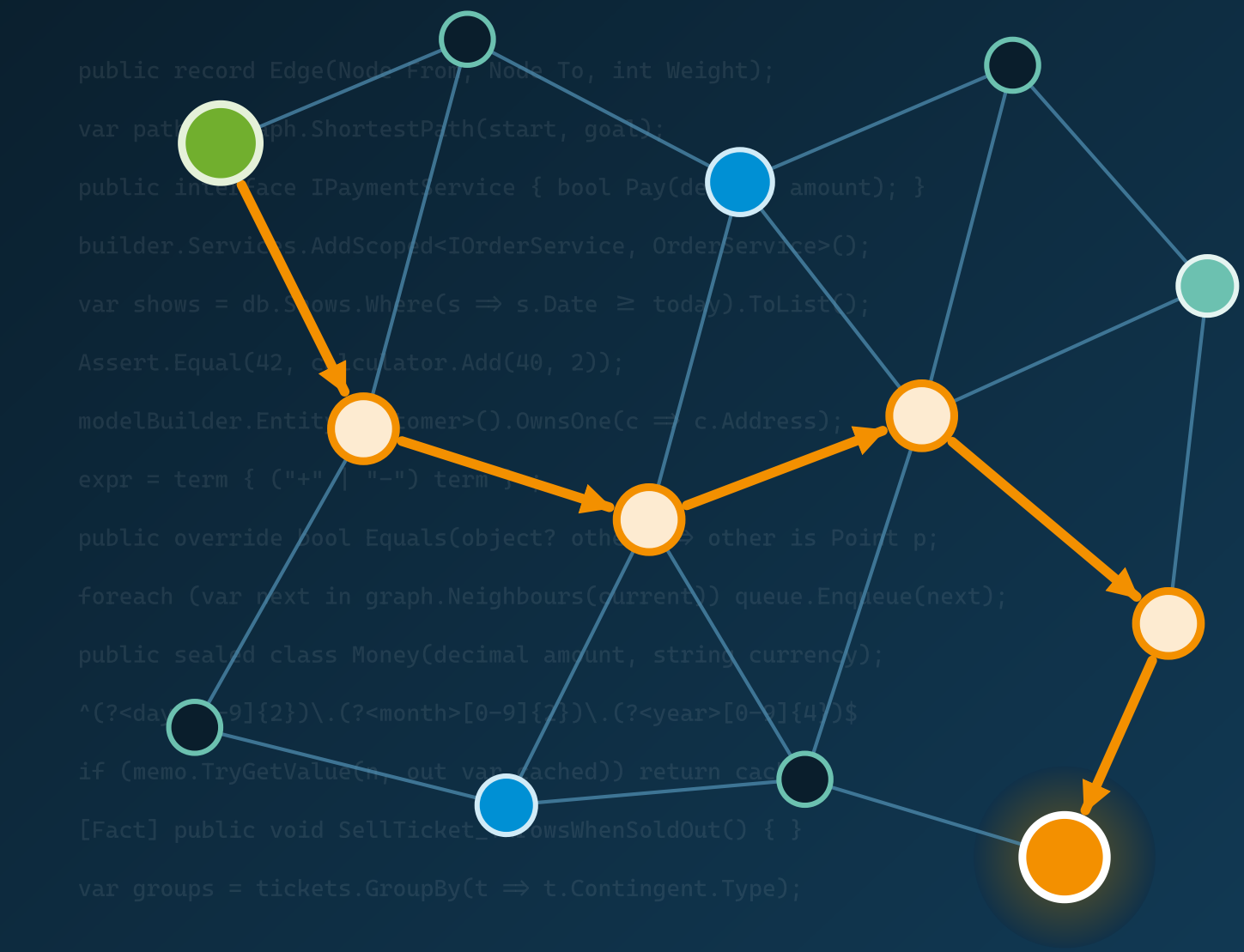
HTL SPENGERGASSE · WIEN



Softwareentwicklung mit .NET

C# · Unittests · SOLID · EF Core · LINQ · Theoretische Informatik

Michael Schletz

Version 18. Oktober 2026

Inhaltsverzeichnis

Arbeit mit dem Buch . . . 3
Types in C# . . . 4
- Wertetypen in C# . . . 5
- Referenztypen und der Umgang mit null in C# . . . 13
- Properties und Initializer in C# . . . 22
- Collections in .NET . . . 29
- Vererbung 1: Implementierung in C# . . . 39
- Interfaces und Dependencies . . . 47
- Vererbung 2: Interfaces implementieren: IEquatable und IComparable . . . 59

Unittests mit xUnit . . . 66
- Warum testen? . . . 67
- Unit Tests mit xUnit . . . 72
- Testfälle finden . . . 79

SOLID und DRY . . . 84
- Programmieren im Zeitalter der Agents . . . 85
- DRY: Don’t Repeat Yourself . . . 92
- S: Single Responsibility . . . 103
- O, Teil 1: Werte und Regeln an genau einer Stelle . . . 114
- O, Teil 2: Erweitern statt ändern . . . 125
- L: Liskov Substitution . . . 136
- I: Interface Segregation . . . 147
- D: Dependency Inversion . . . 160

Der OR Mapper EF Core . . . 171
- Warum ein OR Mapper (Object-relational mapping) . . . 172
- Klassenmodelle persistieren mit EF Core . . . 175
- Erweiterte Konfigurationsmöglichkeiten mit EF Core . . . 185
- Vererbung und andere OOP Konzepte . . . 194
- Musterdaten erzeugen mit Bogus . . . 205

Serviceklassen . . . 213
- Lambda Expressions . . . 214
- LINQ: Where, Select und Any . . . 222
- Serviceklassen mit EF Core . . . 231
- Erweiterte LINQ Techniken . . . 240

Theoretische Informatik: Formale Sprachen . . . 245
- Grundbegriffe formaler Sprachen . . . 246
- Grammatiken . . . 253
- Darstellungsformen von Grammatiken . . . 262

Endliche Automaten . . . 271
Reguläre Ausdrücke . . . 279
Parser für arithmetische Ausdrücke . . . 292
Theoretische Informatik: Algorithmen . . . 307
Graphen . . . 308
Breiten- und Tiefensuche . . . 318
Kürzeste Wege mit Dijkstra . . . 329
Topologische Sortierung . . . 337
Divide and Conquer . . . 343
Backtracking . . . 353
Greedy-Algorithmen . . . 361
Dynamische Programmierung . . . 367
Theoretische Informatik: Komplexität . . . 377
Laufzeit messen . . . 378
Die O-Notation . . . 385
Code analysieren . . . 392
Datenstrukturen in .NET . . . 402
Optimierung . . . 410
P und NP . . . 419

Arbeit mit dem Buch

Dieses Buch ist ein PDF Export des Kurses von **https://github.com/Die-Spengergasse/course-pos-csharp_basics**. Lade immer die neueste Version von → diesem Link.

Jedes Kapitel ist auch im Kurs als online Version enthalten. Die **Übungsbeispiele** sind in den einzelnen Kapitel nur in der online Version sichtbar. Zusätzlich gibt es im Kurs auf github zusätzliche Übungen.

Autor:
Michael Schletz
HTBLVA Spengergasse
Spengergasse 20
1050 Wien

Das Werk kann kopiert, verändert und weiterverbreitet werden, ohne um Erlaubnis bitten zu müssen.



KI generierte Inhalte

Inhalte dieses Kurses wurden mit KI erstellt oder überarbeitet.

Types in C#

Wertetypen in C#

Welchen Wert hat `a` nach diesen drei Zeilen?

```
int a = 5;
int b = a;
b = 7;
```

`a` hat noch immer den Wert 5. Die Variable `b` hat bei der Zuweisung eine **Kopie** des Wertes bekommen. Typen mit diesem Verhalten heißen **Wertetypen** (*value types*). Die Variable enthält direkt den Wert und nicht nur einen Verweis darauf. In diesem Kapitel lernst du die eingebauten Wertetypen von C# kennen und erfährst, wo ihre Fallen liegen: Rundungsfehler, Überlauf, Typumwandlung, fehlende Werte und das Einlesen von Zahlen aus Strings.

Projekt anlegen

Um die Beispiele mitzumachen, erstellst du eine .NET Konsolenapplikation. Führe dafür die folgenden Befehle in der Konsole aus. Unter macOS verwendest du statt `rd /S /Q` den Befehl `rm -rf` und statt `start` den Befehl `open`.

```
rd /S /Q TypesDemo
mkdir TypesDemo
cd TypesDemo
mkdir TypesDemo.Application
cd TypesDemo.Application
dotnet new console
cd ..
dotnet new sln -f sln
dotnet sln add TypesDemo.Application
start TypesDemo.sln
```

Öffne danach durch Doppelklick auf das Projekt (*TypesDemo.Application*) die Datei *TypesDemo.Application.csproj*. Ersetze den Inhalt durch die folgende Konfiguration:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

</Project>
```

Option	Bedeutung
`Nullable`	Der Compiler prüft, ob eine Variable `null` sein kann. Mehr dazu im Kapitel *Referenztypen*.
`TreatWarningsAsErrors`	Jede Warnung ist ein Fehler. Das Programm startet erst, wenn der Code sauber ist.

Die Option `ImplicitUsings`, die `dotnet new` anlegt, haben wir entfernt. Du musst daher die verwendeten Namespaces mit `using` selbst angeben. So siehst du, woher ein Typ kommt. Jede *Program.cs* in diesem Kapitel beginnt daher mit `using System;`.

Die eingebauten Wertetypen

Die meisten Typen kennst du schon aus Java. In C# sind sie Kurzschreibweisen für Strukturen (*struct*) im Namespace *System*: `int` ist z. B. dasselbe wie `System.Int32`.

Typ	Größe	Wertebereich bzw. Genauigkeit	Literal
`bool`	1 Byte	`true` oder `false`	`true`
`char`	2 Byte	ein Zeichen (UTF-16)	`'A'`
`int`	4 Byte	-2 147 483 648 bis 2 147 483 647	`42`
`long`	8 Byte	etwa $\pm 9{,}2 \cdot 10^{18}$	`42L`
`float`	4 Byte	etwa 7 Stellen	`1.5f`
`double`	8 Byte	etwa 15 bis 16 Stellen	`1.5`
`decimal`	16 Byte	28 bis 29 Stellen, exakte Dezimalzahlen	`1.5m`
`DateTime`	8 Byte	Datum und Uhrzeit	`new DateTime(2026, 10, 18)`

Daneben gibt es die vorzeichenlosen (*unsigned*) Typen `byte`, `ushort`, `uint` und `ulong`. Du brauchst sie selten, denn fast alle Methoden in .NET arbeiten mit `int` und `long`.

Die Deklaration funktioniert wie in Java. Große Zahlen kannst du mit dem Unterstrich `_` gliedern, der Compiler ignoriert das Zeichen. Mit `var` erkennt der Compiler den Typ selbst aus dem zugewiesenen Wert (*type inference*).

```
int count = 1;
bool isActive = true;
long population = 9_000_000;   // Same as 9000000
var price = 12.5m;             // The compiler infers decimal
Console.WriteLine($"{count} {isActive} {population} {price}");

long total;
// error CS0165: Use of unassigned local variable 'total'
Console.WriteLine(total);
```

Lokale Variablen musst du vor dem ersten Lesen initialisieren. Nur Felder einer Klasse bekommen automatisch einen Standardwert (0, `false`, …).

Ganzzahlen: Division und Überlauf

Bei Ganzzahlen gibt es zwei typische Fehler. Der Compiler meldet keinen davon.

```
Console.WriteLine(7 / 2);            // 3, not 3.5
Console.WriteLine(7 / 2.0);          // 3.5
Console.WriteLine((double)7 / 2);    // 3.5

int max = int.MaxValue;
Console.WriteLine(max + 1);          // -2147483648
Console.WriteLine(checked(max + 1)); // OverflowException
```

- **Ganzzahldivision:** Sind beide Operanden ganzzahlig, ist auch das Ergebnis ganzzahlig. Die Nachkommastellen werden abgeschnitten. Erst wenn ein Operand ein Gleitkommatyp ist, rechnet C# mit Nachkommastellen.
- **Überlauf (*overflow*):** Überschreitet ein Ergebnis den Wertebereich, beginnt der Wert am anderen Ende des Bereichs. Mit `checked` wirft C# stattdessen eine *OverflowException*.

Gleitkommazahlen: double und decimal

Was gibt dieses Programm aus?

```
Console.WriteLine(0.1 + 0.2);           // 0.30000000000000004
Console.WriteLine(0.1 + 0.2 == 0.3);    // False
Console.WriteLine(0.1m + 0.2m == 0.3m); // True
```

`double` speichert Zahlen im Binärsystem. Die Zahl 0.1 ist im Binärsystem ein periodischer Bruch, so wie 1/3 = 0.333… im Dezimalsystem. Sie wird daher gerundet gespeichert, und die kleinen Fehler summieren sich. Der Typ `decimal` speichert Dezimalzahlen exakt, ist aber langsamer.

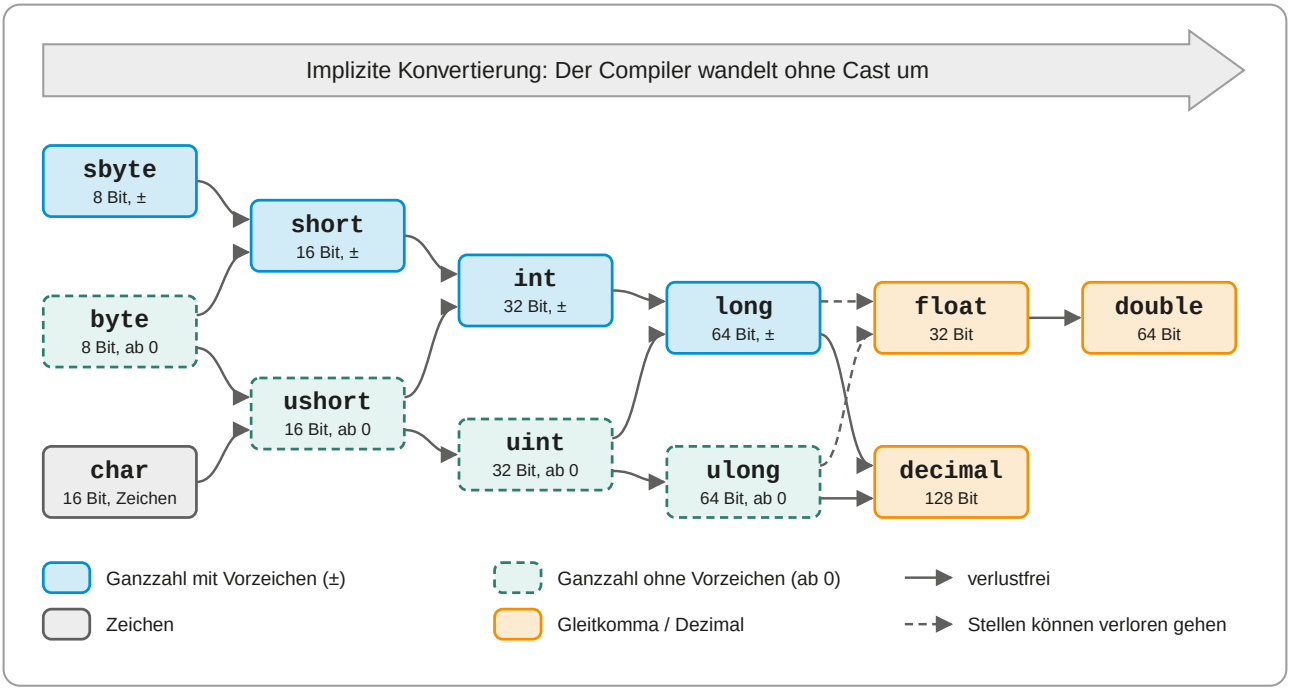


Auf einem deutschsprachigen Windows gibt `Console.WriteLine` ein Komma statt eines Punktes aus.

	`double`	`decimal`
Genauigkeit	etwa 15 Stellen, binär gerundet	28 Stellen, Dezimalzahlen exakt
Geschwindigkeit	schnell (direkt im Prozessor)	langsamer
Verwende für	Messwerte, Physik, Statistik	Geldbeträge, Steuersätze
Vergleich	nie mit `==`, sondern `Math.Abs(a - b) < 1e-9`	mit `==` möglich

Typumwandlung (Casts)

Darf ich einen `int` in einer `long` Variable speichern? Ja, denn jeder mögliche `int` Wert passt in einen `long`. Der Compiler wandelt den Wert ohne Hinweis um. Das nennt man *implizite Konvertierung*.



Die Pfeile lassen sich verketten: *byte* wird über *short* und *int* auch implizit zu *long*. Die gestrichelten Pfeile sind eine Ausnahme. Ein *long* passt zwar ohne Cast in einen *float*, aber *float* speichert nur rund 7 Stellen genau.

In die Gegenrichtung brauchst du einen *expliziten Cast*. Damit sagst du dem Compiler: Ich weiß, dass Information verloren gehen kann.

```
int count = 42;
long total = count;           // OK, implicit
// error CS0266: Cannot implicitly convert type 'long' to 'int'
int small = total;
int small2 = (int)total;      // OK, explicit cast

Console.WriteLine((int)3.9);  // 3: decimal places are cut off, not rounded
long big = 3_000_000_000;
Console.WriteLine((int)big);  // -1294967296: the value does not fit
```

Brauchst du einen gerundeten Wert, verwende `Math.Round()` vor dem Cast.

Nullable value types

Ein Sensor liefert eine Temperatur. Manchmal fällt er aus, dann gibt es keinen Messwert. Oft findet man dafür Code wie diesen:

```
int temperature = -999;   // -999 means "no value"
```

Diese Lösung hat zwei Probleme:

- Nur der Autor weiß, dass -999 ein Spezialwert ist.

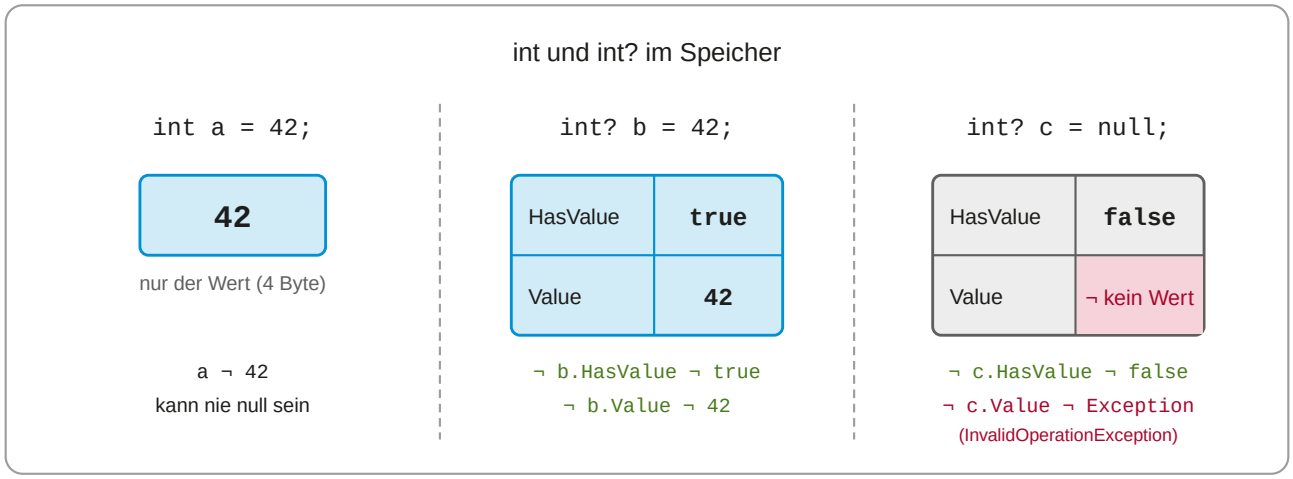
- Rechnet jemand versehentlich mit -999, läuft das Programm ohne Fehler weiter. Das Ergebnis ist aber sinnlos.

C# bietet dafür *nullable value types*. Schreibst du ein `?` nach dem Typ (`int?`, `double?`, `DateTime?`, ...), kann die Variable zusätzlich den Wert `null` speichern. `null` bedeutet: Es gibt keinen Wert.

```
int? temperature = null;  // Valid assignment
temperature = 21;
```

Wie funktioniert int?

`int?` ist die Kurzschreibweise für `Nullable<int>`. Das ist eine Struktur mit zwei Feldern: *HasValue* gibt an, ob ein Wert vorhanden ist. *Value* enthält den Wert.



Die Variable selbst ist also nie "leer". Du kannst *HasValue* deshalb auch dann abfragen, wenn kein Wert gespeichert ist. Liest du *Value* ohne Wert, wirft C# eine *InvalidOperationException*.

```
int? temperature = null;
if (temperature.HasValue)               // false, no exception
{
    Console.WriteLine(temperature.Value);
}
if (temperature is int value)           // Pattern matching, value is an int
{
    Console.WriteLine(value);
}
```

Rechnen mit null und der Operator ??

Rechnest du mit einem nullable Typ, kann das Ergebnis wieder `null` sein. Ist ein Operand `null`, ist das Ergebnis `null`.

```
int? temperature = null;
int? higher = temperature + 1;  // null
```

```
// error CS0266: Cannot implicitly convert type 'int?' to 'int'
int result = temperature + 1;
```

Der *null-coalescing operator* `??` löst das Problem. `x ?? y` liefert `x`, wenn `x` nicht `null` ist, sonst `y`.

```
int result = temperature ?? 0;          // 0 if temperature is null
int result2 = (temperature ?? 0) + 1;   // Parentheses matter, see below
```

Ob 0 ein sinnvoller Ersatzwert ist, hängt von der Aufgabe ab. Achte auf die Rangfolge der Operatoren: `+` wird vor `??` ausgeführt.

Ausdruck	**`a` ist `null`**	**`a` ist 5**
`a ?? 0 + 2` (bedeutet `a ?? (0 + 2)`)	2	5
`(a ?? 0) + 2`	2	7

Werte aus Strings lesen (Parsen)

Eingaben von der Tastatur oder aus einer Datei sind immer Strings. Das Umwandeln eines Strings in eine Zahl nennt man *parsen*. Ein Cast wie `(int)"12"` funktioniert nicht, denn der String muss Zeichen für Zeichen interpretiert werden.

Methode	**Verhalten bei einem ungültigen String**
`int.Parse("A")`	wirft eine *FormatException*
`int.TryParse("A", out int value)`	liefert `false`. Bei Erfolg liefert die Methode `true` und schreibt die Zahl in `value`.

```
int number = int.Parse("12");
try
{
    int invalid = int.Parse("A");
}
catch (FormatException)
{
    Console.WriteLine("A is not a number.");
}

// out: the method writes the result into parsed
if (int.TryParse("12", out int parsed))
{
    Console.WriteLine($"Success: {parsed}");
}
```

Kommazahlen und die Spracheinstellung

Ohne weitere Angabe verwendet `Parse()` die Spracheinstellung (*culture*) des Betriebssystems. Sie legt fest, welches Zeichen das Dezimalzeichen ist. Derselbe String liefert daher auf verschie-

denen Computern verschiedene Ergebnisse:

String	Deutsch (Österreich)	Deutsch (Deutschland)	Englisch (USA)
`"12.1"`	Fehler	121	12.1
`"12,1"`	12.1	12.1	121

Für Daten aus Dateien oder Schnittstellen gibst du deshalb immer `CultureInfo.InvariantCulture` an. Sie verwendet unabhängig vom Computer den Punkt als Dezimalzeichen.

```
using System;
using System.Globalization;

double value = double.Parse("12.1", CultureInfo.InvariantCulture);  // 12.1
Console.WriteLine(value);
if (double.TryParse("12.1", CultureInfo.InvariantCulture, out double v))
{
    Console.WriteLine(v);
}
```

Datumswerte

Für Datumswerte gibst du mit `ParseExact()` bzw. `TryParseExact()` das genaue Format an. Achte auf die Groß- und Kleinschreibung: `MM` ist der Monat, `mm` die Minute. `HH` ist die Stunde im 24-Stunden-Format, `hh` im 12-Stunden-Format. Mit `hh` scheitert daher das Parsen von 18:07 Uhr.

```
if (DateTime.TryParseExact(
        "2026-01-22 18:07:08",              // String to parse
        "yyyy-MM-dd HH:mm:ss",              // Format
        CultureInfo.InvariantCulture,
        DateTimeStyles.None,
        out DateTime date))
{
    Console.WriteLine(date.ToString("yyyy-MM-dd HH:mm:ss"));
}
```

Für Profis: Wertetypen als Parameter, ref und out

Übergibst du einen Wertetyp an eine Methode, bekommt die Methode eine Kopie. Ändert die Methode den Parameter, bleibt die Variable des Aufrufers unverändert. Das ist dasselbe Verhalten wie in Java.

Mit den Schlüsselwörtern `ref` und `out` übergibst du stattdessen die Variable selbst. Die Methode kann dann die Variable des Aufrufers ändern.

Schlüsselwort	Bedeutung
`ref`	Die Variable muss vor dem Aufruf einen Wert haben. Die Methode darf ihn lesen und ändern.
`out`	Die Variable braucht vorher keinen Wert. Die Methode **muss** ihr einen Wert zuweisen.

`out` begegnet dir vor allem bei `TryParse()`. Für eigene Methoden gibt es eine besser lesbare Lösung: Die Methode gibt ein *Tuple* mit mehreren Werten zurück.

```
using System;

int value = 1;
Increment(value);
Console.WriteLine(value);                  // 1: the method got a copy
IncrementRef(ref value);
Console.WriteLine(value);                  // 2

if (TryGetCount(out int itemCount))        // Declares itemCount in the call
{
    Console.WriteLine($"{itemCount} items found.");
}

(bool success, int count) = GetCount();    // Better: return a tuple
if (success) { Console.WriteLine($"{count} items found."); }

// A switch expression on the tuple, _ (discard) matches any value
string message = GetCount() switch
{
    (true, 0) => "No items found.",
    (true, _) => "Some items found.",
    (false, _) => "Error."
};
Console.WriteLine(message);

static void Increment(int number) => number++;

static void IncrementRef(ref int number) => number++;

// The compiler reports an error if count is not assigned in every path.
static bool TryGetCount(out int count)
{
    count = 1;
    return true;
}

static (bool success, int count) GetCount() => (success: true, count: 1);
```

Referenztypen und der Umgang mit null in C#

Im Kapitel *Wertetypen* hast du gesehen: Nach `int b = a;` ist `b` eine unabhängige Kopie. Bei Objekten einer Klasse ist das anders. Welcher Vorname wird hier ausgegeben?

```
Person p1 = new Person("Max", "Muster");
Person p2 = p1;
p2.FirstName = "Eva";
Console.WriteLine(p1.FirstName);
```

Die Ausgabe ist *Eva*, obwohl wir nur `p2` geändert haben. Eine Klasse ist ein **Referenztyp** (*reference type*). Die Variable enthält nicht das Objekt selbst, sondern nur einen Verweis (eine *Referenz*) darauf. In diesem Kapitel lernst du, wie Referenzen funktionieren und wie C# dich vor dem häufigsten Laufzeitfehler schützt: dem Zugriff auf `null`.

Projekt anlegen

Für die Beispiele brauchst du eine .NET Konsolenapplikation. Führe diese Befehle in der Konsole aus. Unter macOS verwendest du statt `rd /S /Q` den Befehl `rm -rf` und statt `start` den Befehl `open`.

```
rd /S /Q ReferenceTypesDemo
mkdir ReferenceTypesDemo
cd ReferenceTypesDemo
mkdir ReferenceTypesDemo.Application
cd ReferenceTypesDemo.Application
dotnet new console
cd ..
dotnet new sln -f sln
dotnet sln add ReferenceTypesDemo.Application
start ReferenceTypesDemo.sln
```

Öffne danach die Projektdatei *ReferenceTypesDemo.Application.csproj* (Doppelklick auf das Projekt). Entferne die Zeile *ImplicitUsings* und füge *TreatWarningsAsErrors* hinzu. Die Datei sieht dann so aus:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

</Project>
```

Die Option *Nullable* ist für dieses Kapitel zentral. Was sie bewirkt, erfährst du im Abschnitt *Nullable reference types*.

Eine eigene Klasse

Lege im Projekt die Datei *Person.cs* an. Die Klasse hat zwei öffentliche Felder. Öffentliche Member schreibt man in C# groß (*PascalCase*). In der Praxis verwendet man statt öffentlicher Felder Properties, sie kommen im nächsten Kapitel.

Person.cs

```
namespace ReferenceTypesDemo.Application;

public class Person
{
    public string FirstName;
    public string LastName;

    public Person(string firstName, string lastName)
    {
        FirstName = firstName;
        LastName = lastName;
    }
}
```

Ohne den Konstruktor kompiliert die Klasse nicht:

```
error CS8618: Non-nullable field 'FirstName' must contain a non-null value
when exiting constructor.
Consider adding the 'required' modifier or declaring the field as nullable.
```

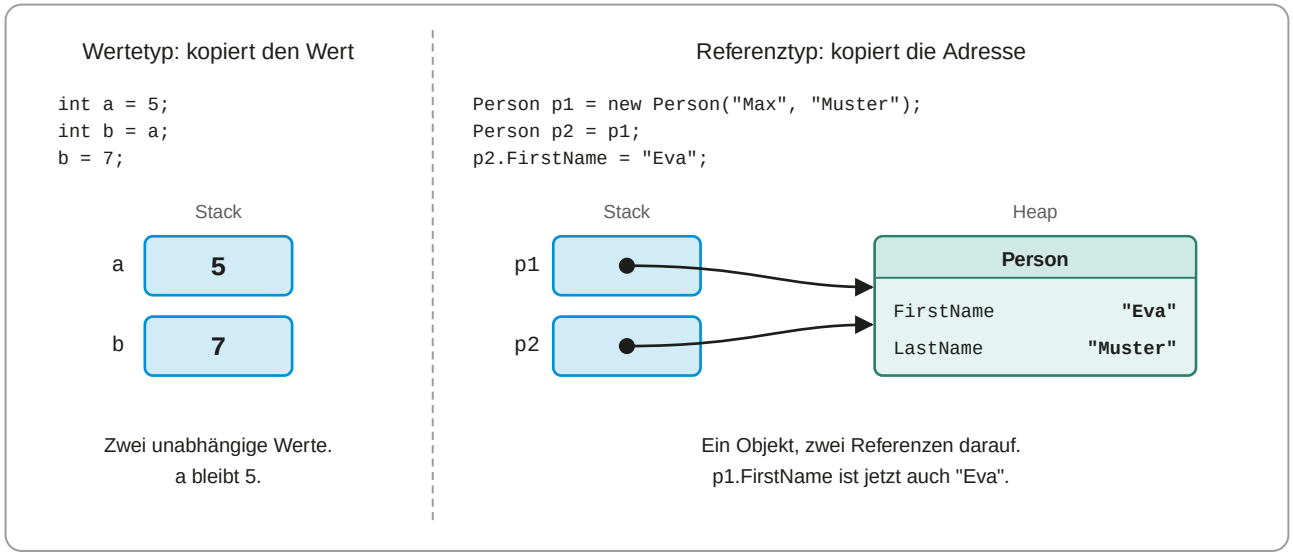
Der Compiler stellt sicher, dass *FirstName* und *LastName* nie `null` sind. Der Konstruktor erzwingt, dass jede Person einen Vor- und Nachnamen bekommt.

Beim Aufruf kannst du die Parameter mit Namen angeben (*named arguments*). Das macht den Code lesbarer. Außerdem kannst du Vor- und Nachname nicht verwechseln, obwohl beide vom Typ `string` sind.

```
Person p1 = new Person(firstName: "Max", lastName: "Muster");
```

Wertetyp und Referenztyp

`new Person(…)` erzeugt ein Objekt in einem eigenen Speicherbereich, dem *Heap*. Die Variable `p1` speichert nur die Adresse dieses Objektes. `p2 = p1` kopiert diese Adresse. Es gibt danach **ein** Objekt und **zwei** Referenzen darauf.



Art	Typen
Wertetypen	`int`, `long`, `double`, `decimal`, `bool`, `char`, `DateTime`, `enum`, eigene `struct` Typen
Referenztypen	`string`, Arrays, `List<T>`, eigene `class` Typen

Dasselbe gilt für Parameter. Übergibst du eine Person an eine Methode, bekommt die Methode eine Kopie der Adresse. Sie arbeitet also mit demselben Objekt und kann es ändern.

```
Rename(p1);
Console.WriteLine(p1.FirstName);   // Anna

static void Rename(Person person) => person.FirstName = "Anna";
```



`string` ist ein Referenztyp, verhält sich aber wie ein Wertetyp. Ein String kann nicht verändert werden (*immutable*). Methoden wie `ToUpper()` liefern immer einen neuen String. Wie Stack und Heap genau aufgebaut sind, zeigt das Kapitel *Collections*.

null: Eine Referenz ohne Objekt

Eine Referenzvariable kann auch auf gar kein Objekt zeigen. Sie hat dann den Wert `null`. Greifst du über so eine Variable auf ein Feld oder eine Methode zu, gibt es eine *NullReferenceException*. Das ist einer der häufigsten Laufzeitfehler.

```
Person? nobody = null;
// Without the Nullable option: NullReferenceException at runtime
// With the Nullable option: error CS8602: Dereference of a possibly null reference.
Console.WriteLine(nobody.FirstName);
```

In diesem Projekt kompiliert der Code nicht einmal. Der Compiler meldet den Fehler schon vor dem Start. Wie das funktioniert, zeigt der nächste Abschnitt.

Nullable reference types

Mit der Option *Nullable* unterscheidet der Compiler zwei Arten von Referenzvariablen:

Deklaration	Bedeutung
`string name`	Darf nie `null` sein. Du kannst ohne Prüfung auf `name.Length` zugreifen.
`string? name`	Darf `null` sein. Vor dem Zugriff musst du prüfen, ob ein Objekt vorhanden ist.

```
string neverNull = string.Empty;
string? canBeNull = null;

// (1) error CS8600: Converting null literal or possible null value
//     to non-nullable type.
neverNull = null;
// (2) error CS8602: Dereference of a possibly null reference.
Console.WriteLine(canBeNull.Length);
Console.WriteLine(canBeNull?.Length);  // (3) OK
if (!string.IsNullOrEmpty(canBeNull))
{
    Console.WriteLine(canBeNull.Length);  // (4) OK
}
```

- **(1)** Einer Variable vom Typ `string` darfst du nicht `null` zuweisen.
- **(2)** Bei `string?` darfst du nicht ungeprüft auf Properties oder Methoden zugreifen.
- **(3)** Der Operator `?.` (siehe unten) ist erlaubt.
- **(4)** Der Compiler analysiert den Programmablauf. Er erkennt, dass `canBeNull` im if-Block nicht `null` sein kann. Das funktioniert, weil `IsNullOrEmpty()` mit einem *Attribute* versehen ist: `IsNullOrEmpty([NotNullWhen(false)] string? value)`. Es sagt dem Compiler: Liefert die Methode `false`, ist der Parameter nicht `null`. Mehr dazu findest du unter Attributes for null-state static analysis.

`string?` sieht aus wie `int?`, ist aber etwas anderes. `int?` ist eine eigene Struktur mit *HasValue* und *Value*. `string?` ist nur ein Hinweis für den Compiler. Im kompilierten Programm sind `string` und `string?` derselbe Typ.

Operatoren für null

C# hat mehrere Operatoren, die den Umgang mit `null` kürzer machen.

Ausdruck	Bedeutung
`x is null`, `x is not null`	Prüft, ob `x` null ist.
`x?.Length`	Liefert `null`, wenn `x` null ist. Sonst wird `x.Length` gelesen.
`x ?? y`	Liefert `x`, wenn `x` nicht null ist. Sonst `y`.
`x ??= y`	Weist `x` den Wert `y` zu, wenn `x` null ist.

Ausdruck	Bedeutung
`x!`	Schaltet die Prüfung des Compilers für `x` ab.

is null

```
string? text = null;
if (text is null) { Console.WriteLine("text is null."); }
if (text is not null) { Console.WriteLine("text is not null."); }
```

`text == null` funktioniert auch. In C# kann der Autor einer Klasse aber festlegen, was `==` bedeutet (*operator overloading*). Der Vergleich mit `==` ist daher nicht in jedem Fall verlässlich. `is` kann nicht überladen werden und prüft immer, ob die Referenz `null` ist.

Der null-conditional operator ?.

Der Operator `?.` liefert das Ergebnis der Property bzw. Methode. Ist das Objekt `null`, liefert er `null` und wirft keine Exception.

```
string? name = "Max";
string? empty = null;

Console.WriteLine(name?.Length);             // 3
Console.WriteLine(empty?.Length);            // null (an empty line), not 0!
Console.WriteLine(empty?.ToUpper().Length);  // null, stops at the first ?.
// error CS0266: Cannot implicitly convert type 'int?' to 'int'
int length = name?.Length;
int length2 = name?.Length ?? 0;             // OK
```

Das Ergebnis von `name?.Length` ist ein `int?`, denn es kann `null` sein. Mit `?? 0` machst du daraus wieder einen `int`.

Der Operator ??=

`??=` weist nur dann einen Wert zu, wenn die Variable `null` ist.

```
string? nickname = null;
// Same as: if (nickname is null) { nickname = "unknown"; }
nickname ??= "unknown";
```

Der null-forgiving operator !

Mit `!` sagst du dem Compiler: Diese Variable ist sicher nicht `null`. Der Compiler prüft dann nicht mehr.

```
string? canBeNull = null;
```

```
// Compiles, but throws a NullReferenceException at runtime
Console.WriteLine(canBeNull!.Length);
```

Verwende `!` nur in begründeten Fällen. Du übernimmst damit die Verantwortung, die sonst der Compiler hat. Begründete Fälle sind z. B.:

- Ältere Bibliotheken, die nullable reference types noch nicht unterstützen.
- Frameworks wie EF Core, die Felder erst nach dem Konstruktor befüllen.
- Komplexe Logik, die der Compiler nicht analysieren kann.

Vererbung und Typencasts

Lege die Datei *Student.cs* an. Ein Student ist eine Person mit Geburtsdatum.

Student.cs

```
using System;

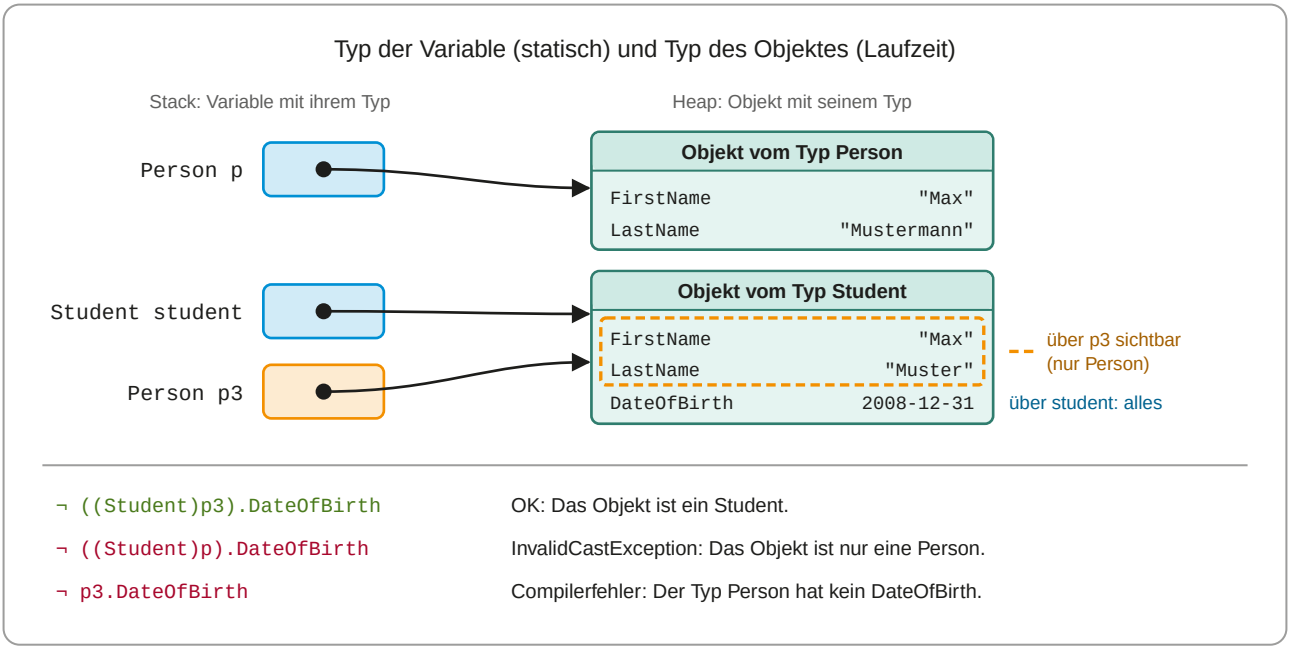
namespace ReferenceTypesDemo.Application;

public class Student : Person              // Java: Student extends Person
{
    public DateTime DateOfBirth;

    // : base(...) calls the constructor of Person, like super(...) in Java.
    public Student(string firstName, string lastName, DateTime dateOfBirth)
        : base(firstName, lastName)
    {
        DateOfBirth = dateOfBirth;
    }
}
```

Eine Referenzvariable hat einen Typ (*statischer Typ*). Das Objekt am Heap hat ebenfalls einen Typ (*Laufzeittyp*). Die beiden Typen müssen nicht gleich sein.

```
Person p = new Person("Max", "Mustermann");
Student student = new Student("Max", "Muster", new DateTime(2008, 12, 31));
Person p3 = student;                                    // (1)
p3.DateOfBirth = new DateTime(2009, 1, 1);              // (2) error CS1061
Console.WriteLine(((Student)p3).DateOfBirth);           // (3)
Console.WriteLine(((Student)p).DateOfBirth);            // (4) Exception
```



- **(1)** Ein Student *ist eine* Person (*is-a* Beziehung). Die Zuweisung an eine Variable vom Typ *Person* ist daher ohne Cast möglich.
- **(2)** Der Compiler sieht nur den Typ der Variable. Eine *Person* hat kein *DateOfBirth*, daher gibt es einen Compilerfehler.
- **(3)** Mit dem Cast sagst du: Hinter `p3` steckt ein Student. Das stimmt, daher wird der 31.12.2008 ausgegeben. Das Objekt hat sich durch die Zuweisung an `p3` nicht verändert.
- **(4)** Hinter `p` steckt nur eine Person. Der Compiler kann das nicht prüfen, erst zur Laufzeit gibt es eine *InvalidCastException*.

Sicher casten mit is und as

Ein Cast wie in (4) bringt das Programm zum Absturz. Mit `is` und `as` prüfst du vorher, ob der Cast möglich ist.

```
if (p3 is Student) { Console.WriteLine("p3 is a student."); }   // (1)
Student? s2 = p as Student;                                      // (2)
if (p3 is Student s3)                                            // (3)
{
    Console.WriteLine(s3.DateOfBirth);
}
```

- **(1)** `is` prüft, ob das Objekt in den Typ umgewandelt werden kann.
- **(2)** `as` versucht den Cast. Ist er nicht möglich, liefert `as` den Wert `null`. Die Variable muss daher den Typ `Student?` haben. Hier ist `s2` null, denn `p` ist nur eine Person.
- **(3)** Seit C# 7 kannst du prüfen und casten in einem Schritt (*type pattern*). Ist `p3` ein Student, ist `s3` die Referenz mit dem Typ *Student*. Diese Form findest du in aktuellem C# Code am häufigsten.

Gleichheit: == und Equals

Sind zwei Personen mit demselben Namen gleich?

```
Person max1 = new Person("Max", "Muster");
Person max2 = new Person("Max", "Muster");
Person max3 = max1;
Console.WriteLine(max1 == max2);          // False: two different objects
Console.WriteLine(max1 == max3);          // True: the same object
```

Bei eigenen Klassen vergleicht `==` die Referenzen, nicht den Inhalt. Bei `string` vergleicht `==` dagegen den Inhalt, denn die Klasse *String* überlädt den Operator. In Java brauchst du dafür `equals()`, in C# funktionieren beide Varianten.

```
string str1 = "MAX";
string str2 = "max".ToUpper();                        // A new object

Console.WriteLine(str1 == str2);                       // True: same content
Console.WriteLine(str1.Equals(str2));                  // True: same content
Console.WriteLine(object.ReferenceEquals(str1, str2)); // False: 2 objects
```

Mit `object.ReferenceEquals()` prüfst du immer, ob zwei Variablen auf dasselbe Objekt zeigen. Wie du für eigene Klassen festlegst, wann zwei Objekte gleich sind, lernst du im Kapitel *Equality*.

Für Profis: Typprüfung mit switch

Ein Serializer wandelt Objekte in Text um, z. B. in JSON. Er bekommt Werte beliebiger Typen und muss je nach Typ anders reagieren. Die *switch expression* prüft den Typ und castet in einem Schritt.

```
using System;
using System.Globalization;

Console.WriteLine(Format(true));     // 1
Console.WriteLine(Format(12));       // 12
Console.WriteLine(Format(3.123));    // 3.12
Console.WriteLine(Format(3.123m));   // Unknown type

// T is a type parameter (generics), inferred from the argument.
static string Format<T>(T value) => value switch
{
    bool b => b ? "1" : "0",
    int i => i.ToString(),
    long l => l.ToString(),
    float or double =>
        string.Format(CultureInfo.InvariantCulture, "{0:0.00}", value),
    null => "null",
    _ => "Unknown type"
```

```
};
```

Die Zweige werden von oben nach unten geprüft. Der erste passende Zweig liefert das Ergebnis. Der Unterstrich _ (*discard*) passt auf jeden Wert.

Properties und Initializer in C#

Erstellen einer Visual Studio Solution

Um die Beispiele mitmachen zu können, muss eine .NET Konsolenapplikation erstellt werden. Führe dafür die folgenden Befehle in der Konsole aus. Unter macOs müssen md und rd durch die entsprechenden Befehle ersetzt werden.

```
rd /S /Q PropertiesDemo
md PropertiesDemo
cd PropertiesDemo
md PropertiesDemo.Application
cd PropertiesDemo.Application
dotnet new console
cd ..
dotnet new sln -f sln
dotnet sln add PropertiesDemo.Application
start PropertiesDemo.sln
```

Öffne danach durch Doppelklick auf das Projekt (*PropertiesDemo.Application*) die Datei *PropertiesDemo.Application.csproj* und füge die Optionen für *Nullable* und *TreatWarningsAsError* hinzu. Die gesamte Konfiguration muss nun so aussehen:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

</Project>
```

Wozu Properties?

Betrachten wir eine Klasse *Student*, mit den get und set Methoden, wie wir sie aus Java kennen:

```
class Student
{
    string firstname;
    string lastname;
    int age;

    public Student(string firstname, string lastname)
    {
        this.firstname = firstname;
        this.lastname = lastname;
```

```
        setAge(0);
    }

    public Student(string firstname, string lastname, int age)
    {
        this.firstname = firstname;
        this.lastname = lastname;
        setAge(age);
    }

    public void setAge(int age)
    {
        if (age >= 0)
        {
            this.age = age;
        }
        else
        {
            throw new ArgumentException("Ungültiges Age");
        }
    }

    public int getAge()
    {
        return age;
    }
}
```

Es fallen 2 Dinge auf:

- Die get und set Methoden, die für Datenprüfungen zuständig sind, haben oft den gleichen Aufbau.
- Es sind mehrere Konstruktoren vorhanden, je nach dem welche Variablen initialisiert werden sollen.

Properties ersetzen get und set Methoden

Wir schreiben nun für die Felder *age*, *firstname* und *lastname* sogenannte Properties. Properties erscheinen nach außen wie Variablen, es wird aber bei der Zuweisung ein Stück Code - nämlich der in set - ausgeführt. Das Schema ist folgendes:

- Anlegen der private Variable. Sie beginnen in C# mit einem Kleinbuchstaben.
- Anlegen der Properties. Sie beginnen in C# mit einem Großbuchstaben.
- Die get Methode kann beliebige Anweisungen enthalten. Sie muss allerdings einen Wert zurückgeben.
- Die set Methode kann auch beliebig aufgebaut sein. Der zugewiesene Wert ist in *value* enthalten.

```
class Student
{
```

```
    private string firstname, lastname;
    private int age;

    public Student(string firstname, string lastname, int age)
    {
        this.firstname = firstname;
        this.lastname = lastname;
        this.age = age;
    }

    public string Firstname
    {
        get { return firstname; }
        set { firstname = value; }
    }
    public string Lastname
    {
        get { return lastname; }
        set { lastname = value; }
    }
    public int Age
    {
        get { return age; }
        set { age = value >= 0 ? value : throw new ArgumentException("Ungültiges
Age!"); }
    }
}
```

> **Achtung:** Oft ist folgender Fehler zu beobachten. *Age* ist in der get Methode großgeschrieben, daher wird eine endlose Rekursion erzeugt:

```
public int Age
{
    get { return Age; }
}
```

Default Properties

Die Properties für *Firstname* und *Lastname* weisen nur 1:1 zu bzw. geben den Wert 1:1 zurück. In C# gibt es mit den *Default Properties* einen eleganteren Weg, das zu bewerkstelligen. Der Compiler erledigt folgende Dinge:

- Es wird automatisch eine private Variable im Hintergrund (das *backing field*) vom Compiler angelegt.
- Die get und set Methode liefert diese 1:1 zurück bzw. schreibt in diese hinein.

```
class Student
{
    public Student(string firstname, string lastname)
    {
```

```
        Firstname = firstname;
        Lastname = lastname;
    }

    public string Firstname { get; set; }
    public string Lastname { get; set; }
}
```

Möchte man die Default Properties gleich initialisieren, ist dies seit C# 6 auch möglich. Der Konstruktor kann hier weggelassen werden, da sicher eine Initialisierung erfolgt.

```
class Student
{
    public string Firstname { get; set; } = string.Empty;
    public string Lastname { get; set; } = string.Empty;
}
```

Read-only Properties

Berechnete Werte

Wird nur eine get Methode definiert, so kann diesem Property nichts zugewiesen werden. Dies ist bei berechneten Werten sinnvoll.

```
class Student
{
    public string Longname
    {
        get { return $"{Firstname} {Lastname}"; }
    }
}
```

Ab C# 7 wurden Expression-bodied members auch für Properties eingeführt. Dadurch kann das folgende Property kürzer definiert werden. Es handelt sich aber immer noch um eine Methode, d. h. sie wird bei jedem Zugriff auf *Longname* ausgeführt und liefert die aktuellen Werte.

```
class Student
{
    public string Longname => $"{Firstname} {Lastname}";
}
```

Unveränderliche Werte (immutable)

Oft sollen Properties nach ihrer Initialisierung nicht mehr verändert werden. So ist z. B. die Änderung von Vor- und Lastname nach Instanzierung der *Student* Klasse nicht notwendig. Die E-Mail Adresse soll jedoch geändert werden können. Durch die Definition mit *get* lassen sich die Properties *Firstname* und *Lastname* nur im Konstruktor oder durch Initialisierung mit = setzen.

```
class Student
{
    public Student(string firstname, string lastname)
    {
        Firstname = firstname;
        Lastname = lastname;
    }
    public string Firstname { get; }
    public string Lastname { get; }
    public string? Email { get; set; }  // Nullable, daher nicht im Konstruktor.
}
```

private set Properties

Sollen Properties wie Id nur innerhalb der Klasse geschrieben werden können, so kann die set Methode auch private (oder protected) definiert werden.

```
class Student
{
    public Student(string firstname, string lastname)
    {
        Firstname = firstname;
        Lastname = lastname;
    }
    public int Id { get; private set; }
    public string Firstname { get; }
    public string Lastname { get; }
    public string? Email { get; set; }
    public void GenerateId()
    {
        Id = new Random().Next();
    }
}
```

Überblick: Getter und Setter in Java vs. Properties in C#

Die folgende Grafik fasst die Arten von Properties zusammen und stellt sie dem gleichwertigen Java Code gegenüber.

Properties in C# – der Vergleich mit Java

Von außen sieht ein Property aus wie eine Variable. Innen besteht es aus einer get und einer set Methode.

Unter der Haube: Aus `public int Age { get; set; }` erzeugt der Compiler die Methoden `get_Age()` und `set_Age(int value)`.
Das sind genau die Getter und Setter, die du in Java selbst schreibst. In C# schreibt sie der Compiler für dich.

ART DES PROPERTIES	Java: Getter und Setter	C#: Property

1 Property mit Logik

Eigene private Variable, set prüft den Wert. value enthält den zugewiesenen Wert.

VON AUSSEN
¬ lesen
¬ schreiben

Klasse Student — 10 Zeilen

```java
private int age;

public int getAge() {
    return age;
}
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException();
    }
    this.age = age;
}
```

```java
s.setAge(s.getAge() + 1);
```

Klasse Student — 10 Zeilen

```csharp
private int age;

public int Age
{
    get { return age; }
    set
    {
        if (value < 0) { throw new ArgumentException(); }
        age = value;
    }
}
```

```csharp
s.Age++;   // ruft get und set auf
```

2 Default Property

Nur 1:1 lesen und schreiben. Das backing field legt der Compiler selbst an.

VON AUSSEN
¬ lesen
¬ schreiben

Klasse Student — 7 Zeilen

```java
private String email;

public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
```

```java
s.setEmail("test@mail.at");
```

Klasse Student — 1 Zeile

```csharp
public string? Email { get; set; }
```

```csharp
s.Email = "test@mail.at";
```

3 Read-only: immutable

Nur get. Der Wert wird im Konstruktor gesetzt und ist danach unveränderlich.

VON AUSSEN
¬ lesen
¬ schreiben: nur Konstruktor

Klasse Student — 7 Zeilen

```java
private final String firstname;

public Student(String firstname) {
    this.firstname = firstname;
}
public String getFirstname() {
    return firstname;
}
```

```java
s.setFirstname("X");  // Fehler: cannot find symbol
```

Klasse Student — 5 Zeilen

```csharp
public Student(string firstname)
{
    Firstname = firstname;
}
public string Firstname { get; }
```

```csharp
s.Firstname = "X";  // Fehler CS0200
```

4 Read-only: berechnet

Kein gespeicherter Wert. get rechnet bei jedem Zugriff neu.

VON AUSSEN
¬ lesen
¬ schreiben

Klasse Student — 3 Zeilen

```java
public String getLongname() {
    return firstname + " " + lastname;
}
```

```java
String name = s.getLongname();
```

Klasse Student — 1 Zeile

```csharp
public string Longname => $"{Firstname} {Lastname}";
```

```csharp
string name = s.Longname;
```

5 private set

Von außen nur lesbar, innerhalb der Klasse auch schreibbar.

VON AUSSEN
¬ lesen
¬ schreiben: nur in der Klasse

Klasse Student — 10 Zeilen

```java
private int id;

public int getId() {
    return id;
}
private void setId(int id) {
    this.id = id;
}
public void generateId() {
    setId(new Random().nextInt());
}
```

```java
s.setId(1);  // Fehler: setId(int) has private access
```

Klasse Student — 5 Zeilen

```csharp
public int Id { get; private set; }

public void GenerateId()
{
    Id = new Random().Next();
}
```

```csharp
s.Id = 1;  // Fehler CS0272
```

Die farbige Zeile unter dem Code zeigt den Zugriff von außen, also z. B. aus der Main Methode (s ist eine Variable vom Typ Student).

Verwendung der Properties, Initializer

Der große Vorteil von Properties liegt in ihrer eleganten Verwendung. Folgende Anweisungen sind dadurch möglich:

```
Student student = new Student(firstname: "VN1", lastname: "ZN1")
{
    Email = "test@mail.at"
};
```

Was passiert hier? Zuerst wird ein Objekt vom Typ Student mit *new Student(...)* erzeugt. Da ein Konstruktor definiert wurde, existiert kein default Konstruktor und wir müssen daher die notwendigen Argumente übergeben. In C# ist es durch den *initializer* möglich, Properties gleich *nach* der Instanzierung zu initialisieren.

Da die set Methoden durchlaufen werden, wird die Datenprüfung natürlich auch im Initializer ausgeführt.

Target-typed new (C# 9)

In den bisherigen Beispielen steht der Typ *Student* zweimal in der Zeile. Ist der Typ der Variable schon bekannt, kann er bei *new* weggelassen werden:

```
Student student = new(firstname: "VN1", lastname: "ZN1") { Email = "test@mail.at" };
```

Collections in .NET

Projekt anlegen

Für die Beispiele brauchst du eine .NET Konsolenapplikation. Führe dafür die folgenden Befehle in der Konsole aus. Unter macOS und Linux ersetzt du *rd /S /Q* durch *rm -rf* und *md* durch *mkdir*.

```
rd /S /Q CollectionDemo
md CollectionDemo
cd CollectionDemo
md CollectionDemo.Application
cd CollectionDemo.Application
dotnet new console
cd ..
dotnet new sln -f sln
dotnet sln add CollectionDemo.Application
start CollectionDemo.sln
```

Öffne danach die Datei *CollectionDemo.Application.csproj*. In Visual Studio geht das mit einem Doppelklick auf das Projekt *CollectionDemo.Application*. Ergänze die Optionen *Nullable* und *TreatWarningsAsErrors*. Die Datei sieht dann so aus:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

</Project>
```

Das Array als einfachste Collection

Ein Array speichert mehrere Elemente vom gleichen Typ, so wie in Java. Die Elemente können auch Referenztypen sein, z. B. Objekte deiner eigenen Klassen. Ein Array wird mit *new* erzeugt. Es liegt daher am Heap und wird später vom Garbage Collector entfernt.

```
int length = 6;
int[] numbers = new int[6];         // (1)
int[] numbers2 = new int[length];   // (2)

int[] numbersDrawn = new int[] { 1, 2, 3, 4, 5, 6 };  // (3)
```

- **(1)** Gibst du nur die Länge an, bekommen alle Elemente den Default-Wert. *numbers* enthält

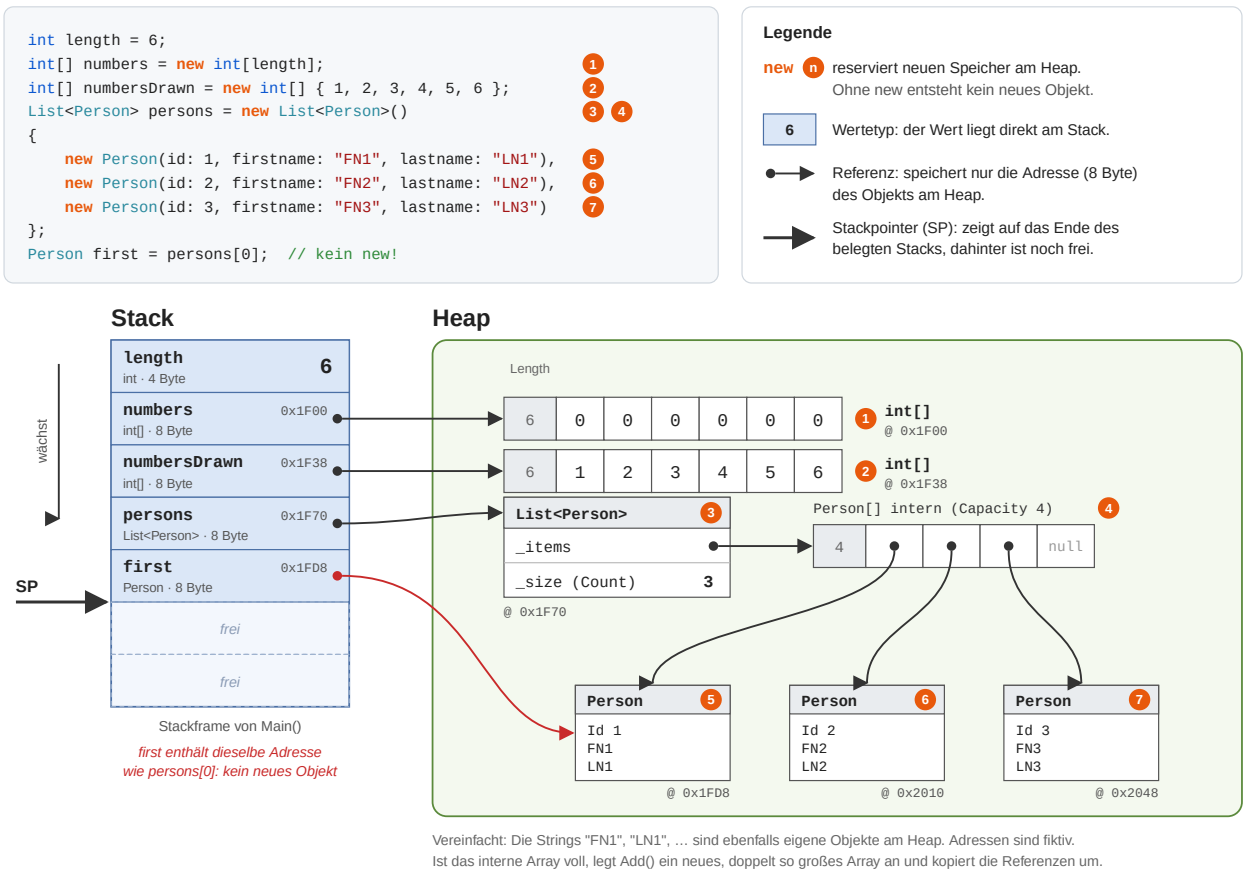
daher [0, 0, 0, 0, 0, 0].

- **(2)** Die Länge kann auch in einer Variable stehen. Sie muss nicht beim Kompilieren feststehen.
- **(3)** Kennst du die Werte schon, befüllst du das Array mit einem Initializer. Die Länge musst du dann nicht angeben. Gibst du sie trotzdem an und passt sie nicht zur Anzahl der Werte, meldet der Compiler einen Fehler.

Stack und Heap

Lokale Variablen liegen am *Stack*. Bei Wertetypen wie *int* steht dort direkt der Wert. Bei Referenztypen steht dort nur die Adresse (8 Byte) des Objekts. Der *Stackpointer* zeigt auf das Ende des belegten Bereichs.

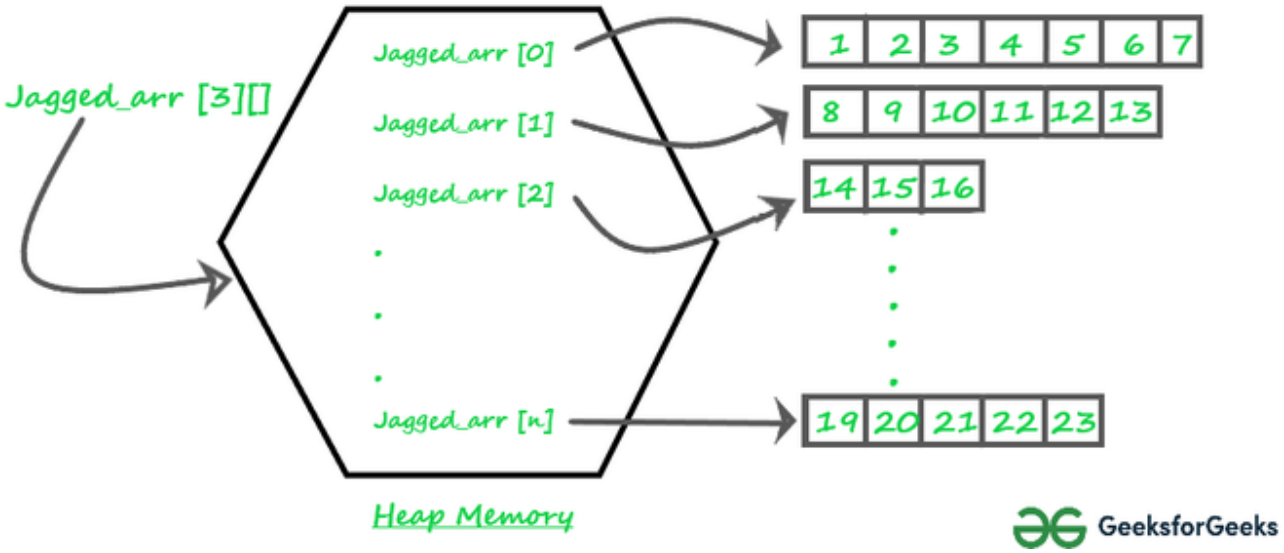
Speicher am *Heap* wird nur mit *new* reserviert. Jedes *new* im Code (orange Nummer in der Grafik) erzeugt genau ein Objekt am Heap. Auch eine Liste ist ein Objekt. Sie verwaltet intern ein Array, das nur Referenzen auf die Personen speichert. Die Zuweisung *first = persons[0]* kopiert daher nur die Adresse. Es entsteht keine neue Person.



Vereinfacht: Die Strings "FN1", "LN1", … sind ebenfalls eigene Objekte am Heap. Adressen sind fiktiv.
Ist das interne Array voll, legt Add() ein neues, doppelt so großes Array an und kopiert die Referenzen um.

Jagged Arrays

Das *jagged array* kennst du schon aus Java. Es wird oft "mehrdimensionales Array" genannt, das ist aber nicht ganz richtig: Es ist ein Array, dessen Elemente auf weitere Arrays am Heap verweisen.



Quelle: https://media.geeksforgeeks.org/wp-content/uploads/20201202202711/Untitled4-660x306.png

Deshalb können die inneren Arrays unterschiedlich lang sein:

```
int[][] quickTipp = new int[][]
{
    new int[]{1,2,3,4,5,6},
    new int[]{8,9,10,11,12,13},
    new int[]{14,16}
};
for (int i = 0; i < quickTipp.Length; i++)
{
    Console.Write($"{i + 1}. TIPP: ");
    for (int j = 0; j < quickTipp[i].Length; j++)
    {
        Console.Write($"{quickTipp[i][j]:00} ");
    }
    Console.WriteLine();
}
```

"Echte" mehrdimensionale Arrays

Bei einem "echten" mehrdimensionalen Array liegen alle Elemente direkt hintereinander im Speicher. Die Position eines Elements ist *row* × *Anzahl der Spalten* + *col*. Die erste Dimension ist meist die Zeile. Die innere Schleife geht dann über die Spalten und liest Speicherstellen, die direkt nebeneinander liegen. Das ist schnell, weil der CPU-Cache die nächsten Elemente schon mitlädt.

```
int[,] matrix = new int[,]
{
    { 1,2,3 },
    { 4,5,6 }
};
```

```
for (int row = 0; row < matrix.GetLength(0); row++)
{
    for (int col = 0; col < matrix.GetLength(1); col++)
    {
        Console.Write($"{matrix[row, col]:00} ");
    }
    Console.WriteLine();
}
```

> **Hinweis:** Mehrdimensionale Arrays brauchst du selten. Verwende sie nur, wenn die Anordnung im Speicher für die Performance wichtig ist. Die Bibliothek OpenCV speichert z. B. ihre Transformationsmatrizen so.

List<T> als flexibler Ersatz für Arrays

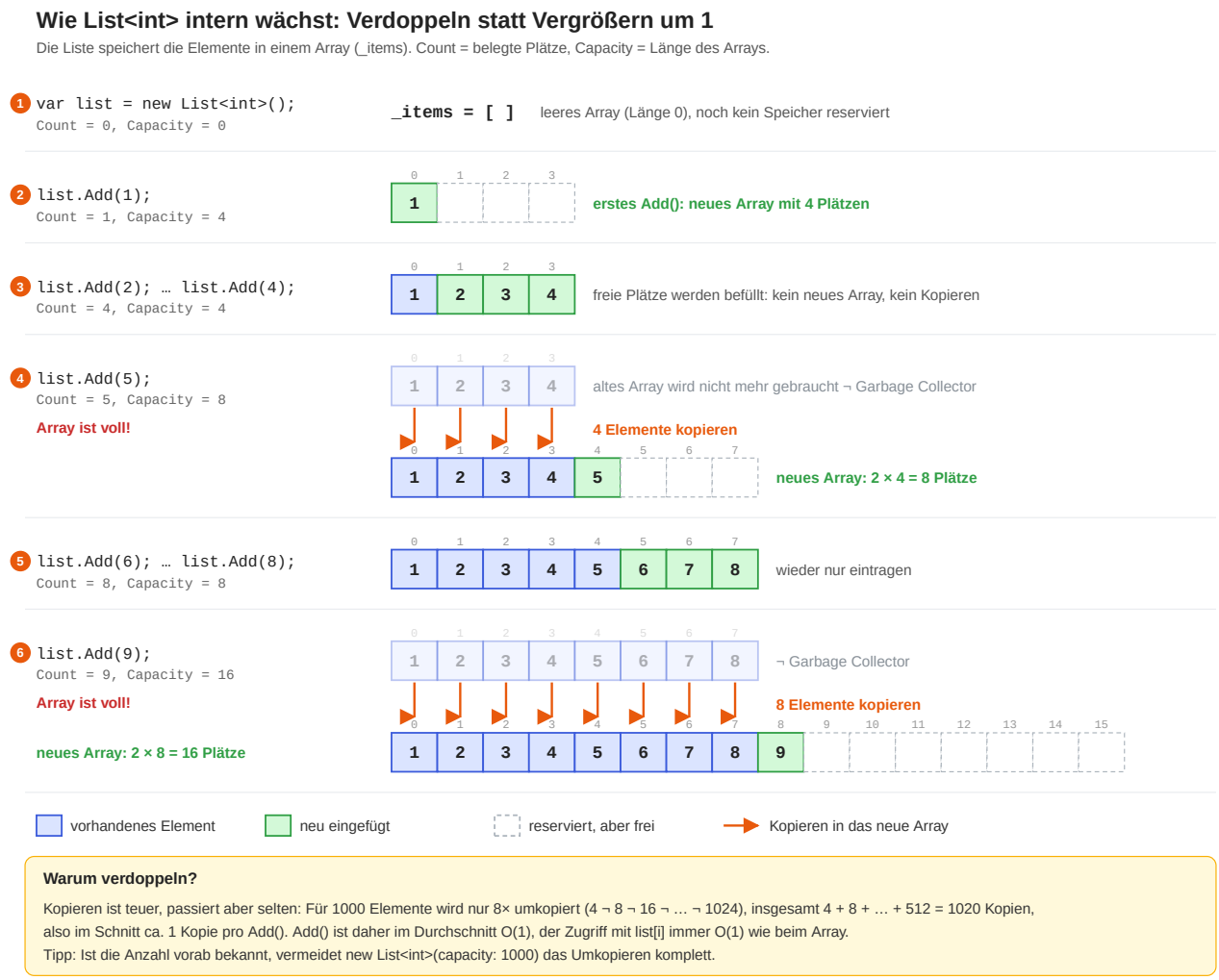
Ein Array hat eine feste Länge. Es gibt keine Methoden *Add()* oder *Remove()*. Deshalb verwenden die meisten Programme den Typ *List<T>*. Die Collections liegen im Namespace *System.Collections.Generic*. Du bindest ihn mit *using* ein:

```
using System.Collections.Generic;
```

Interner Aufbau

List<T> speichert die Daten intern in einem Array. Eine neue, leere Liste hat ein leeres Array. Beim ersten *Add()* bekommt das Array 4 Plätze. Ist das Array voll, legt *Add()* ein neues Array mit doppelter Größe an und kopiert alle Elemente hinein. Das alte Array entfernt der Garbage Collector. Der Zugriff über den Index ist deshalb genauso schnell wie bei einem Array.

Count liefert die Anzahl der Elemente, *Capacity* die Länge des internen Arrays. Die Grafik zeigt die ersten 9 Aufrufe von *Add()*:



Eine Liste anlegen

Für die Beispiele verwenden wir die Klasse *Person* mit 3 Properties:

```
class Person
{
    public Person(int id, string firstname, string lastname)
    {
        Id = id;
        Firstname = firstname;
        Lastname = lastname;
    }

    public int Id { get; }
    public string Firstname { get; set; }
    public string Lastname { get; set; }

    public override string ToString() => $"{Id} - {Firstname} {Lastname}";
}
```

Den Typ der Elemente gibst du in spitzen Klammern an. Mit einem Initializer kannst du die Liste gleich befüllen:

```
// using System.Collections.Generic;
List<Person> emptyList = new List<Person>();
// kürzer: var persons = new List<Person>()
List<Person> persons = new List<Person>()
{
    new Person(id: 1, firstname: "FN1", lastname: "LN1"),
    new Person(id: 2, firstname: "FN2", lastname: "LN2"),
    new Person(id: 3, firstname: "FN3", lastname: "LN3")
};
```

Elemente lesen, hinzufügen und löschen

Add() fügt ein Element am Ende ein. Der Indexer [] liest ein Element an einer Position, wie bei einem Array. Der erste Index ist 0. Mit *foreach* gehst du durch alle Elemente.

```
persons.Add(new Person(id: 4, firstname: "FN4", lastname: "LN4"));

Person thirdPerson = persons[2];                        // (1)
thirdPerson.Lastname = "Other Name";                    // (2)
Console.WriteLine($"Found {persons.Count} Persons");    // (3)
foreach (Person p in persons)                           // (4)
{
    Console.WriteLine(p);
}
```

- **(1)** *persons[2]* liefert das dritte Element.
- **(2)** Bei Referenztypen speichert die Liste nur Referenzen, nicht die Objekte selbst. *thirdPerson* zeigt also auf dasselbe Objekt wie *persons[2]*. Die Ausgabe zeigt daher auch "Other Name".
- **(3)** *Count* liefert die Anzahl der Elemente.
- **(4)** *List<T>* implementiert das Interface *IEnumerable<T>*. Deshalb funktioniert *foreach*.

Remove() löscht ein Element aus der Liste. Die Klasse *Person* überschreibt *Equals()* nicht. *Remove()* vergleicht daher nur die Referenzen (Adressen). Willst du eine Person löschen, brauchst du die Referenz auf genau das Objekt in der Liste, z. B. *persons.Remove(thirdPerson)*. Ein neues Objekt mit denselben Werten hat eine andere Adresse und wird nicht gefunden.

Dictionary<TKey, TValue> (HashMap in Java)

In einer Liste greifst du über den Index zu. Suchst du z. B. die Person mit der ID 2, musst du die Liste durchlaufen. Im schlechtesten Fall (das Element ist nicht vorhanden) sind das n Vergleiche.

Ein *Dictionary* speichert Paare aus *Key* und *Value*. Jeder Key darf nur einmal vorkommen. Über den Key findest du den Value direkt, ohne die Collection zu durchlaufen. Als Key kannst du jeden Typ verwenden, auch eigene Klassen.

Das Beispiel verwendet einen *string* als Key und eine *Person* als Value:

```
// kürzer: var personsDict = new Dictionary<string, Person>()
Dictionary<string, Person> personsDict = new Dictionary<string, Person>()
{
    {"A", new Person(id: 1, firstname: "FN1", lastname: "LN1") },
    {"B", new Person(id: 2, firstname: "FN2", lastname: "LN2") },
    {"C", new Person(id: 3, firstname: "FN3", lastname: "LN3") }
};

// Add benötigt 2 Argumente: Key und Value.
personsDict.Add("D", new Person(id: 4, firstname: "FN4", lastname: "LN4"));
```

Zugriff auf Elemente

Der Indexer liefert den Value zu einem Key, hier die Person mit dem Key "B". Mit *foreach* bekommst du Objekte vom Typ *KeyValuePair*. Sie enthalten den Key im Property *Key* und das Objekt im Property *Value*.

```
Person personB = personsDict["B"];
foreach (KeyValuePair<string, Person> p in personsDict)  // kürzer: foreach (var p in
personsDict)
{
    Console.WriteLine($"Person {p.Key} hat den Zunamen {p.Value.Lastname}");
}
```

Remove() löscht ein Element über seinen Key:

```
personsDict.Remove("A");
```

TryGetValue() und TryAdd()

In zwei Fällen wirft ein Dictionary eine Exception:

- *Add()* mit einem Key, den es schon gibt: *ArgumentException (An item with the same key has already been added).*
- Lesen über den Indexer mit einem Key, den es nicht gibt: *KeyNotFoundException.*

```
// ArgumentException: An item with the same key has already been added
personsDict.Add("D", new Person(id: 5, firstname: "FN5", lastname: "LN5"));
// KeyNotFoundException
Person notFound = personsDict["Z"];
```

Besser sind daher *TryGetValue()* und *TryAdd()*. Sie liefern *true* oder *false* und werfen keine Exception:

```
if (personsDict.TryGetValue("C", out Person? found))
```

```
{
    Console.WriteLine(found.Lastname);
}
if (!personsDict.TryAdd("D", new Person(id: 4, firstname: "FN4", lastname: "LN4")))
{
    Console.WriteLine("Person D ist bereits im Dictionary.");
}
```

HashSet<T> (HashSet in Java)

Ein *HashSet* speichert jeden Wert nur einmal, ähnlich wie *DISTINCT* in SQL. *Add()* fügt nur den ersten Wert ein. Gleiche Werte danach werden ignoriert. Ob zwei Werte gleich sind, prüft das HashSet mit *Equals()* und *GetHashCode()*.

```
HashSet<string> teacherHashSet = new HashSet<string>();
teacherHashSet.Add("SZ");
// Wird einfach ignoriert.
teacherHashSet.Add("SZ");
foreach(string teacher in teacherHashSet)
{
    Console.WriteLine(teacher);                // Gibt 1x SZ aus.
}
```

Ein HashSet hat keinen Index. Du kannst also nicht auf das n-te Element zugreifen. Meist verwendest du es mit *Contains()*. Diese Suche ist sehr schnell: Das HashSet berechnet aus dem Hashcode direkt die Stelle, an der der Wert liegen muss. Es muss nicht alle Elemente durchlaufen.

```
if (teacherHashSet.Contains("SZ")) { ... }
```

Read-only Collections: IReadOnlyList, IReadOnlySet und IReadOnlyDictionary

Oft verwaltet eine Klasse intern eine Collection, die man von außen nur lesen soll. Die Klasse *Course* prüft beim Hinzufügen, ob der Kurs schon voll ist. Wäre die Liste als public *List<Person>* sichtbar, könnte jeder mit *course.Persons.Add()* diese Prüfung umgehen.

```
class Course
{
    private readonly List<Person> _persons = new();
    public IReadOnlyList<Person> Persons => _persons.AsReadOnly();

    public void AddPerson(Person person)
    {
        if (_persons.Count >= 20) { throw new InvalidOperationException("Der Kurs ist
voll."); }
```

```
        _persons.Add(person);
    }
}
```

Die Liste selbst ist *private readonly*. Das Property gibt nach außen nur ein Interface zurück. Dieses Interface hat nur Methoden zum Lesen. Für jede der 3 Collections gibt es ein passendes Interface:

Collection	Read-only Interface	Was ist erlaubt?
List<T>	*IReadOnlyList<T>*	*Count*, Indexer [], *foreach*
HashSet<T>	*IReadOnlySet<T>*	*Count*, *Contains()*, *foreach*
Dictionary<TKey, TValue>	*IReadOnlyDictionary<TKey, TValue>*	*Count*, Indexer [], *ContainsKey()*, *TryGetValue()*, *Keys*, *Values*, *foreach*

Add(), *Remove()* oder *Clear()* gibt es in diesen Interfaces nicht. Rufst du sie trotzdem auf, meldet schon der Compiler einen Fehler.

Warum AsReadOnly() und kein Typecast?

List<T> implementiert *IReadOnlyList<T>*. Deshalb kompiliert auch ein impliziter Typecast:

```
public IReadOnlyList<Person> Persons => _persons;   // Typecast, nicht empfohlen!
```

Das Problem: Ein Typecast ändert nur den Typ der Variable, nicht das Objekt im Speicher. Hinter *Persons* steht weiterhin die interne Liste. Mit einem expliziten Typecast zurück auf *List<Person>* kann man sie wieder verändern. Die Prüfung in *AddPerson()* ist dann wirkungslos:

```
List<Person> hacked = (List<Person>)course.Persons;   // Funktioniert!
hacked.Add(new Person(id: 99, firstname: "FN99", lastname: "LN99"));
hacked.Clear();                                       // Die interne Liste der Klasse
ist leer.
```

AsReadOnly() erzeugt dagegen ein neues Objekt vom Typ *ReadOnlyCollection<T>* (Namespace *System.Collections.ObjectModel*). Dieses Objekt ist ein *Wrapper*: Es umhüllt die interne Liste und erlaubt nur lesende Zugriffe. Ein Typecast auf *List<Person>* ist nicht mehr möglich:

```
List<Person> hacked = (List<Person>)course.Persons;   // InvalidCastException
IList<Person> list = (IList<Person>)course.Persons;   // OK, aber...
list.Add(new Person(id: 99, firstname: "FN99", lastname: "LN99"));  //
NotSupportedException
```

AsReadOnly() kopiert die Daten nicht. Der Wrapper zeigt immer auf die interne Liste. Spätere Änderungen durch *AddPerson()* siehst du daher auch über *Persons*. Der Aufruf kostet kaum Zeit und kann direkt im Property stehen.

Die Methode gibt es für alle 3 Collections:

```
private readonly List<Person> _persons = new();
private readonly HashSet<string> _cities = new();
private readonly Dictionary<int, Person> _personsById = new();

public IReadOnlyList<Person> Persons => _persons.AsReadOnly();                    //
ReadOnlyCollection<T>
public IReadOnlySet<string> Cities => _cities.AsReadOnly();                       //
ReadOnlySet<T>
public IReadOnlyDictionary<int, Person> PersonsById => _personsById.AsReadOnly();  //
ReadOnlyDictionary<TKey, TValue>
```

> Read-only gilt nur für die Collection, nicht für die Objekte darin. Mit *course.Persons[0].Lastname = "X"* kannst du die Person trotzdem ändern, wenn *Lastname* einen public Setter hat.

Vererbung 1: Implementierung in C#

Projekt anlegen

Für die Beispiele brauchst du eine .NET Konsolenapplikation. Führe diese Befehle in der Konsole aus. Unter macOS ersetzt du *rd* und *md* durch *rm -rf* und *mkdir*.

```
rd /S /Q InheritanceDemo
md InheritanceDemo
cd InheritanceDemo
md InheritanceDemo.Application
cd InheritanceDemo.Application
dotnet new console
cd ..
dotnet new sln -f sln
dotnet sln add InheritanceDemo.Application
start InheritanceDemo.sln
```

Öffne danach die Projektdatei *InheritanceDemo.Application.csproj* (Doppelklick auf das Projekt). Entferne die Zeile *ImplicitUsings* und füge *TreatWarningsAsErrors* hinzu. Die Datei sieht dann so aus:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

</Project>
```



- Durch *TreatWarningsAsErrors* ist jede Warnung ein Fehler. Das Programm lässt sich dann nicht kompilieren.
- Ohne *ImplicitUsings* musst du jeden Namespace selbst mit *using* einbinden (z. B. *using System;* für *Console*). So siehst du genau, woher eine Klasse kommt.

Unterschiede zu Java

Vererbung funktioniert in C# fast wie in Java. Es gibt aber einige Unterschiede:

Java	C#
`class B extends A implements I`	`class B : A, I`

Java	C#
Jede Methode kann überschrieben werden.	Nur Methoden mit `virtual` (oder `abstract`) können überschrieben werden.
`@Override` ist optional.	`override` ist Pflicht.
`super(...)` im Konstruktor	`: base(...)` nach dem Konstruktorkopf
`super.method()`	`base.Method()`
`this(...)` im Konstruktor	`: this(...)` nach dem Konstruktorkopf
`final class`	`sealed class`

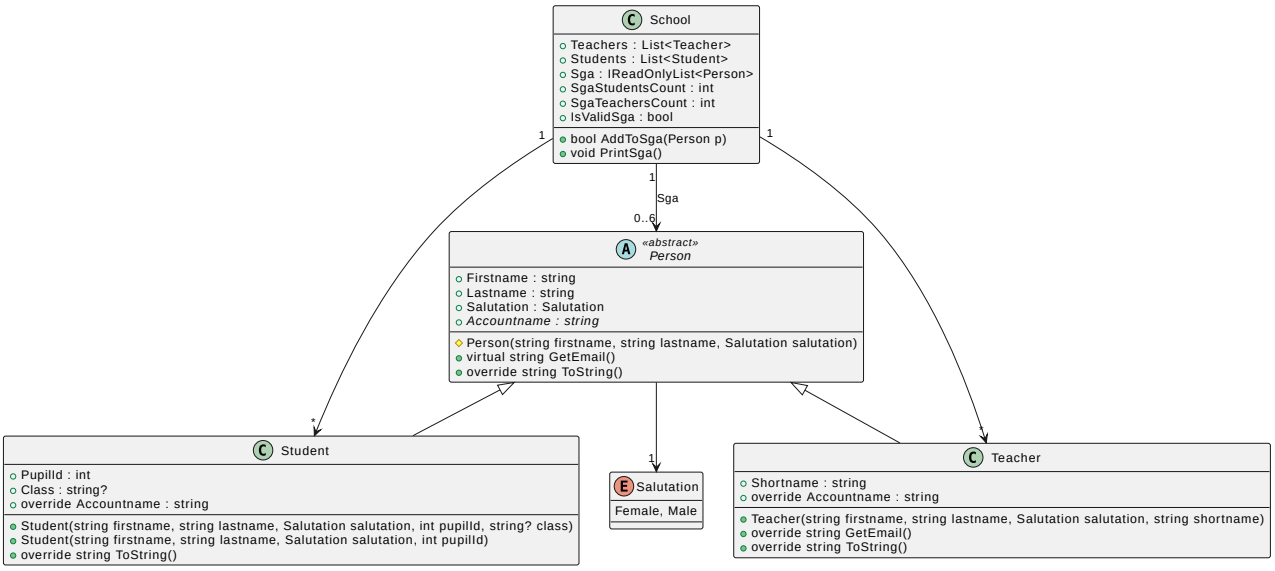
Gleich wie in Java:

- Eine Klasse kann nur von **1 Klasse** erben, aber **beliebig viele Interfaces** implementieren.
- `abstract` hat dieselbe Bedeutung.

Neu in C#:

- **Properties** sind intern Methoden. Sie können daher auch `virtual`, `abstract` oder `override` sein.
- `new` vor einer Methode **verdeckt** eine Methode der Basisklasse (siehe Teacher, Punkt 4).

Das UML Klassendiagramm



PlantUML Quelle: vererbung_diagram.puml

So liest du das Diagramm:

Element	Bedeutung
A im Kreis, *«abstract»*	Abstrakte Klasse, z. B. *Person*
kursiv	Abstraktes Member, z. B. *Accountname*
E im Kreis	Enum, z. B. *Salutation* (= Anrede)
Pfeil mit leerem Dreieck	Vererbung. Der Pfeil zeigt zur Basisklasse.

Element	Bedeutung
Einfacher Pfeil (Assoziation)	*School* speichert Teacher in einem Property (*Teachers*). Der Pfeil zeigt die Navigierbarkeit: Von *School* kommst du zu *Teacher*, aber nicht umgekehrt.
Zahlen an den Pfeilenden	Multiplizität: *1 School* hat ** (beliebig viele) Teacher*. *0..6* heißt 0 bis 6.
+ / #	*public* / *protected*
virtual, *override*	C# Schlüsselwörter der Methode, z. B. *virtual string GetEmail()*

Das ist kein reines UML: Im Diagramm stehen C# Datentypen (*string*, *bool*, …) statt der UML Typen (*String*, *Boolean*, …). Auch *virtual* und *override* gibt es in UML nicht. Wir schreiben sie dazu, damit du siehst, welche Methoden überschrieben werden.

Weitere Beziehungsarten (Abhängigkeit, Komposition) lernst du im Kapitel Interfaces kennen.

Die Klasse Person

Jede Klasse kommt in eine eigene Datei. Der Namespace entspricht dem Projektnamen.

Salutation.cs

```
namespace InheritanceDemo.Application;

enum Salutation { Female = 1, Male }   // (1)
```

Person.cs

```
namespace InheritanceDemo.Application;

abstract class Person   // (2a)
{
    protected Person(string firstname, string lastname, Salutation salutation)
    {
        Firstname = firstname;
        Lastname = lastname;
        Salutation = salutation;
    }

    public string Firstname { get; }
    public string Lastname { get; }
    public Salutation Salutation { get; }
    public abstract string Accountname { get; }                          // (2)
    public virtual string GetEmail() => $"{Accountname}@spengergasse.at";   // (3)
    public override string ToString() => $"{Firstname} {Lastname}";         // (4)
}
```

- **(1)** Ein *enum* gibt einem *int* Wert einen Namen. Wir beginnen mit 1. So ist der Defaultwert 0 kein gültiger Wert. Sonst wäre jeder nicht initialisierte Wert automatisch *Female*.
- **(2)** *Accountname* ist ein abstraktes read-only Property. Es hat keine Implementierung. Jede abgeleitete Klasse muss es überschreiben (außer sie ist selbst abstrakt). Eine Klasse mit abstrakten Membern muss selbst *abstract* sein **(2a)**.
- **(3)** *GetEmail()* ist nicht abstrakt, kann aber das abstrakte Property *Accountname* verwenden. Zur Laufzeit wird immer der *Accountname* des echten Objekts (Teacher oder Student) verwendet. *virtual* bedeutet: Abgeleitete Klassen **können** die Methode mit *override* überschreiben. Sie **müssen** es aber nicht.
- **(4)** *ToString()* ist in *System.Object* als *virtual* definiert. Daher können wir sie mit *override* überschreiben.

Die Klasse Teacher

Teacher.cs

```
namespace InheritanceDemo.Application;

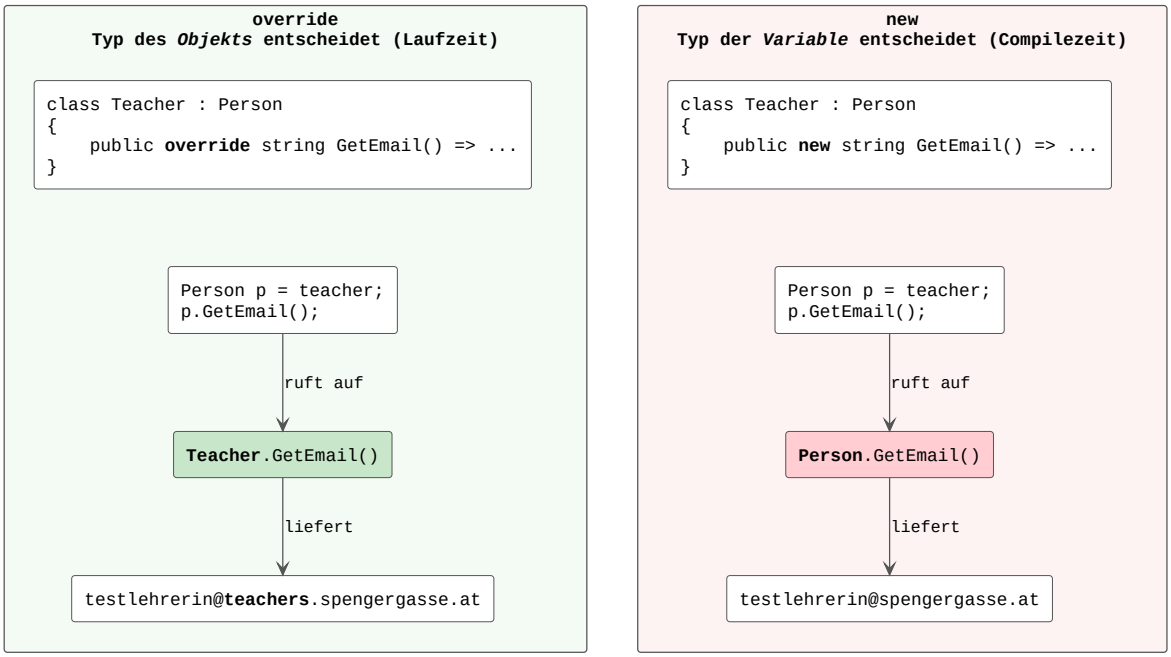
class Teacher : Person   // (1)
{
    public Teacher(string firstname, string lastname, Salutation salutation, string
shortname)   // (2)
        : base(firstname: firstname, lastname: lastname, salutation: salutation)
    {
        Shortname = shortname;
    }

    public string Shortname { get; }
    public override string Accountname => Lastname.ToLower();
// (3)
    public override string GetEmail() => $"{Accountname}@teachers.spengergasse.at";
// (4)
    public override string ToString() => $"Teacher {Shortname} - {Firstname} {Lastname
}";   // (5)
}
```

- **(1)** Der Doppelpunkt kennzeichnet die Vererbung. Es gibt kein *extends* oder *implements*.
- **(2)** Ein Teacher ist auch eine Person. Beim Erstellen eines Teachers muss daher auch der Person-Teil erstellt werden. *Person* hat keinen Defaultkonstruktor. Mit *base(...)* rufen wir den Konstruktor von *Person* auf (wie *super(...)* in Java).
- **(3)** *Accountname* ist abstrakt. Wir **müssen** es mit *override* überschreiben.
- **(4)** *GetEmail()* ist *virtual*. Wir **können** sie mit *override* überschreiben. Was passiert ohne *override*?
  - Der Compiler meldet eine Warnung: Die Methode **verdeckt** *Person.GetEmail()*. Wegen *TreatWarningsAsErrors* ist das ein Fehler.
  - Mit dem Schlüsselwort *new* (`public new string GetEmail()`) verschwindet die Warnung. Die Methode wird dann aber nicht überschrieben, sondern nur verdeckt:

```
Person p = teacher;
p.GetEmail();   // Calls Person.GetEmail(), not Teacher.GetEmail()!
```

Das ist fast nie gewünscht. Verwende daher *override*.



PlantUML Quelle: override_vs_new.puml

- **(5)** *Person* überschreibt *ToString()* bereits. Eine *override* Methode ist selbst wieder *virtual*. Daher kann *Teacher* sie noch einmal überschreiben.

Die Klasse Student

Student.cs

```
namespace InheritanceDemo.Application;

class Student : Person
{
    public Student(string firstname, string lastname, Salutation salutation, int
pupilId, string? @class)
        : base(firstname: firstname, lastname: lastname, salutation: salutation)
    {
        PupilId = pupilId;
        Class = @class;     // (1)
    }

    public Student(string firstname, string lastname, Salutation salutation, int
pupilId)   // (2)
        : this(firstname: firstname, lastname: lastname, salutation: salutation,
pupilId: pupilId, @class: null)
```

```
    {
    }

    public int PupilId { get; }
    public string? Class { get; }
    public override string Accountname =>
        (Lastname.Length < 3 ? Lastname : Lastname.Substring(0, 3)).ToLower() +
PupilId.ToString("000000");
    public override string ToString() => $"Student {base.ToString()}";   // (3)
}
```

- **(1)** *class* ist ein reserviertes Wort. Mit dem Zeichen @ davor (*@class*) können wir es trotzdem als Variablenname verwenden.
- **(2)** *Student* hat 2 Konstruktoren. Mit *this(...)* leiten wir die Argumente an den anderen Konstruktor weiter. So müssen wir den Code nicht kopieren. Für die fehlende Klasse übergeben wir *null*.
- **(3)** Mit *base.ToString()* rufen wir die Implementierung der Basisklasse (*Person*) auf.

Die Klasse School

Der SGA (Schulgemeinschaftsausschuss) ist ein Gremium der Schule. Er hat 9 Mitglieder. In unserem Beispiel verwalten wir nur die 3 Lehrer- und die 3 Schülervertreter.

School.cs

```
using System;                          // (7)
using System.Collections.Generic;

namespace InheritanceDemo.Application;

class School
{
    private readonly List<Person> _sga = new(9);              // (1)
    public List<Teacher> Teachers { get; } = new();
    public List<Student> Students { get; } = new();
    public IReadOnlyList<Person> Sga => _sga;                // (2)
    public int SgaStudentsCount { get; private set; } = 0;   // (3)
    public int SgaTeachersCount { get; private set; } = 0;
    public bool IsValidSga => SgaTeachersCount == 3 && SgaStudentsCount == 3;

    public bool AddToSga(Person p)   // (4)
    {
        if (p is Student && SgaStudentsCount < 3) { _sga.Add(p); SgaStudentsCount++;
return true; }
        if (p is Teacher && SgaTeachersCount < 3) { _sga.Add(p); SgaTeachersCount++;
return true; }
        return false;
    }

    public void PrintSga()
    {
```

```
        foreach (Person p in Sga)
        {
            if (p is Teacher)   // (5)
            {
                Console.WriteLine($"Lehrervertreter {p.Lastname}, Email: {p.GetEmail
()}");
            }
            if (p is Student s)   // (6)
            {
                Console.WriteLine($"Schülervertreter {p.Lastname} in der Klasse {s
.Class ?? "?"}, Email: {p.GetEmail()}");
            }
        }
    }
}
```

- **(1)** *readonly*: Die Variable *_sga* kann nicht neu zugewiesen werden. *Add()*, *Remove()*, … sind aber trotzdem möglich! *new(9)* ist die Kurzform von *new List<Person>(9)*. Der Compiler kennt den Typ von der linken Seite. Die 9 ist die Anfangsgröße (Capacity) der Liste.
- **(2)** Von außen soll niemand die Liste ändern können (z. B. 10 Schülervertreter einfügen). *List<Person>* implementiert das Interface *IReadOnlyList<Person>*. Das Property liefert die Liste als *IReadOnlyList<Person>*. Der Aufrufer sieht daher nur die Lesemethoden.
- **(3)** Die Anzahl kann von außen gelesen werden (*get*). Ändern kann sie nur die Klasse *School* (*private set*).
- **(4)** Nur über diese Methode kommen Personen in den SGA. So können wir prüfen, dass es maximal 3 Schüler- und 3 Lehrervertreter gibt.
- **(5)** Die Liste enthält Personen, also Lehrer **und** Schüler. Mit *is* prüfen wir den Typ. Für *GetEmail()* brauchen wir keinen Cast. Die Methode ist überschrieben. Daher wird auch über eine Variable vom Typ *Person* die Methode von *Teacher* aufgerufen.
- **(6)** Für *Class* brauchen wir ein *Student* Objekt. *p is Student s* prüft den Typ und castet in einem Schritt. Das kennst du aus Java: `if (p instanceof Student s)`.
- **(7)** *Console* liegt im Namespace *System*, *List<T>* und *IReadOnlyList<T>* liegen in *System.Collections.Generic*. Ohne diese *using* Anweisungen kompiliert der Code nicht.

Testprogramm und Ausgabe

Program.cs

```
namespace InheritanceDemo.Application;

class Program
{
    private static void Main(string[] args)
    {
        Teacher teacher = new Teacher(firstname: "Eva", lastname: "Testlehrerin",
salutation: Salutation.Female, shortname: "TES");
        Student student = new Student(firstname: "Stefan", lastname: "Eifrig",
salutation: Salutation.Male, pupilId: 1001, @class: "3AHIF");
```

```
        Student student2 = new Student(firstname: "Lukas", lastname: "Abschreiber",
salutation: Salutation.Male, pupilId: 1002);

        School school = new School();
        school.Teachers.Add(teacher);
        school.Students.Add(student);
        school.Students.Add(student2);
        school.AddToSga(teacher);
        school.AddToSga(student);
        school.AddToSga(student2);

        school.PrintSga();
    }
}
```

```
Lehrervertreter Testlehrerin, Email: testlehrerin@teachers.spengergasse.at
Schülervertreter Eifrig in der Klasse 3AHIF, Email: eif001001@spengergasse.at
Schülervertreter Abschreiber in der Klasse ?, Email: abs001002@spengergasse.at
```

Interfaces und Dependencies

Was sind Dependencies (Abhängigkeiten)?

Klasse A ist von Klasse B **abhängig**, wenn A die Klasse B braucht, um zu kompilieren. Das ist der Fall, wenn A den Typ B verwendet: als Feld, Property, Parameter, Rückgabewert, lokale Variable oder Basisklasse.

Warum ist das wichtig? Jede Abhängigkeit ist eine Verbindung im Code. Ändert sich B, muss eventuell auch A geändert werden. Viele Abhängigkeiten machen Software schwer änderbar und schwer testbar. Man spricht von **starker Kopplung**. Ein Ziel guter Software ist daher **lose Kopplung**: wenige Abhängigkeiten, und zwar zu Typen, die sich selten ändern. Interfaces sind dafür das wichtigste Werkzeug in C#.

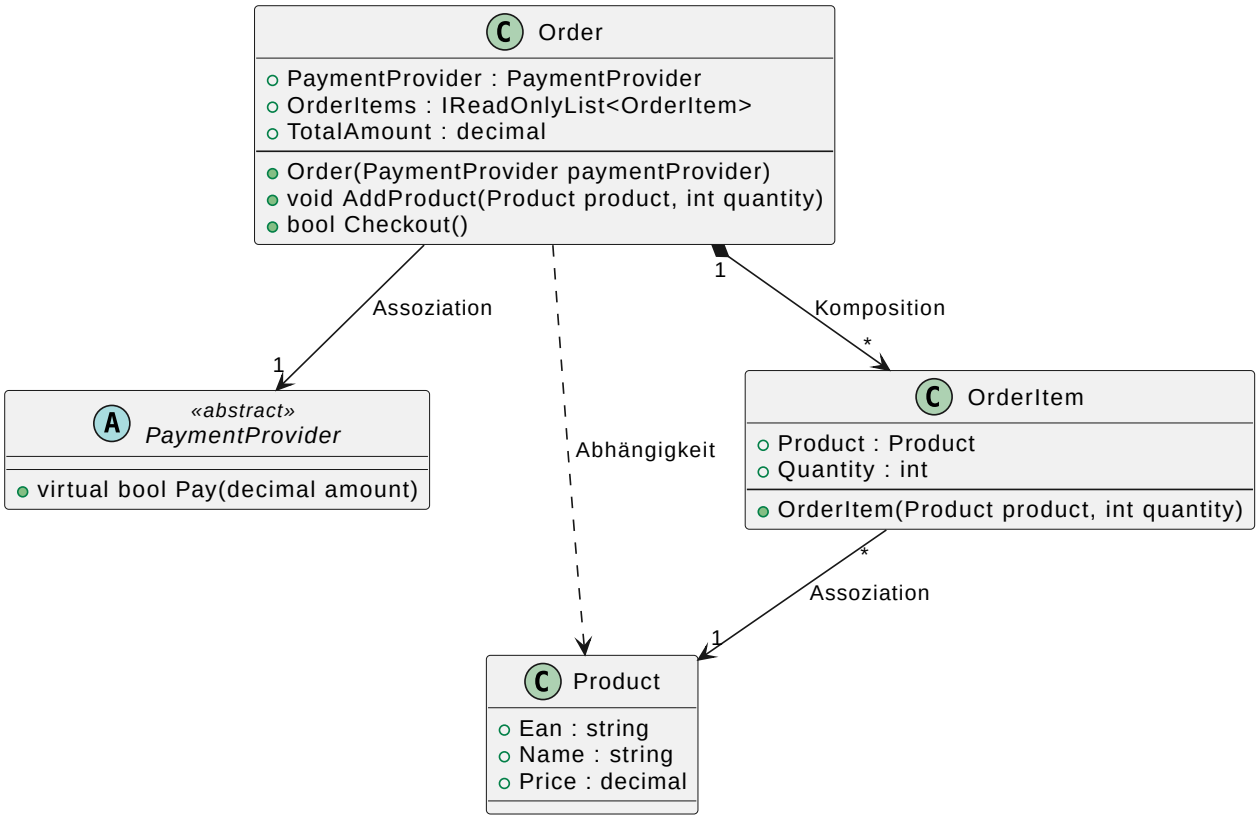
Beziehungsarten im Klassendiagramm

Für die Praxis sind 3 Beziehungsarten wichtig. Du erkennst sie im Code, und sie haben echte Konsequenzen. Sie sind nach Stärke der Bindung sortiert, die schwächste steht oben:

Beziehung	Symbol	Im Code erkennbar an	Konsequenz
Abhängigkeit	gestrichelter Pfeil `..>`	B nur als Parameter, Rückgabewert oder lokale Variable	A muss neu kompiliert werden, wenn sich B ändert.
Assoziation	Pfeil `-->`	B als Feld oder Property	A hält eine Referenz auf B. B lebt unabhängig von A.
Komposition	gefüllte (schwarze) Raute `*-->`	A erzeugt B selbst mit *new* und gibt B nicht zum Ändern heraus	B gehört nur A. B lebt und stirbt mit A.

Beispiel: Bestellung

Wir verwenden *Product* und *PaymentProvider* aus der Übung im Kapitel Vererbung. Eine Bestellung (*Order*) besteht aus Positionen (*OrderItem*). Jede Position verweist auf ein Produkt und hat eine Menge.



PlantUML Quelle: assoziationen.puml

OrderItem.cs

```
class OrderItem
{
    public OrderItem(Product product, int quantity)
    {
        Product = product;
        Quantity = quantity;
    }

    public Product Product { get; }   // (1)
    public int Quantity { get; }
}
```

Order.cs

```
using System.Collections.Generic;
using System.Linq;

class Order
{
    private readonly List<OrderItem> _orderItems = new();

    public Order(PaymentProvider paymentProvider)
    {
        PaymentProvider = paymentProvider;
```

```
    }

    public PaymentProvider PaymentProvider { get; }                    // (2)
    public IReadOnlyList<OrderItem> OrderItems => _orderItems;         // (3)
    public decimal TotalAmount => _orderItems.Sum(i => i.Product.Price * i.Quantity);

    public void AddProduct(Product product, int quantity)              // (4)
    {
        _orderItems.Add(new OrderItem(product, quantity));             // (3)
    }

    public bool Checkout() => PaymentProvider.Pay(TotalAmount);
}
```

- **(1) Assoziation OrderItem → Product:** *OrderItem* speichert ein *Product*. Das Produkt existiert unabhängig davon im Katalog, und viele Positionen verweisen auf dasselbe Produkt.
- **(2) Assoziation Order → PaymentProvider:** *Order* speichert den Payment Provider. Eine Kreditkarte bezahlt viele Bestellungen und existiert auch ohne sie.
- **(3) Komposition Order → OrderItem:** *Order* erzeugt die Positionen selbst mit *new*. Von außen kann niemand eine Position hinzufügen, denn *OrderItems* ist eine *IReadOnlyList*. Eine Position gehört also immer zu genau einer Bestellung. Gibt es die Bestellung nicht mehr, hat niemand mehr eine Referenz auf ihre Positionen. Der Garbage Collector entfernt sie.
- **(4) Abhängigkeit Order → Product:** *Order* verwendet *Product* nur als Parameter. Es speichert kein Produkt in einem eigenen Feld.

Multiplizität: Muss es das Objekt geben?

Die Zahlen an den Pfeilenden sind die **Multiplizität**. Sie haben direkte Folgen für den Code und für die Datenbank:

Multiplizität	Bedeutung	C#	Datenbank
1	genau eines, Pflicht	Konstruktorparameter, nicht nullable (*PaymentProvider*)	Fremdschlüssel *NOT NULL*
0..1	höchstens eines, optional	nullable Property (*PaymentProvider?*)	Fremdschlüssel *NULL*
*	beliebig viele	Collection (*IReadOnlyList<OrderItem>*)	Fremdschlüssel in der anderen Tabelle

Order kann ohne *PaymentProvider* nicht erstellt werden. Das drückt die **1** am Pfeil zu *PaymentProvider* aus, **nicht** eine Komposition.

Komposition: Wann ist das wichtig?



Verwechslungsgefahr: Komposition heißt nicht „A braucht B“. Komposition heißt „B lebt und stirbt mit A“. Stelle dir 2 Fragen:

1. Verschwindet B, wenn A verschwindet?
2. Gehört B nur zu diesem einen A?

Nur wenn beide Antworten *ja* sind, ist es eine Komposition. Bei *Order → OrderItem* ist das so. Bei *Order → PaymentProvider* nicht, denn die Kreditkarte gibt es weiter.

Die Komposition hat in der Praxis diese Konsequenzen:

- **Kapselung:** Positionen werden nur über *Order* geändert (*AddProduct()*). So kann *Order* Regeln prüfen, z. B. eine maximale Anzahl an Positionen.
- **Datenbank:** Wird eine Bestellung gelöscht, werden ihre Positionen mitgelöscht (*ON DELETE CASCADE*). Die Produkte bleiben.
- **Domain-Driven Design:** Eine Gruppe von Objekten, die nur über ein Hauptobjekt geändert wird, heißt dort *Aggregate*. Das Hauptobjekt (hier *Order*) heißt *Aggregate Root*.

Pfeilrichtung

Der Pfeil zeigt die **Navigierbarkeit**: Von *OrderItem* kommst du über das Property *Product* zum Produkt. Umgekehrt geht das nicht. Ein solches Property heißt *navigation property*. Bei der Komposition liegt die Raute beim **Ganzen** (*Order*).



Und die Aggregation (leere Raute `o-->`)? Du wirst sie in manchen Diagrammen sehen. Sie soll eine Teil-Ganzes Beziehung zeigen, bei der das Teil unabhängig existiert. Im Code sieht sie aber genau wie eine Assoziation aus und hat keine eigenen Konsequenzen. Selbst die UML Spezifikation legt ihre Bedeutung nicht genau fest. Behandle sie daher wie eine Assoziation und verwende sie selbst nicht.

Interfaces als Contract (Vertrag)

Sieh dir *Checkout()* in *Order* an:

```
public bool Checkout() => PaymentProvider.Pay(TotalAmount);
```

Order braucht vom Payment Provider nur **eine** Sache: die Methode *Pay()*. Wie sie bezahlt, ist für *Order* egal. Trotzdem hängt *Order* von der Klasse *PaymentProvider* ab, also von einer konkreten Klasse mit Feldern (*Limit*) und Logik.

Besser: Wir beschreiben nur, **was** *Order* braucht. Dafür definieren wir ein Interface:

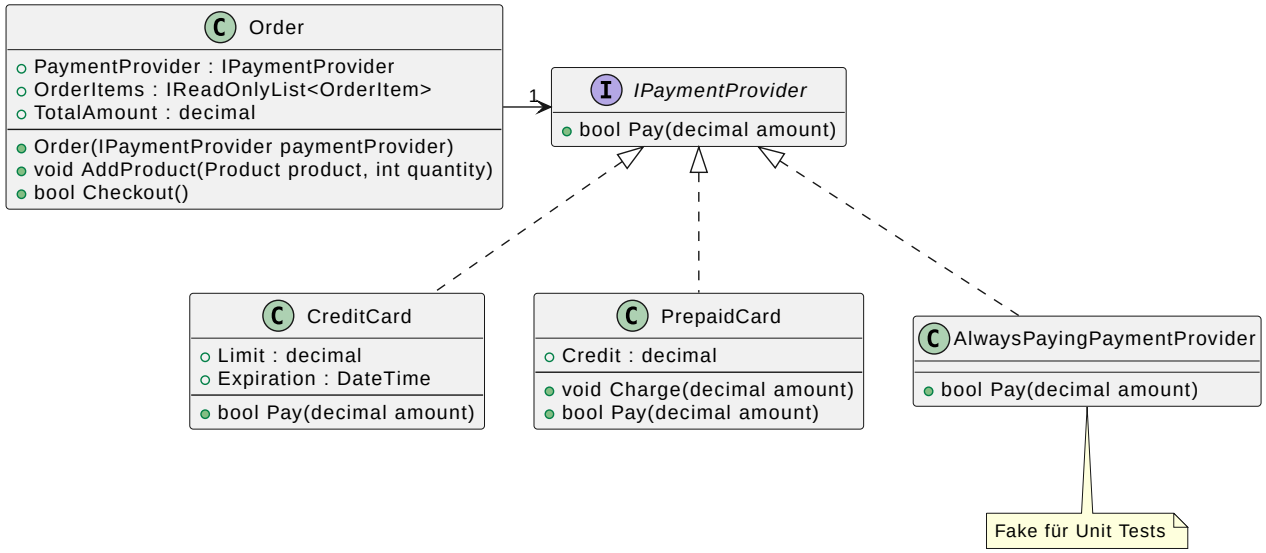
```
interface IPaymentProvider
{
    bool Pay(decimal amount);
}
```

- Interfaces beginnen in .NET mit einem großen **I** (Konvention).
- Alle Members sind automatisch *public*. Ein Vertrag muss für alle sichtbar sein.
- Ein Interface enthält keine Felder, also keinen Zustand.
- Wie in Java ab Version 8 kann ein Interface seit C# 8 auch Default-Implementierungen haben. Das brauchst du aber selten.

Order verwendet jetzt nur noch das Interface:

```
public Order(IPaymentProvider paymentProvider)
{
    PaymentProvider = paymentProvider;
}

public IPaymentProvider PaymentProvider { get; }
```



PlantUML Quelle: iPaymentProvider.puml

Die Klassen *CreditCard*, *PrepaidCard* usw. **implementieren** (realisieren) das Interface. Im Diagramm ist das der gestrichelte Pfeil mit leerem Dreieck. *Order* kennt keine dieser Klassen.

CreditCard.cs

```
using System;

class CreditCard : IPaymentProvider
{
    public CreditCard(decimal limit, DateTime expiration)
    {
        Limit = limit;
        Expiration = expiration;
    }

    public decimal Limit { get; }
    public DateTime Expiration { get; }
```

```
    public bool Pay(decimal amount)
    {
        if (amount > Limit) { return false; }
        if (Expiration < DateTime.UtcNow) { return false; }
        return true;
    }
}
```

Limit und Ablaufdatum sind Details von *CreditCard*. *Order* sieht davon nichts.

Warum Interfaces?

Vorteil	Erklärung
Austauschbarkeit	Eine neue Zahlungsart (z. B. PayPal) ist eine neue Klasse. *Order* bleibt unverändert. Das ist das *Open/Closed Principle* (offen für Erweiterung, geschlossen für Änderung).
Testbarkeit	Im Unit Test übergeben wir eine einfache Fake-Klasse. Wir brauchen keine echte Kreditkarte.
Lose Kopplung	*Order* hängt nur von einem kleinen, stabilen Interface ab, nicht von Details. Das ist das *Dependency Inversion Principle*: Klassen mit Geschäftslogik hängen von Abstraktionen (Interfaces) ab, nicht von technischen Details.
Paralleles Arbeiten	Das Interface ist schnell geschrieben. Danach kann ein Entwickler *Order* programmieren und ein anderer *CreditCard*.

Ein Fake für Tests ist nur ein paar Zeilen lang:

```
class AlwaysPayingPaymentProvider : IPaymentProvider
{
    public bool Pay(decimal amount) => true;
}

Order order = new Order(new AlwaysPayingPaymentProvider());
```

Wie die richtige Implementierung zur Laufzeit in die Klasse kommt, lernst du im Kapitel D: Dependency Inversion.



Erstelle nicht für jede Klasse ein Interface. Ein Interface lohnt sich, wenn es mehrere Implementierungen gibt oder geben wird. Das gilt auch für einen Fake im Test.

Interface oder abstrakte Klasse?

	Interface	Abstrakte Klasse
Bedeutung	*kann* (Fähigkeit, Vertrag)	*ist ein* (gemeinsame Basis)
Felder / Zustand	nein	ja
Konstruktor	nein	ja
Anzahl pro Klasse	beliebig viele	nur 1 Basisklasse
Verwende es, wenn …	Klassen dasselbe *können* sollen	Klassen gemeinsamen *Code* teilen

Beides lässt sich kombinieren. Die Limitprüfung aus der Übung im Kapitel Vererbung bleibt in einer abstrakten Klasse. Diese implementiert das Interface:

```
abstract class PaymentProvider : IPaymentProvider
{
    protected PaymentProvider(decimal limit) { Limit = limit; }
    public decimal Limit { get; }
    public virtual bool Pay(decimal amount) => amount <= Limit;
}
```

Order kennt trotzdem nur *IPaymentProvider*. Ob eine Implementierung von *PaymentProvider* erbt oder nicht, ist für *Order* egal.

Properties in Interfaces

Interfaces können auch Properties enthalten. Oft steht nur *get* im Interface. Die Implementierung darf zusätzlich ein *set* haben:

```
interface IPaymentProvider
{
    string NotificationEmail { get; }
    bool Pay(decimal amount);
}

class CreditCard : IPaymentProvider
{
    /* ... */
    public string NotificationEmail { get; private set; } = string.Empty;
    public bool Pay(decimal amount) { /* ... */ }
}
```

Das Interface Segregation Principle

Das **I** in SOLID steht für das *Interface Segregation Principle*:

> "Many client-specific interfaces are better than one general-purpose interface."

Das bedeutet: Eine Klasse soll nur von den Methoden abhängen, die sie wirklich braucht. Daher sind mehrere kleine Interfaces besser als ein großes.

Ein Beispiel ist *List<T>*. Drücke in Visual Studio *F12* auf *List*:

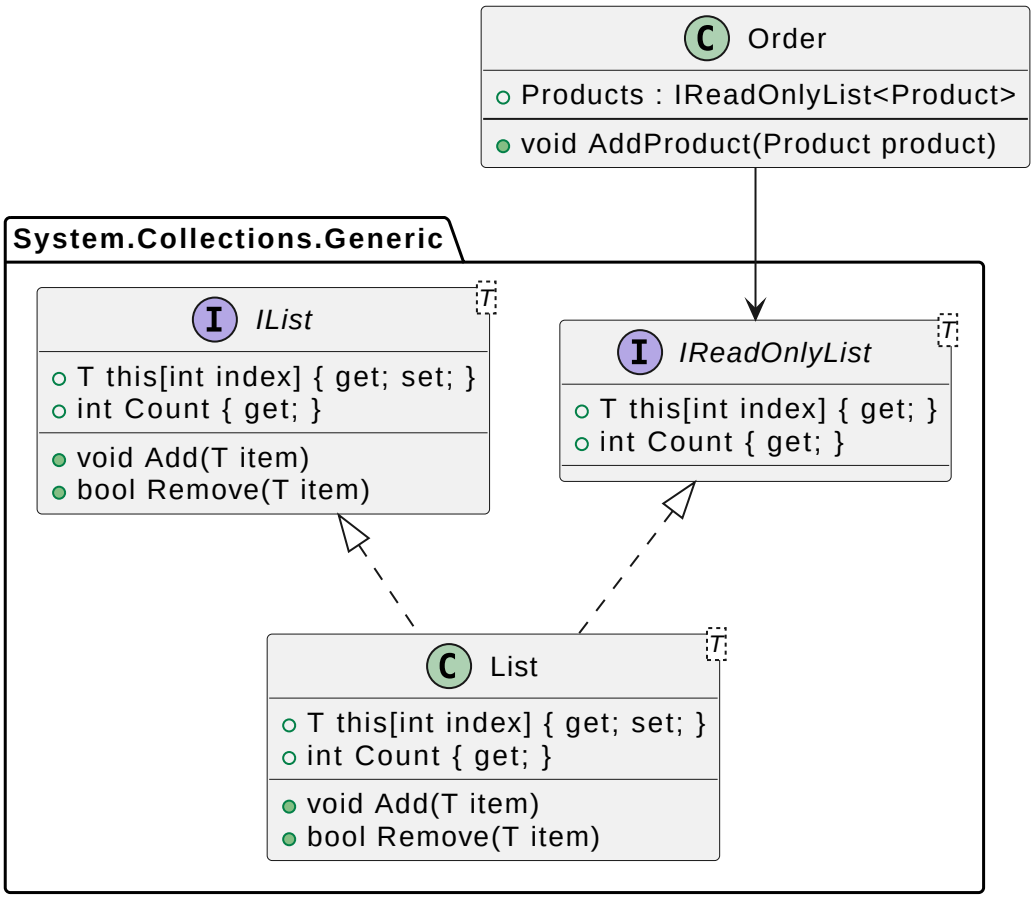
```
public class List<T> : IList<T>, IReadOnlyList<T> // ...
{
    // ...
}
```

List<T> implementiert mehrere Interfaces. Jedes beschreibt eine andere Fähigkeit:

Interface	**Fähigkeit**	**Brauchst du für …**
IEnumerable<T>	Elemente durchlaufen	*foreach*, LINQ
IReadOnlyCollection<T>	+ Anzahl (*Count*)	Anzahl abfragen
IReadOnlyList<T>	+ Zugriff über Index (*list[0]*)	Lesen an einer Position
IList<T>	+ Ändern (*Add*, *Remove*, …)	Liste verändern

Daraus folgen zwei Regeln:

- **Parameter:** Verlange das kleinste Interface, das du brauchst. Eine Methode, die nur mit *foreach* durchläuft, nimmt *IEnumerable<T>*. Dann kann der Aufrufer ein Array, eine Liste, ein Set, … übergeben.
- **Rückgabe:** Gib nur so viel preis, wie der Aufrufer darf. Deshalb liefert *Order.Products* eine *IReadOnlyList<Product>* und keine *List<Product>*.



PlantUML Quelle: ireadonlylist.puml

List<Product> implementiert *IReadOnlyList<Product>*. Daher wandelt der Compiler den Typ automatisch um (implizite Konvertierung):

```
class Order
{
    private readonly List<Product> _products = new();
    public IReadOnlyList<Product> Products => _products;
    public void AddProduct(Product product) => _products.Add(product);
}

Order order = new Order();
Product product = new Product(ean: "1001", name: "Apple iPhone", price: 1800);
order.AddProduct(product);
Product first = order.Products[0];   // OK: IReadOnlyList has an indexer.
order.Products.Remove(product);      // Compiler error: IReadOnlyList has no Remove().
```



Mit einem Cast (*(List<Product>)order.Products*) kann man das umgehen. Ein Interface ist also kein Schutz vor Absicht. Es zeigt aber klar, wie die Klasse verwendet werden soll, und verhindert Fehler aus Versehen.

Das Kapitel I: Interface Segregation vertieft das Prinzip mit Abbildungen zur Collection-Hierarchie und zu weiteren Interfaces aus .NET.

Praktisches Beispiel: Fluent API

Interfaces können auch **Zustände** abbilden. Der Compiler prüft dann, ob Methoden in der richtigen Reihenfolge aufgerufen werden.

Beispiel: Eine Lottoziehung.

- **Vor der Ziehung:** Nur *AddTip()* und *DrawNumbers()* sind erlaubt → Interface *ITippableLottery*.
- **Nach der Ziehung:** Keine neuen Tipps mehr. Nur *CountTips()* und die Ergebnisse sind erlaubt → Interface *IDrawnLottery*.

Die Klasse *Lottery* implementiert beide Interfaces. Jede Methode liefert das Interface für den nächsten Zustand zurück. So kannst du die Aufrufe verketten (*Fluent API*).

Erstelle eine Konsolenapplikation und ersetze *Program.cs* durch den folgenden Code. Prüfe: Kannst du in *Main()* vor *DrawNumbers()* die Methode *CountTips()* aufrufen?

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.Json;

namespace LotteryDemo.Application;

interface ITippableLottery
{
    ITippableLottery AddTip(IEnumerable<int> numbers);
    IDrawnLottery DrawNumbers();
}

interface IDrawnLottery
{
    IReadOnlyList<IReadOnlySet<int>> Tips { get; }
    IReadOnlySet<int> DrawnNumbers { get; }
    int CountTips(int correctNumbers);
}

class Lottery : ITippableLottery, IDrawnLottery
{
    private readonly List<HashSet<int>> _tips = new();
    private readonly HashSet<int> _drawnNumbers = new(6);
    private readonly Random _rnd;

    /// <summary>
    /// Factory method. Starts a new drawing in the state "tippable".
    /// </summary>
    public static ITippableLottery NewDrawing(int seed) => new Lottery(new Random(
seed));

    /// <summary>
    /// Private, so callers cannot create an instance with new and call every method.
```

```
    /// </summary>
    private Lottery(Random rnd)
    {
        _rnd = rnd;
    }

    /// <summary>
    /// List of HashSet can be returned as list of IReadOnlySet because
    /// IReadOnlyList&lt;out T&gt; is covariant (see the out keyword):
    /// https://learn.microsoft.com/en-us/dotnet/csharp/language-
reference/keywords/out-generic-modifier
    /// </summary>
    public IReadOnlyList<IReadOnlySet<int>> Tips => _tips;

    /// <summary>
    /// The drawn numbers. Only accessible after DrawNumbers().
    /// </summary>
    public IReadOnlySet<int> DrawnNumbers => _drawnNumbers;

    /// <summary>
    /// Adds a tip. Accepts the weakest collection type (IEnumerable), so arrays,
lists, ...
    /// can be passed. Tips without 6 different numbers are ignored.
    /// </summary>
    public ITippableLottery AddTip(IEnumerable<int> numbers)
    {
        var set = numbers.ToHashSet();   // Removes duplicate numbers.
        if (set.Count == 6) { _tips.Add(set); }
        return this;
    }

    /// <summary>
    /// Draws 6 different numbers between 1 and 45. Random numbers can repeat,
    /// so we add to a HashSet until it contains 6 numbers.
    /// </summary>
    public IDrawnLottery DrawNumbers()
    {
        while (_drawnNumbers.Count != 6)
        {
            _drawnNumbers.Add(_rnd.Next(1, 46));
        }
        return this;
    }

    /// <summary>
    /// Counts the tips with exactly the given number of correct numbers.
    /// </summary>
    public int CountTips(int correctNumbers)
    {
        int count = 0;
        foreach (var tip in _tips)
        {
            // LINQ, see next chapter: How many numbers of the tip were drawn?
            if (tip.Count(t => _drawnNumbers.Contains(t)) == correctNumbers)
            {
                count++;
```

```
            }
        }
        return count;
    }
}

class Program
{
    public static void Main()
    {
        // A fixed seed returns the same numbers on every run. This makes the result
        // reproducible and testable.
        IDrawnLottery drawing = Lottery
            .NewDrawing(2022)
            .AddTip(new int[] { 1, 2, 3, 4, 5, 6 })
            .AddTip(new int[] { 7, 8, 9, 10, 11, 12 })
            .AddTip(new int[] { 4, 5, 6, 7, 27, 40 })
            .DrawNumbers();

        Console.WriteLine("Abgegebene Tipps:");
        Console.WriteLine(JsonSerializer.Serialize(drawing.Tips));

        Console.WriteLine("Gezogene Zahlen:");
        Console.WriteLine(JsonSerializer.Serialize(drawing.DrawnNumbers));

        foreach (var i in Enumerable.Range(1, 6))
        {
            Console.WriteLine($"{i} richtige: {drawing.CountTips(i)} abgegebene
Tipps");
        }
    }
}
```

Lösung: Nein. *NewDrawing()* und *AddTip()* liefern *ITippableLottery*. Dieses Interface hat keine Methode *CountTips()*. Der Compiler verhindert also den falschen Aufruf.

Vererbung 2: Interfaces implementieren: IEquatable und IComparable

Vererbung wird hauptsächlich dazu verwendet, um ein bestehendes Framework (in unserem Fall das .NET Framework) zu erweitern. Oft werden Interfaces, die bereitgestellt werden, in eigenen Klassen implementiert.

Wir implementieren in diesem Beispiel 2 bekannte Interfaces: *IEquatable<T>* und *IComparable<T>*. Diese Interfaces sind generisch, die bei der Vererbung für einen Typ spezifiziert werden. Diese Technik wird uns bei der Vererbung noch oft begegnen. In unserem Musterbeispiel implementieren wir die Klasse Position, die den Längen- und Breitengrad eines Punktes speichert:

```
class Position : IEquatable<Position>, IComparable, IComparable<Position>
{
    public Position(double lat, double lng)
    {
        Lat = lat;
        Lng = lng;
    }

    // Read only Properties für Lat und Lng, damit sich der Hash nicht mehr ändert.
    public double Lat { get; }
    public double Lng { get; }
    public double LatRad => Lat * Math.PI / 180;
    public double LngRad => Lng * Math.PI / 180;
    /// <summary>
    /// Vergleicht 2 Positionen über die geografische Breite (Lat)
    /// </summary>
    public int CompareTo(Position? other) => Lat.CompareTo(other?.Lat);
    public int CompareTo(object? obj) => CompareTo(obj as Position);

    /// <summary>
    /// 2 Punkte sind ident, wenn sie die selbe Länge und Breite haben.
    /// </summary>
    public bool Equals(Position? other) => Lat.Equals(other?.Lat) && Lng.Equals(other
?.Lng);
    public override bool Equals(object? obj) => Equals(obj as Position);

    public double GetDistance(Position p)
    {
        return Math.Acos(
            Math.Sin(LatRad) * Math.Sin(p.LatRad) +
            Math.Cos(LatRad) * Math.Cos(p.LatRad) * Math.Cos(p.LngRad - LngRad)) *
6370;
    }
    /// <summary>
    /// Überschreibt GetHashCode() von object, sodass ihr Verhalten mit Equals
übereinstimmt . Kein Interface nötig.
    /// </summary>
    public override int GetHashCode() => HashCode.Combine(Lat, Lng);
```

```
    /// <summary>
    /// Überschreibt ToString von object. Kein Interface nötig.
    /// </summary>
    public override string ToString() => $"({Lat} {Lng} (Hash: {GetHashCode()})";

    // Überschreiben der Operatoren für den Vergleich, damit sie das selbe Verhalten
wie
    // CompareTo() zeigen.
    public static bool operator <(Position p1, Position p2) => p1.CompareTo(p2) < 0;
    public static bool operator <=(Position p1, Position p2) => p1.CompareTo(p2) <= 0;
    public static bool operator >(Position p1, Position p2) => p1.CompareTo(p2) > 0;
    public static bool operator >=(Position p1, Position p2) => p1.CompareTo(p2) >= 0;
}

class Program
{
    static void Main(string[] args)
    {
        // Wien
        Position p1 = new Position(48.2083, 16.3725);
        // St. Pölten
        Position p2 = new Position(48.2043, 15.6229);
        // Eisenstadt
        Position p3 = new Position(47.8387, 16.5362);
        // Wien nocheinmal
        Position p4 = new Position(48.2083, 16.3725);

        Console.WriteLine($"Abstand von Wien nach Eisenstadt: {p3.GetDistance(p1)}
km.");
        Console.WriteLine($"Wien = St. Pölten?:              {p1 == p2}");
        Console.WriteLine($"Eisenstadt südlicher als Wien?:  {p3 < p1}");

        Console.WriteLine($"Instanzen von Wien ident?:               {p1.Equals(p4)}
");
        Console.WriteLine($"Instanzen von Wien auch mit == ident?:   {p1 == p4}");
        Console.WriteLine($"Instanzen von Wien auch als object ident?: {p1.Equals
((object)p4)}");
        Console.WriteLine($"Gleiche Referenz der Instanzen von Wien?:  {object
.ReferenceEquals(p1, p4)}");

        List<Position> positions = new List<Position>() { p1, p2, p3, p4 };
        Console.WriteLine("Punkte sortiert von Süd nach Nord.");
        positions.Sort();
        foreach (Position p in positions)
        {
            Console.WriteLine(p);
        }
        Console.WriteLine("Hashset:");
        HashSet<Position> positionsHash = new HashSet<Position>() { p1, p2, p3, p4 };
        foreach (Position p in positionsHash)
        {
            Console.WriteLine(p);
        }
    }
}
```

```
Abstand von Wien nach Eisenstadt: 42.8561852967433 km.
Wien = St. Pölten?:               False
Eisenstadt südlicher als Wien?:   True
Instanzen von Wien ident?:                 True
Instanzen von Wien auch mit == ident?:     False
Instanzen von Wien auch als object ident?: True
Gleiche Referenz der Instanzen von Wien?:  False
Punkte sortiert von Süd nach Nord.
(47.8387 16.5362 (Hash: -490148674)
(48.2043 15.6229 (Hash: 1061893839)
(48.2083 16.3725 (Hash: -1148819547)
(48.2083 16.3725 (Hash: -1148819547)
Hashset:
(48.2083 16.3725 (Hash: -1148819547)
(48.2043 15.6229 (Hash: 1061893839)
(47.8387 16.5362 (Hash: -490148674)
```

Punkt ist ein *Immutable Type*

Es scheint vielleicht seltsam, dass wir Länge und Breite im Konstruktor setzen und diese Werte im Nachhinein nicht mehr geändert werden können. Dies wird durch ein read-only Property erzielt. Diese Properties können jedoch im Konstruktor initialisiert werden. Nach dem Durcharbeiten dieses Kapitels ist der Sinn klar.

IEquatable<T> und *object.Equals()*

Bei Referenztypen wird zwischen Referenzgleichheit (Referenzvariable zeigt auf die gleiche Instanz) und Inhaltsgleichheit (der Inhalt der Instanz ist ident) unterschieden. Aus Java bekannt ist das Überschreiben der *Equals()* Methode, welche in *object* definiert ist. Da wir auch beim Cast auf *object* diese Methode aufrufen wollen, verwenden wir das Schlüsselwort *override*. Damit nicht bei jedem Vergleich zwischen 2 identen Typen ein Cast auf *object* vorgenommen werden muss, implementiert der gute .NET Programmierer zusätzlich das Interface *IEquatable<T>*. Diese Implementierungen müssen natürlich das selbe Verhalten zeigen. Dies lässt sich am Leichtesten so erreichen, dass *object.Equals()* die spezifischere *Equals()* Methode aufruft.

Es ist darauf zu achten, dass die Methode keine Exception verursacht. Wird null übergeben, so soll diese Methode einfach *false* liefern. Der *?.* Operator hilft uns hierbei.

```
public bool Equals(Position? other) => Lat.Equals(other?.Lat) && Lng.Equals(other?.
Lng);
public override bool Equals(object? obj) => Equals(obj as Position);
```

Was ist der Hashcode?

Equals wird in den darunterliegenden Frameworkmethoden häufig aufgerufen. Daher ist die Performance bei Vergleichen wesentlich. Damit nicht ganze Felder verglichen werden müssen, ist in .NET die Methode *object.GetHashCode()* implementiert, die wir entsprechend überschrei-

ben sollten. Sie muss folgendes Verhalten zeigen:

- Instanzen, die in *Equals()* als ident bestimmt werden, müssen den selben Hashcode liefern.
- Instanzen mit gleichem Hashcode müssen nicht immer ident sein. Da die Instanz auf einen einzigen *int* Wert abgebildet wird, ist eine Kollision nicht immer vermeidbar. Diese Kollisionen sollen aber minimiert werden.

Um das zu erreichen, verwenden wir die Methode *HashCode.Combine()* die uns im Namespace System zur Verfügung steht:

```
public override int GetHashCode() => HashCode.Combine(Lat, Lng);
```

Wer verwendet *Equals()*?

In diesem Beispiel wurden Instanzen von *Position* in einem *HashSet<T>* gespeichert. Beim Hinzufügen wird über *GetHashCode()* und in weiterer Folge mit *Equals()* geprüft, ob es sich um idente Instanzen handelt. Deswegen ist es auch wichtig, dass *GetHashCode()* korrekt implementiert ist.

Equals() und der == Operator

Wird *Equals()* überschrieben, so liefert die Methode ein anderes Vergleichsergebnis als der == und der != Operator. Die Guidelines zum Überschreiben (vgl. docs.microsoft.com) besagen, dass dies optional, aber empfohlen ist.

IComparable sowie *IComparable<T>*

Sollen Instanzen verglichen werden, so können diese beiden Interfaces implementiert werden. Aus Performancegründen gibt es hier ein generisches Interface, es soll jedoch auch das nicht generische Interface entsprechend implementiert werden. Dies wird durch folgende Implementierung erreicht:

```
public int CompareTo(Position? other) => Lat.CompareTo(other?.Lat);
public int CompareTo(object? obj) => CompareTo(obj as Position);
```

Auch hier ist es am Einfachsten, die *CompareTo()* Methode des zu vergleichenden Feldes einfach aufzurufen. Damit ist die korrekte Implementierung sichergestellt:

- Es wird ein Wert kleiner als 0 geliefert, wenn die aktuelle Instanz vor *other* eingeordnet werden soll.
- Es wird 0 geliefert, wenn die aktuelle Instanz gleich mit *other* ist.
- Es wird ein Wert größer als 0 geliefert, wenn die aktuelle Instanz nach *other* eingeordnet werden soll.

Guter Stil: Operatoren ebenfalls überschreiben

> If you implement IComparable<T>, you should overload the op_GreaterThan, op_GreaterThanOrEqual, op_LessThan, and op_LessThanOrEqual operators to return values that are consistent with CompareTo(T). In addition, you should also implement IEquatable<T>. See the IEquatable<T> article for complete information. (Vgl. MSDN: IComparable<T> Interface)

In C# besteht die Möglichkeit, Operatoren zu überladen. Wenn dies nicht gemacht wird, liefern *CompareTo()* und die Vergleichsoperatoren unterschiedliche Ergebnisse. Auf die Technik des Überladens wird hier nicht eingegangen, die nachfolgenden Methoden implementieren bereits die Lambda Schreibweise. Hier wird vom Compiler automatisch ein *return* vor dem Ausdruck rechts vom => ergänzt.

```
public static bool operator < (Position p1, Position p2) => p1.CompareTo(p2) < 0;
public static bool operator <=(Position p1, Position p2) => p1.CompareTo(p2) <= 0;
public static bool operator > (Position p1, Position p2) => p1.CompareTo(p2) > 0;
public static bool operator >=(Position p1, Position p2) => p1.CompareTo(p2) > 0;
```

CompareTo() und *Equals()* sollten das selbe Verhalten zeigen

> Typically, types that provide an IComparable<T> implementation also implement the IEquatable<T> interface. The IEquatable<T> interface defines the Equals method, which determines the equality of instances of the implementing type. (Vgl. MSDN: IComparable<T> Interface)

Wann soll *Equals()* überschrieben werden?

> Guidelines for Reference Types
>
> The following guidelines apply to overriding Equals(Object) for a reference type: Consider overriding Equals if the semantics of the type are based on the fact that the type represents some value(s).
>
> Most reference types must not overload the equality operator, even if they override Equals. However, if you are implementing a reference type that is intended to have value semantics, such as a complex number type, you must override the equality operator.
>
> You should not override Equals on a mutable reference type. This is because overriding Equals requires that you also override the GetHashCode method, as discussed in the previous section. This means that the hash code of an instance of a mutable reference type can change during its lifetime, which can cause the object to be lost in a hash table. (Vgl. MSDN: Object.Equals Method)

Diese Erklärung gibt uns schon eindeutig zu verstehen, dass das Überschreiben von *Equals()* und *CompareTo()* zwar beherrscht werden sollten, ihre Anwendung jedoch weit seltener als

vielleicht gedacht ist.

Warum diese Strenge?

Wir betrachten diese technisch vollkommen korrekte Implementierung von *Equals()*:

```
class Person : IEquatable<Person>
{
    public int Nr { get; set; }
    public string Zunamne { get; set; } = string.Empty;
    public string Vorname { get; set; } = string.Empty;

    public bool Equals(Person? other) => Nr.Equals(other?.Nr);
    public override bool Equals(object? obj) => Equals(obj);
    public override int GetHashCode() => Nr.GetHashCode();
}
```

Speichern wir nun Instanzen in einem *HashSet<T>* und ändern nachträglich die Nummer einer Person, hat unser Hashset auf einmal 2 idente Instanzen:

```
Person pe1 = new Person { Nr = 1, Vorname = "VN1", Zunamne = "ZN1" };
Person pe2 = new Person { Nr = 2, Vorname = "VN2", Zunamne = "ZN2" };
HashSet<Person> personHash = new HashSet<Person>() { pe1, pe2 };
pe2.Nr = 1;                          // Nachträgliche Änderung von pe2.
// personHash hat nun 2 Einträge, die jedoch lt. Equals() ident sind!
```

Records in C# 9

Die Klasse *Position* hat 2 Eigenschaften, die oft benötigt werden:

- Alle Properties sind read-only.
- Sie besitzt einen Konstruktor mit allen Properties zur Zuweisung der Werte.
- Sie überschreibt Equals() so, dass 2 Instanzen gleich sind, wenn alle Properties gleich sind.

Seit C# 9 können wir die Klasse wesentlich kürzer definieren:

```
using System;

record Position(double Lat, double Lng);   // "Positional record" in C# 9

public class Program
{
    public static void Main()
    {
        Position position1 = new Position(Lat: 48, Lng: 16);
        Position position2 = new Position(Lat: 48, Lng: 16);
        Console.WriteLine(position1 == position2);              // True
        Console.WriteLine(position1.Equals(position2));         // True
```

```
        // Erzeugt eine neue Position, indem die Werte von position1 geändert in ein
neues
        // Objekt geschrieben werden (Position selbst ist natürlich immutable).
        Position position3 = position1 with {Lng = 17};
        Console.WriteLine(position3);           // ToString liefert Position { Lat =
48, Lng = 17 }
        // Deconstruct
        var (lat, lng) = position3;
        Console.WriteLine($"{lat}° NB, {lng}° ÖL");
    }
}
```

Es ist natürlich auch möglich, zusätzliche Properties (sowohl get und set) oder Methoden in einem Record zu definieren:

```
record Position(double Lat, double Lng)
{
    public double LatRad => Lat * Math.PI / 180;
    public double LngRad => Lng * Math.PI / 180;
    public double GetDistance(Position p)
    {
        return Math.Acos(
            Math.Sin(LatRad) * Math.Sin(p.LatRad) +
            Math.Cos(LatRad) * Math.Cos(p.LatRad) * Math.Cos(p.LngRad - LngRad)) *
6370;
    }
}
```

Unittests mit xUnit

Warum testen?

Du programmierst einen Notenrechner: Er bekommt die erreichten Prozent und liefert die Note. Du probierst 90 % aus, die Konsole zeigt 1. Passt. Eine Woche später änderst du die Grenze für das „Gut“. Funktioniert 90 % noch? Und 87,5 %? Und was passiert bei -5 %? Um das zu wissen, müsstest du alles wieder von Hand ausprobieren. In diesem Kapitel siehst du, warum man das **automatisch** macht und welche Arten von Tests es gibt. Im nächsten Kapitel schreibst du die Tests dann selbst mit xUnit.

Das Problem: Testen von Hand

Im Kapitel *Types in C#* hast du die Konsolenausgabe angesehen und mit dem erwarteten Ergebnis verglichen:

```
Console.WriteLine(GradeCalculator.CalculateGrade(90m));     // Expected: 1
Console.WriteLine(GradeCalculator.CalculateGrade(87.5m));   // Expected: 1
Console.WriteLine(GradeCalculator.CalculateGrade(49.99m));  // Expected: 5
```

Das funktioniert bei drei Zeilen. Bei einem echten Projekt mit hunderten Methoden nicht mehr:

Problem	Folge
Ein Mensch muss vergleichen.	Bei 200 Zeilen Ausgabe übersieht man einen Fehler.
Man testet nur, was man gerade ändert.	Ein Fehler in einem anderen Teil fällt nicht auf.
Das Wissen steht nur im Kopf.	Ein Kollege weiß nicht, welche Fälle du geprüft hast.
Es kostet bei jeder Änderung Zeit.	Irgendwann testet niemand mehr.

Ein Fehler, der durch eine Änderung in Code entsteht, der vorher schon funktioniert hat, heißt **Regression**. Gegen Regressionen hilft nur, nach *jeder* Änderung *alle* Fälle zu prüfen. Das schafft nur ein Programm.

Was ist ein Unit Test?

Ein **Unit Test** ist eine Methode, die eine kleine Einheit (engl. *unit*) deines Programms aufruft und das Ergebnis automatisch prüft. Die Einheit ist meist eine Methode oder eine Klasse.

```
[Fact]
public void CalculateGrade_With90Percent_Returns1()
{
    int grade = GradeCalculator.CalculateGrade(90m);
    Assert.Equal(1, grade);
}
```

- `[Fact]` markiert die Methode als Test. Ein Testprogramm (der *Test Runner*) findet alle Methoden mit diesem Attribut und führt sie aus.
- `Assert.Equal(expected, actual)` vergleicht. Stimmen die Werte nicht überein, schlägt der Test fehl („rot“). Sonst ist er bestanden („grün“).

Jetzt prüft ein Programm in Millisekunden, was du vorher von Hand gemacht hast. Und es macht das nach jeder Änderung wieder, ohne etwas zu vergessen.

Tests sind eine Spezifikation. Ein Test beschreibt genau, was der Code tun soll: „Bei 90 % kommt 1 heraus.“ Wenn du später nicht mehr weißt, was bei 87,5 % herauskommen soll, liest du den Test. Deshalb bekommst du in den Übungen dieses Kurses oft fertige Tests: Sie sind die Angabe.

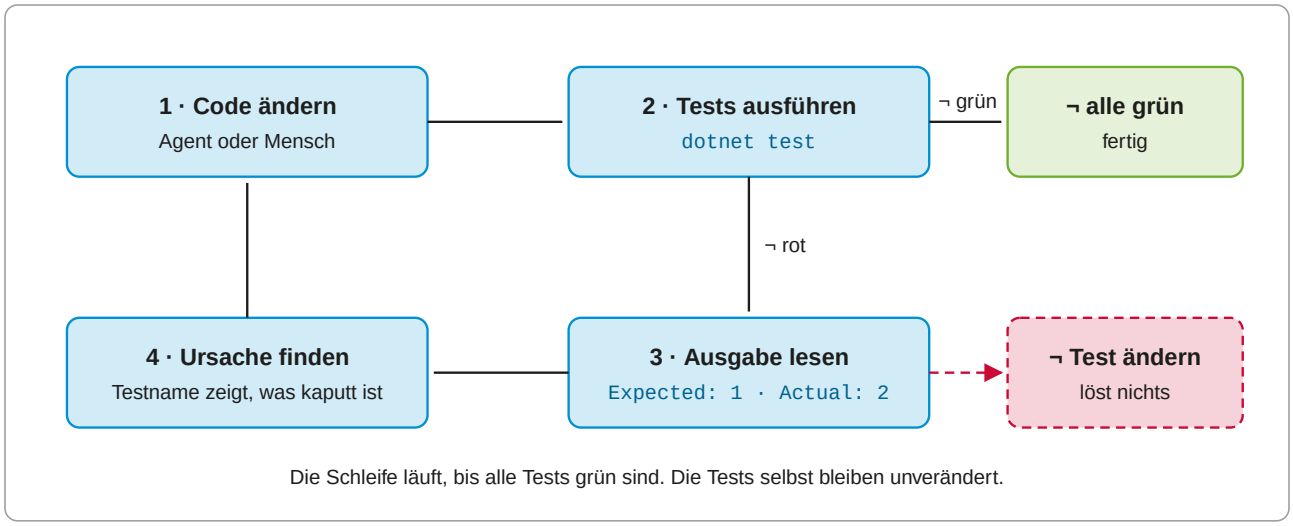
Was macht einen guten Unit Test aus?

Die Abkürzung **FIRST** fasst die Eigenschaften zusammen:

Buchstabe	Eigenschaft	Bedeutung
F	*Fast*	Ein Test läuft in Millisekunden. Sonst führt sie niemand oft aus.
I	*Independent*	Kein Test hängt von einem anderen ab. Die Reihenfolge ist egal.
R	*Repeatable*	Gleiches Ergebnis bei jedem Lauf, auf jedem Rechner, zu jeder Uhrzeit.
S	*Self-validating*	Der Test sagt selbst „grün“ oder „rot“. Niemand muss Ausgaben lesen.
T	*Timely*	Der Test entsteht zusammen mit dem Code, nicht Wochen später.

Tests als Feedback für Agents

Immer mehr Code schreiben *Coding Agents*. Das sind KI-Programme wie GitHub Copilot, Claude Code oder Codex, die selbstständig Dateien ändern und Befehle ausführen. Ein Agent kann aber nicht „sehen“, ob sein Code das Richtige tut. Er braucht ein Signal von außen. Tests liefern dieses Signal:



Die Schleife läuft, bis alle Tests grün sind. Die Tests selbst bleiben unverändert.

1. Der Agent ändert den Code.
2. Er führt `dotnet test` aus.
3. Ist ein Test rot, liest er die Ausgabe, zum Beispiel `Expected: 1, Actual: 2`.
4. Aus Testname und Ausgabe schließt er, wo der Fehler liegt, und beginnt wieder bei 1.

Ohne Tests weiß der Agent nur, dass der Code **kompiliert**. Das ist zu wenig. Mit Tests weiß er, ob der Code **das Richtige** tut. Für dich als Mensch gilt übrigens dieselbe Schleife.

Daraus folgen zwei Regeln:

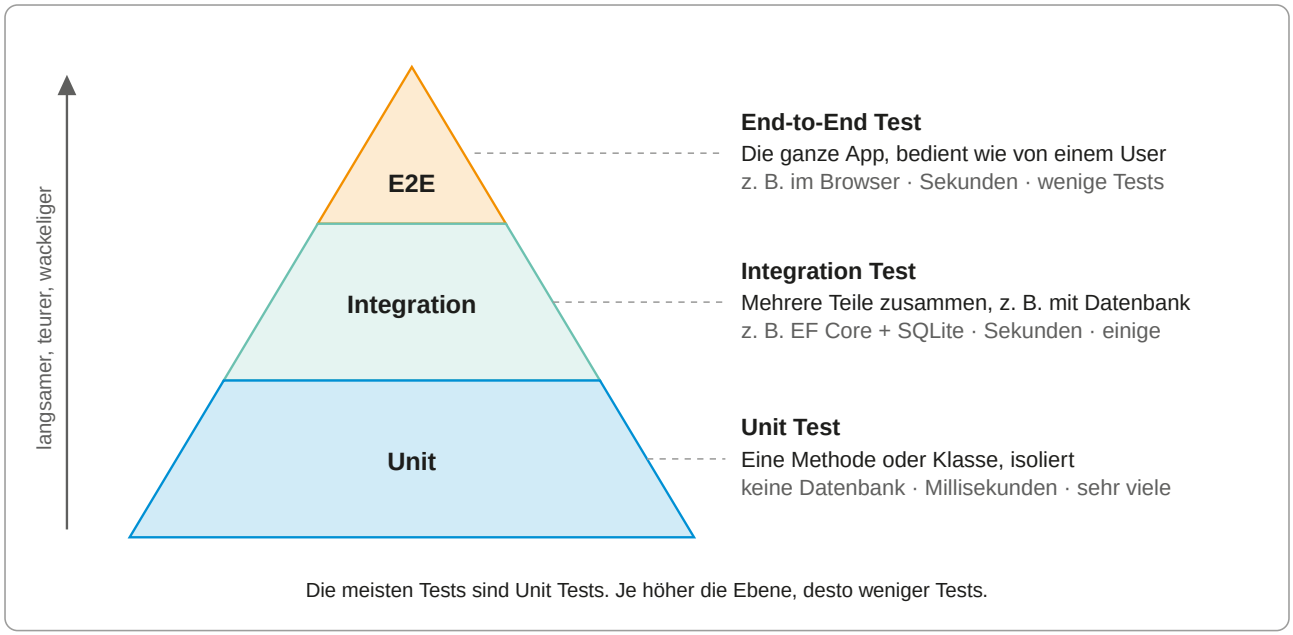
- **Tests sind die Spezifikation, nicht das Hindernis.** Ein Agent (oder ein Schüler), der einen Test so lange ändert, bis er grün ist, hat nichts gelöst. Er hat nur die Angabe umgeschrieben. Deshalb steht in jeder Übung: *Die Tests darfst du nicht ändern.*
- **Gute Tests erklären den Fehler.** Ein Testname wie `CalculateGrade_At87Point5Percent_Returns1` und eine klare Ausgabe zeigen sofort, was kaputt ist. Ein Test namens `Test1` zeigt nur, *dass* etwas kaputt ist. Das hilft Menschen und Agents gleichermaßen.



Lässt du einen Agent Code schreiben, sieh dir im Diff (in der Liste der Änderungen) immer auch die Testdateien an. Ein geändertes `Assert` oder ein gelöschter Test ist ein Warnsignal.

Die Testpyramide

Unit Tests prüfen eine Methode isoliert. Das reicht nicht für alles: Speichert das Programm die Daten wirklich in der Datenbank? Kann ein User im Browser bestellen? Dafür gibt es weitere Testebenen. Die **Testpyramide** zeigt, wie viele Tests man von jeder Ebene schreibt:



Die meisten Tests sind Unit Tests. Je höher die Ebene, desto weniger Tests.

Ebene	Was wird geprüft?	Beispiel in diesem Kurs
Unit Test	Eine Methode oder Klasse ohne Datenbank, Netzwerk oder Dateien	`GradeCalculator.CalculateGrade(90m)` liefert `1`.
Integration Test	Mehrere Teile zusammen	Im Kapitel EF Core: Ein Objekt wird in eine SQLite-Datenbank geschrieben und wieder gelesen.
End-to-End Test (E2E)	Die ganze Anwendung, so wie sie ein User bedient	Ein Programm öffnet den Browser, klickt auf „Bestellen“ und prüft die Bestätigung.

Warum eine Pyramide und nicht ein Würfel mit gleich vielen Tests pro Ebene?

- **Geschwindigkeit:** 1 000 Unit Tests laufen in einer Sekunde. Ein einziger E2E-Test braucht oft mehrere Sekunden, weil ein Browser startet.
- **Fehlersuche:** Schlägt ein Unit Test fehl, weißt du genau, welche Methode schuld ist. Schlägt ein E2E-Test fehl, kann der Fehler überall liegen: im Code, in der Datenbank, im Netzwerk.
- **Stabilität:** Je mehr Teile beteiligt sind, desto öfter schlägt ein Test ohne echten Fehler fehl, zum Beispiel weil das Netzwerk langsam war. Solche Tests heißen *flaky* (wackelig).



Auch Integration Tests schreibt man mit xUnit. Die Tests im Kapitel EF Core verwenden eine echte SQLite-Datenbank und sind daher streng genommen Integration Tests. Umgangssprachlich sagt man trotzdem oft „Unittest“ zu allem, was mit xUnit läuft.

Testarten nach Zweck

Unabhängig von der Ebene unterscheidet man Tests danach, *welchen Fall* sie prüfen:

Testart	Prüft …	Beispiel Notenrechner
Happy Path	den normalen Fall mit gültigen Daten. Alles geht gut.	90 % → Note 1
Edge Case (Grenzfall)	Werte am Rand des Erlaubten: Grenzen, 0, Maximum, leere Liste	87,5 % → Note 1, 0 % → Note 5, 100 % → Note 1
Fehlerfall (Negativtest)	ungültige Eingaben. Der Code muss sie ablehnen.	-5 % → `ArgumentOutOfRangeException`
Smoke Test	ganz grob, ob das Programm überhaupt läuft	Die App startet, eine Note wird berechnet.

Happy Path ist nicht genug

Anfänger testen oft nur den Happy Path. Der funktioniert aber meistens, weil du beim Programmieren genau an diesen Fall gedacht hast. Fehler stecken an den Rändern: Hast du `>` oder `>=` geschrieben? Was passiert bei einer leeren Liste? Bei `null`? Ein typischer Fehler ist der *Off-by-one-Fehler*: Eine Grenze ist um eins (oder um eine Kommastelle) verschoben.

```
if (percent > 87.5m) { return 1; }     // Bug: 87.5 % should give grade 1
```

Ein Test mit 90 % findet diesen Fehler nie. Ein Test mit genau 87,5 % findet ihn sofort.

Smoke Test

Der Name kommt aus der Elektronik: Man schaltet ein neues Gerät ein und schaut, ob Rauch aufsteigt. Ein Smoke Test prüft also nicht die Details, sondern nur: Startet die Anwendung? Erreicht sie die Datenbank? Liefert die Startseite eine Antwort? Smoke Tests laufen zuerst, zum Beispiel direkt nach dem Installieren einer neuen Version auf dem Server. Schlägt schon der Smoke Test fehl, braucht man die restlichen Tests gar nicht mehr auszuführen.

Im Kapitel Testfälle finden lernst du, wie du Edge Cases nicht nur errätst, sondern **systematisch** findest.

Unit Tests mit xUnit

Im Kapitel Warum testen? hast du gesehen, was Unit Tests leisten. Jetzt schreibst du sie selbst, und zwar für den Notenrechner. Du lernst das Paket xUnit kennen, prüfst Ergebnisse und Exceptions und erfährst, warum ein Test nur ein Verhalten prüfen soll. Welche Testfälle du brauchst, leitest du im nächsten Kapitel systematisch ab.

Projekt anlegen

Eine Solution mit Tests besteht aus zwei Projekten: Die *Application* enthält den Code, das Projekt *Tests* enthält die Tests und verweist auf die Application. Führe diese Befehle in der Konsole aus. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q UnittestDemo
md UnittestDemo
cd UnittestDemo
md UnittestDemo.Application
cd UnittestDemo.Application
dotnet new classlib
cd ..
md UnittestDemo.Tests
cd UnittestDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\UnittestDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add UnittestDemo.Application
dotnet sln add UnittestDemo.Tests
start UnittestDemo.sln
```

- `dotnet new install xunit.v3.templates` installiert die Vorlage für xUnit in Version 3. Das musst du nur einmal pro Rechner machen.
- `-f net10.0` ist nötig, sonst erzeugt die Vorlage ein Projekt für .NET 8.
- Die Datei *global.json* gehört in den Ordner der Solution. Sie legt fest, mit welcher Testplattform `dotnet test` arbeitet. Fehlt sie, bricht `dotnet test` mit einer Fehlermeldung ab. Liegt dein Ordner in einem git-Repository, legt die Vorlage *global.json* gleich im obersten Ordner des Repositorys an. Dann meldet *move* „Datei nicht gefunden“. Das kannst du ignorieren, denn `dotnet test` sucht die Datei auch in den übergeordneten Ordnern.

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge *TreatWarningsAsErrors* hinzu. Im Testprojekt entfernst du zusätzlich die Zeile `<Using Include="Xunit" />`. Dann schreibst du `using Xunit;` selbst in jede Testdatei und siehst, woher `Assert` kommt. *UnittestDemo.Tests.csproj* sieht dann so aus:

```
<Project Sdk="Microsoft.NET.Sdk">
```

```
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>

  <ItemGroup>
    <Content Include="xunit.runner.json" CopyToOutputDirectory="PreserveNewest" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="xunit.v3.mtp-v2" Version="4.0.2" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference
Include="..\UnittestDemo.Application\UnittestDemo.Application.csproj" />
  </ItemGroup>

</Project>
```

Das Testprojekt ist ein eigenes, ausführbares Programm (`OutputType Exe`). Du startest die Tests im Test Explorer von Visual Studio oder Rider, in der Konsole mit `dotnet test` im Ordner der Solution.

Die Klasse, die wir testen

Der Notenrechner bekommt die erreichten Prozent als `decimal`. Mit `decimal` sind Werte wie 87,5 oder 49,99 exakt darstellbar. Bei `double` können an den Grenzen Rundungsfehler auftreten.

GradeCalculator.cs (im Projekt *UnittestDemo.Application*)

```
using System;

namespace UnittestDemo.Application;

public static class GradeCalculator
{
    public static int CalculateGrade(decimal percent)
    {
        if (percent < 0 || percent > 100)
        {
            throw new ArgumentOutOfRangeException(nameof(percent), "Percent must be
between 0 and 100.");
        }
        if (percent >= 87.5m) { return 1; }
        if (percent >= 75m) { return 2; }
        if (percent >= 62.5m) { return 3; }
        if (percent >= 50m) { return 4; }
        return 5;
```

```
    }
}
```

Die Klasse muss `public` sein, sonst sieht das Testprojekt sie nicht.

Der erste Test: [Fact]

GradeCalculatorTests.cs (im Projekt *UnittestDemo.Tests*)

```
using System;
using UnittestDemo.Application;
using Xunit;

namespace UnittestDemo.Tests;

public class GradeCalculatorTests
{
    [Fact]
    public void CalculateGrade_With90Percent_Returns1()
    {
        // Arrange
        decimal percent = 90m;

        // Act
        int grade = GradeCalculator.CalculateGrade(percent);

        // Assert
        Assert.Equal(1, grade);
    }
}
```

- Eine **Testklasse** ist `public` und heißt wie die getestete Klasse plus *Tests*.
- `[Fact]` markiert eine Testmethode ohne Parameter. Sie ist `public void`.
- Der **Name** folgt dem Muster `Methode_Situation_ErwartetesErgebnis`. Schlägt der Test fehl, sagt schon der Name, was nicht stimmt.

Jeder Test hat drei Teile. Man nennt das **Arrange-Act-Assert** (AAA):

Teil	Was passiert?	Im Beispiel
Arrange	Die Ausgangslage herstellen: Objekte, Eingaben	`percent = 90m`
Act	Genau **eine** Aktion ausführen	`CalculateGrade(percent)` aufrufen
Assert	Das Ergebnis prüfen	Ist die Note `1`?

Bei kurzen Tests lässt man die Kommentare weg. Die Reihenfolge bleibt trotzdem gleich.

Ist ein Test rot, zeigt xUnit, was erwartet war und was herauskam. Stünde im Notenrechner `percent > 87.5m` statt `percent >= 87.5m`, meldet ein Test für 87,5 %:

```
failed
UnittestDemo.Tests.GradeCalculatorTests.CalculateGrade_With87Point5Percent_Returns1
  Assert.Equal() Failure: Values differ
  Expected: 1
  Actual:   2
```

Mehrere Werte: [Theory] und [InlineData]

Für die Grenze bei 87,5 % willst du zwei Werte prüfen: 87,5 (Note 1) und 87,49 (Note 2). Zwei fast gleiche `[Fact]`-Methoden wären doppelter Code. Eine **Theory** ist ein Test mit Parametern. Jedes `[InlineData]` liefert einen Satz Argumente:

```
[Theory]
[InlineData(87.5, 1)]
[InlineData(87.49, 2)]
public void CalculateGrade_AtBoundary_ReturnsExpectedGrade(decimal percent, int
expected)
{
    int grade = GradeCalculator.CalculateGrade(percent);
    Assert.Equal(expected, grade);
}
```

- Die Werte in `[InlineData]` stehen in der Reihenfolge der Parameter.
- Der Test Explorer zeigt jede Zeile als eigenen Testfall. Du siehst also, *welcher* Wert fehlschlägt.
- In einem Attribut darf kein `decimal` stehen (`87.5m` ist dort nicht erlaubt). Du schreibst `87.5`, xUnit wandelt den Wert in den Parametertyp `decimal` um.

Die wichtigsten Asserts

Die Klasse `Assert` hat viele Methoden. Diese brauchst du in den nächsten Kapiteln am häufigsten:

Assert	Prüft, ob …
`Assert.Equal(expected, actual)`	zwei Werte gleich sind. Zuerst der erwartete Wert!
`Assert.NotEqual(expected, actual)`	zwei Werte verschieden sind
`Assert.True(condition)`, `Assert.False(condition)`	eine Bedingung erfüllt (nicht erfüllt) ist
`Assert.Null(obj)`, `Assert.NotNull(obj)`	ein Wert `null` ist (nicht ist)
`Assert.Empty(list)`, `Assert.Single(list)`	eine Collection leer ist bzw. genau ein Element hat
`Assert.Contains(item, list)`	eine Collection (oder ein String) das Element enthält

Assert	Prüft, ob …
`Assert.Equivalent(expected, actual)`	zwei Objekte in allen Properties übereinstimmen
`Assert.Throws<T>(...)`	eine Exception vom Typ `T` geworfen wird (nächster Abschnitt)

Nimm das genaueste Assert. `Assert.True(grade == 1)` funktioniert auch. Schlägt der Test fehl, meldet xUnit aber nur `Expected: True, Actual: False`. `Assert.Equal(1, grade)` meldet dagegen `Expected: 1, Actual: 2`. Damit weißt du (oder ein Agent) sofort mehr.

`Assert.True` und `Assert.False` können eine eigene Meldung bekommen. Sie erscheint statt `Expected: True` in der Ausgabe:

```
Assert.True(success, "Payment within the limit must succeed.");
```

Exceptions testen

Bei -5 % soll `CalculateGrade` eine `ArgumentOutOfRangeException` werfen. Würdest du die Methode direkt im Test aufrufen, bricht die Exception den Test ab. Daher übergibst du den Aufruf an `Assert.Throws`. Diese Methode führt ihn selbst aus und fängt die Exception:

```
[Fact]
public void CalculateGrade_WithNegativePercent_ThrowsArgumentOutOfRangeException()
{
    ArgumentOutOfRangeException ex = Assert.Throws<ArgumentOutOfRangeException>(
        () => GradeCalculator.CalculateGrade(-0.01m));

    Assert.Equal("percent", ex.ParamName);
    Assert.StartsWith("Percent must be between 0 and 100.", ex.Message);
}
```

- `() => GradeCalculator.CalculateGrade(-0.01m)` ist eine **Lambda Expression**: eine kurze Methode ohne Namen. `Assert.Throws` ruft sie auf. Lambdas lernst du im Kapitel Lambda Expressions genauer kennen.
- Wird keine Exception geworfen, ist der Test rot.
- `Assert.Throws` liefert die gefangene Exception zurück. So prüfst du auch die **Message**. Damit erkennst du, ob die *richtige* Prüfung angeschlagen hat. `ArgumentException` und ihre Unterklassen hängen an die Message noch `(Parameter 'percent')` an. Daher steht hier `Assert.StartsWith` statt `Assert.Equal`.



Schreibe in Tests kein `try`/`catch`, um Exceptions zu prüfen. Vergisst du im `try` ein `Assert.Fail()`, ist der Test auch grün, wenn gar keine Exception kommt.

Throws oder ThrowsAny?

`Assert.Throws<T>` verlangt **genau** den Typ `T`. `ArgumentOutOfRangeException` ist eine Unterklasse von `ArgumentException`. Trotzdem ist dieser Test rot:

```
Assert.Throws<ArgumentException>(() => GradeCalculator.CalculateGrade(101m));
```

```
Assert.Throws() Failure: Exception type was not an exact match
Expected: typeof(System.ArgumentException)
Actual:   typeof(System.ArgumentOutOfRangeException)
```

`Assert.ThrowsAny<T>` akzeptiert auch alle Unterklassen von `T`:

```
Assert.ThrowsAny<ArgumentException>(() => GradeCalculator.CalculateGrade(101m));   //
green
```

Methode	Akzeptiert	Verwende es, wenn …
`Assert.Throws<T>`	genau `T`	der genaue Typ zur Spezifikation gehört (der Normalfall).
`Assert.ThrowsAny<T>`	`T` und alle Unterklassen	nur die Basisklasse zählt.

`ThrowsAny` wird wichtig, sobald du eigene Exceptions schreibst. Eine Serviceklasse wirft dann zum Beispiel eine `ServiceException` oder eine davon abgeleitete `NotFoundException`. Ein Test, der nur prüft „der Service lehnt ab“, verwendet `Assert.ThrowsAny<ServiceException>`.

Ein Verhalten pro Test

Dieser Test prüft alles auf einmal:

```
[Fact]
public void CalculateGrade_Works()
{
    Assert.Equal(1, GradeCalculator.CalculateGrade(90m));
    Assert.Equal(1, GradeCalculator.CalculateGrade(87.5m));
    Assert.Equal(5, GradeCalculator.CalculateGrade(0m));
    Assert.Throws<ArgumentOutOfRangeException>(() => GradeCalculator.CalculateGrade(-
1m));
}
```

Das hat zwei Nachteile:

- **Das erste rote Assert beendet den Test.** Schlägt die Zeile mit 87,5 fehl, laufen die anderen Prüfungen gar nicht. Du weißt nicht, ob 0 % und -1 % funktionieren.
- **Der Name sagt nichts.** `CalculateGrade_Works` ist rot. Aber was genau ist kaputt?

Daher gilt die Regel: **Ein Test prüft ein Verhalten.** Oft wird sie als „ein Assert pro Test“ zitiert. Mehrere Asserts sind aber in Ordnung, wenn sie *dasselbe* Verhalten prüfen. Im Abschnitt über Exceptions prüfen Typ, `ParamName` und Message gemeinsam: „Die richtige Exception kommt.“

Prüfst du mehrere Properties eines Objekts, gibt es zwei Hilfen. Beide melden *alle* Abweichungen und nicht nur die erste. Im Beispiel ist `card` eine *CreditCard* aus dem Kapitel Interfaces, erzeugt mit `new CreditCard(limit: 1000, expiration: new DateTime(2030, 12, 31))`:

```
// Assert.Multiple runs every check and reports all failures together.
Assert.Multiple(
    () => Assert.Equal(1000m, card.Limit),
    () => Assert.Equal(new DateTime(2030, 12, 31), card.Expiration));

// Assert.Equivalent compares the properties of an anonymous object with the real
object.
Assert.Equivalent(new { Limit = 1000m, Expiration = new DateTime(2030, 12, 31) },
card);
```

Testfälle finden

Der Notenrechner aus dem Kapitel Unit Tests mit xUnit hat Tests für 90 %, 87,5 % und 87,49 %. Reicht das? Zehn Tests? Hundert? In diesem Kapitel leitest du die nötigen Testfälle systematisch ab, statt sie zu raten. Du erfährst, was Code Coverage aussagt und was nicht. Außerdem machst du Code testbar, der von der Uhrzeit abhängt.

Projekt anlegen

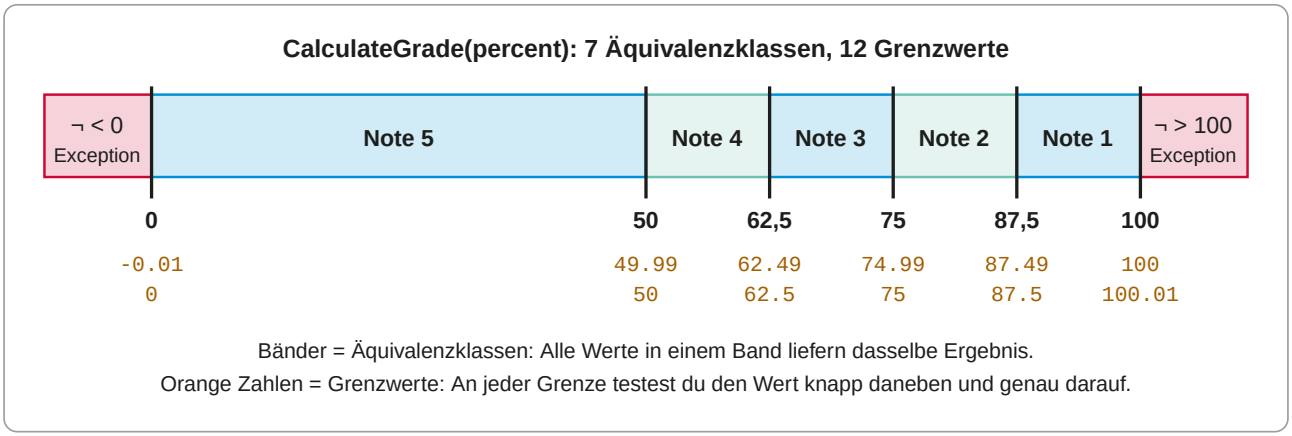
Du arbeitest in der Solution *UnittestDemo* aus dem Kapitel Unit Tests mit xUnit weiter. Die Klasse *GradeCalculator* bleibt unverändert. Im Abschnitt *Deterministische Tests* kommen die Dateien *CreditCard.cs* (Application) und *FixedTimeProvider.cs* (Tests) dazu.

Wie viele Tests brauche ich?

Alle möglichen Prozentwerte zu testen, geht nicht. Allein zwischen 0 und 100 gibt es mit zwei Kommastellen 10 001 Werte. Zwei Techniken liefern eine kleine, aber vollständige Auswahl.

Äquivalenzklassen

Eine **Äquivalenzklasse** ist eine Menge von Eingaben, bei denen sich der Code gleich verhält. Ob du 90 % oder 95 % testest, ist egal: Beide liefern die Note 1 über dieselbe Zeile im Code. Daher reicht **ein Wert pro Klasse**. Ungültige Eingaben bilden eigene Klassen.



Klasse	Bereich	Erwartet
ungültig, zu klein	unter 0	`ArgumentOutOfRangeException`
Note 5	0 bis unter 50	5
Note 4	50 bis unter 62,5	4
Note 3	62,5 bis unter 75	3
Note 2	75 bis unter 87,5	2
Note 1	87,5 bis 100	1
ungültig, zu groß	über 100	`ArgumentOutOfRangeException`

Grenzwertanalyse

Die meisten Fehler entstehen an den Grenzen der Klassen: `>` statt `>=` oder ein Tippfehler in einer Grenze. Daher testest du an **jeder Grenze** den Wert genau darauf und den Wert knapp daneben. Wir nehmen an, dass Prozent höchstens zwei Kommastellen haben. „Knapp daneben“ ist dann 0,01 entfernt. Beim Notenrechner ergibt das 12 Grenzwerte (die orangen Zahlen in der Grafik). Liegt ein Grenzwert schon in einer Klasse, brauchst du für diese Klasse keinen eigenen Wert mehr.

```
[Theory]
[InlineData(0, 5)]
[InlineData(49.99, 5)]
[InlineData(50, 4)]
[InlineData(62.49, 4)]
[InlineData(62.5, 3)]
[InlineData(74.99, 3)]
[InlineData(75, 2)]
[InlineData(87.49, 2)]
[InlineData(87.5, 1)]
[InlineData(100, 1)]
public void CalculateGrade_AtBoundary_ReturnsExpectedGrade(decimal percent, int
expected)
{
    Assert.Equal(expected, GradeCalculator.CalculateGrade(percent));
}

[Theory]
[InlineData(-0.01)]
[InlineData(100.01)]
public void CalculateGrade_OutOfRange_ThrowsArgumentOutOfRangeException(decimal
percent)
{
    Assert.Throws<ArgumentOutOfRangeException>(() => GradeCalculator.CalculateGrade
(percent));
}
```

12 Testfälle prüfen den Notenrechner vollständig. Das ist die Antwort auf „Wie viele Tests?“: **Jede Äquivalenzklasse und jede Grenze kommt mindestens einmal vor.**

Code Coverage

Code Coverage (Testabdeckung) misst, welcher Teil des Codes beim Ausführen der Tests durchlaufen wurde:

Maß	Frage
Line Coverage	Wie viele Zeilen wurden mindestens einmal ausgeführt?
Branch Coverage	Wurde jede Verzweigung in *beide* Richtungen durchlaufen? Bei `if` also einmal `true` und einmal `false`.

Testest du den Notenrechner nur mit 90 % und 30 %, laufen diese Zeilen:

```
if (percent < 0 || percent > 100)                // ✓ executed, but always false
{
    throw new ArgumentOutOfRangeException(...);  // ✗ never executed
}
if (percent >= 87.5m) { return 1; }              // ✓ true (90) and false (30)
if (percent >= 75m) { return 2; }                // ✗ "return 2" never executed
if (percent >= 62.5m) { return 3; }              // ✗ "return 3" never executed
if (percent >= 50m) { return 4; }                // ✗ "return 4" never executed
return 5;                                        // ✓ executed (30)
```

Coverage zeigt dir also, welche Fälle **fehlen**. Werkzeuge dafür sind in Visual Studio, Rider und `dotnet test` eingebaut. Wichtig ist, die Zahl richtig zu deuten.

100 % Coverage heißt nicht fehlerfrei

Angenommen, im Code steht `percent > 87.5m` statt `percent >= 87.5m`. Diese Tests führen jede Zeile und jede Verzweigung aus:

```
90 → 1,  80 → 2,  70 → 3,  55 → 4,  30 → 5,  -1 → Exception,  101 → Exception
```

Line und Branch Coverage sind 100 %, alle Tests sind grün. Trotzdem liefert 87,5 % die Note 2. Der Fehler sitzt an einer Grenze, und keine Grenze wurde getestet. Coverage kann außerdem Code nicht messen, der **fehlt**. Hätte jemand die Prüfung auf `> 100` vergessen, gäbe es auch keine Zeile, die als „nicht ausgeführt“ auffällt.

Werkzeug	Beantwortet
Äquivalenzklassen + Grenzwerte	Welche Tests **brauche** ich?
Code Coverage	Welchen Code habe ich **noch nicht** getestet?

Deterministische Tests

Ein Test muss bei jedem Lauf dasselbe Ergebnis liefern (das *R* aus FIRST). Erinnere dich an *CreditCard* aus dem Kapitel Interfaces:

```
if (Expiration < DateTime.UtcNow) { return false; }
```

Ein Test mit einer Karte, die am 31.12.2030 abläuft, ist heute grün. Ab 2031 wird er rot, obwohl niemand den Code geändert hat. Und den Fall „Karte läuft genau heute ab“ kannst du gar nicht gezielt testen. `DateTime.UtcNow` ist eine **versteckte Abhängigkeit**: *CreditCard* holt sich die Zeit selbst, und der Test kann sie nicht beeinflussen.

Die Lösung: Die Klasse holt sich die Zeit nicht selbst. Sie bekommt ein Objekt, das die Zeit liefert, als Parameter im Konstruktor. Im Programm ist das die echte Uhr, im Test eine feste Zeit. Diese Technik heißt **Dependency Injection**. Mehr dazu steht im Kapitel D: Dependency Inversion.

.NET hat dafür die abstrakte Klasse `TimeProvider`. *CreditCard* bekommt sie im Konstruktor und fragt sie nach der Zeit:

CreditCard.cs (im Projekt *UnittestDemo.Application*)

```
using System;

namespace UnittestDemo.Application;

public class CreditCard
{
    private readonly TimeProvider _timeProvider;

    public CreditCard(decimal limit, DateTime expiration, TimeProvider timeProvider)
    {
        Limit = limit;
        Expiration = expiration;
        _timeProvider = timeProvider;
    }

    public decimal Limit { get; }
    public DateTime Expiration { get; }

    public bool Pay(decimal amount)
    {
        if (amount > Limit) { return false; }
        if (Expiration < _timeProvider.GetUtcNow().UtcDateTime) { return false; }
        return true;
    }
}
```

Im Programm übergibst du `TimeProvider.System`, das liefert die echte Uhrzeit. Im Test übergibst du einen Fake, der immer dieselbe Zeit liefert:

FixedTimeProvider.cs (im Projekt *UnittestDemo.Tests*)

```
using System;

namespace UnittestDemo.Tests;

public class FixedTimeProvider : TimeProvider
{
    private readonly DateTimeOffset _now;

    public FixedTimeProvider(DateTimeOffset now)
    {
        _now = now;
    }

    public override DateTimeOffset GetUtcNow() => _now;
}
```

```
[Fact]
public void Pay_AfterExpiration_ReturnsFalse()
{
    TimeProvider afterExpiration = new FixedTimeProvider(new DateTimeOffset(2031, 1, 1,
0, 0, 0, TimeSpan.Zero));
    CreditCard card = new CreditCard(limit: 1000, expiration: new DateTime(2030, 12,
31), afterExpiration);

    bool success = card.Pay(100);

    Assert.False(success);
}
```

Dieser Test liefert heute und in zehn Jahren dasselbe Ergebnis. Dasselbe gilt für Zufallszahlen: Eine Klasse, die `new Random()` selbst erzeugt, ist nicht testbar. Bekommt sie das `Random`-Objekt von außen, übergibt der Test eines mit festem Startwert (`new Random(42)`). Dann ist die Folge der Zufallszahlen bei jedem Lauf gleich.

Test Driven Development

Bisher hast du zuerst den Code geschrieben und dann die Tests. Beim **Test Driven Development** (TDD) ist es umgekehrt. Du arbeitest in kurzen Zyklen:

Schritt	Was du tust
Red	Schreibe einen Test für das nächste kleine Verhalten. Er ist rot, denn der Code fehlt noch.
Green	Schreibe gerade so viel Code, dass der Test grün wird.
Refactor	Räume den Code auf. Die Tests zeigen, dass dabei nichts kaputtgeht.

Der Vorteil: Du denkst zuerst darüber nach, *was* der Code tun soll, und erst danach, *wie*. Die Übungsprojekte dieses Kurses funktionieren ähnlich: Die Tests sind schon rot, du machst sie grün.

SOLID und DRY

Programmieren im Zeitalter der Agents

Du schreibst in einen Chat: „Programmiere mir den Checkout für einen Webshop in C#.“ Nach 20 Sekunden hast du 300 Zeilen Code. Er kompiliert, und die Tests sind grün. Bist du fertig? Vielleicht. Vielleicht steht die Berechnung der Versandkosten aber an drei Stellen im Code. Beim nächsten Preiswechsel vergisst jemand eine davon, und die Rechnung stimmt nicht mehr mit dem Warenkorb überein. Ob Code *gut* ist, sagt dir der Agent nicht von selbst. Das beurteilst du. In diesem Kapitelblock lernst du, guten und schlechten Code zu unterscheiden und Probleme mit den richtigen Fachbegriffen zu benennen.

Der Mythos: „Programmieren muss ich nicht mehr können“

Viele glauben: Mit LLMs und Agents muss niemand mehr programmieren können. Ein **LLM** (*Large Language Model*) ist ein Sprachmodell wie ChatGPT, Claude oder Gemini. Ein **Coding Agent** ist ein Programm, das mit einem LLM selbstständig Dateien ändert und Befehle wie `dotnet test` ausführt. Man beschreibt, was man will, und der Code entsteht von selbst. Das stimmt nur zur Hälfte:

Agents kennen mehr Patterns und schreiben schneller als die meisten Entwickler. Die Entscheidungen bleiben aber bei dir.

Ein **Pattern** (Entwurfsmuster) ist eine bewährte Lösung für ein Problem, das immer wieder vorkommt. Es hat einen Namen, zum Beispiel *Strategy*. Wer den Namen kennt, muss die Lösung nicht jedes Mal neu erklären.

Das kann ein Agent gut	Das entscheidet er nicht für dich
In Sekunden viel Code schreiben	Was willst du eigentlich?
Hunderte Patterns und Bibliotheken kennen	Was ist in diesem Projekt wichtig?
Fehlermeldungen lesen und Code anpassen, bis Tests grün sind	Wann ist der Code gut genug?

Warum kann der Agent diese drei Fragen nicht für dich beantworten?

Frage	Der Agent …	Beispiel Webshop
Was willst du eigentlich?	kennt nur deinen Auftrag. Was fehlt, ergänzt er mit einer Annahme.	Gilt der Gratisversand ab 50 € vor oder nach dem Rabatt?
Was ist im Projekt wichtig?	kennt die Pläne deiner Firma nicht.	Nächstes Jahr kommt eine neue Zahlungsart dazu. Genau diese Stelle muss leicht erweiterbar sein.
Wann ist der Code gut genug?	kann immer noch eine Schicht Abstraktion ergänzen.	Ein Schulprojekt für zwei Wochen braucht keine Architektur für zehn Jahre.

Diese Entscheidungen kannst du nur treffen, wenn du Code lesen und beurteilen kannst. In Firmen liest vor jeder Übernahme eine zweite Person den Code durch. Das heißt **Code Review**. Schreibt ein Agent den Code, bist du diese zweite Person.

Fachsprache als Kompression

Zwei Expertinnen tauschen in fünf Sätzen mehr Information aus als ein Laie mit einer Expertin in einer Stunde. Der Grund: Fachbegriffe sind komprimiertes Wissen. Ein Wort wie *Constructor Injection* steht für eine ganze Seite Erklärung. Kennen beide die Seite, genügt das Wort.

Mit einem Agent ist es genauso. Er kennt die Fachbegriffe. Kennst du sie auch, schreibst du viel schärfere und klarere Aufträge:

Vager Auftrag	Präziser Auftrag
„Mach den Code besser.“	„Extrahiere die Rabattberechnung in eine Strategie. `CheckoutService` bekommt sie per Constructor Injection.“

Was passiert bei den beiden Aufträgen?

- **Vager Auftrag:** Der Agent muss raten, was „besser“ heißt. Er benennt vielleicht Variablen um, ergänzt Kommentare und baut drei Klassen um, die du gar nicht ändern wolltest. Du bekommst einen Diff (die Liste der Änderungen) mit 400 Zeilen und musst alle prüfen.
- **Präziser Auftrag:** Der Agent weiß genau, was er tun soll. Es entstehen ein Interface, eine Klasse und ein neuer Konstruktorparameter. Der Diff ist klein, und du prüfst ihn in einer Minute.

Die beiden Begriffe im präzisen Auftrag lernst du in diesem Block kennen:

- **Strategie** (*Strategy Pattern*): Ein Verfahren, zum Beispiel eine Rabattberechnung, steckt hinter einem Interface. Man kann es austauschen, ohne den Aufrufer zu ändern. Siehe O, Teil 2: Erweitern statt ändern.
- **Constructor Injection:** Eine Klasse erzeugt ihre Helfer nicht selbst mit `new`. Sie bekommt sie als Parameter im Konstruktor. Siehe D: Dependency Inversion.

Weitere präzise Formulierungen, wie du sie nach diesem Block verwenden kannst:

Du siehst im Code …	Du formulierst …
Die Steuer wird an drei Stellen fast gleich berechnet.	„Die Steuerberechnung ist dreifach vorhanden (DRY). Fasse sie in einer Methode zusammen.“ (DRY)
`OrderService` validiert, rechnet, speichert und formatiert.	„`OrderService` hat vier Zuständigkeiten. Trenne die Formatierung der Bestätigung in eine eigene Klasse.“ (S)

Du siehst im Code …	Du formulierst …
`if (order.Total > 50)` steht in mehreren Klassen.	„Die Regel für Gratisversand gehört zu `Order`. Ersetze die Bedingung durch eine Property `QualifiesForFreeShipping`.“ (O, Teil 1)
`GiftCard.Refund()` wirft `NotSupportedException`.	„`GiftCard` verletzt Liskov. Trenne `Refund()` in ein eigenes Interface `IRefundable`.“ (L)

Diese Sätze helfen dir nicht nur mit einem Agent. Sie sind auch die Sprache im Code Review mit Kolleginnen und Kollegen. Statt „das gefällt mir nicht“ sagst du, *was* das Problem ist und *was* du stattdessen erwartest.

Was ist „guter Code“?

Zwei Programme liefern bei allen Tests dasselbe Ergebnis. Ist eines davon besser? Ja, denn „funktioniert“ ist nur die Mindestanforderung. Guter Code hat noch drei weitere Eigenschaften:

Eigenschaft	Prüffrage
Funktioniert	Liefert der Code das richtige Ergebnis? Sind die Tests grün?
Lesbar	Verstehst du in einer Minute, was eine Methode tut und warum?
Änderbar	Musst du für eine neue Anforderung nur eine Stelle ändern?
Testbar	Kannst du eine Klasse ohne Datenbank, Netzwerk oder aktuelle Uhrzeit testen?

Der Kern dahinter sind die **Kosten einer Änderung**. Code wird viel öfter gelesen und geändert als neu geschrieben. Ein Beispiel: Der Gratisversand gilt ab jetzt ab 60 € statt ab 50 €.

- Im guten Code änderst du **eine** Zeile.
- Im schlechten Code steht `50` an fünf Stellen. Du findest vier davon. Die fünfte zeigt im Warenkorb weiter „Gratisversand“, die Rechnung verlangt aber Versandkosten.

Sprechender Code hilft Menschen und Agents

Sieh dir diese Bedingung an:

```
if (employee.MonthlySalary > 5000m)
{
    bonus = 0m;
}
```

Was bedeutet 5000? Ist es eine Grenze für die Steuer, für den Bonus oder für etwas anderes? Gilt sie auch an anderen Stellen? Eine Zahl ohne Namen heißt **Magic Number**. Wer den Code liest, muss die Absicht erraten. Vergleiche:

```
if (employee.HasHighSalary)
{
    bonus = 0m;
}
```

```
namespace WebshopDemo.Application;

/// <summary>
/// An employee with a monthly gross salary.
/// </summary>
public class Employee
{
    /// <summary>Monthly gross salary above this limit counts as high.</summary>
    public const decimal HighSalaryLimit = 5000m;

    public Employee(decimal monthlySalary)
    {
        MonthlySalary = monthlySalary;
    }

    public decimal MonthlySalary { get; }

    /// <summary>True if the salary is above <see cref="HighSalaryLimit"/>.</summary>
    public bool HasHighSalary => MonthlySalary > HighSalaryLimit;
}
```

Jetzt steht die Absicht im Namen, und die Grenze ist an genau einer Stelle definiert. Code mit solchen Namen heißt **sprechend**: Er erklärt sich selbst.

Das gilt auch für ein LLM. Ein LLM liest Code in **Tokens** (Wortteilen). Bei `> 5000m` muss es mehr Code lesen, zum Beispiel andere Klassen, Kommentare und Tests. Es muss auch länger nachdenken, um die Absicht zu erschließen. Viele Agents haben dafür eine Einstellung für den Denkaufwand (*Effort*). Oft rät das Modell richtig. Verlassen kannst du dich aber nicht darauf. Bei `HasHighSalary` muss niemand raten, weder du noch der Agent.



Guter Code für Menschen ist auch guter Code für Agents. Was dich beim Lesen verwirrt, kostet auch ein LLM Tokens und macht Fehler wahrscheinlicher.

Code Smells

Die Magic Number ist ein **Code Smell** (wörtlich „Code-Geruch“). Das ist ein Merkmal im Code, das auf ein Problem im Design hinweist. Der Code funktioniert, aber er „riecht“: Er wird schwer zu ändern oder zu testen sein. Bekannt wurde der Begriff durch das Buch *Refactoring* von Martin Fowler.

Ein Smell ist kein Fehler, sondern ein Hinweis. Du schaust genauer hin und entscheidest dann. Typische Smells:

- Derselbe Code steht an mehreren Stellen (*Duplicated Code*).
- Eine Klasse hat 500 Zeilen und macht alles (*God Class*).
- Ein `switch` über eine Art oder einen Typ steht in mehreren Methoden.
- Eine Methode wirft `NotSupportedException`, weil sie „eigentlich nicht passt“.

Zu jedem Prinzip in diesem Block gehören bestimmte Smells. Die Übersicht am Ende des Kapitels zeigt sie.

Tests als Sicherheitsnetz

Du hast einen Smell gefunden und willst den Code verbessern. Woher weißt du danach, dass noch alles funktioniert?

Refactoring heißt: Du änderst die *Struktur* des Codes, aber nicht sein *Verhalten*. Beispiele sind eine Methode umbenennen, doppelten Code in eine Methode auslagern oder eine große Klasse aufteilen. Eine neue Funktion oder ein Bugfix ist **kein** Refactoring. Mach das in einem eigenen Schritt.

„Verhalten bleibt gleich“ musst du prüfen. Genau das leisten Tests. Sie sind dein **Sicherheitsnetz**:

1. Vor dem Refactoring sind alle Tests grün.
2. Du machst einen kleinen Schritt, zum Beispiel eine Methode auslagern.
3. Du führst `dotnet test` aus.
4. Ist ein Test rot, hast du Verhalten geändert. Mach den Schritt rückgängig oder korrigiere ihn.
5. Ist alles grün, machst du den nächsten Schritt.

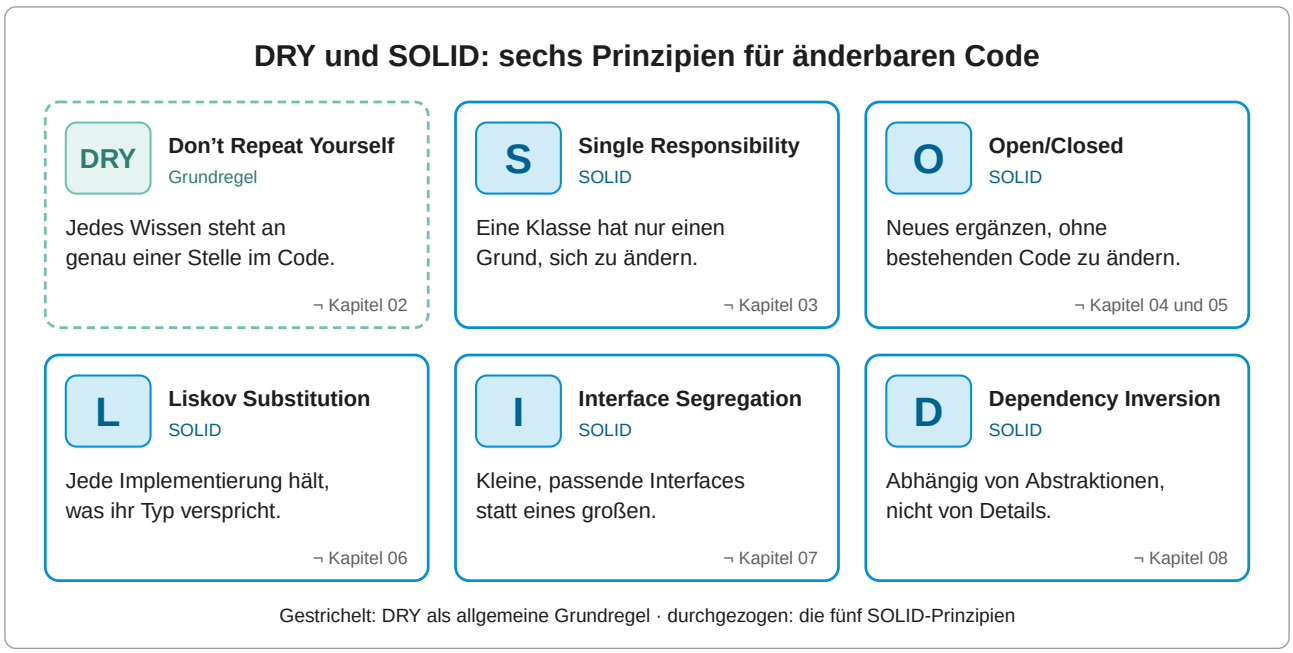
Für einen Agent gilt dieselbe Schleife. Ohne Tests weiß er nur, dass der Code kompiliert. Mehr dazu steht im Abschnitt Tests als Feedback für Agents.



In den Übungen dieses Blocks bekommst du funktionierenden, aber schlechten Code. Die Tests sind von Anfang an grün. Nach deinem Refactoring müssen sie immer noch grün sein. Die Tests darfst du nicht ändern.

Überblick: DRY und SOLID

Für änderbaren Code gibt es bekannte Prinzipien. **DRY** stammt aus dem Buch *The Pragmatic Programmer* von Andrew Hunt und David Thomas. **SOLID** fasst fünf Prinzipien zusammen, die vor allem Robert C. Martin bekannt gemacht hat. Jeder Buchstabe steht für ein Prinzip.



Prinzip	In einem Satz	Typischer Smell	Kapitel
DRY – Don’t Repeat Yourself	Jedes Wissen steht an genau einer Stelle im Code.	Fast gleicher Code an mehreren Stellen	DRY
S – Single Responsibility	Eine Klasse hat nur einen Grund, sich zu ändern.	Klasse validiert, rechnet, speichert und formatiert	S
O – Open/Closed	Neues ergänzt du, ohne bestehenden Code zu ändern.	Dieselbe Bedingung (`Total > 50`) an mehreren Stellen; `switch` über eine Art	O, Teil 1, O, Teil 2
L – Liskov Substitution	Jede Implementierung hält, was ihr Interface oder ihre Basisklasse verspricht.	`NotSupportedException`; Aufrufer prüft den Typ mit `is`	L
I – Interface Segregation	Viele kleine, passende Interfaces statt eines großen.	Klassen implementieren Methoden, die sie nicht brauchen	I
D – Dependency Inversion	Klassen hängen von Abstraktionen ab, nicht von konkreten Details.	`new` für Logger, Datenbank oder Zahlungsdienst mitten in der Logik	D

Eine **Abstraktion** ist hier ein Interface oder eine abstrakte Klasse: Sie beschreibt, *was* etwas kann, aber nicht, *wie* es das macht.

Prinzipien mit Augenmaß

Prinzipien sind keine Gesetze. Wer jede Regel bis zum Ende befolgt, bekommt für jede Klasse ein

Interface und für jede Zeile eine eigene Klasse. Das nennt man **Over-Engineering**: Der Code ist komplizierter, als das Problem verlangt. Auch Agents neigen manchmal dazu, weil sie viele Patterns kennen und gerne anwenden.

Frag dich deshalb bei jedem Umbau:

- Welches konkrete Problem löse ich damit? Welcher Smell verschwindet?
- Wird die nächste wahrscheinliche Änderung dadurch billiger?
- Versteht jemand anderer den Code danach schneller oder langsamer?

Kannst du keine der Fragen mit „ja“ oder „schneller“ beantworten, lass den Code, wie er ist.

Wie die folgenden Kapitel aufgebaut sind

Jedes Kapitel dieses Blocks folgt demselben Muster:

1. Ein Beispiel aus einem Webshop zeigt schlechten Code und das Problem dahinter.
2. Du siehst, wie der Code nach dem Prinzip aussieht und warum er besser ist.
3. Die Tabelle *Im Code-Review erkennen* fasst Warnzeichen und präzise Formulierungen zusammen.
4. In der Übung baust du meist ein Projekt mit grünen Tests um, ohne die Tests zu ändern.

DRY: Don’t Repeat Yourself

Ein Webshop berechnet den Gesamtpreis einer Bestellung an drei Stellen: für Kunden in Österreich, für Kunden in Deutschland und für Firmenkunden. Jede Stelle addiert die Rechnungszeilen, rechnet die Versandkosten dazu und schlägt die Steuer auf. Dann ändert sich eine Regel: Gratisversand gibt es nicht mehr ab 50 €, sondern erst ab 60 €. Die Entwicklerin ändert zwei Stellen. Die dritte übersieht sie. Ab jetzt bekommen Firmenkunden bei 55 € den Versand geschenkt, alle anderen nicht. In diesem Kapitel lernst du, solche Wiederholungen zu erkennen und in **Methoden** an genau einer Stelle zusammenzufassen, ohne dabei etwas kaputt zu machen.



Zum Selbst-Erarbeiten: Duplikation erkennen und Parameter finden

In der interaktiven Präsentation zu DRY siehst du Schritt für Schritt, wie aus zwei fast gleichen Blöcken eine Methode mit Parametern wird. Danach entscheidest du in mehreren Runden selbst, welche Teile gleich bleiben und welche zu Parametern werden. So trainierst du den Blick, den du für jedes Refactoring brauchst.

Projekt anlegen

Du brauchst eine Solution mit einer Application und einem Testprojekt. Führe diese Befehle in der Konsole aus. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings*, im Testprojekt auch `<Using Include="Xunit" />`. Füge in beiden `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Was die einzelnen Befehle und Einstellungen bedeuten, steht im Kapitel Unit Tests mit xUnit.

Das Problem: dreimal fast derselbe Code

Eine Rechnungszeile besteht aus Produkt, Nettopreis pro Stück und Menge:

OrderLine.cs (im Projekt *WebshopDemo.Application*)

```
namespace WebshopDemo.Application;

/// <summary>
/// One line of an order: a product, its net unit price and the ordered quantity.
/// </summary>
public record OrderLine(string Product, decimal UnitPrice, int Quantity);
```

Die Klasse `PriceCalculator` berechnet den Gesamtpreis für drei Gruppen von Kunden. Die Regeln sind erfunden: In Österreich kostet der Versand 4,90 € und die Steuer 20 %, in Deutschland 6,90 € und 19 %. Firmenkunden zahlen 4,90 € Versand und in unserem Beispiel keine Steuer.

PriceCalculator.cs (im Projekt *WebshopDemo.Application*)

```
using System;
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>
/// Calculates the total of an order for different groups of customers.
/// </summary>
public class PriceCalculator
{
    public decimal CalculateTotalAustria(IReadOnlyList<OrderLine> lines)
    {
        decimal net = 0m;
        foreach (OrderLine line in lines)
        {
            net += line.UnitPrice * line.Quantity;
        }
        decimal shipping = net >= 60m ? 0m : 4.90m;
        return Math.Round((net + shipping) * 1.20m, 2);
    }

    public decimal CalculateTotalGermany(IReadOnlyList<OrderLine> lines)
    {
        decimal sum = 0m;
        foreach (OrderLine line in lines)
        {
            sum += line.Quantity * line.UnitPrice;
        }
        decimal shippingCost = 6.90m;
        if (sum >= 60m)
        {
```

```
            shippingCost = 0m;
        }
        return Math.Round((sum + shippingCost) * 1.19m, 2);
    }

    public decimal CalculateTotalBusiness(IReadOnlyList<OrderLine> lines)
    {
        decimal net = 0m;
        foreach (OrderLine line in lines)
        {
            net += line.UnitPrice * line.Quantity;
        }
        decimal shipping = net >= 50m ? 0m : 4.90m;
        return Math.Round(net + shipping, 2);
    }
}
```

Lege die drei Methoden nebeneinander:

Teil	Österreich	Deutschland	Firmenkunden
Rechnungszeilen addieren	✔ gleich	✔ gleich, aber anders geschrieben (`sum`, `Quantity * UnitPrice`)	✔ gleich
Gratisversand ab	60 €	60 €	**50 € ✗** vergessen
Versandkosten	4,90 €	6,90 €	4,90 €
Steuersatz	20 %	19 %	0 %

Die Grenze für den Gratisversand wurde in zwei Methoden geändert, in der dritten nicht. Die Methode für Deutschland sieht außerdem anders aus. Eine Suche nach dem Text `net += line.UnitPrice` findet sie nicht. **Doppelten Code erkennst du nicht immer am gleichen Text.**

Ein Test zeigt den Fehler:

PriceCalculatorTests.cs (im Projekt *WebshopDemo.Tests*)

```
using System.Collections.Generic;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class PriceCalculatorTests
{
    private readonly PriceCalculator _calculator = new PriceCalculator();

    // 2 x 12.50 + 1 x 20.00 = 45.00 net, below the limit for free shipping
    private readonly List<OrderLine> _smallOrder = new List<OrderLine>
    {
        new OrderLine("USB cable", 12.50m, 2),
        new OrderLine("Mouse", 20.00m, 1)
    };
```

```
    [Fact]
    public void CalculateTotalAustria_SmallOrder_AddsShippingAndTax()
    {
        // (45.00 + 4.90) * 1.20 = 59.88
        Assert.Equal(59.88m, _calculator.CalculateTotalAustria(_smallOrder));
    }

    [Fact]
    public void CalculateTotalGermany_SmallOrder_AddsShippingAndTax()
    {
        // (45.00 + 6.90) * 1.19 = 61.761, rounded to 61.76
        Assert.Equal(61.76m, _calculator.CalculateTotalGermany(_smallOrder));
    }

    [Fact]
    public void CalculateTotalBusiness_OrderOf55Euro_AddsShipping()
    {
        List<OrderLine> lines = new List<OrderLine> { new OrderLine("Keyboard", 55.00m,
1) };
        // 55.00 + 4.90 = 59.90, business customers pay no tax in this example
        Assert.Equal(59.90m, _calculator.CalculateTotalBusiness(lines));
    }
}
```

```
failed
WebshopDemo.Tests.PriceCalculatorTests.CalculateTotalBusiness_OrderOf55Euro_AddsShippin
g
  Assert.Equal() Failure: Values differ
  Expected: 59.90
  Actual:   55.00
```

Die Erwartungswerte in den Kommentaren rechnest du von Hand nach. So prüft der Test wirklich die Regel und nicht nur, ob der Code das tut, was er eben tut.

DRY: Wissen an genau einer Stelle

Andrew Hunt und David Thomas formulieren das Prinzip in ihrem Buch *The Pragmatic Programmer* so:

> *Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.*

Auf Deutsch: Jedes Stück Wissen hat im System genau **eine** eindeutige und maßgebliche Darstellung. Mit **Wissen** sind Regeln und Entscheidungen gemeint, zum Beispiel „Gratisversand ab 60 €“ oder „so berechnet man die Summe einer Bestellung“.

DRY betrifft also das **Wissen** und nicht nur gleichen Text:

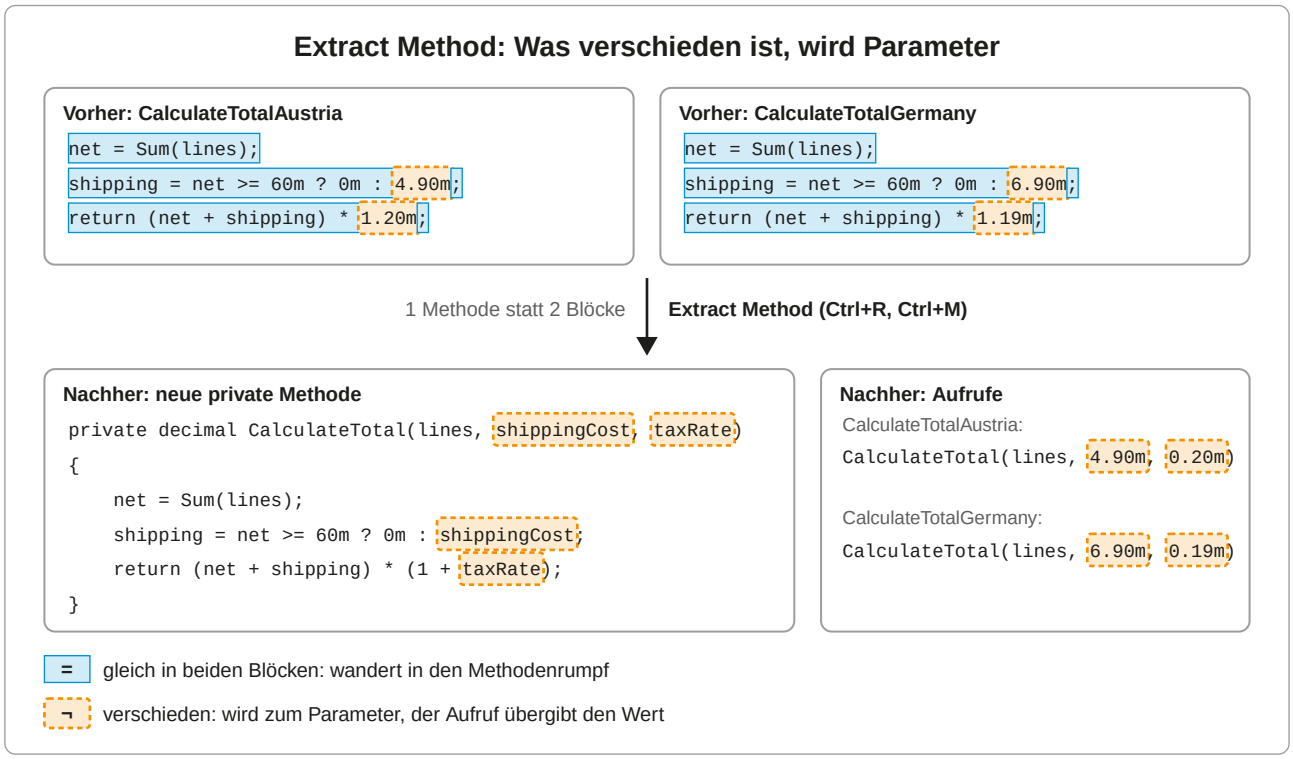
Situation	DRY verletzt?
Die Grenze 60 € steht in drei Methoden.	✔ Ja, dieselbe Regel steht dreimal da.
Die Summe wird einmal mit `UnitPrice * Quantity`, einmal mit `Quantity * UnitPrice` berechnet.	✔ Ja, gleiches Wissen in anderer Form.
Zwei Methoden enthalten beide die Zeile `decimal total = 0m;`.	✗ Nein, das ist kein Wissen, sondern Handwerk.
Rückgabefrist und Zahlungsziel sind beide 14 Tage.	✗ Nein, das sind zwei Regeln mit zufällig gleichem Wert (siehe *Wann DRY schadet*).

Die Prüffrage lautet: **Wenn sich diese Regel ändert, wie viele Stellen muss ich anpassen?** Ist die Antwort größer als eins, ist DRY verletzt.

Methoden extrahieren

Die Lösung ist eine Methode, die das gemeinsame Wissen enthält. So gehst du vor:

1. **Vergleichen:** Lege die Blöcke nebeneinander. Markiere, was gleich ist und was sich unterscheidet.
2. **Unterschiede werden Parameter:** Versandkosten und Steuersatz sind in jedem Block anders. Sie werden zu den Parametern `shippingCost` und `taxRate`.
3. **Gleiches wird der Methodenrumpf:** Zeilen addieren, Versand prüfen, Steuer aufschlagen.
4. **Sprechender Name:** Der Name sagt, *was* die Methode liefert, nicht *wie*.
5. **Ersetzen:** Jeder Block wird zu einem Aufruf der neuen Methode.



PriceCalculator.cs nach dem Umbau:

```
using System;
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>
/// Calculates the total of an order for different groups of customers.
/// </summary>
public class PriceCalculator
{
    /// <summary>Orders with at least this net value are shipped for free.</summary>
    private const decimal FreeShippingLimit = 60m;

    public decimal CalculateTotalAustria(IReadOnlyList<OrderLine> lines)
    {
        return CalculateTotal(lines, shippingCost: 4.90m, taxRate: 0.20m);
    }

    public decimal CalculateTotalGermany(IReadOnlyList<OrderLine> lines)
    {
        return CalculateTotal(lines, shippingCost: 6.90m, taxRate: 0.19m);
    }

    public decimal CalculateTotalBusiness(IReadOnlyList<OrderLine> lines)
    {
        return CalculateTotal(lines, shippingCost: 4.90m, taxRate: 0m);
    }

    private static decimal CalculateTotal(IReadOnlyList<OrderLine> lines, decimal
shippingCost, decimal taxRate)
    {
        decimal net = SumLines(lines);
        decimal shipping = CalculateShipping(net, shippingCost);
        return Math.Round((net + shipping) * (1 + taxRate), 2);
    }

    private static decimal SumLines(IReadOnlyList<OrderLine> lines)
    {
        decimal net = 0m;
        foreach (OrderLine line in lines)
        {
            net += line.UnitPrice * line.Quantity;
        }
        return net;
    }

    private static decimal CalculateShipping(decimal net, decimal shippingCost)
    {
        return net >= FreeShippingLimit ? 0m : shippingCost;
    }
}
```

Was ist jetzt besser?

- **Die Regel steht einmal da.** Gratisversand ab 70 € heißt: eine Zeile ändern. Der Fehler bei den Firmenkunden ist ganz nebenbei verschwunden, alle Tests sind grün.
- **Private Hilfsmethoden:** `CalculateTotal`, `SumLines` und `CalculateShipping` sind `private`. Sie sind ein Detail der Klasse. Von außen sieht man nur die drei öffentlichen Methoden. Die Tests prüfen weiterhin nur diese.
- `static` zeigt: Die Hilfsmethode verwendet keine Felder und ändert den Zustand nicht.
- **Benannte Argumente:** `shippingCost: 4.90m` zeigt beim Lesen, welche Zahl was bedeutet.
- **Sprechende Namen:** Jede Methode hat eine Aufgabe, ihr Name ersetzt einen Kommentar. `CalculateShipping` sagt mehr als `Calc2` oder `Helper`. `SumLines` beschreibt das *Was*, `DoLoop` nur das *Wie*. Ein „And“ im Namen (`CheckAndCalc`) verrät zwei Aufgaben.



DRY spart nicht unbedingt Zeilen. Die Klasse ist nach dem Umbau sogar 3 Zeilen länger. Was sinkt, sind die Stellen, die du bei einer Änderung anfassen musst. Eine vierte Kundengruppe kostet jetzt 4 Zeilen statt 10, und sie kann die Versandregel nicht mehr falsch übernehmen.

Regressionstest: erst absichern, dann umbauen

Ein Fehler, der durch eine Änderung in Code entsteht, der vorher funktioniert hat, heißt **Regression** (wörtlich: Rückschritt), siehe Warum testen?. Ein **Regressionstest** ist ein Test, der vor einer Änderung grün war und danach grün bleiben muss. Er schützt vor Rückschritten. Für jedes Refactoring gilt deshalb diese Reihenfolge:

Schritt	Was du tust	Tests
1	Tests für das bestehende Verhalten schreiben, falls sie fehlen	grün
2	Gefundene Fehler **getrennt** beheben, hier: `50m` → `60m`	grün
3	Einen kleinen Umbau machen, zum Beispiel `SumLines` extrahieren	grün?
4	Nach **jedem** Umbau `dotnet test` ausführen	grün!
5	Ist alles grün: Commit. Dann zurück zu Schritt 3	grün

Ein Bugfix ist kein Refactoring (siehe Tests als Sicherheitsnetz). Mischst du beides, weißt du bei einem roten Test nicht, ob der Fix oder der Umbau schuld ist. Kleine Schritte haben denselben Grund: Nach fünf geänderten Zeilen ist ein Fehler schnell gefunden, nach fünfzig nicht. Testfälle findest du wie in Testfälle finden.

Werkzeuge der IDE

Code von Hand in eine neue Methode zu kopieren ist langsam und fehleranfällig. Die IDE erkennt selbst, welche Variablen die neue Methode als Parameter braucht und was sie zurückgeben muss.

Extract Method und Rename

Markiere die Zeilen, die in die neue Methode sollen, und drücke:

Aktion	Visual Studio	Rider (Windows/Linux)	Rider (macOS)	VS Code (C# Dev Kit)
Extract Method	`Ctrl+R, Ctrl+M`	`Ctrl+Alt+M`	`⌥⌘M`	`Ctrl+.` → *Extract method*
Rename	`Ctrl+R, Ctrl+R`	`Shift+F6`	`⇧F6`	`F2`
Alle Aktionen an der Stelle	`Ctrl+.` (*Quick Actions*)	`Ctrl+Alt+Shift+T` (*Refactor This*)	`^T`	`Ctrl+.`

`Ctrl+R, Ctrl+M` heißt: Halte `Ctrl` gedrückt und drücke nacheinander `R` und `M`. Die IDE legt die Methode an und markiert ihren Namen. Tippe gleich einen sprechenden Namen ein. **Rename** ändert einen Namen überall, wo er verwendet wird, auch in anderen Dateien. Suchen und Ersetzen erwischt dagegen auch gleichnamige Variablen in anderen Methoden.

Code Metrics: messen statt schätzen

„Die Methode ist zu lang“ ist eine Meinung. Eine Zahl ist ein Argument. **Code Metrics** sind Kennzahlen, die die IDE aus dem Quelltext berechnet:

Metrik	Bedeutung	Richtwert pro Methode
Lines of Source Code	Zeilen im Quelltext, mit Leerzeilen und Kommentaren	so wenig wie nötig
Lines of Executable Code	ungefähre Anzahl der ausführbaren Anweisungen	so wenig wie nötig
Cyclomatic Complexity	Anzahl der Wege durch die Methode	höchstens 10
Maintainability Index	Wartbarkeit von 0 bis 100. Ab 20 grün, 10 bis 19 gelb, unter 10 rot	möglichst hoch

Die **zyklomatische Komplexität** (*Cyclomatic Complexity*) zählt die unabhängigen Wege durch eine Methode. Du startest bei 1 und zählst für jedes `if`, jede Schleife, jedes `case`, jedes `?:`, jedes `&&` und jedes `||` eins dazu. `CalculateTotalGermany` hat eine Schleife und ein `if`, also den Wert 3. Jeder Weg braucht mindestens einen Testfall. Eine hohe Komplexität heißt also auch: viele Tests und viele Stellen für Fehler.

Bei `PriceCalculator` haben die drei öffentlichen Methoden vor dem Umbau je die Komplexität 3

und einen Maintainability Index zwischen 66 und 70. Nach dem Umbau haben sie die Komplexität 1 und den Index 92. Der Index der ganzen Klasse steigt von 68 auf 82.

In Visual Studio: Markiere im Solution Explorer die Solution. Wähle *View → Other Windows → Code Metrics* und dann *Calculate Code Metrics for Solution*. In Visual Studio 2022 heißt der Befehl *Analyze → Calculate Code Metrics → For Solution*. Im Fenster klappst du Projekt, Namespace und Klasse auf und siehst die Werte jeder Methode.

In Rider, VS Code oder in der Konsole: Der C#-Compiler bringt Analyseregeln mit. Die Regel **CA1502** meldet Methoden, deren Komplexität eine Grenze überschreitet. Sie ist standardmäßig ausgeschaltet. So schaltest du sie ein:

1. Lege im Projektordner die Datei *CodeMetricsConfig.txt* mit der Zeile `CA1502: 6` an.
2. Trage sie in der Projektdatei ein:

```
<ItemGroup>
  <AdditionalFiles Include="CodeMetricsConfig.txt" />
</ItemGroup>
```

3. Lege im Projektordner eine Datei *.editorconfig* an:

```
[*.cs]
dotnet_diagnostic.CA1502.severity = warning
```

Jetzt meldet `dotnet build` jede Methode mit einer Komplexität über 6. Wegen *TreatWarningsAsErrors* bricht der Build sogar ab. Für die Methode `CalculatePrice` aus der Übung *PraterTickets* sieht das so aus:

```
error CA1502: 'CalculatePrice' has a cyclomatic complexity of '19'. Rewrite or refactor
the code
to decrease its complexity below '7'.
```

Metriken sind ein Hinweis, kein Urteil. Ein `switch` mit zwölf einfachen Fällen ist lesbar, hat aber die Komplexität 13. Hohe Werte sagen dir, wo du zuerst hinschauen solltest.

Wann DRY schadet

DRY heißt nicht, jede Ähnlichkeit sofort zu entfernen. Falsch angewendet macht es Code schlechter.

Rule of Three

Martin Fowler beschreibt in seinem Buch *Refactoring* die **Rule of Three**. Er schreibt sie seinem Kollegen Don Roberts zu:

1. Beim ersten Mal schreibst du den Code einfach.
2. Beim zweiten Mal bemerkst du die Wiederholung, schreibst ihn aber noch einmal.
3. Beim dritten Mal fasst du zusammen.

Nach zwei Fällen weißt du oft noch nicht, was wirklich gleich ist. Beim dritten Fall siehst du es. Eine Ausnahme sind Regeln wie „Gratisversand ab 60 €“: Sie gehören schon beim zweiten Mal an eine Stelle.

Zufällig gleich ist nicht doppelt

```
private const int ReturnPeriodDays = 14;
private const int PaymentTermDays = 14;
```

Beide Werte sind 14. Trotzdem wäre eine gemeinsame Konstante `DefaultDays` falsch. Rückgabefrist und Zahlungsziel sind zwei Regeln, die sich aus verschiedenen Gründen ändern. Verlängert der Shop das Zahlungsziel auf 30 Tage, ändert sich mit `DefaultDays` auch die Rückgabefrist. Frag dich deshalb: **Ändern sich beide Stellen immer gemeinsam?** Nur dann ist es dasselbe Wissen.

Die falsche Abstraktion: bool-Parameter

Was passiert, wenn man Blöcke zusammenlegt, die sich nicht nur in Werten, sondern im **Ablauf** unterscheiden?

```
private static decimal CalculateTotal(IReadOnlyList<OrderLine> lines, bool isGermany,
bool isBusiness,
    bool isGiftOrder)
{
    decimal net = SumLines(lines);
    if (isGiftOrder && !isBusiness) { net += 2.50m; }   // gift wrapping
    decimal shipping = net >= 60m || isGiftOrder ? 0m : (isGermany ? 6.90m : 4.90m);
    decimal taxRate = isBusiness ? 0m : (isGermany ? 0.19m : 0.20m);
    return Math.Round((net + shipping) * (1 + taxRate), 2);
}
```

Der Aufruf `CalculateTotal(lines, false, true, false)` ist nicht mehr lesbar. Jeder neue Sonderfall bringt einen weiteren `bool` und ein weiteres `if`. Bei drei `bool`-Parametern gibt es 8 Kombinationen, von denen einige keinen Sinn ergeben. Sandi Metz hat dazu einen bekannten Satz geprägt: *„Duplication is far cheaper than the wrong abstraction.“* Doppelter Code ist billiger als eine falsche Abstraktion.

Unterschied zwischen den Blöcken	**Was du tust**
ein **Wert** (Preis, Satz, Grenze, Text)	Parameter
ein **zusätzlicher Schritt** in einem Block	bleibt beim Aufrufer, vor oder nach dem Aufruf
ein **anderer Ablauf**	getrennte Methoden oder Polymorphie, siehe O, Teil 2

LLMs abstrahieren gern zu früh

Bittest du ein LLM, doppelten Code zu entfernen, bekommst du oft mehr als nötig: eine Methode mit `bool`-Flags oder ein Interface mit nur einer Implementierung. Prüfe jedes Ergebnis mit diesen Fragen:

- Ist es wirklich dasselbe **Wissen**, oder nur zufällig gleicher Code?
- Sind die Unterschiede **Werte** (gut als Parameter) oder **Abläufe** (schlecht als `bool`)?
- Ist der Aufruf ohne Blick in die Methode verständlich?

Im Code-Review erkennen

Warnzeichen im Code	Verletztes Prinzip	Was du verlangst bzw. wie du es formulierst
Zwei oder mehr Blöcke unterscheiden sich nur in Zahlen oder Texten	DRY	„Die Blöcke in X und Y sind bis auf Versandkosten und Steuersatz gleich. Extrahiere eine private Methode mit diesen beiden Parametern.“
Dieselbe Grenze (`>= 60m`) steht an mehreren Stellen	DRY	„Die Grenze für Gratisversand steht dreimal im Code. Definiere sie an einer Stelle.“
Eine Methode ist länger als eine Bildschirmseite	Lesbarkeit	„`CalculatePrice` hat eine Komplexität von 19. Teile sie in private Methoden mit je einer Aufgabe auf.“
Kommentare gliedern eine Methode in Abschnitte (`// discount`)	Lesbarkeit	„Mach aus jedem kommentierten Abschnitt eine Methode, deren Name den Kommentar ersetzt.“
Eine Hilfsmethode hat mehrere `bool`-Parameter	falsche Abstraktion	„Die Flags zeigen verschiedene Abläufe. Trenne die Fälle wieder, statt sie über `bool` zu steuern.“
Ein Refactoring und ein Bugfix stecken im selben Commit	Arbeitsweise	„Trenne den Bugfix vom Umbau, damit die Regressionstests aussagekräftig bleiben.“

Zusatzmaterial

In der interaktiven Präsentation zu DRY siehst du die Schritte von zwei fast gleichen Blöcken bis zur extrahierten Methode mit grünen Tests. Danach übst du in mehreren Runden, welche Teile gleich bleiben und welche zu Parametern werden.

S: Single Responsibility

Das Marketing will einen neuen Text in der Bestellbestätigung. Du änderst eine Zeile in *OrderService.cs*. Am selben Tag ändert eine Kollegin dort die Grenze für den Gratisversand. Beim Zusammenführen gibt es einen Konflikt, obwohl ihr an verschiedenen Dingen gearbeitet habt. Die Klasse macht zu viele Dinge. Hier lernst du, solche Klassen zu erkennen und aufzuteilen.

Projekt anlegen

Du brauchst eine Solution mit Application und Testprojekt. Die Befehle und alle Anpassungen der Projektdateien sind im Kapitel Unit Tests mit xUnit genau erklärt. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Im Testprojekt entfernst du zusätzlich `<Using Include="Xunit" />`.

Ausgangslage: Ein OrderService für alles

Eine Bestellung besteht aus Zeilen mit Produkt, Stückpreis und Menge. Lege dafür *OrderLine.cs* und *Order.cs* im Projekt *WebshopDemo.Application* an:

```
namespace WebshopDemo.Application;

/// <summary>One line of an order: a product, its unit price and the
quantity.</summary>
public record OrderLine(string ProductName, decimal UnitPrice, int Quantity);
```

```
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>A placed order with its lines and the total price including
shipping.</summary>
public record Order(int Id, string CustomerEmail, IReadOnlyList<OrderLine> Lines,
decimal Total);
```

So einen Service schreibt ein Agent gerne, wenn du nur „Programmiere das Bestellen“ verlangst:

OrderService.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;

namespace WebshopDemo.Application;

/// <summary>Places orders in the webshop.</summary>
public class OrderService
{
    private readonly List<Order> orders = new List<Order>();
    private int nextId = 1;

    public IReadOnlyList<Order> Orders => orders;

    public string PlaceOrder(string customerEmail, IReadOnlyList<OrderLine> lines)
    {
        // Validate the order.
        if (!customerEmail.Contains('@'))
        {
            throw new ArgumentException("Invalid email address.", nameof(
customerEmail));
        }
        foreach (OrderLine line in lines)
        {
            if (line.Quantity <= 0)
            {
                throw new ArgumentException($"Invalid quantity for {line.ProductName}
.", nameof(lines));
            }
        }

        // Calculate the price.
        decimal subtotal = 0m;
        int itemCount = 0;
        foreach (OrderLine line in lines)
        {
            subtotal += line.UnitPrice * line.Quantity;
            itemCount += line.Quantity;
        }
```

```
        decimal shipping = subtotal >= 50m ? 0m : 4.90m;
        decimal total = subtotal + shipping;

        // Save the order.
        Order order = new Order(nextId, customerEmail, lines, total);
        nextId++;
        orders.Add(order);

        // Format the confirmation.
        string totalText = total.ToString("0.00", CultureInfo.InvariantCulture);
        return $"Thank you! Order {order.Id}: {itemCount} item(s), total {totalText}
EUR.";
    }
}
```

Der Code funktioniert, und die Tests sind grün:

OrderServiceTests.cs (im Projekt *WebshopDemo.Tests*)

```
using System.Collections.Generic;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class OrderServiceTests
{
    [Theory]
    [InlineData(24.99, 54.88)]
    [InlineData(25.00, 50.00)]
    public void PlaceOrder_TwoMugs_SavesTotalWithShippingRule(decimal unitPrice,
decimal expectedTotal)
    {
        OrderService service = new OrderService();
        List<OrderLine> lines = new List<OrderLine> { new OrderLine("Mug", unitPrice,
2) };

        service.PlaceOrder("anna@example.com", lines);

        Assert.Equal(expectedTotal, service.Orders[0].Total);
    }

    [Fact]
    public void PlaceOrder_ValidOrder_ReturnsConfirmation()
    {
        OrderService service = new OrderService();
        List<OrderLine> lines = new List<OrderLine> { new OrderLine("Mug", 12.50m, 2)
};

        string confirmation = service.PlaceOrder("anna@example.com", lines);

        Assert.Equal("Thank you! Order 1: 2 item(s), total 29.90 EUR.", confirmation);
    }
```

```
}
```

Nachrechnen: 2 × 24,99 € = 49,98 € ist unter 50 €, mit 4,90 € Versand also 54,88 €. Ab 50,00 € ist der Versand gratis.

Das Problem: Vier Aufgaben in einer Methode

Die Kommentare in `PlaceOrder` zeigen es schon: Die Methode hat vier Abschnitte mit vier verschiedenen Aufgaben. Was passiert, wenn sich eine davon ändert?

Änderung	**Was passiert in *OrderService.cs*?**
Marketing will einen neuen Bestätigungstext.	Du änderst eine Klasse, in der auch die Preisberechnung steht.
Gleichzeitig wird die Grenze für Gratisversand auf 60 € erhöht.	Zwei Personen ändern dieselbe Datei. Beim Zusammenführen (*Merge*) gibt es einen Konflikt.
Jemand schreibt die Preisberechnung mit LINQ neu und löscht die Schleife.	`itemCount` wird nicht mehr berechnet. Die **Bestätigung** zeigt jetzt „0 item(s)“, obwohl niemand sie angefasst hat.

Besonders tückisch ist `itemCount`: Der Abschnitt *Calculate the price* berechnet die Variable, nur *Format the confirmation* verwendet sie. Die Abschnitte sehen getrennt aus, hängen aber zusammen.

Tests brauchen zu viel Setup

Sieh dir den ersten Test noch einmal an. Er soll nur die Versandregel prüfen. Trotzdem braucht er:

- ein `OrderService`-Objekt mit seiner Liste im Speicher,
- eine gültige E-Mail-Adresse, obwohl die E-Mail für den Preis egal ist,
- den Umweg über `service.Orders[0]`, weil `PlaceOrder` nur den Text zurückgibt.

Wird die Prüfung der E-Mail strenger (zum Beispiel „muss mit *.at* enden“), wird dieser Test rot, obwohl sich am Preis nichts geändert hat. Ein Test, der aus fremden Gründen rot wird, kostet Zeit.

Das Prinzip: Ein Grund für Änderungen

Robert C. Martin formuliert das **Single Responsibility Principle (SRP)** so:

> A class should have only one reason to change.

Eine Klasse soll also nur **einen Grund** haben, sich zu ändern. Doch was ist ein „Grund für eine Änderung“? Frag dich: **Wer verlangt die Änderung?** Später hat Martin das Prinzip genau so

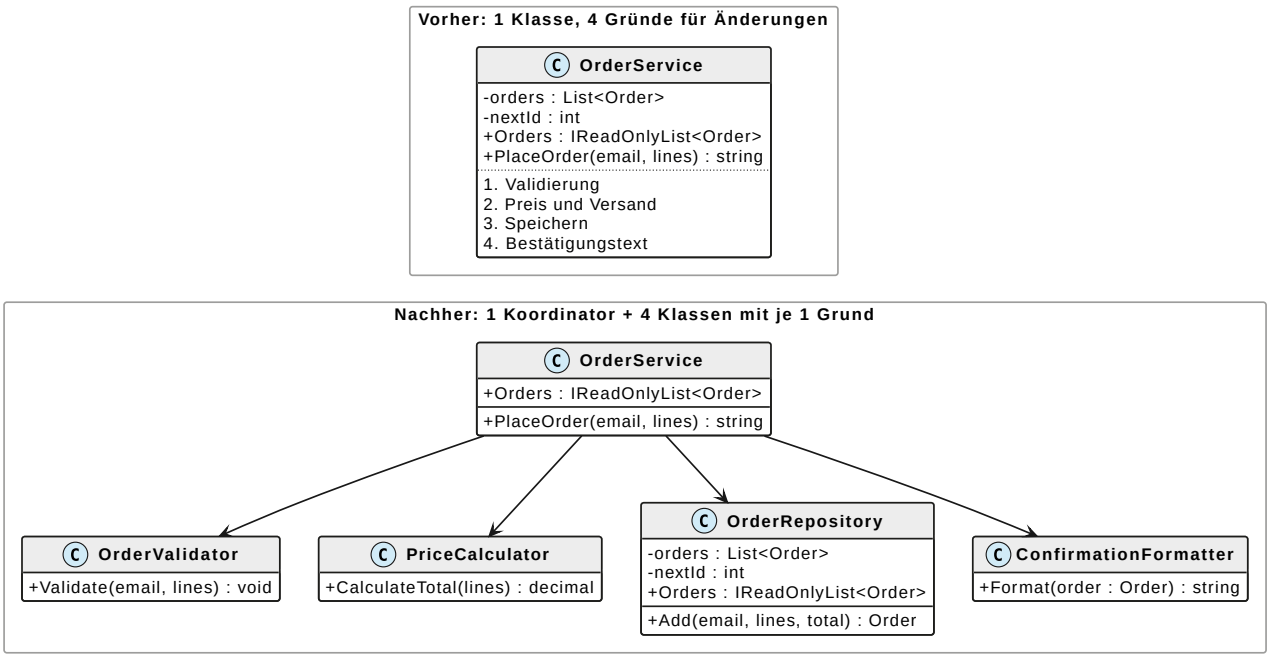
präzisiert: Eine Klasse ist nur *einer* Gruppe von Personen (*Actor*) verantwortlich.

Abschnitt in `PlaceOrder`	**Wer verlangt eine Änderung?**	**Beispiel**
Validierung	Fachabteilung Verkauf	„Pro Zeile höchstens 10 Stück.“
Preis und Versand	Geschäftsführung, Controlling	„Gratisversand erst ab 60 €.“
Speichern	IT-Betrieb	„Bestellungen kommen in eine SQL-Server-Datenbank.“
Bestätigungstext	Marketing	„Die Bestätigung soll freundlicher klingen.“

Vier Gruppen, also vier Gründe für Änderungen. `OrderService` verletzt das SRP. Eine Klasse mit nur einer Zuständigkeit nennt man **kohäsiv** (*cohesive*): Alles in ihr gehört zusammen.

Refactoring Schritt für Schritt

Du teilst die Klasse in vier Klassen mit je einer Zuständigkeit. `OrderService` bleibt bestehen und **koordiniert** nur noch: Er ruft die anderen Klassen in der richtigen Reihenfolge auf.



Führe nach jedem Schritt `dotnet test` aus. **Schritt 1:** Die Validierung kommt unverändert nach *OrderValidator.cs*.

```
using System;
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>Checks if an order may be placed.</summary>
public class OrderValidator
{
    /// <summary>Throws an <see cref="ArgumentException"/> if the order is
```

```
invalid.</summary>
    public void Validate(string customerEmail, IReadOnlyList<OrderLine> lines)
    {
        if (!customerEmail.Contains('@'))
        {
            throw new ArgumentException("Invalid email address.", nameof(
customerEmail));
        }
        foreach (OrderLine line in lines)
        {
            if (line.Quantity <= 0)
            {
                throw new ArgumentException($"Invalid quantity for {line.ProductName}
.", nameof(lines));
            }
        }
    }
}
```

Schritt 2: Die Preisberechnung kommt nach *PriceCalculator.cs*. `itemCount` gehört nicht hierher, der Preis braucht ihn nicht. Jetzt kannst du die Schleife gefahrlos durch LINQ ersetzen. Die Zahlen bekommen bei der Gelegenheit einen Namen.

```
using System.Collections.Generic;
using System.Linq;

namespace WebshopDemo.Application;

/// <summary>Calculates the price of an order with the shipping rules of the
shop.</summary>
public class PriceCalculator
{
    public const decimal FreeShippingLimit = 50m;
    public const decimal ShippingCost = 4.90m;

    /// <summary>Returns the sum of all lines plus shipping.</summary>
    public decimal CalculateTotal(IReadOnlyList<OrderLine> lines)
    {
        decimal subtotal = lines.Sum(line => line.UnitPrice * line.Quantity);
        decimal shipping = subtotal >= FreeShippingLimit ? 0m : ShippingCost;
        return subtotal + shipping;
    }
}
```

Schritt 3: Das Speichern kommt nach *OrderRepository.cs*. Ein **Repository** speichert Objekte und liefert sie wieder. Hier ist es eine Liste, später vielleicht eine Datenbank.

```
using System.Collections.Generic;

namespace WebshopDemo.Application;
```

```
/// <summary>Stores orders in memory and assigns their ids.</summary>
public class OrderRepository
{
    private readonly List<Order> orders = new List<Order>();
    private int nextId = 1;

    public IReadOnlyList<Order> Orders => orders;

    /// <summary>Creates an order with the next free id and stores it.</summary>
    public Order Add(string customerEmail, IReadOnlyList<OrderLine> lines, decimal
total)
    {
        Order order = new Order(nextId, customerEmail, lines, total);
        nextId++;
        orders.Add(order);
        return order;
    }
}
```

Schritt 4: Der Bestätigungstext kommt nach *ConfirmationFormatter.cs*. Die Stückzahl berechnet der Formatter selbst aus `order.Lines`, nicht mehr in der Preisberechnung.

```
using System.Globalization;
using System.Linq;

namespace WebshopDemo.Application;

/// <summary>Creates the confirmation text that the customer sees.</summary>
public class ConfirmationFormatter
{
    /// <summary>Returns a confirmation with the order id, the number of items and the
total.</summary>
    public string Format(Order order)
    {
        int itemCount = order.Lines.Sum(line => line.Quantity);
        string totalText = order.Total.ToString("0.00", CultureInfo.InvariantCulture);
        return $"Thank you! Order {order.Id}: {itemCount} item(s), total {totalText}
EUR.";
    }
}
```

Schritt 5: `OrderService` koordiniert nur noch. `PlaceOrder` liest sich jetzt wie eine Beschreibung des Ablaufs:

```
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>
/// Places orders in the webshop. Coordinates validation, price calculation,
/// storage and confirmation.
/// </summary>
```

```
public class OrderService
{
    private readonly OrderValidator validator = new OrderValidator();
    private readonly PriceCalculator calculator = new PriceCalculator();
    private readonly OrderRepository repository = new OrderRepository();
    private readonly ConfirmationFormatter formatter = new ConfirmationFormatter();

    public IReadOnlyList<Order> Orders => repository.Orders;

    public string PlaceOrder(string customerEmail, IReadOnlyList<OrderLine> lines)
    {
        validator.Validate(customerEmail, lines);
        decimal total = calculator.CalculateTotal(lines);
        Order order = repository.Add(customerEmail, lines, total);
        return formatter.Format(order);
    }
}
```

`OrderService` erzeugt seine Helfer hier noch selbst mit `new`. Wie er sie von außen bekommt, zeigt das Kapitel D: Dependency Inversion.

Die Tests: alte bleiben, neue werden kleiner

Die öffentliche API von `OrderService` hat sich nicht geändert. Deshalb laufen alle Tests aus *OrderServiceTests.cs* **unverändert** weiter. Sie sind dein **Regressionstest** (den Begriff kennst du aus DRY): Sie zeigen, dass das Verhalten nach dem Umbau gleich ist.

Neu ist: Jede Zuständigkeit kannst du jetzt **allein** testen. Für die Versandregel brauchst du keine E-Mail und keine Liste mehr:

PriceCalculatorTests.cs

```
using System.Collections.Generic;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class PriceCalculatorTests
{
    [Theory]
    [InlineData(49.99, 54.89)]
    [InlineData(50.00, 50.00)]
    public void CalculateTotal_AtFreeShippingLimit_AppliesShippingRule(decimal
unitPrice, decimal expectedTotal)
    {
        PriceCalculator calculator = new PriceCalculator();
        List<OrderLine> lines = new List<OrderLine> { new OrderLine("Lamp", unitPrice,
1) };

        decimal total = calculator.CalculateTotal(lines);
```

```
        Assert.Equal(expectedTotal, total);
    }
}
```

Für den Bestätigungstext erzeugst du eine `Order` direkt. Der Test hängt nicht mehr davon ab, wie der Preis berechnet wird:

ConfirmationFormatterTests.cs

```
using System.Collections.Generic;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class ConfirmationFormatterTests
{
    [Fact]
    public void Format_OrderWithThreeItems_ReturnsConfirmation()
    {
        List<OrderLine> lines = new List<OrderLine> { new OrderLine("Mug", 12.50m, 3)
};
        Order order = new Order(7, "anna@example.com", lines, 42.40m);
        ConfirmationFormatter formatter = new ConfirmationFormatter();

        string text = formatter.Format(order);

        Assert.Equal("Thank you! Order 7: 3 item(s), total 42.40 EUR.", text);
    }
}
```

Was hat das Aufteilen gebracht?

Vorher	Nachher
Neuer Bestätigungstext ändert die Klasse mit dem Preis.	Nur *ConfirmationFormatter.cs* ändert sich. Kein Merge-Konflikt.
Preis-Test braucht E-Mail, Service und Liste.	Preis-Test braucht nur Zeilen.
Ein roter Test sagt: „irgendwo in `PlaceOrder`“.	Ein roter Test zeigt auf genau eine Klasse.

Verstöße erkennen

Ein Verstoß fällt nicht immer so deutlich auf wie die Abschnittskommentare in `PlaceOrder`. Diese Warnzeichen helfen dir:

Warnzeichen	Beispiel
Vage Namen mit *Manager*, *Helper*, *Utils* oder *And*	`OrderManager`, `StringHelper`, `ValidateAndSave()`. Der Name sagt nicht, *wofür* die Klasse da ist. Also landet alles darin.

Warnzeichen	Beispiel
Beschreibung braucht „und“	„Die Klasse prüft die Bestellung **und** berechnet den Preis **und** speichert sie.“
Methoden nutzen getrennte Felder	Siehe unten.
`using`s aus vielen Bereichen	`System.IO` (Dateien), `System.Net.Http` (Netzwerk), `System.Text.Json` (Daten umwandeln) und `System.Globalization` (Formatierung) in einer Klasse.
Viele Codezeilen	Die Code Metrics (siehe DRY) zeigen für eine Klasse 300 *Lines of Code*. Es gibt keine feste Grenze. Ab etwa 200 Zeilen lohnt sich aber ein genauer Blick.

Getrennte Felder: Schreib für jede Methode auf, welche Felder sie verwendet. Eine Klasse `CustomerManager` könnte so aussehen:

Methode	`connectionString`	`smtpServer`	`exchangeRates`
`Save()`, `Load()`	✔		
`SendNewsletter()`		✔	
`ConvertToUsd()`			✔

Die Tabelle zerfällt in drei Blöcke ohne gemeinsame Felder: drei Klassen in einer Datei. In einer kohäsiven Klasse verwenden die meisten Methoden die meisten Felder.

Wie weit aufteilen?

SRP heißt **nicht** „eine Methode pro Klasse“. Sieh dir diese Aufteilung an:

```
SubtotalCalculator.CalculateSubtotal(lines)
ShippingCalculator.CalculateShipping(subtotal)
TotalCalculator.CalculateTotal(subtotal, shipping)
```

Kommt ein Rabatt dazu, änderst du wahrscheinlich alle drei. Sie ändern sich aus **demselben** Grund: Die Geschäftsführung ändert die Preisregeln. Sie gehören in **eine** Klasse. Drei Klassen sind hier Over-Engineering: mehr Dateien und Namen, mehr Sprünge beim Lesen, aber kein Gewinn.

Martin fasst beide Seiten in einem Satz zusammen:

> Gather together the things that change for the same reasons. Separate those things that change for different reasons.

Zu wenig getrennt ist eine *God Class*: Jede Änderung trifft fremden Code. Zu viel getrennt heißt: Eine Änderung betrifft viele kleine Klassen. Auch `OrderService` hat nach dem Umbau genau eine Zuständigkeit. Er kennt den **Ablauf** einer Bestellung und ändert sich nur, wenn sich die Reihen-

folge der Schritte ändert.

Im Code-Review erkennen

Warnzeichen im Code	Verletzt	Was du verlangst / wie du es formulierst
Eine Methode hat Abschnittskommentare wie `// Validate`, `// Save`, `// Format`.	SRP	„`PlaceOrder` hat vier Zuständigkeiten. Extrahiere die Formatierung in eine Klasse `ConfirmationFormatter`.“
Ein Test für die Preisberechnung braucht eine gültige E-Mail.	SRP	„Die Preisberechnung ist nicht allein testbar. Lagere sie in einen `PriceCalculator` aus.“
Eine Variable aus einem Abschnitt wird in einem anderen verwendet (`itemCount`).	SRP	„Der Bestätigungstext hängt an einer Hilfsvariable der Preisberechnung. Der Formatter soll seine Werte aus `Order` lesen.“
Klassenname mit *Manager*, *Helper*, *Utils* oder *And*.	SRP	„Der Name `OrderManager` sagt nicht, wofür die Klasse zuständig ist. Benenne die Zuständigkeiten und teile die Klasse danach auf.“
Gruppen von Methoden verwenden getrennte Felder.	SRP	„`Save`/`Load` und `SendNewsletter` haben keine gemeinsamen Daten. Trenne das Speichern in ein `CustomerRepository`.“
Fachklasse mit `using System.IO` oder `System.Net.Http`.	SRP	„Die Klasse mischt Fachlogik mit Ein- und Ausgabe. Verschiebe den Dateizugriff in eine eigene Klasse.“
Mehrere Mini-Klassen, die sich immer gemeinsam ändern.	Over-Engineering	„`SubtotalCalculator`, `ShippingCalculator` und `TotalCalculator` ändern sich aus demselben Grund. Fasse sie zu `PriceCalculator` zusammen.“

O, Teil 1: Werte und Regeln an genau einer Stelle

Der Webshop verschickt ab 50 € gratis. Diese Regel steht im Warenkorb, im Checkout und auf der Rechnung, jedes Mal als `if (order.Total >= 50m)`. Nun ändert der Shop die Grenze auf 60 €. Du findest zwei der drei Stellen. Der Warenkorb zeigt bei 55 € weiter „Gratisversand“, der Checkout verlangt aber Versandkosten. In diesem Kapitel lernst du, Werte und Regeln so zu schreiben, dass eine Änderung genau **eine** Stelle betrifft.

Projekt anlegen

Du brauchst eine Solution mit Application und Testprojekt. Die Befehle und alle Anpassungen der Projektdateien sind im Kapitel Unit Tests mit xUnit genau erklärt. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Im Testprojekt entfernst du zusätzlich `<Using Include="Xunit" />`.

Das O in SOLID

Das **Open/Closed Principle** (OCP) hat Bertrand Meyer 1988 formuliert. Robert C. Martin hat es als Teil von SOLID bekannt gemacht:

> Software entities should be open for extension, but closed for modification.

Auf Deutsch: Eine Klasse ist *offen* für Erweiterungen, aber *geschlossen* für Änderungen. Neues Verhalten ergänzt du, ohne bestehenden, getesteten Code zu ändern. Der Kern dieses Prinzips

ist **Polymorphie**. Ihn lernst du in O, Teil 2: Erweitern statt ändern kennen.

Dieses Kapitel zeigt die „kleine“ Form davon. Sie kommt im Alltag viel öfter vor:

	Teil 1 (dieses Kapitel)	Teil 2
Was ändert sich?	ein Wert oder eine Regel (50 € → 60 €)	ein neues Verhalten (eine neue Zahlungsart)
Wie viel änderst du?	genau eine Stelle	nichts Bestehendes, du ergänzt eine Klasse
Werkzeuge	Konstanten, Collections, Properties, Konstruktor	Interfaces und Polymorphie

Das Problem: Magic Values

So sieht der Code oft aus, wenn ein Anfänger oder ein Agent ohne genaue Vorgaben ihn schreibt:

Order.cs

```
namespace WebshopDemo.Application;

public class Order
{
    public Order(decimal total, string country)
    {
        Total = total;
        Country = country;
    }

    public decimal Total { get; }
    public string Country { get; }
}
```

CartView.cs

```
namespace WebshopDemo.Application;

public class CartView
{
    public string GetShippingHint(Order order)
    {
        if (order.Total >= 50m)
        {
            return "Free shipping!";
        }
        return "Free shipping from 50 EUR";
    }
}
```

CheckoutService.cs

```
namespace WebshopDemo.Application;

public class CheckoutService
{
    public decimal GetShippingCost(Order order)
    {
        if (order.Total >= 50m)
        {
            return 0m;
        }
        return 4.90m;
    }
}
```

InvoiceWriter.cs

```
namespace WebshopDemo.Application;

public class InvoiceWriter
{
    public string GetShippingLine(Order order)
    {
        if (order.Total > 50m)
        {
            return "Shipping: free";
        }
        return "Shipping: 4.90 EUR";
    }
}
```

Jede Klasse für sich ist kurz und verständlich. Das Problem siehst du erst, wenn du alle drei vergleichst:

- Die Zahl 50 steht an **vier** Stellen: in drei Bedingungen und in einem Text.
- Die Versandkosten 4,90 € stehen an zwei Stellen, einmal als Zahl, einmal im Text.
- *InvoiceWriter* verwendet `>` statt `>=`. Bei genau 50 € ist der Versand im Checkout gratis, auf der Rechnung aber nicht. Das zeigt dieser Test:

```
[Fact]
public void GetShippingLine_At50Euro_MatchesFreeShippingInCheckout()
{
    Order order = new Order(total: 50m, country: "AT");

    decimal cost = new CheckoutService().GetShippingCost(order);
    string line = new InvoiceWriter().GetShippingLine(order);

    Assert.Equal(0m, cost);
    Assert.Equal("Shipping: free", line);
```

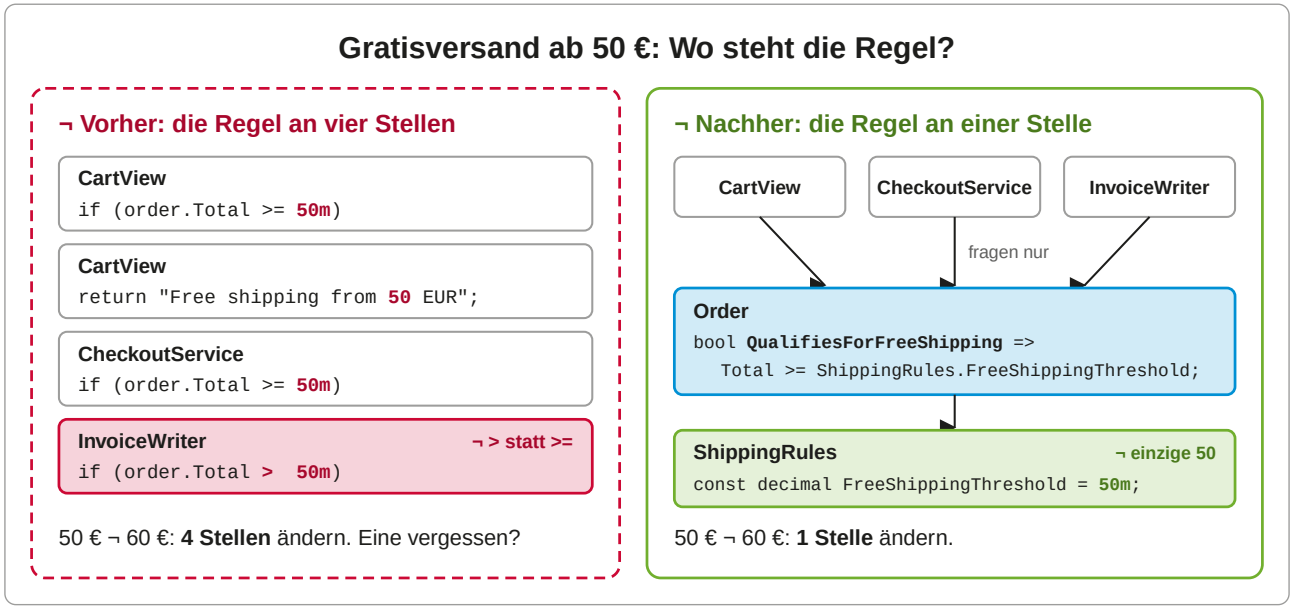
```
}
```

```
Assert.Equal() Failure: Strings differ
                     ↓ (pos 10)
Expected: "Shipping: free"
Actual:   "Shipping: 4.90 EUR"
                     ↑ (pos 10)
```

Ein **Magic Value** ist ein Literal mitten in der Logik, dessen Bedeutung nicht im Code steht. Bei Zahlen sagt man auch **Magic Number**, bei Texten **Magic String**. Magic Values machen zwei Probleme:

Problem	Beispiel
Die Bedeutung fehlt	Ist 50 die Grenze für Gratisversand, für einen Rabatt oder für Ratenzahlung?
Die Regel ist verstreut	Eine Änderung braucht vier Stellen. Eine davon vergisst man (siehe DRY).

Suchen und Ersetzen hilft nicht. Im Shop gibt es noch andere 50er: 50 Artikel pro Seite, 50 % Rabatt im Abverkauf. Du musst bei jedem Treffer entscheiden, ob er den Versand meint.



Lösung 1: Ein Name für den Wert

Der erste Schritt: Der Wert bekommt einen Namen und steht an genau einer Stelle.

ShippingRules.cs

```
namespace WebshopDemo.Application;

/// <summary>
```

```
/// The shipping rules of the shop.
/// </summary>
public static class ShippingRules
{
    /// <summary>Orders with at least this total (in EUR) are shipped for
free.</summary>
    public const decimal FreeShippingThreshold = 50m;

    /// <summary>Shipping cost in EUR for orders below the threshold.</summary>
    public const decimal StandardShippingCost = 4.90m;
}
```

Im Checkout steht jetzt `if (order.Total >= ShippingRules.FreeShippingThreshold)`. Auch der Text im Warenkorb verwendet die Konstante: `$"Free shipping from {ShippingRules.FreeShippingThreshold} EUR"`.

const, static readonly oder Konstruktor?

C# bietet drei Möglichkeiten, einen Wert an einer Stelle festzulegen:

Variante	Beispiel	Verwende sie, wenn …	Einschränkung
`const`	`public const decimal FreeShippingThreshold = 50m;`	sich der Wert nur mit einer neuen Programmversion ändert.	Nur Zahlen, `bool`, `char` und `string`.
`static readonly`	`public static readonly IReadOnlySet<string> DeliveryCountries = ...;`	du ein Objekt brauchst (Collection, `TimeSpan`, `DateTime`).	Wird einmal beim Start gesetzt, danach nicht mehr.
Konstruktorparameter	`new ShippingPolicy(freeShippingThreshold: 60m, ...)`	der Wert aus der Konfiguration kommt, pro Shop verschieden ist oder Tests andere Werte brauchen.	Jemand muss das Objekt erzeugen und weitergeben.

Bei `const` kopiert der Compiler den Wert an jede Stelle, die ihn verwendet. Ändert sich eine Konstante in einer DLL, musst du alle Projekte neu kompilieren, die sie verwenden. Innerhalb einer Solution merkst du davon nichts.

Lösung 2: Eine Liste an einer Stelle

Der Shop liefert nur nach Österreich, Deutschland und in die Schweiz. Diese Regel steht im Checkout und im Formular für die Lieferadresse:

```
// CheckoutService
if (order.Country == "AT" || order.Country == "DE" || order.Country == "CH") { /* ...
*/ }

// AddressForm
```

```
if (country == "AT" || country == "DE") { /* ... */ }
```

Im Formular fehlt die Schweiz. Kommt Italien dazu, musst du jede dieser Bedingungen finden und erweitern. Das ist eine **hartcodierte Liste**: eine Aufzählung von Werten mitten in einer Bedingung. Besser ist eine Collection an genau einer Stelle:

```
using System.Collections.Generic;

namespace WebshopDemo.Application;

public static class ShippingRules
{
    // FreeShippingThreshold and StandardShippingCost as above

    /// <summary>Country codes (ISO 3166) the shop delivers to.</summary>
    public static readonly IReadOnlySet<string> DeliveryCountries = new HashSet<string>
{ "AT", "DE", "CH" };
}
```

Die Bedingung heißt jetzt überall `ShippingRules.DeliveryCountries.Contains(country)`. Ein neues Land ist eine Änderung in einer Zeile.

- **Warum ein Set?** Ein `HashSet` enthält jeden Wert nur einmal, und `Contains` ist auch bei vielen Einträgen schnell. Siehe Kapitel Collections.
- **Warum `IReadOnlySet`?** `readonly` schützt nur die Variable, nicht den Inhalt. Mit dem Typ `IReadOnlySet<string>` kann niemand von außen `Add()` aufrufen.

Properties statt Bedingungen

Mit der Konstante steht der *Wert* an einer Stelle. Die *Regel* steht aber immer noch dreimal im Code:

```
if (order.Total >= ShippingRules.FreeShippingThreshold)   // CartView, CheckoutService,
InvoiceWriter
```

Der Fehler mit `>` statt `>=` kann also weiter passieren. Außerdem rechnet jede Klasse mit den Daten einer *anderen* Klasse. Die Lösung: Die Klasse, der die Daten gehören, beantwortet die Frage selbst. Dafür eignet sich ein **berechnetes Property** (siehe Kapitel Properties):

Order.cs

```
namespace WebshopDemo.Application;

/// <summary>
/// An order of the webshop.
/// </summary>
public class Order
```

```
{
    public Order(decimal total, string country)
    {
        Total = total;
        Country = country;
    }

    public decimal Total { get; }
    public string Country { get; }

    /// <summary>True if the order is shipped for free.</summary>
    public bool QualifiesForFreeShipping => Total >= ShippingRules
.FreeShippingThreshold;

    /// <summary>The shipping cost in EUR.</summary>
    public decimal ShippingCost => QualifiesForFreeShipping ? 0m : ShippingRules
.StandardShippingCost;

    /// <summary>True if the shop delivers to the country of the order.</summary>
    public bool CanBeDelivered => ShippingRules.DeliveryCountries.Contains(Country);
}
```

Die anderen Klassen fragen nur noch: `if (order.QualifiesForFreeShipping)`. Der Checkout gibt einfach `order.ShippingCost` zurück. Die Tests prüfen die Regel jetzt an einer Stelle, auch an der Grenze:

```
[Theory]
[InlineData(49.99, false)]
[InlineData(50, true)]
public void QualifiesForFreeShipping_AtThreshold_ReturnsExpectedResult(decimal total,
bool expected)
{
    Order order = new Order(total: total, country: "AT");

    Assert.Equal(expected, order.QualifiesForFreeShipping);
}

[Fact]
public void ShippingCost_Below50Euro_Returns4Point90()
{
    Order order = new Order(total: 49.99m, country: "AT");

    Assert.Equal(4.90m, order.ShippingCost);
}
```

Feature Envy und „Tell, don’t ask“

Dahinter stehen zwei bekannte Begriffe:

Begriff	Bedeutung	Im Beispiel
Feature Envy (Smell)	Eine Methode interessiert sich mehr für die Daten einer anderen Klasse als für die eigenen.	*CheckoutService* liest `order.Total` und entscheidet selbst.
Tell, don't ask	Frag ein Objekt nicht nach seinen Daten, um dann selbst zu entscheiden. Lass das Objekt entscheiden.	*CheckoutService* fragt `order.QualifiesForFreeShipping`.

Streng genommen meint „Tell, don't ask" Methoden, die etwas *tun* (`order.MarkAsPaid()`). Für dieses Kapitel genügt die Grundidee: **Die Entscheidung gehört zu den Daten.**

Property oder Methode? Ein Property passt, wenn die Berechnung schnell ist, keine Parameter braucht und nichts verändert. Braucht die Regel einen Parameter, ist es eine Methode, zum Beispiel `order.CanBeDeliveredTo(address)`.

Wenn sich der Wert ändern kann: Konfiguration

Das Marketing will die Grenze für Gratisversand selbst ändern, ohne neue Programmversion. Außerdem gibt es bald einen Shop in der Schweiz mit einer anderen Grenze. Eine `const` reicht dafür nicht. Der Wert kommt jetzt von außen, zum Beispiel aus einer Konfigurationsdatei. Dafür bekommt die Regel eine eigene Klasse, und die Werte kommen über den Konstruktor:

ShippingPolicy.cs

```
using System.Collections.Generic;

namespace WebshopDemo.Application;

/// <summary>
/// The configurable shipping rules of one shop.
/// </summary>
public class ShippingPolicy
{
    public ShippingPolicy(decimal freeShippingThreshold, decimal standardShippingCost,
        IReadOnlySet<string> deliveryCountries)
    {
        FreeShippingThreshold = freeShippingThreshold;
        StandardShippingCost = standardShippingCost;
        DeliveryCountries = deliveryCountries;
    }

    public decimal FreeShippingThreshold { get; }
    public decimal StandardShippingCost { get; }
    public IReadOnlySet<string> DeliveryCountries { get; }

    public bool QualifiesForFreeShipping(Order order) => order.Total >=
FreeShippingThreshold;

    public decimal GetShippingCost(Order order) => QualifiesForFreeShipping(order) ? 0m
```

```
: StandardShippingCost;

    public bool CanDeliverTo(string country) => DeliveryCountries.Contains(country);
}
```

Order hat jetzt nur noch `Total` und `Country`. Die Regel ist trotzdem an genau einer Stelle, nämlich in *ShippingPolicy*. Ein Test kann jede Grenze verwenden:

```
[Fact]
public void GetShippingCost_WithThreshold60And55Euro_Returns4Point90()
{
    ShippingPolicy policy = new ShippingPolicy(
        freeShippingThreshold: 60m,
        standardShippingCost: 4.90m,
        deliveryCountries: new HashSet<string> { "AT" });
    Order order = new Order(total: 55m, country: "AT");

    decimal cost = policy.GetShippingCost(order);

    Assert.Equal(4.90m, cost);
}
```

Die Klassen *CartView*, *CheckoutService* und *InvoiceWriter* bekommen die *ShippingPolicy* im Konstruktor. Diese Technik heißt *Constructor Injection*. Du lernst sie im Kapitel D: Dependency Inversion kennen.

Der Wert …	Wohin damit?
ändert sich nur mit einer neuen Programmversion	`const` oder `static readonly`, Regel als Property der Datenklasse
kommt aus der Konfiguration oder ist pro Shop verschieden	eigene Klasse wie *ShippingPolicy*, Werte über den Konstruktor

Lesbar für Menschen und für LLMs

Vergleiche diese beiden Zeilen:

```
if (order.Total >= 50m && ShippingRules.DeliveryCountries.Contains(order.Country)) { /*
... */ }

if (order.QualifiesForFreeShipping && order.CanBeDelivered) { /* ... */ }
```

Beide tun dasselbe. Die erste Zeile musst du *übersetzen*: „50, das ist wohl die Grenze für Gratisversand.“ Die zweite Zeile liest du wie einen Satz. Für einen Agent gilt dasselbe wie für dich (siehe Sprechender Code hilft Menschen und Agents):

Aufgabe für den Agent	Mit `order.Total >= 50m`	Mit `order.QualifiesForFreeShipping`
Verstehen, was die Bedingung bedeutet	Er muss die Absicht aus Aufrufern, Texten und Tests erschließen.	Die Absicht steht im Namen.
„Gratisversand ab 60 €“ umsetzen	Er sucht alle `50` und entscheidet bei jeder, ob sie den Versand meint.	Er ändert eine Konstante.
Kosten	mehr Tokens lesen, mehr Denkaufwand (*Effort*)	wenige Tokens

Moderne Modelle raten oft richtig. Verlassen kannst du dich darauf aber nicht. Jede Stelle, an der ein Agent raten muss, ist eine Stelle, an der er falsch raten kann. Ein guter Name nimmt ihm das Raten ab.

Wo ist die Grenze?

Nicht jedes Literal ist ein Magic Value. Diese Werte sind in Ordnung:

Code	Magic Value?	Warum?
`if (items.Count == 0)`	nein	0 heißt „leer“. Das versteht jeder.
`items[i + 1]`, `for (int i = 0; ...)`	nein	Technik der Schleife, keine Geschäftsregel.
`percent / 100m`	nein	Allgemein bekannte Umrechnung.
`new Order(total: 49.99m, ...)` im Test	nein	Testdaten sollen konkret und nachrechenbar sein.
`if (order.Total >= 50m)`	**ja**	Geschäftsregel, die sich ändern kann.
`if (status == 3)`	**ja**	Niemand weiß, was 3 bedeutet. Hier passt ein `enum`.

Die Faustregel: Würde eine neue Kollegin fragen „Warum genau dieser Wert?“ oder „Kann sich der Wert ändern?“, dann bekommt er einen Namen.

Der Name muss die *Bedeutung* nennen, nicht den Wert. `const decimal Fifty = 50m;` ist nutzlos. Ändert sich die Grenze auf 60, steht dann `Fifty = 60m` im Code.

Im Code-Review erkennen

Warnzeichen im Code	Verletztes Prinzip	Was du verlangst bzw. formulierst
Dieselbe Zahl in mehreren Klassen (`50m`, `4.90m`)	Magic Value, DRY	„Die Grenze für Gratisversand steht an vier Stellen. Lege sie als Konstante `FreeShippingThreshold` an einer Stelle an.“
Ein Wert steckt in einem Text (`"from 50 EUR"`)	Magic Value	„Der Text wiederholt die Grenze. Baue ihn aus der Konstante zusammen.“
`x == "AT" || x == "DE" || ...`	Hartcodierte Liste	„Lege die Lieferländer als `IReadOnlySet<string>` an einer Stelle an und prüfe mit `Contains`.“
Dieselbe Bedingung auf fremden Daten (`order.Total >= ...`) an mehreren Stellen	Feature Envy, OCP Teil 1	„Die Regel gehört zu `Order`. Ersetze die Bedingung durch ein Property `QualifiesForFreeShipping`.“
`>` an einer Stelle, `>=` an einer anderen	Regel mehrfach implementiert	„Die Regel ist zweimal verschieden umgesetzt. Es darf nur eine Implementierung geben.“
`const` für einen Wert, der pro Shop oder Kunde verschieden ist	Konfiguration im Code	„Die Grenze ist Konfiguration. Übergib sie über den Konstruktor einer `ShippingPolicy`.“
`if (status == 3)`	Magic Value	„Ersetze den Statuscode durch ein `enum` mit sprechenden Namen.“
`const decimal Fifty = 50m;`	Name ohne Bedeutung	„Der Name wiederholt nur den Wert. Benenne, wofür der Wert steht.“

O, Teil 2: Erweitern statt ändern

Der Webshop bietet drei Zahlungsarten an: Kreditkarte, PayPal und Rechnung. Jetzt soll die Lastschrift dazukommen. Eine Entwicklerin ergänzt das Enum und zwei `switch`-Anweisungen. Alle Tests sind grün, der Code geht online. Beim ersten Kunden mit Lastschrift bricht der Checkout mit einer Exception ab. Sie hat eine dritte `switch`-Anweisung übersehen. In diesem Kapitel lernst du, wie du Code so baust, dass eine neue Variante eine **neue Klasse** ist und kein Eingriff an vielen Stellen.

Im Kapitel O, Teil 1 ging es um Werte und Regeln. Jetzt geht es um **Verhalten**, das von einer Art abhängt: Jede Zahlungsart berechnet die Gebühr anders und bezahlt anders.

Projekt anlegen

Du brauchst eine Solution mit Application und Testprojekt. Die Befehle und alle Anpassungen der Projektdateien sind im Kapitel Unit Tests mit xUnit genau erklärt. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Im Testprojekt entfernst du zusätzlich `<Using Include="Xunit" />`.

Das Problem: dieselbe Fallunterscheidung an drei Stellen

Für die Zahlungsarten gelten diese (erfundenen) Regeln:

Zahlungsart	Gebühr	Höchstbetrag
Kreditkarte	1,5 % des Betrags	1 000 €

Zahlungsart	Gebühr	Höchstbetrag
PayPal	0,35 € + 2 % des Betrags	10 000 €
Rechnung	keine	500 €

Ein typischer erster Entwurf verwendet ein Enum und eine Serviceklasse. Die Methode `Pay()` simuliert hier nur die Zahlung. Ein echter Shop würde an dieser Stelle einen Zahlungsdienst im Internet aufrufen.

PaymentMethod.cs

```
namespace WebshopDemo.Application;

public enum PaymentMethod
{
    CreditCard = 1,
    PayPal,
    Invoice
}
```

PaymentService.cs

```
using System;

namespace WebshopDemo.Application;

public class PaymentService
{
    public decimal CalculateFee(PaymentMethod method, decimal amount)
    {
        switch (method)
        {
            case PaymentMethod.CreditCard:
                return Math.Round(amount * 0.015m, 2);
            case PaymentMethod.PayPal:
                return Math.Round(0.35m + amount * 0.02m, 2);
            case PaymentMethod.Invoice:
                return 0m;
            default:
                throw new ArgumentOutOfRangeException(nameof(method), $"Unknown payment
method {method}.");
        }
    }

    // Simulates the payment: every payment method accepts amounts up to a limit.
    public bool Pay(PaymentMethod method, decimal amount)
    {
        switch (method)
        {
            case PaymentMethod.CreditCard:
                return amount <= 1000m;
            case PaymentMethod.PayPal:
```

```
                return amount <= 10000m;
            case PaymentMethod.Invoice:
                return amount <= 500m;
            default:
                throw new ArgumentOutOfRangeException(nameof(method), $"Unknown payment
method {method}.");
        }
    }

    public string GetDisplayName(PaymentMethod method) => method switch
    {
        PaymentMethod.CreditCard => "Credit card",
        PaymentMethod.PayPal => "PayPal",
        PaymentMethod.Invoice => "Invoice",
        _ => throw new ArgumentOutOfRangeException(nameof(method), $"Unknown payment
method {method}.")
    };
}
```

Der Code funktioniert. Jede Methode fragt aber dieselbe Frage: *Welche Zahlungsart ist es?* Das Wissen über eine Zahlungsart ist auf drei Methoden verteilt. In einem echten Shop stehen solche Abfragen oft in verschiedenen Klassen: im Warenkorb, auf der Rechnung, in der Bestätigungsmail.

Eine Zahlungsart kommt dazu

Die Lastschrift (`DirectDebit`) hat keine Gebühr und einen Höchstbetrag von 2 000 €. Die Entwicklerin ergänzt das Enum, `CalculateFee()` und `GetDisplayName()`:

```
DirectDebit                                     // new value in the enum PaymentMethod

case PaymentMethod.DirectDebit:                 // new case in CalculateFee()
    return 0m;

PaymentMethod.DirectDebit => "Direct debit",    // new arm in GetDisplayName()
```

`Pay()` hat sie vergessen. Der Compiler meldet nichts, denn der `default`-Zweig fängt jeden Wert ab. Erst zur Laufzeit zeigt sich der Fehler. Ein Test macht ihn sichtbar:

```
[Fact]
public void Pay_DirectDebitWith200Euro_ReturnsTrue()
{
    PaymentService service = new PaymentService();
    Assert.True(service.Pay(PaymentMethod.DirectDebit, 200m));
}
```

```
failed WebshopDemo.Tests.PaymentServiceTests.Pay_DirectDebitWith200Euro_ReturnsTrue
  System.ArgumentOutOfRangeException : Unknown payment method DirectDebit. (Parameter
```

```
'method')
```

Gibt es diesen Test nicht, findet ihn der erste Kunde mit Lastschrift.



Kann der Compiler nicht helfen? Bei einer `switch`-Anweisung warnt er nie, wenn ein `case` fehlt. Lässt du bei einer `switch`-Expression den Zweig `_ =>` weg, warnt er mit CS8524. Ein Enum kann nämlich auch Werte ohne Namen enthalten, zum Beispiel `(PaymentMethod)0`. Darum schreibt man fast immer `_ => throw ...`. Genau dieser Zweig versteckt dann jeden neuen Wert, den du vergessen hast.

Das Problem ist nicht die eine vergessene Zeile. Das Problem ist das Design: **Jede neue Zahlungsart verlangt Änderungen an bestehendem, getestetem Code, und zwar an vielen Stellen.**

Das Open/Closed Principle

Das **O** in SOLID steht für das *Open/Closed Principle* (OCP). Bertrand Meyer hat es 1988 in seinem Buch *Object-Oriented Software Construction* formuliert. Robert C. Martin hat es später in der Form bekannt gemacht, die wir heute verwenden:

> "Software entities should be open for extension, but closed for modification."

Eine *Software Entity* ist eine Klasse, ein Modul oder eine Methode. Der Satz hat zwei Teile:

Teil	Bedeutung	Im Webshop
Open for extension (offen für Erweiterung)	Du kannst neues Verhalten ergänzen.	Eine neue Zahlungsart ist möglich.
Closed for modification (geschlossen für Änderung)	Dafür änderst du bestehenden Code nicht.	Der Checkout und die anderen Zahlungsarten bleiben, wie sie sind.

Klingt widersprüchlich? Der Trick: Das neue Verhalten kommt in **neuen Code**. Der alte Code ist fertig und getestet. Wer ihn nicht anfasst, kann darin auch nichts kaputt machen.

Die Lösung: Jede Zahlungsart ist eine Klasse

Im Kapitel Interfaces hast du das Interface `IPaymentProvider` mit der Methode `bool Pay(decimal amount)` kennengelernt. Wir erweitern es um alles, was sich je nach Zahlungsart unterscheidet:

IPaymentProvider.cs

```
namespace WebshopDemo.Application;

/// <summary>A payment method of the webshop. Every payment method is its own
class.</summary>
```

```
public interface IPaymentProvider
{
    string DisplayName { get; }
    decimal CalculateFee(decimal amount);
    bool Pay(decimal amount);
}
```

Jeder Zweig der `switch`-Anweisungen wird zu einer Klasse. Alles, was zur Kreditkarte gehört, steht jetzt in `CreditCard`:

CreditCard.cs

```
using System;

namespace WebshopDemo.Application;

public class CreditCard : IPaymentProvider
{
    public CreditCard(decimal limit)
    {
        Limit = limit;
    }

    public decimal Limit { get; }
    public string DisplayName => "Credit card";
    public decimal CalculateFee(decimal amount) => Math.Round(amount * 0.015m, 2);
    public bool Pay(decimal amount) => amount <= Limit;
}
```

PayPal.cs

```
using System;

namespace WebshopDemo.Application;

public class PayPal : IPaymentProvider
{
    public string DisplayName => "PayPal";
    public decimal CalculateFee(decimal amount) => Math.Round(0.35m + amount * 0.02m,
2);
    public bool Pay(decimal amount) => amount <= 10000m;
}
```

Invoice.cs

```
namespace WebshopDemo.Application;

public class Invoice : IPaymentProvider
{
    public string DisplayName => "Invoice";
```

```
    public decimal CalculateFee(decimal amount) => 0m;
    public bool Pay(decimal amount) => amount <= 500m;
}
```

Eine Klasse kann auch eigene Daten haben. Mit dem Enum hatte jede Kreditkarte das Limit 1 000 €. Jetzt bekommt jede Karte ihr eigenes Limit im Konstruktor.

Der Checkout kennt nur das Interface. Er enthält keinen `switch` mehr:

CheckoutService.cs

```
namespace WebshopDemo.Application;

/// <summary>Calculates the total of an order and pays it. Knows only the interface
IPaymentProvider.</summary>
public class CheckoutService
{
    private readonly IPaymentProvider _paymentProvider;

    public CheckoutService(IPaymentProvider paymentProvider)
    {
        _paymentProvider = paymentProvider;
    }

    public decimal CalculateTotal(decimal orderAmount) =>
        orderAmount + _paymentProvider.CalculateFee(orderAmount);

    public bool Pay(decimal orderAmount) => _paymentProvider.Pay(CalculateTotal
(orderAmount));
}
```

Die Tests übergeben die Zahlungsart als Objekt:

CheckoutServiceTests.cs

```
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class CheckoutServiceTests
{
    [Fact]
    public void CalculateTotal_CreditCardWith100Euro_Returns101Point50()
    {
        CheckoutService checkout = new CheckoutService(new CreditCard(limit: 1000m));
        Assert.Equal(101.50m, checkout.CalculateTotal(100m));
    }

    [Fact]
    public void Pay_InvoiceAbove500Euro_ReturnsFalse()
```

```
    {
        CheckoutService checkout = new CheckoutService(new Invoice());
        Assert.False(checkout.Pay(600m));
    }
}
```

Die Lastschrift kommt dazu, diesmal als neue Klasse

DirectDebit.cs

```
namespace WebshopDemo.Application;

public class DirectDebit : IPaymentProvider
{
    public string DisplayName => "Direct debit";
    public decimal CalculateFee(decimal amount) => 0m;
    public bool Pay(decimal amount) => amount <= 2000m;
}
```

DirectDebitTests.cs (`using`s und Namespace wie in *CheckoutServiceTests.cs*)

```
public class DirectDebitTests
{
    [Fact]
    public void Pay_DirectDebitWith200Euro_ReturnsTrue()
    {
        CheckoutService checkout = new CheckoutService(new DirectDebit());
        Assert.True(checkout.Pay(200m));
    }
}
```

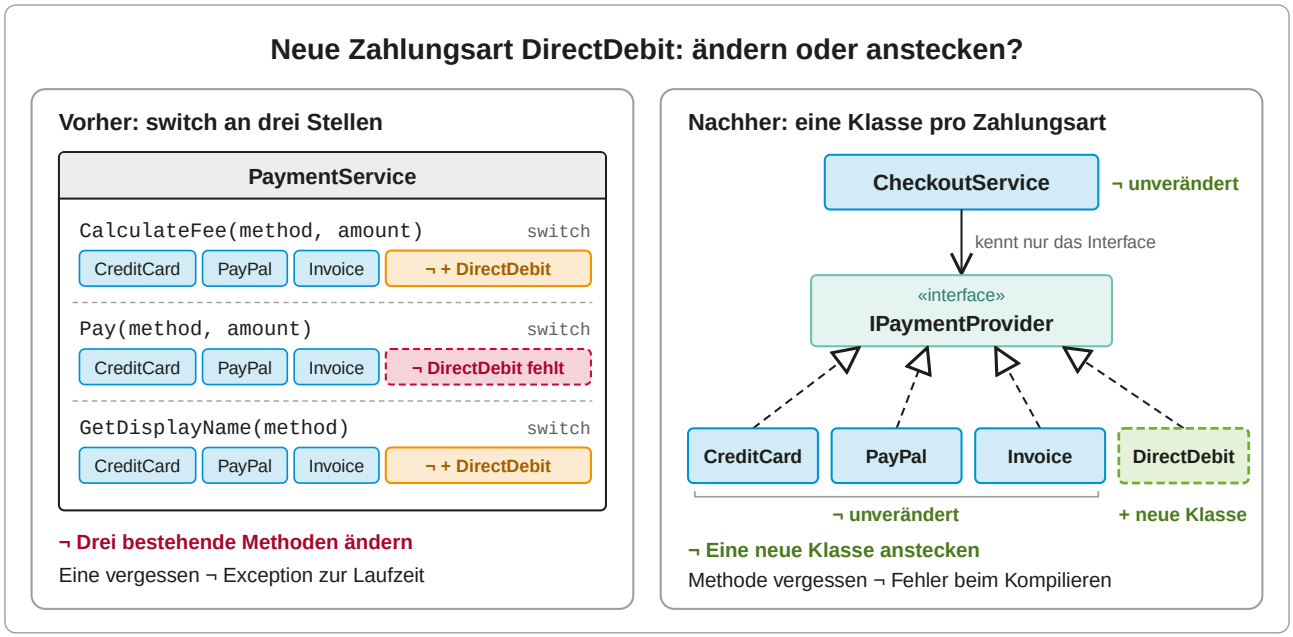
Vergleiche, was sich für die neue Zahlungsart geändert hat:

Datei	Mit `switch`	Mit Interface
`PaymentMethod` / `IPaymentProvider`	geändert (neuer Wert)	unverändert
`PaymentService` / `CheckoutService`	an drei Stellen geändert	unverändert
`CreditCard`, `PayPal`, `Invoice`	–	unverändert
`CheckoutServiceTests`	–	unverändert und grün
`DirectDebit`, `DirectDebitTests`	–	neu

Und wenn die Entwicklerin diesmal `Pay()` vergisst? Dann kompiliert der Code nicht:

```
DirectDebit.cs(3,28): error CS0535: 'DirectDebit' does not implement interface member
'IPaymentProvider.Pay(decimal)'
```

Der Fehler kommt jetzt vom Compiler und nicht vom ersten Kunden.



Das Strategy Pattern

Diese Lösung hat einen Namen: **Strategy Pattern**. Ein Verfahren, das es in mehreren Varianten gibt, steckt hinter einem Interface. Die Klasse, die es verwendet, kennt nur das Interface.

Rolle im Pattern	Im Webshop
Strategy (das Interface)	`IPaymentProvider`
Concrete Strategy (eine Variante)	`CreditCard`, `PayPal`, `Invoice`, `DirectDebit`
Context (verwendet die Strategie)	`CheckoutService`

Schreibst du einem Agent „Ersetze den `switch` über `PaymentMethod` durch das Strategy Pattern“, weiß er genau, was du meinst.

Interface oder abstrakte Klasse?

Sieh dir `CreditCard` und `PayPal` an. Beide runden die Gebühr auf Cent, und beide vergleichen den Betrag mit einem Limit. Kommen weitere Zahlungsarten dazu, wiederholt sich dieser Code (siehe DRY). Gemeinsamen Code legst du in eine **abstrakte Basisklasse**, wie im Kapitel Vererbung:

PaymentProvider.cs

```
using System;

namespace WebshopDemo.Application;

/// <summary>Shared code of all payment methods: the limit and the rounding of the
fee.</summary>
public abstract class PaymentProvider : IPaymentProvider
```

```
{
    protected PaymentProvider(decimal limit)
    {
        Limit = limit;
    }

    public decimal Limit { get; }
    public abstract string DisplayName { get; }

    // Template method: the fixed steps are here, the variable step is in the subclass.
    public decimal CalculateFee(decimal amount) => Math.Round(CalculateRawFee(amount),
2);

    public bool Pay(decimal amount) => amount <= Limit;

    protected abstract decimal CalculateRawFee(decimal amount);
}
```

PayPal.cs

```
namespace WebshopDemo.Application;

public class PayPal : PaymentProvider
{
    public PayPal() : base(limit: 10000m) { }

    public override string DisplayName => "PayPal";
    protected override decimal CalculateRawFee(decimal amount) => 0.35m + amount *
0.02m;
}
```

`CalculateFee()` ist in der Basisklasse fertig: Sie ruft den abstrakten Schritt `CalculateRawFee()` auf und rundet das Ergebnis. Jede Unterklasse liefert nur ihren Schritt. Diese Idee heißt **Template Method**: Die Basisklasse legt den Ablauf fest, die Unterklassen füllen einzelne Schritte.

Verwende …	**wenn …**
ein **Interface**	die Varianten nur dasselbe *können* müssen. Das ist der Normalfall.
zusätzlich eine **abstrakte Basisklasse**	mehrere Varianten denselben *Code* teilen.

`CheckoutService` kennt weiterhin nur `IPaymentProvider`. Ob eine Zahlungsart von `PaymentProvider` erbt, ist ihm egal.

Wo ein switch in Ordnung ist

Der Kunde wählt im Checkout-Formular eine Zahlungsart. Vom Browser kommt ein String wie `"paypal"`. Irgendwo muss das Programm daraus das passende Objekt machen. Dafür ist ein `switch` richtig:

PaymentProviderFactory.cs

```
using System;

namespace WebshopDemo.Application;

/// <summary>Creates the payment provider for the code the customer selected in the
checkout form.</summary>
public static class PaymentProviderFactory
{
    public static IPaymentProvider Create(string code) => code switch
    {
        "creditcard" => new CreditCard(limit: 1000m),
        "paypal" => new PayPal(),
        "invoice" => new Invoice(),
        "directdebit" => new DirectDebit(),
        _ => throw new ArgumentException($"Unknown payment method '{code}'.", nameof
(code))
    };
}
```

Eine Methode oder Klasse, die nur Objekte erzeugt, heißt **Factory**. Dieser `switch` ist in Ordnung, weil er drei Bedingungen erfüllt:

- Er steht an **genau einer** Stelle.
- Er **erzeugt nur** Objekte und enthält keine Geschäftsregeln (keine Gebühr, kein Limit).
- Eine neue Zahlungsart braucht hier eine neue Zeile. Bestehende Zeilen bleiben gleich.

In größeren Programmen steht diese Stelle beim Programmstart. Sie heißt dann *Composition Root*. Mehr dazu im Kapitel D: Dependency Inversion.

OCP mit Augenmaß: YAGNI

Musst du jetzt jede Fallunterscheidung durch Klassen ersetzen? Nein. **YAGNI** steht für *You Aren't Gonna Need It*: Baue nichts für eine Zukunft, die vielleicht nie kommt. Auch Agents bauen Interfaces gerne auf Vorrat. Frag dann: Welche zweite Variante gibt es wirklich?

Situation	Entscheidung
Ein `if` oder `switch` an **einer** Stelle, die Varianten ändern sich selten	Lass ihn stehen.
Dieselbe Fallunterscheidung an **mehreren** Stellen	Ersetze sie durch Klassen mit einem gemeinsamen Interface.
Es kommen **tatsächlich** neue Varianten dazu (Zahlungsarten, Versandarten, Tarife)	Interface. Hier lohnt sich OCP.
„Vielleicht brauchen wir das irgendwann“	Kein Interface auf Vorrat. Baue um, wenn die zweite Variante kommt.

Im Code-Review erkennen

Warnzeichen im Code	Verletztes Prinzip	Was du verlangst bzw. wie du es formulierst
`switch (paymentMethod)` mit denselben `case`-Werten in mehreren Methoden oder Klassen	OCP	„Die Fallunterscheidung über `PaymentMethod` steht an drei Stellen. Ersetze sie durch ein Interface `IPaymentProvider` mit einer Klasse pro Zahlungsart (Strategy Pattern).“
`if`/`else if`-Kette über einen Typcode (`if (type == "A") ... else if (type == "B")`)	OCP	„Jeder Zweig ist eine eigene Variante. Mach daraus eine Klasse pro Variante.“
Vergleich mit Typnamen: `GetType().Name == "PayPal"` oder Strings wie `"creditcard"` mitten in der Logik	OCP	„Der Aufrufer soll den konkreten Typ nicht kennen. Verschiebe das Verhalten in die Klasse und rufe es über das Interface auf.“
`default: throw new ...("Unknown ...")` in mehreren `switch`-Anweisungen	OCP	„Eine neue Variante fällt erst zur Laufzeit auf. Mit einem Interface meldet der Compiler fehlende Methoden.“
Gleicher Code in mehreren Implementierungen eines Interfaces	DRY	„Lege den gemeinsamen Code in eine abstrakte Basisklasse (Template Method).“
Ein `switch`, der nur Objekte erzeugt, an genau einer Stelle	–	In Ordnung. Das ist eine Factory.
Interface mit einer einzigen Implementierung, keine zweite in Sicht	YAGNI	„Wofür brauchen wir hier das Interface? Welche zweite Variante gibt es?“

L: Liskov Substitution

Der Webshop kennt zwei Zahlungsarten: Kreditkarte und Gutschein. Beide implementieren `IPaymentProvider`. Jetzt kommen Rücksendungen dazu, und das Interface bekommt eine Methode `Refund()`. Der Gutschein kann aber kein Geld zurückzahlen. Ein Agent löst das in Sekunden: Er wirft eine `NotSupportedException`. Der Code kompiliert, die Tests für die Kreditkarte sind grün. Bei der ersten Rücksendung mit Gutschein stürzt der Shop ab. In diesem Kapitel lernst du das **L** in SOLID kennen: das *Liskov Substitution Principle*. Es sagt, wann eine Implementierung wirklich zu ihrem Interface oder ihrer Basisklasse passt.

Projekt anlegen

Du brauchst eine Solution mit Application und Testprojekt. Die Befehle und alle Anpassungen der Projektdateien sind im Kapitel Unit Tests mit xUnit genau erklärt. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Im Testprojekt entfernst du zusätzlich `<Using Include="Xunit" />`.

Das Problem: Ein Gutschein kann nicht erstatten

Das Interface `IPaymentProvider` kennst du aus dem Kapitel Interfaces. Dort hat es nur `Pay()`. Für Rücksendungen kommt `Refund()` dazu. Der Kommentar beschreibt, was jede Implementierung verspricht.

IPaymentProvider.cs, **CreditCard.cs** und **GiftCard.cs** (im Projekt *WebshopDemo.Application*)

```
namespace WebshopDemo.Application;

/// <summary>
/// A payment method of the webshop, for example a credit card or a gift card.
/// </summary>
public interface IPaymentProvider
{
    /// <summary>Charges the amount. Returns false if the payment is not
possible.</summary>
    bool Pay(decimal amount);

    /// <summary>Pays back the amount after a return. Returns false if not
possible.</summary>
    bool Refund(decimal amount);
}
```

```
namespace WebshopDemo.Application;

public class CreditCard : IPaymentProvider
{
    public CreditCard(decimal limit)
    {
        Limit = limit;
    }

    public decimal Limit { get; }
    public decimal Charged { get; private set; }

    public bool Pay(decimal amount)
    {
        if (Charged + amount > Limit) { return false; }
        Charged += amount;
        return true;
    }

    public bool Refund(decimal amount)
    {
        if (amount > Charged) { return false; }
        Charged -= amount;
        return true;
    }
}
```

Unser Shop zahlt den Wert eines Gutscheins nie aus. Das ist eine Regel des Geschäfts. Deshalb wirft `GiftCard.Refund()` eine Exception:

```
using System;

namespace WebshopDemo.Application;

public class GiftCard : IPaymentProvider
```

```
{
    public GiftCard(decimal balance)
    {
        Balance = balance;
    }

    public decimal Balance { get; private set; }

    public bool Pay(decimal amount)
    {
        if (amount > Balance) { return false; }
        Balance -= amount;
        return true;
    }

    public bool Refund(decimal amount)
    {
        // The shop never pays out the value of a gift card.
        throw new NotSupportedException("Gift cards cannot be refunded.");
    }
}
```

Der `ReturnService` bearbeitet Rücksendungen. Er kennt nur das Interface:

ReturnService.cs

```
namespace WebshopDemo.Application;

public class ReturnService
{
    /// <summary>Pays back the price of a returned product.</summary>
    public bool ProcessReturn(IPaymentProvider paymentProvider, decimal amount)
    {
        return paymentProvider.Refund(amount);
    }
}
```

Die Tests zeigen das Problem:

ReturnServiceTests.cs (im Projekt *WebshopDemo.Tests*)

```
using System;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class ReturnServiceTests
{
    [Fact]
    public void ProcessReturn_WithCreditCard_RefundsAmount()
    {
```

```
        CreditCard creditCard = new CreditCard(limit: 500m);
        creditCard.Pay(80m);
        ReturnService returnService = new ReturnService();

        bool refunded = returnService.ProcessReturn(creditCard, 30m);

        Assert.True(refunded);
        Assert.Equal(50m, creditCard.Charged);
    }

    [Fact]
    public void ProcessReturn_WithGiftCard_ThrowsNotSupportedException()
    {
        GiftCard giftCard = new GiftCard(balance: 100m);
        giftCard.Pay(80m);
        ReturnService returnService = new ReturnService();

        Assert.Throws<NotSupportedException>(() => returnService.ProcessReturn(
giftCard, 30m));
    }
}
```

`ProcessReturn()` verlangt einen `IPaymentProvider`. Der Compiler ist zufrieden, denn `GiftCard` *ist* ein `IPaymentProvider`. Trotzdem stürzt das Programm ab. Das Interface verspricht „`Refund()` liefert `true` oder `false`“. Der Gutschein hält dieses Versprechen nicht.

Die schnelle Reparatur macht es schlimmer

Ein typischer Fix, den du oft in generiertem Code siehst:

```
public bool ProcessReturn(IPaymentProvider paymentProvider, decimal amount)
{
    if (paymentProvider is GiftCard)
    {
        return false;
    }
    return paymentProvider.Refund(amount);
}
```

Der Absturz ist weg, aber:

Problem	Folge
`ReturnService` kennt jetzt `GiftCard`.	Das Interface sollte genau das verhindern.
Jeder Aufrufer von `Refund()` braucht die Prüfung.	Eine Stornierung (`CancellationService`) stürzt weiter ab, wenn jemand die Prüfung vergisst.
Eine neue Zahlungsart ohne Erstattung kommt dazu.	Du musst alle `is`-Prüfungen finden und erweitern. Das verletzt auch Open/Closed.
Der Fehler zeigt sich erst zur Laufzeit.	Der Compiler kann dir nicht helfen.

Das Prinzip

Die Informatikerin Barbara Liskov stellte das Prinzip 1987 in ihrem Vortrag *Data Abstraction and Hierarchy* vor. Für ihre Arbeit bekam sie später den Turing Award. Verständlich formuliert:

> Ein Objekt einer abgeleiteten Klasse oder einer Implementierung muss **überall funktionieren**, wo der Basistyp erwartet wird, und zwar **ohne Überraschung** für den Aufrufer.

Substitution heißt „Ersetzen“: Du ersetzt den Basistyp durch einen Untertyp, und alles läuft weiter wie versprochen.

Wichtig: Es geht um **Verhalten**, nicht nur um Typen. Der Compiler prüft, ob `GiftCard` eine Methode `Refund(decimal)` mit Rückgabetyp `bool` hat. Er prüft nicht, ob sie tut, was das Interface verspricht. Dieses Versprechen heißt **Vertrag** (*Contract*). Er besteht aus drei Teilen:

Teil des Vertrags	Frage	Beispiel bei `Pay(amount)`
Vorbedingung	Was muss der Aufrufer vorher sicherstellen?	`amount` ist positiv.
Nachbedingung	Was gilt nach dem Aufruf?	Bei `true` wurde genau `amount` abgebucht.
Exceptions	Welche Fehler können kommen?	Keine. „Geht nicht“ heißt `false`.

Eine Implementierung darf die Vorbedingung nicht verschärfen und die Nachbedingung nicht abschwächen. Sie darf auch keine neuen Exceptions werfen, mit denen der Aufrufer nicht rechnet.



Das bekannteste Beispiel ist Rechteck und Quadrat. In der Mathematik *ist* jedes Quadrat ein Rechteck. Im Code macht `Square : Rectangle` aber Probleme: Wer beim Rechteck die Breite ändert, erwartet, dass die Höhe gleich bleibt. Beim Quadrat ändert sie sich mit. „Ist ein“ heißt also nicht automatisch „ist ersetzbar“.

Drei Warnzeichen

1. NotSupportedException oder leere Implementierung

Eine Methode wirft `NotSupportedException` oder `NotImplementedException`, weil sie „eigentlich nicht passt“. Das ist das Beispiel von oben. Noch gefährlicher ist die leere Variante:

```
public bool Refund(decimal amount)
{
    return true;   // does nothing
}
```

Der Aufrufer glaubt, die Kundin hat ihr Geld zurück. Niemand merkt den Fehler, bis sie sich beschwert.

2. Typprüfungen beim Aufrufer

`is GiftCard`, `as GiftCard`, `GetType() == typeof(...)` oder ein `switch` über den Typ in einer Methode, die eigentlich nur das Interface kennen sollte. Die Prüfung ist ein Zeichen dafür, dass sich eine Implementierung anders verhält, als der Basistyp verspricht. Der Aufrufer hat das bemerkt und baut eine Ausnahme ein.

3. Verschärfte Vorbedingung oder abgeschwächte Nachbedingung

Eine neue Zahlungsart „Kauf auf Rechnung“ erlaubt nur Beträge bis 100 €:

```
using System;

namespace WebshopDemo.Application;

public class InvoicePayment : IPaymentProvider
{
    public bool Pay(decimal amount)
    {
        if (amount > 100m)
        {
            throw new ArgumentException("Invoice payment is limited to 100 EUR.");
        }
        return true;
    }

    public bool Refund(decimal amount) => true;
}
```

Das Limit selbst ist in Ordnung, die Kreditkarte hat auch eines. Aber das Interface verspricht: „Geht nicht“ heißt `false`. `InvoicePayment` wirft stattdessen eine Exception. Die Vorbedingung ist verschärft: Der Aufrufer muss jetzt vorher wissen, welche Beträge erlaubt sind.

```
[Fact]
public void Pay_AmountAboveLimit_CreditCardReturnsFalseButInvoiceThrows()
{
    IPaymentProvider creditCard = new CreditCard(limit: 100m);
    IPaymentProvider invoice = new InvoicePayment();

    Assert.False(creditCard.Pay(150m));
    Assert.Throws<ArgumentException>(() => invoice.Pay(150m));
}
```

Beide Objekte haben denselben Typ `IPaymentProvider`. Trotzdem braucht der Aufrufer zwei verschiedene Arten, mit ihnen umzugehen. Die Korrektur ist hier einfach: `return false;` statt `throw`.

Die Nachbedingung ist abgeschwächt, wenn eine Implementierung weniger oder etwas anderes liefert als versprochen. Beispiel: `Pay(amount)` liefert `true`, bucht aber `amount + 0.50m` ab, weil eine Gebühr dazukommt. Der Kunde zahlt mehr, als die Rechnung zeigt.

Auch .NET verletzt Liskov

Ein Array implementiert in .NET das Interface `IList<T>`. Das Interface hat eine Methode `Add()`. Ein Array hat aber eine feste Länge. Was passiert?

DotNetCollectionTests.cs (im Projekt *WebshopDemo.Tests*)

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using Xunit;

namespace WebshopDemo.Tests;

public class DotNetCollectionTests
{
    [Fact]
    public void Add_OnArrayAsIList_ThrowsNotSupportedException()
    {
        IList<int> numbers = new int[] { 1, 2, 3 };

        Assert.True(numbers.IsReadOnly);
        Assert.Throws<NotSupportedException>(() => numbers.Add(4));
    }

    [Fact]
    public void Add_OnReadOnlyCollection_ThrowsNotSupportedException()
    {
        IList<int> numbers = new ReadOnlyCollection<int>(new List<int> { 1, 2, 3 });

        Assert.True(numbers.IsReadOnly);
        Assert.Throws<NotSupportedException>(() => numbers.Add(4));
    }
}
```

Das ist genau Warnzeichen 1. Damit der Aufrufer nicht abstürzt, gibt es die Property `IsReadOnly`. Wer `Add()` sicher aufrufen will, muss sie vorher prüfen. Das ist Warnzeichen 2 in anderer Form. Noch verwirrender: Beim Array ist `IsReadOnly` zwar `true`, aber `numbers[0] = 9` funktioniert trotzdem.

Warum ist das so? `IList<T>` gibt es seit .NET 2.0 (2005). Ein Interface nur zum Lesen kam erst mit .NET 4.5 (2012) dazu: `IReadOnlyList<T>`. Es hat gar keine Methode `Add()`. Ein falscher Aufruf kompiliert nicht:

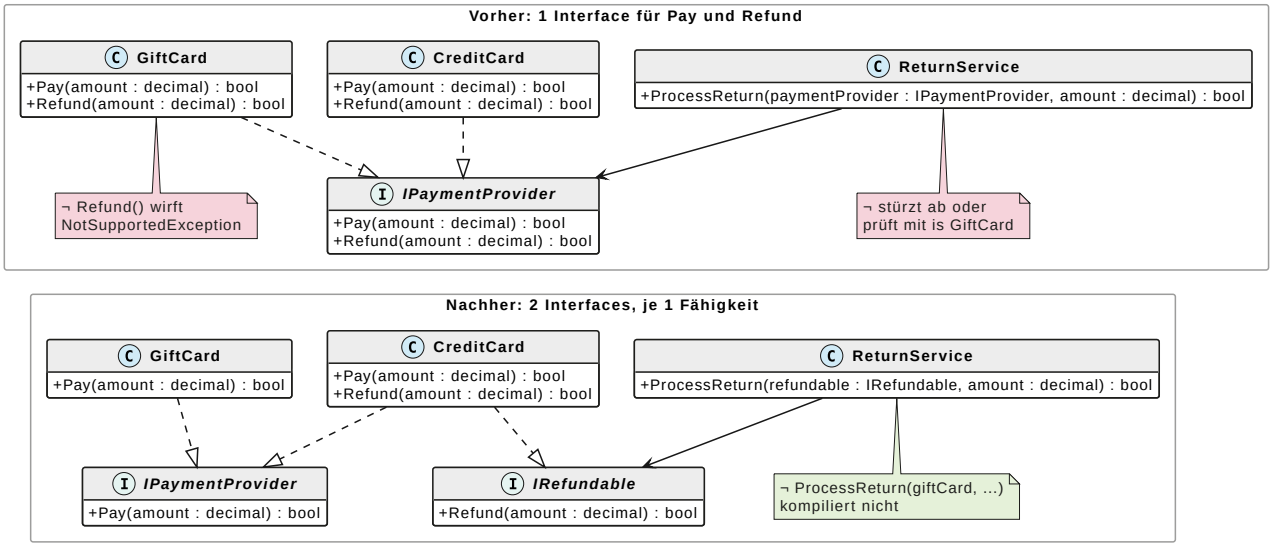
```
IReadOnlyList<int> numbers = new int[] { 1, 2, 3 };
numbers.Add(4);   // error CS1061: 'IReadOnlyList<int>' does not contain a definition
```

```
for 'Add'
```

Ein so verbreitetes Interface wie `IList<T>` kann Microsoft nicht mehr ändern, ohne Millionen Programme zu brechen. Es ist ein historischer Kompromiss. In deinem eigenen Code verwendest du deshalb `IReadOnlyList<T>`, wenn der Aufrufer nur lesen soll. Die Idee dahinter, kleine Interfaces für einzelne Fähigkeiten, ist das nächste Prinzip: I: Interface Segregation.

Refactoring: Fähigkeiten trennen

Nicht jede Zahlungsart kann erstatten. Also gehört `Refund()` nicht in den Vertrag für *alle* Zahlungsarten. Wir trennen die Fähigkeiten in zwei Interfaces:



PlantUML Quelle: 06_lsp_refactoring.puml

IPaymentProvider.cs und **IRefundable.cs**

```
namespace WebshopDemo.Application;

/// <summary>
/// A payment method of the webshop, for example a credit card or a gift card.
/// </summary>
public interface IPaymentProvider
{
    /// <summary>Charges the amount. Returns false if the payment is not
possible.</summary>
    bool Pay(decimal amount);
}
```

```
namespace WebshopDemo.Application;

/// <summary>
/// A payment method that can pay back money after a return.
/// </summary>
```

```
public interface IRefundable
{
    /// <summary>Pays back the amount. Returns false if not possible.</summary>
    bool Refund(decimal amount);
}
```

Die Klassen deklarieren nur noch, was sie wirklich können. Der Code in den Methoden bleibt gleich, `GiftCard` verliert die Methode `Refund()`:

```
public class CreditCard : IPaymentProvider, IRefundable
{
    /* Pay() and Refund() as before */
}

public class GiftCard : IPaymentProvider
{
    /* Pay() as before, no Refund() */
}
```

Der `ReturnService` verlangt jetzt genau die Fähigkeit, die er braucht:

ReturnService.cs

```
namespace WebshopDemo.Application;

public class ReturnService
{
    /// <summary>Pays back the price of a returned product.</summary>
    public bool ProcessReturn(IRefundable refundable, decimal amount)
    {
        return refundable.Refund(amount);
    }
}
```

Der Test für die Kreditkarte bleibt unverändert grün. Den Test mit dem Gutschein kannst du nicht mehr schreiben:

```
GiftCard giftCard = new GiftCard(balance: 100m);
returnService.ProcessReturn(giftCard, 30m);
// error CS1503: Argument 1: cannot convert from
// 'WebshopDemo.Application.GiftCard' to 'WebshopDemo.Application.IRefundable'
```

Der Fehler zeigt sich jetzt beim Kompilieren, nicht bei der Kundin. Es gibt keine `NotSupportedException` und keine `is`-Prüfung mehr.

Wo fällt die Entscheidung?

Irgendwo muss das Programm trotzdem wissen, was bei einer Rücksendung mit Gutschein pas-

siert. Das ist eine fachliche Frage, keine technische. Es gibt zwei saubere Wege:

Weg	Idee	Passt, wenn …
Fähigkeiten trennen	`IRefundable` nur dort, wo es Erstattungen gibt. Gutscheine laufen über einen anderen Ablauf.	nur ein Teil der Implementierungen die Fähigkeit hat.
Vertrag allgemeiner fassen	Statt `Refund()` heißt die Methode `Compensate()`. Die Kreditkarte bucht zurück, der Gutschein bekommt den Betrag wieder gutgeschrieben.	jede Implementierung den Vertrag sinnvoll erfüllen kann.

Welcher Weg richtig ist, entscheidet das Geschäft, nicht das Prinzip. Prüfst du an einer Stelle doch den Typ, dann auf die Fähigkeit (`is IRefundable`), nicht auf die Klasse (`is GiftCard`). Eine neue Zahlungsart muss dann nur das Interface implementieren, der Aufrufer bleibt gleich.



Bevor du eine Methode überschreibst oder ein Interface implementierst, frag dich: Kann ich den Kommentar des Basistyps für meine Klasse unverändert unterschreiben? Wenn nicht, passt die Vererbung nicht.

Im Code-Review erkennen

Warnzeichen im Code	Verletztes Prinzip	Was du verlangst bzw. wie du es formulierst
`throw new NotSupportedException()` in einem Override oder einer Interface-Methode	Liskov	„`GiftCard` erfüllt den Vertrag von `IPaymentProvider` nicht. Trenne `Refund()` in ein eigenes Interface `IRefundable`.“
Leere Implementierung, die nur `return true;` oder nichts macht	Liskov	„`Refund()` meldet Erfolg, tut aber nichts. Entweder erfüllt die Klasse den Vertrag, oder sie implementiert das Interface nicht.“
`if (x is GiftCard)`, `as`, `GetType()` oder `switch` über den Typ beim Aufrufer	Liskov (oft auch Open/Closed)	„`ReturnService` darf keine konkreten Zahlungsarten kennen. Der Parameter soll genau die Fähigkeit verlangen, die gebraucht wird.“
Override wirft bei Werten, die der Basistyp erlaubt (`amount > 100`)	Liskov: Vorbedingung verschärft	„`InvoicePayment.Pay()` wirft, wo der Vertrag `false` verspricht. Gib `false` zurück.“

Warnzeichen im Code	Verletztes Prinzip	Was du verlangst bzw. wie du es formulierst
Override liefert mehr, weniger oder etwas anderes als dokumentiert	Liskov: Nachbedingung abgeschwächt	„`Pay()` bucht eine Gebühr dazu. Der Vertrag sagt ‚genau `amount`'. Weise die Gebühr als eigene Position aus."
Basisklasse bekommt Dummy-Werte im Konstruktor (`base(name, 0, 0)`)	Liskov	„Die Unterklasse braucht diese Daten nicht. Die Daten gehören in eine andere Unterklasse, nicht in die Basis."
Parameter vom Typ `IList<T>`, obwohl die Methode nur liest	Liskov / Interface Segregation	„Verwende `IReadOnlyList<T>`. Dann kann der Aufrufer auch ein Array oder eine `ReadOnlyCollection` übergeben, ohne Überraschung."

I: Interface Segregation

Du öffnest im Webshop die Klasse *GiftCard*. Sie hat sechs Methoden. Vier davon bestehen nur aus `throw new NotSupportedException()`. Und der *CheckoutService*, der nur bezahlen will, hängt trotzdem von allen sechs ab. Das **I** in SOLID, das *Interface Segregation Principle* (ISP), beschreibt genau dieses Problem. *Segregation* heißt Trennung.

Ein Grundgedanke dieses Kapitels: Du schreibst selten ein Framework, sondern meist eine **Business App**, also eine Anwendung für die Abläufe einer Firma. Gut geschnittene Interfaces **benutzt** du trotzdem jeden Tag: bei `foreach`, `using`, `List<T>.Sort()` oder `ILogger<T>`. Du lernst hier, solche Interfaces zu lesen, richtig zu nutzen und schlechte Schnitte im Code Review zu erkennen. Die Grundlagen stehen im Abschnitt Das Interface Segregation Principle. Dieses Kapitel vertieft sie.



Zum Selbst-Erarbeiten: Interfaces nach Rollen schneiden

In der interaktiven Präsentation zum Interface Segregation Principle siehst du Schritt für Schritt, wie ein fettes Interface in Rollen zerfällt und wie die Collection-Interfaces in .NET aufeinander aufbauen. Danach wählst du in mehreren Runden das kleinste passende Interface für einen Parameter. Genau diese Entscheidung triffst du in jedem Code Review.

Das Problem: ein Interface für alles

Ein Agent hat für die Zahlungsarten im Webshop dieses Interface geschrieben:

```
namespace WebshopDemo.Application;

public interface IPaymentProvider
{
    bool Pay(decimal amount);
    bool Refund(decimal amount);
    string Authorize(decimal amount);            // reserves the amount, returns an id
    bool Capture(string authorizationId);        // charges the reserved amount
    IReadOnlyList<Transaction> GetStatement(DateOnly from, DateOnly to);
    string ExportForAccounting(int year, int month);
}
```

Ein Interface mit vielen Methoden für verschiedene Zwecke heißt **fettes Interface** (*fat interface*). Jede Zahlungsart muss alle sechs Methoden implementieren, auch wenn sie nicht passen:

```
public class GiftCard : IPaymentProvider
{
    // Pay() and GetStatement() work as expected.

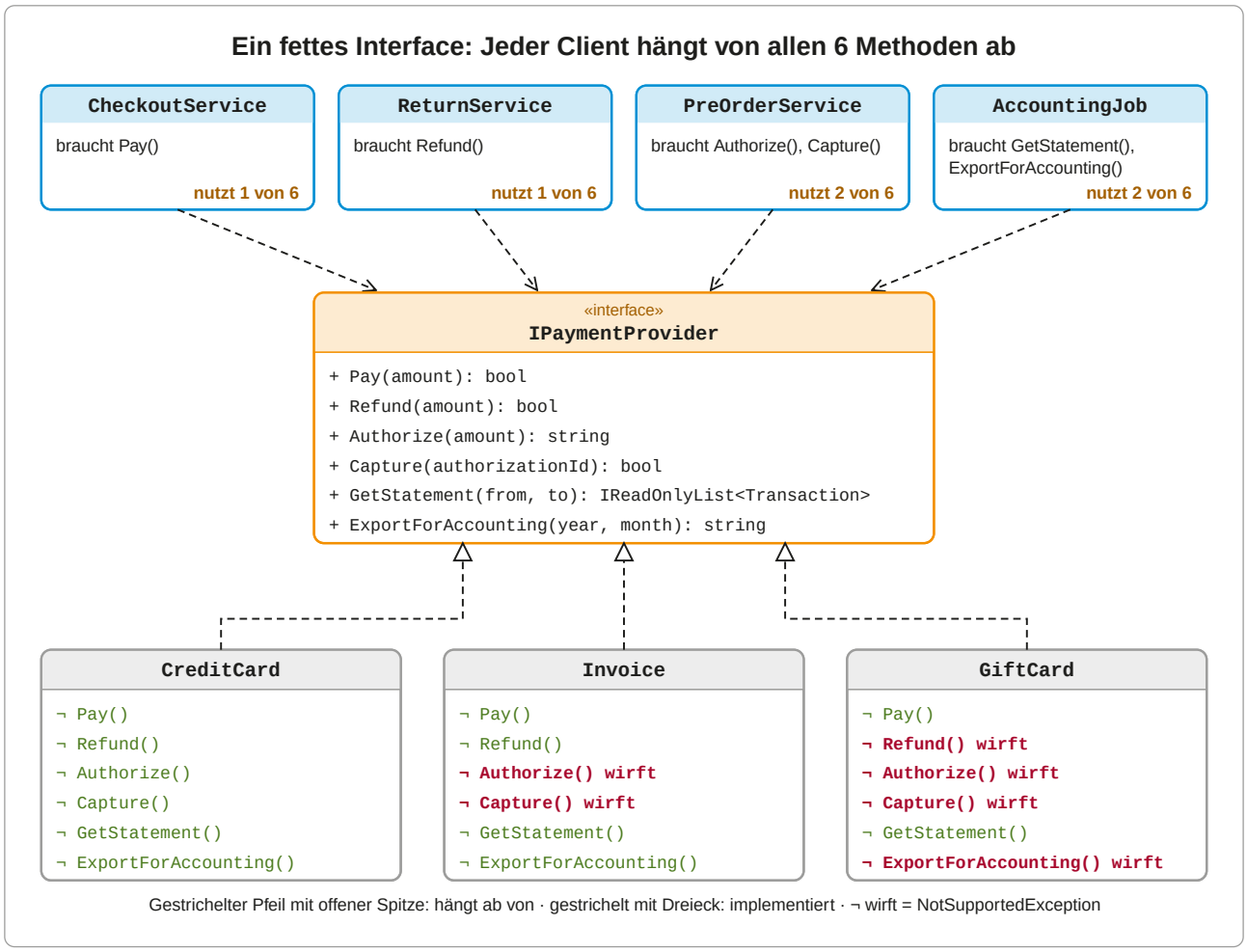
    public bool Refund(decimal amount) =>
        throw new NotSupportedException("A gift card cannot be refunded.");
```

```
    public string Authorize(decimal amount) =>
        throw new NotSupportedException("A gift card cannot authorize payments.");

    public bool Capture(string authorizationId) =>
        throw new NotSupportedException("A gift card cannot capture payments.");

    public string ExportForAccounting(int year, int month) =>
        throw new NotSupportedException("Gift cards are booked when they are sold.");
}
```

Sieh dir an, wer das Interface verwendet. Eine Klasse, die ein Interface benutzt, heißt **Client** des Interfaces.



Was kostet das fette Interface?

Wer	Problem
GiftCard, *Invoice*	Sie implementieren Methoden, die sie nicht können. Ein Aufruf stürzt zur Laufzeit ab.
CheckoutService	Er hängt von sechs Methoden ab und braucht eine. Ändert sich `ExportForAccounting()`, muss er neu kompiliert werden.

Wer	Problem
Unit Test	Ein Fake für den *CheckoutService* braucht sechs Methoden. Fünf davon sind leer oder werfen.
Leserin im Code Review	Sie sieht am Konstruktor nicht, was der *CheckoutService* wirklich braucht.
Neue Zahlungsart	Wer z. B. Kauf auf Raten ergänzt, muss sechs Methoden schreiben, bevor er bezahlen kann.

Das Prinzip

Robert C. Martin hat das Prinzip so formuliert:

> „Clients should not be forced to depend on methods they do not use.“
>
> Kein Client soll von Methoden abhängen müssen, die er nicht verwendet.

Abhängen heißt hier: Der Client braucht den Typ, um zu kompilieren (siehe Was sind Dependencies?). Martin entdeckte das Problem in den 1990er Jahren bei Xerox. Dort hatte eine einzige Klasse für Druckaufträge Methoden für alle Aufgaben. Jede kleine Änderung führte zu langen Builds im ganzen System.

Wichtig ist die **Sicht des Clients**. Du fragst nicht: „Was kann eine Kreditkarte alles?“ Du fragst: „Was braucht der *CheckoutService*?“ Ein Interface, das genau die Bedürfnisse einer Gruppe von Clients beschreibt, heißt **Role Interface** (Rollen-Interface). Das Gegenteil ist ein **Header Interface**: Es kopiert einfach alle öffentlichen Methoden einer Klasse.

Interfaces nach Rollen schneiden

Schreibe zu jedem Client auf, was er braucht. Jede Zeile wird eine Rolle:

Client	braucht	Rolle (Interface)
CheckoutService	`Pay()`	*IPaymentProvider*
ReturnService	`Refund()`	*IRefundable*
PreOrderService	`Authorize()`, `Capture()`	*IAuthorizable*
AccountingExporter	`GetStatement()`	*IStatementSource*

```
public interface IPaymentProvider
{
    bool Pay(decimal amount);
}

public interface IRefundable
{
    bool Refund(decimal amount);
}

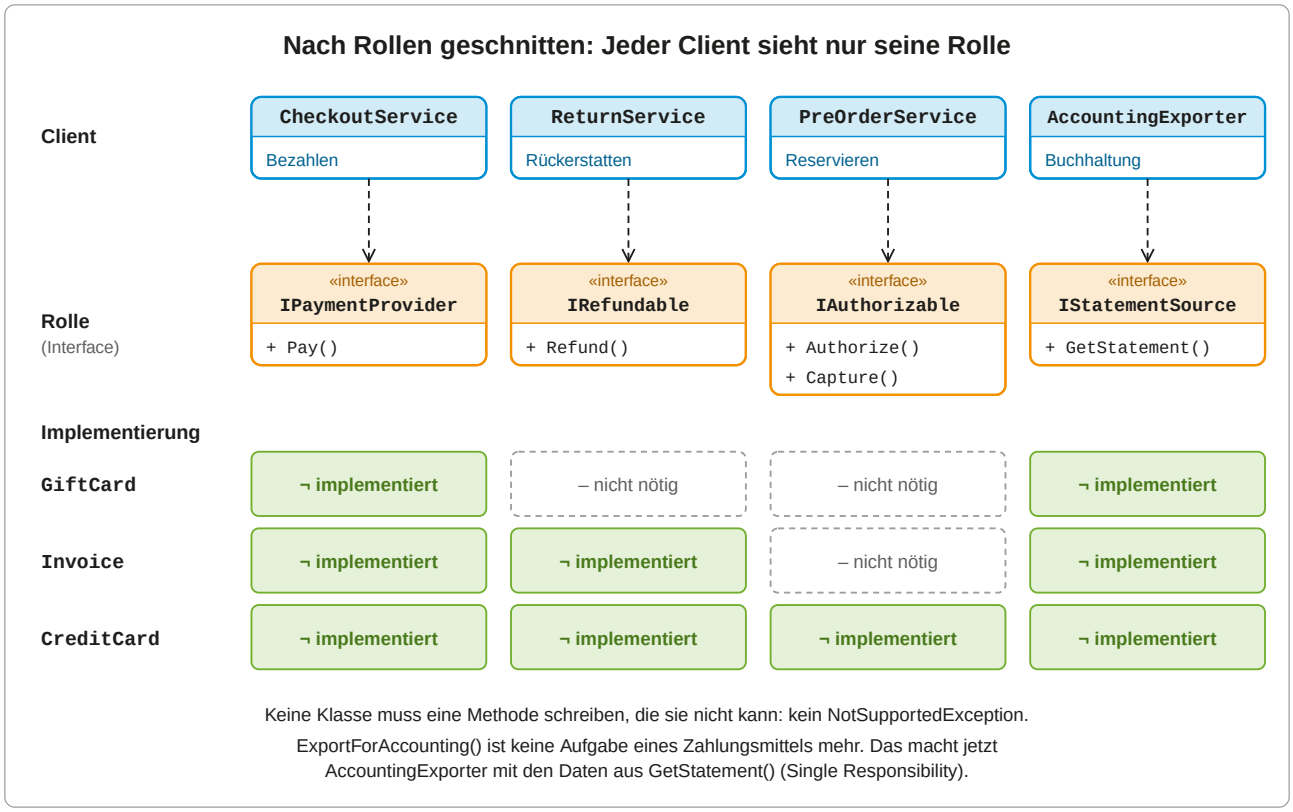
public interface IAuthorizable
```

```
{
    string Authorize(decimal amount);
    bool Capture(string authorizationId);
}

public interface IStatementSource
{
    IReadOnlyList<Transaction> GetStatement(DateOnly from, DateOnly to);
}
```

Jede Klasse implementiert nur die Rollen, die sie wirklich erfüllt:

```
public class GiftCard : IPaymentProvider, IStatementSource { /* ... */ }
public class CreditCard : IPaymentProvider, IRefundable, IAuthorizable,
IStatementSource { /* ... */ }
```



Drei Beobachtungen:

- **Eine Rolle kann mehrere Methoden haben.** `Authorize()` ohne `Capture()` ist nutzlos. Was ein Client immer gemeinsam braucht, bleibt in einem Interface. Ein Interface pro Methode ist nicht das Ziel.
- **Beim Schneiden findest du falsche Zuständigkeiten.** `ExportForAccounting()` erzeugt eine Datei für die Buchhaltung. Das ist keine Aufgabe eines Zahlungsmittels. Die Methode wandert in die Klasse *AccountingExporter*. Sie holt sich die Daten über *IStatementSource* (siehe S: Single Responsibility).
- **Der Fake im Test wird winzig.** Für den *CheckoutService* genügt eine Zeile:

```
public class AlwaysPayingProvider : IPaymentProvider
{
    public bool Pay(decimal amount) => true;
}
```

Braucht ein Client zwei Rollen, bekommt er zwei Parameter im Konstruktor. Brauchen viele Clients immer dieselben Rollen zusammen, kannst du sie kombinieren: `interface IFullPaymentProvider : IPaymentProvider, IRefundable { }`. Ein Interface darf also von mehreren Interfaces erben.

ISP und Liskov

Im Kapitel L: Liskov Substitution hast du *IRefundable* schon abgetrennt. Beide Prinzipien führen oft zum selben Umbau. Sie schauen aber aus verschiedenen Richtungen:

Prinzip	Blickrichtung	Frage
L	von der Implementierung	Hält *GiftCard* alles, was *IPaymentProvider* verspricht?
I	vom Client	Muss der *CheckoutService* Methoden kennen, die er nicht nutzt?

Ein fettes Interface verleitet zu Liskov-Verletzungen. Eine `NotSupportedException` ist deshalb fast immer ein Hinweis auf beide Prinzipien.

Das große Beispiel: Collections in .NET

Die Rechnung im Webshop summiert die Positionen einer Bestellung:

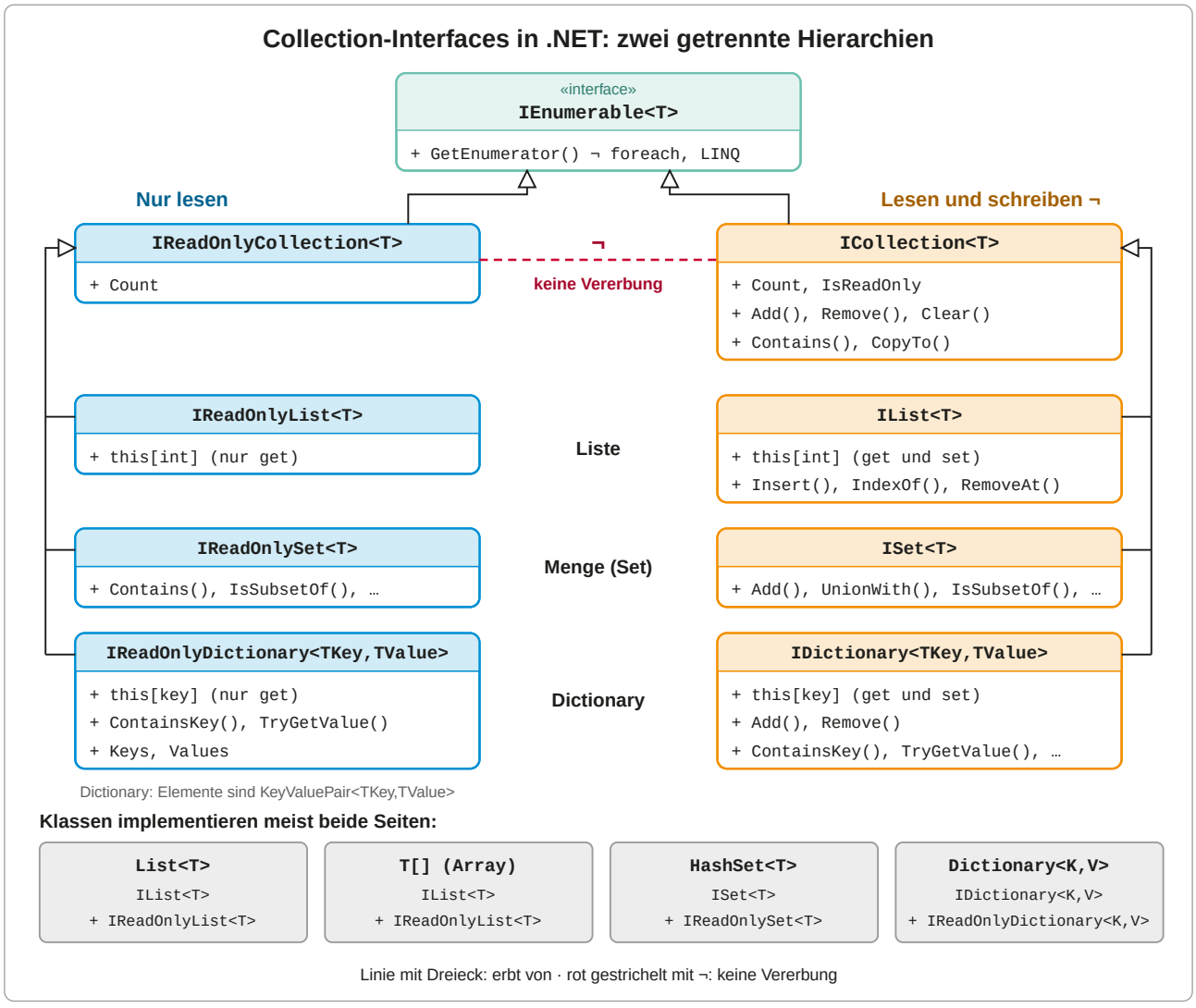
```
public static decimal Total(List<OrderItem> items)
{
    decimal total = 0m;
    foreach (OrderItem item in items) { total += item.Price * item.Quantity; }
    return total;
}
```

Die Methode kompiliert, aber beim Aufruf gibt es Probleme:

```
OrderItem[] fromJson = LoadItems();
Total(fromJson);                // CS1503: cannot convert from 'OrderItem[]' to
'List<OrderItem>'
Total(order.Items);             // CS1503: IReadOnlyList<OrderItem> is not a
List<OrderItem>
Total(order.Items.ToList());    // compiles, but copies every element first
```

`Total()` verwendet nur `foreach`. Trotzdem verlangt es eine `List<T>` mit `Add()`, `Remove()`, `Sort()`

und vielem mehr. Mit `IEnumerable<OrderItem> items` funktionieren alle drei Aufrufe ohne Kopie. Das Framework bietet dafür eine ganze Familie kleiner Interfaces an. Sie ist das beste Beispiel für ISP, das du täglich verwendest.



Zwei Hierarchien nebeneinander

- **Links (nur lesen):** Jede Ebene fügt genau eine Fähigkeit hinzu. `IReadOnlyCollection<T>` kennt die Anzahl, `IReadOnlyList<T>` zusätzlich den Index. `IReadOnlySet<T>` und `IReadOnlyDictionary<TKey, TValue>` sind die Varianten für Mengen und Schlüssel-Wert-Paare.
- **Rechts (lesen und schreiben):** `ICollection<T>` bringt gleich *Count*, *Add*, *Remove*, *Clear* und *Contains* mit. `IList<T>`, `ISet<T>` und `IDictionary<TKey, TValue>` bauen darauf auf.
- **Kein Pfeil dazwischen:** `ICollection<T>` erbt **nicht** von `IReadOnlyCollection<T>`, und `IList<T>` erbt nicht von `IReadOnlyList<T>`.

Der Grund ist historisch. `IList<T>` gibt es seit .NET 2.0 (2005). Die *IReadOnly*-Interfaces kamen erst mit .NET Framework 4.5 (2012). Eine neue Basis für ein bestehendes Interface hätte vorhandenen Code gebrochen. Für dich hat das eine Folge:

```
IList<int> numbers = new List<int> { 1, 2, 3 };
```

```
PrintAll(numbers);   // CS1503: cannot convert from 'IList<int>' to
'IReadOnlyList<int>'

static void PrintAll(IReadOnlyList<int> values) { /* ... */ }
```

Eine `List<int>` oder ein `int[]` kannst du übergeben, denn die Klassen implementieren beide Seiten. Eine Variable vom Typ `IList<int>` aber nicht. Verwende deshalb in deinem Code die *IReadOnly*-Interfaces, wenn du nur liest.

Welche Klasse implementiert was?

Interface	`List<T>`	`T[]`	`HashSet<T>`	`Dictionary<TKey, TValue>`
`IEnumerable<T>`	✔	✔	✔	✔ (Elemente: *KeyValuePair*)
`IReadOnlyCollection<T>`	✔	✔	✔	✔ (Elemente: *KeyValuePair*)
`IReadOnlyList<T>`	✔	✔	✗	✗
`IReadOnlySet<T>`	✗	✗	✔	✗
`IReadOnlyDictionary<TKey, TValue>`	✗	✗	✗	✔
`ICollection<T>`	✔	✔ (wirft bei *Add*)	✔	✔ (Elemente: *KeyValuePair*)
`IList<T>`	✔	✔ (wirft bei *Add*)	✗	✗
`ISet<T>`	✗	✗	✔	✗
`IDictionary<TKey, TValue>`	✗	✗	✗	✔

Der historische Bruch: `int[]` ist eine `IList<T>`

Ein Array hat eine feste Länge. Trotzdem implementiert es `IList<T>` mit `Add()` und `Remove()`:

```
IList<int> numbers = new int[] { 1, 2, 3 };
Console.WriteLine(numbers.IsReadOnly);   // True
numbers[0] = 10;                         // works, although IsReadOnly is true
numbers.Add(4);                          // NotSupportedException: Collection was of a
fixed size.
```

Das ist genau das Problem von *GiftCard*: Eine Klasse implementiert ein Interface, dessen Versprechen sie nicht halten kann. Das verletzt Liskov. Die Property *IsReadOnly* ist ein Notausgang. Der Client soll zur Laufzeit fragen, was der Compiler mit kleineren Interfaces schon beim Kompilieren prüfen könnte. `ReadOnlyCollection<T>` hat dasselbe Problem: Auch sie implementiert `IList<T>` und wirft bei jedem Schreibzugriff.



Die *IReadOnly*-Interfaces sind die nachträgliche Reparatur. Sie trennen Lesen und Schreiben, wie es ISP verlangt. Die alten Interfaces bleiben aus Kompatibi-

> litätsgründen.

Ein-Methoden-Interfaces: eine Fähigkeit, ein Interface

Du schreibst `using`, `foreach` und `list.Sort()`, ohne an Interfaces zu denken. Dahinter steckt aber jedes Mal ein winziges Interface mit einer einzigen Methode:

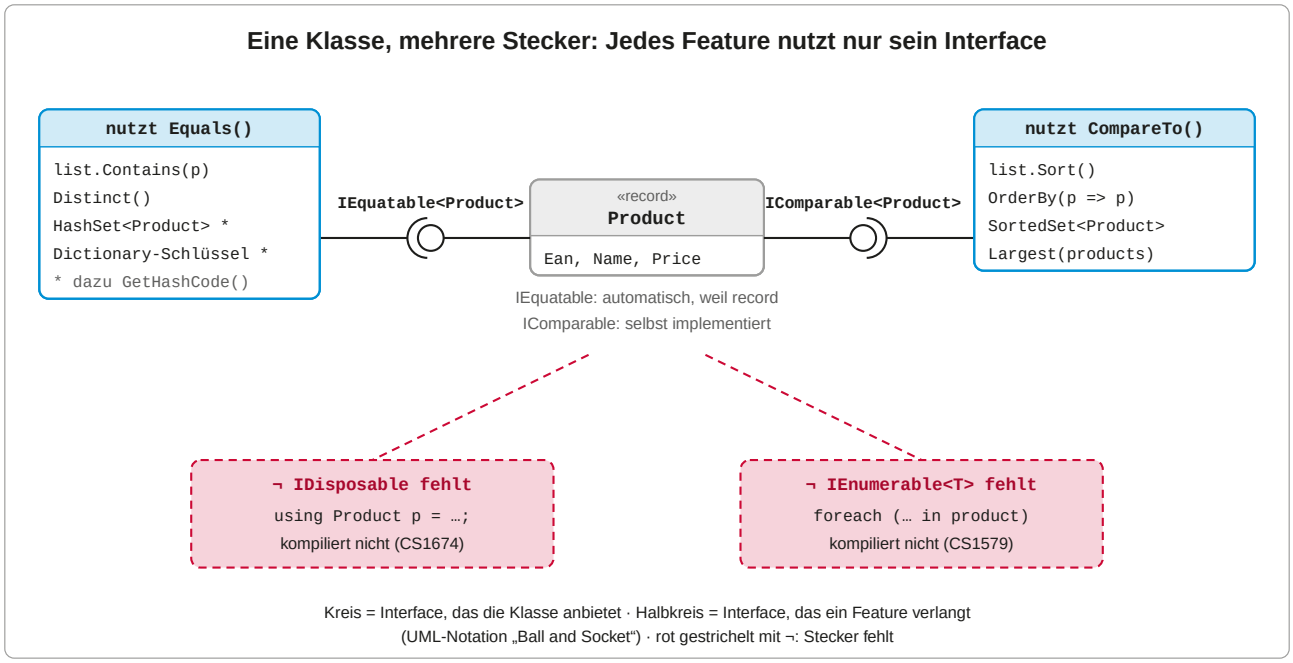
Interface	Methode	Wer fragt danach
`IDisposable`	`Dispose()`	die `using`-Anweisung: gibt Dateien, Verbindungen usw. frei
`IAsyncDisposable`	`DisposeAsync()`	`await using`
`IEnumerable<T>`	`GetEnumerator()`	`foreach` und alle LINQ-Methoden
`IComparable<T>`	`CompareTo(other)`	`List<T>.Sort()`, `Array.Sort()`, `OrderBy()`, `SortedSet<T>`
`IEquatable<T>`	`Equals(other)`	`Contains()`, `Distinct()`, `HashSet<T>`, Schlüssel im Dictionary

Mehr zu `IEquatable<T>` und `IComparable<T>` steht im Kapitel Equality. Genau genommen sucht `foreach` nur eine Methode `GetEnumerator()`. Fast immer kommt sie über `IEnumerable<T>`.

Jedes Feature fragt nur nach **seiner** Fähigkeit. `List<T>.Sort()` interessiert sich nicht dafür, ob ein Produkt `Dispose()` kann. `using` interessiert sich nicht für `CompareTo()`. Eine Klasse bietet so viele Fähigkeiten an, wie sie braucht:

```
public record Product(string Ean, string Name, decimal Price) : IComparable<Product>
{
    public int CompareTo(Product? other) => other is null ? 1 : Price.CompareTo(other
.Price);
}
```

Ein *record* implementiert `IEquatable<Product>` automatisch. `IComparable<Product>` ergänzt du selbst. Damit funktionieren `products.Sort()` und `new HashSet<Product>()`. `using` und `foreach` mit einem Produkt kompilieren dagegen nicht.



So fragt eine Methode nach genau einer Fähigkeit

Die Methode `Largest()` liefert das größte Element einer Sequenz. Sie braucht zwei Fähigkeiten: die Elemente durchlaufen und sie vergleichen.

```
public static T Largest<T>(IEnumerable<T> items) where T : IComparable<T>
{
    using IEnumerator<T> enumerator = items.GetEnumerator();
    if (!enumerator.MoveNext()) { throw new InvalidOperationException("The sequence is
empty."); }
    T largest = enumerator.Current;
    while (enumerator.MoveNext())
    {
        if (enumerator.Current.CompareTo(largest) > 0) { largest = enumerator.Current;
}
    }
    return largest;
}
```

- `IEnumerable<T> items`: Jede Collection passt, auch eine LINQ-Abfrage.
- `where T : IComparable<T>` ist eine **Constraint** (Einschränkung): `T` muss `IComparable<T>` implementieren. Sonst kompiliert der Aufruf nicht.
- `using IEnumerator<T>`: Auch der Enumerator ist `IDisposable`. In fünf Zeilen stecken also drei Ein-Methoden-Interfaces.

Vergleiche mit `List<T>.Sort()`. Diese Methode hat keine Constraint, weil eine Liste beliebige Typen enthalten darf. Fehlt `IComparable<T>`, merkst du es erst zur Laufzeit:

```
List<Customer> customers = [new Customer("A"), new Customer("B")];
customers.Sort();           // InvalidOperationException: Failed to compare two
elements in the array.
```

```
Largest(customers);         // CS0311: Customer cannot be used as type parameter 'T'
... IComparable<Customer>
```

Ein kleines Interface in der Signatur macht die Anforderung sichtbar. Der Compiler prüft sie für dich.

Ausblick: ASP.NET Core und EF Core

In den Kapiteln zu ASP.NET Core und EF Core triffst du viele Interfaces. Sie sind fast alle nach Rollen geschnitten. Du implementierst sie selten selbst. Du **verlangst** sie im Konstruktor und nutzt sie:

Interface	Members	Rolle
`ILogger<T>`	`Log()`, `IsEnabled()`, `BeginScope()`	Meldungen protokollieren. `LogInformation()` usw. sind Extension Methods darauf.
`IOptions<T>`	nur `Value`	Einstellungen einmal lesen
`IOptionsMonitor<T>`	`CurrentValue`, `Get()`, `OnChange()`	Einstellungen lesen und auf Änderungen reagieren
`IHostedService`	`StartAsync()`, `StopAsync()`	Hintergrunddienst, der mit der App startet und stoppt
`IServiceProvider`	nur `GetService()`	Objekte aus dem DI-Container holen
`IQueryable<T>`	(erbt von `IEnumerable<T>`)	Abfragen, die EF Core in SQL übersetzt. `DbSet<T>` implementiert es.

Bei den Optionen siehst du ISP besonders deutlich. Wer Einstellungen nur einmal liest, verlangt `IOptions<T>` mit einem einzigen Property. Wer auf Änderungen reagieren muss, verlangt `IOptionsMonitor<T>`. Jeder Client wählt die Rolle, die er braucht:

```
public class ShippingCalculator
{
    private readonly ILogger<ShippingCalculator> _logger;
    private readonly ShopSettings _settings;

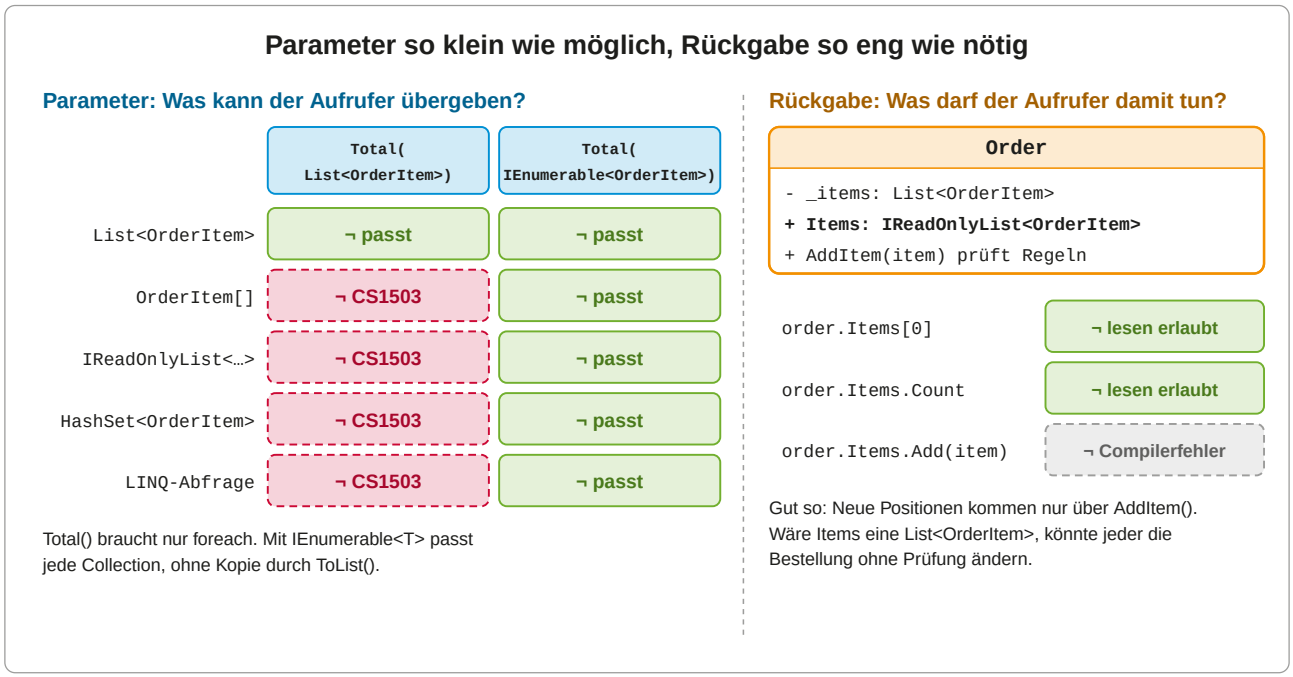
    public ShippingCalculator(ILogger<ShippingCalculator> logger, IOptions<
ShopSettings> options)
    {
        _logger = logger;
        _settings = options.Value;
    }

    public decimal GetShippingCost(decimal total)
    {
        _logger.LogInformation("Calculating shipping for {Total}", total);
        return total >= _settings.FreeShippingLimit ? 0m : _settings.ShippingCost;
```

```
    }
}
```

Wie die Objekte in den Konstruktor kommen, zeigt das Kapitel D: Dependency Inversion.

Was heißt das für dich als Business-App-Entwickler?



Regel	Bedeutung	Beispiel
Parameter so klein wie möglich	Verlange nur die Fähigkeit, die die Methode nutzt. Dann passt jede Collection ohne Kopie.	`Total(IEnumerable<OrderItem> items)`
Bei mehrfachem Durchlaufen größer	Eine LINQ-Abfrage läuft bei jedem `foreach` neu. Brauchst du *Count* oder mehrere Durchläufe, verlange `IReadOnlyCollection<T>` oder `IReadOnlyList<T>`.	`Average(IReadOnlyCollection<decimal> prices)`
Rückgabe so eng wie nötig	Gib eine interne Liste nie als `List<T>` heraus. Der Aufrufer könnte sie ohne Prüfung ändern.	`public IReadOnlyList<OrderItem> Items => _items;`
Rückgabe nicht zu arm	Liefert eine Methode eine fertige Liste, gib `IReadOnlyList<T>` zurück und nicht nur `IEnumerable<T>`. Sonst muss der Aufrufer für *Count* alles durchlaufen.	`IReadOnlyList<Product> FindByCategory(...)`

Regel	Bedeutung	Beispiel
Eigene Interfaces nach Rollen	Schneide nach dem, was ein Client braucht, nicht nach dem, was eine Klasse kann.	*IRefundable* statt *IPaymentProvider* mit allem
Nicht für jede Klasse ein Interface	Ein Interface lohnt sich bei mehreren Implementierungen oder einem Fake im Test.	siehe Warnung in Interfaces



Agents erzeugen gerne **Header Interfaces**: zu *OrderService* ein *IOrderService* mit allen 15 öffentlichen Methoden. Das sieht nach sauberer Architektur aus. Jeder Client hängt aber wieder von allem ab. Frag nach: Welche Clients gibt es, und was braucht jeder davon?

Im Code-Review erkennen

Warnzeichen	Prinzip	Was du verlangst bzw. wie du es präzise formulierst
Parameter `List<T>`, die Methode macht nur `foreach`	I	„`Total()` liest nur. Verlange `IEnumerable<OrderItem>` statt `List<OrderItem>`, dann brauchen Aufrufer kein `ToList()`.“
Aufrufer schreiben `.ToList()` oder `.ToArray()` nur für einen Methodenaufruf	I	„Der Parameter ist zu groß. Ändere ihn auf das kleinste Interface, das die Methode braucht.“
Parameter `IList<T>` oder `ICollection<T>`, aber die Methode ändert nichts	I	„Die Methode schreibt nicht. Verwende `IReadOnlyList<T>` bzw. `IReadOnlyCollection<T>`.“
Property oder Methode gibt eine interne `List<T>` zurück	I, Kapselung	„Gib `_items` als `IReadOnlyList<OrderItem>` heraus. Änderungen laufen nur über `AddItem()`.“
`NotImplementedException` oder `NotSupportedException` in einer Interface-Implementierung	I, L	„`GiftCard` muss Methoden implementieren, die sie nicht kann. Trenne das Interface nach Rollen, z. B. `IRefundable`.“
Interface mit 10 oder mehr Methoden für verschiedene Zwecke	I	„`IPaymentProvider` ist ein fettes Interface. Schneide es nach den Clients: Was braucht `CheckoutService`, was `ReturnService`?“
Fake im Unit Test hat viele leere oder werfende Methoden	I	„Der Test zeigt, dass der Client nur `Pay()` nutzt. Er soll nur ein Interface mit `Pay()` verlangen.“

Warnzeichen	Prinzip	Was du verlangst bzw. wie du es präzise formulierst
`IXxxService` mit genau allen öffentlichen Methoden von `XxxService`	I	„Das ist ein Header Interface. Leite die Interfaces aus den Clients ab oder verzichte darauf, wenn es nur eine Implementierung gibt.“
`if (provider.IsReadOnly)` oder `if (x is IRefundable)` vor einem Aufruf	I, L	„Der Client prüft zur Laufzeit eine Fähigkeit. Verlange die passende Rolle im Typ, dann prüft der Compiler.“

Zusatzmaterial

In der 07_Interface_Segregation.html zerlegst du ein fettes Interface Schritt für Schritt in Rollen, baust die Collection-Hierarchie auf und übst in mehreren Runden, welches Interface als Parameter reicht und welche Klasse du übergeben kannst.

D: Dependency Inversion

Ein Agent schreibt dir einen *CheckoutService* für den Webshop. Die Tests sind grün. Mitten in `Checkout()` steht aber `new ConsoleLogger()`. Jeder Test schreibt jetzt in die Konsole, und ob der Service etwas protokolliert, kannst du nicht prüfen. In diesem Kapitel lernst du das **D** in SOLID kennen, das *Dependency Inversion Principle*, und die Technik dazu: *Dependency Injection*.

Projekt anlegen

Du brauchst eine Solution mit Application und Testprojekt. Die Befehle und alle Anpassungen der Projektdateien sind im Kapitel Unit Tests mit xUnit genau erklärt. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q WebshopDemo
md WebshopDemo
cd WebshopDemo
md WebshopDemo.Application
cd WebshopDemo.Application
dotnet new classlib
cd ..
md WebshopDemo.Tests
cd WebshopDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\WebshopDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add WebshopDemo.Application
dotnet sln add WebshopDemo.Tests
start WebshopDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings* und füge `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Im Testprojekt entfernst du zusätzlich `<Using Include="Xunit" />`.

Ausgangslage

Wir setzen das Webshop-Beispiel aus dem Kapitel Interfaces fort. *Order* speichert nur Daten. Ein **Service** wie *CheckoutService* führt einen Ablauf aus, hier „Bestellung bezahlen“. Alle Klassen sind `public`, denn das Testprojekt muss sie sehen.

Order.cs, **IPaymentProvider.cs** und **CreditCard.cs** (im Projekt *WebshopDemo.Application*)

```
namespace WebshopDemo.Application;

public class Order
{
```

```
    public Order(string customerEmail, decimal totalAmount)
    {
        CustomerEmail = customerEmail;
        TotalAmount = totalAmount;
    }

    public string CustomerEmail { get; }
    public decimal TotalAmount { get; }
    public bool IsPaid { get; private set; }

    public void MarkAsPaid() => IsPaid = true;
}
```

```
namespace WebshopDemo.Application;

public interface IPaymentProvider
{
    bool Pay(decimal amount);
}
```

```
using System;

namespace WebshopDemo.Application;

public class CreditCard : IPaymentProvider
{
    public CreditCard(decimal limit, DateTime expiration)
    {
        Limit = limit;
        Expiration = expiration;
    }

    public decimal Limit { get; }
    public DateTime Expiration { get; }

    public bool Pay(decimal amount)
    {
        if (amount > Limit) { return false; }
        if (Expiration < DateTime.UtcNow) { return false; }
        return true;
    }
}
```

Das Problem: Eine Klasse erzeugt ihre Abhängigkeiten selbst

Der Service soll protokollieren, ob eine Zahlung geklappt hat. Den Payment Provider bekommt er schon von außen. Den Logger erzeugt er aber selbst, direkt in der Methode:

```
using System;

namespace WebshopDemo.Application;

public class ConsoleLogger
{
    public void Log(string message) => Console.WriteLine($"[LOG] {message}");
}

public class CheckoutService
{
    private readonly IPaymentProvider _paymentProvider;

    public CheckoutService(IPaymentProvider paymentProvider)
    {
        _paymentProvider = paymentProvider;
    }

    public bool Checkout(Order order)
    {
        ConsoleLogger logger = new ConsoleLogger();   // (!)
        if (order.IsPaid) { return false; }
        if (!_paymentProvider.Pay(order.TotalAmount))
        {
            logger.Log($"Payment of {order.TotalAmount} for {order.CustomerEmail}
declined.");
            return false;
        }
        order.MarkAsPaid();
        logger.Log($"Payment of {order.TotalAmount} for {order.CustomerEmail}
succeeded.");
        return true;
    }
}
```

Die Zeile mit `new ConsoleLogger()` macht mehrere Probleme:

Problem	Warum?
Nicht austauschbar	Sollen die Logs in eine Datei oder an einen Logging-Dienst gehen, musst du *CheckoutService* ändern.
Nicht testbar	Im Test kannst du nicht prüfen, ob und was geloggt wurde. Jeder Test schreibt in die Konsole.
Nicht abschaltbar	Jeder, der den Service verwendet, bekommt Konsolenausgaben. Auch eine Web-App, die gar keine Konsole hat.
Versteckt	Der Konstruktor zeigt nicht, dass der Service in die Konsole schreibt.

Das Dependency Inversion Principle

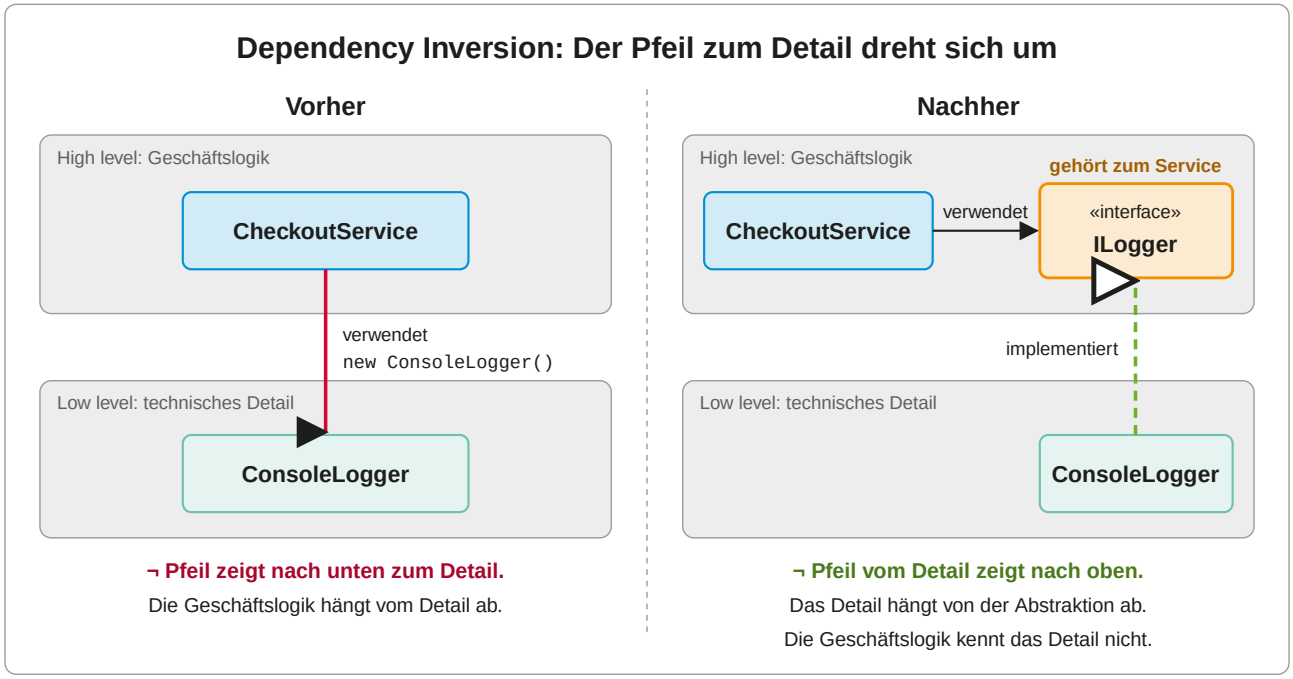
Robert C. Martin hat das Prinzip so formuliert:

> 1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
> 2. Abstractions should not depend on details. Details should depend on abstractions.

Begriff	Bedeutung	Im Beispiel
High-level Modul	Code mit Geschäftslogik. Er beschreibt, *was* die Firma tut.	*CheckoutService*
Low-level Modul	Technisches Detail. Es beschreibt, *wie* etwas technisch passiert.	*ConsoleLogger*, *CreditCard*
Abstraktion	Interface oder abstrakte Klasse. Sie beschreibt nur, was etwas kann.	*ILogger*, *IPaymentProvider*

Die Regel „Bestellung zuerst bezahlen, dann als bezahlt markieren“ ändert sich selten. Ob die Logs in die Konsole oder in eine Datei gehen, ändert sich oft. Daher soll die stabile Geschäftslogik nicht vom wechselnden Detail abhängen.

Was wird „umgedreht“?



- **Vorher:** Der Pfeil im Klassendiagramm zeigt von *CheckoutService* nach unten zu *ConsoleLogger*. Die Geschäftslogik hängt vom Detail ab.
- **Nachher:** *CheckoutService* verwendet nur noch *ILogger*. *ConsoleLogger* implementiert *ILogger*. Der Pfeil vom Detail zeigt jetzt **nach oben**. Diese Umkehrung (*Inversion*) gibt dem Prin-

zip den Namen.
- Zur Laufzeit ruft der Service weiterhin `Log()` von *ConsoleLogger* auf. Umgedreht wird nur die Abhängigkeit im **Quellcode**: *CheckoutService.cs* erwähnt *ConsoleLogger* nicht mehr.

Das Interface gehört zum Service, nicht zur Implementierung. Der Service legt fest, was er braucht (`Log(string message)`), jede Implementierung richtet sich danach. In größeren Solutions liegt *ILogger* daher im Projekt *Application* und *ConsoleLogger* in einem Projekt wie *Infrastructure*. *Infrastructure* referenziert *Application*, nie umgekehrt.

Teil 2: Abstraktionen hängen nicht von Details ab

Auch ein Interface kann ein Detail „verraten“:

```
public interface IPaymentProvider
{
    bool Pay(decimal amount);
    string CreditCardNumber { get; }   // Detail of one implementation
}
```

Eine Prepaid Karte hat keine Kreditkartennummer. Ein Interface beschreibt daher nur, was der **Aufrufer** braucht. Typen wie `SqlConnection` oder `HttpClient` darin sind ein Warnzeichen.

Depend on interfaces, not on concretes



Depend on interfaces, not on concretes: Eine Klasse mit Logik hängt von **Interfaces** ab, nicht von **konkreten Klassen**.

Prüffrage: Steht im Konstruktor, in einem Feld oder in einem Property eines Services eine konkrete Klasse wie *ConsoleLogger*? Oder steht `new ConsoleLogger()` in einer Methode? Dann ist der Service fest an diese Klasse gebunden.

Ausnahme: Datenklassen wie *Order*, *Product* oder *string* darfst du direkt verwenden. Sie enthalten nur Daten, keine austauschbare Logik.

Dependency Injection: die Technik dazu

Dependency Injection (DI) bedeutet: Eine Klasse erzeugt ihre Abhängigkeiten **nicht selbst** mit `new`. Sie bekommt sie **von außen** übergeben („injiziert“). Sie verlangt dabei nur ein Interface. DI ist die Technik, das Dependency Inversion Principle ist das Ziel.

Beim Payment Provider machen wir das schon. Jetzt behandeln wir den Logger genauso:

ILogger.cs und **ConsoleLogger.cs**

```
namespace WebshopDemo.Application;
```

```
public interface ILogger
{
    void Log(string message);
}
```

```
using System;

namespace WebshopDemo.Application;

public class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine($"[LOG] {message}");
}
```

CheckoutService.cs

```
namespace WebshopDemo.Application;

public class CheckoutService
{
    private readonly IPaymentProvider _paymentProvider;

    public CheckoutService(IPaymentProvider paymentProvider)   // (1)
    {
        _paymentProvider = paymentProvider;
    }

    public ILogger? Logger { get; set; }                        // (2)

    public bool Checkout(Order order)
    {
        if (order.IsPaid) { return false; }
        if (!_paymentProvider.Pay(order.TotalAmount))
        {
            Logger?.Log($"Payment of {order.TotalAmount} for {order.CustomerEmail}
declined.");
            return false;
        }
        order.MarkAsPaid();
        Logger?.Log($"Payment of {order.TotalAmount} for {order.CustomerEmail}
succeeded.");
        return true;
    }
}
```

- **(1) Constructor Injection:** Ohne Payment Provider kann der Service nicht arbeiten. Daher verlangt der Konstruktor ihn, sonst lässt sich der Service gar nicht erstellen.
- **(2) Property Injection:** Statt `new ConsoleLogger()` steht hier nur noch das Interface *ILogger*. Logging ist optional. Daher ist der Logger ein nullable Property, das man von außen setzen **kann**. Mit `?.` wird `Log()` nur aufgerufen, wenn ein Logger gesetzt ist.

Constructor oder Property Injection?

Die Entscheidung folgt der **Multiplizität** aus dem Kapitel Interfaces:

	Constructor Injection	Property Injection
Multiplizität	**1** (Pflicht)	**0..1** (optional)
C#	Konstruktorparameter, *private readonly* Feld	nullable Property mit *set*
Beispiel	*IPaymentProvider*	*ILogger?*
Vorteil	Objekt ist immer vollständig. Der Compiler erzwingt die Abhängigkeit.	Objekt funktioniert auch ohne.
Risiko	–	Vergisst man das Setzen, fehlt die Funktion ohne Fehlermeldung.

Regel: Verwende Constructor Injection. Property Injection nur, wenn die Abhängigkeit wirklich optional ist.

Wer erzeugt die Objekte? Der Composition Root

Irgendwo muss `new CreditCard(...)` stehen. Das passiert an **einer** Stelle, dem **Composition Root**. Dort entscheidest du, welche Implementierungen verwendet werden. Alle anderen Klassen kennen nur Interfaces. In einem Test ist der Composition Root der Arrange-Teil:

CheckoutServiceTests.cs (im Projekt *WebshopDemo.Tests*)

```
using System;
using WebshopDemo.Application;
using Xunit;

namespace WebshopDemo.Tests;

public class CheckoutServiceTests
{
    [Fact]
    public void Checkout_WithCreditCardWithinLimit_ReturnsTrue()
    {
        // Composition root of this test: here you decide which implementations are
used.
        IPaymentProvider paymentProvider = new CreditCard(limit: 1000, expiration: new
DateTime(2030, 12, 31));
        CheckoutService checkoutService = new CheckoutService(paymentProvider);
        Order order = new Order(customerEmail: "max@example.com", totalAmount: 250);

        bool success = checkoutService.Checkout(order);

        Assert.True(success);
    }
```

```
}
```

Der Test setzt keinen Logger. Das ist erlaubt, denn Logging ist optional. Willst du die Meldungen sehen, ergänzt du `checkoutService.Logger = new ConsoleLogger();`. Für eine Prepaid Karte änderst du nur die erste Zeile. *CheckoutService* bleibt in beiden Fällen unverändert.

Ausblick: der DI Container in ASP.NET Core

In ASP.NET Core ist der Composition Root die Datei *Program.cs*. Dort schreibst du kein `new`. Du **registrierst** nur, welche Klasse zu welchem Interface gehört:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using WebshopDemo.Application;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

// Composition root: register which class is used for which interface.
builder.Services.AddScoped<IPaymentProvider, CreditCard>();
builder.Services.AddScoped<CheckoutService>();
```

Der **DI Container** erzeugt die Objekte und übergibt sie an die Konstruktoren. Das Beispiel ist vereinfacht: Der Container füllt nur Konstruktorparameter mit registrierten Typen, *CreditCard* bräuchte dafür einen anderen Konstruktor. Property Injection kennt er nicht. Für Logging gibt es dort `ILogger<T>` aus *Microsoft.Extensions.Logging*, es kommt über den Konstruktor.

Testen mit einem Fake

Ob die Zahlung mit *CreditCard* klappt, hängt von Limit und Ablaufdatum ab. Ein echter Zahlungsdienst würde sogar Geld abbuchen. Mit DI übergibst du im Test eine **Fake**-Implementierung. Sie zahlt nicht wirklich, sondern liefert ein festgelegtes Ergebnis und merkt sich jeden Aufruf.

FakePaymentProvider.cs (im Projekt *WebshopDemo.Tests*)

```
using System.Collections.Generic;
using WebshopDemo.Application;

namespace WebshopDemo.Tests;

public class FakePaymentProvider : IPaymentProvider
{
    private readonly bool _result;
    private readonly List<decimal> _payments = new();

    public FakePaymentProvider(bool result)
    {
        _result = result;
```

```
    }

    public IReadOnlyList<decimal> Payments => _payments;

    public bool Pay(decimal amount)
    {
        _payments.Add(amount);
        return _result;
    }
}
```

Damit testest du Fälle, die mit einer echten Kreditkarte schwer herzustellen sind. Der zweite Test prüft sogar, ob der Service den Payment Provider **aufgerufen** hat:

```
[Fact]
public void Checkout_WhenPaymentDeclined_ReturnsFalse()
{
    FakePaymentProvider paymentProvider = new FakePaymentProvider(result: false);
    CheckoutService checkoutService = new CheckoutService(paymentProvider);
    Order order = new Order(customerEmail: "max@example.com", totalAmount: 250);

    bool success = checkoutService.Checkout(order);

    Assert.False(success);
}

[Fact]
public void Checkout_WhenOrderAlreadyPaid_DoesNotCallPaymentProvider()
{
    FakePaymentProvider paymentProvider = new FakePaymentProvider(result: true);
    CheckoutService checkoutService = new CheckoutService(paymentProvider);
    Order order = new Order(customerEmail: "max@example.com", totalAmount: 250);
    order.MarkAsPaid();

    checkoutService.Checkout(order);

    Assert.Empty(paymentProvider.Payments);
}
```



Begriffe: Eine einfache Ersatzimplementierung für Tests heißt *Fake*. Bibliotheken wie Moq oder NSubstitute erzeugen solche Klassen automatisch. Man spricht dann von *Mocks*. Beides funktioniert nur mit DI.

Versteckte Abhängigkeiten

Der erste Test oben ist heute grün. Ab 2031 wird er rot, obwohl niemand den Code geändert hat. Der Grund steht in *CreditCard*: `DateTime.UtcNow`. Auch das ist eine Abhängigkeit, nur sieht man sie nicht im Konstruktor. Solche **versteckten Abhängigkeiten** gibt es oft:

Im Code	Problem	Ersatz über DI
`DateTime.Now`, `DateTime.UtcNow`	Ergebnis hängt vom Tag ab, Grenzfälle sind nicht testbar.	`TimeProvider` im Konstruktor
`new Random()`	Jeder Lauf liefert andere Zahlen.	`Random` im Konstruktor, im Test mit festem Startwert
`Console.WriteLine()`	Ausgabe ist nicht prüfbar und nicht abschaltbar.	`ILogger` oder ein eigenes Interface
`new SmtpClient()`, `new SqlConnection()`	Test braucht Mailserver oder Datenbank.	Interface wie `IEmailSender`

Wie du `TimeProvider` übergibst und im Test durch einen Fake ersetzt, steht im Abschnitt Deterministische Tests.

Was bringt das?

Vorteil	Im Beispiel
Testbarkeit	Wir testen „Zahlung abgelehnt“ mit einem Fake, ohne echte Zahlung.
Austauschbarkeit	Konsole oder Datei, Kreditkarte oder Prepaid Karte: nur der Composition Root ändert sich.
Sichtbare Abhängigkeiten	Der Konstruktor zeigt, was der Service braucht.



Nicht übertreiben: Ein Interface lohnt sich für Abhängigkeiten mit Logik oder Technik (Zahlung, E-Mail, Datenbank, Zeit). Ein Interface *IOrder* für die Datenklasse *Order* nicht.

Im Code-Review erkennen

Warnzeichen im Code	Verletzt	Was du verlangst
`new ConsoleLogger()`, `new SmtpClient()` in einer Methode eines Services	DIP: Service erzeugt sein Detail selbst	„`CheckoutService` erzeugt `ConsoleLogger` selbst. Definiere `ILogger` und übergib ihn per Constructor Injection.“
Konstruktorparameter, Feld oder Property mit konkreter Klasse (`CreditCard card`)	Depend on interfaces, not on concretes	„Der Service soll von `IPaymentProvider` abhängen, nicht von `CreditCard`.“
`DateTime.Now`, `new Random()`, `Console.WriteLine()` in der Logik	Versteckte Abhängigkeit	„Die Zeit ist eine versteckte Abhängigkeit. Übergib einen `TimeProvider` im Konstruktor.“
Interface mit Typen wie `SqlConnection` oder Members, die nur eine Implementierung hat	DIP Teil 2: Abstraktion hängt vom Detail ab	„`IPaymentProvider` verrät ein Detail von `CreditCard`. Entferne `CreditCardNumber` aus dem Interface.“

Warnzeichen im Code	Verletzt	Was du verlangst
Das Projekt *Application* referenziert *Infrastructure*	DIP: Pfeil zeigt nach unten	„Das Interface gehört zur Application. Verschiebe `ILogger` nach *Application* und drehe die Referenz um.“
Pflicht-Abhängigkeit als nullable Property, im Code `PaymentProvider!.Pay(...)`	Falsche Injection-Art	„Ohne Payment Provider kann der Service nicht bezahlen. Verlange ihn im Konstruktor.“
Ein Interface für jede Datenklasse (`IOrder`, `IProduct`)	Over-Engineering	„`Order` enthält nur Daten. Das Interface `IOrder` bringt nichts, entferne es.“

Weitere Informationen

- Dependency Injection in .NET (Microsoft Learn)
- Dependency inversion principle (Wikipedia)
- Unit Testing Best Practices (Microsoft Learn)

Der OR Mapper EF Core

Warum ein OR Mapper (Object-relational mapping)

Der Namespace System.Data als Unterbau

Natürlich kann auch in .NET Core - so wie in anderen Frameworks auch - ein "roher" Datenbankzugriff erfolgen. Folgendes Programm zeigt diese Möglichkeit für eine SQLite Datenbank:

```
//
**********************************************************************************************
**********
// ACHTUNG: DIESER CODE KANN IHRE GESUNDHEIT GEFÄHRDEN!!!
// Vorher die Testdatenbank in den Ausgabeordner kopieren
//
**********************************************************************************************
**********

using System;
using System.Data.SQLite;   // NuGet: Install-Package System.Data.SQLite.Core
namespace DbDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string nameBeginsWith = "A";
            SQLiteConnection conn = new SQLiteConnection("DataSource=Tests.db");
            conn.Open();

            // Einladung für SQL Injections...
            SQLiteCommand command = new SQLiteCommand("SELECT * FROM Pupil WHERE
P_Lastname LIKE '" + nameBeginsWith + "%'", conn);
            SQLiteDataReader reader = command.ExecuteReader();
            while (reader.Read())
            {
                // Wir casten auf "gut Glück", denn alles ist ein object.
                long id = (long)reader["P_ID"];
                string lastname = reader["P_Lastname"] == DBNull.Value ? null : (
string)reader["P_Lastname"];
                Console.WriteLine($"{id}: {lastname}");
            }

            // Wer schließt reader und conn?? Es wurde kein using verwendet.
        }
    }
}
```

Allerdings hat diese Technik einige (gravierende) Nachteile:

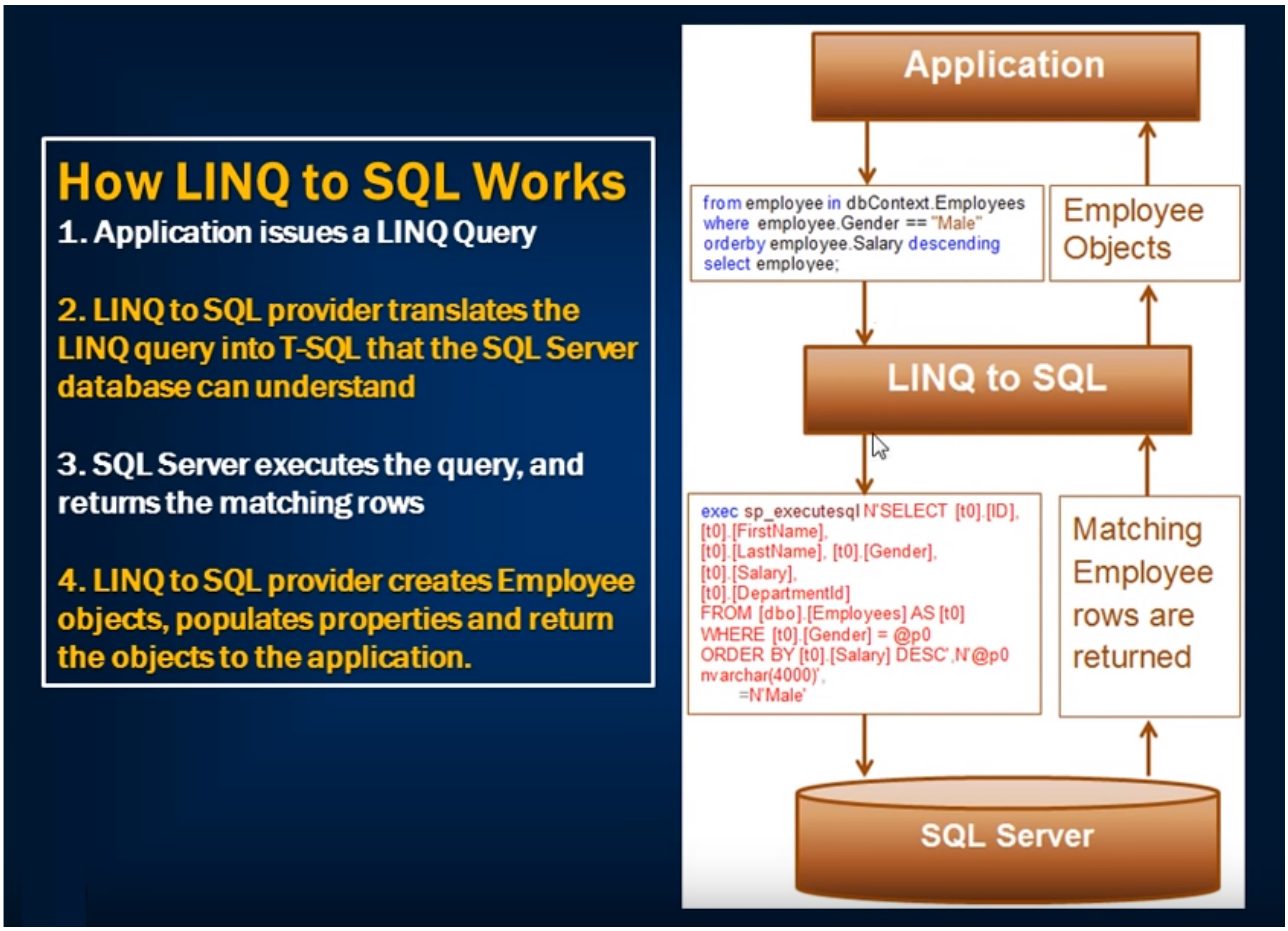
- Es wird *System.Data.SQLite* eingebunden. Möchten wir ein anderes DBMS verwenden (z. B.

SQL Server), so müssen alle Funktionsaufrufe geändert werden.

- Das Resultset liefert die Spalten als object. Wir müssen "auf gut Glück" casten, wir haben keine Information über den Datentyp in der Tabelle.
- SQL Statements werden durch Stringverknüpfung natürlich ohne Syntaxprüfung mit allen Gefahren erzeugt.
- Der Zugriff auf die Spalten erfolgt mittels der Spaltenbezeichnung als String. Wird diese Spalte nicht gefunden, entsteht ein Laufzeitfehler.
- Das Schließen der Verbindungen und Resultsets muss durch den Programmierer händisch gemacht werden.

Der Datenbankzugriff in heute gängigen Frameworks im Enterprise Architecture Bereich erfolgt daher nie nach dieser Methode, sondern durch eine zusätzliche Zwischenschicht: den OR Mapper. Er erzeugt generisch aus den Tabellendefinitionen Klassen und ermöglicht einen typisierten Zugriff auf eine Relationale Datenbank. In .NET ist das *Entity Framework* dieser OR Mapper und ermöglicht auch den Zugriff über LINQ.

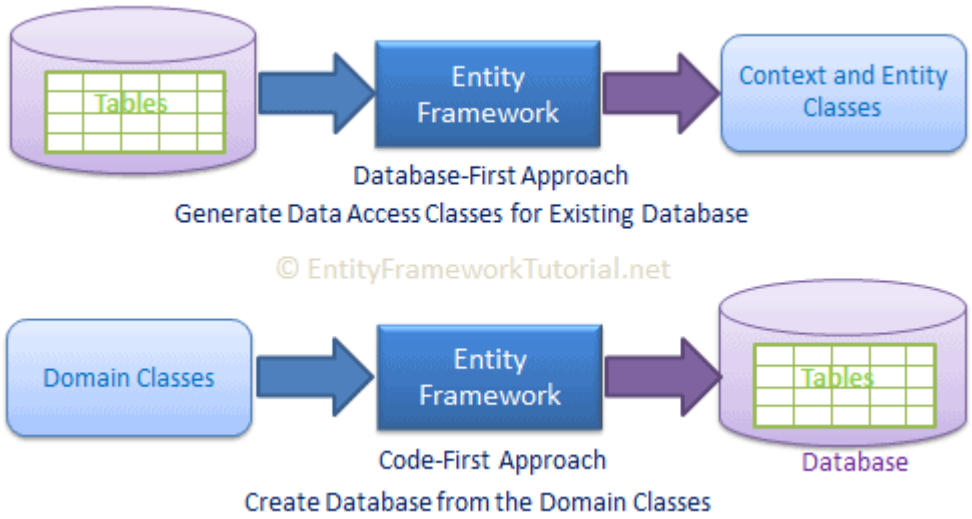
Die wohl größte Stärke der Microsoft .NET Entwicklungsumgebung ist die Kombination von LINQ mit dem OR Mapper. Er ermöglicht es, mit Hilfe der gewohnten LINQ Syntax (oder den Extension Methoden) SQL Code zu generieren.



Quelle: https://www.youtube.com/watch?v=bsncc8dYIgY

Es gibt 2 Ansätze, ein Projekt mit EF Core anzulegen: *Database First* oder *Code First*. Bei Database First wird die Datenbank klassisch über CREATE TABLE Statements erstellt. Danach wer-

den mit einem Skript die Modelklassen erstellt. Bei Code First werden zuerst die Modelklassen in C# erstellt. Diese werden dann über sogenannte Migrations in Tabellen umgesetzt.



Quelle: https://www.entityframeworktutorial.net/efcore/entity-framework-core.aspx

Klassenmodelle persistieren mit EF Core

> Im Ordner CodeFirstDemo ist ein lauffähiges Beispiel dieser Erklärungen.

Anlegen des Musterprojektes (.NET 8)

Um ein Klassenmodell umsetzen zu können, legen wir eine kleine Solution an. Wir nutzen nun *2 Projekte* und keine Konsolenapplikation:

- **CodeFirstDemo.Application** beinhaltet die Modelklassen und die Logik für den Datenbankzugriff. Mit *dotnet add package* können wir die NuGet Pakete für EF Core und Bogus (Musterdaten Generator) hinzufügen.
- **CodeFirstDemo.Test** beinhaltet sogenannte *Unit Tests*. Diese rufen unseren Programmcode im Application Projekt auf. Dafür wird eine Referenz auf das Application Projekt hinzugefügt.
- **CodeFirstDemo.sln** ist die Solution, die die 2 Projekte beinhaltet und wird in Visual Studio (oder Rider) gestartet.

```
rd /S /Q CodeFirstDemo
md CodeFirstDemo
cd CodeFirstDemo
md CodeFirstDemo.Application
cd CodeFirstDemo.Application
dotnet new classlib
dotnet add package Microsoft.EntityFrameworkCore --version 8.*
dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 8.*
dotnet add package Microsoft.EntityFrameworkCore.Proxies --version 8.*
dotnet add package Bogus --version 35.6.1
cd ..
md CodeFirstDemo.Test
cd CodeFirstDemo.Test
dotnet new xunit
dotnet add reference ..\CodeFirstDemo.Application
cd ..
dotnet new sln
dotnet sln add CodeFirstDemo.Application
dotnet sln add CodeFirstDemo.Test
start CodeFirstDemo.sln
```

Nun stellen wir durch Doppelklick auf die Projektdatei (*CodeFirstDemo.Application*) die Option *TreatWarningsAsErrors* ein.

```
<PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
</PropertyGroup>
```

Im Ordner CodeFirstDemo des Kapitels *03 EF Core* befindet sich eine fertige Applikation, die eine fertige Implementierung beinhaltet.

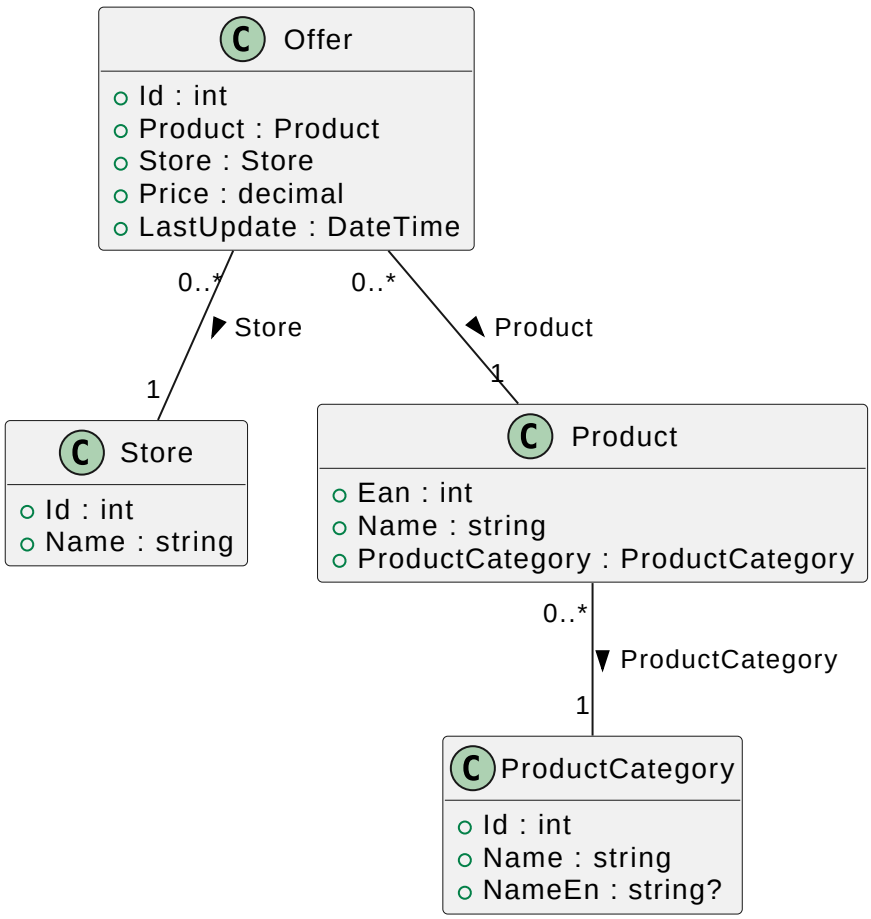
Nullable reference types und EF Core

In einer Datenbank können manche Felder NULL Werte enthalten (nullable), andere Felder werden mit NOT NULL definiert. Wir können mit den Datentypen steuern, ob EF Core ein Feld mit NULL oder NOT NULL anlegt. Deaktivieren wir die nullable Features von C# 8, dreht sich die Bedeutung um und aus ihrem Modell wird ein anderes Datenbankschema erzeugt! Wir müssen sich also schon am Beginn Ihres Projektes entscheiden, ob wir dieses Feature aktivieren oder nicht.

Daher ist das nachträgliche Aktivieren des nullable Features bei EF Core in bestehenden Projekten sehr gefährlich!

Code first: Vom Klassenmodell zur Datenbank

Seit Beginn der Programmierausbildung werden Klassen und die Beziehungen zwischen diesen Klassen als Assoziationen umgesetzt. Betrachten wir das folgende Modell. Es setzt ein kleines Bestellsystem wie z. B. Geizhals um. Produkte werden in verschiedenen Stores zu unterschiedlichen Preisen angeboten (offer).



[.small]# https://www.plantuml.com/plantuml/uml/ZP1FIuCn3CRl_HGXHsL3h-

z11xg382ge7w2qsLRedz7cFGpPTxSlQNSe0-sboQ_9yoJT5DQPv-3LmLa22aS-GI0-95kKyReK-EIK2NuKr7FxOc425rcoDnseveMpYtjrLgtZVRUZVBtZ8oSx6wweHOB5OnAlNkdCWYmP5xGN-AeBVn-jvkNbkLvVBj1YB9PujrxV8AnMWFS8NHGV9CHhLNqEgUi9jbKwUAfM-vHF__c62zh62tnKvq6nAsrr_4Dg7662WLJjHeRLnaxPSQ2eDXJj7Fml #

Beachte die Kardinalitäten. Ein Store hat mehrere (*0..n*) Offers, während ein Offer genau einem (*1*) Store zuzuordnen ist.

Nun wollen wir dieses Klassenmodell speichern, also *persistieren*. Dafür stehen uns mehrere Techniken zur Verfügung:

- Dateien (Serialisierung)
- NoSQL Datenbanken
- Relationale Datenbanken

EF Core unterstützt das Erstellen einer Datenbank, damit diese das Klassenmodell speichern kann. Dafür schreiben wir wie gewohnt diese Klassen in C#. Um Ordnung zu halten, wird im Projekt *Application* ein Ordner *Model* erstellt.

Conventions und Annotations in EF Core

EF Core kann in den meisten Fällen ohne besondere Anweisungen eine Datenbank erzeugen. Das liegt an den sogenannten *Conventions*, die wir in den nachfolgenden Klassen einhalten:

- Properties mit dem Namen *Id* werden automatisch als Primärschlüssel definiert.
- Id Properties mit dem Datentyp *int* werden automatisch zu AutoIncrement Feldern.
- Properties vom Typ *List<Typname>* finden "automatisch" ihren Weg in die richtige Tabelle. So verweist das Property *Pupils* in den vorigen LINQ Beispielen automatisch auf alle Schüler dieser Klasse.
- Properties vom Typ *Typname* verweisen automatisch auf die Tabelle dieses Typs. So referenziert das Property vom Typ *Store* automatisch auf die Tabelle *Store*.
- Fremdschlüsselfelder mit dem Namen *NavigationProperty* + *PK Name* (wie *StoreId*) werden automatisch zum Fremdschlüsselfeld (in diesem Beispiel für die Tabelle *Store*).
- Über NULL oder NOT NULL entscheidet der Datentyp (bei den nullable reference types werden Typen mit ? am Ende zu Feldern mit der NULLABLE Eigenschaft).
- Read-only Properties werden nicht in der Datenbank abgebildet.

Erstellen von Modelklassen: Konstruktoren verwenden

Erstellen wir die Klasse *Store*, bekommen wir durch die Option *Nullable* und *TreatWarningsAsErrors* eine Fehlermeldung beim Property *Name*:

```
public class Store
{
    public int Id { get; set; }
    public string Name { get; set; }   // Error: not initialized
```

```
}
```

Wir müssen daher *Konstruktoren* verwenden, um alle Felder zu initialisieren. Allerdings sollte nicht blind jedes Feld im Konstruktor initialisiert werden. *Id* ist ein AutoIncrement Wert (Details unter Conventions), daher kann dieser Wert gar nicht im Konstruktor zugewiesen werden.

Die verbesserte Version sieht nun so aus:

```
public class Store
{
    public Store(string name)
    {
        Name = name;
    }

    public int Id { get; private set; }      // ID by convention, AutoIncrement by
convention
    public string Name { get; set; }         // NOT NULL because nullable reference
types are enabled
}
```

Id hat nun einen private setter. Da der Primärschlüssel in EF Core nicht veränderbar ist, definieren wir alle Schlüsselfelder mit dieser Sichtbarkeit. Es macht wenig Sinn von einem bestehenden Eintrag diesen Wert zu ändern. Alle anderen Properties haben einen public setter, da wir die Spalten in der Datenbank ja auch ändern wollen (UPDATE Befehl). Wir bestimmen also, welche Properties wir im Programmverlauf aktualisieren dürfen. Warum wir nicht nur *get* verwenden ist in den Conventions erklärt.

Optionale Werte

Die Klasse ProductCategory hat ein Property *NameEn* (englischer Name). Diese Spalte ist optional, darf also den Wert NULL enthalten. Bei aktivierten nullable reference Types legen wir diese Spalte daher mit dem Datentyp *string?* an. Sollen optionale int, DateTime, … Werte gespeichert werden, wird der entsprechende nullable Datentyp (int?, DateTime?, …) verwendet.

Es ist in C# möglich, ein *Argument mit default value* zu dafür zu definieren. Diese Form hat den Vorteil, dass das zusätzliche Property *nameEn* in IntelliSense erscheint. Durch den default value muss kein Wert angegeben werden, d. h. der Konstruktor von ProductCategory kann auch mit einem Argument aufgerufen werden.

```
public class ProductCategory
{
    public ProductCategory(string name, string? nameEn = null)
    {
        Name = name;
        NameEn = nameEn;
    }
```

```
      public int Id { get; private set; }      // ID by convention, AutoIncrement by
convention
      public string Name { get; set; }
      public string? NameEn { get; set; }      // nullable
  }
```

Navigationen

Das Produkt (Klasse *Product*) verwendet die Klasse *ProductCategory*. Dies ist im Klassenmodell durch die Verwendung des Typs *ProductCategory* leicht ersichtlich. In einer relationalen Datenbank kennen wir das Konzept des Fremdschlüssels. Möchten wir Produkte und Kategorien speichern, würde die Produkttabelle einfach eine Spalte für den Fremdschlüssel (der Kategorie ID) beinhalten.

EF Core macht dies automatisch. Legen wir ein Property *ProductCategory* an, wird automatisch ein Feld *ProductCategoryId* erstellt.

Schlüssel und UNIQUE Constraints

Da der Schlüssel *Ean* heißt, greift die Convention (Id als Schlüsselname) nicht mehr. Wir müssen daher mit Annotations aus dem Namespace *System.ComponentModel.DataAnnotations* das jeweils nachfolgende Property genauer definieren. Damit der int Wert für die EAN Nummer nicht als auto increment Wert angelegt wird, setzen wir diese Information mittels der Annotation *DatabaseGenerated(DatabaseGeneratedOption.None)*

Die Annotation `[Index(nameof(Name), IsUnique = true)]` muss über der Klasse stehen. Sie gibt an, dass EF Core für das Feld *Name* einen *unique index* anlegt.

```
[Index(nameof(Name), IsUnique = true)]
public class Product
{
    public Product(int ean, string name, ProductCategory productCategory)
    {
        Ean = ean;
        Name = name;
        ProductCategory = productCategory;
    }

    // Ean is the PK and not an auto increment column. Annotations are used
    // for the next property (ean)
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int Ean { get; private set; }
    public string Name { get; set; }
    public ProductCategory ProductCategory { get; set; }  // Navigation property
}
```

Der Konstruktor von Product verlangt diesmal den Primärschlüssel (die EAN), da dieser Wert von externen Quellen kommt und nicht in der Datenbank generiert wird. Für die Navigation

verlagen wir nur die Instanz von *ProductCategory*.

Nun fehlt noch die Klasse *Offer*, die mit bestehendem Wissen angelegt werden kann:

```
public class Offer
{
    public Offer(Product product, Store store, decimal price, DateTime lastUpdate)
    {
        Product = product;
        Store = store;
        Price = price;
        LastUpdate = lastUpdate;
    }

    public int Id { get; private set; }
    public Product Product { get; set; }
    public Store Store { get; set; }
    public decimal Price { get; set; }
    public DateTime LastUpdate { get; set; }
}
```

Anlegen von protected Konstruktoren

Zum Abschluss müssen wir noch eine Besonderheit von EF Core berücksichtigen. EF Core versucht beim Lesen eines Datensatzes eine Instanz der entsprechenden Klasse zu erzeugen. Dafür braucht es aber einen Default Konstruktor. Diese Default Konstruktoren wollen wir allerdings vermeiden. Ein guter Weg ist das Anlegen dieses Konstruktors als private oder protected Konstruktor. Damit kann niemand mit *new Store()* ein uninitialisiertes Store Objekt anlegen. Wir verwenden protected, da wir im nächsten Kapitel auch Vererbung verwenden wollen.

Legen wir nun einen Default Konstruktor an, ergeben sich aufgrund des aktivierten nullable Features Warnungen bzw. Fehlermeldungen. Wir können aber in diesem Fall mit einer etwas seltsamen Anweisung Warnungen unterdrücken. In Visual Studio können die *#pragma* Anweisungen mit *STRG* + . und *Suppress or Configure issues* - *Suppress CS8618 in Source Code* deaktiviert werden.

> **Hinweis:** Das Unterdrücken von Warnungen ist nur gerechtfertigt, wenn wir mit Sicherheit ausschließen können, dass dadurch ein Laufzeitfehler entsteht. EF Core garantiert das Initialisieren der Felder, daher kann diese Technik hier verwendet werden.

```
public class Store
{
    public Store(string name)
    {
        Name = name;
    }
    #pragma warning disable CS8618
    protected Store() { }
    #pragma warning restore CS8618
```

```
    public int Id { get; private set; }
    public string Name { get; set; }
}
```

Anlegen der Datenbank: Der Datenbankkontext

Damit eine Datenbank erzeugt werden kann, brauchen wir einen *Datenbankkontext*. Er ist die Verbindung zur darunterliegenden Datenbank. Dafür legen wir einen Ordner *Infrastructure* an und erstellen eine Klasse *StoreContext*. Die Klasse kann beliebig benannt werden, die Endung *Context* hat sich aber bei .NET Entwicklern durchgesetzt.

```
public class StoreContext : DbContext
{
    public StoreContext(DbContextOptions opt) : base(opt) { }

    public DbSet<Store> Stores => Set<Store>();
    public DbSet<Offer> Offers => Set<Offer>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<ProductCategory> ProductCategories => Set<ProductCategory>();
}
```

Wir erkennen 2 Dinge:

- Es gibt einen Konstruktor, der eine Konfiguration verlangt. Diese Konfiguration gibt an, welche Datenbank verwendet werden soll.
- Jede Tabelle wird als *DbSet<T>* definiert. Sie geben einfach mit der *Set()* Methode die entsprechende Tabelle zurück.

Schreiben eines Tests zum Anlegen der Datenbank

Wir können das Programm zwar kompilieren, es wird aber nie ausgeführt. Das Projekt *Application* ist eine Klassenbibliothek und hat daher keine *Main()* Methode. Um den Code aufzurufen, verwenden wir ein in der Softwareentwicklung sehr bekanntes Werkzeug: Der Unittest.

Unittests sind dafür da, Code aufzurufen und das Ergebnis mit einem erwarteten Wert zu vergleichen.

Zu Beginn haben wir bereits ein Projekt *CodeFirstDemo.Test* angelegt. Nun legen wir eine Klasse *StoreContextTests* darin an. Die Namensgebung sollte immer *zu testende Klasse + Tests* sein.

Die Logik zur Erstellung der Datenbankverbindung geben wir in eine gemeinsame Basisklasse *DatabaseTest*. Sie erstellt im Konstruktor eine SQLite Datenbank. Damit nach dem Unittest alle Ressourcen geschlossen werden, implementiert diese Klasse das Interface *IDisposeable*.

DatabaseTest.cs

```
public class DatabaseTest : IDisposable
```

```
{
    protected readonly StoreContext _db;

    public DatabaseTest()
    {
        var opt = new DbContextOptionsBuilder()
            .UseSqlite("Data Source=Stores.db")
            .Options;

        _db = new StoreContext(opt);
    }
    public void Dispose()
    {
        _db.Dispose();
    }
}
```

Nun implementieren wir die eigentliche Testklasse *StoreContextTests*. Sie erbt von *DatabaseTest*, daher steht die Membervariable *_db* zur Verfügung. Dieser Test prüft nur, ob die Anweisung ohne Exception ausgeführt werden kann.

StoreContextTests.cs

```
[Collection("Sequential")] // A file database does not support parallel test execution.
public class StoreContextTests : DatabaseTest
{
    [Fact]
    public void CreateDatabaseTest()
    {
        _db.Database.EnsureCreated();
    }
}
```

Bis jetzt haben wir nirgends gesagt, welche Datenbank wir verwenden wollen. Nun müssen wir dies angeben. Dafür verwenden wir die Klasse *DbContextOptionsBuilder()* aus dem Namespace *Microsoft.EntityFrameworkCore*. EF Core arbeitet datenbankenunabhängig. Wir verwenden mit *UseSqlite()* eine SQLite Datenbank. Mit *UseMysql()* z. B. können wir jederzeit auch eine MySQL Datenbank erzeugen. Der Provider abstrahiert datenbankspezifische Anweisungen wie das Erstellen von Autoincrement Werten und die Definition der SQL Datentypen.

3 Dinge sind zudem noch wichtig:

- Die Testklasse und die Testmethode muss public sein.
- Über jeder Testmethode muss die Annotation *[Fact]* aus dem Namespace *Xunit* geschrieben werden.
- Tests werden normalerweise parallel ausgeführt. Da wir die Filedatenbank jedoch immer löschen und neu erzeugen, ist eine parallele Ausführung nicht möglich. *[Collection("Sequential")]* konfiguriert xUnit so, dass die Tests nacheinander ausgeführt werden.

Klicken wir mit der rechten Maustaste in Visual Studio auf die Testmethode *CreateDatabaseTest* finden wir im Kontextmenü den Punkt *Run Tests*. Nach erfolgreicher Ausführung erscheint ein grünes Häkchen. Im Text Explorer (Menü *Test - Test Explorer*) ist der Test ebenfalls aufgelistet.

Nutzen einer in-memory SQLite Datenbank

SQLite Datenbanken können auch vollständig im Speicher gehalten werden. Gerade für Unittests ergeben sich mehrere Vorteile:

- Das Erstellen und befüllen der Datenbank ist natürlich viel schneller.
- Die Datenbank wird immer "frisch", also leer bei einer neuen Verbindung erstellt. Dadurch beeinflussen sich die Tests nicht.

Im Programm CodeFirstDemo werden die Unittests mit einer in-memory durchgeführt. Hier sind auch Infos über den Connectionstring und die Besonderheiten im Code abgebildet. Wichtig ist, dass eine offene SqliteConnection bei `UseSqlite` übergeben wird. Sonst wird EF Core nach `EnsureCreated` die Verbindung schließen und beim nächsten Öffnen ist die Datenbank wieder leer.

```
public class DatabaseTest : IDisposable
{
    private readonly SqliteConnection _connection;
    protected readonly StoreContext _db;

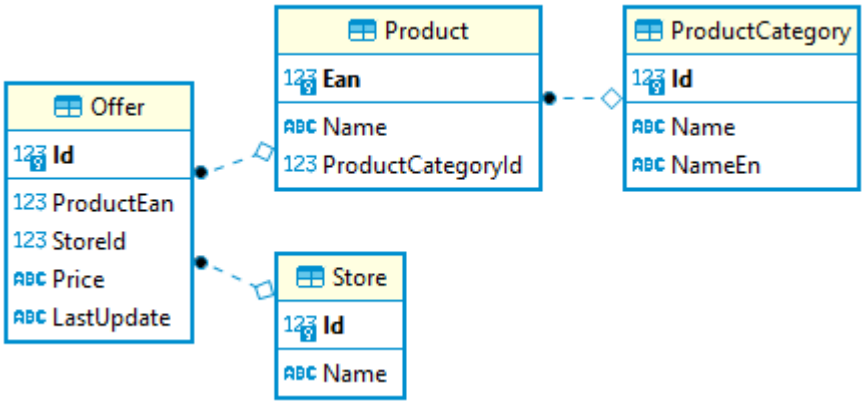
    public DatabaseTest()
    {
        _connection = new SqliteConnection("DataSource=:memory:");
        _connection.Open();

        // Enable Data Source=stores.db instead of UseSqlite(_connection)
        // if you want to open the database in DBeaver.
        var opt = new DbContextOptionsBuilder()
            //.UseSqlite("Data Source=stores.db")
            .UseSqlite(_connection)  // Keep connection open (only needed with SQLite
in memory db)
            .LogTo(message => Debug.WriteLine(message), Microsoft.Extensions.Logging
.LogLevel.Information)
            .EnableSensitiveDataLogging()
            .Options;

        _db = new StoreContext(opt);
    }
    public void Dispose()
    {
        _db.Dispose();
        _connection.Dispose();
    }
}
```

Ansehen der Datenbank in DBeaver

Nachdem der Unittest ausgeführt wurde, findet sich im Ordner *CodeFirst-Demo.Test\bin\Debug\net8.0* die Datei *Stores.db*. Zum Betrachten der Datenbank kann z. B. DBeaver (https://dbeaver.io/) verwendet werden. Öffnen wir mit DBeaver die Datenbank und klicken doppelt auf *Tables*, kann das ER Diagramm der erstellen Datenbank angezeigt werden:



Die Fremdschlüssel wurden also korrekt gesetzt. Sehen wir uns die DDL Statements der Tabelle *Stores* an, findet sich ein klassisches *CREATE TABLE* Statement, wie es auch von Hand geschrieben worden wäre:

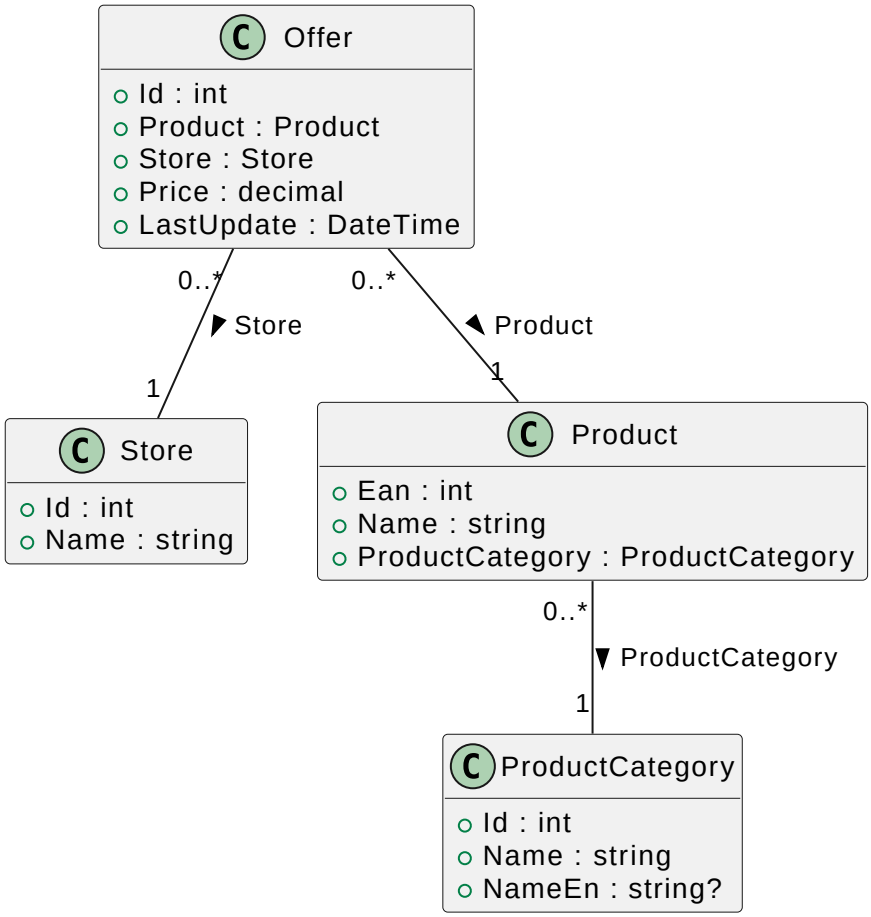
```
CREATE TABLE "ProductCategories" (
    "Id" INTEGER NOT NULL CONSTRAINT "PK_ProductCategories" PRIMARY KEY AUTOINCREMENT,
    "Name" TEXT NOT NULL,
    "NameEn" TEXT NULL
);

CREATE TABLE "Products" (
    "Ean" INTEGER NOT NULL CONSTRAINT "PK_Products" PRIMARY KEY,
    "Name" TEXT NOT NULL,
    "ProductCategoryId" INTEGER NOT NULL,
    CONSTRAINT "FK_Products_ProductCategories_ProductCategoryId" FOREIGN KEY
("ProductCategoryId") REFERENCES "ProductCategories" ("Id") ON DELETE CASCADE
);
```

Falls Fehler in der Datenbank auftreten, können die Modelklassen einfach geändert und mittels Unittest die Datenbank neu erzeugt werden.

> **Achtung:** Trenne in DBeaver vor dem Starten des Unittests die Verbindung (roter Stecker). Sonst kann der Test die Datenbank nicht löschen und bricht mit einem Zugriffsfehler ab.

Erweiterte Konfigurationsmöglichkeiten mit EF Core



[.small]# https://www.plantuml.com/plantuml/uml/ZP1FIuCn3CRl_HGXHsL3h-z11xg382ge7w2qsLRedz7cFGpPTxSlQNSe0-sboQ_9yoJT5DQPv-3LmLa22aS-GI0-95kKyReK-EIK2NuKr7FxOc425rcoDnseveMpYtjrLgtZVRUZVBtZ8oSx6wweHOB5OnAlNkdCWYmP5xGN-AeBVn-jvkNbkLvVBj1YB9PujrxV8AnMWFS8NHGV9CHhLNqEgUi9jbKwUAfM-vHF__c62zh62tnKvq6nAsrr_4Dg7662WLJjHeRLnaxPSQ2eDXJj7Fml #

Collection Navigations

Im Klassenmodell hat der Pfeil zwischen *Product* und *Offer* zwei Pfeilspitzen. An einem Ende kann von der Klasse *Offer* über das Property *Product* zum Produkt navigiert werden. In der Klasse *Product* ist auch eine besondere Navigation vom Typ *ICollection<Offer>* vorhanden. EF Core unterstützt auch diese Art der Navigation. In *Offers* werden automatisch alle Angebote zum aktuellen Produkt abgerufen. Das macht LINQ Abfragen besonders einfach.

Hier ein Beispiel, wie in *ProductCategory* die Liste der Produkte leichter abgerufen werden kann:

```
public class Product
```

```
{
    #pragma warning disable CS8618
    protected Product() { }
    #pragma warning restore CS8618
    public Product(int ean, string name, ProductCategory productCategory)
    {
        Ean = ean;
        Name = name;
        ProductCategory = productCategory;
    }

    [Key]                                                   // bestimmt den Key
    [DatabaseGenerated(DatabaseGeneratedOption.None)]   // verhindert AUTO_INCREMENT
    public int Ean { get; private set; }                    // PK
    public string Name { get; set; }
    public ProductCategory ProductCategory { get; set; }
}


public class ProductCategory
{
    public ProductCategory(string name, string? nameEn = null)
    {
        Name = name;
        NameEn = nameEn;
    }
    public int Id { get; set; }
    public string Name { get; set; }
    public string? NameEn { get; set; }
    // Liste aller Produkte, die diese Produktkategorie verwenden.
    public List<Product> Products = new();
}
```

Festlegen der Tabellennamen

Die Tabellennamen werden durch den Namen des Properties in der Kontextklasse bestimmt. So erzeugt der folgende DbContext die Tabellen Stores, Offers, Products und ProductCategories.

```
public class StoreContext : DbContext
{
    public StoreContext(DbContextOptions opt) : base(opt) { }
    public DbSet<Store> Stores => Set<Store>();
    public DbSet<Offer> Offers => Set<Offer>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<ProductCategory> ProductCategories => Set<ProductCategory>();
}
```

Oft sind aber Tabellennamen in der Einzahl, also Store, Offer, ... erwünscht. Mit der Annotation *Table* über der *Modelklasse* (nicht im Kontext) aus dem Namespace *System.Component-Model.DataAnnotations.Schema* kann das Verhalten abgeändert werden.

```
[Table("Offer")]
public class Offer
{
    /* ... */
}
```

Annotations vs. Konfiguration im DbContext

Der Einsatz von Annotations (also EF Core spezifischen Anweisungen) in den Modelklassen wird oft diskutiert. Es gibt für alle Konfigurationseinstellungen immer die Möglichkeit, diese auch im Kontext zu definieren. Dafür wird mit override die Methode *OnModelCreating()* überschrieben. Der ModelBuilder stellt mit *Entity<T>()* eine Methode bereit, die die Konfiguration aller Properties erlaubt. So kann z. B. der Tabellenname mit *ToTable()* definiert werden:

```
public class StoreContext : DbContext
{
    public StoreContext(DbContextOptions opt) : base(opt) { }
    /* .. */
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Offer>().ToTable("Offer");
    }
}
```

Manche Konfigurationsanweisungen müssen in *OnModelCreating()* durchgeführt werden, da es keine Annotations dafür gibt. Es steht allerdings für jede Annotation eine entsprechende Methode des ModelBuilders in *OnModelCreating()* bereit.

Welche Technik man wählt ist wohl Geschmackssache. Der Vorteil der Annotations, nämlich die Konfiguration in derselben Datei wie die Klasse, ist allerdings ein unschlagbarer Vorteil.

Definieren der Datentypen

Der Datenbankprovider versucht, die C# Datentypen bestmöglich in SQL Datentypen der entsprechenden Datenbank umzuwandeln. Für SQL Server ist z. B. auf https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql-server-data-type-mappings eine genaue Auflistung zu sehen.

Manchmal gibt es mehrere Möglichkeiten, einen .NET Datentyp in der Datenbank umzusetzen. So können Stringspalten mit einer Länge als *VARCHAR(n)* definiert werden. Außerdem ist bei numerischen Datentypen die Angabe der precision möglich wie z. B. *DECIMAL(9,4)*.

Für *decimal* werden standardmäßig 2 Nachkommastellen definiert.

Problematisch: Die Column Annotation

Oftmals wird die Annotation *Column* zur Definition des SQL Datentyps verwendet:

```
public class Store
{
    /* ... */
    [Column(TypeName = "VARCHAR(255")]
    public string Name { get; set; }
}

public class Offer
{
     /* ... */
    [Column(TypeName = "DECIMAL(9,4")]
    public decimal Price { get; set; }
}
```

Allerdings ist der Datentyp abhängig von der Datenbank. So wird z. B. in Oracle *VARCHAR2* zur Speicherung von strings verwendet. Bei numerischen Werten gibt es mit *DECIMAL* und *NUMBER* (Oracle) ebenfalls mehrere Möglichkeiten, die abhängig von der Datenbank unterstützt werden oder nicht.

Besser: Steuerung über entsprechende Annotations

Um die maximale Länge eines String Properties zu steuern, kann die Annotation *MaxLength* verwendet werden. Der EF Core Provider kann dadurch weiterhin den geeigneten Typ auswählen (*VARCHAR*, *VARCHAR2*, ...) und eine Längendefinition - wenn unterstützt- angeben.

Für die Angabe der Precision gibt es (in EF Core 6) keine Annotation. Daher muss dieser Wert in der Methode *OnModelCreating()* der Klasse *StoreContext* definiert werden.

```
public class Store
{
    /* ... */
    [MaxLength(255)]
    public string Name { get; set; }
}

public class Offer
{
     /* ... */
    public decimal Price { get; set; }   // No annotation for precision :( -->
OnModelCreating
}

public class StoreContext : DbContext
{
    /* ... */
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Offer>().Property(o => o.Price).HasPrecision(9, 4);
    }
}
```

Definieren von Indizes und mehrteiligen Schlüsseln

In einem *CREATE TABLE* Statement können Schlüssel auch aus mehreren Spalten bestehen. Durch die Methode *HasKey()* kann dies auch im Context definiert werden:

```
public class StoreContext : DbContext
{
    /* ... */
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Offer>().HasKey("StoreId", "ProductEan");
    }
}
```



In der Klasse *Offer* muss mit einer *ForeignKey* Annotation über den Navigations zu *Store* und *Product* der Name des Fremdschlüsselfeldes explizit gesetzt werden. Beispiel: `[ForeignKey("StoreId")]`

Wir werden allerdings mehrteilige Schlüssel vermeiden! Navigationen müssen - falls z. B. eine Bestellung auf ein Offer verweist - auch mehrteilig sein und immer händisch definiert werden. Daher verwenden wir eine bessere Technik: Wir belassen die auto increment Id und definieren einen *Unique Index*.

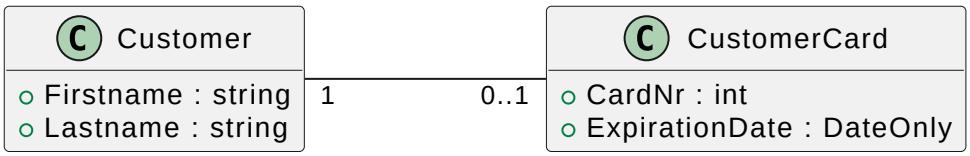
```
public class StoreContext : DbContext
{
    /* ... */
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Hinweis: Setze [ForeignKey("StoreId")] und [ForeignKey("ProductEan")]
        // über den Navigations, um den Nanem des FK genau zu spezifizieren.
        modelBuilder.Entity<Offer>().HasIndex("StoreId" "ProductEan").IsUnique();
    }
}
```

Durch den Unique Index ist sichergestellt, dass ein Produkt im selben Store nicht zu unterschiedlichen Preisen angeboten werden kann.

Unter https://docs.microsoft.com/en-us/ef/core/modeling/ gibt es eine sehr detaillierte Beschreibung der weiteren Einstellungsmöglichkeiten. Meist kommt man aber mit den hier gezeigten Möglichkeiten aus.

1:1 Beziehungen

Betrachten wir folgendes Klassendiagramm. Es zeigt Kunden, die eine Kundenkarte besitzen können.



Wir stellen bei der genaueren Analyse der 1:1 Beziehung fest:

- Es handelt sich um eine **1 : 0 oder 1** Beziehung, da ein Kunde auch keine Kundenkarte besitzen kann.
- Eine Kundenkarte kann nicht ohne Kunde existieren, umgekehrt jedoch schon.

Besonders die letzte Feststellung ist für den Begriff der *dependent entity* relevant. Wir müssen vor der konkreten Umsetzung identifizieren, welche Entity von der anderen abhängig ist. Diese Entity wird dann das Fremdschlüsselfeld beinhalten.

Die Modelklassen haben folgenden Aufbau:

```
public class Customer
{
    #pragma warning disable CS8618
    protected Customer() { }
    #pragma warning restore CS8618
    public Customer(string firstname, string lastname)
    {
        Firstname = firstname;
        Lastname = lastname;
    }

    public int Id { get; set; }
    public string Firstname { get; set; }
    public string Lastname { get; set; }
    public CustomerCard? CustomerCard { get; set; }
}

public class CustomerCard
{
    #pragma warning disable CS8618
    protected CustomerCard() { }
    #pragma warning restore CS8618
    public CustomerCard(int cardNr, DateOnly expirationDate, Customer customer)
    {
        CardNr = cardNr;
        ExpirationDate = expirationDate;
        Customer = customer;
    }

    public int Id { get; set; }
    public int CardNr { get; set; }
    public DateOnly ExpirationDate { get; set; }
    [ForeignKey("CustomerId")]
    public Customer Customer { get; set; }
```

```
}
```

Wir erkennen folgende Besonderheiten bei der Umsetzung:

- Das Property *CustomerCard* von *Customer* ist *nullable*. Ein Kunde muss schließlich keine Kundenkarte haben.
- Das Property *CustomerCard* ist nicht im Konstruktor von *Customer* enthalten. Wir müssen ja zuerst einen Kunden anlegen, um ihn eine Kundenkarte zuweisen zu können.
- Das Entity *Customer* ist das sogenannte *principal entity*. Es kann alleine existieren.

Das ForeignKey Attribut

Im Code fällt das Attribut *[ForeignKey("CustomerId")]* auf. Würden wir keinen ForeignKey definieren, entsteht beim Erstellen der Datenbank eine Exception:

```
System.InvalidOperationException : The dependent side could not be determined for the
one-to-one relationship between 'CustomerCard.Customer' and 'Customer.CustomerCard'. To
identify the dependent side of the relationship, configure the foreign key property. If
these navigations should not be part of the same relationship, configure them
independently via separate method chains in 'OnModelCreating'. See
https://go.microsoft.com/fwlink/?LinkId=724062 for more details.
```

Diese Exception sagt folgendes aus: EF Core muss wissen, in welcher Tabelle der Fremdschlüssel angelegt werden soll. Bei 1:n Beziehungen ist dies klar (auf der 1er Seite), bei 1:1 Beziehungen wird der FK üblicherweise beim *dependent entity* angelegt. Durch diese Annotation bestimmen wir, dass *CustomerCard* das *dependent entity* ist und dass dort der Fremdschlüssel angelegt werden kann.

Grenzen der 1:1 Beziehung

Die Datenbank hat keine eigenen Constraints für eine 1:1 Beziehung. Es wird ein "normaler" Fremdschlüssel wie bei einer 1:n Beziehung angelegt. Dadurch ist es aber möglich, dass ein Kunde mehrere Kundenkarten besitzt. Um dies zu verhindern, legt der OR Mapper einen *unique index* auf das Feld *CustomerId* in *CustomerCard* an.

Wir haben in diesem Fall eine "1 : 0 oder 1" Beziehung modelliert. Wie können wir eine exakte 1:1 Beziehung abbilden, sprich: Jeder Kunde *muss* eine Kundenkarte haben?

In der Doku von EF Core gibt es einen Hinweis, dass dies nicht möglich ist:

> A required relationship ensures that every dependent entity must be associated with some principal entity. However, a principal entity can always exist without any dependent entity. That is, a required relationship does not indicate that there will always be a dependent entity. There is no way in the EF model, and also no standard way in a relational database, to ensure that a principal is associated with a dependent. If this is needed, then it must be implemented in application (business) logic. See Required navigations for more informa-

> tion.
>
> https://learn.microsoft.com/en-us/ef/core/modeling/relationships/one-to-one

Alle möglichen Szenarien von Beziehungen und deren Konfiguration sind auf https://docs.microsoft.com/en-us/ef/core/modeling/relationships beschrieben.

Guid Werte als Alternate Key

Im Bereich der Webapplikationen wird oft eine eindeutige ID benötigt, die als Parameter einer URL verwendet werden kann. Die interne ID soll dabei nicht verwendet werden, da z. B. aus der URL */Store/Delete/4* leicht ableitbar ist, dass es auch einen Store 3 oder 5 geben kann.

Die GUID (oft auch UUID genannt) ist ein 128 bit langer Wert, der oft als Hexstring angegeben wird. Somit könnte die URL */Store/Delete/6d925883-f6f9-46c4-b47f-0bc80524d0b9* lauten. Hier ist es schon schwieriger, auf einen anderen Store zu schließen.

Die GUID wird mit dem .NET Typ *Guid* einfach im Model definiert. Da wir sie nicht zuweisen wollen, hat sie eine *private set* Methode.

```
public class Store
{
    public Store(string name, User? manager = null)
    {
        Name = name;
        Manager = manager;
        ManagerId = manager?.Id;
        Guid = Guid.NewGuid();
    }
    #pragma warning disable CS8618 // Non-nullable field must contain a non-null value
when exiting constructor. Consider declaring as nullable.
    protected Store() { }
    #pragma warning restore CS8618 // Non-nullable field must contain a non-null value
when exiting constructor. Consider declaring as nullable.
    public int Id { get; private set; }
    public Guid Guid { get; private set; }
    [MaxLength(255)]      // Produces NVARCHAR(255) in SQL Server
    public string Name { get; set; }
    public int? ManagerId { get; set; }
    public User? Manager { get; set; }
}
```

Konfiguration

Das GUID Property bekommt noch zwei spezielle Einstellungen: Mit *HasAlternateKey()* können wir EF Core veranlassen, dass es einen UNIQUE Index bekommt. Außerdem ist dies notwendig für die zweite Konfiguration. Mit *ValueGeneratedOnAdd()* wird EF Core - wie der Name schon sagt - selbstständig einen GUID Wert beim Einfügen des Datensatzes generieren. Dies funktioniert allerdings nur, wenn das Property mit *HasAlternateKey()* konfiguriert wurde.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    /* ... */
    modelBuilder.Entity<Store>().HasAlternateKey(s => s.Guid);
    modelBuilder.Entity<Store>().Property(s => s.Guid).ValueGeneratedOnAdd();
}
```

In der Klasse *Store* wird im Konstruktor trotzdem eine neue GUID zugewiesen, da wir vom Standpunkt des Domain Modellings nicht warten wollen, bis EF Core durch SaveChanges() diese ID geniert.

Es ist auch möglich, die Properties mit dem Namen *Guid* in *allen Entity Klassen* mit der oben beschriebenen Konfiguration zu versehen:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    /* ... */
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Generic config for all entities
        // ON DELETE RESTRICT instead of ON DELETE CASCADE
        foreach (var key in entityType.GetForeignKeys())
            key.DeleteBehavior = DeleteBehavior.Restrict;

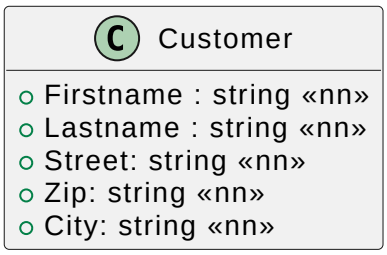
        foreach (var prop in entityType.GetDeclaredProperties())
        {
            // Define Guid as alternate key. The database can create a guid fou you.
            if (prop.Name == "Guid")
            {
                modelBuilder.Entity(entityType.ClrType).HasAlternateKey("Guid");
                prop.ValueGenerated = Microsoft.EntityFrameworkCore.Metadata
.ValueGenerated.OnAdd;
            }
            // Default MaxLength of string Properties is 255.
            if (prop.ClrType == typeof(string) && prop.GetMaxLength() is null) prop
.SetMaxLength(255);
            // Seconds with 3 fractional digits.
            if (prop.ClrType == typeof(DateTime)) prop.SetPrecision(3);
            if (prop.ClrType == typeof(DateTime?)) prop.SetPrecision(3);
        }
    }
}
```

Vererbung und andere OOP Konzepte

Value Objects

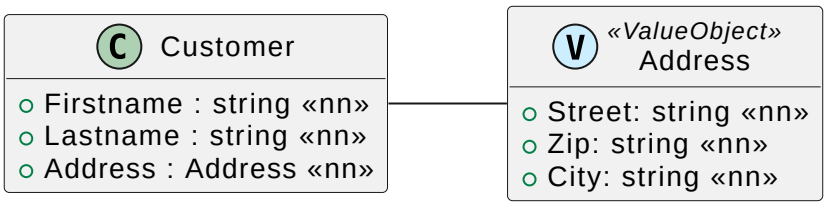
Einbinden eines (1) value objects

Oft bestehen Entities aus Properties, die thematisch eng zusammen gehören. So besteht eine Adresse aus Straße, PLZ und Ort.



PlantUML Source

Im OOP Design würden wir mit diesen 3 Properties einen eigenen Typ definieren, damit wir ihn im Programmcode öfters verwenden können.



PlantUML Source

Mit der 1:1 Beziehung haben wir schon das Wissen, diese Relation mit 2 Tabellen in der Datenbank darzustellen. Daraus ergibt sich jedoch ein Nachteil, dass beim Lesen des Kunden samt Adresse ein JOIN durchgeführt werden muss. Effektiver wäre es, wenn wir zwar getrennte Typen für Customer und Adresse verwenden, diese aber in einer Datenbanktabelle gespeichert werden würden:

Firstname	Lastname	Address_Street	Address_Zip	Address_City
Max	Mustermann	Spengergasse 20	1050	Wien

Wir definieren zuerst die Modelklassen für *Customer* und *Address*. Die Klasse *Customer* ist eine Modelklasse. Sie hat daher einen protected default Konstruktor. Die Klasse *Address* ist ein *value object*. Sie hat **keinen primary key und auch keinen default Konstruktor**.

```
public class Customer
{
    protected Customer() { }
    public Customer(string firstname, string lastname, Address address)
    {
```

```
        Firstname = firstname;
        Lastname = lastname;
        Address = address;
    }
    public int Id { get; set; }
    public string Firstname { get; set; }
    public string Lastname { get; set; }
    public Address Address { get; set; }
}

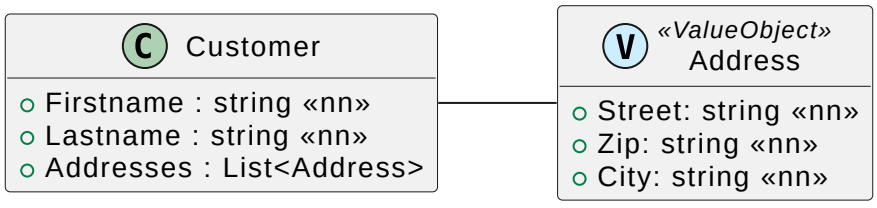
public class Address
{
    public Address(string street, string zip, string city)
    {
        Street = street;
        Zip = zip;
        City = city;
    }
    public string Street { get; set; }
    public string Zip { get; set; }
    public string City { get; set; }
}
```

Mit dieser Konfiguration würde EF Core allerdings eine 1:n Beziehung in der Datenbank erstellen und einen Fehler liefern, da Address keinen primary key besitzt. Wir müssen in der Methode *OnModelCreating* im Context das value object konfigurieren. Dafür gibt es die Methode *OwnsOne*.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>().OwnsOne(c => c.Address);
}
```

Listen von value objects

Wenn Kunden mehrere Adressen haben können, brauchen wir eine Liste von Adressen.



PlantUML Source

Die Abbildung in den Modelklassen erfolgt über eine Liste. Das Entity *Customer* besitzt keine Liste von Adressen im Konstruktor, da wir diese nach dem Anlegen befüllen.

```
public class Customer
{
```

```
    protected Customer() { }
    public Customer(string firstname, string lastname)
    {
        Firstname = firstname;
        Lastname = lastname;
    }
    public int Id { get; set; }
    public string Firstname { get; set; }
    public string Lastname { get; set; }
    public List<Address> Addresses { get; } = new();
}

public class Address
{
    public Address(string street, string zip, string city)
    {
        Street = street;
        Zip = zip;
        City = city;
    }
    public string Street { get; set; }
    public string Zip { get; set; }
    public string City { get; set; }
}
```

Nun müssen wir das value object mit *OwnsMany* im Context konfigurieren:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>().OwnsMany(c => c.Addresses).HasKey("Id");
}
```

Interne Abbildung in der Datenbank

Eine Liste von value objects kann nicht direkt in der Customer Tabelle gespeichert werden. Deswegen legt EF Core intern eine eigene Tabelle *Address* an.

```
CREATE TABLE "Address" (
    "Id" INTEGER NOT NULL CONSTRAINT "PK_Address" PRIMARY KEY AUTOINCREMENT,
    "Street" TEXT NOT NULL,
    "Zip" TEXT NOT NULL,
    "City" TEXT NOT NULL,
    "CustomerId" INTEGER NOT NULL,
    CONSTRAINT "FK_Address_Customers_CustomerId" FOREIGN KEY ("CustomerId") REFERENCES
"Customers" ("Id") ON DELETE CASCADE
);
```



Ohne die Option *HasKey("Id")* würde EF Core einen zusammengesetzten Schlüssel aus *Id* und *CustomerId* anlegen. SQLite unterstützt aber keine zusammengesetzten Schlüssel mit einem AUTO_INCREMENT Teil. Es würde die Fehlermel-

> dung `SqliteException: SQLite Error 19: 'NOT NULL constraint failed: Address.Id'.` beim Einfügen einer Adresse entstehen.

Converter

Eingebaute Converter

In der vorigen Übung mussten wir die enum *Weekday* als *string* in der Datenbank speichern. Das hat folgenden Grund: Enums sind Zahlenwerte. Ohne Converter würde statt *MO* einfach der Wert 0 gespeichert werden. Ändern wir nun die enum im Code, würden wir auch den Wochentag ändern. Um das zu verhindern, wollen wir die *string Repräsentation* speichern. Hier gibt es einen vordefinierten Converter, den wir im Context konfigurieren können:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OpeningHour>().Property(o => o.Weekday).HasConversion<
string>();
}
```

HasConversion<string>() bedeutet, dass einfach die *ToString()* Methode vor der Speicherung aufgerufen wird. Nach der Speicherung wird mit *Parse* der Wert im Objekt wiederhergestellt.

Userdefinierte Converter

Die vorige Technik hat bei manchen Datentypen einen Nachteil: Es wird die locale Einstellung des Servers verwendet, der die App ausführt. Dies kann zu unerwarteten Ergebnissen führen, wenn mehrere Server auf die Datenbank zugreifen. Wir können daher auch einen benutzerdefinierten Converter schreiben, der den zu speicherten Wert selbst festlegt.

Wir möchten nun die Werte *From* und *To* in *OpeningHour* aus dem letzten Übungsbeispiel nicht als Zeitwert, sondern auf Minute genau als String speichern. Der Converter besteht aus 2 Argumenten:

- Das erste Argument konvertiert den Objektwert in den Datenbankwert (in unserem Fall *string*).
- Das zweite Argument konvertiert den Datenbankwert in den Objektwert (*TimeOnly*).

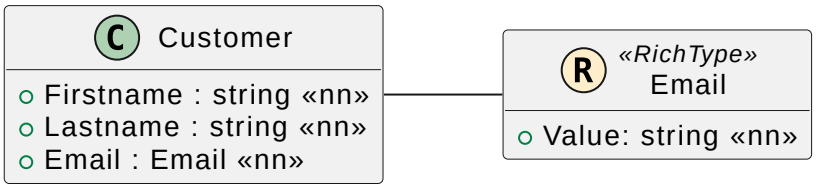
```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OpeningHour>().Property(o => o.From).HasConversion(
        objVal => objVal.ToString("HH:mm"),
        dbVal => TimeOnly.Parse(dbVal));
    modelBuilder.Entity<OpeningHour>().Property(o => o.To).HasConversion(
        objVal => objVal.ToString("HH:mm"),
        dbVal => TimeOnly.Parse(dbVal));
}
```

Das Ergebnis ist nun ein String der Form *HH:mm* in der Datenbanktabelle *OpeningHour*:

Id	StoreDetailId	Weekday	From	To
1	1	MO	08:00	16:00

Rich types

Wir möchten nun in der Entity *Customer* die E-Mail Adresse speichern. Strings haben den Nachteil, dass jeder Wert zulässig ist. Bei E-Mail Adressen wird allerdings ein bestimmter Aufbau verlangt. Um nur gültige E-Mail Adressen anlegen zu können, definieren wir einen eigenen Typ für *Email*:



PlantUML Source

In der Implementierung können wir die Klasse *MailAddress* aus .NET verwenden. Diese prüft automatisch, ob die E-Mail Adresse korrekt ist.

```
public class Customer
{
    protected Customer() { }
    public Customer(string firstname, string lastname, Email email)
    {
        Firstname = firstname;
        Lastname = lastname;
        Email = email;
    }

    public int Id { get; set; }
    public string Firstname { get; set; }
    public string Lastname { get; set; }
    public Email Email { get; set; }
}

public class Email
{
    public Email(string value)
    {
        Value = new System.Net.Mail.MailAddress(value);
    }
    public System.Net.Mail.MailAddress Value { get; }
}
```

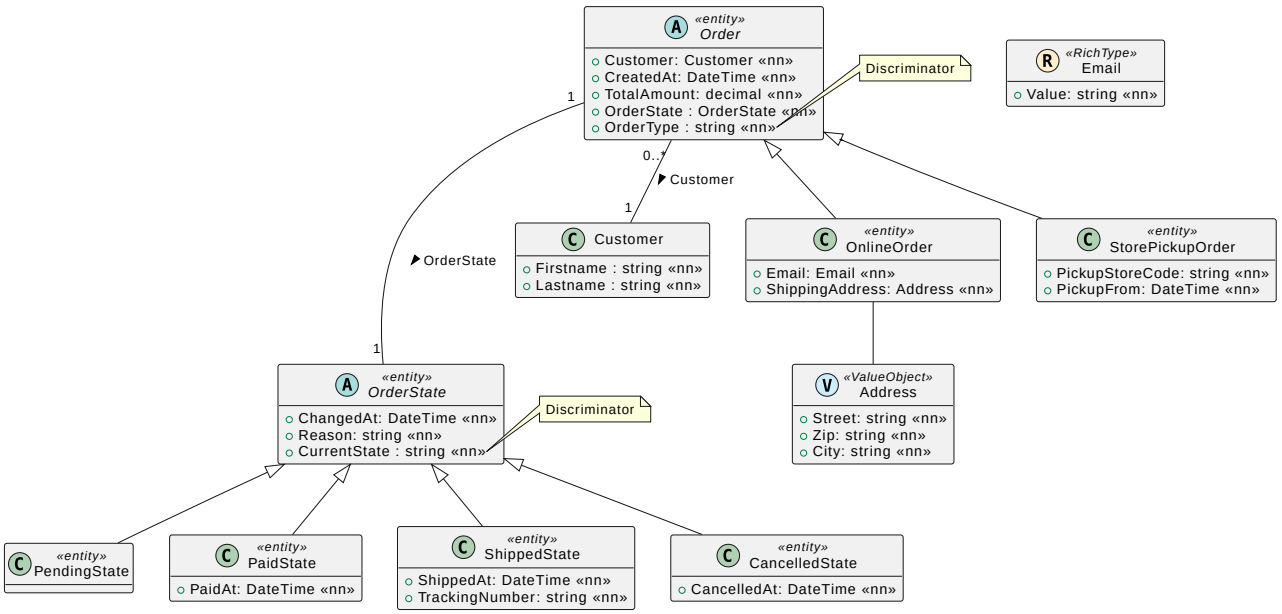
So können wir die Klasse *Customer* allerdings nicht mit EF Core in der Datenbank abbilden. Schließlich weiß EF Core nicht, welcher Wert für den user defined type *Email* in die Datenbank

geschrieben werden soll. Ein Converter behebt dieses Problem, und sorgt für eine definierte Umwandlung in einen String:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>().Property(c => c.Email).HasConversion(
        objVal => objVal.Value.Address,
        dbVal => new Email(dbVal));
}
```

Vererbung

Ein wesentliches Konzept der OOP ist die *Vererbung*. Gemeinsam mit den vorher gelernten Konzepten können wir nun ein kleines Domainmodel eines Bestellmanagements modellieren:



PlantUML Source

Umsetzung der Modelklassen

Die Klasse *Order* wird syntaktisch wie gewohnt umgesetzt, hat aber einige Besonderheiten:

```
public abstract class Order  ①
{
    protected Order() { }  ②
    protected Order(Customer customer, DateTime createdAt, decimal totalAmount)
    {
        Customer = customer;
        CreatedAt = createdAt;
        TotalAmount = totalAmount;
    }

    public int Id { get; private set; }
    public DateTime CreatedAt { get; set; }
```

```
    public Customer Customer { get; set; }
    public OrderState? OrderState { get; set; }  ③
    public decimal TotalAmount { get; set; }
    public string OrderType { get; private set; } = default!;  ④
}
```

① Im Model ist die Klasse *Order* als *abstract* ausgewiesen. Schließlich macht es keinen Sinn, eine allgemeine Order zu erstellen. Daher verwenden wir das Schlüsselwort *abstract*.

② Der gewohnte default Konstruktor für EF Core wird ebenfalls angelegt.

③ Die 1:1 Beziehung von *Order* zu *OrderState* können wir nur als 1:0 oder 1 definieren. Order ist die *principal entity*, und hat daher eine nullable Navigation zu *OrderState*. Ein Order hat daher *0 oder 1 States*. Dieses Feld wird auch nicht im Konstruktor angefordert, da wir zunächst einmal eine Order in der Datenbank anlegen müssen.

④ Im Modell gibt es den Hinweis *Discriminator* beim Feld *OrderType*. Der Discriminator speichert einfach den Typnamen des Orders (*OnlineOrder* oder *StorePickupOrder*). Daher wird er auch mit null (*default!*) initialisiert und hat keinen öffentlichen Setter. Er wird von EF Core selbstständig verwaltet.



Initialisiere den Discriminator nicht mit *string.Empty* oder anderen Werten außer *null*. EF Core kann den Datensatz dann nicht korrekt zurücklesen.

Die Klassen *OnlineOrder* und *StorePickupOrder* werden nun angelegt:

```
public class OnlineOrder : Order
{
    public OnlineOrder(
        Customer customer, DateTime createdAt, decimal totalAmount,
        Email email, Address shippingAddress)
        : base(customer, createdAt, totalAmount)
    {
        Email = email;
        ShippingAddress = shippingAddress;
    }

    protected OnlineOrder() { }

    public Email Email { get; set; }
    public Address ShippingAddress { get; set; }
}

public class StorePickupOrder : Order
{
    public StorePickupOrder(
        Customer customer, DateTime createdAt, decimal totalAmount,
        string pickupStoreCode, DateTime pickupFrom)
        : base(customer, createdAt, totalAmount)
    {
        PickupStoreCode = pickupStoreCode;
        PickupFrom = pickupFrom;
    }
```

```
    protected StorePickupOrder() { }

    public DateTime PickupFrom { get; set; }
    public string PickupStoreCode { get; set; }
}

public class Email
{
    public Email(string value)
    {
        Value = new System.Net.Mail.MailAddress(value);
    }

    public System.Net.Mail.MailAddress Value { get; }
}

public class Address
{
    public Address(string street, string zip, string city)
    {
        Street = street;
        Zip = zip;
        City = city;
    }

    public string City { get; set; }
    public string Street { get; set; }
    public string Zip { get; set; }
}
```

Beachte den Konstruktor. Er leitet die Felder der Basisklasse an den *public* Konstruktor der Basisklasse weiter. Die Klassen *Email* als rich type und *Address* als value object werden wie vorher erklärt angelegt.



Verwende immer die Konstruktorweiterleitung zum public Konstruktor der Basisklasse. Sonst wird der protected default Konstruktor aufgerufen und die Felder in *Order* haben den Wert *null*!

Die Unterklassen haben **kein Property Id**, da es von *Order* vererbt wird!

Unseren OrderState setzen wir ebenfalls mit Vererbung um. Da *OrderState* das *dependent entity* der 1:1 Beziehung zu *Order* ist, besitzt es das *ForeignKey* Attribut.

```
public abstract class OrderState
{
    protected OrderState() { }
    protected OrderState(Order order, DateTime changedAt, string reason)
    {
        Order = order;
        ChangedAt = changedAt;
        Reason = reason;
    }
```

```
    public int Id { get; private set; }
    public DateTime ChangedAt { get; set; }
    [ForeignKey("OrderId")]
    public Order Order { get; set; }
    public string Reason { get; set; }
    // Discriminator, managed by EF Core.
    public string CurrentState { get; private set; } = default!;
}

public class PendingState : OrderState
{
    protected PendingState() { }
    public PendingState(Order order, DateTime changedAt, string reason)
        : base(order, changedAt, reason) { }

}
public class PaidState : OrderState
{
    protected PaidState() { }
    public PaidState(Order order, DateTime changedAt, string reason, DateTime paidAt)
        : base(order, changedAt, reason)
    {
        PaidAt = paidAt;
    }
    public DateTime PaidAt { get; set; }
}
public class ShippedState : OrderState
{
    protected ShippedState() { }
    public ShippedState(Order order, DateTime changedAt, string reason, DateTime
shippedAt,
        string trackingNumber) : base(order, changedAt, reason)
    {
        ShippedAt = shippedAt;
        TrackingNumber = trackingNumber;
    }
    public DateTime ShippedAt { get; set; }
    public string TrackingNumber { get; set; }
}
public class CancelledState : OrderState
{
    protected CancelledState() { }
    public CancelledState(Order order, DateTime changedAt, string reason, DateTime
cancelledAt)
        : base(order, changedAt, reason)
    {
        CancelledAt = cancelledAt;
    }
    public DateTime CancelledAt { get; set; }
}
```

Konfiguration des Discriminator Feldes

By convention würde EF Core ein Feld mit dem Namen *Discriminator* als solchen automatisch

erkennen. Da wir aber *OrderType* bzw. *CurrentState* verwenden, müssen wir mit der Methode *HasDiscriminator()* in *OnModelCreating* im Context eine Konfiguration vornehmen:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrderState>().HasDiscriminator<string>(o => o.CurrentState);
    modelBuilder.Entity<Order>().HasDiscriminator<string>(o => o.OrderType)
        .HasValue<OnlineOrder>("Online")
        .HasValue<StorePickupOrder>("Pickup");

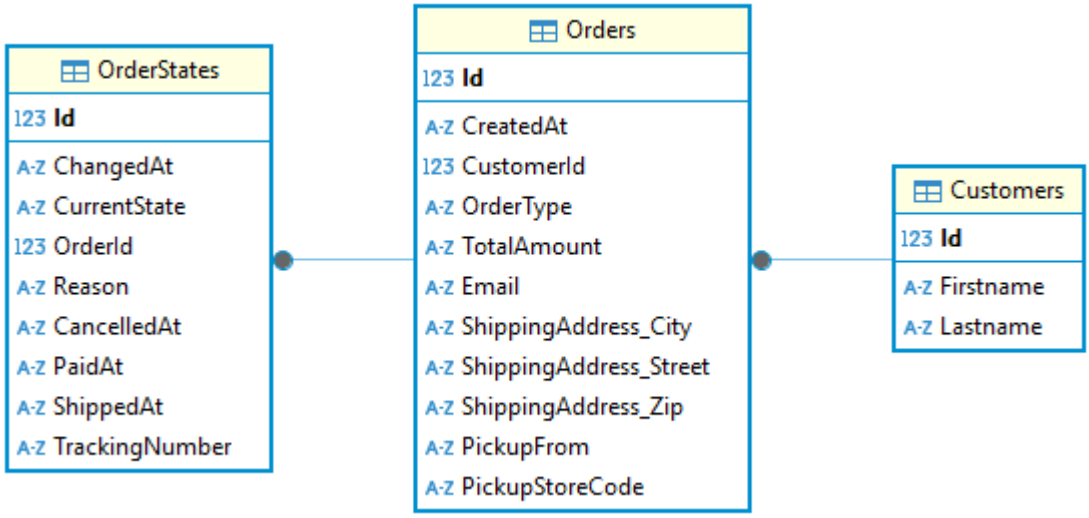
    // Other config.
    modelBuilder.Entity<OnlineOrder>().Property(o => o.Email).HasConversion(
        objVal => objVal.Value.Address,
        dbVal => new Email(dbVal));
    modelBuilder.Entity<OnlineOrder>().OwnsOne(o => o.ShippingAddress);
}
```

Wir können den Discriminator auf 2 Arten definieren:

- In der ersten Form (ohne *HasValue()*) wird einfach der Typname als String verwendet (also *PaidState*, *PendingState*, *ShippedState* und *CancelledState*).
- In der zweiten Form können wir eigene Werte für das Datenbankfeld verwenden.

Das erzeugte Datenbankschema

Das erzeugte Datenbankschema hat überraschend wenig mit unserem Modelklassen zu tun. **Es werden nur 3 Tabellen erzeugt.**



Bei genauerer Betrachtung erkennen wir aber:

- Address ist ein *value object*. Es findet sich in den Spalten *ShippingAddress_City*, *ShippingAddress_Street* und *ShippingAddress_Zip* wieder.
- Email ist ein *rich type*. Er wird mit Hilfe des Converters als String abgebildet.
- Die Vererbung wird nur mit einer Tabelle, nämlich der Basisklasse abgebildet. Dies wird als

table per hierarchy (TPH) bezeichnet. Sie besitz mit _OrderType den Discriminator. Die Felder der Unterklassen von *Order* (*Email ShippingAddress…*, *PickupFrom* und *PickupStoreCode*, sind stets *nullable*. In der Datenmodellierung wird dies als *rollup* bezeichnet.

```
CREATE TABLE "Orders" (
    "Id" INTEGER NOT NULL CONSTRAINT "PK_Orders" PRIMARY KEY AUTOINCREMENT,
    "CreatedAt" TEXT NOT NULL,
    "CustomerId" INTEGER NOT NULL,
    "OrderType" TEXT NOT NULL,
    "TotalAmount" TEXT NOT NULL,
    "Email" TEXT NULL,
    "ShippingAddress_City" TEXT NULL,
    "ShippingAddress_Street" TEXT NULL,
    "ShippingAddress_Zip" TEXT NULL,
    "PickupFrom" TEXT NULL,
    "PickupStoreCode" TEXT NULL,
    CONSTRAINT "FK_Orders_Customers_CustomerId" FOREIGN KEY ("CustomerId") REFERENCES
"Customers" ("Id") ON DELETE CASCADE
);
```

Table 1. Orders

Id	CreatedAt	CustomerId	OrderType	TotalAmount	Email	ShippingAddress_City	ShippingAddress_Street	ShippingAddress_Zip	PickupFrom	PickupStoreCode
1	2025-11-05 13:00:00	1	Online	123.45	ada@example.org	Wien	Main St 1	1010	[NULL]	[NULL]

Da eine *OnlineOrder* angelegt wurde (siehe *OrderType*), haben die Felder *PickupFrom* und *PickupStoreCode* den Wert NULL.

Table 2. OrderStates

Id	ChangedAt	CurrentState	OrderId	Reason	CancelledAt	PaidAt	ShippedAt	TrackingNumber
1	2025-11-05 14:00:00	PaidState	1	Payment received	[NULL]	2025-11-05 14:00:00	[NULL]	[NULL]

Da ein *PaidState* angelegt wurde (siehe *CurrentState*), haben die Felder *CancelledAt*, *ShippedAt* und *TrackingNumber* den Wert NULL.

Table 3. Customers

Id	Firstname	Lastname
1	Ada	Lovelace

Musterdaten erzeugen mit Bogus

In der Enwicklung eines Projektes möchte man Programmteile wie z. B. das Frontend mit eingetragenen Daten testen. Für diesen Zweck gibt es in den verschiedenen Programmiersprachen sogenannte *Faker* wie *fakerjs*, etc. In .NET gibt es das NuGet Paket Bogus. Um es zu verwenden, muss es zuerst installiert werden. Dies geht am Schnellsten über das Terminal im Ordner des C# Application Projektes mit

```
dotnet add package Bogus --version 35.6.1
```

Im Musterprogramm *CodeFirstDemo* gibt es in der Datei StoreContext.cs die Methode `Seed()`. Sie zeigt die Verwendung des Paketes. Eine ausführliche Beschreibung ist im Repo auf github.com/bchavez/Bogus zu sehen.

```
1  public void Seed()
2  {
3      Randomizer.Seed = new Random(1335);     ①
4      var productCategories = new Faker<ProductCategory>("de")     ②
5          .CustomInstantiator(f =>
6              new ProductCategory(name: f.Commerce.ProductAdjective())  ③
7          {
8              NameEn = f.Commerce.ProductAdjective().OrNull(f, 0.5f)     ④
9          })
10         .Generate(5)     ⑤
11         .ToList();       ⑥
12     ProductCategories.AddRange(productCategories);
13     SaveChanges();
14 }
```

① Um bei jedem Programmstart gleiche Daten zu erzeugen, setzen wir ein fixes Seed. Das ist wichtig, um Fehler zu finden. Sonst würde beim nächsten Programmstart der Fehler vielleicht nicht mehr auftreten, da andere Daten generiert werden.

② Zuerst speichern wir die Daten in eine Liste vom Typ `List<ProductCategory>`. Dadurch können wir später auf die Daten zugreifen, wenn wir sie brauchen. Außerdem setzen wir mit *de* die locale Einstellung. So werden z. B. Namen mit Umlauten generiert.

③ Da die Klasse `ProductCategory` keinen public default Konstruktor besitzt, erzeugen wir die Instanz mit `CustomInstantiator()`.

④ Der Faker bietet verschiedene Profile für Musterdaten an. Im Property `Commerce` finden wir z. B. eine Methode `ProductAdjective()`, um Produkteigenschaften zu generieren. Da das Feld auch null sein kann, müssen wir auch solche Werte generieren. Die Methode `OrNull()` definiert, wie viel Prozent der Werte *null* sein sollen. Hier sind es 50%.

⑤ Nun wollen wir *n* Werte generieren. Mit `Generate(n)` können wir die Anzahl der zu generierenden Datensätze angeben.

⑥ Da `Generate()` nur einen Enumerator zurückgibt, müssen wir mit `ToList()` die Daten fix in eine Liste ablegen.

Wenn wir die Tabelle *ProductCategory* nun in einem Datenbankeditor betrachten, sehen wir die generierten Daten:

Table 4. ProductCategory

Id	Name	NameEn
1	Rustic	Awesome
2	Sleek	[NULL]
3	Ergonomic	[NULL]
4	Awesome	Refined
5	Fantastic	[NULL]

Die verschiedenen Profile, die für die Musterdatengenerierung zur Verfügung stehen, sind in der Doku oder in der Klasse `Faker` zu sehen. Am Wichtigsten ist das Property `Random`. Es gibt einen `Randomizer` zurück, der zufällige Zahlen, Listenelemente, etc. generieren kann. Im nächsten Punkt sehen wir, warum dies nützlich ist.

▼ *Klasse Faker*

```
public class Faker : ILocaleAware, IHasRandomizer, IHasContext
{
    public static bool DefaultStrictMode;
    public static int GlobalUniqueIndex;
    public int IndexFaker;
    public int IndexVariable;
    protected SeedNotifier Notifier;
    public Faker(string locale = "en");
    public Lorem Lorem { get; set; }
    public Images Image { get; set; }
    public Finance Finance { get; set; }
    public Address Address { get; set; }
    public Date Date { get; set; }
    public Company Company { get; set; }
    public Internet Internet { get; set; }
    public DataSets.Database Database { get; set; }
    public DataSets.System System { get; set; }
    public Name Name { get; set; }
    public Rant Rant { get; set; }
    public Vehicle Vehicle { get; set; }
    public Music Music { get; set; }
    public string Locale { get; set; }
    public int UniqueIndex { get; }
    public Commerce Commerce { get; set; }
    public PhoneNumbers Phone { get; set; }
    public Person Person { get; }
    public DateTime? DateTimeReference { get; set; }
    public Hacker Hacker { get; set; }
    public Hashids Hashids { get; set; }
    public int IndexGlobal { get; }
    public Randomizer Random { get; set; }
    public IList<T> Make<T>(int count, Func<int, T> action);
    public IList<T> Make<T>(int count, Func<T> action);
```

```
    public IEnumerable<T> MakeLazy<T>(int count, Func<int, T> action);
    public IEnumerable<T> MakeLazy<T>(int count, Func<T> action);
    public string Parse(string str);
    public T PickRandom<T>() where T : struct, Enum;
    public T PickRandom<T>(params T[] items);
    public T PickRandom<T>(List<T> items);
    public T PickRandom<T>(ICollection<T> items);
    public T PickRandom<T>(IList<T> items);
    public T PickRandom<T>(IEnumerable<T> items);
    public IEnumerable<T> PickRandom<T>(IEnumerable<T> items, int amountToPick);
    public T PickRandomParam<T>(params T[] items);
    public T PickRandomWithout<T>(params T[] exclude) where T : struct, Enum;
}
```

▼ *Klasse Randomizer*

```
public class Randomizer
{
    public static Random Seed;
    protected Random localSeed;
    public Randomizer();
    public Randomizer(int localSeed);
    public string AlphaNumeric(int length);
    public T ArrayElement<T>(T[] array);
    public BValue ArrayElement(BArray props, int? min = null, int? max = null);
    public string ArrayElement(Array array);
    public T[] ArrayElements<T>(T[] array, int? count = null);
    public bool Bool(float weight);
    public bool Bool();
    public byte Byte(byte min = 0, byte max = 255);
    public byte[] Bytes(int count);
    public char Char(char min = '\0', char max = '\uffff');
    public char[] Chars(char min = '\0', char max = '\uffff', int count = 5);
    public string ClampString(string str, int? min = null, int? max = null);
    public T CollectionItem<T>(ICollection<T> collection);
    public decimal Decimal(decimal min = 0.0M, decimal max = 1.0M);
    public int[] Digits(int count, int minDigit = 0, int maxDigit = 9);
    public double Double(double min = 0, double max = 1);
    public T Enum<T>(params T[] exclude) where T : struct, Enum;
    public T[] EnumValues<T>(int? count = null, params T[] exclude) where T : Enum;
    public int Even(int min = 0, int max = 1);
    public float Float(float min = 0, float max = 1);
    public Guid Guid();
    public string Hash(int length = 40, bool upperCase = false);
    public string Hexadecimal(int length = 1, string prefix = "0x");
    public int Int(int min = int.MinValue, int max = int.MaxValue);
    public T ListItem<T>(List<T> list);
    public T ListItem<T>(IList<T> list);
    public List<T> ListItems<T>(IList<T> items, int? count = null);
    public IList<T> ListItems<T>(List<T> items, int? count = null);
    public long Long(long min = long.MinValue, long max = long.MaxValue);
    public int Number(int max);
    public int Number(int min = 0, int max = 1);
    public int Odd(int min = 0, int max = 1);
```

```
    public string RandomLocale();
    public string Replace(string format);
    public string ReplaceNumbers(string format, char symbol = '#');
    public string ReplaceSymbols(string format, char symbol, Func<char> func);
    public sbyte SByte(sbyte min = sbyte.MinValue, sbyte max = sbyte.MaxValue);
    public short Short(short min = short.MinValue, short max = short.MaxValue);
    [IteratorStateMachine(typeof(Randomizer.<Shuffle>d__47<>))]
    public IEnumerable<T> Shuffle<T>(IEnumerable<T> source);
    public string String(int? length = null, char minChar = '\0', char maxChar =
'\uffff');
    public string String(int minLength, int maxLength, char minChar = '\0', char
maxChar = '\uffff');
    public string String2(int minLength, int maxLength, string chars =
"abcdefghijklmnopqrstuvwxyz");
    public string String2(int length, string chars = "abcdefghijklmnopqrstuvwxyz");
    public uint UInt(uint min = 0, uint max = uint.MaxValue);
    public ulong ULong(ulong min = 0, ulong max = ulong.MaxValue);
    public ushort UShort(ushort min = 0, ushort max = ushort.MaxValue);
    public string Utf16String(int minLength = 40, int maxLength = 80, bool
excludeSurrogates = false);
    public Guid Uuid();
    public T WeightedRandom<T>(T[] items, float[] weights);
    public string Word();
    public string Words(int? count = null);
    public string[] WordsArray(int min, int max);
    public string[] WordsArray(int count);
}
```

Generieren von abhängigen Daten

Die Klasse `Product` verwendet die Klasse `ProductCategory`, da hier eine 1:n Beziehung vorliegt. Wenn Musterdaten für `Product` generiert werden, muss also eine zufällige `ProductCategory` gewählt werden.

```
1 var products = new Faker<Product>("de")
2     .CustomInstantiator(f => new Product(
3         ean: f.Random.Int(1000, 9999),
4         name: f.Commerce.ProductName(),
5         productCategory: f.Random.ListItem(productCategories)))    ①
6     .Generate(25)
7     .DistinctBy(p => p.Ean)    ②
8     .ToList();
9 Products.AddRange(products);
10 SaveChanges();
```

① Die Methode `ListItem()` der Klasse `Randomizer` liefert ein zufälliges Element einer Liste. So generieren wir einen gültigen Fremdschlüsselwert.

② Da das Property `Ean` eindeutig sein muss, brauchen wir `DistinctBy`. Beim Generieren von Zufallszahlen ist es nämlich möglich, dass eine Zahl nochmals auftritt (*"ziehen mit Zurücklegen"*). Es werden in diesem Fall zwar nicht genau 25 Elemente generiert, das ist jedoch für uns nicht wichtig.

Generieren von Zeitwerten, mehrteilige unique constraints

Interessant ist das Generieren von Offers. Hier werden Zahlen und Datumswerte generiert.

```
1 var offers = new Faker<Offer>("de")
2     .CustomInstantiator(f => new Offer(
3         product: f.Random.ListItem(products),
4         store: f.Random.ListItem(stores),
5         price: Math.Round(f.Random.Decimal(100, 999), 2),    ①
6         lastUpdate: new DateTime(2021, 1, 1)
7             .AddSeconds(f.Random.Int(0, 180 * 86_400))))     ②
8     .Generate(125)
9     .DistinctBy(o => new { o.Store, o.Product })      ③
10    .ToList();
```

① Normalerweise haben die generierten `decimal` Werte viele Nachkommastellen. Um realistische Preise zu generieren, verwenden wir `Math.Round()`.

② Wir wollen den Zeitstempel auf Sekunde genau generieren, ohne Millisekunden. Daher verwenden wir einen Trick: Beginnend mit 1.01.2021 addieren wir ganzzahlig Sekunden dazu. 1 Tag hat 86.400 Sekunden, somit ergibt `180 * 86_400` einen Zeitraum von 180 Tagen.

③ Da wir ein UNIQUE Constraint über die Spalten *StoreId* und *ProductEan* definiert haben, müssen wir sicherstellen, dass nicht doppelte Werte auftreten.

Generieren von einer gewissen Anzahl an abhängigen Daten

Manchmal möchten wir bei einer 1:n Beziehung sicherstellen, dass für jedes Element der 1-Seite mindestens n Elemente der n-Seite generiert werden. Dafür können wir die Collection Navigation in ProductCategory verwenden und dort mit der entsprechenden Anzahl an Produkten füllen. Mit *GenerateForever()* können wir unendlich viele Werte generieren und diese dann mit *Take(n)* auf die gewünschte Anzahl beschränken. Da wir vorher ein *DistinctBy()* auf die eindeutigen Felder anwenden, können wir sicherstellen, dass die gewünschten eindeutigen Werte generiert werden.



Verwende *GenerateForever()* immer mit einer Begrenzung wie *Take(n)*, da sonst unendlich viele Werte generiert werden. Wenn das Dataset zu klein ist, um die gewünschten eindeutigen Werte zu generieren, läuft die Methode ebenfalls endlos. Daher verwenden wir - wenn möglich - das *DistinctBy()* im Nachhinein und nehmen eine kleinere Anzahl an Werten in Kauf.

```
var productCategories = new Faker<ProductCategory>("de")
    .CustomInstantiator(f =>
    {
        var name = f.Commerce.Categories(1)[0];
        var nameEn = f.Commerce.Categories(1)[0].OrNull(f, 0.5f);
        var productCategory = new ProductCategory(name, nameEn);
```

```
        var products = new Faker<Product>("de")
            .CustomInstantiator(f =>
            {
                var ean = f.Random.Int(10000, 99999);  // PK
                var name = $"{f.Commerce.ProductName()} {f.Commerce.ProductName()}";
                return new Product(ean, name, productCategory);
            })
            .GenerateForever()
            .DistinctBy(p => p.Ean)
            .DistinctBy(p => p.Name)
            .Take(f.Random.Int(9, 10))   // Zwischen 9 und 10 Produkte pro Kategorie
            .ToList();

        productCategory.Products.AddRange(products);
        return productCategory;
    })
    .Generate(10)
    .DistinctBy(p => p.Name)
    .ToList();
```

Die Anzahl der generierten Produkte pro Kategorie ist zufällig zwischen 9 und 10.

Id	Name	Count
1	Beauty	9
2	Movies	9
3	Computers	10
4	Music	10
5	Outdoors	9
6	Automotive	9
7	Toys	10
8	Home	9

Durch diese Technik haben wir allerdings keine Liste aller Produkte mehr, da wir sie pro Kategorie generiert haben. Mit *SelectMany()* können wir jedoch alle Produkte aus *productCategories* in einer Liste sammeln.

```
var products = productCategories.SelectMany(p => p.Products).ToList();
```

Generieren von DateTime Werten mit Minuten- und Sekundenauflösung

Wir betrachten folgende Berechnung:

```
125247 / 100 * 100
```

Das Ergebnis ist mathematsch klarerweise wieder 125247, da wir durch 100 teilen und danach wieder mit 100 multiplizieren. Da aber in C#, Java, C++ und vielen anderen Programmiersprachen die Division von Ganzzahlen ebenfalls eine Ganzzahl zurückgibt, wird hier zuerst 1252 berechnet. Multiplikation mit 100 ergibt dann 125200. Wir haben also den Wert auf Hunderter abgerundet.

Für DateTime Werte können wir diese Technik verwenden, um auf Minuten- oder Sekundenauflösung zu runden.

```
var dateTime = f.Date.Between(new DateTime(2024, 1, 1), new DateTime(2025, 1, 1));
dateTime = new DateTime(dateTime.Ticks / TimeSpan.TicksPerMinute * TimeSpan
.TicksPerMinute);
```

Ticks ist das interne Zeitmaß von DateTime Werten. Es ist die Anzahl der 100-Nanosekunden Intervalle seit dem 1.1.0001, ähnlich wie Unix Time oder dem Timestamp in Javascript.

Für "normale" Zahlen wie double oder decimal können wir *Math.Round()* verwenden, um auf eine bestimmte Anzahl an Nachkommastellen zu runden.

Für Profis: Generische Hilfsmethoden

Wir erkennen in den oberen Beispielen, dass sich einige Codeblöcke wiederholen:

- Instanzieren des Fakers vom Typ *T*
- *Generate()* und *ToList()*
- Eventuell auch *DistinctBy()*
- *AddRange()* und *SaveChanges()*

Die nachfolgenden Methoden *Generate()* und *GenerateDistinct()* bündeln diesen Ablauf für Entities vom Typ *T*:

```
public class StoreContext : DbContext
{
    // ...

    public void Seed()
    {
        // ...
    }

    private List<T> Generate<T>(Func<Faker, T> generator, int count) where T : class
    {
        var data = new Faker<T>("de")
            .CustomInstantiator(generator)
            .Generate(count)
            .ToList();
        var set = Set<T>();
        set.AddRange(data);
```

```
        SaveChanges();
        return data;
    }

    private List<T> GenerateDistinct<T, Tkey>(Func<Faker, T> generator, int count,
params Func<T, Tkey>[] distinctBys) where T : class
    {
        var dataEnumerable = new Faker<T>("de")
            .CustomInstantiator(generator)
            .GenerateForever();
        foreach (var distinctBy in distinctBys)
            dataEnumerable = dataEnumerable.DistinctBy(distinctBy);
        var data = dataEnumerable.Take(count).ToList();
        var set = Set<T>();
        set.AddRange(data);
        SaveChanges();
        return data;
    }
}
```

Nun können z. B. Entities mit weniger Codezeilen generiert und gespeichert werden:

```
// Generates 10 customers.
var candidates = Generate(f =>
{
    var lastName = f.Name.LastName();
    var email = f.Internet.Email();
    return new Customer(f.Name.FirstName(), lastName, email);
}, 10);
// Generates 5 departments, distinct by name.
var departments = GenerateDistinct(
    f => new Department(f.Commerce.Department()),
    5, d => d.Name);
// Generates 5 products, first distinct by ean, then by name.
var products = GenerateDistinct(
    f => new Product(f.Random.Int(10000, 99999), f.Commerce.ProductName()),
    5, p => p.Ean, p => p.Name);
```

Serviceklassen

Lambda Expressions

Eine App verwaltet Personen in einer Liste. Die Leitung möchte alle Personen aus Wien sehen. Dann alle Volljährigen. Dann alle, deren Nachname mit „B“ beginnt. Du schreibst drei Methoden, und alle drei sehen fast gleich aus: eine Schleife, ein `if`, ein `Add`. Nur die Bedingung im `if` ist anders. In diesem Kapitel lernst du, wie du **die Bedingung selbst als Parameter übergibst**. Dafür gibt es in C# **Lambda Expressions** (kurz: Lambdas). Sie sind die Grundlage für LINQ, mit dem du später Datenbanken abfragst.



Zum Selbst-Erarbeiten: Lambdas lesen und schreiben

In der interaktiven Präsentation zu Lambda Expressions siehst du Schritt für Schritt, wie aus einer Methode eine Lambda wird. Danach übst du mit sofortigem Feedback, welchen Typ (`Func` oder `Action`) eine Lambda hat und wie du sie kurz schreibst. Das trainiert genau das Lesen, das du für LINQ brauchst.

Projekt anlegen

Du brauchst eine Solution mit einer Application und einem Testprojekt. Führe diese Befehle in der Konsole aus. Unter macOS ersetzt du *rd*, *md* und *move* durch *rm -rf*, *mkdir* und *mv*.

```
rd /S /Q LambdaDemo
md LambdaDemo
cd LambdaDemo
md LambdaDemo.Application
cd LambdaDemo.Application
dotnet new classlib
cd ..
md LambdaDemo.Tests
cd LambdaDemo.Tests
dotnet new install xunit.v3.templates
dotnet new xunit3 -f net10.0
move global.json ..
dotnet reference add ..\LambdaDemo.Application
cd ..
dotnet new sln -f sln
dotnet sln add LambdaDemo.Application
dotnet sln add LambdaDemo.Tests
start LambdaDemo.sln
```

Lösche *Class1.cs* und *UnitTest1.cs*. Entferne in beiden Projektdateien die Zeile *ImplicitUsings*, im Testprojekt auch `<Using Include="Xunit" />`. Füge in beiden `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` hinzu. Was die einzelnen Befehle und Einstellungen bedeuten, steht im Kapitel Unit Tests mit xUnit.

Lege im Projekt *LambdaDemo.Application* den Record `Person` an:

Person.cs (im Projekt *LambdaDemo.Application*)

```
namespace LambdaDemo.Application;

public record Person(string FirstName, string LastName, string City, int Age);
```

Das Problem: dreimal fast dieselbe Methode

PersonFilter.cs (im Projekt *LambdaDemo.Application*)

```
using System.Collections.Generic;

namespace LambdaDemo.Application;

public static class PersonFilter
{
    public static List<Person> FilterByCity(List<Person> persons, string city)
    {
        var result = new List<Person>();
        foreach (Person p in persons)
        {
            if (p.City == city) { result.Add(p); }
        }
        return result;
    }

    public static List<Person> FilterAdults(List<Person> persons)
    {
        var result = new List<Person>();
        foreach (Person p in persons)
        {
            if (p.Age >= 18) { result.Add(p); }
        }
        return result;
    }
}
```

Das widerspricht dem Prinzip *DRY* (siehe Kapitel DRY: Don't Repeat Yourself). Mit einem `string`-Parameter kommen wir nicht weiter: Die Bedingungen sind ganz verschieden (`==` bei der Stadt, `>=` beim Alter). Was sich unterscheidet, ist nicht ein Wert, sondern **ein Stück Code**: die Bedingung im `if`.

Die Bedingung als Parameter

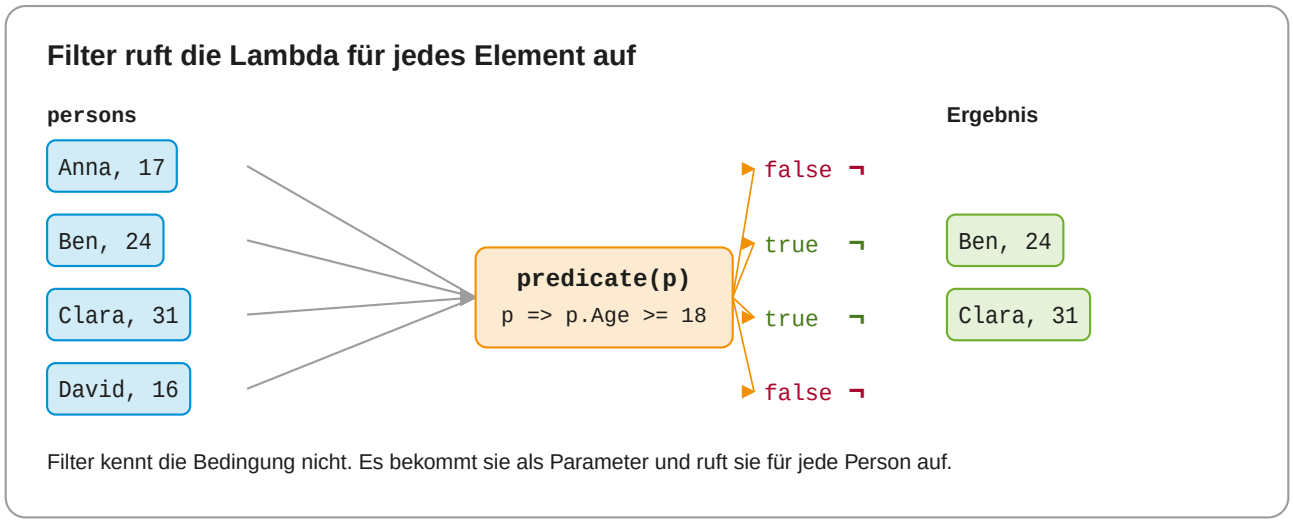
C# erlaubt es, eine Funktion als Parameter zu übergeben. Der Parameter `condition` hat den Typ `Func<Person, bool>`. Das bedeutet: eine Funktion, die eine `Person` bekommt und einen `bool` liefert.

```
public static List<Person> Filter(List<Person> persons, Func<Person, bool> condition)
{
```

```
    var result = new List<Person>();
    foreach (Person p in persons)
    {
        if (condition(p)) { result.Add(p); }
    }
    return result;
}
```

`condition(p)` ruft die übergebene Funktion auf, wie eine normale Methode. Für `Func` brauchst du `using System;`. Beim Aufruf schreibst du die Bedingung als **Lambda** direkt in die Klammern:

```
List<Person> fromVienna = PersonFilter.Filter(persons, p => p.City == "Wien");
List<Person> adults = PersonFilter.Filter(persons, p => p.Age >= 18);
List<Person> startsWithB = PersonFilter.Filter(persons, p => p.LastName.StartsWith(
"B"));
```

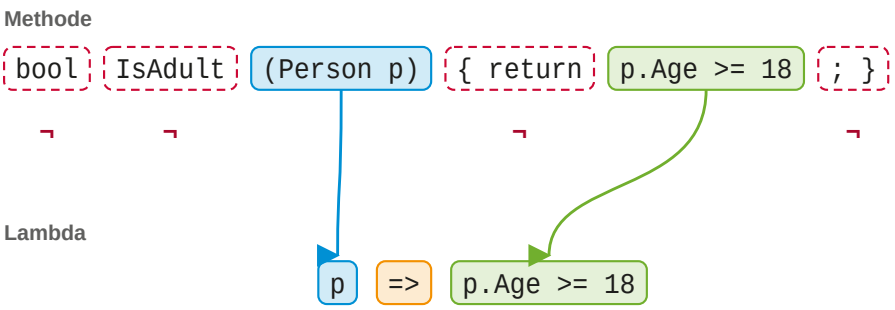


Die Methode `Filter` enthält die Schleife nur einmal. Die Bedingung kommt von außen. Eine Funktion, die eine andere Funktion als Parameter bekommt, nennt man **Higher-Order Function**. Eine Funktion, die für ein Element `true` oder `false` liefert, heißt **Predicate**.

Von der Methode zur Lambda

Eine Lambda ist eine **Methode ohne Namen**, die du direkt dort schreibst, wo du sie brauchst. Du kannst jede kurze Methode in eine Lambda umformen:

Von der Methode zur Lambda Expression



Parameter: Den Typ Person leitet der Compiler ab, die Klammern entfallen bei einem Parameter.
=> ist der Lambda-Operator. Lies: „p geht über zu p.Age >= 18“.
Body: Ein einzelner Ausdruck ist automatisch der Rückgabewert, return und { } entfallen.

Was	Methode	Lambda
Ein Parameter	`bool IsAdult(Person p) { return p.Age >= 18; }`	`p => p.Age >= 18`
Zwei Parameter	`int Add(int a, int b) { return a + b; }`	`(a, b) => a + b`
Kein Parameter	`bool AlwaysTrue() { return true; }`	`() => true`
Kein Rückgabewert	`void Print(Person p) { Console.WriteLine(p); }`	`p => Console.WriteLine(p)`
Mehrere Anweisungen	`bool Check(Person p) { var a = p.Age; return a >= 18; }`	`p => { var a = p.Age; return a >= 18; }`
Typ angeben	`bool IsAdult(Person p) { ... }`	`(Person p) => p.Age >= 18`

Die Regeln dahinter:

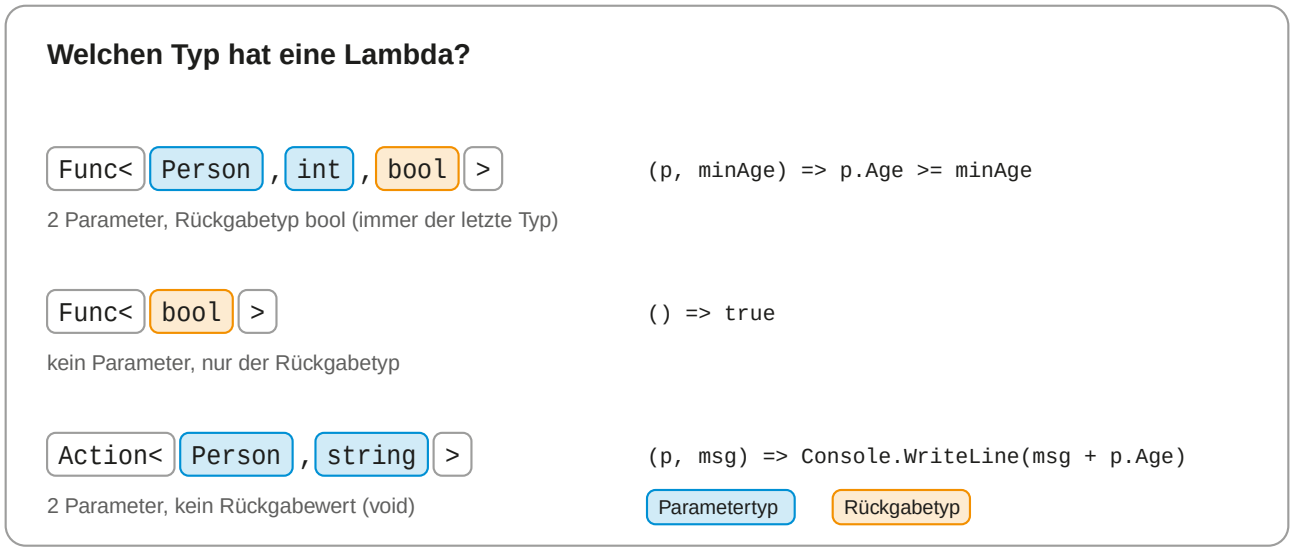
- Den **Namen** und den **Rückgabetyp** lässt du weg. Der Compiler kennt sie aus dem Zusammenhang.
- Den **Typ der Parameter** leitet der Compiler ebenfalls ab. Bei genau einem Parameter fallen auch die runden Klammern weg.
- Besteht der Body aus **einem Ausdruck**, ist dieser automatisch der Rückgabewert. Dann entfallen `{ }` und `return`. Brauchst du mehrere Anweisungen, schreibst du `{ }` und `return` wie in einer Methode.
- Die Namen der Parameter sind frei wählbar. Üblich ist der erste Buchstabe des Typs: `p` für `Person`, `t` für `Trip`.



Den Pfeil `=>` kennst du schon von Properties wie `public string Longname => $"{Firstname} {Lastname}";` (siehe Kapitel Properties). Dort heißt er *expression-bodied member* und ist eine Kurzform für `get { return ...; }`. Das ist keine Lambda, verwendet aber dieselbe Idee: Rechts vom Pfeil steht ein Ausdruck, sein Wert ist das Ergebnis.

Welchen Typ hat eine Lambda? Func und Action

C# ist statisch typisiert. Auch eine Lambda hat daher einen Typ. Die Typen heißen **Delegates**. Für fast alle Fälle reichen zwei generische Delegates aus dem Namespace `System`:



Delegate	Bedeutung	Beispiel
`Func<TResult>`	kein Parameter, liefert `TResult`	`Func<bool> f = () => true;`
`Func<T, TResult>`	ein Parameter vom Typ `T`	`Func<Person, bool> f = p => p.Age >= 18;`
`Func<T1, T2, TResult>`	zwei Parameter	`Func<int, int, int> f = (a, b) => a + b;`
`Action`	kein Parameter, kein Rückgabewert	`Action a = () => Console.WriteLine("Hi");`
`Action<T>`	ein Parameter, kein Rückgabewert	`Action<Person> a = p => Console.WriteLine(p);`

Merkregel: Bei `Func` ist der **letzte** Typ immer der Rückgabetyp. `Action` liefert nichts (`void`). Beide gibt es mit bis zu 16 Parametern.

Eine Lambda in einer Variable rufst du wie eine Methode auf:

```
Func<int, int> square = x => x * x;
Func<Person, string> fullName = p => $"{p.FirstName} {p.LastName}";
Action<string> log = text => Console.WriteLine($"LOG: {text}");

int nine = square(3);
log(fullName(new Person("Anna", "Huber", "Wien", 17)));   // LOG: Anna Huber
```

Mit `var` musst du den Parametertyp in die Lambda schreiben, sonst kennt der Compiler ihn nicht: `var square = (int x) => x * x;`. In älteren Teilen von .NET findest du auch den Delegate `Predicate<T>`. Er entspricht `Func<T, bool>`, z. B. bei `List<T>.FindAll`.

Lambdas, die Variablen von außen verwenden

Eine Lambda darf Variablen verwenden, die außerhalb von ihr deklariert sind:

```
int minAge = 18;
Func<Person, bool> isOldEnough = p => p.Age >= minAge;
minAge = 21;
List<Person> result = PersonFilter.Filter(persons, isOldEnough);   // uses 21, not 18
```

Die Lambda speichert **nicht den Wert** 18, sondern einen Verweis auf die Variable `minAge`. Sie wird erst ausgeführt, wenn `Filter` sie aufruft. Zu diesem Zeitpunkt ist `minAge` schon 21. Unterscheide daher immer zwischen dem **Schreiben** einer Lambda und ihrem **Ausführen**. Eine Lambda, die Variablen von außen verwendet, heißt **Closure**.

Das ist besonders praktisch, wenn eine Methode eine Lambda **erzeugt**:

```
public static Func<Person, bool> LivesIn(string city) => p => p.City == city;
```

Das erste `=>` gehört zur Methode (expression-bodied member), das zweite ist der Pfeil der Lambda. Der Aufruf `LivesIn("Graz")` liefert eine Lambda, die sich den Wert `"Graz"` merkt:

```
Func<Person, bool> livesInGraz = PersonFilter.LivesIn("Graz");
List<Person> fromGraz = PersonFilter.Filter(persons, livesInGraz);
```

Lambdas testen

Weil die Application eine Klassenbibliothek ist, probierst du die Beispiele als Tests aus:

PersonFilterTests.cs (im Projekt *LambdaDemo.Tests*)

```
using System;
using System.Collections.Generic;
using System.Linq;
using LambdaDemo.Application;
using Xunit;

namespace LambdaDemo.Tests;

public class PersonFilterTests
{
    private readonly List<Person> _persons = new()
    {
        new Person("Anna", "Huber", "Wien", 17),
        new Person("Ben", "Bauer", "Graz", 24),
        new Person("Clara", "Berger", "Wien", 31),
    };
```

```
    [Fact]
    public void FilterWithLambda()
    {
        List<Person> result = PersonFilter.Filter(_persons, p => p.City == "Wien");

        Assert.Equal(new[] { "Anna", "Clara" }, result.Select(p => p.FirstName));
    }

    [Fact]
    public void ClosureUsesCurrentValue()
    {
        int minAge = 18;
        Func<Person, bool> isOldEnough = p => p.Age >= minAge;
        minAge = 30;

        List<Person> result = PersonFilter.Filter(_persons, isOldEnough);

        Assert.Single(result);
    }
}
```

`result.Select(p => p.FirstName)` ist schon LINQ. Das lernst du im nächsten Kapitel.

Wo du Lambdas schon verwendet hast

Lambdas begegnen dir in .NET überall. Einige kennst du bereits:

Wo	Beispiel	Typ der Lambda
xUnit	`Assert.Throws<ArgumentException>(() => calc.Calculate(-1))`	`Action`
xUnit	`Assert.All(shows, s => Assert.True(s.Date.Year > 2000))`	`Action<Show>`
EF Core	`modelBuilder.Entity<Ticket>().Property(t => t.TicketState)`	Ausdruck, der ein Property auswählt
Bogus	`new Faker<Event>().CustomInstantiator(f => new Event(f.Music.Genre()))`	`Func<Faker, Event>`
`List<T>`	`persons.RemoveAll(p => p.Age < 18)`	`Predicate<Person>`
`List<T>`	`persons.Sort((a, b) => a.Age.CompareTo(b.Age))`	`Comparison<Person>`
`List<T>`	`persons.ForEach(p => Console.WriteLine(p))`	`Action<Person>`

Bei EF Core schreibst du eine Lambda, aber EF Core **führt sie nicht aus**. Es liest daraus, welches Property gemeint ist. Wie das funktioniert, siehst du im Kapitel LINQ: Where, Select und Any.

Zusammenfassung

Begriff	Bedeutung
Lambda Expression	Methode ohne Namen: `Parameter => Ausdruck`
Delegate	Typ einer Funktion, z. B. `Func<Person, bool>`
`Func<…, TResult>`	Funktion mit Rückgabewert, der letzte Typ ist der Rückgabetyp
`Action<…>`	Funktion ohne Rückgabewert (`void`)
Predicate	Funktion, die `true` oder `false` liefert
Higher-Order Function	Methode, die eine Funktion als Parameter bekommt oder liefert
Closure	Lambda, die eine Variable von außen verwendet

Zusatzmaterial

In der interaktiven Präsentation zu Lambda Expressions siehst du die Umformung von der Methode zur Lambda Schritt für Schritt. Danach bestimmst du in mehreren Runden den Typ von Lambdas und schreibst Methoden in Kurzform um.

LINQ: Where, Select und Any

In SQL schreibst du `SELECT FirstName FROM Persons WHERE City = 'Wien'`. Du **filterst** Zeilen und **projizierst** Spalten. Dasselbe brauchst du in C# ständig: bei einer Liste im Speicher und bei einer Datenbank, auf die du mit EF Core zugreifst. Dafür gibt es **LINQ** (Language Integrated Query). LINQ ist eine Sammlung von Methoden, die Lambdas als Parameter bekommen. Heute schreibt oft ein KI-Agent die Abfragen. Du musst sie trotzdem lesen und beurteilen können: Filtert die Abfrage das Richtige? Läuft sie in der Datenbank oder lädt sie die ganze Tabelle? Genau das lernst du hier.



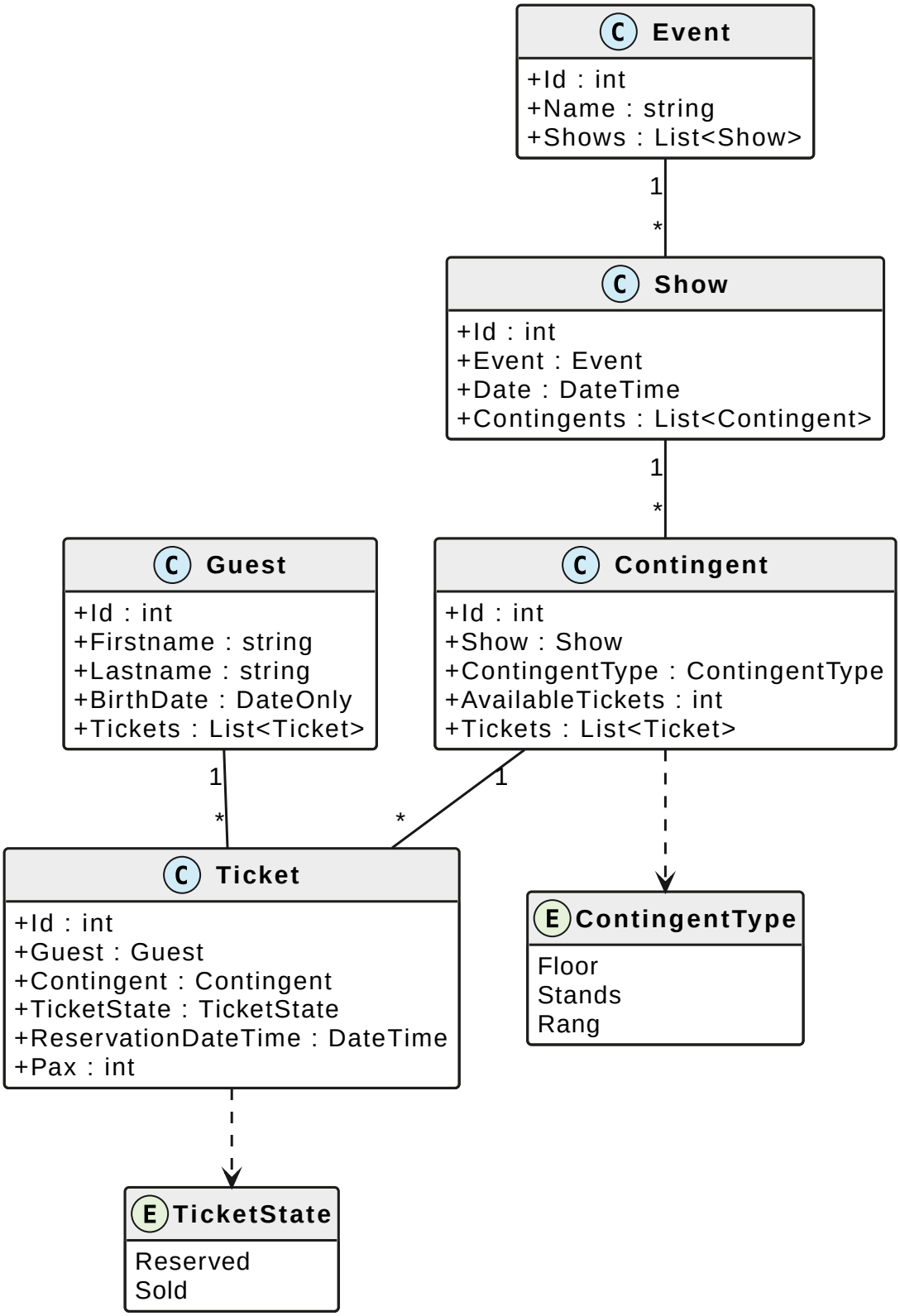
Zum Selbst-Erarbeiten: LINQ und SQL verbinden

In der interaktiven Präsentation zu Where, Select und Any siehst du als Animation, wie Elemente durch `Where` und `Select` laufen. Danach ordnest du LINQ-Ausdrücke den passenden SQL-Abfragen zu und bekommst sofort Feedback. So verbindest du dein SQL-Wissen mit LINQ.

Projekt anlegen

Für dieses Kapitel gibt es ein fertiges Demoprojekt. Lade Eventmanager.7z herunter und entpacke es. Die Beschreibung steht im Archiv in der Datei *README.md*. Öffne *Eventmanager.sln*. Die Beispiele aus diesem Kapitel stehen in *Eventmanager.Tests/PersonQueryTests.cs* und *LinqPlaygroundTests.cs*. Führst du einen Test aus, siehst du in seiner Ausgabe das SQL, das EF Core erzeugt hat.

Das Modell ist ein Ticketsystem: Ein Event findet in mehreren Shows statt, jede Show hat Kontingente, Gäste kaufen oder reservieren Tickets.



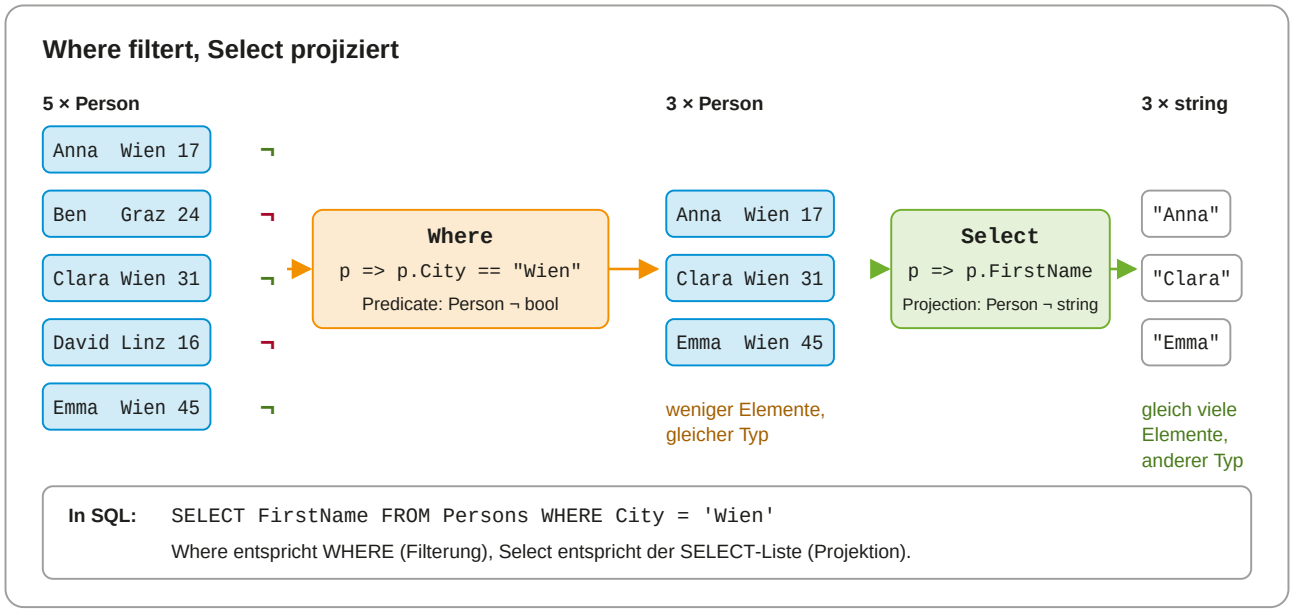
Die Idee: Where filtert, Select projiziert

Wir beginnen mit einer Liste im Speicher:

```
public record Person(string FirstName, string LastName, string City, int Age);
```

```
List<Person> persons = new()
{
    new Person("Anna", "Huber", "Wien", 17),
    new Person("Ben", "Gruber", "Graz", 24),
    new Person("Clara", "Bauer", "Wien", 31),
    new Person("David", "Wagner", "Linz", 16),
    new Person("Emma", "Pichler", "Wien", 45),
};

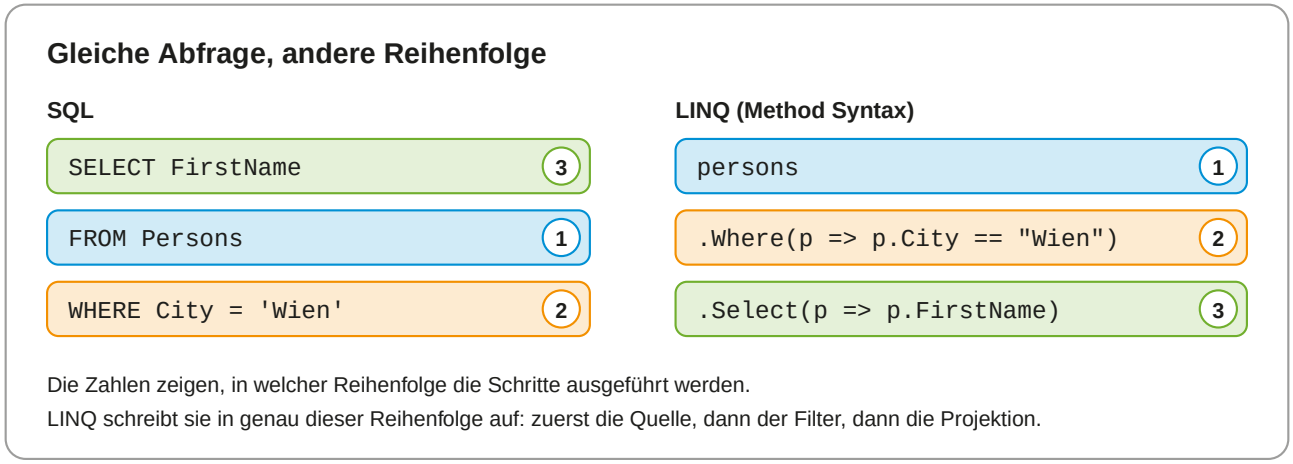
List<string> names = persons
    .Where(p => p.City == "Wien")
    .Select(p => p.FirstName)
    .ToList();                      // ["Anna", "Clara", "Emma"]
```



Methode	Parameter	Ergebnis	SQL
`Where`	**Predicate** `Func<Person, bool>`: Soll das Element bleiben?	weniger (oder gleich viele) Elemente, **gleicher Typ**	`WHERE`
`Select`	**Projection** `Func<Person, string>`: Was soll aus dem Element werden?	**gleich viele** Elemente, **anderer Typ**	Spalten nach `SELECT`

`Where` und `Select` sind genau die Methoden `Filter` und `Map` aus dem Kapitel Lambda Expressions. Sie funktionieren aber für jeden Typ, weil sie generisch sind. Für LINQ brauchst du `using System.Linq;`.

LINQ schreibt die Schritte in der Reihenfolge auf, in der sie passieren. In SQL steht `SELECT` vorne, obwohl die Datenbank zuerst `FROM` und `WHERE` verarbeitet:



Where: Filtern

Die Bedingung in `Where` ist ein normaler C#-Ausdruck vom Typ `bool`. Mit EF Core übersetzt der Provider ihn in SQL:

```
var start = new DateTime(2024, 2, 1);
List<Show> shows = _db.Shows
    .Where(s => s.Date >= start && s.Date < start.AddMonths(2))
    .ToList();
```

```
SELECT "s"."Id", "s"."Date", "s"."EventId"
FROM "Shows" AS "s"
WHERE "s"."Date" >= @start AND "s"."Date" < @AddMonths
```

`start` ist eine Variable außerhalb der Lambda (eine Closure, siehe voriges Kapitel). EF Core schickt ihren Wert als **Parameter** `@start` an die Datenbank. Damit ist auch SQL Injection ausgeschlossen. So übersetzt du typische SQL-Bedingungen:

SQL	C# in der Lambda
`City = 'Wien'`, `City <> 'Wien'`	`p.City == "Wien"`, `p.City != "Wien"`
`AND`, `OR`, `NOT`	`&&`, `||`, `!`
`Age BETWEEN 18 AND 30`	`p.Age >= 18 && p.Age <= 30`
`LastName LIKE 'B%'`, `LIKE '%er%'`	`p.LastName.StartsWith("B")`, `p.LastName.Contains("er")`
`City IN ('Wien', 'Graz')`	`cities.Contains(p.City)` mit `var cities = new[] { "Wien", "Graz" };`
`Email IS NULL`	`p.Email == null`
`e.Name = 'Funk'` über einen Fremdschlüssel	`s.Event.Name == "Funk"` (Navigation, EF Core erzeugt den JOIN)

Mehrere `Where` hintereinander sind wie `AND`. Das ist praktisch, wenn du Bedingungen nur unter bestimmten Umständen hinzufügen willst (`if (onlyAdults) query = query.Where(...)`).

Select: Projizieren

`Select` wandelt jedes Element in etwas anderes um. Das Ergebnis kann ein einfacher Wert, ein Record oder ein anonymer Typ sein:

```
// A simple value.
List<string> names = _db.Guests
    .Where(g => g.BirthDate.Year >= 2003)
    .Select(g => g.Firstname + " " + g.Lastname)
    .ToList();

// A record with values from several tables (navigation properties).
public record ShowInfo(int ShowId, DateTime Date, string EventName);
List<ShowInfo> infos = _db.Shows
    .Select(s => new ShowInfo(s.Id, s.Date, s.Event.Name))
    .ToList();

// An anonymous type: only inside a method, the type has no name, so you need var.
var eventInfos = _db.Events.Select(e => new { e.Name, ShowCount = e.Shows.Count
}).ToList();
```

```
SELECT "g"."Firstname" || ' ' || "g"."Lastname"
FROM "Guests" AS "g"
WHERE CAST(strftime('%Y', "g"."BirthDate") AS INTEGER) >= 2003
```

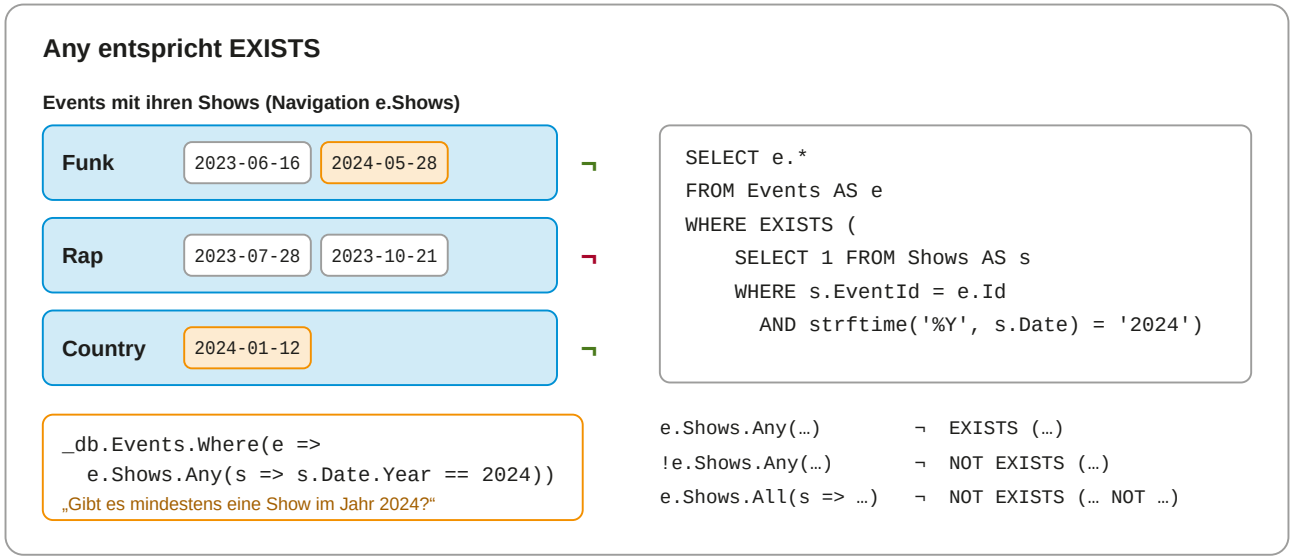
EF Core fragt nur die Spalten ab, die in der Projektion vorkommen. Greifst du auf eine Navigation wie `s.Event.Name` zu, erzeugt es den JOIN selbst. Wozu Records als Rückgabetyp dienen und warum man sie **DTOs** nennt, lernst du im Kapitel Serviceklassen mit EF Core.

Any und All: Gibt es so ein Element?

`Any` liefert `true`, wenn **mindestens ein** Element die Bedingung erfüllt. `All` liefert `true`, wenn **alle** Elemente sie erfüllen. Ohne Parameter fragt `Any()`: Ist die Liste nicht leer?

```
bool hasMinors = persons.Any(p => p.Age < 18);     // true
bool allAdults = persons.All(p => p.Age >= 18);    // false
bool notEmpty = persons.Any();                      // true
```

Richtig stark wird `Any` auf einer **Navigation**. Die Frage „Welche Events haben eine Show im Jahr 2024?“ löst du in SQL mit einer Unterabfrage und `EXISTS`. In LINQ schreibst du `Any` auf die Liste `e.Shows`:




```
List<Event> events = _db.Events
    .Where(e => e.Shows.Any(s => s.Date.Year == 2024))
    .ToList();

List<Guest> guestsWithoutPurchase = _db.Guests
    .Where(g => !g.Tickets.Any(t => t.TicketState == TicketState.Sold))
    .ToList();
```

```
SELECT "g"."Id", "g"."BirthDate", "g"."Firstname", "g"."Lastname"
FROM "Guests" AS "g"
WHERE NOT EXISTS (
    SELECT 1
    FROM "Tickets" AS "t"
    WHERE "g"."Id" = "t"."GuestId" AND "t"."TicketState" = 'Sold')
```



`All` auf einer **leeren** Liste liefert `true`. Es gibt ja kein Element, das die Bedingung verletzt. Ein Event ohne Shows erfüllt also `e.Shows.All(s => s.Date.Year == 2024)`. Willst du das nicht, kombiniere: `e.Shows.Any() && e.Shows.All(...)`.

Was du sonst noch brauchst

Diese Methoden kommen in fast jeder Abfrage vor. Die meisten kennst du aus SQL:

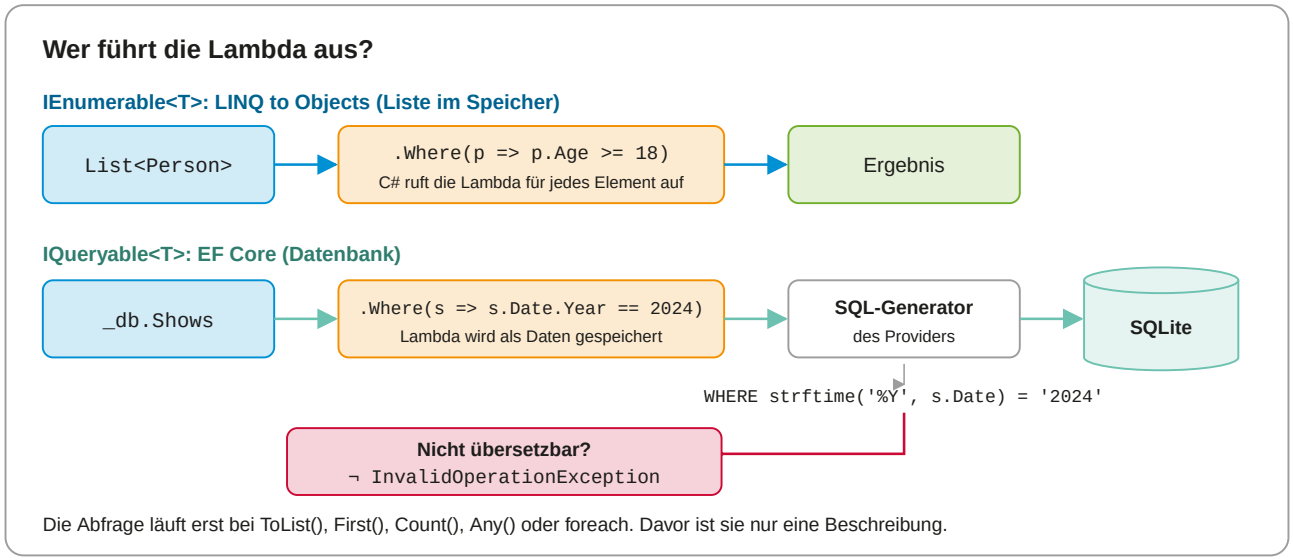
LINQ	Ergebnis	SQL
`ToList()`	führt die Abfrage aus und liefert eine `List<T>`	
`First(pred)`, `FirstOrDefault(pred)`	erstes passendes Element. `First` wirft eine Exception, wenn es keins gibt, `FirstOrDefault` liefert `null`	`LIMIT 1`

LINQ	Ergebnis	SQL
`Single(pred)`, `SingleOrDefault(pred)`	wie `First`, aber eine Exception bei **mehr** als einem Treffer	`LIMIT 2` + Prüfung
`Count()`, `Count(pred)`	Anzahl als `int`	`COUNT(*)`
`OrderBy(x => …)`, `OrderByDescending`, `ThenBy`	sortierte Folge	`ORDER BY …, …`
`Sum`, `Min`, `Max`, `Average`	ein Wert	`SUM`, `MIN`, `MAX`, `AVG`
`Distinct()`	ohne Duplikate	`DISTINCT`
`Skip(n)`, `Take(n)`	Teil der Folge, z. B. zum Blättern	`OFFSET`, `LIMIT`

`FirstOrDefault` liefert bei Entities `null`, wenn nichts gefunden wurde. Der Rückgabetyp ist daher nullable (`Show?`). Prüfe das Ergebnis, bevor du es verwendest.

IEnumerable und IQueryable: Wer führt die Lambda aus?

Die Abfrage auf die Liste `persons` und die Abfrage auf `_db.Shows` sehen gleich aus. Intern passiert aber etwas ganz anderes:



	`IEnumerable<T>` **(LINQ to Objects)**	`IQueryable<T>` **(EF Core)**
Quelle	Liste, Array im Speicher	`DbSet` wie `_db.Shows`
Die Lambda ist	eine Funktion (`Func<T, bool>`)	eine Beschreibung des Ausdrucks (`Expression<Func<T, bool>>`)
Ausgeführt von	C#, für jedes Element	der Datenbank, als SQL

Eine Abfrage auf einem `IQueryable` ist zuerst **nur eine Beschreibung**. Erst `ToList()`, `First()`, `Count()`, `Any()` oder ein `foreach` schicken das SQL an die Datenbank. Mit `ToQueryString()` siehst du das SQL, ohne die Abfrage auszuführen:

```
IQueryable<DateTime> query = _db.Shows
    .Where(s => s.Event.Name == "Funk")
    .Select(s => s.Date);
Console.WriteLine(query.ToQueryString());
```

Wenn EF Core die Abfrage nicht übersetzen kann

Nicht jeder C#-Ausdruck hat ein Gegenstück in SQL. Wir suchen Reservierungen, die mehr als 20 Tage vor der Show gemacht wurden:

```
// Throws InvalidOperationException: The LINQ expression ... could not be translated.
var tickets = _db.Tickets
    .Where(t => (t.Contingent.Show.Date - t.ReservationDateTime).TotalDays > 20)
    .ToList();
```

Die Differenz zweier `DateTime` ist ein `TimeSpan`. Dafür hat SQLite keinen Datentyp. EF Core wirft daher eine `InvalidOperationException` mit dem Text *could not be translated*. Du hast zwei Möglichkeiten:

1. Die Bedingung umformulieren. Das ist die beste Lösung, denn die Datenbank filtert weiterhin. `AddDays` kann SQLite berechnen:

```
var tickets = _db.Tickets
    .Where(t => t.ReservationDateTime < t.Contingent.Show.Date.AddDays(-20))
    .ToList();
```

2. Erst in der Datenbank filtern und projizieren, dann mit `ToList()` laden und im Speicher weiterarbeiten. Nach `ToList()` hast du eine `List<T>`, ab dann läuft LINQ in C#:

```
var tickets = _db.Tickets
    .Where(t => t.TicketState == TicketState.Reserved)        // SQL
    .Select(t => new { t.Id, t.ReservationDateTime, ShowDate = t.Contingent.Show.Date
})
    .ToList()                                                  // query runs here
    .Where(t => (t.ShowDate - t.ReservationDateTime).TotalDays > 20)   // C#
    .ToList();
```



`_db.Tickets.ToList().Where(...)` lädt **die ganze Tabelle** in den Speicher und filtert erst dann. Bei einer Million Tickets ist das ein ernstes Problem. Filtere so viel wie möglich vor dem `ToList()`. Achte darauf auch, wenn ein KI-Agent die Abfrage geschrieben hat.

Typische Ausdrücke, die der SQLite-Provider von EF Core 10 nicht übersetzt:

Ausdruck	Beispiel
Rechnen mit `TimeSpan` in `Where` oder `OrderBy`	`(a - b).TotalMinutes > 5`
`TimeOnly` aus einem `DateTime`	`TimeOnly.FromDateTime(t.Planned) < new TimeOnly(8, 0)`
eigene C#-Methoden	`.Where(p => IsAdult(p))`

Eine Ausnahme gibt es: Im **letzten** `Select` darf etwas stehen, das nicht übersetzbar ist. EF Core lädt dann die nötigen Spalten und rechnet den Rest in C#.

Zusatzmaterial

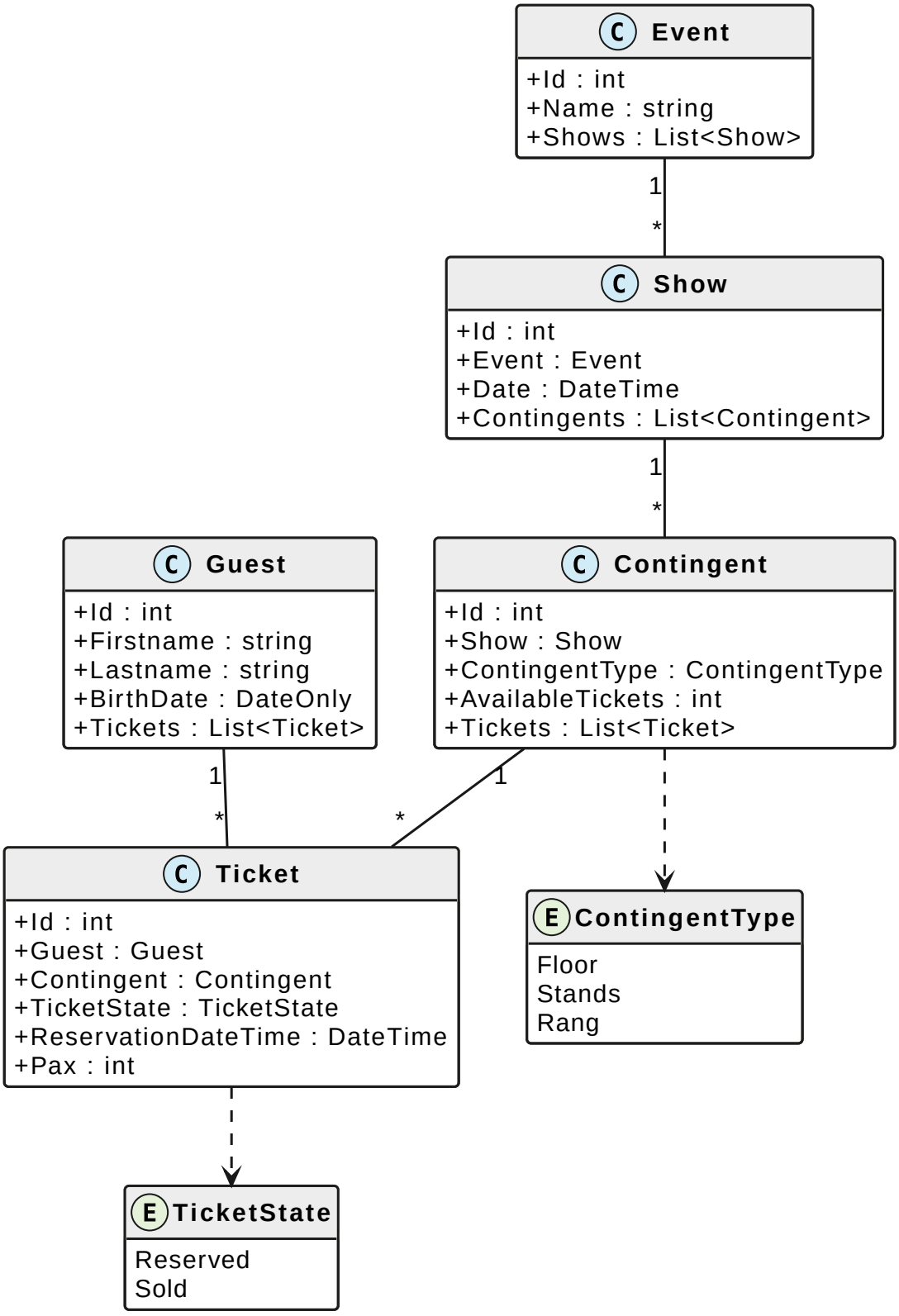
In der interaktiven Präsentation zu Where, Select und Any siehst du die Pipeline aus `Where` und `Select` als Animation. Danach übst du in mehreren Runden, SQL-Abfragen und LINQ-Ausdrücke einander zuzuordnen.

Serviceklassen mit EF Core

Ein Ticketsystem soll Reservierungen annehmen. Die Web-Oberfläche prüft, ob die Show ausverkauft ist. Die App fürs Handy prüft das auch, aber etwas anders. Der Import aus einer Excel-Liste prüft es gar nicht. Bald gibt es mehr verkaufte Plätze, als die Halle hat. Die Regel „nicht mehr Tickets als Plätze“ gehört an **genau eine Stelle**. Diese Stelle ist eine **Serviceklasse**: Sie enthält die Abfragen und die Geschäftsregeln (Business Logic) einer Anwendung. Alle Oberflächen rufen nur ihre Methoden auf. In diesem Kapitel schreibst du Serviceklassen, die Daten als **DTOs** liefern und die Datenbank nach festen Regeln ändern.

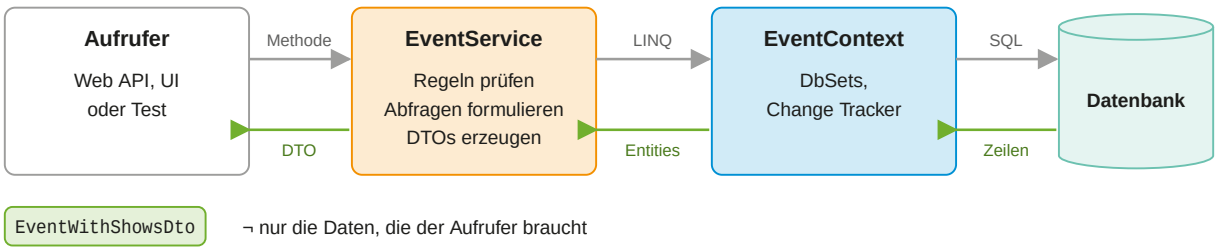
Projekt anlegen

Du verwendest wieder das Demoprojekt aus dem Kapitel LINQ: Where, Select und Any. Lade Eventmanager.7z herunter, falls du es noch nicht hast. Die Beschreibung steht im Archiv in der Datei *README.md*. Die Serviceklasse aus diesem Kapitel steht in *Eventmanager.Application/Services/EventService.cs*, die Tests dazu in *Eventmanager.Tests/EventServiceTests.cs*.



Was ist eine Serviceklasse?

Wo liegt die Serviceklasse?



`EventWithShowsDto` ¬ nur die Daten, die der Aufrufer braucht

Der Aufrufer kennt keine Entities und keinen DbContext. Er ruft eine Methode auf und bekommt ein DTO.

Eine Serviceklasse bekommt den DbContext im **Konstruktor** übergeben. Sie erzeugt ihn nicht selbst. Das ist **Dependency Injection** (siehe Kapitel D: Dependency Inversion). Im Programm übergibt das Framework einen DbContext für die echte Datenbank, im Test einen für eine SQLite-Datenbank im Speicher.

```
public class EventService
{
    private readonly EventContext _db;

    public EventService(EventContext db)
    {
        _db = db;
    }
}
```

Über das Feld `_db` schreiben die Methoden ihre Abfragen. Die erste Methode liefert alle Shows in einem Zeitraum. Sie gibt die Entities direkt zurück:

```
public List<Show> GetShowsInDateRange(DateTime start, DateTime end)
{
    return _db.Shows
        .AsNoTracking()
        .Where(s => s.Date >= start && s.Date < end)
        .OrderBy(s => s.Date)
        .ToList();
}
```

Was `AsNoTracking()` bedeutet, steht weiter unten. Oft passen die Entities aber nicht zu dem, was der Aufrufer braucht.

DTOs: typsichere Rückgabewerte

Eine Übersichtsseite soll jedes Event mit der **Anzahl** seiner Shows zeigen. Die Entity `Event` hat kein Property dafür. Wir definieren daher einen eigenen Typ, ein **DTO** (Data Transfer Object). Es enthält genau die Daten, die der Aufrufer braucht:

```
public record EventWithShowCountDto(int EventId, string EventName, int ShowCount);
```

Das ist ein **positional record** (siehe Kapitel Equality). Der Compiler erzeugt daraus eine Klasse mit drei read-only Properties und einem passenden Konstruktor. Die Parameter beginnen deshalb mit einem Großbuchstaben: Sie werden zu Properties.

```
public List<EventWithShowCountDto> GetEventsWithShowCount()
{
    return _db.Events
        .Select(e => new EventWithShowCountDto(e.Id, e.Name, e.Shows.Count))
        .ToList();
}
```

```
SELECT "e"."Id", "e"."Name", (
    SELECT COUNT(*)
    FROM "Shows" AS "s"
    WHERE "e"."Id" = "s"."EventId") AS "Count"
FROM "Events" AS "e"
```

```
EventId  EventName         ShowCount
      1  Stage And Screen          3
      2  Funk                      4
      3  World                     5
      4  Rap                       9
      5  Country                   4
```

Warum ein eigener Record und nicht einfach ein anonymer Typ (`new { e.Id, e.Name }`) oder die Entity?

Rückgabe	Problem oder Vorteil
Entity (`List<Event>`)	liefert alle Spalten, aber keine berechneten Werte. Der Aufrufer sieht den internen Aufbau der Datenbank. Beim Umwandeln in JSON können Navigationen Endlosschleifen erzeugen.
Anonymer Typ	hat keinen Namen. Eine Methode kann ihn nur als `object` zurückgeben, der Aufrufer weiß nicht, welche Properties es gibt.
DTO	hat einen Namen und feste Properties. Der **Compiler** prüft jeden Zugriff. Es enthält nur, was der Aufrufer braucht, auch berechnete Werte.

Eine Web API liefert fast immer DTOs. Das Frontend bekommt so eine stabile Struktur, auch wenn sich die Datenbank ändert.

Verschachtelte DTOs

Eine Detailseite zeigt ein Event mit allen Shows. Ein DTO kann eine Liste anderer DTOs enthalten:

```
public record ShowDto(int ShowId, DateTime Date);
public record EventWithShowsDto(int EventId, string EventName, List<ShowDto> Shows);

public EventWithShowsDto? GetEventWithShows(int eventId)
{
    return _db.Events
        .Where(e => e.Id == eventId)
        .Select(e => new EventWithShowsDto(
            e.Id, e.Name,
            e.Shows.OrderBy(s => s.Date).Select(s => new ShowDto(s.Id, s.Date)).
ToList()))
        .FirstOrDefault();
}
```

```
{"EventId":1,"EventName":"Stage And Screen","Shows":[
  {"ShowId":15,"Date":"2023-12-23T19:00:00"},
  {"ShowId":14,"Date":"2024-04-27T18:00:00"},
  {"ShowId":1,"Date":"2024-05-04T19:00:00"}]}
```

Beachte drei Dinge:

- Das innere `Select` braucht ein `ToList()`, weil das DTO eine `List<ShowDto>` erwartet.
- `FirstOrDefault()` steht **am Ende**. Davor ist die Abfrage noch ein `IQueryable`, und EF Core kann die Navigation `e.Shows` in SQL übersetzen (hier ein `LEFT JOIN`).
- Gibt es das Event nicht, liefert `FirstOrDefault()` `null`. Der Rückgabetyp ist daher `EventWithShowsDto?`.

Ein DTO kann auch Werte enthalten, die erst berechnet werden. Die folgende Methode zählt die verkauften und die reservierten Personen eines Kontingents. Ein Ticket gilt für `Pax + 1` Personen (Gast plus Begleitpersonen):

```
public record ContingentStatisticsDto(int ContingentId, DateTime ShowDate, int
SoldPersons, int ReservedPersons);

public ContingentStatisticsDto GetContingentStatistics(int contingentId)
{
    var statistics = _db.Contingents
        .Where(c => c.Id == contingentId)
        .Select(c => new ContingentStatisticsDto(
            c.Id,
            c.Show.Date,
            c.Tickets.Where(t => t.TicketState == TicketState.Sold).Sum(t => t.Pax +
1),
            c.Tickets.Where(t => t.TicketState == TicketState.Reserved).Sum(t => t.Pax
```

```
+ 1)))
        .FirstOrDefault();
    if (statistics is null)
        throw new EventServiceException("Invalid contingent id.");
    return statistics;
}
```

Eager Loading oder Projektion?

Ein häufiger Irrtum: „Für verknüpfte Daten brauche ich immer `Include()`." Es gibt zwei Wege.

Eager Loading lädt ganze Entities mit ihren Navigationen. Das Ergebnis sind Objekte deiner Model-Klassen, die du ändern und mit `SaveChanges()` speichern kannst:

```
Event? ev = _db.Events
    .Include(e => e.Shows)
    .FirstOrDefault(e => e.Id == 1);
int count = ev?.Shows.Count ?? 0;
```

Bei einer **Projektion** beschreibt `Select`, welche Daten du brauchst. EF Core erzeugt die nötigen JOINs selbst. Ein zusätzliches `Include()` ist dann **unnötig**, EF Core ignoriert es.

	Eager Loading (`Include`)	**Projektion (`Select`)**
Ergebnis	Entities (`Event`)	DTOs (`EventWithShowCountDto`)
Geladene Spalten	alle Spalten aller Tabellen	nur die benötigten
Verwendung	Daten ändern oder löschen	Daten anzeigen, Web API

AsNoTracking: Nur lesen

EF Core merkt sich jede geladene Entity im **Change Tracker**. Dazu legt es eine Kopie der Werte an (Snapshot). Bei `SaveChanges()` vergleicht es die Entity mit dem Snapshot und erzeugt ein `UPDATE` für jede Änderung. Das kostet Speicher und Zeit.

Liefert eine Methode Entities nur zum **Anzeigen**, schaltest du das mit `AsNoTracking()` ab (siehe `GetShowsInDateRange` oben). Projektionen auf DTOs werden ohnehin nicht getrackt, dort brauchst du es nicht.



Hast du eine Entity mit `AsNoTracking()` geladen, speichert `SaveChanges()` Änderungen an ihr **nicht**. Verwende es daher nur in lesenden Methoden.

Methoden, die die Datenbank ändern

Die DbSets erlauben auch schreibende Zugriffe. Die Grundbausteine:

```
// INSERT: add a new entity and save.
```

```
var guest = new Guest("Lena", "Moser", new DateOnly(2004, 3, 12));
_db.Guests.Add(guest);
_db.SaveChanges();              // guest.Id is set now

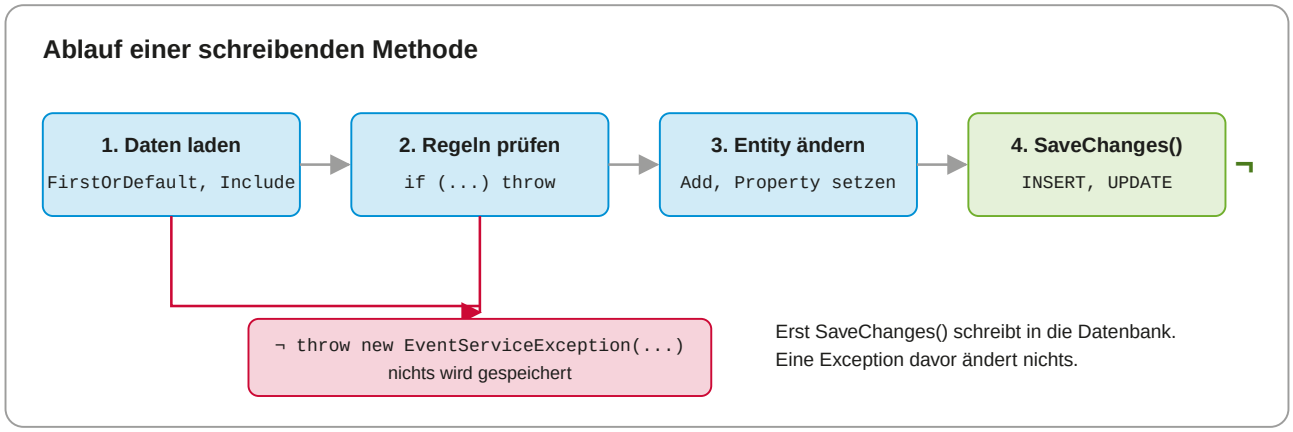
// UPDATE: load a tracked entity, change it, save.
var ticket = _db.Tickets.First(t => t.Id == 1);
ticket.TicketState = TicketState.Sold;
_db.SaveChanges();              // UPDATE "Tickets" SET "TicketState" = 'Sold' WHERE
"Id" = 1

// DELETE without loading: one DELETE statement, SaveChanges is not needed.
int deleted = _db.Tickets
    .Where(t => t.TicketState == TicketState.Reserved && t.ReservationDateTime < limit)
    .ExecuteDelete();
```

Für viele Datensätze gibt es analog `ExecuteUpdate(s => s.SetProperty(t => t.Pax, 0))`.

Geschäftsregeln prüfen

Fast jede schreibende Methode muss Regeln prüfen. Der Ablauf ist immer gleich:



Verletzt ein Aufruf eine Regel, wirft die Methode eine **eigene Exception**. So kann der Aufrufer Regelverletzungen von Programmfehlern unterscheiden:

```
public class EventServiceException : Exception
{
    public EventServiceException(string message) : base(message)
    { }
}
```

Die Methode `CreateReservation` legt eine Reservierung an. Sie prüft diese Regeln:

- Die Werte in `guestId` und `contingentId` müssen existieren.
- Eine Reservierung ist nur bis 14 Tage vor der Show möglich.
- Pro Gast und Kontingent gibt es höchstens ein Ticket.
- Im Kontingent müssen noch genug Plätze frei sein.

```
public int CreateReservation(int guestId, int contingentId, int pax, DateTime dateTime)
{
    var contingent = _db.Contingents
        .Include(c => c.Show)
        .Include(c => c.Tickets)
        .FirstOrDefault(c => c.Id == contingentId);
    if (contingent is null)
        throw new EventServiceException("Invalid contingent id.");
    if (contingent.Show.Date < dateTime.AddDays(14))
        throw new EventServiceException("The show is too close in time.");

    var guest = _db.Guests.FirstOrDefault(g => g.Id == guestId);
    if (guest is null)
        throw new EventServiceException("Invalid guest id.");

    if (_db.Tickets.Any(t => t.Contingent.Id == contingentId && t.Guest.Id == guestId))
        throw new EventServiceException("A reservation or purchase has already been
made for this contingent.");

    // Each ticket admits the guest plus the accompanying persons (pax).
    var soldOrReserved = contingent.Tickets.Sum(t => t.Pax + 1);
    if (soldOrReserved + pax + 1 > contingent.AvailableTickets)
        throw new EventServiceException("Show is sold out.");

    var ticket = new Ticket(guest, contingent, TicketState.Reserved, dateTime, pax);
    _db.Tickets.Add(ticket);
    _db.SaveChanges();
    return ticket.Id;
}
```

Hier brauchen wir `Include()`: Die Methode arbeitet mit der Entity `contingent` und greift auf `contingent.Show` und `contingent.Tickets` zu. Ohne `Include()` wären diese Navigationen `null` bzw. leer. Die Prüfung auf ein vorhandenes Ticket läuft dagegen mit `Any()` direkt in der Datenbank:

```
SELECT EXISTS (
    SELECT 1
    FROM "Tickets" AS "t"
    INNER JOIN "Contingents" AS "c" ON "t"."ContingentId" = "c"."Id"
    INNER JOIN "Guests" AS "g" ON "t"."GuestId" = "g"."Id"
    WHERE "c"."Id" = @contingentId AND "g"."Id" = @guestId)
```



Prüfe **alle** Regeln, bevor du etwas änderst. `SaveChanges()` steht am Ende. Wirft eine Prüfung davor eine Exception, bleibt die Datenbank unverändert.

Serviceklassen testen

Weil die Serviceklasse den DbContext von außen bekommt, kannst du sie mit einer SQLite-Datenbank im Speicher testen. Die Basisklasse *DatabaseTest.cs* im Demoprojekt erzeugt für jeden Test eine neue Datenbank mit den Musterdaten:

```
[Fact]
public void CreateReservation_ThrowsIfShowIsTooClose()
{
    var contingent = _db.Contingents.Include(c => c.Show).First();
    var guest = _db.Guests.First();

    var e = Assert.Throws<EventServiceException>(
        () => _service.CreateReservation(guest.Id, contingent.Id, 0, contingent.Show
.Date.AddDays(-3)));
    Assert.Equal("The show is too close in time.", e.Message);
}
```

Prüfst du nach einer schreibenden Methode die Datenbank, rufe vorher `_db.ChangeTracker.Clear()` auf. Sonst liefert EF Core die Objekte aus dem Change Tracker und nicht die gespeicherten Werte. Mehr zum Testen steht im Kapitel Unit Tests mit xUnit.

Erweiterte LINQ Techniken

Der Veranstalter aus dem Eventmanager möchte Statistiken. Wie viele Tickets gibt es pro Kontingent-Typ? Welche Gäste waren bei einem Event, egal bei welcher Show? Welche Shows fallen auf einen Feiertag, wenn die Feiertage aus einer Datei kommen? Mit `Where`, `Select` und `Any` kommst du hier nicht weiter. Du brauchst **GroupBy**, **SelectMany**, **Join** und **Aggregate**. Aus SQL kennst du `GROUP BY` und `JOIN` schon. In diesem Kapitel siehst du, wie sie in LINQ aussehen.



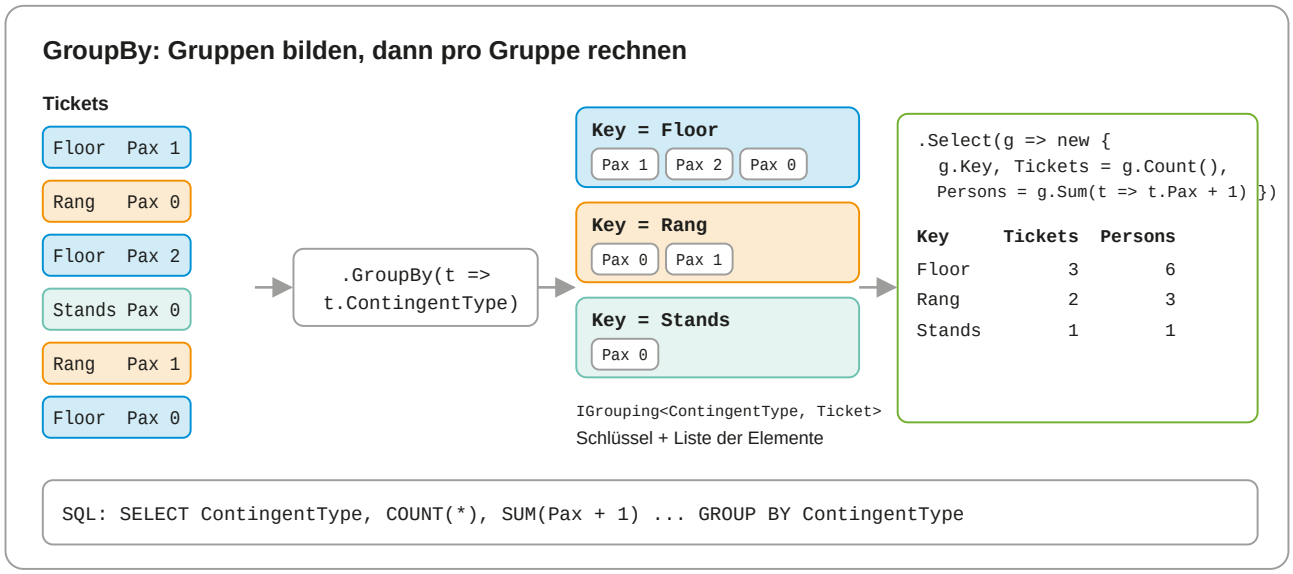
Dieses Kapitel ist für das **Selbststudium** gedacht. Es setzt die Kapitel LINQ: Where, Select und Any und Serviceklassen mit EF Core voraus.

Projekt anlegen

Du verwendest das Demoprojekt aus den vorigen Kapiteln. Lade Eventmanager.7z herunter, falls du es noch nicht hast. Die Beschreibung steht im Archiv in der Datei *README.md*. Alle Beispiele dieses Kapitels stehen in *Eventmanager.Tests/LinqAdvancedTests.cs*.

GroupBy: Gruppen bilden

Wir wollen pro Kontingent-Typ (Floor, Rang, Stands) wissen, wie viele Tickets es gibt und für wie viele Personen sie gelten. In SQL schreibst du `GROUP BY`. In LINQ heißt die Methode `GroupBy`:




```
var personsPerType = _db.Tickets
    .GroupBy(t => t.Contingent.ContingentType)
    .Select(g => new
    {
        ContingentType = g.Key,
        Tickets = g.Count(),
        Persons = g.Sum(t => t.Pax + 1)
    })
    .ToList();
```

```
SELECT "c"."ContingentType", COUNT(*) AS "Tickets", COALESCE(SUM("t"."Pax" + 1), 0) AS
"Persons"
FROM "Tickets" AS "t"
INNER JOIN "Contingents" AS "c" ON "t"."ContingentId" = "c"."Id"
GROUP BY "c"."ContingentType"
```

`GroupBy` bekommt eine Lambda, die den **Schlüssel** (Key) liefert. Das Ergebnis ist eine Folge von Gruppen vom Typ `IGrouping<ContingentType, Ticket>`. Jede Gruppe hat ein Property `Key` und ist selbst eine Liste ihrer Elemente. Deshalb funktionieren `g.Count()` und `g.Sum(...)` wie bei jeder anderen Liste. Das `Select` danach entspricht der SELECT-Liste mit den Aggregatfunktionen.

Brauchst du **mehrere Schlüssel**, verwendest du einen anonymen Typ, wie `GROUP BY Year, Month`:

```
var showsPerMonth = _db.Shows
    .GroupBy(s => new { s.Date.Year, s.Date.Month })
    .OrderBy(g => g.Key.Year).ThenBy(g => g.Key.Month)
    .Select(g => new { g.Key.Year, g.Key.Month, Count = g.Count() })
    .ToList();
```

Zwei Regeln für EF Core:

- Nach `GroupBy` sollte ein `Select` mit Aggregatfunktionen (`Count`, `Sum`, `Max` …) folgen. Nur das kann SQL direkt berechnen.
- Sortiere **vor** einem `Select`, das einen Record mit Konstruktor erzeugt (`.OrderBy(g => g.Key).Select(g => new MyDto(...))`). Ein `OrderBy` auf die Properties des Records danach kann EF Core nicht übersetzen. Bei anonymen Typen funktioniert beides.

Oft brauchst du gar kein GroupBy

Wie viele Shows hat jedes Event? Das geht mit `GroupBy` über die Shows. Mit der Navigation `e.Shows` ist es einfacher, und Events **ohne** Shows kommen mit 0 ins Ergebnis:

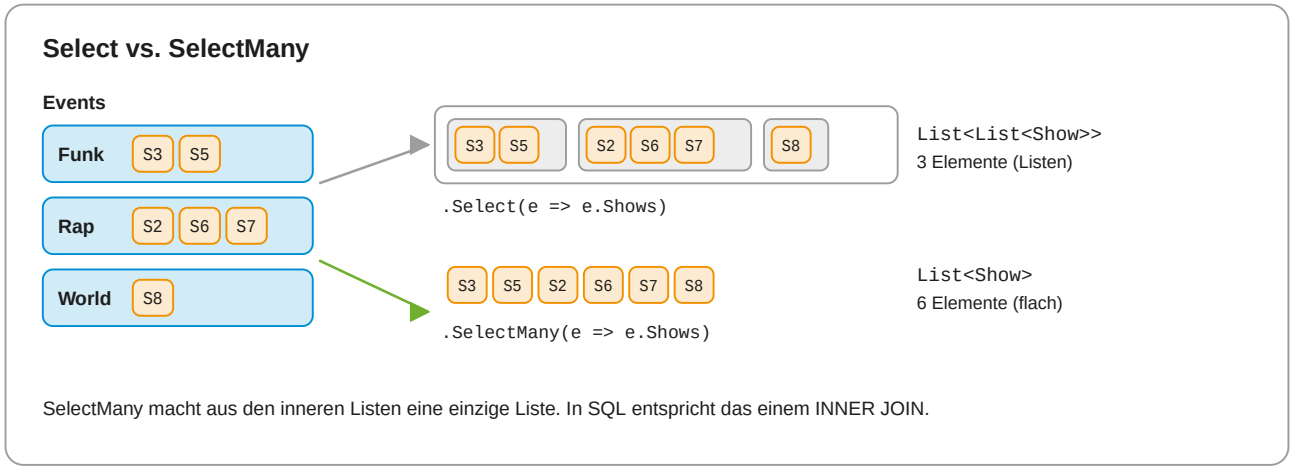
```
var showsPerEvent = _db.Events
    .Select(e => new { e.Name, Count = e.Shows.Count })
    .ToList();
```

`_db.Shows.GroupBy(s => s.Event.Name)` würde Events ohne Shows nicht liefern, denn es gibt keine Show, die eine Gruppe für sie bilden könnte. Gruppiere daher nur, wenn es keine passende Navigation gibt (z. B. nach einem Enum, einem Datum oder einem Wert wie der Stadt).

SelectMany: Listen von Listen flach machen

Welche Gäste waren bei einem Event? Ein Event hat Shows, eine Show hat Kontingente, ein Kontingent hat Tickets. Mit `Select(e => e.Shows)` bekommst du eine **Liste von Listen**. `SelectMany`

macht daraus eine einzige Liste:



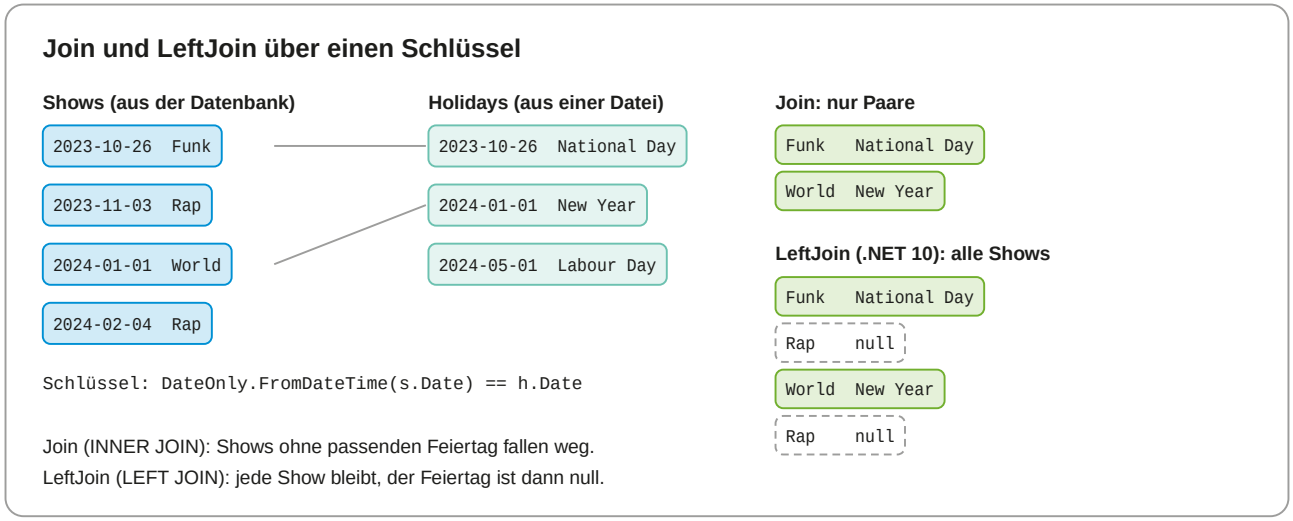

```
List<string> guestNames = _db.Events
    .Where(e => e.Id == 1)
    .SelectMany(e => e.Shows)
    .SelectMany(s => s.Contingents)
    .SelectMany(c => c.Tickets)
    .Select(t => t.Guest.Lastname)
    .Distinct()
    .OrderBy(n => n)
    .ToList();
```

```
SELECT DISTINCT "g"."Lastname"
FROM "Events" AS "e"
INNER JOIN "Shows" AS "s" ON "e"."Id" = "s"."EventId"
INNER JOIN "Contingents" AS "c" ON "s"."Id" = "c"."ShowId"
INNER JOIN "Tickets" AS "t" ON "c"."Id" = "t"."ContingentId"
INNER JOIN "Guests" AS "g" ON "t"."GuestId" = "g"."Id"
WHERE "e"."Id" = 1
```

Jedes `SelectMany` über eine Navigation wird zu einem `INNER JOIN`. Du schreibst die JOINs nicht selbst, du gehst einfach den Navigationen entlang.

Join: Verbinden ohne Navigation

In EF Core brauchst du `Join` fast nie, denn die Navigationen erzeugen die JOINs. `Join` brauchst du, wenn es **keine Beziehung** gibt. Ein Beispiel: Die Feiertage kommen aus einer Datei oder einem Webservice. Sie haben keinen Fremdschlüssel zu den Shows. Das Einzige, was sie verbindet, ist das Datum.




```
private record Holiday(DateOnly Date, string Name);

var shows = _db.Shows
    .Select(s => new { s.Id, s.Date, EventName = s.Event.Name })
    .ToList();

var showsOnHolidays = shows
    .Join(holidays,                          // the second list
        s => DateOnly.FromDateTime(s.Date),  // key of a show
        h => h.Date,                         // key of a holiday
        (s, h) => new { s.EventName, s.Date, Holiday = h.Name })   // result of a pair
    .ToList();
```

`Join` liefert nur die Paare, bei denen die Schlüssel gleich sind, wie ein `INNER JOIN`. Seit .NET 10 gibt es auch `LeftJoin`. Dabei bleibt jedes Element der ersten Liste im Ergebnis. Gibt es keinen Partner, ist das zweite Element `null`:

```
var allShows = shows
    .LeftJoin(holidays,
        s => DateOnly.FromDateTime(s.Date),
        h => h.Date,
        (s, h) => new { s.EventName, s.Date, Holiday = h?.Name })
    .ToList();
```

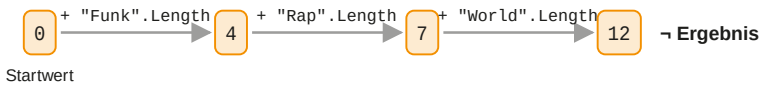
Liegen beide Tabellen in der Datenbank (zwei DbSets ohne Beziehung), übersetzt EF Core `Join` in einen `INNER JOIN` und `LeftJoin` in einen `LEFT JOIN`. Die Holidays im Beispiel sind dagegen eine Liste im Speicher. Deshalb laden wir die Shows zuerst mit `ToList()`.

Aggregate: Eine Liste zu einem Wert zusammenfassen

In JavaScript kennst du `reduce`. In .NET heißt die Methode `Aggregate`. Sie geht die Liste Element für Element durch und reicht einen Zwischenwert (Akkumulator) weiter:

Aggregate: eine Liste zu einem Wert zusammenfassen

```
names.Aggregate(0, (sum, name) => sum + name.Length)
```



JavaScript: `names.reduce((sum, name) => sum + name.length, 0)`

Der Wert sum wird von Schritt zu Schritt weitergereicht (Akkumulator).
Aggregate wird nicht nach SQL übersetzt: zuerst ToList(), dann Aggregate.

JavaScript	C#
`names.reduce((sum, n) => sum + n.length, 0)`	`names.Aggregate(0, (sum, n) => sum + n.Length)`
`names.reduce((text, n) => text + ", " + n)`	`names.Aggregate((text, n) => text + ", " + n)`

Beachte die Reihenfolge der Parameter: In C# kommt der Startwert **zuerst**. Ohne Startwert beginnt `Aggregate` mit dem ersten Element. Bei einer leeren Liste wirft es dann eine `InvalidOperationException`. Mit Startwert liefert es bei einer leeren Liste den Startwert.

Für viele Fälle gibt es fertige Methoden: `Sum`, `Max`, `string.Join`. `Aggregate` lohnt sich, wenn es keine passende Methode gibt. Ein Beispiel: `Sum` gibt es nicht für `TimeSpan`.

```
TimeSpan total = durations.Aggregate(TimeSpan.Zero, (sum, d) => sum + d);
```



EF Core übersetzt `Aggregate` nicht in SQL. Lade die Daten zuerst mit `ToList()` und rufe `Aggregate` danach auf.

Theoretische Informatik: Formale Sprachen

Grundbegriffe formaler Sprachen

Der C#-Compiler muss für jede Datei entscheiden: Ist das ein gültiges Programm oder nicht? Dafür braucht er eine exakte Beschreibung, welche Zeichenketten erlaubt sind. Dasselbe gilt für ein JSON-Dokument, eine IBAN oder ein Datum in einem Formular. Die Theorie der formalen Sprachen liefert die Begriffe, mit denen man solche Regeln präzise aufschreibt. In diesem Kapitel lernst du das Vokabular kennen, das du in den folgenden Kapiteln zu Grammatiken, Automaten, regulären Ausdrücken und Parsern brauchst.

Alphabet

Ein **Alphabet** Σ (Sigma) ist eine endliche, nicht leere Menge von Zeichen. Die Zeichen heißen auch **Symbole**. Was ein Zeichen ist, legst du selbst fest:

Alphabet	Verwendung
Σ = {0, 1}	Binärzahlen
Σ = {a, b}	Beispiele in der Theorie, weil sie kurz sind
Σ = {0, 1, …, 9, .}	Datumsangaben wie `24.12.2025`
Σ = alle Unicode-Zeichen	Quelltext einer C#-Datei
Σ = {`if`, `(`, `)`, `id`, `==`, …}	Tokens, die ein Parser verarbeitet (siehe Parser)

Das letzte Beispiel zeigt: Ein Symbol muss kein einzelner Buchstabe sein. Für einen Parser ist das Schlüsselwort `while` *ein* Symbol, auch wenn es aus 5 Buchstaben besteht.

Wörter

Ein **Wort** über Σ ist eine endliche Folge von Symbolen aus Σ. Über Σ = {a, b} sind z. B. `a`, `abba` und `bbb` Wörter. Man schreibt die Symbole einfach hintereinander, ohne Komma und ohne Klammern.

- Die **Länge** |w| ist die Anzahl der Symbole: |`abba`| = 4, |`b`| = 1.
- Das **leere Wort** ε (Epsilon) hat keine Symbole, also |ε| = 0. Es entspricht dem Leerstring `""` in C#. ε ist *kein* Symbol des Alphabets, sondern ein Wort.
- **Konkatenation** hängt zwei Wörter aneinander: u = `ab`, v = `ba` ergibt u·v = uv = `abba`. Den Punkt lässt man meistens weg. Es gilt εw = wε = w, so wie `"" + s == s` in C#.
- **Potenz** eines Wortes: $w^0 = \varepsilon$, $w^1 = w$, $w^2 = ww$, … So ist $(ab)^3$ = `ababab`. Bei einem einzelnen Symbol schreibt man kurz a^n, z. B. a^3 = `aaa` und a^3b^3 = `aaabbb`.

Für w = `abc` gilt:

- **Präfixe** (Anfangsstücke): ε, `a`, `ab`, `abc`
- **Suffixe** (Endstücke): ε, `c`, `bc`, `abc`
- **Teilwörter** (zusammenhängende Stücke): alle Präfixe und Suffixe, dazu `b`

ε und das Wort selbst sind also immer Präfix und Suffix. In C# entsprechen dem die Methoden `StartsWith`, `EndsWith` und `Contains`.

Σ^n, Σ^* und Σ^+

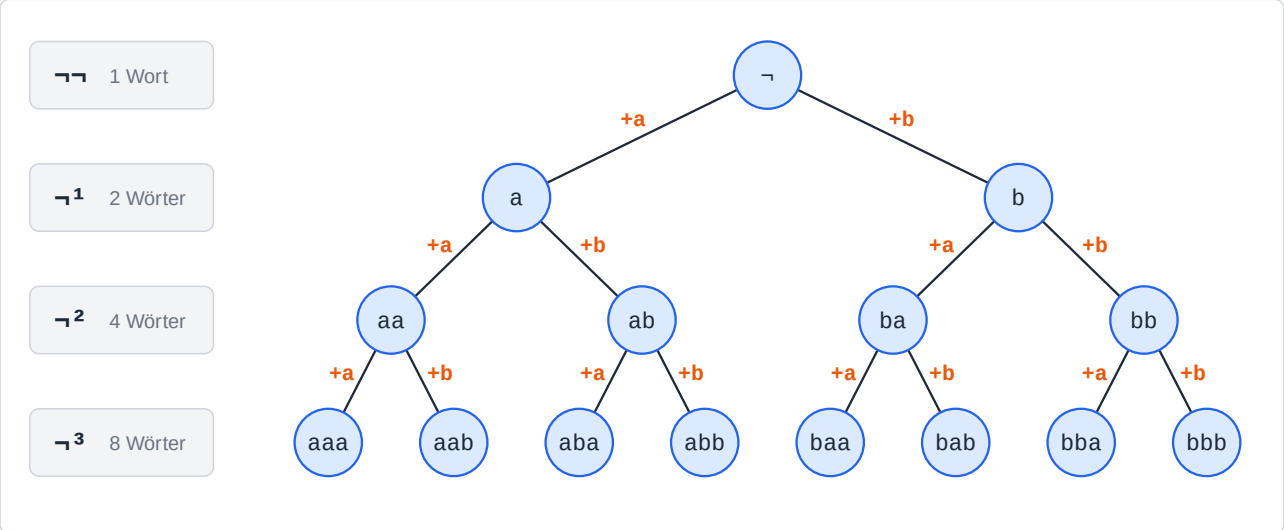
Aus einem Alphabet bildest du Mengen von Wörtern:

- Σ^n ist die Menge aller Wörter mit genau n Symbolen.
- Σ^* (Kleene-Stern) ist die Menge *aller* Wörter, inklusive ε: $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$
- Σ^+ ist dasselbe ohne das leere Wort: $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots = \Sigma^* \setminus \{\varepsilon\}$

Für $\Sigma = \{a, b\}$:

Menge	Wörter	Anzahl
Σ^0	ε	1
Σ^1	a, b	2
Σ^2	aa, ab, ba, bb	4
Σ^3	aaa, aab, aba, abb, baa, bab, bba, bbb	8

Jedes Wort aus Σ^n entsteht, indem du an ein Wort aus Σ^{n-1} ein Symbol anhängst. Daraus ergibt sich der folgende Baum. Jede Ebene enthält Σ^n, und die Anzahl verdoppelt sich mit jeder Ebene. Allgemein hat Σ^n genau $|\Sigma|^n$ Wörter.



Σ^* ist unendlich groß, aber jedes einzelne Wort darin ist endlich lang. Du kannst Σ^* auch der Länge nach aufzählen (ε, a, b, aa, ab, …) und erreichst so jedes Wort nach endlich vielen Schritten.

Wörter über einem Alphabet in C#

In C# stellst du ein Alphabet als `HashSet<char>` und ein Wort als `string` dar. Ob ein Wort über dem Alphabet liegt, prüfst du Zeichen für Zeichen:

```
HashSet<char> binary = ['0', '1'];

Console.WriteLine(IsWordOver("1011", binary));   // True
Console.WriteLine(IsWordOver("", binary));       // True  (1)
Console.WriteLine(IsWordOver("1021", binary));   // False

static bool IsWordOver(string word, IReadOnlySet<char> alphabet) =>
    word.All(alphabet.Contains);                  // (2)
```

- **(1)** Das leere Wort liegt über jedem Alphabet, denn es enthält kein Zeichen, das nicht in Σ vorkommt.
- **(2)** `All` liefert `true`, wenn die Bedingung für jedes Zeichen gilt. Bei einem leeren String ist das automatisch erfüllt.

Σ^n erzeugst du am besten mit einem **Iterator**. Eine Methode mit `yield return` liefert die Wörter einzeln, wenn der Aufrufer sie braucht. Es wird keine Liste mit $|\Sigma|^n$ Elementen im Speicher aufgebaut. Die Methode folgt direkt dem Baum: Für n = 0 gibt es nur ε, sonst hängst du an jedes Wort aus Σ^{n-1} jedes Symbol an.

```
Console.OutputEncoding = System.Text.Encoding.UTF8;   // (1)
char[] alphabet = ['a', 'b'];

for (int n = 0; n <= 3; n++)
{
    var words = WordsOfLength(alphabet, n).Select(w => w == "" ? "ε" : w);
    Console.WriteLine($"Σ^{n} = {{ {string.Join(", ", words)} }}");
}

static IEnumerable<string> WordsOfLength(IReadOnlyList<char> alphabet, int n)
{
    if (n == 0)
    {
        yield return "";                              // (2)
        yield break;
    }
    foreach (string shorter in WordsOfLength(alphabet, n - 1))   // (3)
    {
        foreach (char symbol in alphabet)
        {
            yield return shorter + symbol;
        }
    }
}
```

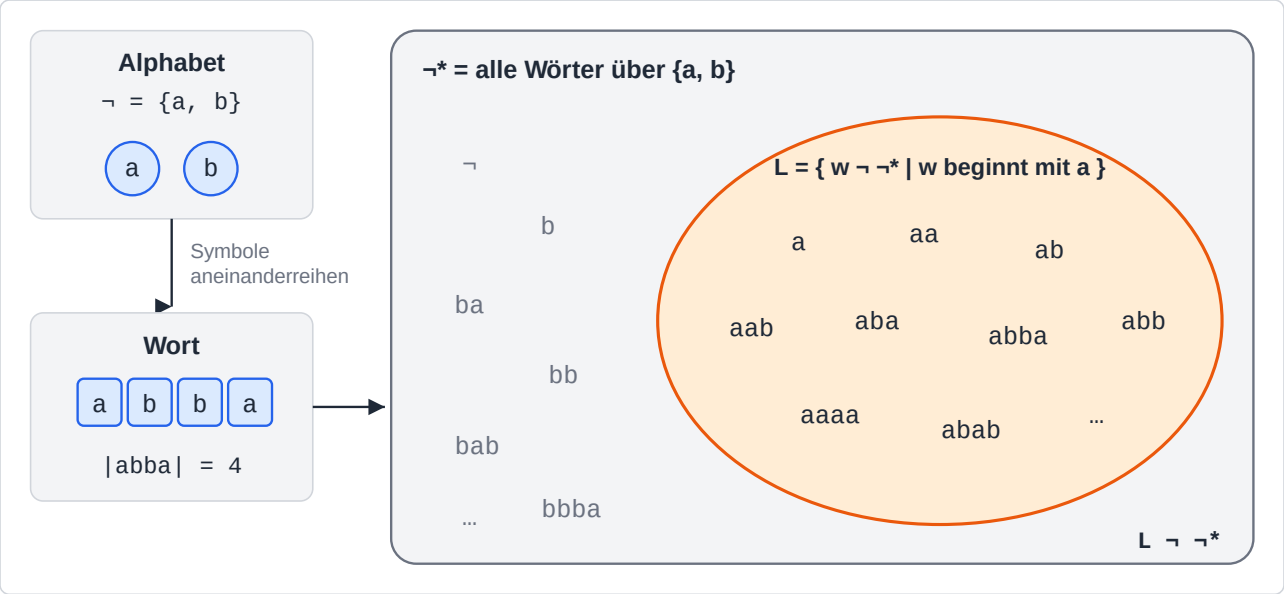
```
Σ^0 = { ε }
Σ^1 = { a, b }
Σ^2 = { aa, ab, ba, bb }
Σ^3 = { aaa, aab, aba, abb, baa, bab, bba, bbb }
```

- **(1)** Damit die Windows-Konsole Σ und ε richtig anzeigt.
- **(2)** $\Sigma^0 = \{\varepsilon\}$. `yield break` beendet den Iterator.
- **(3)** Der rekursive Aufruf liefert die Wörter der Länge n − 1, also die Ebene darüber im Baum.

Rufst du `WordsOfLength` für n = 0, 1, 2, … nacheinander auf, zählst du Σ* der Länge nach auf.

Sprachen

Eine **(formale) Sprache** L über Σ ist eine beliebige Teilmenge von Σ*, also $L \subseteq \Sigma^*$. Mehr steckt in der Definition nicht. Eine Sprache ist einfach eine Menge von Wörtern. Die Kunst liegt darin, diese Menge genau zu beschreiben.



Die Grafik zeigt Σ* für Σ = {a, b} und darin die Sprache L = { w ∈ Σ* | w beginnt mit `a` }. Die Schreibweise { w ∈ Σ* | … } liest du als „alle Wörter w aus Σ*, für die gilt …“.

Eine Sprache kann **endlich** oder **unendlich** sein:

- L_1 = {`if`, `else`, `while`} ist endlich. Sie hat 3 Wörter.
- L_2 = { $a^n b^n$ | n ≥ 0 } = {ε, `ab`, `aabb`, `aaabbb`, …} ist unendlich.
- Die **leere Sprache** Ø enthält kein einziges Wort. Sie ist nicht dasselbe wie {ε}, die Sprache, die genau ein Wort enthält, nämlich das leere.

Beispiele

Binärzahlen. Σ = {0, 1}. Die Sprache L_bin enthält `0` und alle Wörter, die mit `1` beginnen: `0`, `1`, `10`, `11`, `100`, … Das Wort `0101` liegt in Σ*, aber nicht in L_bin, weil es eine führende Null hat.

C#-Bezeichner (vereinfacht). Σ enthält Buchstaben, Ziffern und `_`. Ein Bezeichner beginnt mit einem Buchstaben oder `_`, danach folgen beliebig viele Buchstaben, Ziffern oder `_`. Also gilt `count`, `_tmp`, `x1` ∈ L, aber `1x` ∉ L und ε ∉ L. Die genaue Regel aus der C#-Spezifikation findest du in Darstellungsformen.

Datumsangaben. Σ = {0, …, 9, .}. Die Sprache aller Wörter der Form `TT.MM.JJJJ` enthält `24.12.2025`, aber auch `31.02.2025` und `00.00.0000`. Willst du nur echte Kalenderdaten, ist das eine andere, kleinere Sprache. Formal ist auch sie eine Teilmenge von Σ*, nur ist ihre Beschreibung aufwändiger, weil sie Monatslängen und Schaltjahre berücksichtigen muss. Bevor du eine Sprache beschreibst, musst du also genau festlegen, welche Wörter dazugehören sollen.

Operationen auf Sprachen

Weil Sprachen Mengen sind, gibt es die üblichen Mengenoperationen wie Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$) und Differenz ($L_1 \setminus L_2$). Dazu kommen Operationen, die Wörter aneinanderhängen. Für die Beispiele verwenden wir $L_1 = \{a, ab\}$ und $L_2 = \{b, c\}$.

Vereinigung: Alle Wörter, die in L_1 oder in L_2 liegen.

```
L₁ ∪ L₂ = { a, ab, b, c }
```

Konkatenation: Jedes Wort aus L_1 wird mit jedem Wort aus L_2 verbunden, $L_1 \cdot L_2 = \{ uv \mid u \in L_1, v \in L_2 \}$.

```
L₁·L₂ = { ab, ac, abb, abc }
L₂·L₁ = { ba, bab, ca, cab }
```

Die Reihenfolge ist wichtig, $L_1 \cdot L_2$ und $L_2 \cdot L_1$ sind verschieden. Die Konkatenation hat $\{\varepsilon\}$ als neutrales Element ($L \cdot \{\varepsilon\} = L$), und es gilt $L \cdot \emptyset = \emptyset$, weil es aus $\emptyset$ kein Wort zum Anhängen gibt.

Potenz: $L^0 = \{\varepsilon\}$, $L^1 = L$, $L^2 = L \cdot L$, $L^3 = L \cdot L \cdot L$, …

```
L₁⁰ = { ε }
L₁² = { aa, aab, aba, abab }
```

L_1^2 hat 4 Wörter, weil $2 \cdot 2$ Kombinationen möglich sind. Das gilt nicht immer. Für $L = \{a, aa\}$ ergibt `a·aa` und `aa·a` dasselbe Wort, daher hat $L^2 = \{aa, aaa, aaaa\}$ nur 3 Wörter.

Kleene-Stern: $L^* = L^0 \cup L^1 \cup L^2 \cup \ldots$ enthält alle Wörter, die man aus beliebig vielen (auch null) Wörtern aus L zusammensetzen kann. $L^+ = L^1 \cup L^2 \cup \ldots$ verlangt mindestens ein Wort.

```
{ab}*   = { ε, ab, abab, ababab, ⋯ }
{ab}⁺   = { ab, abab, ababab, ⋯ }
{a, b}* = Σ*  für Σ = {a, b}
```

Die letzte Zeile zeigt, woher der Name Σ* kommt: Du fasst das Alphabet als Sprache aus einbuchstabigen Wörtern auf und wendest den Stern an. Für jede Sprache L gilt $\varepsilon \in L^*$, sogar für $L = \emptyset$, denn $\emptyset^* = \{\varepsilon\}$.

Diese drei Operationen (Vereinigung, Konkatenation, Stern) begegnen dir in Reguläre Ausdrücke wieder: `a|b`, `ab` und `a*` sind genau Vereinigung, Konkatenation und Kleene-Stern.

Operationen auf endlichen Sprachen in C#

Für endliche Sprachen kannst du die Operationen mit LINQ direkt nachbauen. Eine Sprache ist dann ein `HashSet<string>`, damit doppelte Wörter automatisch verschwinden.

```
HashSet<string> l1 = ["a", "ab"];
HashSet<string> l2 = ["b", "c"];

Console.WriteLine(Show(Union(l1, l2)));         // { a, ab, b, c }
Console.WriteLine(Show(Concat(l1, l2)));        // { ab, ac, abb, abc }
Console.WriteLine(Show(Power(l1, 2)));          // { aa, aab, aba, abab }
Console.WriteLine(Show(Power(["a", "aa"], 2)));// { aa, aaa, aaaa }

static HashSet<string> Union(IEnumerable<string> l1, IEnumerable<string> l2) =>
    [.. l1.Union(l2)];

static HashSet<string> Concat(IEnumerable<string> l1, IEnumerable<string> l2) =>
    [.. l1.SelectMany(_ => l2, (u, v) => u + v)];   // (1)

static HashSet<string> Power(IEnumerable<string> language, int n)
{
    HashSet<string> result = [""];                  // (2)
    for (int i = 0; i < n; i++)
    {
        result = Concat(result, language);
    }
    return result;
}

static string Show(IEnumerable<string> language) =>
    "{ " + string.Join(", ", language.Select(w => w == "" ? "ε" : w)) + " }";
```

- **(1)** `SelectMany` bildet alle Paare (u, v) mit u aus L_1 und v aus L_2, so wie zwei verschachtelte `foreach`-Schleifen.
- **(2)** Der Startwert ist $L^0 = \{\varepsilon\}$, also eine Menge mit dem Leerstring. Eine leere Menge wäre falsch, denn $\emptyset \cdot L = \emptyset$.

L* kannst du so nicht berechnen, weil die Menge unendlich ist, sobald L ein nicht leeres Wort enthält. Du kannst aber alle Wörter von L* bis zu einer Maximallänge erzeugen.

Syntax und Semantik

Eine formale Sprache legt nur fest, *welche* Zeichenketten erlaubt sind. Das ist die **Syntax**. Was eine Zeichenkette *bedeutet*, ist die **Semantik**. Der Satz „Der Kühlschrank singt grüne Primzahlen.“ ist syntaktisch korrektes Deutsch, hat aber keinen sinnvollen Inhalt.

Bei C# meldet der Compiler einen Syntaxfehler, wenn der Quelltext nicht zur Sprache gehört:

```
Console.WriteLine("Hallo")

error CS1002: ; expected
```

Der Compiler prüft aber mehr als die reine Syntax. `int x = "Hallo";` ist syntaktisch korrekt, denn es hat dieselbe Form wie `int x = 42;`. Trotzdem lehnt der Compiler es ab, weil die Typen nicht passen. Solche Regeln heißen **statische Semantik**. Sie lassen sich nicht mehr mit den Grammatiken aus den nächsten Kapiteln ausdrücken und werden in einem eigenen Schritt geprüft.

Ob ein Programm das tut, was du *gemeint* hast, prüft kein Compiler. Der folgende Code ist syntaktisch korrekt, kompiliert ohne Warnung und liefert trotzdem das falsche Ergebnis:

```
int[] grades = [1, 2, 2, 5];
double average = grades.Sum() / grades.Length;    // (1)
Console.WriteLine($"Average: {average}");
```

```
Average: 2
```

- **(1)** `Sum()` und `Length` sind `int`, daher ist `/` eine Ganzzahldivision: 10 / 4 = 2. Erst danach wird das Ergebnis in `double` umgewandelt. Gemeint war 2.5, richtig wäre `(double)grades.Sum() / grades.Length` oder einfach `grades.Average()`.

Für diesen Abschnitt merkst du dir:

Ebene	Frage	Wer prüft?
Syntax	Ist die Zeichenkette ein Wort der Sprache?	Lexer und Parser
Statische Semantik	Passen Typen, sind Variablen deklariert?	Compiler nach dem Parsen
Semantik	Was bedeutet das Programm, tut es das Richtige?	Tests und Menschen

Die Kapitel Grammatiken bis Parser behandeln die erste Zeile: Wie beschreibt man eine Sprache, und wie prüft man, ob ein Wort dazugehört?

Grammatiken

Eine unendliche Sprache wie „alle korrekt geklammerten Ausdrücke“ kannst du nicht aufzählen. Du brauchst eine endliche Beschreibung, aus der sich jedes Wort der Sprache bauen lässt. Genau das leistet eine Grammatik: ein paar Ersetzungsregeln, die vom Startsymbol zum fertigen Wort führen. Jede Programmiersprache ist so definiert, auch C#. Wer die Grammatik lesen kann, versteht Compilerfehler besser und kann später selbst einen Parser schreiben (siehe Parser für arithmetische Ausdrücke).



Zum Selbst-Erarbeiten: der interaktive Ableitungstrainer

Im Ableitungstrainer siehst du als Animation, wie bei einer Linksableitung der Ableitungsbaum wächst. Danach leitest du selbst Wörter ab, vom einfachen $a^n b^n$ bis zu mehrdeutigen Ausdrücken, und bekommst sofort Rückmeldung. Nach ein paar Aufgaben dort sitzt das Ableiten viel sicherer als nur vom Lesen.

Ein erstes Beispiel

Die folgende Grammatik erzeugt Wörter wie `ab`, `aabb` oder `aaabbb`:

```
S → aSb | ε
```

Lies das so: Das Symbol `S` darfst du durch `aSb` ersetzen *oder* durch das leere Wort ε. Der Strich `|` trennt die Alternativen. Du beginnst immer mit `S` und ersetzt so lange, bis kein `S` mehr dasteht:

```
S ⇒ aSb ⇒ aaSbb ⇒ aabb
```

Jeder Pfeil ⇒ ist ein *Ableitungsschritt*. In den ersten beiden Schritten hast du `S → aSb` angewendet, im letzten `S → ε`. Jede Anwendung von `S → aSb` setzt ein `a` vorne und ein `b` hinten dazu. Deshalb erzeugt die Grammatik genau die Wörter $a^n b^n$ mit $n \geq 0$, also gleich viele `a` gefolgt von gleich vielen `b`. Wörter wie `abab` oder `aab` kannst du nicht ableiten.

Formale Definition

Eine Grammatik ist ein Quadrupel $G = (N, \Sigma, P, S)$:

- **N** ist eine endliche Menge von *Nichtterminalen* (auch Variablen). Sie sind Platzhalter und kommen im fertigen Wort nicht mehr vor.
- **Σ** ist das Alphabet der *Terminale*. Das sind die Zeichen, aus denen die Wörter der Sprache bestehen. N und Σ haben keine gemeinsamen Elemente.
- **P** ist eine endliche Menge von *Produktionen* (Regeln) der Form $\alpha \to \beta$. Links steht mindestens ein Nichtterminal.
- **S** $\in N$ ist das *Startsymbol*.

Für das Beispiel oben gilt also G = ({S}, {a, b}, {S → aSb, S → ε}, S).

In diesem Kapitel gilt folgende Konvention: Großbuchstaben (`S`, `E`, `T`) sind Nichtterminale, alles andere sind Terminale. Bis zum Abschnitt über die Chomsky-Hierarchie steht links in jeder Produktion genau **ein** Nichtterminal. Solche Grammatiken heißen *kontextfrei*.

Ableitung und Sprache

Eine Folge aus Terminalen und Nichtterminalen heißt *Satzform*. `aaSbb` ist eine Satzform, `aabb` auch (sie enthält nur Terminale).

- **u** ⇒ **v** (ein Ableitungsschritt): Du ersetzt in u ein Nichtterminal A durch die rechte Seite einer Produktion A → γ und bekommst v. Beispiel: `aSb ⇒ aaSbb`.
- **u** ⇒* **v** (Ableitung): v entsteht aus u in null oder mehr Schritten. Beispiel: `S ⇒* aabb`.

Die *Sprache* der Grammatik besteht aus allen Wörtern, die nur aus Terminalen bestehen und sich aus dem Startsymbol ableiten lassen:

```
L(G) = { w ∈ Σ* | S ⇒* w }
```

Satzformen mit Nichtterminalen wie `aSb` sind Zwischenschritte. Sie gehören nicht zu L(G).

Weitere Beispiele

Korrekt geklammerte Ausdrücke

Jede öffnende Klammer braucht eine passende schließende. `()`, `(())` und `()(())` sind korrekt, `)(` und `(()` nicht:

```
K → ( K ) K | ε
```

Die Regel sagt: Ein korrekter Ausdruck ist leer, oder er beginnt mit einem Klammerpaar, in dem ein korrekter Ausdruck steht. Danach folgt wieder ein korrekter Ausdruck. Hier ist N = {K} und Σ = {(,)}.

Einfache arithmetische Ausdrücke

```
E → E + E | E * E | ( E ) | id
```

Das Terminal `id` steht für eine Zahl oder einen Variablennamen. Ein Lexer fasst die Zeichen `42` oder `count` vorher zu einem Symbol zusammen (das machst du in 15). So kannst du z. B. `id * ( id + id )` ableiten:

```
E ⇒ E * E ⇒ id * E ⇒ id * ( E ) ⇒ id * ( E + E ) ⇒ id * ( id + E ) ⇒ id * ( id +
```

```
id )
```

Links- und Rechtsableitung

Enthält eine Satzform mehrere Nichtterminale, darfst du dir aussuchen, welches du ersetzt. Damit Ableitungen vergleichbar werden, legt man die Reihenfolge fest:

- Bei der **Linksableitung** ersetzt du immer das *linkeste* Nichtterminal.
- Bei der **Rechtsableitung** ersetzt du immer das *rechteste* Nichtterminal.

Beide Varianten für das Wort `()()` mit der Klammer-Grammatik:

```
Linksableitung:   K ⇒ (K)K ⇒ ()K ⇒ ()(K)K ⇒ ()()K ⇒ ()()
Rechtsableitung:  K ⇒ (K)K ⇒ (K)(K)K ⇒ (K)(K) ⇒ (K)() ⇒ ()()
```

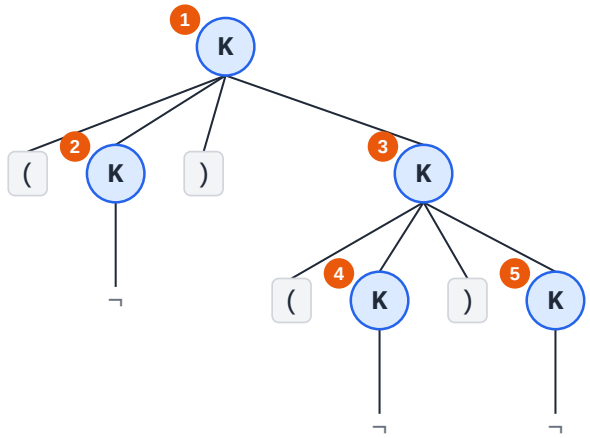
Die Ableitung von `id * ( id + id )` im letzten Abschnitt war eine Linksableitung. Die Linksableitung ist wichtig, weil ein Parser, der den Text von links nach rechts liest, genau sie nachbaut. Der rekursive Abstiegsparser in Parser für arithmetische Ausdrücke macht das so.

Der Ableitungsbaum

Ein *Ableitungsbaum* (Syntaxbaum) zeigt, welche Produktion auf welches Nichtterminal angewendet wurde. Die Reihenfolge der Schritte zeigt er nicht.

- Die Wurzel ist das Startsymbol.
- Jeder innere Knoten ist ein Nichtterminal. Seine Kinder sind die Symbole der rechten Seite der angewendeten Produktion, von links nach rechts.
- Die Blätter sind Terminale oder ε. Liest du sie von links nach rechts, bekommst du das Wort.

Ableitungsbaum für ()() mit K ¬ (K) K | ¬



Linksableitung

	K	
1	¬ (K)K	K ¬ (K) K
2	¬ ()K	K ¬ ¬
3	¬ ()(K)K	K ¬ (K) K
4	¬ ()()K	K ¬ ¬
5	¬ ()()	K ¬ ¬

Blätter von links nach rechts gelesen (¬ weggelassen): ( ) ( ) = ()()

Die orangen Nummern zeigen, in welchem Schritt der Linksableitung der Knoten ersetzt wird.

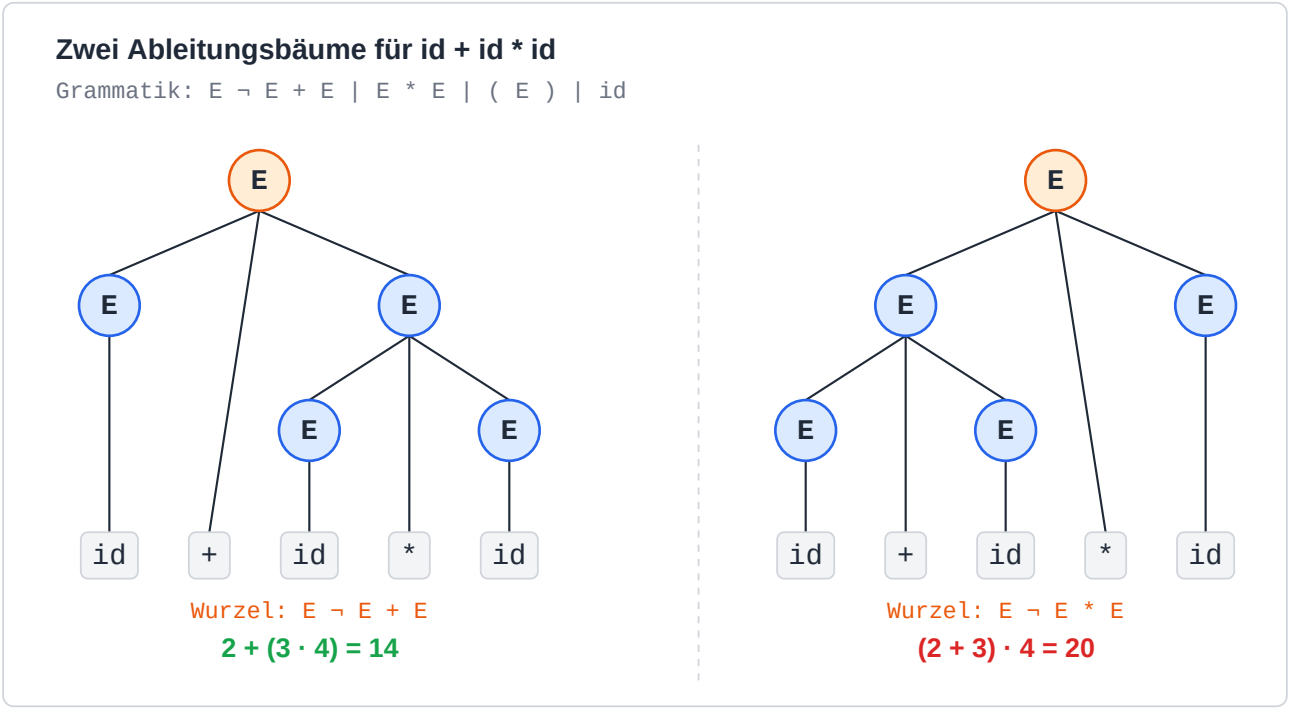
Die Rechtsableitung von oben ersetzt dieselben Knoten in anderer Reihenfolge, liefert aber **denselben Baum**. Zu jedem Baum gibt es genau eine Linksableitung und umgekehrt. Deshalb kannst du Bäume zählen, indem du Linksableitungen zählst.

Mehrdeutigkeit

Eine Grammatik ist *mehrdeutig*, wenn es ein Wort mit zwei verschiedenen Ableitungsbäumen gibt. Gleichwertig: Das Wort hat zwei verschiedene Linksableitungen. Die Grammatik für arithmetische Ausdrücke von oben ist mehrdeutig. Für `id + id * id` gibt es zwei Linksableitungen:

```
E ⇒ E + E ⇒ id + E ⇒ id + E * E ⇒ id + id * E ⇒ id + id * id
E ⇒ E * E ⇒ E + E * E ⇒ id + E * E ⇒ id + id * E ⇒ id + id * id
```

Die erste Ableitung beginnt mit `E → E + E`, die zweite mit `E → E * E`. Das ergibt zwei verschiedene Bäume:



Für die Syntax ist das egal, beide Male entsteht dasselbe Wort. Für die Bedeutung ist es fatal: Ein Compiler wertet den Baum von unten nach oben aus. Setzt du für die drei `id` die Zahlen 2, 3 und 4 ein, liefert der linke Baum 2 + (3 · 4) = 14, der rechte (2 + 3) · 4 = 20. Eine Grammatik für eine Programmiersprache muss daher eindeutig sein.

Auflösung über Präzedenzstufen

Die Lösung kennst du aus der Mathematik: Punkt- vor Strichrechnung. Du führst für jede Präzedenzstufe ein eigenes Nichtterminal ein:

```
E → E + T | T          (1)
T → T * F | F          (2)
```

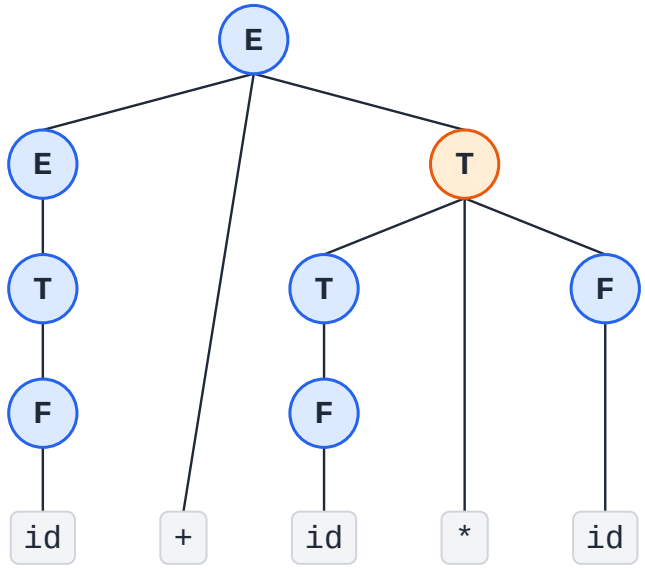
```
F → ( E ) | id          (3)
```

- **(1)** Ein Ausdruck E (expression) ist eine Summe von Termen T. Auf dieser Ebene gibt es nur `+`.
- **(2)** Ein Term T ist ein Produkt von Faktoren F. `*` gibt es nur auf dieser Ebene, also tiefer im Baum. Was tiefer liegt, wird zuerst ausgewertet.
- **(3)** Ein Faktor F ist ein `id` oder ein geklammerter Ausdruck. Mit der Klammer kommst du wieder ganz nach oben zu E. So kannst du `( id + id ) * id` schreiben.

Jetzt gibt es für `id + id * id` nur noch eine Linksableitung und einen Baum:

```
E ⇒ E + T ⇒ T + T ⇒ F + T ⇒ id + T ⇒ id + T * F ⇒ id + F * F ⇒ id + id * F ⇒ id
+ id * id
```

Eindeutiger Baum für id + id * id




```
E ¬ E + T | T
T ¬ T * F | F
F ¬ ( E ) | id
```

+ hängt an E (oben),
* hängt an T (tiefer).

T ¬ T * F wird zuerst ausgewertet: 3 · 4 = 12, **dann 2 + 12 = 14**

Die Produktion `E → E + T` hat ihr E links. Deshalb wird `id + id + id` als `(id + id) + id` gelesen, so wie in C#. Man nennt das *Linksrekursion*, sie macht den Operator linksassoziativ. Ein rekursiver Abstiegsparser kann Linksrekursion nicht direkt umsetzen, er würde sich endlos selbst aufrufen. In Darstellungsformen von Grammatiken lernst du die EBNF kennen. Dort schreibst du die Wiederholung mit `{ }`, und die Grammatik wird zu genau der Form, die du in Parser für arithmetische Ausdrücke programmierst (dort mit `-`, `/`, Zahlen und Vorzeichen):

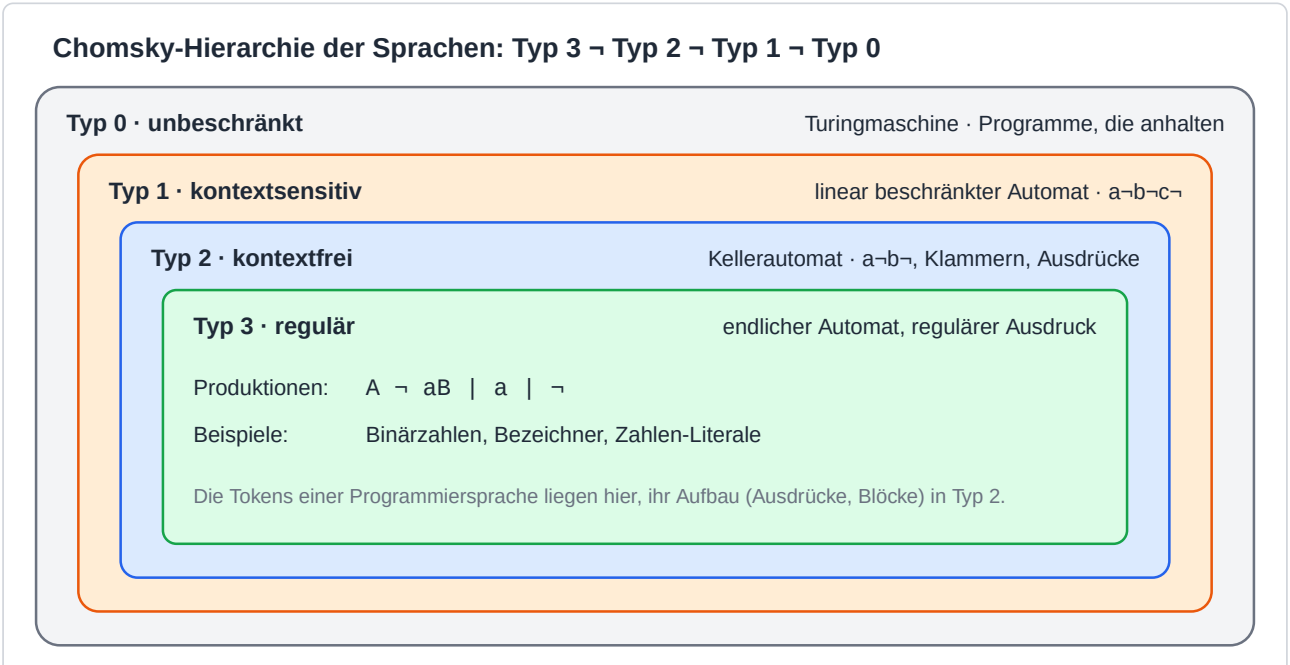
```
expr = term { "+" term }
term = factor { "*" factor }
factor = "(" expr ")" | "id"
```

Die Chomsky-Hierarchie

Bisher stand links in jeder Produktion genau ein Nichtterminal. Erlaubt man links mehrere Symbole, werden Grammatiken mächtiger. Der Linguist Noam Chomsky hat in den 1950er-Jahren vier Typen unterschieden. Je höher die Nummer, desto strenger die Regeln für die Produktionen und desto einfacher der Automat, der die Sprache erkennt:

Typ	Name	Form der Produktionen	Automatenmodell	Beispiel
3	regulär	$A \to aB$, $A \to a$, $A \to \varepsilon$	endlicher Automat	Binärzahlen, Bezeichner
2	kontextfrei	$A \to \gamma$ (links genau ein Nichtterminal)	Kellerautomat	$a^n b^n$, Klammern, Ausdrücke
1	kontextsensitiv	$\alpha \to \beta$ mit $\lvert\alpha\rvert \le \lvert\beta\rvert$	linear beschränkter Automat	$a^n b^n c^n$
0	unbeschränkt	$\alpha \to \beta$ (α enthält ein Nichtterminal)	Turingmaschine	Programme, die anhalten

Dabei sind $A, B \in N$, $a \in \Sigma$ und α, β, γ beliebige Folgen aus $N \cup \Sigma$. Bei Typ 1 darf die rechte Seite nie kürzer sein als die linke. Damit ε trotzdem zur Sprache gehören kann, ist zusätzlich $S \to \varepsilon$ erlaubt, wenn S auf keiner rechten Seite vorkommt.



Ein paar Beispiele zu den Typen:

- **Typ 3:** Binärzahlen ohne führende Nullen erzeugt `Z → 0 | 1R` mit `R → 0R | 1R | ε`. Rechts steht höchstens ein Nichtterminal, und zwar ganz hinten. Solche Sprachen erkennt ein endlicher Automat (Endliche Automaten) oder ein regulärer Ausdruck (Reguläre Ausdrücke).
- **Typ 2:** `S → aSb | ε` ist kontextfrei, aber nicht regulär, weil rechts nach dem `S` noch ein `b` steht. Ein endlicher Automat kann $a^n b^n$ nicht erkennen, denn er müsste sich die Anzahl der `a`

merken, und die ist unbegrenzt. Ein Kellerautomat hat dafür einen Stack.

- **Typ 1:** $a^n b^n c^n$ ($n \geq 1$) ist nicht kontextfrei. Eine Grammatik dafür braucht Regeln mit zwei Symbolen links:

```
S → aSBC | aBC      CB → BC      aB → ab      bB → bb      bC → bc      cC → cc

S ⇒ aSBC ⇒ aaBCBC ⇒ aaBBCC ⇒ aabBCC ⇒ aabbCC ⇒ aabbcC ⇒ aabbcc
```

Die Regel `CB → BC` vertauscht zwei Nichtterminale. Die anderen Regeln ersetzen ein Nichtterminal nur, wenn links davon das passende Terminal steht, also in einem bestimmten *Kontext*. Daher kommt der Name.

Grammatik oder Sprache einordnen

Den Typ einer **Grammatik** liest du an den Produktionen ab: Es zählt der höchste Typ, dessen Form alle Produktionen erfüllen. Bei einer **Sprache** zählt der höchste Typ, für den es irgendeine Grammatik gibt. Das kann ein Unterschied sein:

```
S → aS | Sb | ε
```

Die Grammatik ist wegen `S → Sb` vom Typ 2, das Nichtterminal steht nicht hinten. Sie erzeugt aber die Sprache „beliebig viele `a`, dann beliebig viele `b`“, und die ist regulär (Typ 3).

Für Programmiersprachen bedeutet das: Einzelne Tokens wie Zahlen und Bezeichner sind regulär. Der Aufbau von Ausdrücken, Anweisungen und Klassen ist kontextfrei. Regeln wie „eine Variable muss deklariert sein, bevor du sie verwendest“ sind nicht kontextfrei. Der Compiler prüft sie deshalb in einem eigenen Schritt nach dem Parsen.

Grammatiken in C#

Eine kontextfreie Grammatik lässt sich leicht als Datenstruktur speichern. Eine Produktion ist ein `record` aus der linken Seite und der Liste der Symbole rechts. Mit einer Linksableitung, bei der du die Produktion zufällig wählst, erzeugst du Wörter der Sprache:

```
Console.OutputEncoding = System.Text.Encoding.UTF8;

var anbn = new Grammar("S",
[
    new Production("S", ["a", "S", "b"]),
    new Production("S", []),                        // (1)
]);
var binary = new Grammar("Z",
[
    new Production("Z", ["0"]),
    new Production("Z", ["1", "R"]),
    new Production("R", ["0", "R"]),
    new Production("R", ["1", "R"]),
```

```
    new Production("R", []),
]);

var random = new Random(1);                          // (2)
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(string.Join(" ⇒ ", anbn.Derive(random)));
}
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(string.Join(" ⇒ ", binary.Derive(random)));
}

/// <summary>A production Left → Right. An empty Right stands for ε.</summary>
record Production(string Left, string[] Right);

/// <summary>
/// A context-free grammar. Every symbol that appears on the left side of a production
is a
/// nonterminal, all other symbols are terminals.
/// </summary>
class Grammar(string start, Production[] productions)
{
    /// <summary>Returns true if <paramref name="symbol"/> is a nonterminal of this
grammar.</summary>
    public bool IsNonterminal(string symbol) => productions.Any(p => p.Left == symbol);

    /// <summary>
    /// Creates a random word with a leftmost derivation. Returns all sentential forms,
    /// from the start symbol to the word.
    /// </summary>
    public List<string> Derive(Random random)
    {
        List<string> form = [start];
        List<string> steps = [Show(form)];
        int index;
        while ((index = form.FindIndex(IsNonterminal)) != -1)             // (3)
        {
            Production[] candidates = [.. productions.Where(p => p.Left == form[
index])];
            Production chosen = candidates[random.Next(candidates.Length)];
            form.RemoveAt(index);                                          // (4)
            form.InsertRange(index, chosen.Right);
            steps.Add(Show(form));
        }
        return steps;
    }

    private static string Show(List<string> form) => form.Count == 0 ? "ε" : string
.Concat(form);
}
```

- **(1)** Die ε-Produktion hat eine leere rechte Seite. Symbole sind Strings, damit auch Terminale wie `id` aus mehreren Zeichen möglich sind.

- **(2)** Mit einem festen Seed liefert `Random` bei jedem Start dieselben Zahlen. Das ist praktisch zum Testen.
- **(3)** `FindIndex` sucht das linkeste Nichtterminal der Satzform. Gibt es keines mehr, ist das Wort fertig.
- **(4)** Das Nichtterminal wird durch die rechte Seite der gewählten Produktion ersetzt. Bei ε verschwindet es einfach.

Die Ausgabe:

```
S ⇒ aSb ⇒ aaSbb ⇒ aaaSbbb ⇒ aaabbb
S ⇒ ε
S ⇒ aSb ⇒ aaSbb ⇒ aabb
Z ⇒ 0
Z ⇒ 1R ⇒ 10R ⇒ 100R ⇒ 1000R ⇒ 1000
Z ⇒ 1R ⇒ 11R ⇒ 110R ⇒ 1101R ⇒ 1101
```

Jedes erzeugte Wort liegt in L(G), denn es entsteht durch eine Ableitung aus dem Startsymbol. Je öfter du `Derive` aufrufst, desto mehr verschiedene Wörter der Sprache siehst du.

Zusatzmaterial

Im Ableitungstrainer siehst du animiert, wie bei einer Linksableitung der Baum wächst, und leitest danach selbst Zielwörter ab, vom einfachen $a^n b^n$ bis zu mehrdeutigen Ausdrücken.

Darstellungsformen von Grammatiken

In Grammatiken schreibst du Produktionen wie `S → aSb | ε`. Für kleine Beispiele ist das ideal. Echte Sprachspezifikationen wie die von C# oder JSON brauchen aber eine Schreibweise, die man in einer Textdatei tippen kann, die auch bei hunderten Regeln lesbar bleibt und die Wiederholungen ohne Rekursion ausdrückt. Dafür gibt es BNF, EBNF und Syntaxdiagramme. Alle drei beschreiben dasselbe, nämlich kontextfreie Grammatiken. Wenn du sie lesen und ineinander umrechnen kannst, verstehst du jede Sprachspezifikation und kannst in Parser direkt aus einer EBNF einen Parser programmieren.

BNF

Die **Backus-Naur-Form** stammt aus dem Bericht über die Sprache ALGOL 60 (1960) von John Backus und Peter Naur. Sie ist die Grammatik aus dem letzten Kapitel, nur anders aufgeschrieben:

Grammatik	BNF	Bedeutung
`A`	`<a>`	Nichtterminal in spitzen Klammern, der Name darf länger sein
`→`	`::=`	„wird ersetzt durch“
`|`	`|`	Alternative
`x`	`"x"`	Terminal
`ε`	`ε`	leeres Wort

Im Original stehen die Terminale ohne Anführungszeichen. Wir setzen sie in `"…"`, damit du sie sofort von den Metazeichen `|`, `<` und `>` unterscheiden kannst. Eine natürliche Zahl sieht in BNF so aus:

```
<number> ::= <digit> | <number> <digit>
<digit>  ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

BNF kennt keine Schleifen. Eine Zahl mit beliebig vielen Ziffern entsteht nur über die Rekursion `<number> ::= <number> <digit>`. Die Ableitung von `427` zeigt das:

```
<number> ⇒ <number> <digit> ⇒ <number> <digit> <digit> ⇒ <digit> <digit> <digit> ⇒*
427
```

Auch arithmetische Ausdrücke mit den Präzedenzstufen aus Grammatiken lassen sich in BNF aufschreiben. Die Linksrekursion `<expr> "+" <term>` sorgt dafür, dass `1 - 2 - 3` als `(1 - 2) - 3` gelesen wird:

```
<expr>   ::= <term> | <expr> "+" <term> | <expr> "-" <term>
```

```
<term>   ::= <factor> | <term> "*" <factor> | <term> "/" <factor>
<factor> ::= <number> | "(" <expr> ")" | "-" <factor>
```

EBNF

Die **erweiterte BNF** ergänzt die BNF um Klammern für Option und Wiederholung. Niklaus Wirth hat 1977 eine einfache Form vorgeschlagen, 1996 wurde EBNF als ISO/IEC 14977 genormt. In diesem Kurs verwenden wir die folgende Schreibweise. Du findest sie auch in Parser und im Übungsprojekt JsonParser.

Schreibweise	Bedeutung	Beispiel
`name = ⋯`	eine Regel pro Zeile, ohne Abschlusszeichen	`sign = "+" | "-"`
`"x"`	Terminal, steht in Anführungszeichen	`"while"`, `"("`
`a b`	Folge (Konkatenation), getrennt durch Leerzeichen	`"(" expr ")"`
`a | b`	Alternative	`"true" | "false"`
`( ⋯ )`	Gruppe	`("+" | "-") term`
`[ ⋯ ]`	Option: 0- oder 1-mal	`[ "-" ] number`
`{ ⋯ }`	Wiederholung: 0- bis n-mal	`digit { digit }`

Nichtterminale schreiben wir ohne spitze Klammern, klein und mit `_` (`integer_literal`). Die Folge bindet stärker als `|`. `a b | c` bedeutet also `(a b) | c` und nicht `a (b | c)`. Brauchst du das Zweite, setzt du Klammern.

Damit werden die Regeln von oben kürzer und kommen ohne Rekursion aus:

```
number = digit { digit }
digit  = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
expr   = term { ("+" | "-") term }
term   = factor { ("*" | "/") factor }
factor = number | "(" expr ")" | "-" factor
```

- `digit { digit }` heißt „eine Ziffer, dann beliebig viele weitere“, also mindestens eine. Das Muster `x { x }` für „1- bis n-mal“ brauchst du ständig.
- `expr = term { ("+" | "-") term }` liest du als „ein Term, danach beliebig oft ein Plus oder Minus mit einem weiteren Term“. Genau diese drei Regeln setzt du in Parser in C# um.
- In `factor` bleibt Rekursion übrig: `"(" expr ")"` ruft wieder `expr` auf. Klammern können beliebig tief geschachtelt sein. Das lässt sich nicht durch eine Wiederholung ersetzen.

Manche Zeichenmengen sind zum Aufzählen zu groß, z. B. alle Buchstaben. Für Bereiche schreiben wir `"a" | ⋯ | "z"`. Was sich gar nicht sinnvoll aufzählen lässt, beschreibst du wie in ISO 14977 in Worten zwischen Fragezeichen: `letter = ? ein Unicode-Buchstabe ?`.

Andere EBNF-Varianten

Es gibt nicht *die* EBNF. Jede Spezifikation verwendet ihre eigene Variante. Die Ideen sind dieselben, nur die Zeichen unterscheiden sich. Dieselbe Regel in vier Schreibweisen:

Variante	Regel
ISO 14977	`expr = term, { ("+" | "-"), term };`
Wirth (1977)	`expr = term { ("+" | "-") term } .`
W3C (z. B. XML-Spezifikation)	`expr ::= term (("+" | "-") term)*`
ANTLR	`expr : term (('+' | '-') term)* ;`

- **ISO 14977** trennt die Elemente einer Folge mit `,` und beendet jede Regel mit `;`.
- **Wirth** beendet jede Regel mit einem Punkt.
- **W3C** und **ANTLR** verwenden die Quantoren aus den regulären Ausdrücken: `x*` für `{ x }`, `x?` für `[ x ]` und `x+` für `x { x }`. Du triffst sie in Reguläre Ausdrücke wieder.

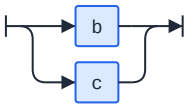
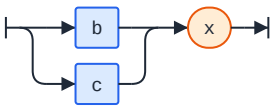
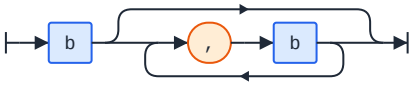
Die C#-Sprachspezifikation verwendet übrigens die ANTLR-Schreibweise. Wenn du die Notation von oben beherrschst, kannst du sie lesen.

Syntaxdiagramme

Wirth hat für die Sprache Pascal (1973) eine grafische Form erfunden: das **Syntaxdiagramm**, auch *railroad diagram* genannt. Du liest es wie einen Gleisplan:

- Du startest links beim senkrechten Strich und folgst den Linien in Pfeilrichtung bis zum Strich rechts.
- Ein **abgerundetes Feld** (orange) ist ein Terminal. Du liest genau diese Zeichen.
- Ein **Rechteck** (blau) ist ein Nichtterminal. Hier springst du in das Diagramm mit diesem Namen und kommst danach zurück.
- An einer **Weiche** darfst du einen der Wege wählen. Jeder Weg vom Start zum Ende ergibt ein gültiges Wort.

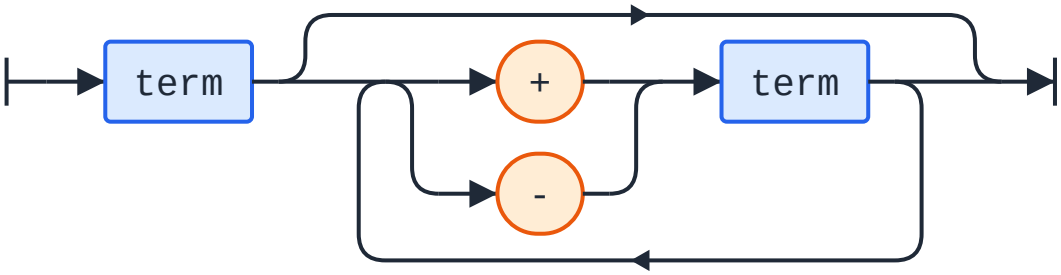
Jedes EBNF-Konstrukt hat ein festes Gegenstück im Diagramm. Die Tabelle stellt alle drei Schreibweisen nebeneinander:

Konstrukt	BNF	EBNF	Syntaxdiagramm
Folge	`<a> ::= <b> "x"`	`a = b "x"`	
Alternative	`<a> ::= <b> | <c>`	`a = b | c`	
Gruppe	`<a> ::= <b> "x" | <c> "x"`	`a = ( b | c ) "x"`	
Option	`<a> ::= "x" <b> | <b>`	`a = [ "x" ] b`	
Wiederholung	`<a> ::= <b> | <a> "," <b>`	`a = b { "," b }`	

- **Folge:** Die Felder stehen hintereinander auf einer Linie.
- **Alternative:** Die Linie teilt sich und läuft danach wieder zusammen.
- **Option:** Eine Umgehung führt am Element vorbei.
- **Wiederholung:** Die Umgehung erlaubt 0 Durchläufe, die Rückführung unten (Pfeil nach links) führt zurück an den Anfang und erlaubt beliebig viele weitere.

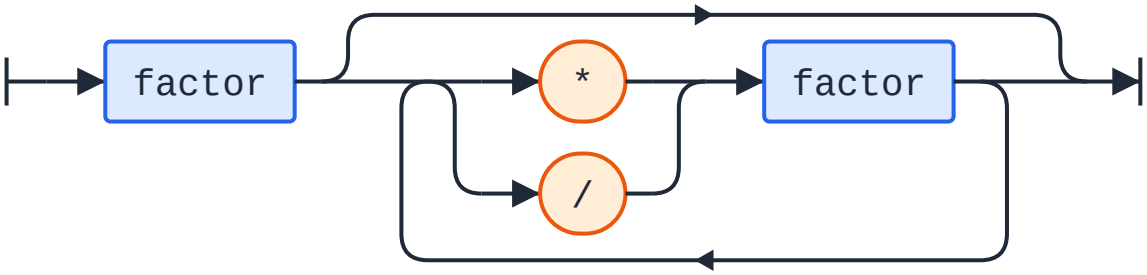
Die drei Regeln für arithmetische Ausdrücke sehen als Syntaxdiagramm so aus:

expr



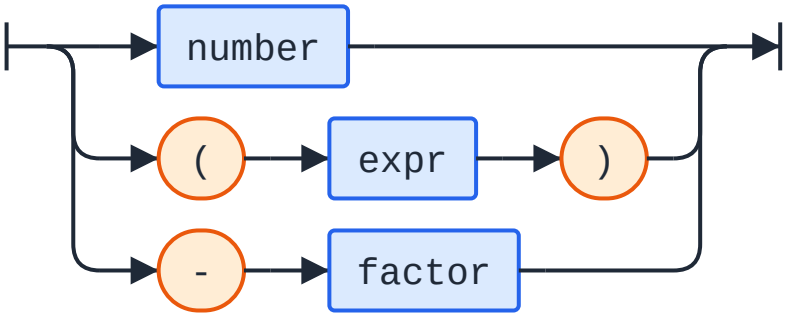

```
expr = term { ("+" | "-") term }
```

term




```
term = factor { ("*" | "/") factor }
```

factor




```
factor = number | "(" expr ")" | "-" factor
```

Probiere es mit `2 * (3 + 4)`: In `expr` gehst du in `term`, dort in `factor` und liest `2` als `number`. Zurück in `term` nimmst du die Schleife, liest `*` und gehst wieder in `factor`. Jetzt nimmst du den mittleren Weg, liest `(` und springst in `expr`. Dort liest du `3 + 4` und kommst über `)` zurück. Weil danach kein Zeichen mehr kommt, verlässt du alle Diagramme über die Umgehung bzw. den Ausgang.

Umrechnen

BNF in EBNF

Die Rekursion in BNF dient fast immer nur dazu, eine Wiederholung auszudrücken. Mit vier Regeln ersetzt du sie durch EBNF-Klammern. x, y und z stehen dabei für beliebige Folgen:

BNF	EBNF	Warum
`<a> ::= <a> x | y`	`a = y { x }`	Linksrekursion: Zuerst kommt einmal y, dann beliebig oft x.
`<a> ::= x <a> | y`	`a = { x } y`	Rechtsrekursion: Beliebig oft x, am Ende einmal y.
`<a> ::= x | ε`	`a = [ x ]`	Eine Alternative ist leer.
`<a> ::= x y | x z`	`a = x ( y | z )`	Gemeinsamer Anfang wird herausgehoben.

Wende das auf `<expr>` an. Die Regel hat zwei rekursive Alternativen und eine ohne Rekursion:

```
<expr> ::= <term> | <expr> "+" <term> | <expr> "-" <term>       (1)
<expr> ::= <term> | <expr> ( "+" <term> | "-" <term> )          (2)
<expr> ::= <term> | <expr> ("+" | "-") <term>                   (3)
expr   = term { ("+" | "-") term }                              (4)
```

- **(1)** Die BNF-Regel aus dem ersten Abschnitt.
- **(2)** Die beiden rekursiven Alternativen beginnen mit `<expr>`. Den gemeinsamen Anfang hebst du heraus. Klammern sind hier schon EBNF, das ist ein Zwischenschritt.
- **(3)** Auch `<term>` ist in beiden Alternativen gleich und wandert nach hinten.
- **(4)** Jetzt hat die Regel die Form `<a> ::= <a> x | y` mit y = `term` und x = `("+" | "-") term`. Daraus wird `y { x }`.

Bei `<number> ::= <digit> | <number> <digit>` funktioniert es genauso: y = `digit`, x = `digit`, also `number = digit { digit }`.

EBNF in BNF

Der umgekehrte Weg geht immer. Für jede Klammer führst du ein neues Nichtterminal ein:

```
a = [ x ]        →   <a> ::= x | ε
a = { x }        →   <a> ::= ε | <a> x
a = y ( p | q )  →   <a> ::= y <g>        <g> ::= p | q
```

Beispiel: `number = digit { digit }` wird zu

```
<number> ::= <digit> <digits>
<digits> ::= ε | <digits> <digit>
```

EBNF ist also nicht mächtiger als BNF. Beide beschreiben genau die kontextfreien Sprachen. EBNF ist nur kürzer und näher an einer Schleife im Programmcode.

EBNF und Syntaxdiagramm

Zwischen EBNF und Syntaxdiagramm übersetzt du Baustein für Baustein, wie in der Vergleichstabelle oben. Beim Lesen eines Diagramms gehst du von links nach rechts und fragst an jeder Weiche:

1. Teilt sich die Linie und läuft weiter rechts wieder zusammen? Dann ist es eine Alternative `( | )`. Ist einer der Wege leer, ist es eine Option `[ ]`.
2. Führt eine Linie nach links zurück? Dann ist es eine Wiederholung `{ }`. Alles, was zwischen dem Abzweig der Rückführung und ihrem Wiedereinstieg liegt, kommt in die Klammer.
3. Alles andere ist eine Folge.

Beispiele aus echten Spezifikationen

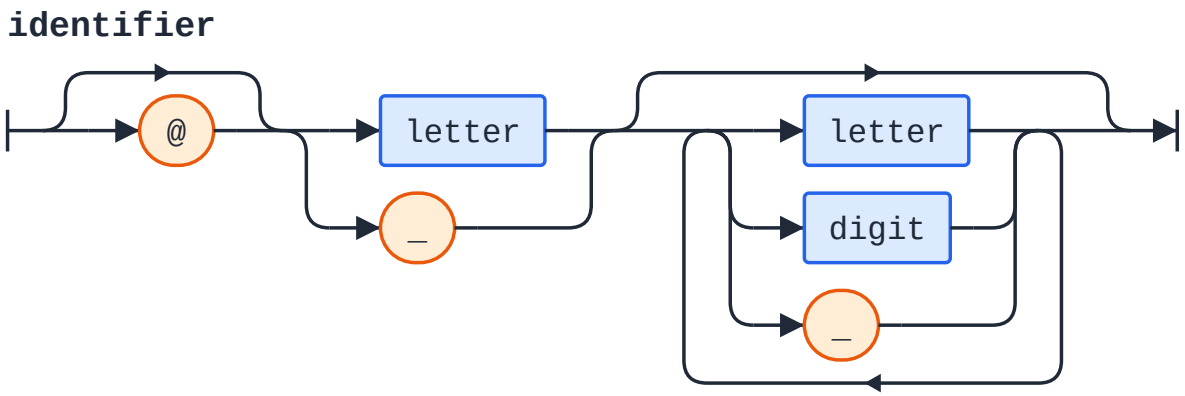
Bezeichner in C#

Die C#-Spezifikation (Abschnitt *Lexical structure*) definiert Bezeichner so, hier gekürzt:

```
Simple_Identifier
    : Available_Identifier
    | Escaped_Identifier
    ;
fragment Available_Identifier
    // excluding keywords or contextual keywords, see note below
    : Basic_Identifier
    ;
fragment Escaped_Identifier
    // Includes keywords and contextual keywords prefixed by '@'.
    : '@' Basic_Identifier
    ;
fragment Basic_Identifier
    : Identifier_Start_Character Identifier_Part_Character*
    ;
fragment Identifier_Start_Character
    : Letter_Character
    | Underscore_Character
    ;
```

`fragment` ist ANTLR-spezifisch und bedeutet „Hilfsregel“. Lässt du die Unicode-Details weg (`Letter_Character` umfasst z. B. auch `ä` und `π`), bleibt in unserer EBNF:

```
identifier = [ "@" ] ( letter | "_" ) { letter | digit | "_" }
letter     = "a" | … | "z" | "A" | … | "Z"
digit      = "0" | … | "9"
```




```
identifier = [ "@" ] ( letter | "_" ) { letter | digit | "_" }
```

Eine Regel steht nur als Kommentar in der Spezifikation: Ohne `@` darf ein Bezeichner kein Schlüsselwort sein. `class` ist laut Diagramm ein Bezeichner, der Compiler lehnt es trotzdem ab. Solche Ausnahmen prüft der Lexer mit einer Liste der Schlüsselwörter. Mit `@` ist `@class` erlaubt.

Integer-Literale in C#

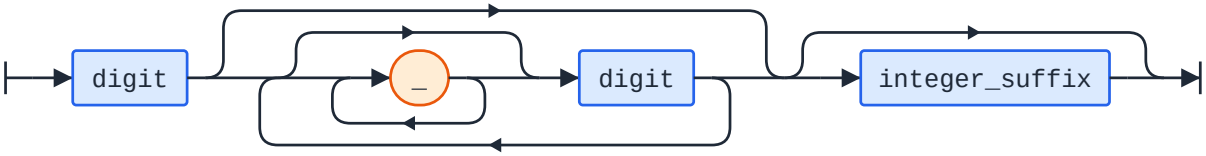
Auch Zahlen im Quelltext sind Wörter einer Sprache. Die Spezifikation definiert dezimale Integer-Literale so:

```
fragment Decimal_Integer_Literal
    : Decimal_Digit Decorated_Decimal_Digit* Integer_Type_Suffix?
    ;
fragment Decorated_Decimal_Digit
    : '_'* Decimal_Digit
    ;
fragment Integer_Type_Suffix
    : 'U' | 'u' | 'L' | 'l' |
      'UL' | 'Ul' | 'uL' | 'ul' | 'LU' | 'Lu' | 'lU' | 'lu'
    ;
```

In unserer EBNF wird aus `X*` eine Wiederholung `{ X }` und aus `X?` eine Option `[ X ]`. Die Hilfsregel `Decorated_Decimal_Digit` setzen wir direkt ein:

```
integer_literal = digit { { "_" } digit } [ integer_suffix ]
integer_suffix  = "U" | "u" | "L" | "l" | "UL" | "Ul" | "uL" | "ul" | "LU" | "Lu" |
"lU" | "lu"
```

integer_literal




```
integer_literal = digit { { "_" } digit } [ integer_suffix ]
```

Die verschachtelte Wiederholung erlaubt beliebig viele `_` zwischen zwei Ziffern. Weil am Ende der äußeren Schleife immer ein `digit` steht, kann ein Literal nie mit `_` enden. Du kannst die Regeln direkt im Compiler ausprobieren:

```
long population = 9_200_000L;   // (1)
int strange = 1__0;             // (2)
int @int = 42;                  // (3)
int _1000 = 1;                  // (4)
uint mask = 0xFF_FFu;           // (5)

Console.WriteLine($"{population} {strange} {@int} {_1000} {mask}");
```

```
9200000 10 42 1 65535
```

- **(1)** Gruppierung mit `_`, Suffix `L` für `long`.
- **(2)** Mehrere `_` hintereinander sind erlaubt, auch wenn das niemand so schreibt.
- **(3)** Mit `@` wird das Schlüsselwort `int` zu einem Bezeichner.
- **(4)** `_1000` beginnt mit `_` und ist laut Diagramm ein *Bezeichner*, keine Zahl.
- **(5)** Hexadezimale Literale haben das Präfix `0x`. Ihre Regel ist ähnlich aufgebaut.

`1_000_` und `1000_L` liefern dagegen den Fehler `CS1013: Invalid number`. Ein Vorzeichen gehört übrigens nicht zum Literal. `-5` ist das Literal `5` mit dem Operator `-` davor, genau wie in der Regel `factor = ⋯ | "-" factor`.

JSON

Auf json.org ist das ganze JSON-Format als Syntaxdiagramme gezeichnet: Objekt, Array, Wert, String, Zahl und Leerraum. Rechts daneben steht dieselbe Grammatik als Text in der McKeeman-Form, einer weiteren BNF-Variante. Sieh dir die Diagramme an. Im Übungsprojekt JsonParser.7z schreibst du daraus selbst eine EBNF. Die Angabe steht im Archiv in der Datei *README.md*.

Endliche Automaten

Ein endlicher Automat ist die einfachste Maschine, die eine Sprache erkennen kann. Er liest ein Wort Zeichen für Zeichen und merkt sich dabei nur, in welchem *Zustand* er gerade ist. Am Ende sagt er „ja“ oder „nein“. Genau so arbeiten Lexer in Compilern, Eingabeprüfungen in Formularen und manche Regex-Engines.



Zum Selbst-Erarbeiten: der interaktive Automaten-Trainer

Im Automaten-Trainer siehst du, wie ein DEA ein Wort Zeichen für Zeichen liest. Danach baust du selbst Automaten per Mausklick, und der Trainer prüft sie sofort. So findest du die Fehler in deinem Automaten, bevor sie dich im Test erwischen.

Ein Automat aus dem Alltag

Ein Getränkeautomat verkauft eine Flasche um 1,50 €. Er nimmt nur 50-Cent-Münzen (h) und 1-Euro-Münzen (e). Der Automat muss sich nicht jede Münze merken. Es reicht, wenn er weiß, wie viel schon eingeworfen wurde. Mehr als 1,50 € muss er sich nie merken, ab da gibt er das Getränk aus. Er hat also nur vier Zustände: *0 ct*, *50 ct*, *1 €* und *bezahlt*.

Zustand	h	e
→ 0 ct	50 ct	1 €
50 ct	1 €	bezahlt
1 €	bezahlt	bezahlt
* bezahlt	–	–

Jede Zeile sagt, in welchen Zustand der Automat nach einer Münze wechselt. Der Pfeil → markiert den Startzustand, der Stern * den Zustand, in dem das Getränk kommt. Die Münzfolge heh endet in *bezahlt*, die Folge hh nicht. Die Striche bedeuten: Nach dem Bezahlen nimmt der Automat keine Münzen mehr an. Was mit solchen Lücken passiert, klärt der Abschnitt über den Fehlerzustand.

Die Folge der Münzen ist ein Wort über dem Alphabet Σ = {h, e}. Der Automat beschreibt damit eine Sprache: alle Münzfolgen, bei denen erst die letzte Münze den Preis von 1,50 € erreicht.

Der deterministische endliche Automat (DEA)

Formal ist ein **deterministischer endlicher Automat** (DEA, englisch *DFA*) ein 5-Tupel M = (Q, Σ, δ, q_0, F):

Teil	**Bedeutung**
Q	endliche Menge von Zuständen
Σ	Eingabealphabet

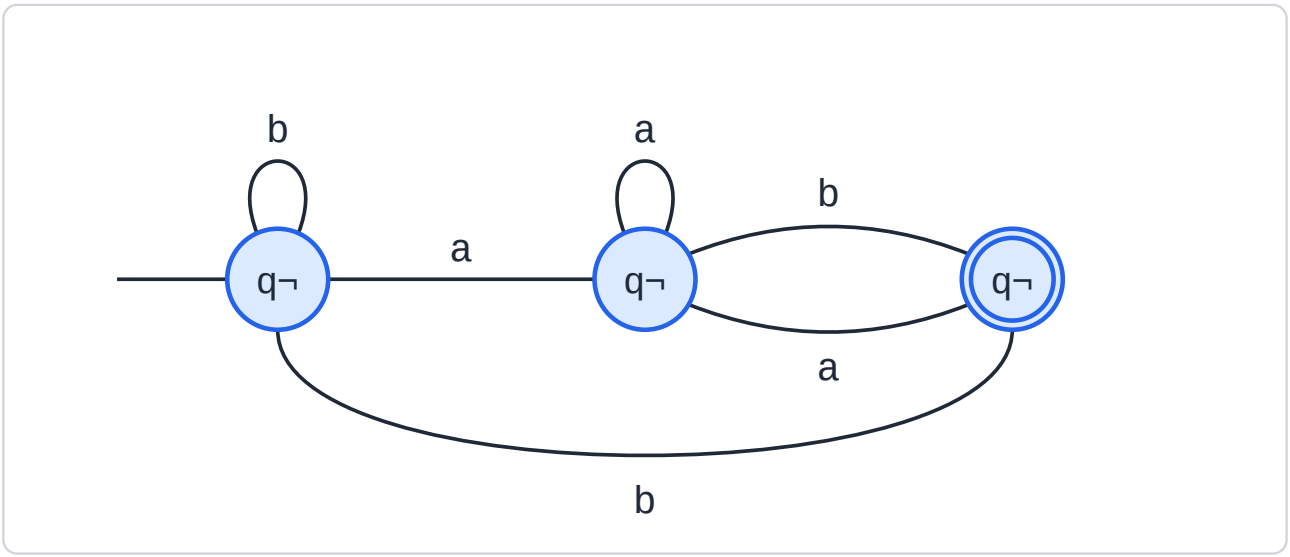
Teil	Bedeutung
$\delta: Q \times \Sigma \to Q$	Übergangsfunktion: aus Zustand und Zeichen folgt der nächste Zustand
$q_0 \in Q$	Startzustand
$F \subseteq Q$	Menge der Endzustände (akzeptierende Zustände)

Deterministisch heißt: Für jeden Zustand und jedes Zeichen gibt es höchstens einen Folgezustand. Der Automat muss nie raten.

Beispiel: Wörter, die auf `ab` enden

Die Sprache $L = \{ w \in \{a, b\}^* \mid w \text{ endet auf } ab \}$ enthält `ab`, `aab`, `bab` und `abab`, aber nicht ε, `a`, `ba` oder `abb`. Der Automat muss sich nur merken, wie viel vom Ende `ab` er gerade gesehen hat:

- q_0: Das Wortende passt noch gar nicht (Start, oder zuletzt `b` ohne `a` davor).
- q_1: Das letzte Zeichen war ein `a`.
- q_2: Die letzten zwei Zeichen waren `ab`.



Im **Zustandsdiagramm** ist jeder Zustand ein Kreis. Der Pfeil von links zeigt auf den Startzustand. Endzustände haben einen Doppelkreis. Ein Pfeil von p nach q mit der Beschriftung `a` bedeutet $\delta(p, a) = q$. Ein Pfeil, der zum selben Zustand zurückführt, heißt *Schleife*.

Interessant sind die Pfeile, die aus q_2 herausführen. Liest der Automat in q_2 ein `a`, ist zwar das `ab` kaputt, aber das neue `a` kann der Anfang eines neuen `ab` sein. Deshalb geht es nach q_1 und nicht nach q_0.

Die Übergangstabelle

Dieselbe Information steht kompakter in einer **Übergangstabelle**. Jede Zeile ist ein Zustand, jede Spalte ein Zeichen. In der Zelle steht der Folgezustand.

δ	a	b
→ q_0	q_1	q_0
q_1	q_1	q_2
* q_2	q_1	q_0

Das Diagramm ist für Menschen leichter zu lesen. Die Tabelle eignet sich besser für Programme, denn ein Programm kann sie direkt als zweidimensionales Array oder `Dictionary` speichern.

Ein Wort prüfen

Der Automat startet in q_0 und wendet für jedes Zeichen δ an. Ist er nach dem letzten Zeichen in einem Endzustand, **akzeptiert** er das Wort. Sonst **lehnt er es ab**.

```
Wort abab:   q₀ --a--> q₁ --b--> q₂ --a--> q₁ --b--> q₂   q₂ ∈ F  -> akzeptiert
Wort aba:    q₀ --a--> q₁ --b--> q₂ --a--> q₁             q₁ ∉ F  -> abgelehnt
Wort ε:      q₀                                           q₀ ∉ F  -> abgelehnt
```

Man schreibt δ*(q, w) für den Zustand, in dem der Automat nach dem ganzen Wort w landet. Die Sprache des Automaten ist dann L(M) = { w ∈ Σ* | δ*(q_0, w) ∈ F }.

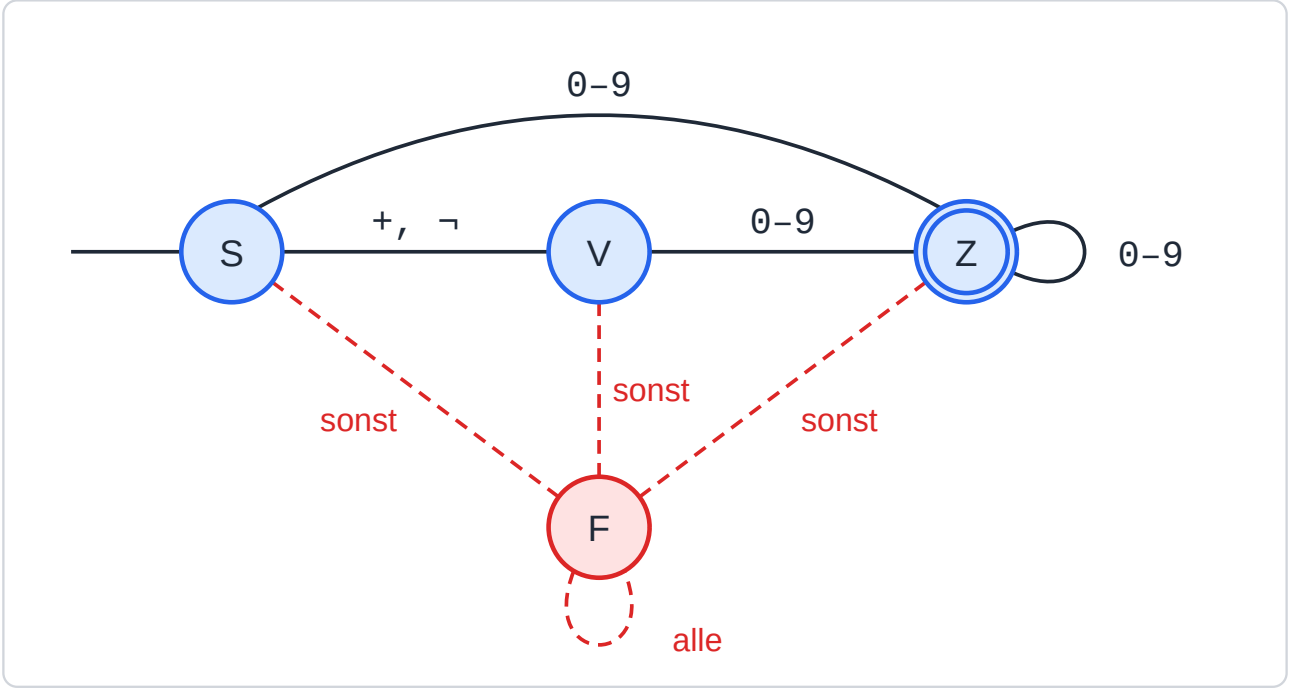
Beachte zwei Dinge:

- Der Automat entscheidet erst **am Ende** des Wortes. Dass er unterwegs in q_2 war, zählt bei `aba` nicht.
- Der Automat liest jedes Zeichen genau **einmal** von links nach rechts. Er kann nicht zurückspringen und hat außer dem Zustand keinen Speicher.

Der Fehlerzustand

Beim Getränkeautomaten fehlten Übergänge. In der Praxis lässt man solche Lücken oft offen, weil das Diagramm sonst unübersichtlich wird. Gemeint ist immer dasselbe: Es gibt einen **Fehlerzustand** (auch *Senke* oder *Trap State*), der kein Endzustand ist und den man nie wieder verlässt. Jeder fehlende Übergang führt dorthin. Ein fehlender Übergang bedeutet also: Das Wort wird abgelehnt, egal was noch kommt.

Als Beispiel prüfen wir Ganzzahl-Literale mit optionalem Vorzeichen: `42`, `-7` und `0815+` gehören dazu, aber nicht ε, ``, `4-2` oder `12a`. Das Alphabet sind alle Zeichen, die in der Eingabe vorkommen können. Damit die Grafik lesbar bleibt, steht `+0–9` für zehn einzelne Pfeile und `sonst` für alle Zeichen, für die kein anderer Pfeil passt.



- S (Start): Noch nichts gelesen. Ein Vorzeichen führt nach V, eine Ziffer direkt nach Z.
- V (Vorzeichen): Es muss jetzt eine Ziffer kommen. Ein zweites Vorzeichen ist ein Fehler.
- Z (Ziffern): Mindestens eine Ziffer gelesen. Das ist der einzige Endzustand.
- F (Fehler): Hier bleibt der Automat bis zum Ende des Wortes.

Ab jetzt lassen wir den Fehlerzustand in Diagrammen meist weg. Du musst ihn dir aber immer dazudenken.

Ein DEA in C#

Ein konkreter DEA lässt sich in C# sehr direkt umsetzen. Die Zustände werden ein `enum`, die Übergangsfunktion δ ein `switch`-Ausdruck über das Tupel `(state, c)`.

```
string[] words = ["42", "-7", "+0815", "", "+", "4-2", "12a"];
foreach (string word in words)
{
    string quoted = $"\"{word}\"";
    Console.WriteLine($"{quoted,-8} {(IsInteger(word) ? "akzeptiert" : "abgelehnt")}");
}

static bool IsInteger(string word)
{
    State state = State.Start;                          // (1)
    foreach (char c in word)
    {
        state = (state, c) switch                       // (2)
        {
            (State.Start, '+' or '-') => State.Sign,
            (State.Start or State.Sign or State.Digits, >= '0' and <= '9') => State
.Digits,
            _ => State.Error                            // (3)
```

```
        };
        if (state == State.Error) { return false; }     // (4)
    }
    return state == State.Digits;                       // (5)
}

enum State { Start, Sign, Digits, Error }
```

```
"42"     akzeptiert
"-7"     akzeptiert
"+0815"  akzeptiert
""       abgelehnt
"+"      abgelehnt
"4-2"    abgelehnt
"12a"    abgelehnt
```

- **(1)** q_0 ist `State.Start`.
- **(2)** Jeder Zweig des `switch` ist ein Pfeil im Diagramm. Die Patterns `'+' or '-'` und `>= '0' and <= '9'` fassen mehrere Zeichen zu einem Pfeil zusammen, wie `0–9` in der Grafik. Mit `State.Start or State.Sign or State.Digits` gilt ein Zweig für drei Zustände.
- **(3)** Der Discard `_` ist der Pfeil `sonst`. Alle fehlenden Übergänge führen in den Fehlerzustand. Das gilt auch für F selbst, deshalb braucht F keinen eigenen Zweig.
- **(4)** Aus dem Fehlerzustand kommt der Automat nie wieder heraus. Wir können also sofort abbrechen, statt den Rest des Wortes zu lesen.
- **(5)** F = {Z}. Das Wort wird akzeptiert, wenn der Automat am Ende in `State.Digits` steht.

Der Compiler hilft dir hier: Fehlt der Discard `_`, warnt er, dass der `switch` nicht alle Fälle abdeckt. Mit `TreatWarningsAsErrors` wird daraus ein Fehler. Du kannst also keinen Übergang vergessen, ohne es zu merken.

Diese Umsetzung ist schnell und gut lesbar, aber fest verdrahtet. Willst du einen anderen Automaten, musst du neuen Code schreiben. Wie du einen Automaten stattdessen aus einer Übergangstabelle als Daten baust, programmierst du im Übungsprojekt.

Nichtdeterministische Automaten (NEA)

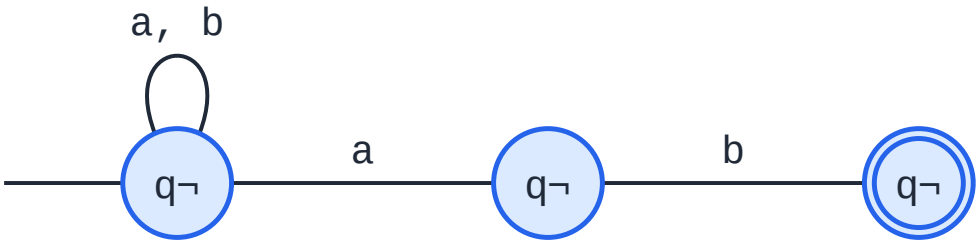
Beim DEA war es knifflig, die Pfeile aus q_2 richtig zu setzen. Einfacher wäre ein Automat, der *rät*: „Ich lese beliebig viel, und wenn das vorletzte Zeichen kommt, springe ich los.“ Genau das erlaubt ein **nichtdeterministischer endlicher Automat** (NEA, englisch *NFA*):

- Aus einem Zustand dürfen für dasselbe Zeichen **mehrere** Pfeile oder **gar keiner** führen.
- Es darf **ε-Übergänge** geben. Über einen solchen Pfeil wechselt der Automat den Zustand, ohne ein Zeichen zu lesen.

Ein NEA **akzeptiert** ein Wort, wenn es **mindestens einen** Weg durch den Automaten gibt, der das ganze Wort liest und in einem Endzustand endet. Die Übergangsfunktion liefert daher eine Menge von Zuständen: $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. $\mathcal{P}(Q)$ ist die *Potenzmenge* von Q, also die Menge

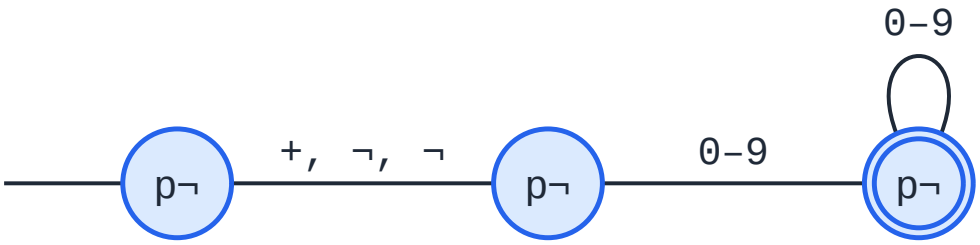
aller Teilmengen.

NEA: Wörter, die auf ab enden



Mit a kann q¬ in q¬ bleiben oder nach q¬ gehen. q¬ hat keinen Pfeil für a.

¬-NEA: Ganzzahl mit optionalem Vorzeichen



Der obere NEA erkennt wieder die Wörter, die auf `ab` enden. In q_0 liest er beliebige Zeichen. Bei einem `a` kann er in q_0 bleiben *oder* nach q_1 gehen. Für `bab` gibt es den Weg $q_0 \to q_0 \to q_1 \to q_2$, also wird `bab` akzeptiert. Andere Wege, etwa $q_0 \to q_0 \to q_0 \to q_0$, enden nicht in einem Endzustand. Das stört nicht, denn ein erfolgreicher Weg genügt.

Der untere NEA erkennt Ganzzahlen mit optionalem Vorzeichen. Der ε-Übergang drückt „optional“ aus: Der Automat kann das Vorzeichen lesen oder ohne Lesen nach p_1 gehen.

Ein NEA ist leichter zu entwerfen, aber schwerer auszuführen. Ein Programm müsste alle Wege gleichzeitig verfolgen. Zum Glück lässt sich jeder NEA in einen DEA umbauen.

Vom NEA zum DEA: die Potenzmengenkonstruktion

Die Idee: Der DEA merkt sich die **Menge aller Zustände**, in denen der NEA gerade sein könnte. Jede solche Menge wird ein Zustand des DEA. So geht die Konstruktion:

1. Der Startzustand des DEA ist die Menge \{q_0}.
2. Für jede neue Menge M und jedes Zeichen x bildest du die Menge aller Zustände, die der NEA von irgendeinem Zustand in M mit x erreicht.
3. Taucht dabei eine neue Menge auf, bekommt sie eine eigene Zeile. Du machst weiter, bis keine neuen Mengen mehr entstehen.
4. Endzustände des DEA sind alle Mengen, die mindestens einen Endzustand des NEA enthalten.

Für den oberen NEA aus der Grafik sieht das so aus:

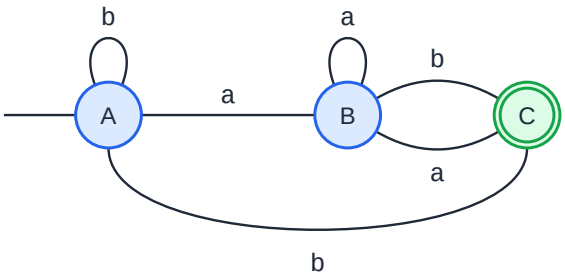
- \{q_0} mit a: Von q_0 geht es nach q_0 und q_1, also {q_0, q_1}. Mit b nur nach \{q_0}.
- {q_0, q_1} mit a: Von q_0 nach q_0 und q_1, von q_1 mit a nirgends hin. Ergebnis {q_0, q_1}.
- {q_0, q_1} mit b: Von q_0 nach q_0, von q_1 nach q_2. Ergebnis {q_0, q_2}. Diese Menge ist neu.
- {q_0, q_2} mit a: {q_0, q_1}. Mit b: \{q_0}. Keine neue Menge mehr, fertig.

Teilmengen-Tabelle

Name	Menge	a	b
¬ A	{q¬}	{q¬, q¬}	{q¬}
B	{q¬, q¬}	{q¬, q¬}	{q¬, q¬}
* C	{q¬, q¬}	{q¬, q¬}	{q¬}

Endzustand ist jede Menge, die den NEA-Endzustand q¬ enthält.
A = {q¬}, B = {q¬,q¬}, C = {q¬,q¬}

Ergebnis-DEA



Nur {q_0, q_2} enthält den Endzustand q_2. Benennst du die Mengen in A, B und C um, bekommst du genau den DEA vom Anfang dieser Einheit.

Ein NEA mit n Zuständen hat 2^n Teilmengen. Im schlimmsten Fall wird der DEA also sehr groß. In der Praxis entstehen aber meist nur wenige Mengen, weil du nur die erreichbaren Mengen aufschreibst.

ε-Übergänge

Gibt es ε-Übergänge, kommt ein Schritt dazu. Die **ε-Hülle** einer Menge M sind alle Zustände, die du von M aus nur über ε-Pfeile erreichst (M selbst gehört dazu). Du bildest die ε-Hülle für die Startmenge und nach jedem gelesenen Zeichen.

Für den unteren NEA aus der Grafik ist die Startmenge die ε-Hülle von \{p_0}, also {p_0, p_1}:

DEA-Zustand	+, -	0–9
→ {p_0, p_1}	\{p_1}	\{p_2}
\{p_1}	–	\{p_2}
* \{p_2}	–	\{p_2}

Die Striche stehen für die leere Menge Ø. Sie ist der Fehlerzustand. Das Ergebnis ist der Ganzzahl-DEA von vorhin: {p_0, p_1} ist S, \{p_1} ist V und \{p_2} ist Z.

Was endliche Automaten können und was nicht

Es gilt ein wichtiger Satz, den wir hier nicht beweisen: **DEA, NEA und reguläre Ausdrücke beschreiben genau dieselben Sprachen.** Man nennt sie die *regulären Sprachen*. Das sind die Typ-3-Sprachen der Chomsky-Hierarchie.

- Zu jedem NEA gibt es einen DEA (die Potenzmengenkonstruktion zeigt, wie).
- Zu jedem regulären Ausdruck gibt es einen NEA. Für jede der drei Operationen Vereinigung, Konkatenation und Stern gibt es einen kleinen NEA-Baustein mit ε-Übergängen. Die Bausteine werden nach dem Aufbau des Ausdrucks zusammengesteckt (*Thompson-Konstruktion*).
- Zu jedem DEA gibt es einen regulären Ausdruck.

Ein endlicher Automat hat nur endlich viele Zustände und sonst keinen Speicher. Deshalb kann er nicht beliebig weit zählen. Die Sprache $a^n b^n$ = {ε, `ab`, `aabb`, …} ist nicht regulär: Der Automat müsste sich die Anzahl der `a` merken, und dafür reichen endlich viele Zustände nicht. Dasselbe gilt für korrekt geklammerte Ausdrücke. Für solche Sprachen brauchst du eine kontextfreie Grammatik und einen Parser.

Zusatzmaterial

Im 13_Endliche_Automaten.html siehst du einen DEA Schritt für Schritt arbeiten und baust danach selbst Automaten per Mausklick, die automatisch geprüft werden.

Reguläre Ausdrücke

Ist `1010` eine gültige Postleitzahl? Wie stellst du in 300 Zeilen jedes Datum von `24.12.2026` auf `2026-12-24` um? Welche Matrikelnummern stehen in einer Teilnehmerliste? Für solche Aufgaben schreibst du keine Schleife über die Zeichen, sondern einen regulären Ausdruck (*regular expression*, kurz *Regex*). Er beschreibt ein Muster in einer kompakten Schreibweise. Du findest ihn in jeder Programmiersprache, in jedem Editor und in vielen Tools.



Zum Selbst-Erarbeiten: der interaktive Regex-Trainer

Im Regex-Trainer siehst du als Animation, wie eine Regex über einen Text läuft und wie Suchen und Ersetzen arbeitet. Danach schreibst du selbst Ausdrücke und siehst live, welche Wörter sie erkennen. Regex lernst du durch Ausprobieren, nicht durch Lesen.

Wofür Regex gut ist und wofür nicht

Regex ist das richtige Werkzeug für vier Aufgaben:

- **Prüfen**, ob eine Eingabe ein Format hat: Postleitzahl, Matrikelnummer, Uhrzeit.
- **Suchen und Ersetzen** im Editor: Datumsformat umstellen, Spalten tauschen, Code umbauen.
- **Daten herausholen**: alle Beträge, Nummern oder Datumsangaben aus einem Text.
- **Zerlegen**: eine Zeile an einem Trennzeichen aufteilen.

Regex prüft aber nur die **Form**. `31.02.2026` sieht aus wie ein Datum, ist aber keines. Ob es eine Postleitzahl wirklich gibt, weiß die Regex auch nicht. Und für **verschachtelte Strukturen** wie HTML, JSON oder Klammerausdrücke ist sie das falsche Werkzeug. Dafür gibt es Parser, z. B. `System.Text.Json`.

Zum Ausprobieren eignet sich regex101.com. Stelle dort links den *Flavor* auf *.NET (C#)*. Die Seite erklärt jeden Teil deines Ausdrucks in Worten.

Die Bausteine

Zeichen und Zeichenklassen

Muster	passt auf	Beispiel
`a`, `7`, `-`	genau dieses Zeichen	`Wien` passt auf `Wien`
`.`	ein beliebiges Zeichen außer dem Zeilenumbruch	`M..er` passt auf `Maier`, `Meier`, `Mayer`
`[aeiou]`	eines der Zeichen in der Klammer	`M[ae][iy]er` passt auf `Maier`, `Meyer`
`[a-z]`, `[0-9A-F]`	ein Zeichen aus dem Bereich	`[0-9]` passt auf eine Ziffer

Muster	passt auf	Beispiel
`[^0-9]`	ein Zeichen, das *nicht* in der Klasse steht	passt auf `a`, `-` oder ein Leerzeichen
`\d`, `\w`, `\s`	Ziffer, Wortzeichen (Buchstabe, Ziffer, _), Leerraum	`\s` passt auf Leerzeichen, Tab, Umbruch
`\D`, `\W`, `\S`	das Gegenteil davon	`\S` passt auf alles außer Leerraum

> **Achtung:** In .NET sind `\d` und `\w` Unicode-fähig. `\d` passt auch auf die arabisch-indische Ziffer `٣`, `\w` passt auf `ä`. Willst du nur die Ziffern 0 bis 9 erlauben, schreibst du `[0-9]`.

Quantoren

Ein Quantor steht hinter einem Zeichen, einer Klasse oder einer Gruppe und gibt an, wie oft sie vorkommen.

Quantor	Anzahl	Beispiel
`*`	0 bis beliebig oft	`Hal*o` passt auf `Hao`, `Halo`, `Hallo`
`+`	1 bis beliebig oft	`[0-9]+` passt auf `7`, `2026`
`?`	0 oder 1 Mal	`-?[0-9]+` passt auf `42`, `-42`
`{4}`	genau 4 Mal	`[0-9]{4}` passt auf `1010`
`{2,}`	mindestens 2 Mal	`[0-9]{2,}` passt auf `10`, `2026`
`{2,4}`	2 bis 4 Mal	`[0-9]{2,4}` passt auf `12`, `1234`

Quantoren sind **gierig** (*greedy*). Sie nehmen so viele Zeichen wie möglich. Ein angehängtes `?` macht sie **genügsam** (*lazy*). Sie nehmen dann so wenig wie möglich.

```
using System.Text.RegularExpressions;

string text = "Treffpunkt [Wien] oder [Graz]";
Console.WriteLine(Regex.Match(text, @"\[.*\]").Value);      // (1)
Console.WriteLine(Regex.Match(text, @"\[.*?\]").Value);     // (2)
Console.WriteLine(Regex.Match(text, @"\[[^\]]*\]").Value);  // (3)
```

```
[Wien] oder [Graz]
[Wien]
[Wien]
```

- **(1)** `.*` nimmt so viel wie möglich und endet erst beim *letzten* `]` im Text.
- **(2)** `.*?` nimmt so wenig wie möglich und endet beim *ersten* `]`.
- **(3)** Oft ist eine negierte Klasse die klarste Lösung: „alles außer `]`“.

Anker

Anker passen nicht auf ein Zeichen, sondern auf eine Position.

Anker	Position
`^`	Anfang des Strings (mit `RegexOptions.Multiline`: Anfang jeder Zeile)
`$`	Ende des Strings **oder vor einem abschließenden** `\n` (mit Multiline: Ende jeder Zeile)
`\z`	nur das echte Ende des Strings (gibt es in .NET, nicht in JavaScript)
`\b`	Wortgrenze: zwischen einem Wortzeichen und einem anderen Zeichen

Ohne Anker sucht die Regex *irgendwo* im String. Für eine Prüfung „der ganze String hat dieses Format“ brauchst du deshalb Anker. Eine österreichische Postleitzahl hat vier Ziffern, die erste ist nicht 0:

```
using System.Text.RegularExpressions;

string[] inputs = ["1010", "0123", "A-1010", "10100", "1010\n"];
foreach (string input in inputs)
{
    bool noAnchors = Regex.IsMatch(input, @"[1-9][0-9]{3}");
    bool withDollar = Regex.IsMatch(input, @"^[1-9][0-9]{3}$");
    bool withZ = Regex.IsMatch(input, @"^[1-9][0-9]{3}\z");
    Console.WriteLine($"{input.ReplaceLineEndings("\\n"),-8} {noAnchors,-6} {
withDollar,-6} {withZ}");
}
```

```
1010     True   True   True
0123     False  False  False
A-1010   True   False  False
10100    True   False  False
1010\n   True   True   False
```

Ohne Anker findet die Regex in `A-1010` und `10100` vier passende Ziffern. Die letzte Zeile ist eine Falle: `$` erlaubt am Ende noch ein `\n`. Für strenge Prüfungen nimmst du in .NET `\z`.

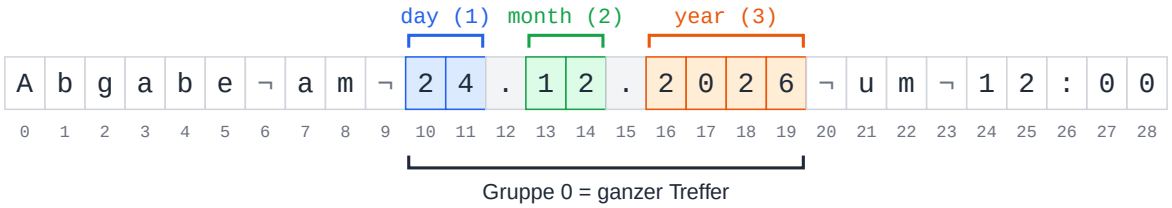
Gruppen und Alternativen

Muster	Bedeutung
`(…)`	Gruppe, die ihren Text speichert, nummeriert ab 1
`(?<name>…)`	benannte Gruppe

Muster	Bedeutung
`(?:⋯)`	Gruppe nur zum Zusammenfassen, speichert nichts
`a|b`	Alternative: links oder rechts

Gruppen fassen Teile zusammen, damit ein Quantor oder eine Alternative für den ganzen Teil gilt: `(ab)+` passt auf `ab`, `abab`, … Vor allem merkt sich die Regex aber, welcher Text in welcher Gruppe gelandet ist. So holst du Tag, Monat und Jahr einzeln aus einem Datum:

Match für `(?<day>[0-9]{2})\.(?<month>[0-9]{2})\.(?<year>[0-9]{4})`




```
m.Value                    "24.12.2026"
m.Index                    10
m.Length                   10
m.Groups["day"].Value      "24" (auch m.Groups[1])
m.Groups["month"].Value    "12" (auch m.Groups[2])
m.Groups["year"].Value     "2026" (auch m.Groups[3])
```

Gruppe 0 ist immer der ganze Treffer. Die benannten Gruppen bekommen in .NET zusätzlich Nummern, hier 1 bis 3.

Im Muster selbst verlangt `\1` denselben Text noch einmal, den Gruppe 1 gefunden hat: `\b(\w+) \1\b` findet verdoppelte Wörter wie `das das`.

> **Falle:** `|` hat die niedrigste Priorität. `^Herr|Frau$` bedeutet „beginnt mit `Herr`“ *oder* „endet mit `Frau`“. Gemeint ist `^(?:Herr|Frau)$`.

Escapen

Die Zeichen `. * + ? ( ) [ ] { } ^ $ | \` haben eine Bedeutung. Meinst du das Zeichen selbst, schreibst du einen Backslash davor: `\.` ist ein Punkt, `\(` eine Klammer, `\\` ein Backslash. In einer Zeichenklasse ist fast alles normal, `[.;]` braucht keinen Backslash. Dort sind nur `]`, `\`, `^` am Anfang und `-` zwischen zwei Zeichen besonders.

In C# schreibst du Muster als Verbatim-String `@"⋯"`. In einem normalen String müsstest du jeden Backslash verdoppeln (`"\\d"` statt `@"\d"`). Baust du Text von Benutzern in ein Muster ein, escapt `Regex.Escape` alle Sonderzeichen für dich. Aus `C++ (.NET)` wird `C\+\+\ \(\.NET\)`.

Lookahead

Ein Lookahead prüft, was *nach* der aktuellen Position kommt, ohne die Zeichen zu verbrauchen. `(?=⋯)` verlangt, dass das Muster folgt, `(?!⋯)` verlangt, dass es *nicht* folgt. In `3 Bücher um 45 EUR` findet `[0-9]+(?= EUR)` nur die `45`. Das ` EUR` gehört nicht zum Treffer. Mehrere Lookaheads am Anfang prüfen mehrere Bedingungen, z. B. bei Kennwortregeln (siehe unten).

Fremde Ausdrücke lesen

Einen fremden Ausdruck liest du von links nach rechts und zerlegst ihn in seine Bausteine. Als Beispiel eine (vereinfachte) österreichische Telefonnummer:

```
^(?:\+43|0)[1-9][0-9]{3,12}\z
```

Teil	Bedeutung
`^` … `\z`	der ganze String, nicht nur ein Teil
`(?:\+43|0)`	entweder `+43` (das Plus escapt) oder `0`
`[1-9]`	eine Ziffer außer 0 (die Vorwahl beginnt nie mit 0)
`[0-9]{3,12}`	danach 3 bis 12 Ziffern

Passt also auf `+436641234567` und `06641234567`, aber nicht auf `0043664⋯` oder `0664 123`.

Validierungsmuster zum Anpassen

Die folgenden Muster prüfen die **Form** einer Eingabe. Kopiere sie und passe sie an deine Regeln an. Sie sind für .NET geschrieben, in JavaScript ersetzt du `\z` durch `$`.

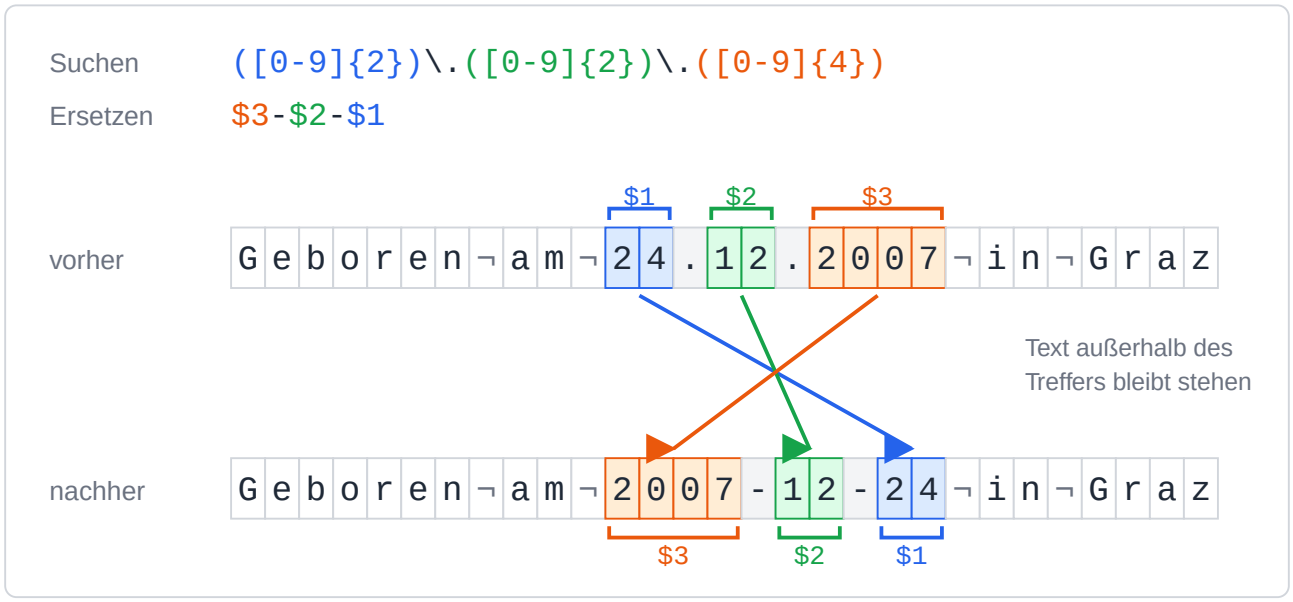
Format	Muster	passt	passt nicht
Postleitzahl (AT)	`^[1-9][0-9]{3}\z`	`1010`, `8010`	`0123`, `10100`
Matrikelnummer	`^[0-9]{8}\z`	`01234567`	`1234567`
Ganze Zahl	`^[+-]?[0-9]+\z`	`42`, `-7`, `+3`	`4.2`, `--7`
Betrag mit Komma	`^[0-9]+(?:,[0-9]{1,2})?\z`	`12`, `12,5`, `0,99`	`12,`, `12,345`
Datum (nur die Form)	`^[0-9]{2}\.[0-9]{2}\.[0-9]{4}\z`	`24.12.2026`	`1.1.2026`, `24-12-2026`
Uhrzeit	`^(?:[01][0-9]|2[0-3]):[0-5][0-9]\z`	`08:15`, `23:59`	`24:00`, `8:15`
Benutzername	`^[a-z][a-z0-9_]{2,15}\z`	`anna_h`, `max2`	`2max`, `Anna`
Kennwort	`^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z]).{8,}\z`	`Sommer2026`	`sommer2026`

Das Kennwort-Muster besteht aus drei Lookaheads (eine Ziffer, ein Klein-, ein Großbuchstabe irgendwo) und `.{8,}` für die Länge. Ob es ein Datum wirklich gibt, prüfst du nicht mit der Regex,

sondern mit `DateOnly.TryParseExact` (siehe unten). Eine Regex, die Schaltjahre kennt, wäre unlesbar.

Suchen und Ersetzen im Editor

Im Alltag brauchst du Regex am häufigsten im Editor. Du willst 200 Zeilen gleich umbauen, und jede Zeile sieht ein bisschen anders aus. Die Idee: Das Suchmuster speichert die Teile, die du behalten willst, in Gruppen. Im Ersetzungstext setzt du sie mit `$1`, `$2`, … neu zusammen.



So schaltest du Regex ein:

	VS Code	Visual Studio
Ersetzen in der Datei	`Strg+H`	`Strg+H`
Ersetzen in allen Dateien	`Strg+Shift+H`	`Strg+Shift+H`
Regex ein/aus	Knopf `.*` oder `Alt+R`	Knopf `.*` oder `Alt+E`
Engine	JavaScript	.NET (wie in C#)
Gruppe im Ersetzungstext	`$1` … `$99`	`$1` … `$99`
ganzer Treffer	`$0` oder `$&`	`$0` oder `$&`
benannte Gruppe	nur über die Nummer (`$1`)	`${name}`
Groß-/Kleinschreibung ändern	`\u`, `\l`, `\U`, `\L` vor der Gruppe	–

In beiden Editoren passen `^` und `$` am Anfang und Ende jeder **Zeile** (in Visual Studio mit einer Falle bei Windows-Zeilenenden, siehe unten). Mit `Strg+Z` machst du eine Ersetzung rückgängig. Klicke vor *Alle ersetzen* ein paar Mal auf *Weitersuchen* und kontrolliere die Treffer.

Datum umstellen

```
Suchen:    \b([0-9]{2})\.([0-9]{2})\.([0-9]{4})\b
Ersetzen:  $3-$2-$1
```

```
vorher:    Abgabe 24.12.2026, Nachtrag 07.01.2027
nachher:   Abgabe 2026-12-24, Nachtrag 2027-01-07
```

In Visual Studio kannst du die Gruppen auch benennen: Suchen `(?<day>[0-9]{2})\.(?<month>[0-9]{2})\.(?<year>[0-9]{4})`, Ersetzen `${year}-${month}-${day}`. Der VS-Code-Editor kennt benannte Gruppen im Ersetzungstext nicht. Dort schreibst du `$3-$2-$1`.

Spalten einer CSV-Datei tauschen

```
Suchen:    ^([^;]*);([^;]*)
Ersetzen:  $2;$1

vorher:    Huber;Anna;5AHIF          nachher:   Anna;Huber;5AHIF
           Gruber;Max;5BHIF                     Max;Gruber;5BHIF
```

`[^;]*` nimmt alles bis zum nächsten Strichpunkt. Die dritte Spalte gehört nicht zum Treffer und bleibt deshalb unverändert stehen.

Leerzeichen am Zeilenende entfernen

```
Suchen:    [ \t]+$
Ersetzen:  (leer lassen)
```

Das funktioniert in VS Code. Visual Studio sieht bei Windows-Zeilenenden noch das `\r` vor dem Zeilenumbruch. Dort schreibst du `[ \t]+(?=\r?$)`: Leerzeichen und Tabs, nach denen höchstens ein `\r` und das Zeilenende kommen. Diese Variante funktioniert in beiden Editoren.

Code umbauen

Aus einer Klasse mit Properties soll ein Record werden. Statt jede Zeile von Hand umzuschreiben, ersetzt du alle auf einmal:

```
Suchen:    public (\S+) (\w+) \{ get; set; \}
Ersetzen:  $1 $2,

vorher:    public string Firstname { get; set; }     nachher:   string Firstname,
           public string Lastname { get; set; }                 string Lastname,
           public DateOnly? Birthdate { get; set; }             DateOnly? Birthdate,
```

Die Klammern `{` und `}` sind Sonderzeichen und brauchen einen Backslash. `\S+` statt `\w+` nimmt auch Typen wie `DateOnly?` oder `List<int>`. In VS Code kannst du mit `\l` den ersten Buchstaben einer Gruppe kleinschreiben. Ersetzen durch `$2 = \l$2;` macht aus derselben Zeile `Firstname = firstname;`, also die Zuweisungen für einen Konstruktor.

Die Klasse Regex in .NET

Alle Typen liegen im Namespace `System.Text.RegularExpressions`. Er gehört nicht zu den impliziten Usings, du musst ihn selbst einbinden.

Methode	Ergebnis	Wofür
`IsMatch`	`bool`	Hat der String das Format?
`Match`	`Match`	Den ersten Treffer samt Gruppen holen
`Matches`	`MatchCollection`	Alle Treffer durchlaufen
`Replace`	`string`	Treffer ersetzen
`Split`	`string[]`	Den String an den Treffern zerlegen

Alle Methoden gibt es statisch (`Regex.IsMatch(input, pattern)`) und als Instanzmethode (`regex.IsMatch(input)`).

Match und Groups

```
using System.Text.RegularExpressions;

string text = "Abgabe am 24.12.2026 um 12:00";
Match m = Regex.Match(text, @"(?<day>[0-9]{2})\.(?<month>[0-9]{2})\.(?<year>[0-
9]{4})");
if (m.Success)                                                    // (1)
{
    Console.WriteLine($"{m.Value} at index {m.Index}");
    Console.WriteLine($"Day {m.Groups["day"].Value}, month {m.Groups["month"].Value}");
    int year = int.Parse(m.Groups["year"].ValueSpan);             // (2)
    Console.WriteLine($"Year {year}");
}

string[] candidates = ["24.12.2026", "31.02.2026"];
foreach (string candidate in candidates)
{
    bool isDate = DateOnly.TryParseExact(candidate, "dd.MM.yyyy", out DateOnly date);
// (3)
    Console.WriteLine($"{candidate}: {(isDate ? date.DayOfWeek.ToString() : "no date"
)}");
}
```

```
24.12.2026 at index 10
Day 24, month 12
Year 2026
24.12.2026: Thursday
31.02.2026: no date
```

- **(1)** Gibt es keinen Treffer, liefert `Match` ein Objekt mit `Success == false`, nicht `null`.

- **(2)** `ValueSpan` liefert den Text ohne neuen String. `int.Parse` nimmt ihn direkt.
- **(3)** Die Regex prüft nur die Form. Ob es den Tag gibt, prüft `DateOnly.TryParseExact`.

Matches: alle Treffer

Eine Matrikelnummer hat 8 Ziffern. Die folgende Regex sucht alle in einem Text. Die Wortgrenzen `\b` verhindern, dass sie aus einer längeren Zahl 8 Ziffern herausschneidet.

```
using System.Text.RegularExpressions;

string text = "Angemeldet: 01234567 (Huber), 12345678 (Gruber), 1234567 (Tippfehler),
123456789";
MatchCollection matches = Regex.Matches(text, @"\b[0-9]{8}\b");
foreach (Match m in matches)
{
    Console.WriteLine($"{m.Value} at index {m.Index}");
}
List<string> numbers = matches.Select(m => m.Value).ToList();            // (1)
Console.WriteLine(string.Join(", ", numbers));
```

```
01234567 at index 12
12345678 at index 30
01234567, 12345678
```

- **(1)** `MatchCollection` implementiert `IEnumerable<Match>`, du kannst also LINQ verwenden.

Replace

Der Ersetzungstext funktioniert wie im Editor: `$1` für eine Gruppe, `${name}` für eine benannte Gruppe. Brauchst du mehr Logik, übergibst du statt des Ersetzungstexts ein Lambda, das jeden Treffer bekommt und den Ersatz liefert.

```
using System.Text.RegularExpressions;

string text = "Prüfung am 24.12.2026, Matrikelnummer 01234567";
string iso = Regex.Replace(text,
    @"(?<day>[0-9]{2})\.(?<month>[0-9]{2})\.(?<year>[0-9]{4})",
    "${year}-${month}-${day}");                                           // (1)
Console.WriteLine(iso);

string masked = Regex.Replace(iso, @"\b[0-9]{8}\b",
    m => new string('*', 5) + m.Value[5..]);                              // (2)
Console.WriteLine(masked);
```

```
Prüfung am 2026-12-24, Matrikelnummer 01234567
Prüfung am 2026-12-24, Matrikelnummer *****567
```

- **(1)** Das Datum wird ins ISO-Format umgestellt. Die Gruppen kommen in neuer Reihenfolge.
- **(2)** Das Lambda bekommt jeden Treffer als `Match` und zeigt nur die letzten drei Ziffern.

Split und eine CSV-Zeile

`string.Split` trennt nur an einem festen Zeichen. `Regex.Split` trennt an einem Muster, z. B. an einem Strichpunkt mit beliebig viel Leerraum davor und danach:

```
using System.Text.RegularExpressions;

string[] parts = Regex.Split("Wien ; Graz;Linz ;   Salzburg", @"\s*;\s*");
Console.WriteLine(string.Join("|", parts));
```

```
Wien|Graz|Linz|Salzburg
```

Echte CSV-Zeilen sind schwieriger. Ein Feld darf in Anführungszeichen stehen und dann selbst einen `;` enthalten. Ein Anführungszeichen im Feld wird verdoppelt (`""`). Hier hilft `Split` nicht mehr. Stattdessen beschreibst du, wie ein *Feld* aussieht, und holst alle Felder mit `Matches`:

```
using System.Text.RegularExpressions;

string line = """4711;"Huber; Anna";;"Er sagt ""Servus""";Wien""";       // (1)
var field = new Regex("""(?:^|;)(?<field>"(?:[^"]|"")*"|[^;]*)""");       // (2)

foreach (Match m in field.Matches(line))
{
    string value = m.Groups["field"].Value;
    if (value.StartsWith('"'))
    {
        value = value[1..^1].Replace("\"\"", "\"");                        // (3)
    }
    Console.WriteLine($"[{value}]");
}
```

```
[4711]
[Huber; Anna]
[]
[Er sagt "Servus"]
[Wien]
```

- **(1)** Ein Raw String Literal braucht keine Escapes. Der Inhalt enthält bis zu drei Anführungszeichen hintereinander, deshalb steht der String zwischen je vier.
- **(2)** Jedes Feld beginnt am Zeilenanfang oder nach einem `;`. Danach kommt entweder ein Feld in Anführungszeichen (beliebige Zeichen außer `"` oder ein verdoppeltes `""`) oder ein Feld ohne `;`. Das zweite `[^;]*` darf auch leer sein, so findest du leere Felder.
- **(3)** Die äußeren Anführungszeichen werden entfernt und `""` wird zu `"`.

Für ganze CSV-Dateien nimmst du besser eine Bibliothek wie *CsvHelper*. Sie kann z. B. auch Felder, die über mehrere Zeilen gehen.

RegexOptions

Optionen übergibst du als weiteres Argument und kombinierst sie mit `|`. Die wichtigsten:

Option	Wirkung
`IgnoreCase`	Groß- und Kleinschreibung egal
`Multiline`	`^` und `$` passen an jedem Zeilenanfang und -ende
`Singleline`	`.` passt auch auf `\n`
`IgnorePatternWhitespace`	Leerzeichen im Muster werden ignoriert, `#` leitet einen Kommentar ein
`ExplicitCapture`	nur benannte Gruppen speichern, `(…)` wirkt wie `(?:…)`

Im Editor gilt immer das Verhalten von `Multiline`: `^` und `$` beziehen sich auf jede Zeile.

[GeneratedRegex]

Eine Regex wird vor der ersten Suche übersetzt. Lege sie deshalb einmal an und verwende sie oft, statt in einer Schleife immer wieder `new Regex(…)` zu schreiben. Am besten geht das mit dem Attribut `[GeneratedRegex]`. Ein *Source Generator* schreibt beim Kompilieren C#-Code, der genau dein Muster sucht. Du deklarierst dafür nur eine `partial`-Methode in einer `partial`-Klasse. In einer Konsolen-App steht die Klasse nach den Top-Level-Statements:

```
using System.Text.RegularExpressions;

string[] inputs = ["1010", "0123", "4020"];
foreach (string input in inputs)
{
    Console.WriteLine($"{input}: {Patterns.PostalCode().IsMatch(input)}");
}

/// <summary>Regular expressions that are generated at compile time.</summary>
static partial class Patterns
{
    /// <summary>Austrian postal code: four digits, the first one is not 0.</summary>
    [GeneratedRegex(@"^[1-9][0-9]{3}\z", RegexOptions.None, matchTimeoutMilliseconds:
100)]
    public static partial Regex PostalCode();
}
```

```
1010: True
0123: False
```

```
4020: True
```

Ein ungültiges Muster ist damit ein **Compilerfehler** und kein Absturz zur Laufzeit. In Visual Studio zeigt *Go To Definition* (F12) den erzeugten Code samt einer Erklärung des Musters in Worten. Der Parameter `matchTimeoutMilliseconds` setzt ein Zeitlimit. Warum das wichtig ist, zeigt der nächste Abschnitt.

Sicherheit: immer ein Zeitlimit

Manche Ausdrücke werden bei langen Eingaben extrem langsam. Typisch sind verschachtelte Quantoren wie `(a+)+$`: Bei `aaa…a!` mit 30 `a` probiert die Regex rund eine Milliarde Möglichkeiten durch, jedes weitere Zeichen verdoppelt die Arbeit. Kommt der Text von Benutzern (Formular, Datei, Web-API), kann ein Angreifer mit einem passenden langen String deinen Server lahmlegen. Das heißt *ReDoS* (Regular Expression Denial of Service).

Prüfst du Eingaben von Benutzern, setze deshalb **immer** ein Zeitlimit. Ohne Angabe gibt es keines. Ist es überschritten, wirft die Regex eine `RegexMatchTimeoutException`:

```
using System.Text.RegularExpressions;

var regex = new Regex(@"(a+)+$", RegexOptions.None, TimeSpan.FromMilliseconds(100));
string input = new string('a', 40) + "!";
try
{
    Console.WriteLine(regex.IsMatch(input));
}
catch (RegexMatchTimeoutException e)
{
    Console.WriteLine($"Timeout after {e.MatchTimeout.TotalMilliseconds} ms: {e.
Pattern}");
}
```

```
Timeout after 100 ms: (a+)+$
```

Verschachtle außerdem keine Quantoren über dieselben Zeichen: `(a+)+` bedeutet dasselbe wie `a+`. Die Option `RegexOptions.NonBacktracking` schließt das Problem ganz aus, unterstützt dafür aber keine Lookaheads und keine Rückreferenzen wie `\1`.

Typische Fehler

```
using System.Text.RegularExpressions;

Console.WriteLine(Regex.IsMatch("PLZ 1010", @"[1-9][0-9]{3}"));             // (1)
Console.WriteLine(Regex.IsMatch("24-12-2026", @"^[0-9]{2}.[0-9]{2}.[0-9]{4}$")); // (2)
Console.WriteLine(Regex.IsMatch("Herrgott", @"^Herr|Frau$"));               // (3)
Console.WriteLine(Regex.IsMatch("Wien", "\bWien\b"));                        // (4)
```

```
Console.WriteLine(Regex.IsMatch("_", @"^[A-z]$"));                    // (5)
```

```
True
True
True
False
True
```

- **(1)** Anker vergessen: Die Regex findet `1010` irgendwo im String. Für eine Prüfung brauchst du `^···\z`.
- **(2)** Punkt nicht escapt: `.` ist *jedes* Zeichen, auch `-`. Für den Punkt selbst schreibst du `\.`.
- **(3)** `|` ohne Gruppe: Alles, was mit `Herr` beginnt, passt. Richtig ist `^(?:Herr|Frau)$`.
- **(4)** Ohne `@` ist `\b` im C#-String das Steuerzeichen *Backspace*. Der Compiler meldet nichts, die Regex findet nie etwas.
- **(5)** `[A-z]` enthält zwischen `Z` und `a` auch `[ \ ] ^ _` und `` ` ``. Gemeint ist `[A-Za-z]`.

Weitere Klassiker, die du schon kennst: das gierige `.*` statt `.*?` oder `[^···]`, `$` statt `\z` bei Prüfungen, `\d` statt `[0-9]`, ein fehlendes Zeitlimit bei Benutzereingaben und der Versuch, HTML oder JSON mit Regex zu zerlegen.

Zusatzmaterial

Im Regex-Trainer 14_Regulaere_Ausdruecke.html siehst du animiert, wie eine Regex über einen Text läuft und wie Suchen und Ersetzen arbeitet. Danach übst du mit Live-Feedback: Ausdrücke, die bestimmte Wörter erkennen und andere nicht, und Ersetzungen, die einen Text in eine Zielform bringen.

Parser für arithmetische Ausdrücke

Ein Taschenrechner bekommt den Text `2 * (3 + 4)` und soll 14 ausgeben. Für das Programm ist der Text aber nur eine Folge von Zeichen. Es muss zuerst die Struktur erkennen: Was gehört zusammen, was wird zuerst gerechnet? Genau das macht ein *Parser*. Parser stecken in jedem Compiler, in Excel-Formeln, in SQL-Datenbanken und in jeder Bibliothek, die JSON oder XML liest. In diesem Kapitel baust du einen vollständigen Parser für Rechenausdrücke, direkt aus einer Grammatik.

Vom Text zum Wert

Die Verarbeitung läuft in drei Stufen. Jede Stufe hat eine eigene Klasse mit genau einer Aufgabe:



Stufe	Eingabe	Ausgabe	Aufgabe
Tokenizer (Lexer)	`string`	`List<Token>`	Zeichen zu Wörtern zusammenfassen, Leerzeichen entfernen
Parser	`List<Token>`	`Expr` (AST)	Struktur nach der Grammatik erkennen
Evaluator	`Expr`	`double`	den Baum ausrechnen

Die Trennung zwischen Tokenizer und Parser hat einen theoretischen Grund. Die Tokens selbst (Zahlen, Operatoren) bilden eine *reguläre* Sprache, ein endlicher Automat erkennt sie (siehe Endliche Automaten). Ausdrücke mit beliebig tief geschachtelten Klammern sind dagegen *kontextfrei* (Typ 2 in der Chomsky-Hierarchie, siehe Grammatiken). Dafür braucht der Parser einen Stack. Diesen Stack bekommst du gratis: Es ist der Aufrufstack der rekursiven Methoden.

Der vollständige Code liegt in Beispiele/15_ExpressionParser. Öffne die Datei *ExpressionParser.sln* oder starte das Programm in der Konsole:

```
cd Beispiele/15_ExpressionParser
dotnet run --project ExpressionParser
```

Das Projekt besteht aus diesen Dateien:

Datei	Inhalt
TokenKind.cs, *Token.cs*	Art eines Tokens, Token als `record`
Tokenizer.cs	zerlegt den Text in Tokens
Expr.cs, *NumberExpr.cs*, *BinaryExpr.cs*, *BinaryOperator.cs*, *NegateExpr.cs*	Knoten des AST
Parser.cs	rekursiver Abstiegsparser
ParseException.cs	Fehler mit Position
Evaluator.cs	rechnet den AST aus
ExpressionCompiler.cs	übersetzt den AST in einen .NET-Ausdrucksbaum
Program.cs	Beispiele und Eingabeschleife

Die Codeblöcke mit *Auszug aus …* stammen aus diesen Dateien. Die XML-Kommentare sind weggelassen, `// ...` markiert ausgelassene Stellen. Einzeln kompilieren die Auszüge nicht, das ganze Projekt schon.

Tokens

Ein *Token* ist die kleinste Einheit mit Bedeutung, so wie ein Wort in einem Satz. Der Tokenizer (auch *Lexer* oder *Scanner* genannt) macht aus `2 * (3 + 4)` die Tokens `2`, `*`, `(`, `3`, `+`, `4` und `)`. Jedes Token kennt seine Art, seinen Text und seine Position im Eingabetext. Die Position brauchst du später für Fehlermeldungen.

Auszug aus TokenKind.cs und Token.cs:

```
public enum TokenKind
{
    Number,
    Plus,
    Minus,
    Star,
    Slash,
    LeftParen,
    RightParen,
    End
}

public record Token(TokenKind Kind, string Text, int Position)
{
    public override string ToString() => $"{Kind} '{Text}' at {Position}";
}
```

Die Art `End` gibt es im Eingabetext nicht. Der Tokenizer hängt sie trotzdem immer an. So hat der Parser immer ein aktuelles Token und muss nie prüfen, ob die Liste schon zu Ende ist.

Zahlen bestehen aus mehreren Zeichen. Ihre Form beschreibt eine kleine Grammatik in EBNF (die Notation kennst du aus Darstellungsformen):

```
number = digit { digit } [ "." digit { digit } ]
digit  = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

`42`, `3.5` und `0.25` sind Zahlen, `3.` und `.5` nicht. Der Tokenizer geht den Text Zeichen für Zeichen durch:

Auszug aus Tokenizer.cs:

```
public class Tokenizer(string source)
{
    private int _pos;

    public List<Token> Tokenize()
    {
        var tokens = new List<Token>();
        _pos = 0;
        while (_pos < source.Length)
        {
            char c = source[_pos];
            if (char.IsWhiteSpace(c))
            {
                _pos++;
                continue;
            }
            if (char.IsAsciiDigit(c))
            {
                tokens.Add(ReadNumber());
                continue;
            }
            TokenKind kind = c switch
            {
                '+' => TokenKind.Plus,
                '-' => TokenKind.Minus,
                '*' => TokenKind.Star,
                '/' => TokenKind.Slash,
                '(' => TokenKind.LeftParen,
                ')' => TokenKind.RightParen,
                _ => throw new ParseException($"Unexpected character '{c}'", _pos)
            };
            tokens.Add(new Token(kind, c.ToString(), _pos));
            _pos++;
        }
        tokens.Add(new Token(TokenKind.End, "", source.Length));
        return tokens;
    }

    private Token ReadNumber()
    {
        int start = _pos;
        SkipDigits();
        if (_pos < source.Length && source[_pos] == '.')
        {
            _pos++;
```

```
            // "3." is not a number: at least one digit has to follow the decimal
point.
            if (_pos >= source.Length || !char.IsAsciiDigit(source[_pos]))
            {
                throw new ParseException("Expected digit after '.'", _pos);
            }
            SkipDigits();
        }
        return new Token(TokenKind.Number, source[start.._pos], start);
    }
    // ...
}
```

- Leerzeichen überspringt der Tokenizer. Der Parser sieht sie nie, deshalb ist `2*(3+4)` dasselbe wie `2 * ( 3 + 4 )`.
- *ReadNumber()* liest so viele Ziffern wie möglich. `123` wird daher ein Token und nicht drei. Diese Regel heißt *longest match* (längster Treffer).
- *ReadNumber()* folgt direkt der EBNF: `digit { digit }` ist *SkipDigits()*, der optionale Teil `[ "." … ]` ist das `if`.

Das Programm gibt die Tokens für `2 * (3 + 4)` aus:

```
Tokens of 2 * (3 + 4):
  Number '2' at 0
  Star '*' at 2
  LeftParen '(' at 4
  Number '3' at 5
  Plus '+' at 7
  Number '4' at 9
  RightParen ')' at 10
  End '' at 11
```

Die Grammatik

Der Parser setzt diese EBNF um:

```
expr   = term { ("+" | "-") term }
term   = factor { ("*" | "/") factor }
factor = number | "(" expr ")" | "-" factor
```

Ein *expr* ist eine Summe oder Differenz von Termen. Ein *term* ist ein Produkt oder Quotient von Faktoren. Ein *factor* ist eine Zahl, ein geklammerter Ausdruck oder ein negierter Faktor.

Präzedenz. Die Grammatik `E → E + E | E * E | id` aus Grammatiken ist mehrdeutig. Für `id + id * id` gibt es zwei Ableitungsbäume. Die Lösung sind Präzedenzstufen: Jede Stufe bekommt eine eigene Regel. Je tiefer eine Regel liegt, desto stärker binden ihre Operatoren. `*` steht in *term*, also eine Stufe tiefer als `+`. Deshalb ist `2 * 3 + 4` immer `(2 * 3) + 4`, es gibt nur noch einen

Baum.

Stufe	Regel	Operatoren	bindet
1	expr	`+ -`	am schwächsten
2	term	`* /`	stärker
3	factor	Vorzeichen `-`, Klammern	am stärksten

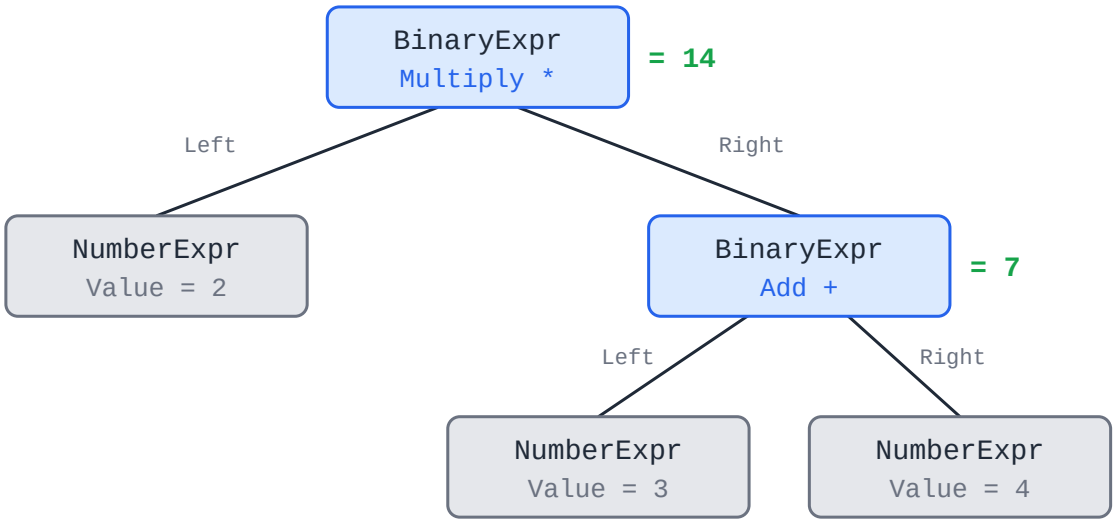
Assoziativität. `1 - 2 - 3` muss `(1 - 2) - 3 = -4` ergeben und nicht `1 - (2 - 3) = 2`. Die Wiederholung `{ }` liest die Terme von links nach rechts. Der Parser hängt jeden neuen Term rechts an den bisherigen Baum an. So entsteht ein linkslastiger Baum, die Operatoren sind *linksassoziativ*.

Keine Linksrekursion. In BNF würdest du `<expr> ::= <expr> "+" <term> | <term>` schreiben. Als Methode umgesetzt ruft *ParseExpr()* dann als Erstes wieder *ParseExpr()* auf, ohne ein Token zu lesen. Das endet mit einer `StackOverflowException`. Die Wiederholung in EBNF vermeidet dieses Problem. Deshalb schreibst du Grammatiken für den rekursiven Abstieg immer mit `{ }`.

Vorzeichen. Das Minus steht zweimal in der Grammatik. In *expr* ist es die Subtraktion, in *factor* das Vorzeichen. Welches gemeint ist, entscheidet die Stelle: Am Anfang eines Faktors ist es ein Vorzeichen. So sind auch `-(2 + 3)` und `2 - -3` gültig.

Der AST

Der Parser liefert einen *abstrakten Syntaxbaum* (AST, *abstract syntax tree*). „Abstrakt“ heißt: Der Baum enthält nur, was für die Bedeutung wichtig ist. Klammern, Leerzeichen und Tokens fehlen. Die Klammern in `2 * (3 + 4)` stecken nur noch in der Form des Baums:



Die Klammern stehen nicht im Baum: (3 + 4) ist der rechte Teilbaum der Multiplikation.

Für die Knoten reichen drei Records mit einer gemeinsamen Basisklasse:

Auszug aus Expr.cs, NumberExpr.cs, BinaryOperator.cs, BinaryExpr.cs und NegateExpr.cs:

```
public abstract record Expr;

public sealed record NumberExpr(double Value) : Expr
{
    public override string ToString() => Value.ToString(CultureInfo.InvariantCulture);
}

public enum BinaryOperator
{
    Add,
    Subtract,
    Multiply,
    Divide
}

public sealed record BinaryExpr(Expr Left, BinaryOperator Operator, Expr Right) : Expr
{
    // ... ToString() returns the prefix notation, e.g. (* 2 (+ 3 4))
}

public sealed record NegateExpr(Expr Operand) : Expr
{
    public override string ToString() => $"(neg {Operand})";
}
```

Records passen gut zu einem AST:

- Sie sind unveränderlich. Ein fertiger Baum ändert sich nicht mehr.
- Sie vergleichen nach Wert. In einem Test kannst du einen erwarteten Baum mit `new` aufbauen und mit `Assert.Equal` gegen das Ergebnis des Parsers prüfen.
- Sie lassen sich in Pattern Matching zerlegen. Das nutzt der Evaluator.

Für die Konsolenausgabe überschreiben die Records `ToString()`. Der Baum oben wird dann zu `(* 2 (+ 3 4))`: zuerst der Operator, dann die beiden Teilbäume. Diese Schreibweise heißt *Präfixnotation*. Sie braucht keine Präzedenzregeln, weil jede Klammer genau einen Knoten zeigt.

Rekursiver Abstieg

Ein *rekursiver Abstiegsparser* (*recursive descent parser*) übersetzt die EBNF fast mechanisch in Code. Jedes Nichtterminal wird eine Methode, die einen Teilbaum zurückgibt:

EBNF	C#
Nichtterminal `term`	Methode *ParseTerm()*, ein Vorkommen in einer Regel ist ein Aufruf
Terminal `")"`	*Expect(TokenKind.RightParen, …)*: prüfen und verbrauchen

EBNF	C#
Wiederholung `{ … }`	`while`, solange das aktuelle Token die Wiederholung beginnen kann
Option `[ … ]`	`if`
Alternative `a | b | c`	`switch` über das aktuelle Token

Der Parser entscheidet immer nur mit dem **aktuellen Token**, welchen Weg er nimmt. Er muss nie zurückgehen und etwas anderes probieren. Das klappt, weil jede Alternative von *factor* mit einem anderen Token beginnt: einer Zahl, `(` oder `-`. Grammatiken mit dieser Eigenschaft heißen *LL(1)*: Lesen von links, Linksableitung, 1 Token Vorschau.

Hilfsmethoden

Der Parser speichert die Tokenliste und die Position des aktuellen Tokens. Drei kleine Methoden reichen für den Zugriff:

Auszug aus Parser.cs:

```
private Token Current => _tokens[_pos];

private Token Advance()
{
    Token token = Current;
    if (token.Kind != TokenKind.End)
    {
        _pos++;
    }
    return token;
}

private Token Expect(TokenKind kind, string description)
{
    if (Current.Kind != kind)
    {
        throw new ParseException(
            $"Expected {description} but found {Describe(Current)}", Current.Position);
    }
    return Advance();
}
```

- *Current* ist das Token, das als Nächstes gelesen wird. Weil *Advance()* nie über `End` hinausgeht, ist der Index immer gültig.
- *Advance()* verbraucht das aktuelle Token und liefert es zurück.
- *Expect()* verbraucht ein Token, das an dieser Stelle stehen **muss**. Fehlt es, meldet der Parser einen Fehler. *Describe()* liefert dafür `'*'` oder `end of input`.

Die Methoden für die Regeln

Auszug aus Parser.cs:

```
// expr = term { ("+" | "-") term }
private Expr ParseExpr()
{
    Expr left = ParseTerm();
    while (Current.Kind is TokenKind.Plus or TokenKind.Minus)    // (1)
    {
        Token op = Advance();
        Expr right = ParseTerm();
        left = op.Kind == TokenKind.Plus                         // (2)
            ? new BinaryExpr(left, BinaryOperator.Add, right)
            : new BinaryExpr(left, BinaryOperator.Subtract, right);
    }
    return left;
}

// term = factor { ("*" | "/") factor }
private Expr ParseTerm()
{
    Expr left = ParseFactor();
    while (Current.Kind is TokenKind.Star or TokenKind.Slash)
    {
        Token op = Advance();
        Expr right = ParseFactor();
        left = op.Kind == TokenKind.Star
            ? new BinaryExpr(left, BinaryOperator.Multiply, right)
            : new BinaryExpr(left, BinaryOperator.Divide, right);
    }
    return left;
}

// factor = number | "(" expr ")" | "-" factor
private Expr ParseFactor()
{
    Token token = Current;
    switch (token.Kind)                                          // (3)
    {
        case TokenKind.Number:
            Advance();
            return new NumberExpr(double.Parse(token.Text, CultureInfo.
InvariantCulture));
        case TokenKind.LeftParen:
            Advance();
            Expr inner = ParseExpr();                            // (4)
            Expect(TokenKind.RightParen, "')'");
            return inner;
        case TokenKind.Minus:
            Advance();
            return new NegateExpr(ParseFactor());                // (5)
        default:
            throw new ParseException(
```

```
                $"Expected number, '(' or '-' but found {Describe(token)}", token
.Position);
    }
}
```

- **(1)** Die Wiederholung `{ ("+" | "-") term }` wird zu `while`. Die Bedingung prüft, ob das aktuelle Token eine Wiederholung beginnt. Bei jedem anderen Token ist *expr* fertig. Ob das Token danach passt (z. B. `)`), entscheidet die aufrufende Methode.
- **(2)** Der bisherige Baum wird zum linken Kind des neuen Knotens. Daraus entsteht die Linksassoziativität: `1 - 2 - 3` ergibt `(- (- 1 2) 3)`.
- **(3)** Die drei Alternativen von *factor* beginnen mit verschiedenen Tokens. Ein `switch` über das aktuelle Token wählt die richtige aus. Passt keine, ist die Eingabe falsch.
- **(4)** Hier schließt sich der Kreis: *factor* ruft *expr* auf. Diese indirekte Rekursion erlaubt beliebig tief geschachtelte Klammern. Die Klammern selbst landen nicht im Baum, die Methode gibt den inneren Ausdruck direkt zurück.
- **(5)** Das Vorzeichen ruft *ParseFactor()* rekursiv auf. So funktionieren auch `--3` und `-(2 + 3)`.

ParseTerm() ist wie *ParseExpr()* gebaut, nur eine Stufe tiefer. Mehr ist für die Präzedenz nicht nötig: Die Rangfolge steckt allein darin, welche Methode welche aufruft.

Der Einstieg

Von außen gibt es nur zwei öffentliche Methoden:

Auszug aus Parser.cs:

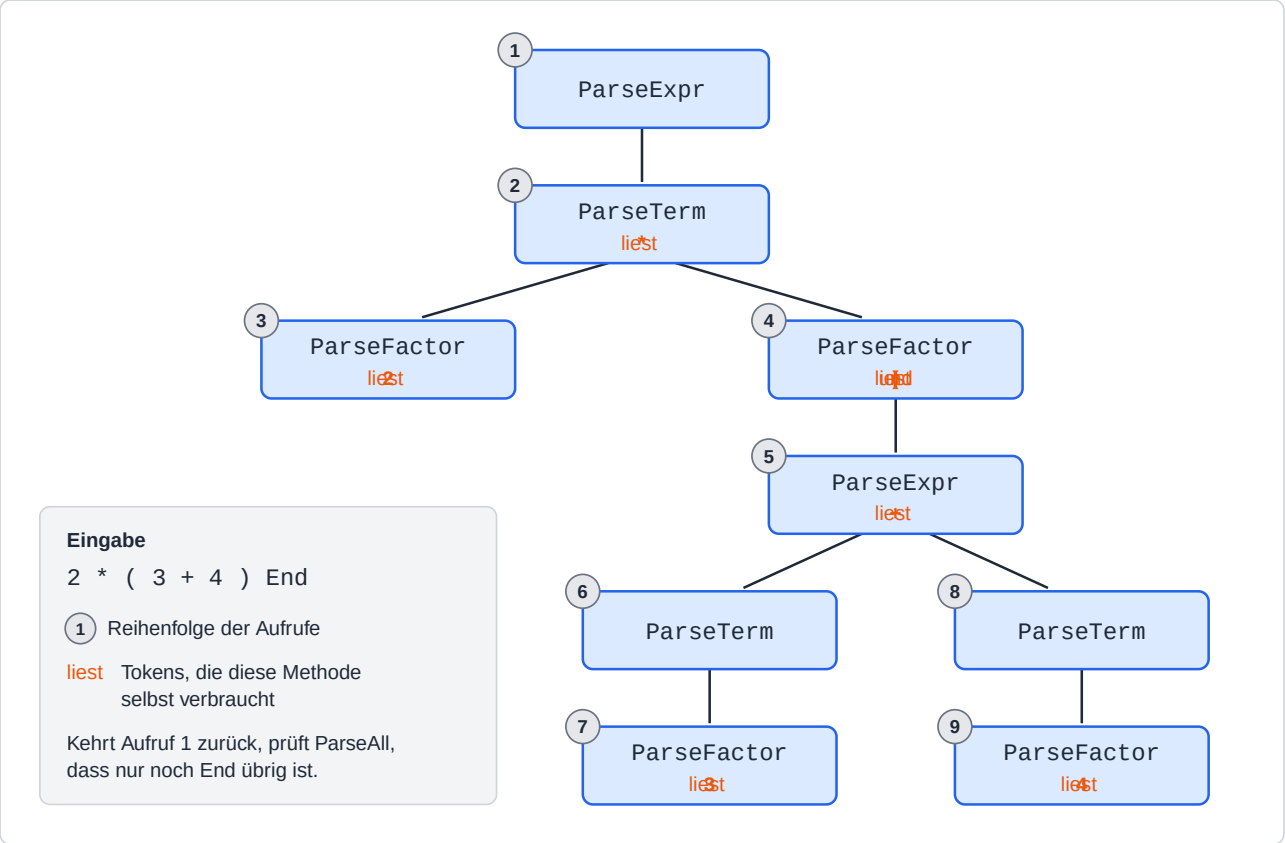
```
public static Expr Parse(string source)
{
    var parser = new Parser(new Tokenizer(source).Tokenize());
    return parser.ParseAll();
}

public Expr ParseAll()
{
    _pos = 0;
    Expr expr = ParseExpr();
    Expect(TokenKind.End, "end of input");
    return expr;
}
```

Das `Expect(TokenKind.End, …)` am Ende ist wichtig. Ohne diese Prüfung würde der Parser bei `2 3` die Zahl 2 lesen, aufhören und `3` stillschweigend ignorieren. Die ganze Eingabe muss aber ein *expr* sein, nicht nur ihr Anfang.

Ablauf für `2 * (3 + 4)`

Die Grafik zeigt, welche Methode welche aufruft und welche Tokens sie selbst verbraucht. Die Nummern geben die Reihenfolge der Aufrufe an:



1. *ParseExpr()* braucht zuerst einen Term und ruft *ParseTerm()* auf.
2. *ParseTerm()* braucht einen Faktor und ruft *ParseFactor()* auf.
3. Das aktuelle Token ist `2`. *ParseFactor()* liefert `NumberExpr(2)`.
4. Zurück in *ParseTerm()*: Das aktuelle Token ist `*`, die Schleife läuft. *ParseTerm()* liest `*` und ruft wieder *ParseFactor()* auf. Dieser Aufruf liest `(` und steigt in *ParseExpr()* ab.
5. Der innere *ParseExpr()*-Aufruf parst `3 + 4` (Aufrufe 6 bis 9) und liefert `(+ 3 4)`. Bei `)` endet seine Schleife, denn `)` ist weder `+` noch `-`.
6. *ParseFactor()* verbraucht `)` mit *Expect()* und gibt `(+ 3 4)` zurück.
7. *ParseTerm()* baut `(* 2 (+ 3 4))`. Das aktuelle Token ist `End`, beide Schleifen enden.

Die Form des Aufrufbaums entspricht der Form des Ableitungsbaums aus Grammatiken. Der Parser baut den Ableitungsbaum aber nie auf, er geht ihn nur einmal mit seinen Aufrufen ab. Daher kommt der Name *rekursiver Abstieg*: Der Parser steigt von der Startregel *expr* zu den Blättern ab.

Fehlermeldungen mit Position

Eine gute Fehlermeldung sagt, **was** erwartet wurde, **was** stattdessen kam und **wo**. Dafür gibt es eine eigene Exception mit der Position:

Auszug aus ParseException.cs:

```
public class ParseException(string message, int position) : Exception(message)
{
    public int Position { get; } = position;
```

```
}
```

Tokenizer und Parser werfen dieselbe Exception. Für die fehlerhaften Beispiele aus *Program.cs* meldet sie diese Positionen und Texte:

Eingabe	Position und Meldung	erkannt von
`4 % 2`	2: `Unexpected character '%'`	Tokenizer
`3. + 1`	2: `Expected digit after '.'`	Tokenizer
`2 + * 3`	4: `Expected number, '(' or '-' but found '*'`	*ParseFactor()*
`2 * (3 + 4`	10: `Expected ')' but found end of input`	*Expect()*
`2 3`	2: `Expected end of input but found '3'`	*ParseAll()*

`1 / 0` ist übrigens syntaktisch korrekt. Der Parser meldet keinen Fehler, der Evaluator liefert für `double` den Wert ∞. Ob das sinnvoll ist, ist eine Frage der **Semantik**, nicht der Syntax.

Auswerten mit einem switch-Ausdruck

Der Evaluator rechnet den Baum rekursiv aus. Zuerst berechnet er die Kinder, dann den Knoten:

Auszug aus Evaluator.cs:

```
public static double Evaluate(Expr expr) => expr switch
{
    NumberExpr(var value) => value,
    NegateExpr(var operand) => -Evaluate(operand),
    BinaryExpr(var left, BinaryOperator.Add, var right) => Evaluate(left) + Evaluate
(right),
    BinaryExpr(var left, BinaryOperator.Subtract, var right) => Evaluate(left) -
Evaluate(right),
    BinaryExpr(var left, BinaryOperator.Multiply, var right) => Evaluate(left) *
Evaluate(right),
    BinaryExpr(var left, BinaryOperator.Divide, var right) => Evaluate(left) /
Evaluate(right),
    _ => throw new ArgumentException($"Unknown node {expr.GetType().Name}", nameof(
expr))
};
```

- Jede Zeile ist ein *Positional Pattern*. `BinaryExpr(var left, BinaryOperator.Add, var right)` passt nur auf Additionen und zerlegt den Knoten gleich in seine Teile. Das geht, weil Records automatisch eine `Deconstruct`-Methode haben.
- Der letzte Fall `_` ist nötig, weil der Compiler nicht weiß, ob es noch weitere Ableitungen von `Expr` gibt. Fügst du später einen neuen Knotentyp hinzu und vergisst ihn hier, bekommst du diese Exception.

Für den Baum aus der Grafik oben rechnet der Evaluator zuerst `3 + 4 = 7` und dann `2 * 7 = 14`.

Program.cs gibt für feste Beispiele den AST und den Wert aus (die fehlerhaften Beispiele stehen in der Tabelle oben). Danach liest es Ausdrücke von der Konsole, bis du eine leere Zeile eingibst:

```
2 * (3 + 4)    AST: (* 2 (+ 3 4))          Value: 14
2 * 3 + 4      AST: (+ (* 2 3) 4)          Value: 10
1 - 2 - 3      AST: (- (- 1 2) 3)          Value: -4
-(2 + 3) * 4   AST: (* (neg (+ 2 3)) 4)    Value: -20
7 / 2 - -1.5   AST: (- (/ 7 2) (neg 1.5))  Value: 5
```

An der AST-Spalte siehst du Präzedenz und Assoziativität direkt: In `2 * 3 + 4` ist `+` die Wurzel, also wird `*` zuerst gerechnet. In `1 - 2 - 3` steht die erste Subtraktion innen.

Ausdrucksbäume in .NET

NET hat im Namespace `System.Linq.Expressions` fertige Klassen für Ausdrucksbäume (expression trees).

Der Basistyp heißt `Expression`, für jede Art von Knoten gibt es eine abgeleitete Klasse.

Der Compiler parst für dich

Weist du ein Lambda einer Variable vom Typ `Expression<Func<···>>` zu, erzeugt der C#-Compiler keinen ausführbaren Code, sondern einen Baum, der das Lambda beschreibt. Dieses eigenständige Programm (kein Auszug) gibt den Baum in derselben Präfixnotation aus wie unser AST:

```
using System.Globalization;
using System.Linq.Expressions;

Expression<Func<double, double>> f = x => 2 * (x + 4);       // (1)
Console.WriteLine(ToPrefix(f.Body));
Expression<Func<double, double>> g = x => -x / 2;
Console.WriteLine(ToPrefix(g.Body));

Func<double, double> compiled = f.Compile();                  // (2)
Console.WriteLine(compiled(3));

// Walks the tree recursively, like the ToString() methods of our AST.
static string ToPrefix(Expression node) => node switch       // (3)
{
    BinaryExpression b => $"({Symbol(b.NodeType)} {ToPrefix(b.Left)} {ToPrefix(b.
Right)})",
    UnaryExpression { NodeType: ExpressionType.Negate } u => $"(neg {ToPrefix(u.
Operand)})",
    ConstantExpression { Value: double value } => value.ToString(CultureInfo
.InvariantCulture),
    ParameterExpression p => p.Name ?? "?",
    _ => throw new NotSupportedException($"Node type {node.NodeType} is not supported")
};
```

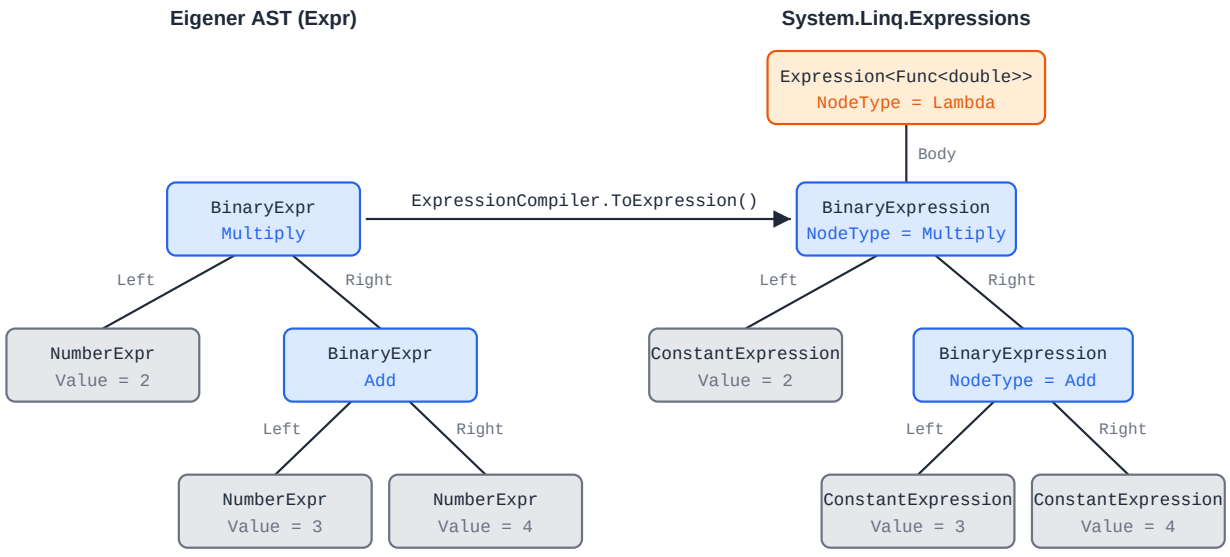
```
static char Symbol(ExpressionType type) => type switch
{
    ExpressionType.Add => '+',
    ExpressionType.Subtract => '-',
    ExpressionType.Multiply => '*',
    ExpressionType.Divide => '/',
    _ => throw new NotSupportedException($"Operator {type} is not supported")
};
```

```
(* 2 (+ x 4))
(/ (neg x) 2)
14
```

- **(1)** Den Text `2 * (x + 4)` hat schon der C#-Compiler geparst. `f.Body` ist die Wurzel des Baums, `f.Parameters` enthält `x`. Mit dem Typ `Func<double, double>` statt `Expression<…>` entsteht ganz normaler Code. Lambdas mit einem Block `{ … }` sind als Baum nicht erlaubt.
- **(2)** *Compile()* erzeugt zur Laufzeit IL-Code aus dem Baum und liefert einen Delegate. `compiled(3)` rechnet `2 * (3 + 4)` und ergibt wie unser Evaluator 14.
- **(3)** *ToPrefix()* ist gebaut wie *Evaluate()*. `BinaryExpression` entspricht `BinaryExpr`, `ConstantExpression` entspricht `NumberExpr`, `UnaryExpression` mit `NodeType` *Negate* entspricht `NegateExpr`. Neu ist `ParameterExpression` für `x`. Andere Knoten wie die `MethodCallExpression` in `x => Math.Sqrt(x)` lehnt die Methode ab.

Vom eigenen AST zum Ausdrucksbaum

Umgekehrt übersetzt die Klasse `ExpressionCompiler` unseren AST in einen `Expression`-Baum. Die Knoten erzeugen statische Methoden wie `Expression.Constant()` und `Expression.Add()`:



Gleiche Form, andere Typen: Aus BinaryExpr wird BinaryExpression, aus NumberExpr wird ConstantExpression.

Auszug aus ExpressionCompiler.cs:

```
public static Func<double> Compile(Expr expr)
{
    Expression body = ToExpression(expr);
    return Expression.Lambda<Func<double>>(body).Compile();
}

public static Expression ToExpression(Expr expr) => expr switch
{
    NumberExpr(var value) => Expression.Constant(value),
    NegateExpr(var operand) => Expression.Negate(ToExpression(operand)),
    BinaryExpr(var left, var op, var right) => Combine(op, ToExpression(left),
ToExpression(right)),
    _ => throw new ArgumentException($"Unknown node {expr.GetType().Name}", nameof(
expr))
};

private static BinaryExpression Combine(BinaryOperator op, Expression left, Expression
right) => op switch
{
    BinaryOperator.Add => Expression.Add(left, right),
    BinaryOperator.Subtract => Expression.Subtract(left, right),
    // ... Multiply and Divide
};
```

ToExpression() rechnet nicht, sondern baut für jeden Knoten unseres AST genau einen neuen Knoten. *Expression.Lambda()* macht daraus das Lambda `() => 2 * (3 + 4)`. *Program.cs* vergleicht den Delegate mit dem Evaluator, die Werte stimmen überein. Der Delegate läuft so schnell wie normaler C#-Code, das Kompilieren kostet aber Zeit. Es lohnt sich bei Ausdrücken, die du oft berechnest.

Praxis: LINQ-Provider und EF Core

Bei einer `List<T>` erwartet *Where()* einen `Func<T, bool>`, das Lambda wird einfach ausgeführt. Bei einem `IQueryable<T>` wie `db.Students` in EF Core erwartet *Where()* dagegen einen `Expression<Func<T, bool>>`. Aus `db.Students.Where(s => s.Age >= 18)` bekommt EF Core also einen Baum, geht ihn rekursiv durch wie *ToPrefix()* und erzeugt daraus SQL mit `WHERE Age >= 18`. Deshalb kannst du in einem EF-Core-*Where()* nicht jede C#-Methode verwenden: Eine eigene Methode ist dort nur ein `MethodCallExpression`-Knoten, für den EF Core kein SQL kennt. Die Abfrage bricht dann mit einer Exception ab.

Ausblick: Parser-Generatoren und Roslyn

Für große Grammatiken gibt es *Parser-Generatoren* wie **ANTLR**. Du schreibst nur die Grammatik (in ANTLR z. B. `expr : term (('+' | '-') term)* ;`), ANTLR erzeugt daraus Lexer und Parser, auf Wunsch auch in C#. Der C#-Compiler **Roslyn** verwendet dagegen einen handgeschriebenen rekursiven Abstiegsparser nach demselben Prinzip wie in diesem Kapitel, nur viel größer. Unser Parser bricht beim ersten Fehler ab, Roslyn hat eine *Fehlererholung*: Er überspringt Tokens bis zu einer sicheren Stelle, z. B. bis zum nächsten `;`, und meldet auch spätere Fehler. Über das NuGet-Paket *Microsoft.CodeAnalysis.CSharp* bekommst du den Syntaxbaum eines C#-

Programms.

Theoretische Informatik: Algorithmen

Graphen

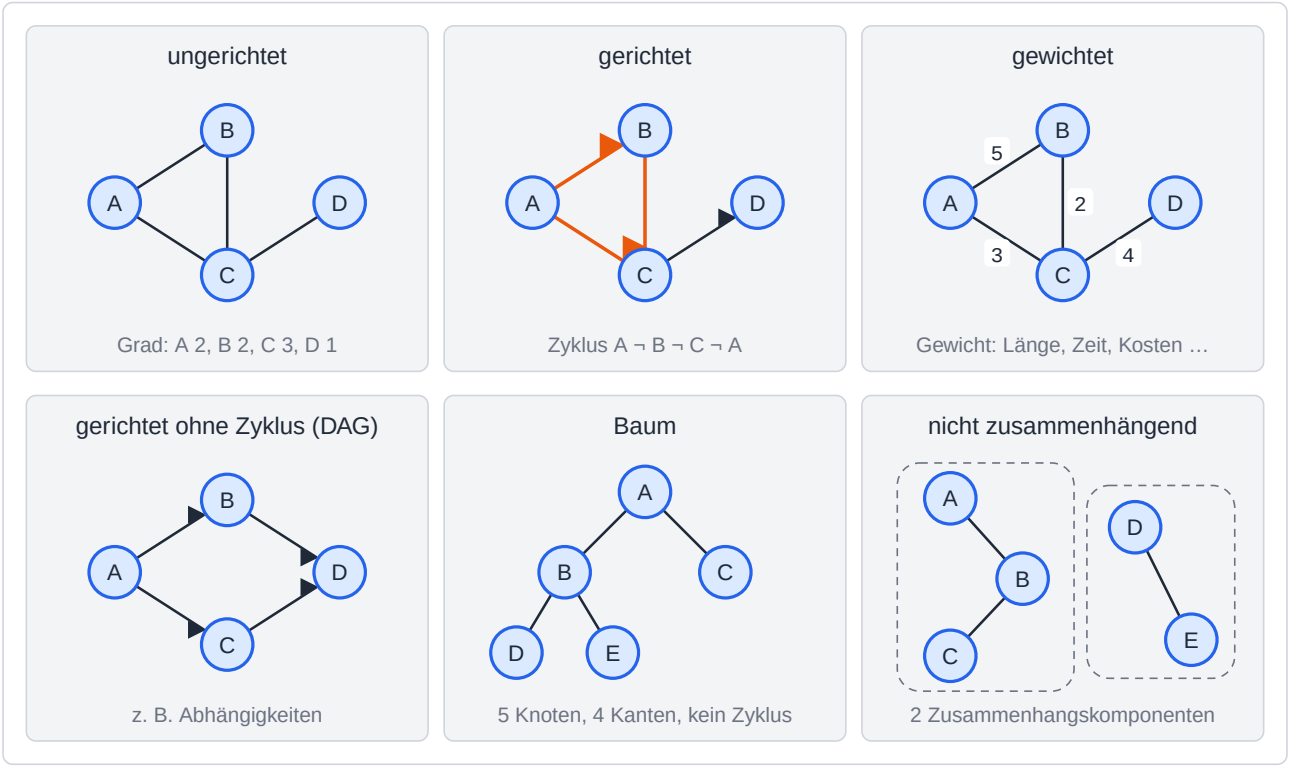
Ein Navi sucht den Weg durch ein Straßennetz. NuGet findet heraus, welche Pakete es vor deinem Projekt bauen muss. Ein soziales Netz schlägt dir Leute vor, die deine Freunde kennen. Das sind drei ganz verschiedene Probleme, aber sie haben dieselbe Struktur: Es gibt *Dinge* und *Beziehungen* zwischen diesen Dingen. Genau das beschreibt ein **Graph**. Wenn du ein Problem einmal als Graph formuliert hast, kannst du dafür bekannte Algorithmen verwenden. Die nächsten Kapitel stellen die wichtigsten davon vor.

Knoten und Kanten

Ein Graph G = (V, E) besteht aus

- einer Menge V von **Knoten** (englisch *vertices* oder *nodes*) und
- einer Menge E von **Kanten** (englisch *edges*). Jede Kante verbindet zwei Knoten.

Gezeichnet wird ein Knoten als Kreis, eine Kante als Linie oder Pfeil. Wo die Knoten auf dem Papier liegen, spielt keine Rolle. Entscheidend ist nur, welche Knoten verbunden sind.



Die Grafik zeigt die wichtigsten Arten von Graphen:

- **Ungerichtet:** Eine Kante {A, B} gilt in beide Richtungen. Beispiele: Freundschaften, Straßen ohne Einbahn, Türen zwischen Räumen.
- **Gerichtet:** Eine Kante (A, B) ist ein Pfeil von A nach B. Der Rückweg gilt nur, wenn es auch einen Pfeil (B, A) gibt. Beispiele: Einbahnstraßen, „folgt“ auf Instagram, Links zwischen Webseiten. Auch der Automat aus Endliche Automaten ist ein gerichteter Graph, dessen Kanten mit Zeichen beschriftet sind.

- **Gewichtet:** Jede Kante trägt eine Zahl, das *Gewicht*. Das kann eine Länge in Kilometern, eine Fahrzeit oder ein Preis sein. Gewichtete Graphen können gerichtet oder ungerichtet sein.

Wichtige Begriffe

Begriff	Bedeutung
benachbart	Zwei Knoten sind *benachbart* (Nachbarn), wenn eine Kante sie verbindet.
Grad deg(v)	Anzahl der Kanten an einem Knoten. In gerichteten Graphen unterscheidet man **Eingangsgrad** (Pfeile hinein) und **Ausgangsgrad** (Pfeile hinaus).
Pfad (Weg)	Folge von Knoten, bei der aufeinanderfolgende Knoten durch eine Kante verbunden sind, z. B. A → B → C. Die **Länge** ist die Anzahl der Kanten.
Zyklus (Kreis)	Pfad, der zu seinem Startknoten zurückkehrt, ohne eine Kante doppelt zu benutzen, z. B. A → B → C → A.
zusammenhängend	Ein ungerichteter Graph ist zusammenhängend, wenn es von jedem Knoten zu jedem anderen einen Pfad gibt.
Zusammenhangskomponente	Ein größtmöglicher zusammenhängender Teil des Graphen. Ein nicht zusammenhängender Graph zerfällt in mehrere Komponenten.
DAG	*Directed Acyclic Graph*: gerichteter Graph ohne Zyklus.
Baum	Zusammenhängender, ungerichteter Graph ohne Zyklus.

Ein paar Beobachtungen an der Grafik:

- Im ungerichteten Graphen hat C den Grad 3, D nur den Grad 1. Zählst du alle Grade zusammen (2 + 2 + 3 + 1 = 8), bekommst du genau doppelt so viel wie die Anzahl der Kanten (4). Das ist kein Zufall: Jede Kante hat zwei Enden und zählt deshalb bei zwei Knoten.
- Im gerichteten Graphen hat C den Eingangsgrad 1 (von B) und den Ausgangsgrad 2 (nach A und D). Der Pfad A → B → C → A ist ein Zyklus.
- Der DAG hat keinen Zyklus. Solche Graphen beschreiben Abhängigkeiten: „B braucht A“, „D braucht B und C“. Ein Zyklus würde bedeuten, dass etwas indirekt von sich selbst abhängt. Wie man daraus eine gültige Reihenfolge berechnet, zeigt die Topologische Sortierung.

Bäume als Spezialfall

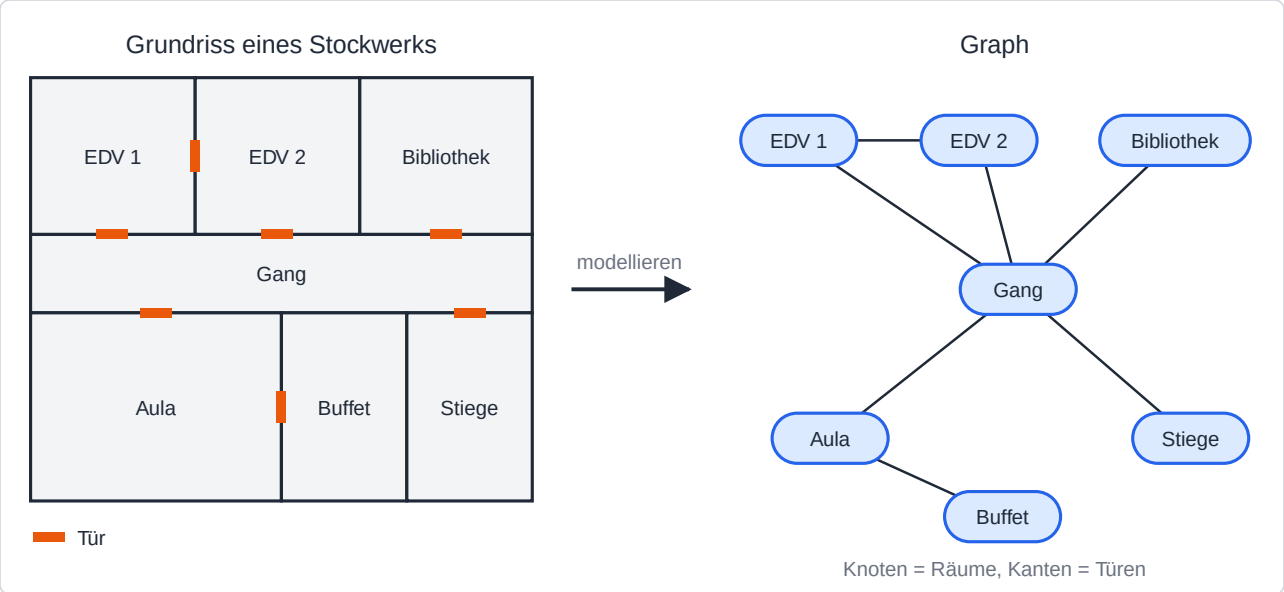
Ein **Baum** ist ein zusammenhängender Graph ohne Zyklus. Daraus folgt:

- Zwischen zwei Knoten gibt es genau einen Pfad (ohne Umwege).
- Ein Baum mit n Knoten hat genau n − 1 Kanten. Nimmst du eine Kante weg, zerfällt er in zwei Teile. Fügst du eine hinzu, entsteht ein Zyklus.

Oft wählt man einen Knoten als **Wurzel** und zeichnet den Baum von oben nach unten. Dann hat jeder Knoten außer der Wurzel genau einen *Elternknoten*. Knoten ohne Kinder heißen *Blätter*. Bäume kennst du schon: die Ordnerstruktur auf deiner Festplatte, das DOM einer Webseite oder den AST aus dem Kapitel Parser.

Probleme als Graph modellieren

Das Modellieren ist der wichtigste Schritt. Du musst dir zwei Fragen beantworten: *Was sind die Knoten?* und *Wann gibt es eine Kante?* Dazu kommt: Ist die Beziehung gerichtet, und brauche ich Gewichte?



Im Beispiel soll ein Programm Besucher durch ein Stockwerk führen. Jeder Raum wird ein Knoten, jede Tür eine Kante. Weil man durch eine Tür in beide Richtungen gehen kann, ist der Graph ungerichtet. Alles andere lässt das Modell weg: wie groß die Räume sind, wo die Tische stehen, welche Farbe die Wände haben. Am Graphen siehst du sofort, was am Grundriss nicht so leicht zu erkennen ist: Ins Buffet kommst du nur durch die Aula, und der Gang ist der zentrale Knoten mit dem höchsten Grad.

Anwendung	Knoten	Kanten	Art
Straßennetz	Kreuzungen, Orte	Straßenabschnitte	gewichtet (km oder Minuten), gerichtet bei Einbahnen
Öffentlicher Verkehr	Haltestellen	direkte Fahrt zur nächsten Haltestelle	gewichtet (Minuten)
Abhängigkeiten	Pakete, Projekte, Lehrveranstaltungen	„braucht“	gerichtet, sollte ein DAG sein

Anwendung	Knoten	Kanten	Art
Soziales Netz	Personen	Freundschaft bzw. „folgt“	ungerichtet bzw. gerichtet
Web	Seiten	Links	gerichtet
Labyrinth, Spielfeld	Felder	Nachbarfeld ohne Mauer	ungerichtet
Spiel, Automat	Zustände	Züge, Übergänge	gerichtet

Dieselbe Situation kann je nach Frage unterschiedlich modelliert werden. Willst du beim öffentlichen Verkehr die Umstiege zählen, reicht ein Knoten pro Haltestelle nicht mehr. Dann brauchst du zum Beispiel einen Knoten pro Haltestelle *und* Linie und eigene Kanten für das Umsteigen.

Graphen speichern

Für das Programm brauchst du eine Datenstruktur. Es gibt drei übliche Darstellungen. Die Grafik zeigt einen ungerichteten Graphen mit 5 Knoten und 6 Kanten als Adjazenzmatrix und als Adjazenzliste. Die Kante B–D ist hervorgehoben.



Adjazenzmatrix

Die Knoten werden durchnummeriert (A = 0, B = 1, …). Eine n × n-Matrix enthält in Zeile i und Spalte j eine 1, wenn es eine Kante von Knoten i zu Knoten j gibt. Bei einem ungerichteten Graphen ist die Matrix symmetrisch zur Hauptdiagonale. Bei einem gewichteten Graphen steht statt der 1 das Gewicht in der Zelle.

```
string[] names = ["A", "B", "C", "D", "E"];
int[,] matrix =
{
    // A  B  C  D  E
    { 0, 1, 1, 0, 0 },  // A
    { 1, 0, 1, 1, 0 },  // B
    { 1, 1, 0, 0, 1 },  // C
```

```
    { 0, 1, 0, 0, 1 },  // D
    { 0, 0, 1, 1, 0 },  // E
};

int b = Array.IndexOf(names, "B");
int d = Array.IndexOf(names, "D");
Console.WriteLine($"Kante B-D: {matrix[b, d] == 1}");
Console.WriteLine($"Kante A-E: {matrix[0, 4] == 1}");
```

```
Kante B-D: True
Kante A-E: False
```

Ob es eine bestimmte Kante gibt, liest du direkt in einer Zelle ab. Dafür braucht die Matrix immer n · n Zellen, egal wie viele Kanten es gibt. Ein Straßennetz mit 10 000 Kreuzungen hätte 100 Millionen Zellen, obwohl an jeder Kreuzung nur drei oder vier Straßen zusammenkommen. Fast alle Zellen wären 0.

Adjazenzliste

Jeder Knoten bekommt die Liste seiner Nachbarn. In C# passt dafür ein `Dictionary<string, List<string>>`: Der Schlüssel ist der Knoten, der Wert die Liste der Nachbarn.

```
var adjacency = new Dictionary<string, List<string>>
{
    ["A"] = ["B", "C"],
    ["B"] = ["A", "C", "D"],
    ["C"] = ["A", "B", "E"],
    ["D"] = ["B", "E"],
    ["E"] = ["C", "D"],
};

foreach (var (node, neighbors) in adjacency)
{
    Console.WriteLine($"{node}: {string.Join(", ", neighbors)}");
}
Console.WriteLine($"Kante B-D: {adjacency["B"].Contains("D")}");
```

```
A: B, C
B: A, C, D
C: A, B, E
D: B, E
E: C, D
Kante B-D: True
```

Die Liste speichert nur Kanten, die es wirklich gibt. Jede ungerichtete Kante steht zweimal darin: B–D bei B *und* bei D. Die Knoten brauchen keine Nummern, jeder Typ mit sinnvollem `Equals` und `GetHashCode` kann ein Knoten sein. Um alle Nachbarn eines Knotens zu besuchen,

gehst du einfach seine Liste durch. Genau das brauchen die Suchalgorithmen in den nächsten Kapiteln.

Kantenliste

Die einfachste Darstellung ist eine Liste aller Kanten, hier als Tupel. So stehen Graphen oft in Dateien: eine Kante pro Zeile, bei Bedarf mit Gewicht, z. B. als CSV. Zum Arbeiten wandelt man die Kantenliste meist in eine Adjazenzliste um:

```
(string From, string To)[] edges = [("A", "B"), ("A", "C"), ("B", "C"), ("B", "D"),
("C", "E"), ("D", "E")];

var adjacency = new Dictionary<string, List<string>>();
foreach (var (from, to) in edges)
{
    adjacency.TryAdd(from, []);       // (1)
    adjacency.TryAdd(to, []);
    adjacency[from].Add(to);          // (2)
    adjacency[to].Add(from);
}
Console.WriteLine($"{edges.Length} Kanten, {adjacency.Count} Knoten");
Console.WriteLine($"C: {string.Join(", ", adjacency["C"])}");
```

```
6 Kanten, 5 Knoten
C: A, B, E
```

- **(1)** `TryAdd` legt einen Knoten nur an, wenn es ihn noch nicht gibt. Knoten entstehen so automatisch aus den Kanten.
- **(2)** Der Graph ist ungerichtet, also wird jede Kante in beide Richtungen eingetragen. In einem gerichteten Graphen fällt die zweite Zeile weg.

Welche Darstellung wann?

Darstellung	praktisch, wenn …	unpraktisch, wenn …
Adjazenzmatrix	der Graph *dicht* ist (fast jeder Knoten mit fast jedem verbunden), du oft fragst „Gibt es die Kante u–v?“, die Knoten ohnehin Nummern 0 … n−1 haben	der Graph *dünn* ist (wenige Kanten pro Knoten) und viele Knoten hat
Adjazenzliste	du die Nachbarn eines Knotens durchgehen willst (Suchen, Wege), der Graph dünn ist, die Knoten Namen oder Objekte sind	du nur wissen willst, ob es eine bestimmte Kante gibt (dafür musst du die Nachbarliste durchsuchen)

Darstellung	praktisch, wenn …	unpraktisch, wenn …
Kantenliste	du Graphen speicherst, einliest oder ausgibst, oder alle Kanten der Reihe nach brauchst (z. B. nach Gewicht sortiert)	du die Nachbarn eines Knotens brauchst

Die meisten Graphen aus der Praxis sind dünn: Eine Kreuzung hat wenige Straßen, eine Person kennt nicht alle anderen Menschen. Deshalb ist die Adjazenzliste der Normalfall.

Eine Klasse für Graphen

Damit du nicht jedes Mal ein `Dictionary` von Hand befüllst, packen wir die Adjazenzliste in eine generische Klasse `Graph<TNode>`. Der Typparameter `TNode` erlaubt Knoten vom Typ `string`, `int` oder auch eigene Records. Kanten sind ein `record Edge<TNode>` mit Start, Ziel und Gewicht. So kann dieselbe Klasse gerichtete, ungerichtete und gewichtete Graphen speichern.

Der vollständige Code liegt in Beispiele/20_Graph. Öffne die Datei *GraphDemo.sln* oder starte das Programm in der Konsole:

```
cd Beispiele/20_Graph
dotnet run --project GraphDemo
```

Datei	Inhalt
Edge.cs	`record Edge<TNode>`
Graph.cs	die Klasse `Graph<TNode>` mit Adjazenzliste
Program.cs	baut zwei Graphen aus Kantenlisten und gibt sie aus

Die Codeblöcke mit *Auszug aus …* stammen aus diesen Dateien. Die XML-Kommentare sind weggelassen. Einzeln kompilieren die Auszüge nicht, das ganze Projekt schon.

Auszug aus Edge.cs:

```
public record Edge<TNode>(TNode From, TNode To, int Weight = 1)
{
    public override string ToString() => Weight == 1 ? $"{From} -> {To}" : $"{From} ->
{To} ({Weight})";
}
```

Ein ungewichteter Graph ist einfach ein Graph, in dem jede Kante das Gewicht 1 hat. Deshalb ist `Weight` optional.

Auszug aus Graph.cs:

```
public class Graph<TNode>(bool isDirected) where TNode : notnull      // (1)
```

```
{
    private readonly Dictionary<TNode, List<Edge<TNode>>> _adjacency = [];   // (2)

    public bool IsDirected => isDirected;
    public IEnumerable<TNode> Nodes => _adjacency.Keys;
    public int NodeCount => _adjacency.Count;

    public void AddNode(TNode node) => _adjacency.TryAdd(node, []);

    public void AddEdge(TNode from, TNode to, int weight = 1)
    {
        AddNode(from);
        AddNode(to);
        _adjacency[from].Add(new Edge<TNode>(from, to, weight));
        if (!isDirected)
        {
            _adjacency[to].Add(new Edge<TNode>(to, from, weight));       // (3)
        }
    }

    public IReadOnlyList<Edge<TNode>> EdgesFrom(TNode node) =>
        _adjacency.TryGetValue(node, out List<Edge<TNode>>? edges)
            ? edges
            : throw new KeyNotFoundException($"Unknown node {node}.");

    public IEnumerable<TNode> Neighbors(TNode node) => EdgesFrom(node).Select(e => e.
To);   // (4)

    public bool HasEdge(TNode from, TNode to) =>
        _adjacency.TryGetValue(from, out List<Edge<TNode>>? edges) && edges.Any(e => e
.To.Equals(to));

    public static Graph<TNode> FromEdgeList(IEnumerable<Edge<TNode>> edges, bool
isDirected)   // (5)
    {
        var graph = new Graph<TNode>(isDirected);
        foreach (Edge<TNode> edge in edges)
        {
            graph.AddEdge(edge.From, edge.To, edge.Weight);
        }
        return graph;
    }
}
```

- **(1)** Der Primary Constructor bekommt nur die Information, ob der Graph gerichtet ist. Die Einschränkung `where TNode : notnull` ist nötig, weil `TNode` als Schlüssel im `Dictionary` dient, und Schlüssel dürfen nicht `null` sein.
- **(2)** Die Adjazenzliste speichert pro Knoten nicht nur die Nachbarn, sondern ganze Kanten. So bleibt das Gewicht erhalten.
- **(3)** Ungerichtete Kanten werden in beide Richtungen gespeichert, wie bei der Kantenliste oben. Wer die Klasse benutzt, muss sich darum nicht kümmern.
- **(4)** `Neighbors` liefert nur die Zielknoten, `EdgesFrom` die Kanten samt Gewicht. Die Nachbarn

kommen in der Reihenfolge, in der die Kanten eingefügt wurden.

- **(5)** Eine statische *Factory-Methode* baut den Graphen aus einer Kantenliste.

Program.cs liest die Kantenliste aus einem Text mit einer Kante pro Zeile. Die Hilfsmethode `ParseEdges` bekommt eine Funktion, die einen Knotennamen in den Knotentyp umwandelt. So funktioniert sie für `string` und `int`.

Auszug aus Program.cs:

```
string exampleEdges = """
    A B
    A C
    B C
    B D
    C E
    D E
    """;
Graph<string> graph = Graph<string>.FromEdgeList(ParseEdges(exampleEdges, name =>
name), isDirected: false);

// ...

string streetEdges = """
    1 2 120
    2 3 90
    3 1 150
    2 4 60
    4 3 70
    """;
Graph<int> streets = Graph<int>.FromEdgeList(ParseEdges(streetEdges, int.Parse),
isDirected: true);

// ...

static IEnumerable<Edge<TNode>> ParseEdges<TNode>(string text, Func<string, TNode>
parseNode)
{
    foreach (string line in text.Split('\n', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries))
    {
        string[] parts = line.Split(' ', StringSplitOptions.RemoveEmptyEntries);
        int weight = parts.Length > 2 ? int.Parse(parts[2]) : 1;
        yield return new Edge<TNode>(parseNode(parts[0]), parseNode(parts[1]), weight);
    }
}
```

Der zweite Graph beschreibt Kreuzungen (Nummern 1 bis 4) mit Einbahnstraßen und ihrer Länge in Metern. Die Ausgabe des Programms:

```
Ungerichteter Graph mit 5 Knoten:
  A: B, C
  B: A, C, D
```

```
  C: A, B, E
  D: B, E
  E: C, D
  Kante B-D? True   Kante A-E? False

Gerichteter, gewichteter Graph mit 4 Knoten:
  1: 1 -> 2 (120)
  2: 2 -> 3 (90), 2 -> 4 (60)
  3: 3 -> 1 (150)
  4: 4 -> 3 (70)
  Kante 1->2? True   Kante 2->1? False
```

Beim gerichteten Graphen gibt es die Kante 1 → 2, aber nicht 2 → 1. Von Kreuzung 2 kommst du trotzdem zu Kreuzung 1 zurück, nur eben über einen Umweg: 2 → 3 → 1. Solche Umwege systematisch zu finden, ist die Aufgabe der Suchalgorithmen im nächsten Kapitel.

Breiten- und Tiefensuche

Sobald ein Problem als Graph vorliegt, lautet die erste Frage oft: Welche Knoten kann ich von hier aus erreichen, und auf welchem Weg? Komme ich durch das Labyrinth zum Ausgang? Sind alle Rechner im Netzwerk miteinander verbunden? Wie viele Umstiege brauche ich mindestens? Für solche Fragen gibt es zwei grundlegende Verfahren: die **Breitensuche** (*Breadth-First Search*, BFS) und die **Tiefensuche** (*Depth-First Search*, DFS). Beide besuchen jeden erreichbaren Knoten genau einmal. Sie unterscheiden sich nur in der Reihenfolge.



Zum Selbst-Erarbeiten: der interaktive Trainer

Im Trainer für Breiten- und Tiefensuche siehst du beide Suchen als Animation, Schritt für Schritt mit Queue und Stack. Danach suchst du selbst und bekommst bei jedem Schritt sofort Rückmeldung. So merkst du schnell, ob du das Verfahren wirklich verstanden hast. Ideal zum Üben vor einem Test!

Das Beispiel und die Spielregeln

Alle Beispiele verwenden diesen ungerichteten Graphen mit 8 Knoten. Wie in Graphen speichern wir ihn als Adjazenzliste:

```
A: C, E          E: A, D, F
B: C, G          F: C, E, G
C: A, B, F       G: B, F
D: E, H          H: D
```

Beide Suchen müssen bei jedem Knoten entscheiden, in welcher Reihenfolge sie die Nachbarn betrachten. Damit alle dasselbe Ergebnis bekommen, gilt hier: **Die Nachbarn werden aufsteigend nach ihrem Namen betrachtet.** Die Listen oben sind schon so sortiert.

Zwei Begriffe brauchst du in diesem Kapitel:

- Ein Knoten ist **entdeckt** (oder *offen*), sobald die Suche ihn als Nachbarn gesehen und vorgemerkt hat.
- Ein Knoten ist **besucht**, wenn die Suche ihn aus ihrem Speicher herausnimmt und seine Nachbarn betrachtet. Die *Besuchsreihenfolge* ist die Reihenfolge, in der das passiert.

Breitensuche (BFS)

Die Breitensuche breitet sich aus wie eine Welle: zuerst der Startknoten, dann alle seine Nachbarn, dann deren Nachbarn und so weiter. Damit das klappt, merkt sie sich die entdeckten Knoten in einer **Queue** (Warteschlange). Was zuerst hineinkommt, kommt zuerst wieder heraus (*FIFO, first in, first out*), genau wie an der Supermarktkasse.

1. Lege den Startknoten in die Queue und markiere ihn als entdeckt.

2. Solange die Queue nicht leer ist: Nimm den vordersten Knoten heraus und besuche ihn. Hänge jeden Nachbarn, der noch nicht entdeckt ist, hinten an die Queue und markiere ihn als entdeckt.

So läuft die BFS ab Knoten A ab:

```
Schritt  besucht  neu entdeckt   Queue danach (vorne ... hinten)
1        A        C, E           C E
2        C        B, F           E B F        A ist schon entdeckt
3        E        D              B F D        A und F sind schon entdeckt
4        B        G              F D G
5        F        –              D G          C, E und G sind schon entdeckt
6        D        H              G H
7        G        –              H
8        H        –              (leer)
```

Die Besuchsreihenfolge ist **A C E B F D G H**. Beachte Schritt 3: F ist zu diesem Zeitpunkt schon in der Queue, weil C ihn entdeckt hat. Deshalb wird F nicht ein zweites Mal angehängt. Ohne die Markierung „entdeckt“ würde die Suche Knoten mehrfach besuchen und bei Zyklen sogar endlos laufen.

In C# gibt es für die Queue die Klasse `Queue<T>`, für die Markierung nimmst du ein `HashSet<T>`:

```
Dictionary<string, string[]> graph = new()
{
    ["A"] = ["C", "E"],
    ["B"] = ["C", "G"],
    ["C"] = ["A", "B", "F"],
    ["D"] = ["E", "H"],
    ["E"] = ["A", "D", "F"],
    ["F"] = ["C", "E", "G"],
    ["G"] = ["B", "F"],
    ["H"] = ["D"],
};

Console.WriteLine($"BFS: {string.Join(" ", BreadthFirst(graph, "A"))}");

static List<string> BreadthFirst(Dictionary<string, string[]> graph, string start)
{
    var order = new List<string>();
    var discovered = new HashSet<string> { start };     // (1)
    var queue = new Queue<string>();
    queue.Enqueue(start);
    while (queue.Count > 0)
    {
        string node = queue.Dequeue();                  // (2)
        order.Add(node);
        foreach (string neighbor in graph[node])        // (3)
        {
            if (discovered.Add(neighbor))               // (4)
            {
```

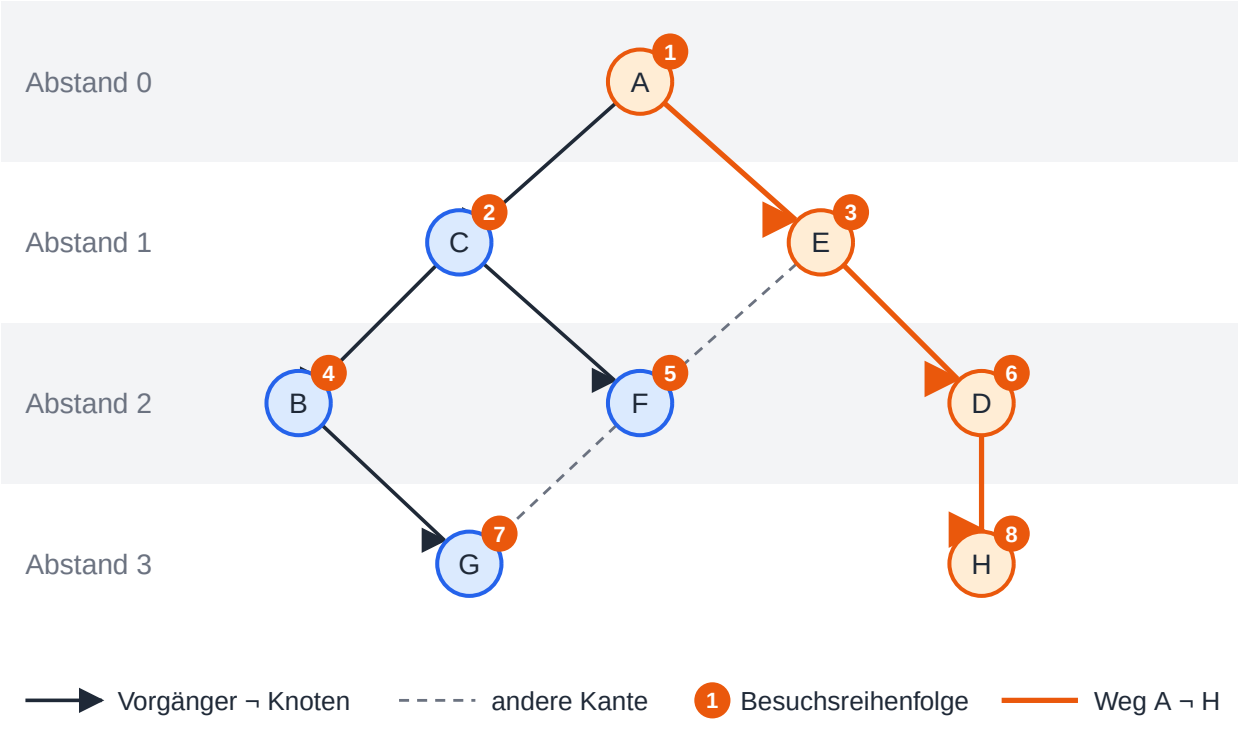
```
                queue.Enqueue(neighbor);
            }
        }
    }
    return order;
}
```

```
BFS: A C E B F D G H
```

- **(1)** Der Startknoten ist von Anfang an entdeckt.
- **(2)** Der vorderste Knoten wird besucht.
- **(3)** Die Nachbarn kommen in der Reihenfolge des Arrays, also aufsteigend nach Namen.
- **(4)** `HashSet.Add` liefert `false`, wenn das Element schon enthalten ist. Damit prüft und markiert eine einzige Zeile.

Ebenen und Abstände

Die Welle der Breitensuche hat eine wichtige Eigenschaft: Sie besucht zuerst alle Knoten mit Abstand 1 vom Start, dann alle mit Abstand 2 und so weiter. Der **Abstand** ist dabei die kleinste Anzahl von Kanten auf einem Weg vom Start zum Knoten.



Merkt sich die BFS bei jedem neu entdeckten Knoten, von welchem Knoten aus sie ihn entdeckt hat, entsteht ein Baum aus **Vorgängern** (die Pfeile in der Grafik). Folgst du den Vorgängern von einem Knoten rückwärts bis zum Start, bekommst du einen kürzesten Weg: H ← D ← E ← A. Umgedreht ist das der Weg **A** → **E** → **D** → **H** mit 3 Kanten. Einen Weg mit weniger Kanten gibt es

nicht, denn H liegt auf Ebene 3.

Für die Vorgänger nimmst du ein `Dictionary`. Es ersetzt gleichzeitig das `HashSet`, denn jeder entdeckte Knoten steht darin. Füge diese Methode im Programm von oben ein (die Aufrufe gehören zu den Top-Level-Statements, die Methode darunter):

```
Console.WriteLine($"A -> H: {string.Join(" ", ShortestPath(graph, "A", "H") ?? [])}");
Console.WriteLine($"A -> G: {string.Join(" ", ShortestPath(graph, "A", "G") ?? [])}");

static List<string>? ShortestPath(Dictionary<string, string[]> graph, string start,
string target)
{
    var previous = new Dictionary<string, string?> { [start] = null };   // (1)
    var queue = new Queue<string>();
    queue.Enqueue(start);
    while (queue.Count > 0)
    {
        string node = queue.Dequeue();
        if (node == target) { break; }                                    // (2)
        foreach (string neighbor in graph[node])
        {
            if (previous.TryAdd(neighbor, node))                          // (3)
            {
                queue.Enqueue(neighbor);
            }
        }
    }
    if (!previous.ContainsKey(target)) { return null; }                   // (4)

    var path = new List<string>();
    for (string? node = target; node is not null; node = previous[node])  // (5)
    {
        path.Add(node);
    }
    path.Reverse();
    return path;
}
```

```
A -> H: A E D H
A -> G: A C B G
```

- **(1)** Der Startknoten hat keinen Vorgänger, deshalb `null`.
- **(2)** Ist das Ziel erreicht, kann die Suche aufhören. Der Weg dorthin steht schon fest.
- **(3)** `TryAdd` trägt den Vorgänger nur ein, wenn der Nachbar noch nicht entdeckt ist.
- **(4)** Wurde das Ziel nie entdeckt, gibt es keinen Weg.
- **(5)** Die Schleife folgt den Vorgängern vom Ziel bis zum Start. Danach steht der Weg verkehrt herum in der Liste und wird umgedreht.

Die Länge des Weges in Kanten ist `path.Count - 1`. Das ist der Abstand des Ziels vom Start.

Tiefensuche (DFS)

Die Tiefensuche geht den umgekehrten Weg: Sie folgt einem Pfad so weit wie möglich. Erst wenn es nicht mehr weitergeht, kehrt sie um und probiert die nächste Abzweigung. So würdest du ein Labyrinth erkunden, wenn du immer an der Wand entlanggehst. Statt einer Queue verwendet die Tiefensuche einen **Stack** (Stapel). Was zuletzt hineinkommt, kommt zuerst heraus (*LIFO, last in, first out*).

Es gibt mehrere Varianten der DFS, die sich in Details unterscheiden. Hier gilt diese:

1. Lege den Startknoten auf den Stack.
2. Solange der Stack nicht leer ist: Nimm den obersten Knoten herunter. Ist er schon besucht, überspringe ihn. Sonst besuche ihn und lege alle noch nicht besuchten Nachbarn auf den Stack, und zwar in **absteigender** Reihenfolge.

Warum absteigend? Der zuletzt hinaufgelegte Knoten wird als Erstes wieder heruntergenommen. Legst du die Nachbarn absteigend hinauf, liegt der Nachbar mit dem kleinsten Namen oben und wird als Nächstes besucht.

```
Schritt  herunter  besucht                    Stack danach (unten ... oben)
1        A         A                          E C
2        C         C                          E F B
3        B         B                          E F G
4        G         G                          E F F
5        F         F                          E F E
6        E         E                          E F D
7        D         D                          E F H
8        H         H                          E F
9        F         schon besucht, überspringen E
10       E         schon besucht, überspringen (leer)
```

Die Besuchsreihenfolge ist **A C B G F E D H**. Ein Knoten kann mehrmals auf dem Stack liegen: F wurde von C und von G hinaufgelegt, als er noch nicht besucht war. Deshalb wird beim Herunternehmen noch einmal geprüft.

In C# verwendest du `Stack<T>`. Füge auch diese Methode im Programm von oben ein:

```
Console.WriteLine($"DFS: {string.Join(" ", DepthFirst(graph, "A"))}");

static List<string> DepthFirst(Dictionary<string, string[]> graph, string start)
{
    var order = new List<string>();
    var visited = new HashSet<string>();
    var stack = new Stack<string>();
    stack.Push(start);
    while (stack.Count > 0)
    {
        string node = stack.Pop();
        if (!visited.Add(node)) { continue; }              // (1)
```

```
        order.Add(node);
        string[] neighbors = graph[node];
        for (int i = neighbors.Length - 1; i >= 0; i--)     // (2)
        {
            if (!visited.Contains(neighbors[i]))
            {
                stack.Push(neighbors[i]);
            }
        }
    }
    return order;
}
```

```
DFS: A C B G F E D H
```

- **(1)** Liegt ein Knoten doppelt auf dem Stack, wird er beim zweiten Mal übersprungen.
- **(2)** Die Schleife läuft rückwärts durch die Nachbarn, damit der kleinste Nachbar oben liegt.

Rekursive Tiefensuche

Die Tiefensuche lässt sich auch rekursiv schreiben. Dann übernimmt der Aufrufstack die Rolle des `Stack<T>`: Jeder Aufruf besucht einen Knoten und ruft sich für jeden noch nicht besuchten Nachbarn selbst auf. Ist ein Aufruf fertig, geht es beim Aufrufer mit dem nächsten Nachbarn weiter. Das ist das „Umkehren und die nächste Abzweigung probieren“.

```
var visited = new HashSet<string>();
var recursiveOrder = new List<string>();
DepthFirstRecursive(graph, "A", visited, recursiveOrder);
Console.WriteLine($"DFS rekursiv: {string.Join(" ", recursiveOrder)}");

static void DepthFirstRecursive(Dictionary<string, string[]> graph, string node,
    HashSet<string> visited, List<string> order)
{
    visited.Add(node);
    order.Add(node);
    foreach (string neighbor in graph[node])                // (1)
    {
        if (!visited.Contains(neighbor))
        {
            DepthFirstRecursive(graph, neighbor, visited, order);
        }
    }
}
```

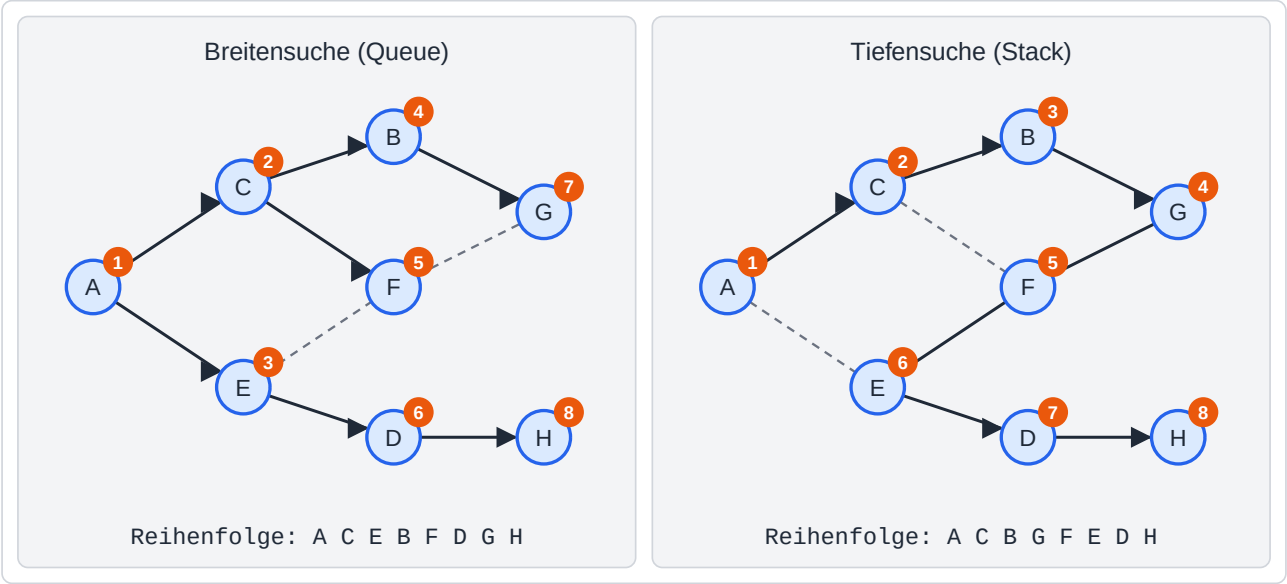
```
DFS rekursiv: A C B G F E D H
```

- **(1)** Hier laufen die Nachbarn aufsteigend durch. Der erste noch nicht besuchte Nachbar wird sofort besucht, alle weiteren erst, wenn dieser Aufruf zurückkommt.

Beide Varianten liefern dieselbe Reihenfolge. Das ist kein Zufall, sondern der Grund für die Regeln oben: Weil die iterative Variante erst beim Herunternehmen markiert und die Nachbarn absteigend hinauflegt, verhält sich ihr Stack genau wie der Aufrufstack der Rekursion. Würdest du dagegen schon beim Hinauflegen markieren (wie bei der BFS), käme eine andere Reihenfolge heraus.

Die rekursive Variante ist kürzer, hat aber eine Grenze: Jeder Aufruf belegt Platz auf dem Aufrufstack, und der ist begrenzt. Bei einem Graphen, in dem ein Pfad über zehntausende Knoten führt, bricht das Programm mit einer `StackOverflowException` ab. Die Variante mit `Stack<T>` hat dieses Problem nicht.

Unterschiede in der Besuchsreihenfolge



Die Grafik zeigt beide Suchen nebeneinander. Die Pfeile führen jeweils vom Knoten, über den ein Knoten erreicht wurde, zu diesem Knoten. Die BFS erzeugt einen breiten, flachen Baum: Jeder Knoten wird über einen Weg mit möglichst wenigen Kanten erreicht. Die DFS erzeugt hier einen einzigen langen Pfad A → C → B → G → F → E → D → H. Sie erreicht E erst als sechsten Knoten, obwohl E direkt neben A liegt.

	Breitensuche (BFS)	**Tiefensuche (DFS)**
Speicher für offene Knoten	Queue (FIFO)	Stack (LIFO) oder Rekursion
Reihenfolge	Ebene für Ebene, nach Abstand vom Start	einem Pfad folgen, dann zurück zur letzten Abzweigung
liefert	kürzeste Wege in Kanten, Abstände	Pfade bis in die Tiefe, Rücksprünge
typische Anwendungen	wenigste Umstiege, kürzester Weg im Labyrinth, „Freunde von Freunden“ bis Stufe 2	Erreichbarkeit, Labyrinth erzeugen, Zyklen finden, Topologische Sortierung, Backtracking

Willst du nur wissen, **ob** ein Knoten erreichbar ist oder welche Knoten zusammenhängen, sind

beide Suchen gleich gut. Brauchst du einen Weg mit möglichst **wenigen Kanten**, nimm die BFS. Der Weg, den die DFS findet, ist oft ein Umweg (im Beispiel A → C → B → G → F → E für das Ziel E).

Zusammenhangskomponenten

Eine Suche findet alle Knoten, die vom Start aus erreichbar sind, also genau eine Zusammenhangskomponente. Um alle Komponenten zu finden, startest du nacheinander bei jedem Knoten, der noch in keiner Komponente liegt. Das folgende Netzwerk besteht aus 8 Rechnern und ihren Kabeln. Auch diese Zeilen kannst du im Programm von oben einfügen:

```
Dictionary<string, string[]> network = new()
{
    ["A"] = ["B"],
    ["B"] = ["A", "C"],
    ["C"] = ["B"],
    ["D"] = ["F"],
    ["E"] = ["G"],
    ["F"] = ["D"],
    ["G"] = ["E"],
    ["H"] = [],
};

var assigned = new HashSet<string>();
int componentCount = 0;
foreach (string node in network.Keys.Order())
{
    if (assigned.Contains(node)) { continue; }             // (1)
    List<string> component = BreadthFirst(network, node);  // (2)
    assigned.UnionWith(component);
    componentCount++;
    Console.WriteLine($"Komponente {componentCount}: {string.Join(", ", component.
Order())}");
}
```

```
Komponente 1: A, B, C
Komponente 2: D, F
Komponente 3: E, G
Komponente 4: H
```

- **(1)** Knoten, die schon zu einer Komponente gehören, sind kein neuer Startpunkt.
- **(2)** Statt `BreadthFirst` könntest du genauso `DepthFirst` verwenden. Die Komponenten sind dieselben, nur die Reihenfolge innerhalb einer Komponente ist anders.

Ein Graph ist genau dann zusammenhängend, wenn es nur eine Komponente gibt, wenn also eine einzige Suche alle Knoten besucht.

Labyrinth als impliziter Graph

Nicht jeder Graph muss als `Dictionary` gespeichert sein. Ein Labyrinth aus Feldern ist schon ein Graph: Jedes freie Feld ist ein Knoten, und zwei Felder sind benachbart, wenn sie oben, rechts, unten oder links nebeneinanderliegen und keines davon eine Mauer ist. Die Nachbarn eines Feldes berechnest du erst, wenn die Suche sie braucht. Man spricht von einem **impliziten Graphen**.

```
string[] maze =
[
    "#############",
    "#S....#.....#",
    "#.###.#.###.#",
    "#...#...#...#",
    "###.#####.#.#",
    "#...#.....#.#",
    "#.###.#####.#",
    "#.....#....Z#",
    "#############",
];

Console.WriteLine($"Nachbarn von (1, 1): {string.Join(" ", Neighbors(maze, (1, 1)))}");
Console.WriteLine($"Nachbarn von (3, 5): {string.Join(" ", Neighbors(maze, (3, 5)))}");

static IEnumerable<(int Row, int Col)> Neighbors(string[] maze, (int Row, int Col)
cell)
{
    (int Row, int Col)[] directions = [(-1, 0), (0, 1), (1, 0), (0, -1)];   // (1)
    foreach (var (dRow, dCol) in directions)
    {
        int row = cell.Row + dRow;
        int col = cell.Col + dCol;
        if (row >= 0 && row < maze.Length && col >= 0 && col < maze[row].Length   //
(2)
            && maze[row][col] != '#')
        {
            yield return (row, col);
        }
    }
}
```

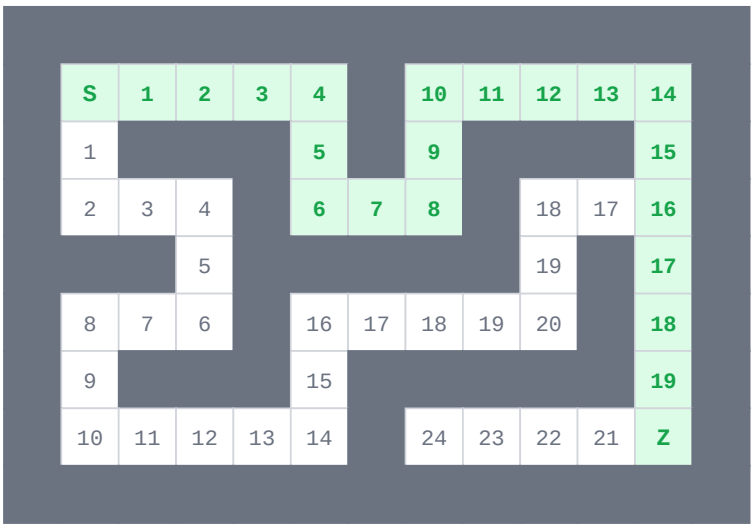
```
Nachbarn von (1, 1): (1, 2) (2, 1)
Nachbarn von (3, 5): (2, 5) (3, 6)
```

- **(1)** Die Richtungen oben, rechts, unten, links als Änderung von Zeile und Spalte. Ihre Reihenfolge legt fest, in welcher Reihenfolge die Nachbarn betrachtet werden.
- **(2)** Felder außerhalb des Labyrinths und Mauern sind keine Nachbarn.

Mit dieser Methode statt `graph[node]` und `(int Row, int Col)` statt `string` als Knotentyp funktionieren BFS und DFS unverändert. Die Grafik zeigt das Ergebnis einer BFS ab S: In jedem Feld

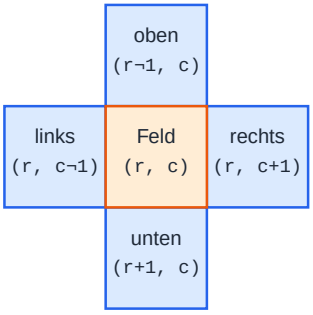
steht der Abstand von S. Die Welle läuft gleichzeitig in alle Gänge, auch in die Sackgassen. Z hat den Abstand 20, und über die Vorgänger findet man den grünen Weg.

Abstand jedes Feldes von S (Breitensuche)



Kürzester Weg (grün): 20 Schritte

Nachbarn eines Feldes



Ein Feld ist ein Knoten. Kanten führen zu den freien Feldern oben, rechts, unten und links. Mauern (#) und der Rand haben keine Kanten. Der Graph wird nie gespeichert, die Nachbarn werden bei Bedarf berechnet (impliziter Graph).

Flood Fill

Das Füllwerkzeug in einem Malprogramm färbt eine zusammenhängende Fläche gleicher Farbe ein. Auch das ist eine Suche in einem impliziten Graphen: Die Pixel sind die Knoten, Nachbarn sind die angrenzenden Pixel mit derselben Farbe wie der Startpixel. Das Verfahren heißt **Flood Fill**. Hier ist die Reihenfolge egal, deshalb reicht ein Stack:

```
string[] lines =
[
    "...........",
    "..#######...",
    "..#.....#...",
    "..#.###.#...",
    "..#.#.#.#...",
    "..#.###.#...",
    "..#.....#...",
    "..#######...",
];
char[][] picture = [.. lines.Select(line => line.ToCharArray())];

FloodFill(picture, (2, 3), 'o');
foreach (char[] row in picture)
{
    Console.WriteLine(new string(row));
}

static void FloodFill(char[][] picture, (int Row, int Col) start, char color)
{
    char oldColor = picture[start.Row][start.Col];
```

```
    if (oldColor == color) { return; }                    // (1)
    picture[start.Row][start.Col] = color;
    var stack = new Stack<(int Row, int Col)>();
    stack.Push(start);
    (int Row, int Col)[] directions = [(-1, 0), (0, 1), (1, 0), (0, -1)];
    while (stack.Count > 0)
    {
        var (row, col) = stack.Pop();
        foreach (var (dRow, dCol) in directions)
        {
            int r = row + dRow;
            int c = col + dCol;
            if (r >= 0 && r < picture.Length && c >= 0 && c < picture[r].Length
                && picture[r][c] == oldColor)             // (2)
            {
                picture[r][c] = color;                    // (3)
                stack.Push((r, c));
            }
        }
    }
}
```

```
............
..#######...
..#ooooo#...
..#o###o#...
..#o#.#o#...
..#o###o#...
..#ooooo#...
..#######...
```

- **(1)** Hat die Fläche schon die neue Farbe, gibt es nichts zu tun. Ohne diese Prüfung würde die Suche bereits gefärbte Pixel wieder als „alte Farbe“ erkennen und endlos laufen.
- **(2)** Nachbar ist nur, wer im Bild liegt und noch die alte Farbe hat.
- **(3)** Die neue Farbe ist gleichzeitig die Markierung „entdeckt“. Ein eigenes `HashSet` braucht Flood Fill deshalb nicht.

Das Feld in der Mitte bleibt leer, denn es ist ringsum von `#` umgeben und gehört zu einer anderen Komponente.

Zusatzmaterial

Im 21_Breiten_und_Tiefensuche.html siehst du beide Suchen Schritt für Schritt mit Queue und Stack. Danach führst du sie auf Zufallsgraphen oder selbst gezeichneten Graphen selbst aus und bekommst bei jedem Schritt Rückmeldung.

Kürzeste Wege mit Dijkstra

Du willst mit dem Auto von Graz nach Bregenz fahren. Das Navi sucht dir nicht die Route mit den wenigsten Straßenabschnitten, sondern die mit den wenigsten Kilometern (oder Minuten). Im Graphen dazu hat jede Kante ein **Gewicht**. Gesucht ist der Weg, bei dem die Summe der Gewichte am kleinsten ist. Das Standardverfahren dafür hat der niederländische Informatiker Edsger W. Dijkstra 1956 erfunden. Es steckt bis heute in Navigationssystemen, in Routern im Internet und in Spielen, in denen Figuren den Weg über eine Karte finden.



Zum Selbst-Erarbeiten: der interaktive Dijkstra-Trainer

Im Dijkstra-Trainer siehst du den Algorithmus mit Distanztabelle und Warteschlange als Animation. Danach führst du ihn selbst Schritt für Schritt aus, auch auf Zufallsgraphen, und bekommst bei jedem Schritt Rückmeldung. So hast du beliebig viele Übungsaufgaben.

Gewichtete Graphen

In einem **gewichteten Graphen** trägt jede Kante eine Zahl, ihr *Gewicht*. Was die Zahl bedeutet, hängt von der Anwendung ab: Kilometer, Fahrzeit in Minuten, Kosten in Euro oder Energie in kWh.

- Die **Länge eines Weges** ist die Summe der Gewichte seiner Kanten. Die Anzahl der Kanten spielt keine Rolle.
- Ein **kürzester Weg** von s nach v ist ein Weg mit der kleinsten Länge. Es kann mehrere gleich lange geben.
- Die **Distanz** d(v) ist die Länge eines kürzesten Weges vom Startknoten s zu v. Es gilt d(s) = 0. Ist v gar nicht erreichbar, schreibt man d(v) = ∞.

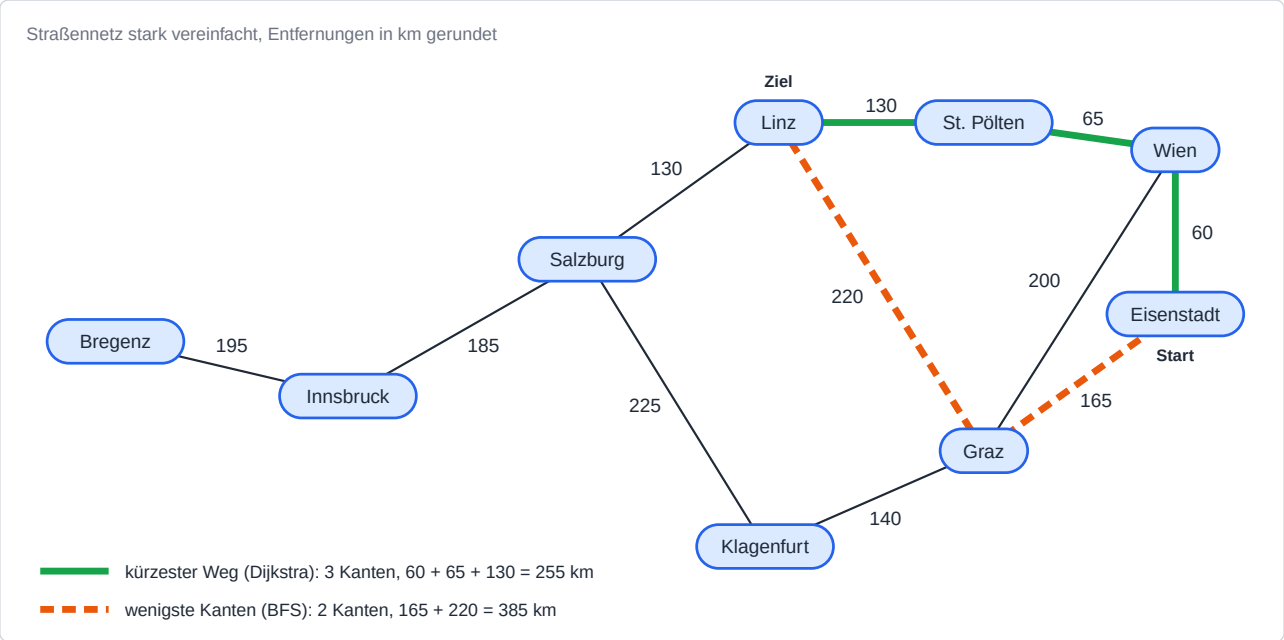
Als Beispiel dient in dieser Einheit ein Straßennetz zwischen den österreichischen Landeshauptstädten. Die Kilometer sind gerundet und das Netz ist **stark vereinfacht**: Es enthält nur einige Hauptverbindungen, echte Routen können anders verlaufen. Jede Straße ist in beide Richtungen befahrbar, der Graph ist also ungerichtet.

Von	Nach	km
Bregenz	Innsbruck	195
Innsbruck	Salzburg	185
Salzburg	Linz	130
Salzburg	Klagenfurt	225
Linz	St. Pölten	130
Linz	Graz	220
St. Pölten	Wien	65
Wien	Eisenstadt	60
Wien	Graz	200

Von	Nach	km
Eisenstadt	Graz	165
Graz	Klagenfurt	140

Warum die Breitensuche hier nicht reicht

Die Breitensuche (BFS) findet den Weg mit den **wenigsten Kanten**. Das ist nur dann der kürzeste Weg, wenn alle Kanten gleich lang sind. Im Straßennetz geht das schief. Suchst du mit BFS einen Weg von Eisenstadt nach Linz, bekommst du Eisenstadt → Graz → Linz. Das sind nur zwei Kanten, aber 385 km. Über Wien und St. Pölten sind es drei Kanten, aber nur 255 km.



Man kann sich Dijkstra wie eine Breitensuche vorstellen, die nicht Kante für Kante, sondern Kilometer für Kilometer wächst. Stell dir vor, in Eisenstadt startet eine Welle, die sich auf allen Straßen gleich schnell ausbreitet. Die Städte werden in der Reihenfolge erreicht, in der die Welle dort ankommt: Wien nach 60 km, St. Pölten nach 125 km, Graz nach 165 km und Linz nach 255 km. Die Reihenfolge, in der die Welle ankommt, ist genau die Reihenfolge der Distanzen.

Die Idee des Algorithmus

Dijkstra führt eine **Distanztabelle**. Darin steht für jeden Knoten die bisher beste bekannte Distanz und der **Vorgänger** auf diesem Weg. Am Anfang hat der Start die Distanz 0 und alle anderen Knoten ∞. Jeder Knoten ist entweder *offen* oder *fertig*. Fertig heißt: Die Distanz in der Tabelle ist endgültig.

1. Wähle unter den offenen Knoten den mit der **kleinsten Distanz**. Nenne ihn u. Er ist ab jetzt fertig.
2. Prüfe jeden Nachbarn v von u, der noch offen ist. Ist d(u) + Gewicht(u, v) **kleiner** als d(v), hast du einen besseren Weg gefunden: Setze d(v) auf den neuen Wert und u als Vorgänger von v. Diesen Schritt nennt man **Relaxieren** der Kante (u, v).

3. Wiederhole, bis kein offener Knoten mit einer Distanz kleiner als ∞ übrig ist.

Warum darf u in Schritt 1 sofort fertig sein? Jeder andere Weg zu u müsste über einen anderen offenen Knoten führen. Der ist aber schon mindestens so weit entfernt wie u, und jede weitere Kante macht den Weg nur länger, denn **Gewichte sind nicht negativ**. Diese Voraussetzung ist der Kern des Algorithmus. Am Ende dieser Einheit siehst du, was ohne sie passiert.

Dijkstra per Hand

Wir berechnen die Distanzen ab **Graz**. In der Tabelle steht jede Zeile für einen Durchlauf. Die Spalten sind schon in der Reihenfolge sortiert, in der die Städte fertig werden.

Schritt	entnommen	Graz	Klagenfurt	Eisenstadt	Wien	Linz	St. Pölten	Salzburg	Innsbruck	Bregenz
Start	–	0	¬	¬	¬	¬	¬	¬	¬	¬
1	Graz (0)	0	140	165	200	220	¬	¬	¬	¬
2	Klagenfurt (140)	0	140	165	200	220	¬	365	¬	¬
3	Eisenstadt (165)	0	140	165	200	220	¬	365	¬	¬
4	Wien (200)	0	140	165	200	220	265	365	¬	¬
5	Linz (220)	0	140	165	200	220	265	350	¬	¬
6	St. Pölten (265)	0	140	165	200	220	265	350	¬	¬
7	Salzburg (350)	0	140	165	200	220	265	350	535	¬
8	Salzburg (365)	*veralteter Eintrag: 365 > 350 in der Tabelle, wird übersprungen*								
9	Innsbruck (535)	0	140	165	200	220	265	350	535	730
10	Bregenz (730)	0	140	165	200	220	265	350	535	730
Vorgänger		–	Graz	Graz	Graz	Graz	Wien	Linz	Salzburg	Innsbruck

wird in diesem Schritt fertig · neue oder verbesserte Distanz · schon fertig · veralteter Eintrag

Die interessanten Stellen:

- **Schritt 1:** Graz ist fertig (0 km). Alle vier Nachbarn bekommen eine Distanz: Klagenfurt 140, Eisenstadt 165, Wien 200, Linz 220.
- **Schritt 2:** Klagenfurt hat die kleinste Distanz. Von dort erreichst du Salzburg mit 140 + 225 = 365 km.
- **Schritt 3:** Von Eisenstadt käme man mit 165 + 60 = 225 km nach Wien. Wien hat aber schon 200. Der neue Weg ist nicht besser, die Tabelle bleibt gleich.
- **Schritt 5:** Über Linz erreichst du Salzburg mit 220 + 130 = 350 km. Das ist besser als 365. Salzburg bekommt die Distanz 350 und den Vorgänger Linz.
- **Schritt 8:** Hier taucht Salzburg ein zweites Mal auf, mit dem alten Wert 365. Warum das passiert, klärt der nächste Abschnitt.

Beachte: Die fertigen Distanzen kommen in aufsteigender Reihenfolge, genau wie bei der Welle. Ein Knoten, der fertig ist, ändert sich nie wieder.

Die Prioritätswarteschlange in .NET

In Schritt 1 musst du immer den offenen Knoten mit der kleinsten Distanz finden. Dafür gibt es in .NET die Klasse `PriorityQueue<TElement, TPriority>`. Jeder Eintrag besteht aus einem Element

und einer Priorität. `Dequeue` und `TryDequeue` liefern immer den Eintrag mit der **kleinsten** Priorität, egal in welcher Reihenfolge du die Einträge hineingelegt hast.

```
PriorityQueue<string, int> queue = new();
queue.Enqueue("Salzburg", 365);
queue.Enqueue("Wien", 200);
queue.Enqueue("Salzburg", 350);
Console.WriteLine($"Einträge: {queue.Count}");

while (queue.TryDequeue(out string? city, out int km))
{
    Console.WriteLine($"{city} {km}");
}
```

```
Einträge: 3
Wien 200
Salzburg 350
Salzburg 365
```

Zwei Eigenschaften musst du kennen:

- Haben zwei Einträge dieselbe Priorität, ist **nicht festgelegt**, welcher zuerst kommt. Brauchst du eine feste Reihenfolge, musst du sie selbst in die Priorität einbauen.
- Es gibt **keine Methode, die die Priorität eines Eintrags verkleinert** (in der Literatur *DecreaseKey* genannt). Findet Dijkstra einen besseren Weg nach Salzburg, legst du Salzburg einfach noch einmal mit der neuen Distanz hinein. Der alte Eintrag (Salzburg, 365) bleibt in der Warteschlange. Er ist **veraltet**.

Ein veralteter Eintrag stört nicht, solange du ihn erkennst. Er kommt immer erst nach dem neuen Eintrag dran, weil seine Priorität größer ist. Wenn du ihn herausnimmst, vergleichst du seine Priorität mit der Distanztabelle. Ist sie größer als die Distanz in der Tabelle, überspringst du den Eintrag. Genau das ist Schritt 8 in der Tabelle oben.

Dijkstra in C#

Der Graph ist ein `Dictionary`, das jeder Stadt eine Liste ihrer Nachbarn mit der Entfernung zuordnet. Die Distanztabelle besteht aus zwei weiteren `Dictionary`-Objekten: eines für die Distanzen und eines für die Vorgänger.

```
Road[] roads =
[
    new("Bregenz", "Innsbruck", 195), new("Innsbruck", "Salzburg", 185),
    new("Salzburg", "Linz", 130), new("Salzburg", "Klagenfurt", 225),
    new("Linz", "St. Pölten", 130), new("Linz", "Graz", 220),
    new("St. Pölten", "Wien", 65), new("Wien", "Eisenstadt", 60),
    new("Wien", "Graz", 200), new("Eisenstadt", "Graz", 165),
    new("Graz", "Klagenfurt", 140)
```

```
];

Dictionary<string, List<(string To, int Km)>> graph = [];
foreach (Road road in roads)
{
    AddNeighbor(graph, road.From, road.To, road.Km);
    AddNeighbor(graph, road.To, road.From, road.Km);
}

(Dictionary<string, int> distance, Dictionary<string, string> previous) = Dijkstra
(graph, "Graz");
foreach ((string city, int km) in distance.OrderBy(entry => entry.Value))
{
    Console.WriteLine($"{city,-11}{km,4} km");
}
Console.WriteLine(string.Join(" -> ", BuildPath(previous, "Graz", "Bregenz")));

static void AddNeighbor(Dictionary<string, List<(string To, int Km)>> graph, string
from, string to, int km)
{
    if (!graph.TryGetValue(from, out List<(string To, int Km)>? neighbors))
    {
        neighbors = [];
        graph[from] = neighbors;
    }
    neighbors.Add((to, km));
}

static (Dictionary<string, int>, Dictionary<string, string>) Dijkstra(
    Dictionary<string, List<(string To, int Km)>> graph, string start)
{
    Dictionary<string, int> distance = new() { [start] = 0 };       // (1)
    Dictionary<string, string> previous = [];
    PriorityQueue<string, int> queue = new();
    queue.Enqueue(start, 0);

    while (queue.TryDequeue(out string? city, out int km))           // (2)
    {
        if (km > distance[city]) { continue; }                       // (3)
        foreach ((string next, int edgeKm) in graph[city])
        {
            int newKm = km + edgeKm;
            if (!distance.TryGetValue(next, out int oldKm) || newKm < oldKm)   // (4)
            {
                distance[next] = newKm;
                previous[next] = city;
                queue.Enqueue(next, newKm);                          // (5)
            }
        }
    }
    return (distance, previous);
}

static List<string> BuildPath(Dictionary<string, string> previous, string start, string
target)
{
```

```
    List<string> path = [target];
    string city = target;
    while (city != start)                                    // (6)
    {
        city = previous[city];
        path.Add(city);
    }
    path.Reverse();
    return path;
}

record Road(string From, string To, int Km);
```

```
Graz          0 km
Klagenfurt  140 km
Eisenstadt  165 km
Wien        200 km
Linz        220 km
St. Pölten  265 km
Salzburg    350 km
Innsbruck   535 km
Bregenz     730 km
Graz -> Linz -> Salzburg -> Innsbruck -> Bregenz
```

- **(1)** Die Distanztabelle enthält am Anfang nur den Start. Ein Knoten, der noch nicht im `Dictionary` steht, hat die Distanz ∞. So brauchst du keinen Ersatzwert wie `int.MaxValue`, bei dem eine Addition überlaufen könnte.
- **(2)** `TryDequeue` liefert den Eintrag mit der kleinsten Distanz und `false`, sobald die Warteschlange leer ist. Dann ist der Algorithmus fertig.
- **(3)** Das ist der Test auf veraltete Einträge. Steht in der Tabelle schon eine kleinere Distanz, wurde die Stadt bereits über einen kürzeren Weg bearbeitet. Ohne diese Zeile würde Salzburg in Schritt 8 ein zweites Mal bearbeitet werden.
- **(4)** Relaxieren: Die Bedingung ist erfüllt, wenn `next` noch ∞ hat oder der neue Weg kürzer ist. Bei einem gleich langen Weg bleibt der alte Vorgänger stehen.
- **(5)** Statt die Priorität zu ändern, kommt ein neuer Eintrag in die Warteschlange.
- **(6)** `BuildPath` folgt den Vorgängern vom Ziel zurück zum Start. Das erklärt der nächste Abschnitt.

Fertige Nachbarn prüft der Code nicht eigens. Ihre Distanz ist schon die kleinste, deshalb ist die Bedingung in (4) für sie nie erfüllt.

Den Weg rekonstruieren

Die Distanztabelle sagt dir, *wie weit* es nach Bregenz ist, aber nicht *wie* du hinkommst. Dafür ist das `Dictionary` mit den Vorgängern da. Es speichert für jede Stadt nur einen einzigen Schritt: die Stadt, von der aus der kürzeste Weg hereinkommt. Den ganzen Weg bekommst du, indem du vom Ziel aus rückwärts den Vorgängern folgst, bis du beim Start ankommst:

```
Bregenz <- Innsbruck <- Salzburg <- Linz <- Graz
```

`BuildPath` sammelt die Städte in dieser Reihenfolge und dreht die Liste am Ende mit `Reverse` um. Das funktioniert, weil jeder Teilweg eines kürzesten Weges selbst ein kürzester Weg ist: Der kürzeste Weg nach Bregenz führt über Innsbruck, also ist sein Anfang der kürzeste Weg nach Innsbruck.

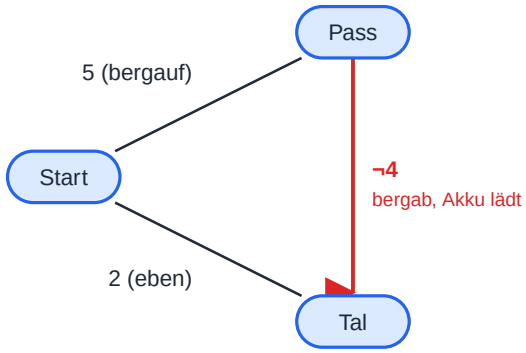
Ist das Ziel nicht erreichbar, steht es nicht in `distance`, und `previous[city]` würde eine `KeyNotFoundException` werfen. Prüfe deshalb vorher mit `distance.ContainsKey(target)`, ob es überhaupt einen Weg gibt.

Ein Lauf von Dijkstra liefert übrigens nicht nur den Weg nach Bregenz, sondern die kürzesten Wege von Graz zu **allen** Städten. Die Vorgänger bilden zusammen einen Baum mit Graz als Wurzel, den *Kürzeste-Wege-Baum*.

Grenzen: negative Kantengewichte

Bei Kilometern und Minuten sind Gewichte nie negativ. Es gibt aber Anwendungen, in denen negative Gewichte Sinn ergeben. Ein Elektroauto verbraucht bergauf viel Energie, bergab lädt es den Akku wieder auf (*Rekuperation*). Misst du die Kanten in kWh, kann eine Bergabfahrt ein negatives Gewicht haben.

Energieverbrauch eines Elektroautos in kWh



entnommen	Start	Pass	Tal
–	0	¬	¬
Start (0)	0	5	2
Tal (2)	0	5	2
Pass (5)	0	5	2

Nach „Tal (2)“ ist das Tal fertig.
Über den Pass wären es 5 + (¬4) = 1 kWh,
aber ein fertiger Knoten ändert sich nicht mehr.

Dijkstra: Tal = 2 ¬ **richtig: Start ¬ Pass ¬ Tal = 1**

Dijkstra nimmt nach dem Start zuerst das Tal heraus (2 kWh), weil es die kleinste Distanz hat. Das Tal ist damit fertig. Erst danach wird der Pass bearbeitet, und über ihn käme man mit 5 + (−4) = 1 kWh ins Tal. Die Annahme „ein Umweg über einen anderen offenen Knoten kann nur länger werden“ stimmt nicht mehr. Dijkstra liefert für das Tal den falschen Wert 2.

Noch schlimmer ist ein **Kreis mit negativer Summe**. Dort wird jeder Weg kürzer, je öfter du im Kreis fährst. Einen kürzesten Weg gibt es dann gar nicht. In einem ungerichteten Graphen ist jede negative Kante schon so ein Kreis: Du fährst sie einfach hin und zurück.

Für Graphen mit negativen Gewichten gibt es den **Bellman-Ford-Algorithmus**. Er verzichtet auf das „fertig“ und relaxiert stattdessen alle Kanten immer wieder, bis sich nichts mehr ändert.

Er erkennt auch Kreise mit negativer Summe. Bellman-Ford ist nicht Teil dieser Einheit.

Zusatzmaterial

Im 22_Kuerzeste_Wege.html siehst du Dijkstra mit Distanztabelle und Warteschlange als Animation und führst den Algorithmus danach auf mehreren Graphen, auch auf Zufallsgraphen, selbst Schritt für Schritt aus.

Topologische Sortierung

Bevor du Entity Framework Core lernst, solltest du LINQ und SQL kennen. Bevor Visual Studio deine Web-API kompilieren kann, muss es die Projekte bauen, die die Web-API referenziert. Bevor NuGet ein Paket installiert, braucht es alle Pakete, von denen dieses Paket abhängt. Immer geht es um dieselbe Frage: **In welcher Reihenfolge kann ich Dinge erledigen, die voneinander abhängen?** Die Antwort liefert die *topologische Sortierung*.

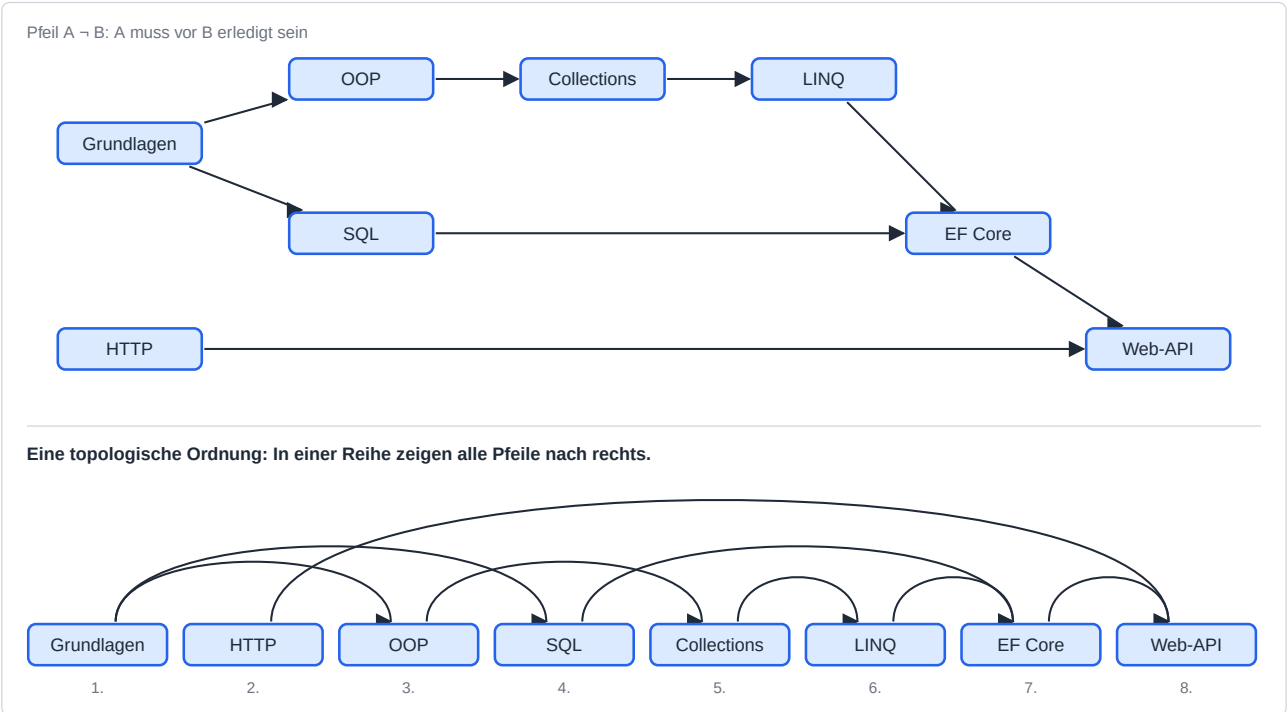
Abhängigkeiten als Graph

Abhängigkeiten lassen sich direkt als gerichteter Graph aufschreiben. Jede Aufgabe ist ein Knoten. Eine Kante A → B bedeutet: **A muss vor B erledigt sein.** Der Pfeil zeigt also von der Voraussetzung zu dem, was darauf aufbaut.

Anwendung	Knoten	Kante A → B
Lehrveranstaltungen	Themen oder Fächer	A ist Voraussetzung für B
Build-Schritte	Restore, Build, Test, Publish	B braucht das Ergebnis von A
Projekte in einer Solution	Projekte	B referenziert A, also muss A zuerst gebaut werden
NuGet-Pakete	Pakete	B hängt von A ab
Tabellenkalkulation	Zellen	die Formel in B verwendet A

Viele Werkzeuge zeichnen den Pfeil übrigens andersherum, als „B hängt ab von A“. Das ist nur eine Konvention. In dieser Einheit zeigt der Pfeil immer in die Richtung, in der gearbeitet wird.

Als Beispiel dienen Themen aus der Programmierausbildung. Die Grafik zeigt, welches Thema auf welchem aufbaut:



Ein solcher Graph darf **keinen Zyklus** haben. Bräuchtest du für LINQ zuerst EF Core und für EF Core zuerst LINQ, könntest du nie anfangen. Ein gerichteter Graph ohne Zyklus heißt **DAG** (*directed acyclic graph*).

Topologische Ordnung

Eine **topologische Ordnung** ist eine Reihenfolge aller Knoten, in der für jede Kante A → B der Knoten A vor B steht. Schreibst du die Knoten in dieser Reihenfolge nebeneinander, zeigen alle Pfeile nach rechts. Das zeigt der untere Teil der Grafik.

Für den Themengraphen ist zum Beispiel diese Reihenfolge gültig:

```
Grundlagen, HTTP, OOP, SQL, Collections, LINQ, EF Core, Web-API
```

Diese Reihenfolge ist dagegen **ungültig**, denn SQL steht nach EF Core, obwohl es die Kante SQL → EF Core gibt:

```
Grundlagen, OOP, Collections, LINQ, EF Core, SQL, HTTP, Web-API
```

Zwei Tatsachen solltest du dir merken:

- Eine topologische Ordnung gibt es **genau dann**, wenn der Graph ein DAG ist. Bei einem Zyklus A → B → C → A müsste A vor B, B vor C und C vor A stehen. Das geht nicht.
- Meist gibt es **mehrere gültige Ordnungen**. HTTP hängt von nichts ab und wird nur für die Web-API gebraucht. Es kann also an fast jeder Stelle vor der Web-API stehen. Auch SQL und der Strang OOP → Collections → LINQ sind unabhängig voneinander. Jede dieser Reihenfolgen ist richtig:

```
Grundlagen, HTTP, OOP, SQL, Collections, LINQ, EF Core, Web-API
HTTP, Grundlagen, SQL, OOP, Collections, LINQ, EF Core, Web-API
Grundlagen, OOP, Collections, LINQ, SQL, EF Core, HTTP, Web-API
```

Welche davon ein Algorithmus liefert, hängt davon ab, welchen Knoten er bei mehreren Möglichkeiten zuerst nimmt.

Der Algorithmus von Kahn

Der **Eingangsgrad** eines Knotens ist die Anzahl der Kanten, die in ihn hineinführen. Bei Abhängigkeiten ist das die Anzahl der Voraussetzungen. EF Core hat den Eingangsgrad 2 (LINQ und SQL), Grundlagen und HTTP haben den Eingangsgrad 0.

Ein Knoten mit Eingangsgrad 0 hat keine offenen Voraussetzungen. Mit ihm kannst du sofort anfangen. Daraus hat Arthur B. Kahn 1962 folgendes Verfahren gemacht:

1. Berechne für jeden Knoten den Eingangsgrad.
2. Lege alle Knoten mit Eingangsgrad 0 in eine Warteschlange.
3. Solange die Warteschlange nicht leer ist: Nimm einen Knoten heraus und hänge ihn an die Ausgabe an. Verringere den Eingangsgrad jedes Nachfolgers um 1. Erreicht ein Nachfolger dabei 0, sind alle seine Voraussetzungen erledigt: Er kommt in die Warteschlange.
4. Enthält die Ausgabe alle Knoten, ist sie eine topologische Ordnung. Sonst hat der Graph einen Zyklus.

Den Eingangsgrad zu verringern bedeutet anschaulich: Du streichst die Kante aus dem Graphen, weil diese Voraussetzung jetzt erfüllt ist. Die Tabelle zeigt den Ablauf für den Themengraphen. Am Anfang kommen die Knoten mit Eingangsgrad 0 alphabetisch sortiert in die Queue, danach in der Reihenfolge, in der ihr Eingangsgrad 0 wird.

Schritt	entnommen	Grundlagen	HTTP	OOP	SQL	Collections	LINQ	EF Core	Web-API	Queue danach
Start	–	0	0	1	1	1	1	2	2	Grundlagen, HTTP
1	Grundlagen	0	0	0	0	1	1	2	2	HTTP, OOP, SQL
2	HTTP	–	0	0	0	1	1	2	1	OOP, SQL
3	OOP	–	–	0	0	0	1	2	1	SQL, Collections
4	SQL	–	–	–	0	0	1	1	1	Collections
5	Collections	–	–	–	–	0	0	1	1	LINQ
6	LINQ	–	–	–	–	–	0	0	1	EF Core
7	EF Core	–	–	–	–	–	–	0	0	Web-API
8	Web-API	–	–	–	–	–	–	–	0	(leer)

Ausgabe: Grundlagen, HTTP, OOP, SQL, Collections, LINQ, EF Core, Web-API

wird entnommen und ausgegeben — Eingangsgrad wird 0, kommt in die Queue — 1 Eingangsgrad sinkt — schon ausgegeben

Schau dir Schritt 4 an: SQL ist fertig, EF Core hat aber erst den Eingangsgrad 1, weil LINQ noch fehlt. EF Core muss warten. Erst in Schritt 6, nach LINQ, erreicht EF Core den Eingangsgrad 0.

Kahn in C#

Der Graph ist ein `Dictionary`, das jedem Knoten die Liste seiner Nachfolger zuordnet. Das Programm sortiert den Themengraphen und danach einen zweiten Graphen, in dem sich ein Zyklus versteckt.

```
(string Before, string After)[] topics =
[
    ("Grundlagen", "OOP"), ("Grundlagen", "SQL"), ("OOP", "Collections"),
    ("Collections", "LINQ"), ("LINQ", "EF Core"), ("SQL", "EF Core"),
    ("EF Core", "Web-API"), ("HTTP", "Web-API")
];
(string Before, string After)[] packages =
[
    ("Core", "Config"), ("Config", "Logging"), ("Logging", "Data"),
    ("Data", "Config"), ("Data", "App")
];

foreach ((string Before, string After)[] edges in new[] { topics, packages })
{
```

```
    (List<string> order, List<string> blocked) = Kahn(BuildGraph(edges));
    Console.WriteLine($"Ordnung:   {string.Join(", ", order)}");
    if (blocked.Count > 0)
    {
        Console.WriteLine($"Zyklus! Nicht sortierbar: {string.Join(", ", blocked)}");
    }
}

static Dictionary<string, List<string>> BuildGraph((string Before, string After)[]
edges)
{
    Dictionary<string, List<string>> graph = [];
    foreach ((string before, string after) in edges)
    {
        graph.TryAdd(before, []);                                    // (1)
        graph.TryAdd(after, []);
        graph[before].Add(after);
    }
    return graph;
}

static (List<string>, List<string>) Kahn(Dictionary<string, List<string>> graph)
{
    Dictionary<string, int> inDegree = graph.Keys.ToDictionary(node => node, _ => 0);
    foreach (List<string> successors in graph.Values)               // (2)
    {
        foreach (string next in successors) { inDegree[next]++; }
    }

    Queue<string> ready = new(inDegree.Keys.Where(node => inDegree[node] == 0).
Order());   // (3)
    List<string> order = [];
    while (ready.TryDequeue(out string? node))
    {
        order.Add(node);
        foreach (string next in graph[node])
        {
            inDegree[next]--;                                        // (4)
            if (inDegree[next] == 0) { ready.Enqueue(next); }
        }
    }

    List<string> blocked = [.. inDegree.Keys.Where(node => inDegree[node] > 0).
Order()];   // (5)
    return (order, blocked);
}
```

```
Ordnung:   Grundlagen, HTTP, OOP, SQL, Collections, LINQ, EF Core, Web-API
Ordnung:   Core
Zyklus! Nicht sortierbar: App, Config, Data, Logging
```

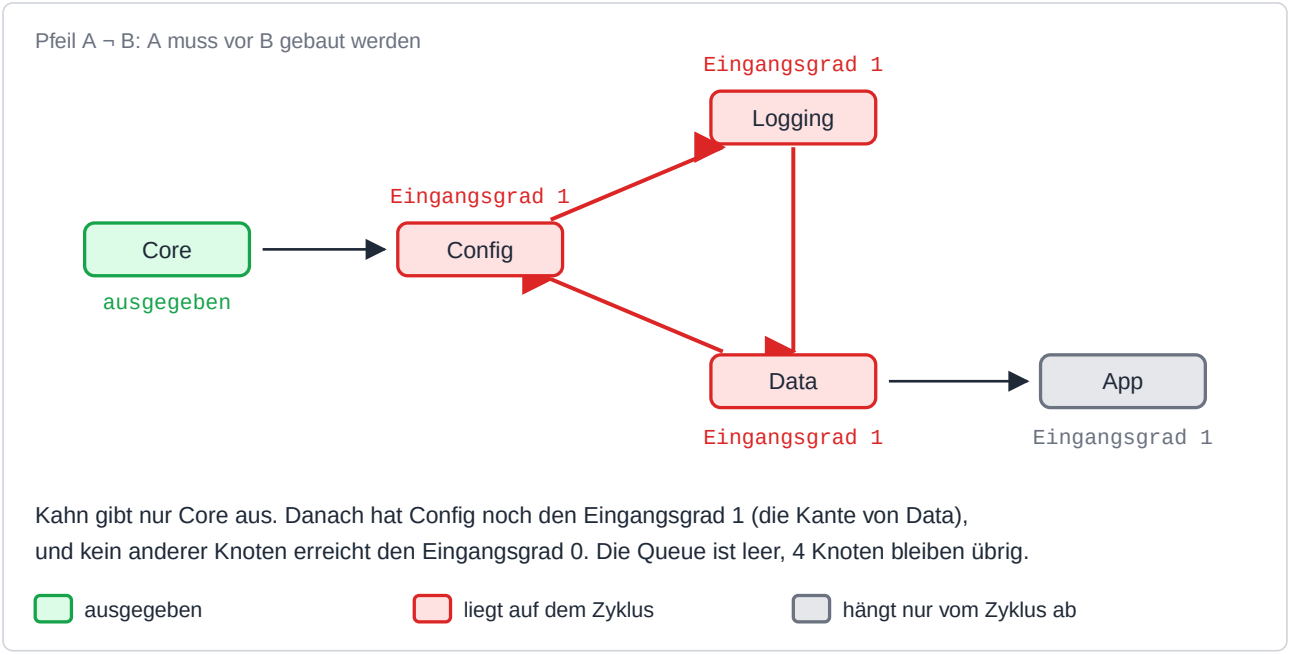
- **(1)** Jeder Knoten bekommt einen Eintrag, auch wenn er keine Nachfolger hat (wie Web-API). `TryAdd` legt den Eintrag nur an, wenn es ihn noch nicht gibt.

- **(2)** Für jede Kante wird der Eingangsgrad ihres Zielknotens um 1 erhöht. Danach hat EF Core den Wert 2.
- **(3)** Die Startknoten kommen alphabetisch sortiert in die Queue. Ohne `Order()` hinge die Ausgabe von der Reihenfolge im `Dictionary` ab. Gültig wäre sie trotzdem.
- **(4)** Das ist das „Streichen“ der Kante. Erreicht der Nachfolger 0, kommt er in die Queue.
- **(5)** Knoten, deren Eingangsgrad nie 0 geworden ist, konnten nicht ausgegeben werden. Ist die Liste nicht leer, hat der Graph einen Zyklus.

Statt einer `Queue` funktioniert auch ein `Stack` oder eine beliebige andere Sammlung. Entscheidend ist nur, dass ein Knoten erst hineinkommt, wenn sein Eingangsgrad 0 ist. Mit einer anderen Sammlung bekommst du eine andere, ebenso gültige Ordnung.

Zyklen erkennen

Der zweite Graph im Programm beschreibt Pakete, die voneinander abhängen. Dabei hat sich ein Fehler eingeschlichen: Data braucht Config, aber Config braucht über Logging indirekt Data.



Kahn gibt Core aus und senkt damit den Eingangsgrad von Config von 2 auf 1. Dann ist die Queue leer. Kein Knoten auf dem Zyklus kann je den Eingangsgrad 0 erreichen, denn jeder wartet auf seinen Vorgänger im Kreis. Der Algorithmus bleibt also nicht hängen, sondern endet mit einer zu kurzen Ausgabe. Genau daran erkennst du den Zyklus: **Die Ausgabe enthält weniger Knoten als der Graph.**

Beachte, dass die übrig gebliebenen Knoten nicht alle auf dem Zyklus liegen. App ist nur blockiert, weil es von Data abhängt. Für eine gute Fehlermeldung („Config → Logging → Data → Config“) musst du den Zyklus selbst noch aus den übrigen Knoten heraussuchen.

Echte Werkzeuge raten in so einem Fall keine Reihenfolge, sondern melden einen Fehler. Visual Studio lässt eine Projektreferenz, die einen Zyklus erzeugen würde, gar nicht erst zu. Excel zeigt bei einem Zirkelbezug eine Warnung an.

Variante mit Tiefensuche

Es gibt einen zweiten Weg zur topologischen Ordnung, der auf der Tiefensuche (DFS) aufbaut. Die Idee: Die Tiefensuche schließt einen Knoten erst ab, wenn alle seine Nachfolger abgeschlossen sind. Schreibst du die Knoten in der Reihenfolge auf, in der sie **abgeschlossen** werden, steht jeder Knoten hinter all seinen Nachfolgern. Diese Liste musst du nur noch umdrehen.

```
Dictionary<string, List<string>> graph = new()
{
    ["Grundlagen"] = ["OOP", "SQL"], ["OOP"] = ["Collections"], ["Collections"] =
["LINQ"],
    ["LINQ"] = ["EF Core"], ["SQL"] = ["EF Core"], ["EF Core"] = ["Web-API"],
    ["HTTP"] = ["Web-API"], ["Web-API"] = []
};

HashSet<string> visited = [];
List<string> finished = [];
foreach (string node in graph.Keys.Order()) { Visit(node); }
finished.Reverse();
Console.WriteLine(string.Join(", ", finished));

void Visit(string node)
{
    if (!visited.Add(node)) { return; }
    foreach (string next in graph[node]) { Visit(next); }
    finished.Add(node);                          // all successors are finished before
node
}
```

```
HTTP, Grundlagen, SQL, OOP, Collections, LINQ, EF Core, Web-API
```

Das Ergebnis ist eine andere, ebenfalls gültige Ordnung. Diese Variante erkennt Zyklen allerdings nicht von selbst. Dafür braucht die Tiefensuche einen dritten Zustand „in Bearbeitung“: Trifft sie auf einen Knoten, der gerade in Bearbeitung ist, hat sie einen Zyklus gefunden. Beim Kahn-Algorithmus ergibt sich die Zykluserkennung dagegen von selbst.

Divide and Conquer

Stell dir vor, du suchst in einem gedruckten Wörterbuch das Wort *Rekursion*. Du fängst nicht auf Seite 1 an. Du schlägst ungefähr in der Mitte auf, siehst *Lampe* und weißt: Das Wort steht weiter hinten. Die vordere Hälfte ist damit erledigt. Mit der hinteren Hälfte machst du dasselbe noch einmal. Dieses Vorgehen heißt **Divide and Conquer** (deutsch *Teile und herrsche*). Es steckt in der binären Suche und in zwei der bekanntesten Sortierverfahren: Mergesort und Quicksort.

Das Prinzip

Ein Divide-and-Conquer-Algorithmus arbeitet in drei Schritten:

1. **Teilen:** Das Problem wird in kleinere Teilprobleme derselben Art zerlegt.
2. **Lösen:** Die Teilprobleme werden mit demselben Verfahren gelöst, also rekursiv. Ist ein Teilproblem so klein, dass die Lösung sofort feststeht, hört die Rekursion auf. Das ist der **Basisfall**.
3. **Zusammenfügen:** Aus den Lösungen der Teilprobleme entsteht die Lösung des ganzen Problems.

Die drei Algorithmen dieser Einheit setzen die Schritte unterschiedlich um:

Algorithmus	Teilen	Basisfall	Zusammenfügen
Binäre Suche	Mitte ansehen, eine Hälfte verwerfen	leerer Bereich oder Treffer	entfällt
Mergesort	Array in der Mitte halbieren	ein Element	zwei sortierte Hälften mischen
Quicksort	nach einem Pivot-Element aufteilen	höchstens ein Element	entfällt

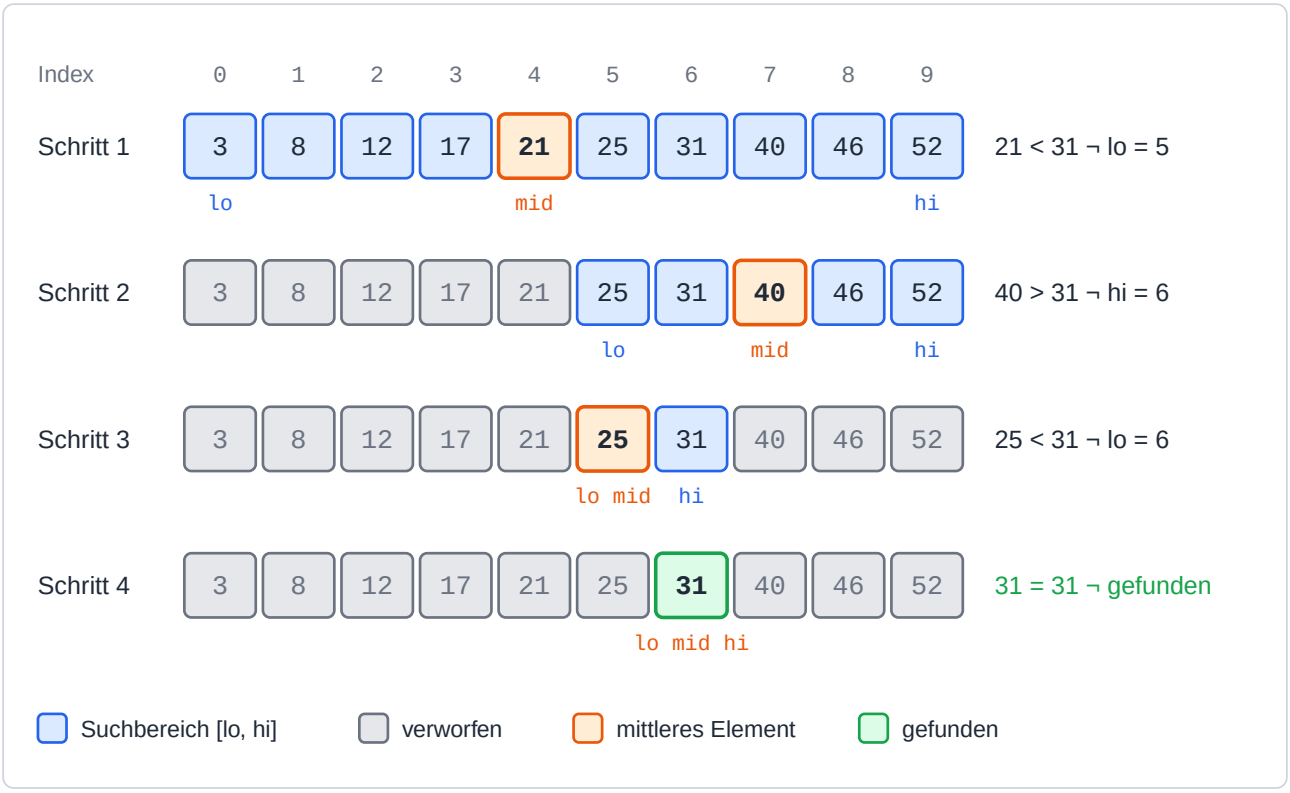
Bei der binären Suche wird nur **ein** Teilproblem weiterverfolgt, bei den Sortierverfahren **beide**. Beim Mergesort steckt die Arbeit im Zusammenfügen, beim Quicksort im Teilen.

Binäre Suche

Die binäre Suche findet einen Wert in einem **sortierten** Array. Sie merkt sich den Bereich, in dem der Wert noch liegen kann, mit zwei Indizes `lo` und `hi`. In jedem Schritt vergleicht sie das mittlere Element `mid` mit dem gesuchten Wert:

- Gleich: gefunden.
- Das mittlere Element ist kleiner: Der Wert kann nur rechts davon liegen, also `lo = mid + 1`.
- Das mittlere Element ist größer: Der Wert kann nur links davon liegen, also `hi = mid - 1`.

Wird der Bereich leer (`lo > hi`), ist der Wert nicht im Array. Die Grafik zeigt die Suche nach 31 in einem Array mit 10 Zahlen. Nach vier Vergleichen ist der Wert gefunden.



Iterativ

```
int[] numbers = [3, 8, 12, 17, 21, 25, 31, 40, 46, 52];
Console.WriteLine(BinarySearch(numbers, 31));
Console.WriteLine(BinarySearch(numbers, 30));

static int BinarySearch(int[] sorted, int target)
{
    int lo = 0, hi = sorted.Length - 1;             // (1)
    while (lo <= hi)                                // (2)
    {
        int mid = lo + (hi - lo) / 2;               // (3)
        if (sorted[mid] == target) { return mid; }
        if (sorted[mid] < target) { lo = mid + 1; } // (4)
        else { hi = mid - 1; }
    }
    return -1;                                      // (5)
}
```

```
6
-1
```

- **(1)** Der Suchbereich ist das geschlossene Intervall `[lo, hi]`. Am Anfang ist das das ganze Array, deshalb `hi = Length - 1`.
- **(2)** Solange der Bereich mindestens ein Element enthält, wird weitergesucht. Bei `lo == hi` ist noch genau ein Element übrig, und das muss auch geprüft werden.
- **(3)** Die Mitte des Bereichs. Die Schreibweise `lo + (hi - lo) / 2` liefert dasselbe wie `(lo +`

`hi) / 2`, kann aber nicht überlaufen (siehe unten).

- **(4)** `mid` selbst ist schon geprüft. Der neue Bereich beginnt deshalb erst bei `mid + 1` bzw. endet bei `mid - 1`. So wird der Bereich in jedem Durchlauf echt kleiner.
- **(5)** Der Bereich ist leer, der Wert kommt nicht vor.

Rekursiv

Die rekursive Variante zeigt das Prinzip *Teile und herrsche* noch deutlicher. Der Bereich wird als Parameter übergeben, jeder Aufruf bearbeitet eine Hälfte des vorigen Bereichs.

```
int[] numbers = [3, 8, 12, 17, 21, 25, 31, 40, 46, 52];
Console.WriteLine(BinarySearch(numbers, 31, 0, numbers.Length - 1));
Console.WriteLine(BinarySearch(numbers, 30, 0, numbers.Length - 1));

static int BinarySearch(int[] sorted, int target, int lo, int hi)
{
    if (lo > hi) { return -1; }                    // (1)
    int mid = lo + (hi - lo) / 2;
    return sorted[mid].CompareTo(target) switch    // (2)
    {
        0 => mid,
        < 0 => BinarySearch(sorted, target, mid + 1, hi),
        _ => BinarySearch(sorted, target, lo, mid - 1)
    };
}
```

```
6
-1
```

- **(1)** Der Basisfall: Ein leerer Bereich enthält den Wert sicher nicht.
- **(2)** `CompareTo` liefert 0 bei Gleichheit, eine negative Zahl, wenn `sorted[mid]` kleiner ist, und sonst eine positive Zahl. Der `switch`-Ausdruck mit dem relationalen Pattern `< 0` wählt die passende Hälfte. Diese Form funktioniert für jeden Typ, der `IComparable<T>` implementiert, also auch für Strings.

Off-by-one-Fehler

Die binäre Suche ist kurz, aber berüchtigt: Schon kleine Änderungen an den Grenzen machen sie falsch. Diese Fehler zeigen sich oft nur bei den Rändern, also beim ersten oder letzten Element oder bei Werten, die nicht vorkommen. Für das Array von oben:

Änderung	Folge
`while (lo < hi)` statt `lo <= hi`	Die Suche nach 3 oder 52 liefert -1. Das letzte verbleibende Element wird nie geprüft.
`hi = sorted.Length` statt `Length - 1`	Die Suche nach 60 berechnet irgendwann `mid = 10` und wirft eine `IndexOutOfRangeException`.

Änderung	Folge
`lo = mid` statt `lo = mid + 1`	Die Suche nach 52 bleibt bei `lo = 8`, `hi = 9` hängen: `mid` ist immer wieder 8, eine Endlosschleife.
`(lo + hi) / 2`	Bei Arrays mit mehr als einer Milliarde Elementen kann `lo + hi` größer als `int.MaxValue` werden und negativ überlaufen.

Der letzte Fehler steckte jahrelang in der Bibliothek von Java, bevor er 2006 auffiel. Die sicherste Methode gegen alle diese Fehler ist eine klare Regel, die du nie brichst: *Wenn der Wert vorkommt, liegt er im Bereich* `[lo, hi]`. Jede Zeile, die `lo` oder `hi` ändert, muss diese Regel erhalten, und der Bereich muss in jedem Durchlauf kleiner werden.

Array.BinarySearch

In der Praxis schreibst du die binäre Suche selten selbst. .NET bietet `Array.BinarySearch` und `List<T>.BinarySearch`. Interessant ist der Rückgabewert, wenn der Wert fehlt:

```
int[] numbers = [3, 8, 12, 17, 21, 25, 31, 40, 46, 52];

int found = Array.BinarySearch(numbers, 31);
Console.WriteLine(found);                          // (1)

int missing = Array.BinarySearch(numbers, 30);
Console.WriteLine(missing);                        // (2)
int insertAt = ~missing;                           // (3)
Console.WriteLine(insertAt);

List<int> list = [.. numbers];
list.Insert(insertAt, 30);                         // (4)
Console.WriteLine(string.Join(", ", list));
```

```
6
-7
6
3, 8, 12, 17, 21, 25, 30, 31, 40, 46, 52
```

- **(1)** Gefunden: Der Index des Elements.
- **(2)** Nicht gefunden: eine **negative** Zahl. Sie ist nicht einfach -1, sondern enthält eine Information: die Stelle, an der der Wert stehen müsste.
- **(3)** Der Operator `~` (bitweises Komplement) macht daraus wieder einen Index. Für jede Zahl x gilt `~x == -x - 1`, also `~(-7) == 6`. 30 gehört vor die 31 an Index 6. Ein negatives Ergebnis bedeutet immer „nicht gefunden“, denn auch die Einfügestelle 0 wird zu `~0 == -1`.
- **(4)** Fügst du an dieser Stelle ein, bleibt die Liste sortiert. So hältst du eine Liste ohne erneutes Sortieren in Ordnung.

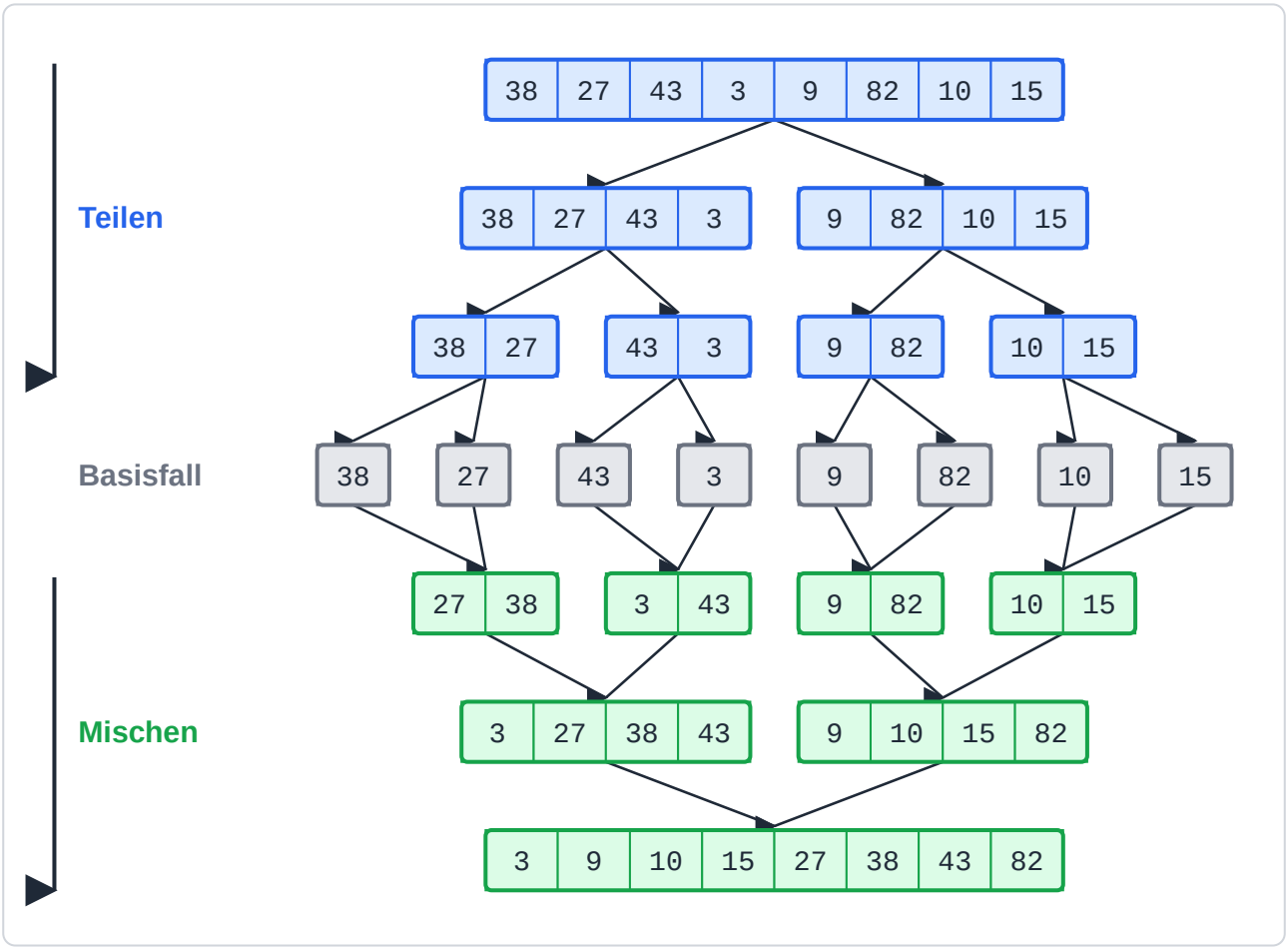
Zwei Dinge musst du beachten: Ist das Array nicht sortiert, liefert `BinarySearch` ein beliebiges

Ergebnis, ohne eine Exception zu werfen. Und kommt ein Wert mehrfach vor, ist nicht festgelegt, welcher der passenden Indizes zurückkommt.

Mergesort

Mergesort setzt das Prinzip ganz direkt um:

1. **Teilen:** Das Array wird in der Mitte in zwei Hälften geteilt.
2. **Lösen:** Beide Hälften werden rekursiv sortiert. Ein Array mit einem Element ist bereits sortiert, das ist der Basisfall.
3. **Zusammenfügen:** Die zwei sortierten Hälften werden zu einem sortierten Array *gemischt* (englisch *merge*).



In der oberen Hälfte der Grafik wird geteilt, ohne dass irgendein Vergleich stattfindet. Die eigentliche Arbeit passiert in der unteren Hälfte beim Mischen.

Der Merge-Schritt

Zwei sortierte Arrays zu mischen ist einfach. Du brauchst für jedes Array einen Zeiger auf das erste noch nicht verwendete Element. In jedem Schritt vergleichst du die beiden Elemente und übernimmst das kleinere. Ist ein Array aufgebraucht, hängst du den Rest des anderen an. Für den letzten Schritt aus der Grafik:

```
links:  3 27 38 43    rechts:  9 10 15 82
3 < 9    -> 3          Ergebnis: 3
27 > 9   -> 9          Ergebnis: 3 9
27 > 10  -> 10         Ergebnis: 3 9 10
27 > 15  -> 15         Ergebnis: 3 9 10 15
27 < 82  -> 27         Ergebnis: 3 9 10 15 27
38 < 82  -> 38         Ergebnis: 3 9 10 15 27 38
43 < 82  -> 43         Ergebnis: 3 9 10 15 27 38 43
links leer, Rest 82    Ergebnis: 3 9 10 15 27 38 43 82
```

Mergesort in C#

```
int[] numbers = [38, 27, 43, 3, 9, 82, 10, 15];
int[] sorted = MergeSort(numbers);
Console.WriteLine(string.Join(", ", sorted));

static int[] MergeSort(int[] items)
{
    if (items.Length <= 1) { return items; }         // (1)
    int mid = items.Length / 2;
    int[] left = MergeSort(items[..mid]);            // (2)
    int[] right = MergeSort(items[mid..]);
    return Merge(left, right);                       // (3)
}

static int[] Merge(int[] left, int[] right)
{
    int[] result = new int[left.Length + right.Length];
    int i = 0, j = 0, k = 0;
    while (i < left.Length && j < right.Length)
    {
        // On ties the left element wins, so equal elements keep their order.
        result[k++] = left[i] <= right[j] ? left[i++] : right[j++];     // (4)
    }
    while (i < left.Length) { result[k++] = left[i++]; }               // (5)
    while (j < right.Length) { result[k++] = right[j++]; }
    return result;
}
```

```
3, 9, 10, 15, 27, 38, 43, 82
```

- **(1)** Basisfall: Ein Array mit höchstens einem Element ist sortiert.
- **(2)** Die Range-Syntax `items[..mid]` liefert ein **neues** Array mit den Elementen 0 bis `mid - 1`, `items[mid..]` den Rest. Bei ungerader Länge bekommt die rechte Hälfte ein Element mehr.
- **(3)** Die beiden sortierten Hälften werden zusammengefügt.
- **(4)** `i`, `j` und `k` sind die Zeiger in `left`, `right` und `result`. `left[i++]` liest das Element und schiebt den Zeiger danach weiter. Das `<=` ist wichtig, siehe *Stabilität* unten.
- **(5)** Nur eine der beiden Schleifen läuft tatsächlich, denn eines der Arrays ist schon aufge-

braucht.

Quicksort

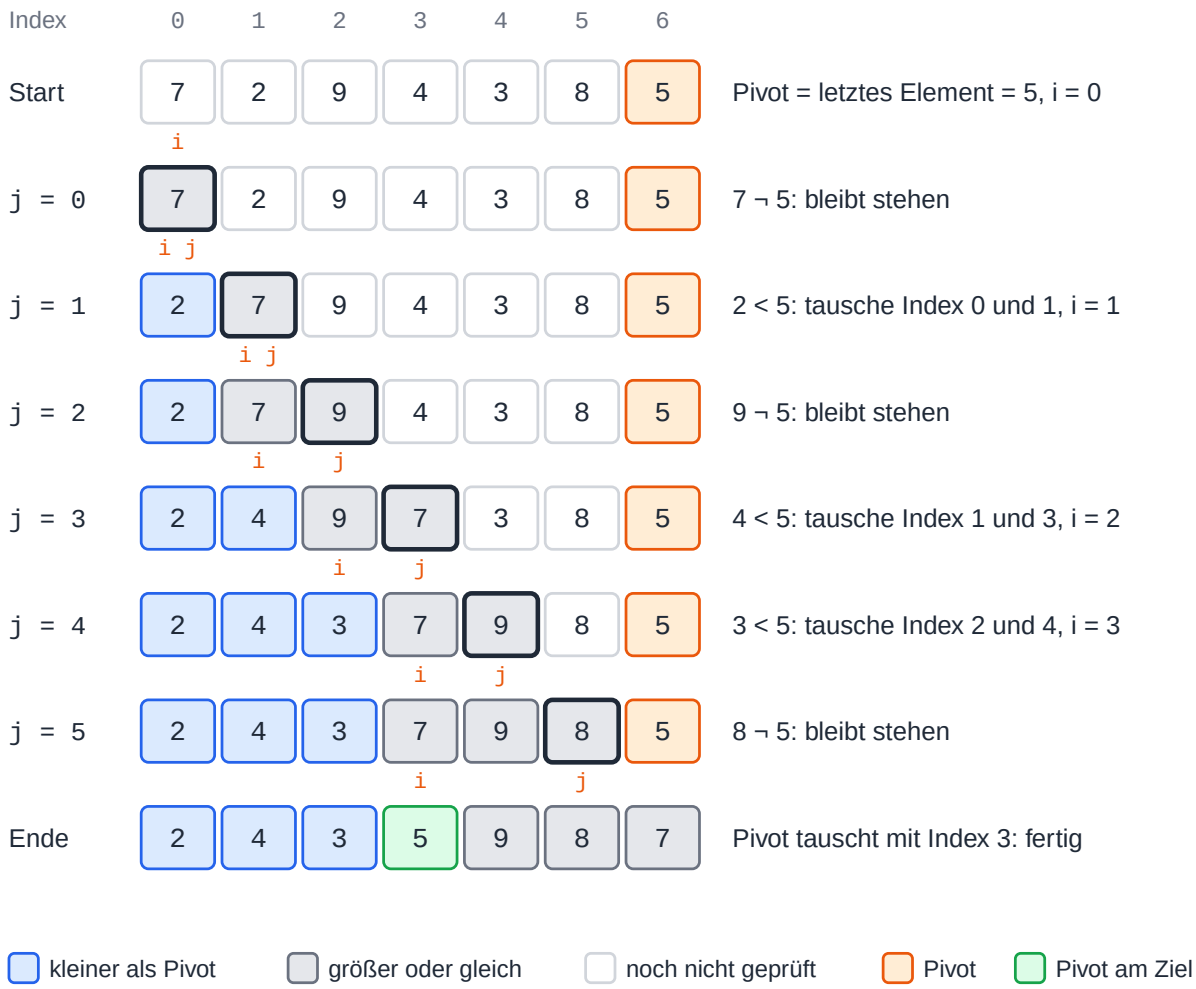
Quicksort geht den umgekehrten Weg. Es wählt ein Element als **Pivot** und ordnet das Array so um, dass alle kleineren Elemente links vom Pivot stehen und alle anderen rechts davon. Dieser Schritt heißt **Partitionieren**. Danach steht das Pivot bereits an seiner endgültigen Stelle. Die beiden Teile links und rechts davon werden rekursiv sortiert. Ein Zusammenfügen ist nicht nötig, weil alles schon an der richtigen Stelle im selben Array steht.

Lomuto-Partitionierung

Es gibt mehrere Arten zu partitionieren. Die Variante nach Nico Lomuto ist am leichtesten zu verstehen. Das Pivot ist das letzte Element des Bereichs. Ein Index `i` markiert die Grenze des Bereichs mit den kleineren Elementen, ein Index `j` läuft einmal von links nach rechts durch:

- Ist `items[j]` kleiner als das Pivot, wird es mit `items[i]` vertauscht und `i` rückt um eins weiter. Damit gehört es zum linken Teil.
- Sonst bleibt es stehen und gehört zum rechten Teil.
- Am Ende wird das Pivot mit `items[i]` vertauscht. Es steht jetzt genau zwischen den beiden Teilen.

Während der Schleife gilt immer: Links von `i` steht nur Kleineres, von `i` bis vor `j` nur Größeres oder Gleiches, ab `j` das noch nicht Geprüfte.



Quicksort in C#

```
int[] numbers = [7, 2, 9, 4, 3, 8, 5];
QuickSort(numbers, 0, numbers.Length - 1);
Console.WriteLine(string.Join(", ", numbers));

static void QuickSort(int[] items, int lo, int hi)
{
    if (lo >= hi) { return; }                       // (1)
    int p = Partition(items, lo, hi);               // (2)
    QuickSort(items, lo, p - 1);                    // (3)
    QuickSort(items, p + 1, hi);
}

static int Partition(int[] items, int lo, int hi)
{
    int pivot = items[hi];                          // (4)
    int i = lo;
    for (int j = lo; j < hi; j++)
    {
        if (items[j] < pivot)
        {
            (items[i], items[j]) = (items[j], items[i]);   // (5)
```

```
            i++;
        }
    }
    (items[i], items[hi]) = (items[hi], items[i]);   // (6)
    return i;
}
```

```
2, 3, 4, 5, 7, 8, 9
```

- **(1)** Basisfall: Ein Bereich mit höchstens einem Element ist sortiert. `lo > hi` tritt auf, wenn das Pivot ganz am Rand gelandet ist und ein Teil leer bleibt.
- **(2)** Nach `Partition` steht das Pivot an Index `p` an seiner endgültigen Stelle.
- **(3)** Das Pivot selbst gehört zu keinem Teil mehr. Deshalb geht es mit `p - 1` und `p + 1` weiter.
- **(4)** Lomuto wählt das letzte Element des Bereichs als Pivot.
- **(5)** Die Tupel-Schreibweise vertauscht zwei Elemente ohne Hilfsvariable.
- **(6)** Das Pivot kommt an die Grenze zwischen den beiden Teilen.

Wie gut die Teile werden, hängt vom Pivot ab. Ist das Array schon sortiert, ist das letzte Element immer das größte. Der rechte Teil bleibt dann leer, der linke ist nur um ein Element kürzer. Die Rekursion wird so tief, wie das Array lang ist. Deshalb wählen echte Implementierungen das Pivot geschickter, z. B. als mittleren Wert aus erstem, mittlerem und letztem Element.

Mergesort und Quicksort im Vergleich

	Mergesort	**Quicksort (Lomuto)**
Teilen	in der Mitte, ohne Vergleich	Partitionieren um das Pivot
Zusammenfügen	zwei sortierte Hälften mischen	entfällt
stabil	ja	nein
in-place	nein	ja

Stabilität

Ein Sortierverfahren ist **stabil**, wenn Elemente mit gleichem Schlüssel ihre ursprüngliche Reihenfolge behalten. Das ist wichtig, wenn du nach mehreren Kriterien sortierst. Hast du eine Klassenliste schon nach Namen sortiert und sortierst sie danach stabil nach der Note, stehen innerhalb jeder Note die Namen weiterhin alphabetisch.

Mergesort ist stabil, weil der Merge-Schritt bei Gleichheit das Element aus der linken Hälfte nimmt (das `<=` bei (4)). Mit `<` statt `<=` wäre er es nicht mehr.

Quicksort mit Lomuto ist nicht stabil. Die Vertauschungen springen über andere Elemente hinweg. Ein Beispiel mit drei Einträgen, sortiert nach der Note:

```
vorher:          Anna 2   Ben 1   Clara 2      Pivot ist Clara (Note 2)
```

```
j = 0:          Anna 2 ist nicht kleiner, bleibt stehen
j = 1:          Ben 1 ist kleiner, tauscht mit Anna:   Ben 1   Anna 2   Clara 2
Pivot setzen:   Clara tauscht mit Anna:                Ben 1   Clara 2  Anna 2
```

Anna stand vor Clara, jetzt steht sie dahinter. In .NET ist das genauso: `Array.Sort` und `List<T>.Sort` sind laut Dokumentation **nicht** stabil, LINQ `OrderBy` dagegen schon.

In-place

Ein Verfahren arbeitet **in-place**, wenn es die Elemente im ursprünglichen Array umordnet und dafür nur wenige zusätzliche Variablen braucht. Quicksort vertauscht nur Elemente innerhalb von `items`. Mergesort legt dagegen bei jedem Aufruf neue Arrays für die Hälften und das Ergebnis an. Es gibt in-place-Varianten von Mergesort, sie sind aber deutlich komplizierter.

Backtracking

Du stehst in einem Labyrinth an einer Abzweigung. Du wählst einen Gang und gehst weiter. Landest du in einer Sackgasse, gehst du zurück bis zur letzten Abzweigung und probierst dort den nächsten Gang. Gibt es dort keinen mehr, gehst du noch eine Abzweigung weiter zurück. So findest du sicher einen Ausgang, wenn es einen gibt. Genau dieses Vorgehen heißt **Backtracking** (deutsch etwa *Zurückverfolgen*). Damit löst ein Programm Rätsel wie Sudoku, plant Stundenpläne oder findet Kombinationen, die eine Bedingung erfüllen.



Zum Selbst-Erarbeiten: der interaktive Backtracking-Trainer

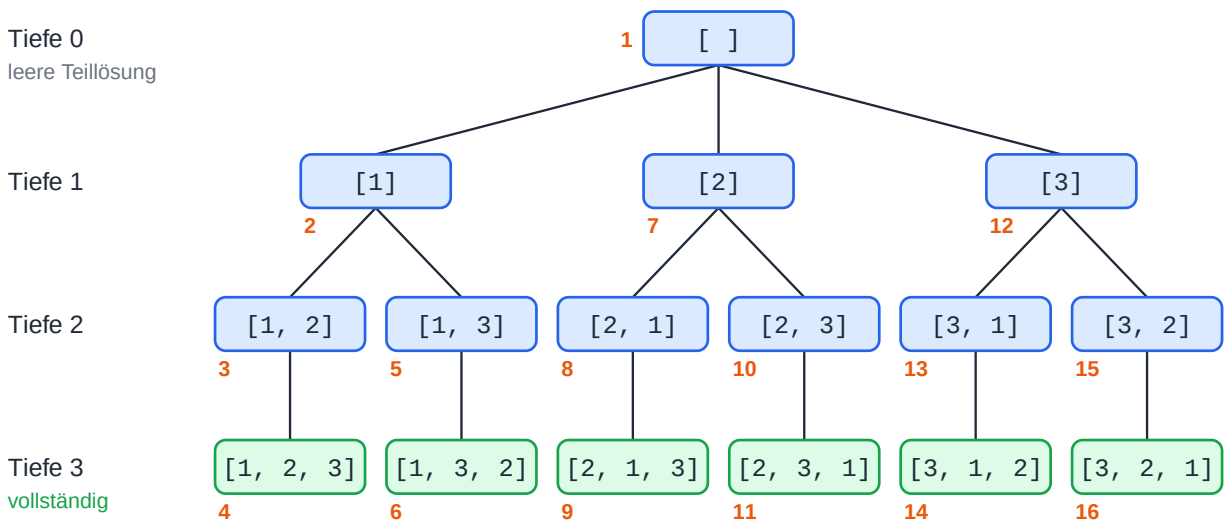
Im Backtracking-Trainer siehst du die Suche nach N Damen, während der Suchbaum mitwächst. Danach setzt du selbst Damen, siehst sofort jeden Konflikt und kannst dir den nächsten Schritt zeigen lassen. So verstehst du, wann und warum die Suche zurückgeht.

Teillösungen und der Suchbaum

Backtracking baut eine Lösung **Schritt für Schritt** auf. Was nach einigen Schritten feststeht, heißt **Teillösung**. In jedem Schritt gibt es mehrere Möglichkeiten, die Teillösung zu erweitern. Der Algorithmus probiert sie der Reihe nach aus:

1. **Setzen:** Eine Möglichkeit wird zur Teillösung hinzugefügt.
2. **Prüfen:** Ist die Teillösung noch gültig? Wenn nein, ist das eine Sackgasse.
3. **Weitersuchen:** Ist sie gültig, geht es mit dem nächsten Schritt weiter, bis die Lösung vollständig ist.
4. **Zurücknehmen:** Danach (oder bei einer Sackgasse) wird die Möglichkeit wieder entfernt und die nächste probiert.

Als erstes Beispiel bilden wir alle Anordnungen (*Permutationen*) der Zahlen 1, 2 und 3. Eine Teillösung ist eine Liste der schon gewählten Zahlen. Erweitern kannst du sie um jede Zahl, die noch nicht in der Liste steht. Vollständig ist sie, wenn alle drei Zahlen darin stehen.



Die orangen Zahlen zeigen die Reihenfolge, in der die Tiefensuche die Teillösungen besucht.
Nach [1, 2, 3] geht sie zurück über [1, 2] zu [1] und probiert dort [1, 3].

Alle Teillösungen zusammen bilden einen **Suchbaum**. Die Wurzel ist die leere Teillösung, jede Kante ist ein Schritt, die Blätter in der untersten Ebene sind die vollständigen Lösungen. Backtracking durchläuft diesen Baum mit einer **Tiefensuche**: Es geht immer zuerst so tief wie möglich und kehrt erst dann um (siehe auch Breiten- und Tiefensuche).

Der Baum wird nie als Datenstruktur gespeichert. Er entsteht nur durch die rekursiven Aufrufe: Jeder Aufruf ist ein Knoten, der Aufrufstack enthält genau den Weg von der Wurzel zum aktuellen Knoten. Deshalb braucht Backtracking wenig Speicher, auch wenn der Baum riesig ist.

Das Schema

Fast jeder Backtracking-Algorithmus folgt demselben Muster:

```
Suche(teillösung):
    wenn teillösung vollständig ist:
        Lösung melden
        zurück
    für jede Möglichkeit m, teillösung zu erweitern:
        wenn teillösung mit m gültig ist:        (prüfen)
            m zu teillösung hinzufügen           (setzen)
            Suche(teillösung)                    (weitersuchen)
            m wieder aus teillösung entfernen    (zurücknehmen)
```

Für ein neues Problem musst du vier Fragen beantworten:

Problem	Teillösung	vollständig, wenn	Möglichkeiten	ungültig, wenn
Permutationen	Liste gewählter Elemente	alle Elemente gewählt	jedes noch freie Element	nie

Problem	Teillösung	vollständig, wenn	Möglichkeiten	ungültig, wenn
N-Damen	Spalten der Damen in den ersten Zeilen	in jeder Zeile eine Dame	jede Spalte der nächsten Zeile	zwei Damen bedrohen sich
Teilsumme	gewählte Zahlen	Summe = Ziel	jede spätere Zahl	Summe > Ziel
Sudoku	teilweise ausgefülltes Gitter	kein Feld mehr leer	Ziffern 1–9 im nächsten leeren Feld	Ziffer doppelt in Zeile, Spalte oder Block

Ob du nur **eine** Lösung suchst oder **alle**, ist ein kleiner Unterschied: Für eine Lösung brichst du die Suche beim ersten Treffer ab. Für alle Lösungen sammelst du jede Lösung und suchst weiter.

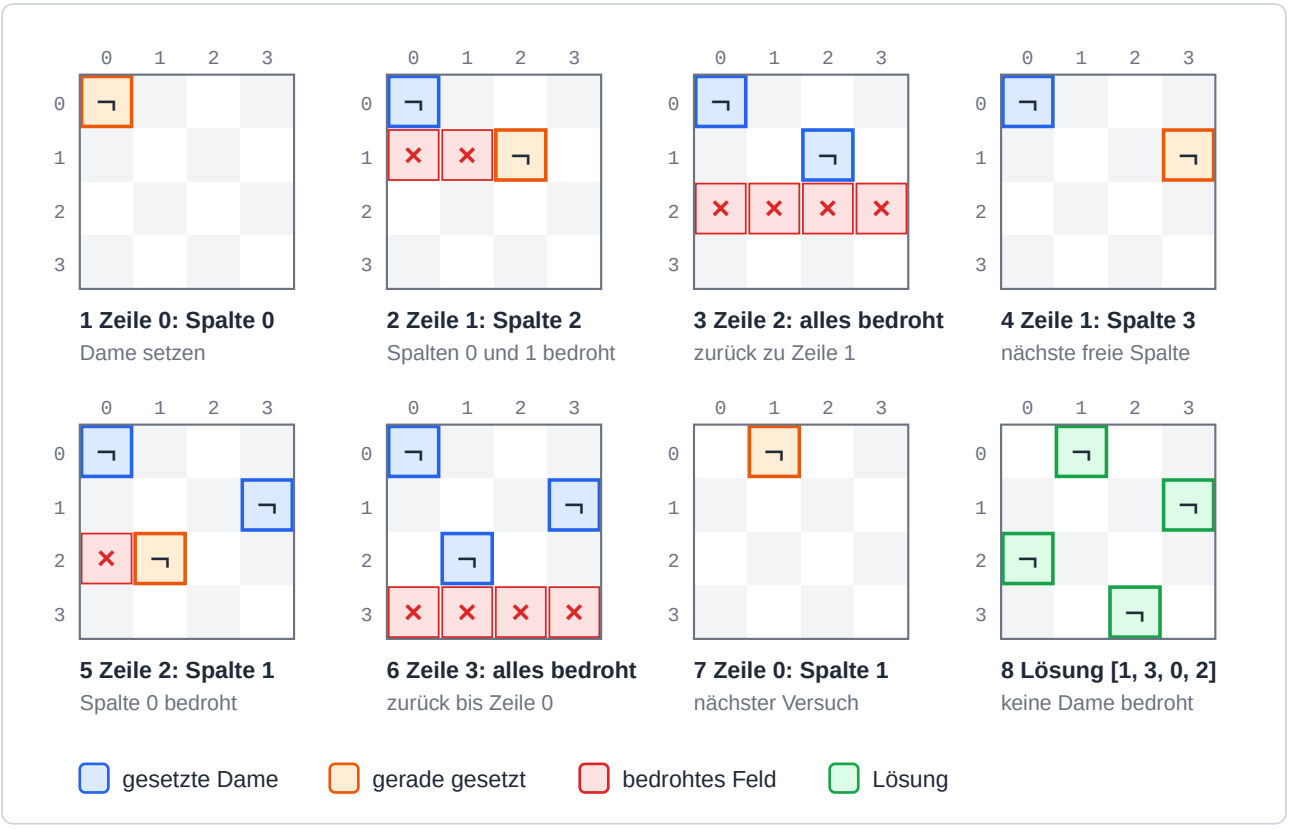
Das N-Damen-Problem

Auf einem Schachbrett mit N × N Feldern sollen N Damen so stehen, dass keine eine andere bedroht. Eine Dame bedroht alle Felder in ihrer Zeile, ihrer Spalte und ihren beiden Diagonalen. Auf dem normalen Schachbrett mit N = 8 gibt es 92 Lösungen.

Zwei Damen in derselben Zeile bedrohen sich sofort. In jeder Zeile steht also genau eine Dame. Diese Überlegung macht die Teillösung sehr einfach: Sie ist eine Liste von Spalten. `[1, 3]` bedeutet: In Zeile 0 steht die Dame in Spalte 1, in Zeile 1 in Spalte 3. Ein Schritt setzt eine Dame in die nächste freie Zeile. Zu prüfen sind nur noch zwei Bedingungen gegenüber allen schon gesetzten Damen:

- **gleiche Spalte:** Die Spalten sind gleich.
- **gleiche Diagonale:** Der Abstand der Spalten ist gleich dem Abstand der Zeilen. Die Damen `(0, 0)` und `(2, 2)` stehen z. B. auf einer Diagonale, ebenso `(1, 3)` und `(3, 1)`.

Die Grafik zeigt die Suche für N = 4 bis zur ersten Lösung. Zeilen und Spalten sind ab 0 nummeriert.



- **1–2:** Die erste Dame kommt in Spalte 0. In Zeile 1 sind Spalte 0 (gleiche Spalte) und Spalte 1 (Diagonale) bedroht, Spalte 2 ist frei.
- **3:** In Zeile 2 ist jedes Feld bedroht. Das ist eine Sackgasse. Die Dame aus Zeile 1 wird zurückgenommen.
- **4–5:** Die Dame in Zeile 1 probiert die nächste Spalte, also 3. In Zeile 2 ist jetzt Spalte 1 frei.
- **6:** In Zeile 3 ist wieder alles bedroht. Auch die restlichen Spalten von Zeile 2 und Zeile 1 führen zu nichts. Die Suche geht bis Zeile 0 zurück.
- **7–8:** Mit der ersten Dame in Spalte 1 gelingt es: `[1, 3, 0, 2]` ist eine Lösung.

N-Damen in C#

```
const int n = 4;
List<int[]> solutions = [];
Solve(n, [], solutions);

Console.WriteLine($"{n} Damen: {solutions.Count} Lösungen");
foreach (int[] solution in solutions)
{
    Console.WriteLine($"[{string.Join(", ", solution)}]");
    Print(solution);
}

static void Solve(int n, List<int> queens, List<int[]> solutions)
{
    int row = queens.Count;                          // (1)
    if (row == n)                                    // (2)
    {
```

```
            solutions.Add([.. queens]);
            return;
        }
        for (int col = 0; col < n; col++)
        {
            if (!IsSafe(queens, col)) { continue; }      // (3)
            queens.Add(col);                             // (4)
            Solve(n, queens, solutions);                 // (5)
            queens.RemoveAt(queens.Count - 1);           // (6)
        }
    }

    static bool IsSafe(List<int> queens, int col)
    {
        int row = queens.Count;
        for (int r = 0; r < row; r++)
        {
            int c = queens[r];
            // Same column, or same diagonal: the column distance equals the row distance.
            if (c == col || Math.Abs(c - col) == row - r) { return false; }
        }
        return true;
    }

    static void Print(int[] queens)
    {
        foreach (int col in queens)
        {
            Console.WriteLine(string.Join(" ", Enumerable.Range(0, queens.Length).Select(c
=> c == col ? "D" : ".")));
        }
        Console.WriteLine();
    }
```

```
4 Damen: 2 Lösungen
[1, 3, 0, 2]
. D . .
. . . D
D . . .
. . D .

[2, 0, 3, 1]
. . D .
D . . .
. . . D
. D . .
```

- **(1)** Die Länge der Teillösung ist gleichzeitig die Zeile, in die die nächste Dame kommt.
- **(2)** In jeder Zeile steht eine Dame, die Teillösung ist vollständig. `[.. queens]` legt eine **Kopie** an. Ohne Kopie stünde in `solutions` immer dieselbe Liste, die das Zurücknehmen später wieder leert.
- **(3)** Prüfen: Bedrohte Spalten werden übersprungen. Der Teilbaum unter diesem Feld wird

nie besucht.

- **(4)** Setzen.
- **(5)** Weitersuchen in der nächsten Zeile. Kommt der Aufruf zurück, sind alle Möglichkeiten unter dieser Teillösung abgearbeitet, egal ob er Lösungen gefunden hat oder nicht.
- **(6)** Zurücknehmen. Danach ist `queens` wieder genau so wie vor (4), und die Schleife probiert die nächste Spalte.

`IsSafe` muss die Zeilen nicht prüfen, weil jede Dame in einer eigenen Zeile steht. Der Ausdruck `row - r` ist immer positiv, deshalb braucht nur der Spaltenabstand `Math.Abs`.

Lösungen zählen

Interessiert nur die Anzahl, muss die Methode keine Lösungen speichern. Sie gibt stattdessen zurück, wie viele Lösungen unter der aktuellen Teillösung liegen:

```
for (int n = 1; n <= 10; n++)
{
    Console.WriteLine($"N = {n,2}: {CountSolutions(n, []),3} Lösungen");
}

static int CountSolutions(int n, List<int> queens)
{
    if (queens.Count == n) { return 1; }
    int count = 0;
    for (int col = 0; col < n; col++)
    {
        if (!IsSafe(queens, col)) { continue; }
        queens.Add(col);
        count += CountSolutions(n, queens);          // (1)
        queens.RemoveAt(queens.Count - 1);
    }
    return count;
}

static bool IsSafe(List<int> queens, int col)
{
    int row = queens.Count;
    for (int r = 0; r < row; r++)
    {
        int c = queens[r];
        if (c == col || Math.Abs(c - col) == row - r) { return false; }
    }
    return true;
}
```

```
N =  1:   1 Lösungen
N =  2:   0 Lösungen
N =  3:   0 Lösungen
N =  4:   2 Lösungen
N =  5:  10 Lösungen
```

```
N =  6:   4 Lösungen
N =  7:  40 Lösungen
N =  8:  92 Lösungen
N =  9: 352 Lösungen
N = 10: 724 Lösungen
```

- **(1)** Die Anzahl der Lösungen eines Knotens ist die Summe über alle seine Kinder. Ein vollständiges Blatt zählt 1, eine Sackgasse 0.

Für N = 2 und N = 3 gibt es keine Lösung. Backtracking stellt das fest, indem es den ganzen Suchbaum durchläuft, ohne ein vollständiges Blatt zu erreichen.

Pruning

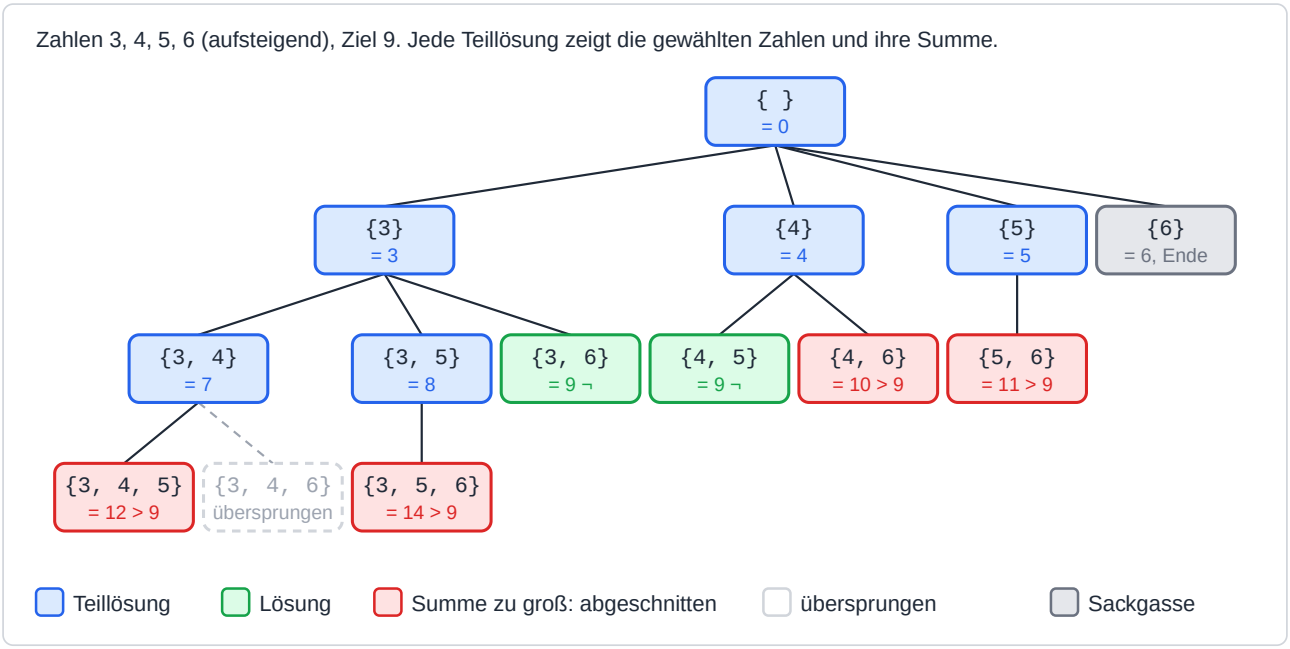
Das Abschneiden ganzer Teilbäume heißt **Pruning** (englisch für *Zurückschneiden* von Ästen). Es ist der eigentliche Grund, warum Backtracking mehr ist als stures Durchprobieren. Je früher eine Sackgasse erkannt wird, desto mehr Knoten fallen weg. Für N = 4 kannst du das nachzählen:

- Ohne jede Prüfung während des Aufbaus gibt es $4 \cdot 4 \cdot 4 \cdot 4 = 256$ vollständige Stellungen mit einer Dame pro Zeile. Erst am Ende würde jede einzelne geprüft.
- Mit der Prüfung nach jeder Dame probiert die Suche nach allen Lösungen 60 Felder aus und setzt nur 16-mal tatsächlich eine Dame.

Beispiel: Teilsummen

Beim **Teilsummenproblem** sind positive Zahlen und eine Zielsumme gegeben. Gesucht sind alle Auswahlen von Zahlen, deren Summe genau das Ziel ergibt. Für die Zahlen 3, 4, 5, 6 und das Ziel 9 sind das {3, 6} und {4, 5}.

Die Teillösung ist eine Auswahl von Zahlen. Damit keine Auswahl doppelt vorkommt ({3, 6} und {6, 3}), wird sie nur um Zahlen erweitert, die in der Liste **nach** der zuletzt gewählten Zahl stehen. Weil alle Zahlen positiv sind, wird die Summe beim Erweitern nur größer. Ist sie einmal über dem Ziel, kann darunter keine Lösung mehr liegen. Dieser Teilbaum wird abgeschnitten.



Sind die Zahlen aufsteigend sortiert, bringt das noch mehr: Ist {3, 4, 5} schon zu groß, ist {3, 4, 6} es erst recht. Die Schleife über die nächsten Zahlen kann deshalb ganz abbrechen, statt nur diesen einen Knoten zu überspringen. Eine zweite Regel schneidet von der anderen Seite: Reicht die Summe aller noch verfügbaren Zahlen nicht mehr bis zum Ziel, ist der Knoten ebenfalls eine Sackgasse. In der Grafik betrifft das z. B. {6}.

Pruning ändert nie das Ergebnis, sondern nur, wie viel vom Suchbaum besucht wird. Eine Pruning-Regel darf deshalb nur Teilbäume abschneiden, in denen **sicher** keine Lösung liegt.

Sudoku

Ein Sudoku ist ein klassisches Backtracking-Problem. Die Teillösung ist das teilweise ausgefüllte Gitter. Ein Schritt wählt das nächste leere Feld und probiert dort die Ziffern 1 bis 9. Eine Ziffer ist gültig, wenn sie in derselben Zeile, derselben Spalte und demselben 3 × 3-Block noch nicht vorkommt. Passt in einem Feld keine einzige Ziffer, ist das eine Sackgasse: Die zuletzt eingetragene Ziffer wird wieder gelöscht, und dort geht es mit der nächsten Ziffer weiter. Ist kein Feld mehr leer, ist das Rätsel gelöst.

Die vorgegebenen Ziffern schränken die Möglichkeiten stark ein, deshalb greift das Pruning bei einem Sudoku sehr früh. Wie viele Schritte die Suche braucht, hängt auch davon ab, in welcher Reihenfolge die leeren Felder drankommen. Das probierst du im Übungsprojekt selbst aus.

Zusatzmaterial

Im 25_Backtracking.html siehst du die Suche nach N Damen Schritt für Schritt, während der Suchbaum mitwächst. Danach setzt du selbst Damen, bekommst sofort Feedback zu Konflikten und kannst dir den nächsten Backtracking-Schritt zeigen lassen.

Greedy-Algorithmen

Viele Probleme bestehen aus einer Folge von Entscheidungen: Welche Münze gebe ich als Nächstes heraus? Welchen Termin trage ich als Nächstes in den Raumplan ein? Ein **Greedy-Algorithmus** (englisch *greedy* = gierig) trifft jede dieser Entscheidungen so, wie sie *im Moment* am besten aussieht. Er überlegt nicht, was später passiert, und er nimmt eine Entscheidung nie zurück.

Das macht Greedy-Algorithmen kurz und leicht verständlich. Die spannende Frage ist immer: Führt die Folge von lokal besten Entscheidungen auch zur besten Gesamtlösung? Manchmal ja, manchmal nein. In dieser Einheit lernst du beide Fälle kennen.

Das Prinzip

Ein Greedy-Algorithmus folgt immer demselben Schema:

```
Lösung = leer
solange das Problem nicht gelöst ist:
    wähle nach einer festen Regel den besten Kandidaten
    wenn er zur bisherigen Lösung passt: füge ihn hinzu
    sonst: verwirf ihn für immer
```

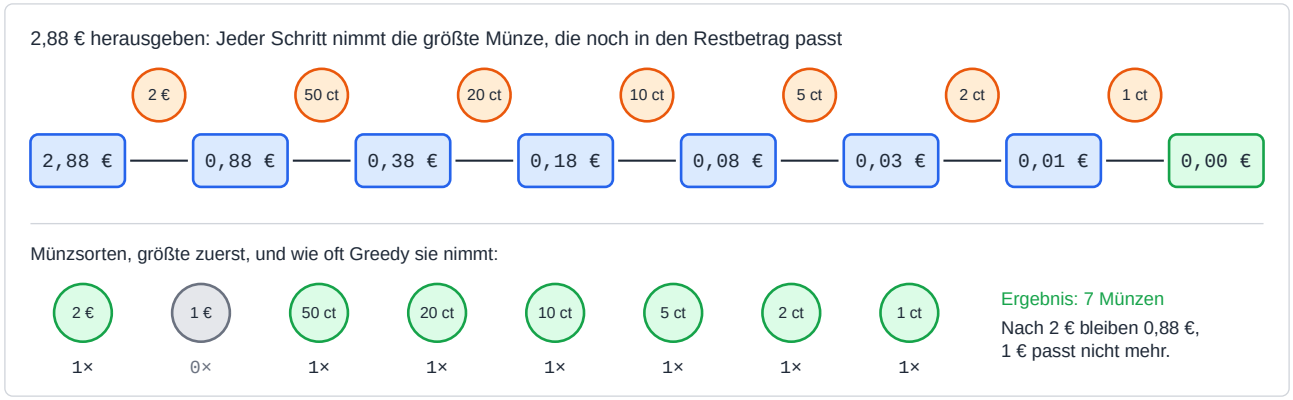
Zwei Dinge musst du dir für jedes Problem überlegen:

1. **Die Auswahlregel.** Was heißt „bester Kandidat“? Die größte Münze, der kürzeste Termin, der Termin, der am frühesten endet? Für dasselbe Problem gibt es oft mehrere Regeln, die vernünftig klingen.
2. **Ist das Ergebnis optimal?** Ein Greedy-Algorithmus liefert immer *irgendeine* Lösung (oder gar keine). Ob es die beste ist, hängt vom Problem und von der Regel ab. Das musst du begründen oder mit einem Gegenbeispiel widerlegen.

Im Unterschied zum Backtracking probiert Greedy keine Alternativen aus. Es gibt keinen Suchbaum, sondern nur einen einzigen Weg vom Anfang bis zum Ende.

Wechselgeld mit Euro-Münzen

An einer Kassa sollen 2,88 € Wechselgeld mit **möglichst wenigen Münzen** herausgegeben werden. Die Euro-Münzen sind 2 €, 1 €, 50 ct, 20 ct, 10 ct, 5 ct, 2 ct und 1 ct. Wahrscheinlich machst du das im Kopf schon greedy: Du nimmst immer die größte Münze, die noch in den Restbetrag passt.



Nach der 2-€-Münze bleiben 0,88 €. Die 1-€-Münze ist jetzt zu groß, also geht es mit 50 ct weiter. Am Ende stehen 7 Münzen. Mit weniger Münzen geht es nicht.

```
int[] euroCoins = [200, 100, 50, 20, 10, 5, 2, 1];             // (1)

List<int> change = MakeChange(288, euroCoins);
Console.WriteLine($"{string.Join(" + ", change)} = {change.Sum()} ct");
Console.WriteLine($"{change.Count} Münzen");

static List<int> MakeChange(int amount, int[] coins)
{
    List<int> result = [];
    foreach (int coin in coins.OrderDescending())               // (2)
    {
        while (amount >= coin)                                  // (3)
        {
            result.Add(coin);
            amount -= coin;
        }
    }
    if (amount != 0)                                            // (4)
    {
        throw new ArgumentException("Der Betrag lässt sich so nicht wechseln.");
    }
    return result;
}
```

```
200 + 50 + 20 + 10 + 5 + 2 + 1 = 288 ct
7 Münzen
```

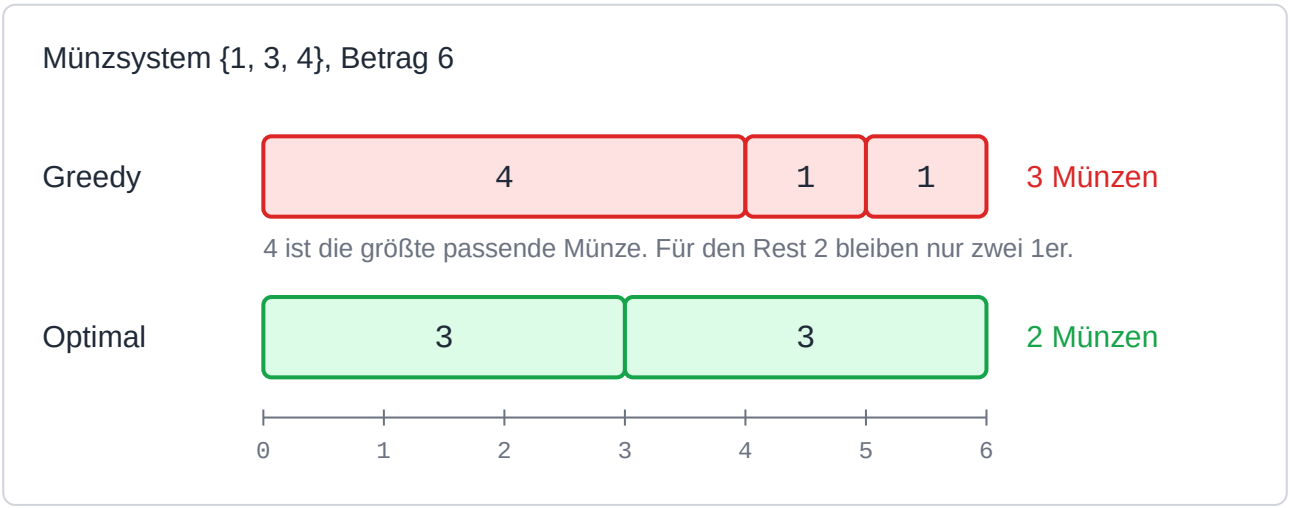
- **(1)** Die Beträge stehen in Cent als `int`. Mit `double` würden Rundungsfehler entstehen, denn 0,1 lässt sich im Binärsystem nicht exakt darstellen. Geldbeträge rechnest du deshalb immer in der kleinsten Einheit oder mit `decimal`.
- **(2)** Die Auswahlregel: größte Münze zuerst. `OrderDescending` sortiert die Münzen absteigend, egal in welcher Reihenfolge sie übergeben werden.
- **(3)** Dieselbe Münze kann mehrmals passen, etwa zweimal 2 € bei 4,50 €. Erst wenn sie zu groß ist, kommt die nächstkleinere Münze dran.
- **(4)** Bleibt ein Rest übrig, gibt es mit diesen Münzen keine Lösung. Bei den Euro-Münzen pas-

siert das nie, weil es eine 1-ct-Münze gibt.

Für das Euro-System liefert diese Regel **immer** die kleinste Anzahl an Münzen. Ein Münzsystem mit dieser Eigenschaft heißt *kanonisch*. Die meisten echten Währungen sind so gebaut, damit Kassierer ohne Nachdenken richtig herausgeben können. Selbstverständlich ist das aber nicht, wie der nächste Abschnitt zeigt.

Wenn Greedy danebenliegt: das Münzsystem {1, 3, 4}

Stell dir eine Währung mit Münzen zu 1, 3 und 4 vor. Es sollen 6 Einheiten herausgegeben werden. Greedy nimmt zuerst die größte Münze, die passt, also die 4. Für den Rest 2 bleiben nur zwei Einser. Das sind 3 Münzen. Es geht aber mit 2 Münzen: 3 + 3.



Rufst du `MakeChange(6, [1, 3, 4])` aus dem Programm oben auf, bekommst du:

```
4 + 1 + 1 = 6 ct
3 Münzen
```

Das Problem: Die 4 sieht im ersten Schritt am besten aus, weil sie den Restbetrag am stärksten verkleinert. Sie führt aber in eine Lage, aus der es nur noch schlecht weitergeht. Greedy merkt das nicht, denn es nimmt keine Entscheidung zurück.

Noch deutlicher wird es mit den Münzen {3, 5} und dem Betrag 9. Greedy nimmt die 5, danach eine 3 und bleibt auf dem Rest 1 sitzen. `MakeChange` wirft deshalb eine Exception, obwohl es mit 3 + 3 + 3 eine Lösung gibt.

Ein einziges Gegenbeispiel genügt, um zu zeigen, dass ein Greedy-Algorithmus nicht optimal ist. Umgekehrt beweisen noch so viele Beispiele, bei denen er funktioniert, *nicht*, dass er immer funktioniert. Dafür braucht es ein Argument wie im nächsten Abschnitt.

Aktivitätsauswahl: möglichst viele Termine in einem Raum

Für den Besprechungsraum deiner Schule gibt es an einem Tag neun Anfragen. Jede hat eine

Beginn- und eine Endzeit (in vollen Stunden). Zwei Termine dürfen sich nicht überschneiden. Endet ein Termin um 10 Uhr, darf der nächste aber um 10 Uhr beginnen. Gesucht ist ein Plan mit **möglichst vielen** Terminen. Wie lange die Termine dauern, ist egal.

Termin	A	B	C	D	E	F	G	H	I
Beginn	8	9	10	11	12	13	14	15	16
Ende	10	11	13	12	15	14	17	16	18

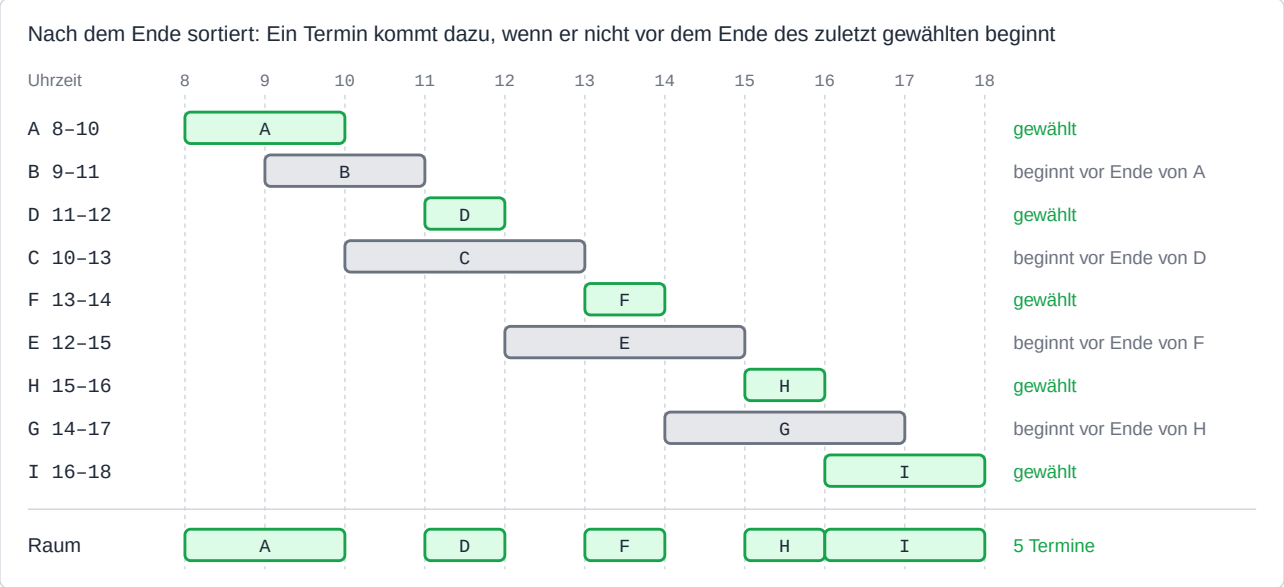
Welche Auswahlregel?

Für dieses Problem klingen mehrere Regeln vernünftig. Nicht jede liefert das Optimum:

- **Frühester Beginn zuerst.** Greedy wählt A (8–10), dann C (10–13), F (13–14) und G (14–17). Das sind 4 Termine. Ein langer Termin, der früh beginnt, blockiert den Raum für viele kurze.
- **Kürzester Termin zuerst.** Klingt gut, scheitert aber an drei Anfragen X (8–11), Y (10–12) und Z (11–14). Y ist am kürzesten und wird gewählt. Danach passt weder X noch Z, obwohl X und Z zusammen zwei Termine ergeben.
- **Frühestes Ende zuerst.** Diese Regel ist optimal, wie du gleich siehst.

Frühestes Ende zuerst

Du sortierst die Termine nach ihrer Endzeit und gehst sie der Reihe nach durch. Ein Termin kommt in den Plan, wenn er nicht vor dem Ende des zuletzt gewählten Termins beginnt.



A endet als Erstes und wird gewählt. B beginnt um 9, also vor dem Ende von A, und fällt weg. D beginnt um 11 und passt. So geht es weiter, bis der Plan A, D, F, H und I enthält. Mit frühestem Beginn waren es nur 4 Termine.

Warum ist das optimal? Der Termin, der als Erstes endet, lässt für den Rest des Tages **am meisten Zeit übrig**. Nimm an, jemand hat einen optimalen Plan, der mit einem anderen Termin beginnt. Dieser erste Termin endet gleichzeitig mit A oder später. Tauschst du ihn gegen A aus,

überschneidet sich A mit keinem der übrigen Termine im Plan, und die Anzahl bleibt gleich. Also gibt es auch einen optimalen Plan, der mit A beginnt. Dasselbe Argument gilt danach für den restlichen Tag ab 10 Uhr, und so weiter. Die gierige Wahl verbaut also nie etwas.

```
List<Activity> requests =
[
    new("A", 8, 10), new("B", 9, 11), new("C", 10, 13),
    new("D", 11, 12), new("E", 12, 15), new("F", 13, 14),
    new("G", 14, 17), new("H", 15, 16), new("I", 16, 18)
];

foreach (Activity a in SelectActivities(requests))
{
    Console.WriteLine($"{a.Name}: {a.Start,2}–{a.End} Uhr");
}

static List<Activity> SelectActivities(IEnumerable<Activity> activities)
{
    List<Activity> selected = [];
    int lastEnd = int.MinValue;                                  // (1)
    foreach (Activity activity in activities.OrderBy(a => a.End))   // (2)
    {
        if (activity.Start >= lastEnd)                           // (3)
        {
            selected.Add(activity);
            lastEnd = activity.End;
        }
    }
    return selected;
}

record Activity(string Name, int Start, int End);
```

```
A:  8–10 Uhr
D: 11–12 Uhr
F: 13–14 Uhr
H: 15–16 Uhr
I: 16–18 Uhr
```

- **(1)** Vor dem ersten Termin ist der Raum frei. `int.MinValue` sorgt dafür, dass der erste Termin in der sortierten Reihenfolge sicher passt.
- **(2)** Die Auswahlregel steckt allein in der Sortierung. Willst du eine andere Regel ausprobieren, tauschst du nur diesen Ausdruck, z. B. `OrderBy(a => a.Start)`.
- **(3)** `>=` erlaubt, dass ein Termin genau dann beginnt, wenn der vorige endet. Mit `>` müsste zwischen zwei Terminen eine Lücke bleiben.

Beachte, dass der Algorithmus **eine** optimale Lösung findet, nicht alle. Auch A, D, F, H und I ist nicht der einzige Plan mit 5 Terminen.

Wann funktioniert Greedy?

Greedy liefert das Optimum, wenn zwei Dinge zutreffen:

- **Die gierige Wahl ist sicher.** Es gibt immer eine optimale Lösung, die mit der Wahl beginnt, die die Regel trifft. Bei der Aktivitätsauswahl zeigt das Tauschargument genau das.
- **Der Rest ist wieder dasselbe Problem.** Nach der ersten Wahl bleibt ein kleineres Problem derselben Art übrig: Wechselgeld für den Restbetrag, Termine ab dem Ende des gewählten Termins.

Beim Münzsystem {1, 3, 4} ist der zweite Punkt erfüllt, der erste aber nicht: Keine optimale Lösung für 6 beginnt mit der 4. Greedy-Algorithmen sind deshalb dann ein gutes Werkzeug, wenn du begründen kannst, warum die gierige Wahl sicher ist. Kannst du das nicht, such zuerst nach einem Gegenbeispiel.

Ausblick: alle Möglichkeiten, aber jede nur einmal

Wie findest du für {1, 3, 4} trotzdem die beste Lösung? Du könntest wie beim Backtracking jede Münze als ersten Schritt ausprobieren und für den Rest wieder alle Münzen. Für den Betrag 6 sieht der Anfang so aus:

```
6 —1—> 5 —1—> 4 ...
       5 —3—> 2 ...
       5 —4—> 1 ...
6 —3—> 3 —1—> 2 ...
       3 —3—> 0
6 —4—> 2 —1—> 1 ...
```

Der Restbetrag 2 taucht hier schon dreimal auf, und jedes Mal würde der Suchbaum für 2 komplett neu durchlaufen. Bei größeren Beträgen wiederholen sich dieselben Restbeträge immer öfter.

Die Lösung: Du rechnest für jeden Restbetrag nur **einmal** aus, wie viele Münzen er mindestens braucht, und schreibst das Ergebnis in eine Tabelle. Braucht ein späterer Schritt denselben Restbetrag, liest du die Tabelle. Genau diese Idee heißt *dynamische Programmierung* und ist das Thema der nächsten Einheit.

Dynamische Programmierung

Am Ende der Greedy-Einheit stand ein Problem: Für das Münzsystem {1, 3, 4} findet Greedy nicht die beste Lösung. Probierst du dagegen alle Möglichkeiten durch, rechnest du dieselben Restbeträge immer wieder neu aus. **Dynamische Programmierung** (kurz *DP*) löst beides: Sie betrachtet alle Möglichkeiten, berechnet aber jedes Teilproblem nur **ein einziges Mal** und merkt sich das Ergebnis.

Der Name klingt nach Programmiersprache, kommt aber aus den 1950er-Jahren. *Programmierung* bedeutete damals so viel wie „Planung mit Tabellen“. Genau darum geht es: Du füllst eine Tabelle mit den Lösungen kleiner Teilprobleme und setzt daraus die Lösung des großen Problems zusammen.



Zum Selbst-Erarbeiten: der interaktive Tabellen-Trainer

Im Tabellen-Trainer siehst du, wie die Tabellen für Editierdistanz und Rucksack Zelle für Zelle entstehen. Danach füllst du selbst Tabellen für zufällige oder eigene Eingaben aus, und jede Zelle wird sofort geprüft. Das Füllen der Tabelle ist der Kern des Verfahrens, hier übst du es, so oft du willst.

Überlappende Teilprobleme: Fibonacci

Die Fibonacci-Zahlen sind so definiert:

```
F(0) = 0
F(1) = 1
F(n) = F(n − 1) + F(n − 2)     für n ≥ 2

n:     0  1  2  3  4  5  6   7   8   9  10
F(n):  0  1  1  2  3  5  8  13  21  34  55
```

Naiv rekursiv

Die Definition lässt sich direkt als rekursive Methode hinschreiben. Ein Zähler zeigt, wie oft sie aufgerufen wird:

```
int calls = 0;
Console.WriteLine($"Fib(5) = {Fib(5)}, Aufrufe: {calls}");
calls = 0;
Console.WriteLine($"Fib(6) = {Fib(6)}, Aufrufe: {calls}");

long Fib(int n)
{
    calls++;                                // (1)
    if (n < 2) { return n; }                // (2)
    return Fib(n - 1) + Fib(n - 2);         // (3)
```

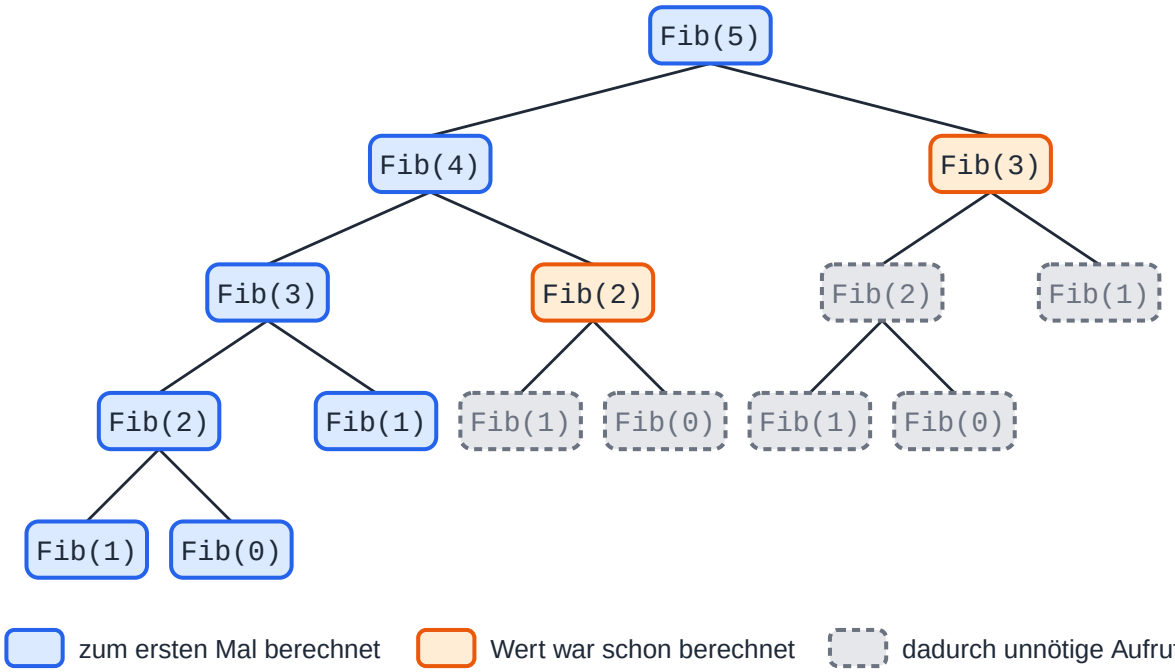
```
}
```

```
Fib(5) = 5, Aufrufe: 15
Fib(6) = 8, Aufrufe: 25
```

- **(1)** `Fib` ist hier eine lokale Funktion ohne `static`, damit sie die Variable `calls` der Top-Level-Statements verwenden kann.
- **(2)** F(0) und F(1) sind die Basisfälle.
- **(3)** Jeder andere Aufruf startet zwei weitere Aufrufe.

Für 6 verschiedene Werte F(0) bis F(5) braucht `Fib(5)` also 15 Aufrufe. Der Aufrufbaum zeigt, woher sie kommen:

Aufrufbaum von Fib(5): 15 Aufrufe, aber nur 6 verschiedene Werte



Mit Memo bleiben nur die blauen und orangen Aufrufe übrig: 9 statt 15.

`Fib(3)` wird zweimal berechnet, `Fib(2)` sogar dreimal, und jedes Mal mit dem ganzen Teilbaum darunter. Die Teilprobleme **überlappen** sich: Verschiedene Zweige des Baums brauchen dieselben Ergebnisse. Je größer n wird, desto öfter wiederholen sich dieselben Berechnungen. Schon bei `Fib(6)` sind es 25 Aufrufe für nur 7 verschiedene Werte.

Top-down mit Memo

Die Idee der **Memoization** (von *Memo*, Notiz): Bevor die Methode rechnet, schaut sie in einem `Dictionary` nach, ob sie das Ergebnis schon kennt. Nach dem Rechnen trägt sie es ein.

```
Dictionary<int, long> memo = [];
Console.WriteLine($"Fib(50) = {FibMemo(50, memo)}");
```

```
Console.WriteLine($"Einträge im Memo: {memo.Count}");

static long FibMemo(int n, Dictionary<int, long> memo)
{
    if (n < 2) { return n; }
    if (memo.TryGetValue(n, out long known)) { return known; }      // (1)
    long result = FibMemo(n - 1, memo) + FibMemo(n - 2, memo);     // (2)
    memo[n] = result;                                               // (3)
    return result;
}
```

```
Fib(50) = 12586269025
Einträge im Memo: 49
```

- **(1)** Ist F(n) schon bekannt, endet der Aufruf sofort. Das sind die orangen Knoten in der Grafik. Die grauen Teilbäume darunter fallen weg.
- **(2)** Die Rekursion ist dieselbe wie vorher. Nur ihr Ergebnis wird aufgehoben.
- **(3)** Das Memo enthält am Ende die Werte F(2) bis F(50), also 49 Einträge.

Für `Fib(5)` sind es mit Memo 9 statt 15 Aufrufe. Jeder Wert F(2) bis F(n) wird genau einmal berechnet, und das Memo wächst nie über n Einträge hinaus. Diese Variante heißt *top-down*: Sie beginnt beim großen Problem F(n) und steigt rekursiv zu den kleinen ab.

Bottom-up mit Tabelle

Man kann auch umgekehrt vorgehen: Zuerst die kleinen Teilprobleme lösen, dann die größeren. Dann braucht es keine Rekursion und keine Prüfung, ob ein Wert schon da ist. Bei F(i) sind F(i − 1) und F(i − 2) immer schon berechnet.

```
Console.WriteLine($"Fib(50) = {FibBottomUp(50)}");
Console.WriteLine($"Fib(90) = {FibTwoVariables(90)}");

static long FibBottomUp(int n)
{
    if (n < 2) { return n; }
    long[] table = new long[n + 1];                 // (1)
    table[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        table[i] = table[i - 1] + table[i - 2];     // (2)
    }
    return table[n];
}

static long FibTwoVariables(int n)
{
    long previous = 0, current = 1;                 // (3)
    for (int i = 0; i < n; i++)
    {
        (previous, current) = (current, previous + current);
```

```
    }
    return previous;
}
```

```
Fib(50) = 12586269025
Fib(90) = 2880067194370816120
```

- **(1)** `table[i]` enthält am Ende F(i). Das Array wird automatisch mit 0 initialisiert, also stimmt `table[0]` schon.
- **(2)** Die Schleife füllt die Tabelle von links nach rechts. Das ist die **Füllreihenfolge**: Jede Zelle braucht nur Zellen, die schon gefüllt sind.
- **(3)** Weil jeder Wert nur seine zwei Vorgänger braucht, reichen auch zwei Variablen statt der ganzen Tabelle. Sie enthalten zu Beginn F(0) und F(1) und rücken in jedem Durchlauf um eins weiter. Mit dem Tupel `(previous, current) = (⋯)` werden beide gleichzeitig zugewiesen.

Ein `long` reicht übrigens nur bis F(92). Danach läuft der Wert ohne Fehlermeldung über. Für größere Zahlen gibt es `System.Numerics.BigInteger`.

Die zwei Zutaten

Dynamische Programmierung funktioniert, wenn ein Problem zwei Eigenschaften hat:

- **Überlappende Teilprobleme.** Beim Zerlegen entstehen immer wieder dieselben Teilprobleme. Sonst gäbe es nichts zu merken. Bei Divide and Conquer (etwa Mergesort) sind die Teilprobleme dagegen verschieden, dort bringt ein Memo nichts.
- **Optimale Teilstruktur.** Bei Optimierungsproblemen setzt sich die beste Lösung aus den besten Lösungen der Teilprobleme zusammen. Das Beispiel im nächsten Abschnitt zeigt, was das heißt.

Für das Vorgehen gibt es ein Rezept:

1. Lege fest, was ein Teilproblem ist und was in einer Tabellenzelle steht.
2. Stelle eine **Rekursionsformel** auf: Wie berechnet sich eine Zelle aus kleineren Zellen?
3. Bestimme die **Basisfälle** und eine **Füllreihenfolge**, bei der jede benötigte Zelle schon gefüllt ist.
4. Lies das Ergebnis ab und rekonstruiere bei Bedarf, *wie* es zustande kam.

Optimales Wechselgeld für {1, 3, 4}

Zurück zum Gegenbeispiel aus der Greedy-Einheit. Gesucht ist die kleinste Anzahl an Münzen für den Betrag 6 mit den Münzen 1, 3 und 4.

1. **Teilproblem:** `minCoins[a]` ist die kleinste Anzahl an Münzen für den Betrag a.
2. **Rekursionsformel:** Die optimale Lösung für a endet mit *irgendeiner* Münze m. Der Rest a −

m muss dann selbst optimal gewechselt sein, sonst könnte man die ganze Lösung verbessern. Das ist die optimale Teilstruktur. Weil du nicht weißt, welche Münze die letzte ist, probierst du alle aus: `minCoins[a] = 1 + Minimum von minCoins[a − m]` über alle Münzen m ≤ a.

3. **Basisfall:** `minCoins[0] = 0`. Gefüllt wird von a = 1 aufwärts, denn a − m ist immer kleiner als a.

a	0	1	2	3	4	5	6
minCoins[a]	0	1	2	1	1	2	2
letzte Münze	–	1	1	3	4	1	3

Für a = 6 stehen drei Möglichkeiten zur Wahl: 1 + minCoins[5] = 3, 1 + minCoins[3] = 2 und 1 + minCoins[2] = 3. Am besten ist die Münze 3. Um die Münzen selbst zu bekommen, merkst du dir in einer zweiten Zeile die letzte Münze und gehst rückwärts: 6 → (Münze 3) → 3 → (Münze 3) → 0.

```
Console.WriteLine(string.Join(" + ", MakeOptimalChange(6, [1, 3, 4])));
Console.WriteLine(string.Join(" + ", MakeOptimalChange(9, [3, 5])));

static List<int> MakeOptimalChange(int amount, int[] coins)
{
    int[] minCoins = new int[amount + 1];                  // (1)
    int[] lastCoin = new int[amount + 1];
    for (int a = 1; a <= amount; a++)
    {
        minCoins[a] = int.MaxValue;                        // (2)
        foreach (int coin in coins)
        {
            if (coin <= a && minCoins[a - coin] != int.MaxValue
                && minCoins[a - coin] + 1 < minCoins[a])    // (3)
            {
                minCoins[a] = minCoins[a - coin] + 1;
                lastCoin[a] = coin;
            }
        }
    }
    if (minCoins[amount] == int.MaxValue)
    {
        throw new ArgumentException($"{amount} lässt sich mit diesen Münzen nicht
wechseln.");
    }
    List<int> result = [];
    for (int a = amount; a > 0; a -= lastCoin[a])           // (4)
    {
        result.Add(lastCoin[a]);
    }
    return result;
}
```

```
3 + 3
3 + 3 + 3
```

- **(1)** `minCoins[0]` ist durch die Initialisierung des Arrays schon 0, der Basisfall.
- **(2)** `int.MaxValue` steht für „noch keine Lösung gefunden“.
- **(3)** Eine Münze kommt nur in Frage, wenn sie passt und der Rest wechselbar ist. Bei Gleichstand bleibt die Münze, die zuerst gefunden wurde.
- **(4)** Die Rekonstruktion geht vom Betrag rückwärts und zieht jeweils die gemerkte letzte Münze ab.

Auch der zweite Fall aus der Greedy-Einheit klappt jetzt: Für 9 mit den Münzen {3, 5} findet DP die Lösung 3 + 3 + 3, an der Greedy gescheitert ist.

Das 0/1-Rucksackproblem

Für den Wandertag darf dein Rucksack höchstens 7 **kg** wiegen. Zur Auswahl stehen vier Gegenstände, jeder mit einem Gewicht und einem Nutzwert:

Gegenstand	**A**	**B**	**C**	**D**
Gewicht (kg)	1	3	4	5
Wert	1	4	5	7

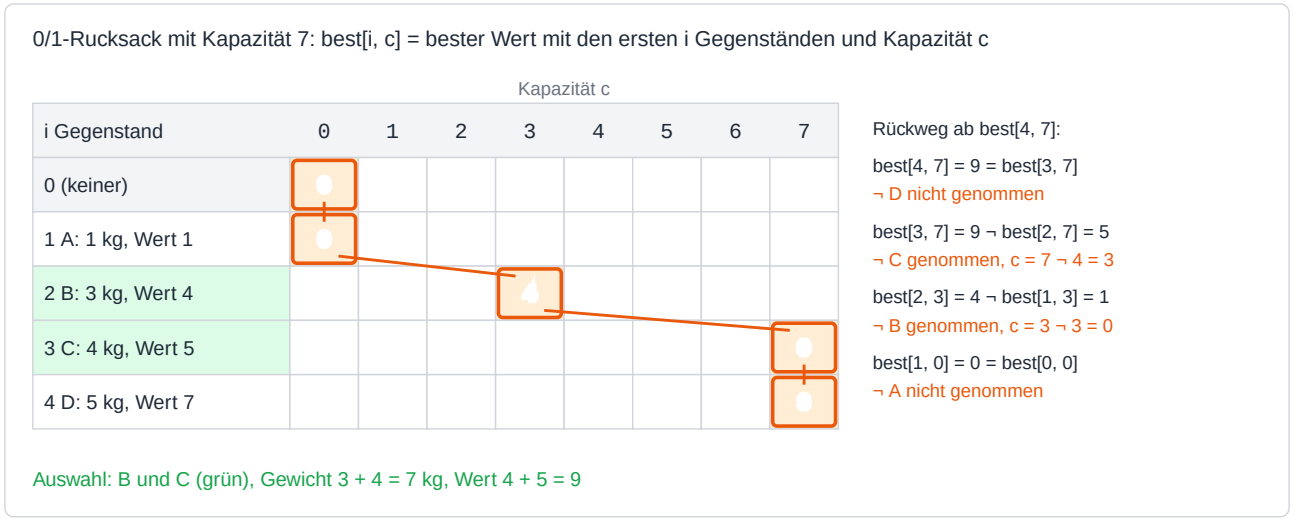
Jeden Gegenstand nimmst du ganz oder gar nicht mit, daher der Name *0/1*. Gesucht ist die Auswahl mit dem größten Gesamtwert, die nicht mehr als 7 kg wiegt.

Greedy liegt auch hier daneben. Wählt es nach dem Wert pro Kilogramm, nimmt es zuerst D (1,4 pro kg). Danach passt nur noch A, zusammen sind das 8. B und C wiegen zusammen genau 7 kg und haben den Wert 9.

Die Tabelle

1. **Teilproblem:** `best[i, c]` ist der größte Wert, den du mit den ersten i Gegenständen und einer Kapazität von c kg erreichen kannst.
2. **Rekursionsformel:** Für den Gegenstand i mit Gewicht w und Wert v gibt es zwei Möglichkeiten:
   - Du nimmst ihn nicht mit. Dann bleibt der Wert aus der Zeile darüber: `best[i − 1, c]`.
   - Du nimmst ihn mit (nur wenn w ≤ c). Dann bekommst du v und hast für die übrigen Gegenstände noch c − w kg: `v + best[i − 1, c − w]`.

     `best[i, c]` ist das Maximum der beiden Möglichkeiten.
3. **Basisfall:** Ohne Gegenstände ist der Wert 0, also ist Zeile 0 überall 0. Gefüllt wird Zeile für Zeile von oben nach unten, in jeder Zeile von links nach rechts. Jede Zelle braucht nur Zellen aus der Zeile darüber.



Ein Beispiel: `best[3, 7]` betrifft Gegenstand C (4 kg, Wert 5). Ohne C ist der Wert `best[2, 7] = 5`. Mit C bleiben 3 kg, also `5 + best[2, 3] = 5 + 4 = 9`. Das Maximum ist 9.

Die Auswahl rekonstruieren

Die Zelle rechts unten sagt dir, *dass* der beste Wert 9 ist, aber nicht, *welche* Gegenstände dazu gehören. Das findest du heraus, indem du von `best[4, 7]` aus rückwärts gehst:

- Steht in der Zeile darüber derselbe Wert, wurde der Gegenstand nicht gebraucht. Du gehst eine Zeile nach oben.
- Ist der Wert anders, gehört der Gegenstand zur Auswahl. Du gehst eine Zeile nach oben und um sein Gewicht nach links.

Der Weg in der Grafik ergibt B und C. Bei Gleichstand gilt die Regel „nicht mitgenommen“. Es kann daher noch andere Auswahlen mit demselben Wert geben, die dieser Rückweg nicht findet.

In C#

```
Item[] items = [new("A", 1, 1), new("B", 3, 4), new("C", 4, 5), new("D", 5, 7)];
int capacity = 7;

int[,] best = FillTable(items, capacity);
for (int i = 0; i <= items.Length; i++)
{
    string name = i == 0 ? "-" : items[i - 1].Name;
    IEnumerable<string> cells = Enumerable.Range(0, capacity + 1).Select(c => $"{best[
i, c],3}");
    Console.WriteLine($"{name} {string.Concat(cells)}");
}
Console.WriteLine($"Bester Wert: {best[items.Length, capacity]}");

static int[,] FillTable(Item[] items, int capacity)
{
    int[,] best = new int[items.Length + 1, capacity + 1];     // (1)
    for (int i = 1; i <= items.Length; i++)
```

```
    {
        Item item = items[i - 1];                                   // (2)
        for (int c = 0; c <= capacity; c++)
        {
            best[i, c] = best[i - 1, c];                            // (3)
            if (item.Weight <= c)
            {
                best[i, c] = Math.Max(best[i, c], item.Value + best[i - 1, c - item
.Weight]);   // (4)
            }
        }
    }
    return best;
}

record Item(string Name, int Weight, int Value);
```

```
-   0  0  0  0  0  0  0  0
A   0  1  1  1  1  1  1  1
B   0  1  1  4  5  5  5  5
C   0  1  1  4  5  6  6  9
D   0  1  1  4  5  7  8  9
Bester Wert: 9
```

- **(1)** Ein zweidimensionales Array mit einer Zeile mehr als Gegenstände. Zeile 0 bleibt 0.
- **(2)** Zeile i gehört zum Gegenstand `items[i - 1]`, weil die Zeilen bei 1 beginnen, das Array aber bei 0.
- **(3)** Zuerst die Möglichkeit „nicht mitnehmen“.
- **(4)** Passt der Gegenstand, wird geprüft, ob „mitnehmen“ besser ist.

Editierdistanz

Eine Rechtschreibprüfung soll zu einem Tippfehler passende Wörter vorschlagen. Dafür braucht sie ein Maß, wie ähnlich sich zwei Wörter sind. Die **Editierdistanz** (auch *Levenshtein-Distanz*) ist die kleinste Anzahl von Einzelschritten, mit denen ein Wort in ein anderes übergeht. Erlaubt sind drei Schritte, jeder kostet 1:

- einen Buchstaben **ersetzen** (`HAUS` → `MAUS`),
- einen Buchstaben **löschen** (`STRAND` → `STAND`),
- einen Buchstaben **einfügen** (`ROT` → `BROT`).

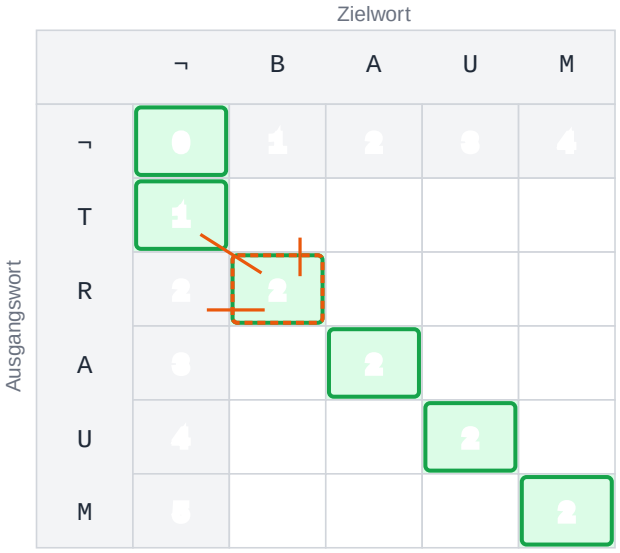
Die Tabelle

1. **Teilproblem:** `d[i, j]` ist die Editierdistanz zwischen den ersten i Buchstaben des Ausgangsworts und den ersten j Buchstaben des Zielworts. Die Zeilen gehören zum Ausgangswort, die Spalten zum Zielwort. Zeile und Spalte 0 stehen für das leere Wort ε.
2. **Basisfälle:** `d[i, 0] = i`, denn aus i Buchstaben wird das leere Wort durch i-mal Löschen.

`d[0, j] = j`, denn aus dem leeren Wort werden j Buchstaben durch j-mal Einfügen.

3. **Rekursionsformel:** Sind die Buchstaben i und j **gleich**, kostet die Zelle nichts extra: `d[i, j] = d[i − 1, j − 1]`. Sind sie **verschieden**, ist `d[i, j]` gleich 1 plus dem Minimum aus drei Nachbarzellen:
   - ↖ `d[i − 1, j − 1]`: Buchstabe i durch Buchstabe j **ersetzen**,
   - ↑ `d[i − 1, j]`: Buchstabe i **löschen**,
   - ← `d[i, j − 1]`: Buchstabe j **einfügen**.
4. **Füllreihenfolge:** Zeile für Zeile von oben nach unten, in jeder Zeile von links nach rechts. Die drei Nachbarn liegen links, oben und links oben und sind damit schon gefüllt.

Editierdistanz von TRAUM nach BAUM: d[i, j] für die ersten i bzw. j Buchstaben



Zelle d[2, 1] (TR nach B, gestrichelt):
R ¬ B, also 1 + Minimum aus
¬ ersetzen: d[1, 0] = 1
¬ löschen: d[1, 1] = 1
¬ einfügen: d[2, 0] = 2
d[2, 1] = 1 + 1 = 2

Rückweg (grün) ab d[5, 4] = 2:
- T löschen
- R durch B ersetzen
- A behalten
- U behalten
- M behalten

In der Grafik wird `TRAUM` in `BAUM` verwandelt. Für die gestrichelte Zelle `d[2, 1]` (aus `TR` soll `B` werden) sind R und B verschieden. Das Minimum der drei Nachbarn ist 1, also ist `d[2, 1] = 2`. Weiter unten in der Tabelle sind die Buchstaben A, U und M gleich, dort wird der Wert diagonal übernommen. Rechts unten steht das Ergebnis: Die Editierdistanz ist **2**.

Den grünen Rückweg findest du wie beim Rucksack: Du startest rechts unten und gehst jeweils zu der Nachbarzelle, aus der der Wert entstanden ist. Ein Schritt nach links oben bedeutet „behalten“ oder „ersetzen“, nach oben „löschen“ und nach links „einfügen“. Rückwärts gelesen ergibt sich: T löschen, R durch B ersetzen, A, U und M behalten.

Manchmal passen mehrere Nachbarn. Bei `d[2, 1]` hätte auch der Weg nach oben gepasst: T durch B ersetzen und R löschen. Auch das sind 2 Schritte. Beide Wege sind gleich gut.

Das Programm dazu schreibst du im Übungsprojekt selbst, zuerst naiv rekursiv und dann mit einer Tabelle.

Zusatzmaterial

Im 27_Dynamische_Programmierung.html siehst du, wie die Tabellen für Editierdistanz und

Rucksack Zelle für Zelle entstehen, und füllst danach selbst Tabellen für zufällige oder eigene Eingaben aus, wobei jede Zelle sofort geprüft wird.

Theoretische Informatik: Komplexität

Laufzeit messen

Zwei Programme lösen dieselbe Aufgabe und liefern dasselbe Ergebnis. Bei 1 000 Datensätzen merkst du keinen Unterschied. Bei einer Million Datensätzen ist das eine nach wenigen Hundertstelsekunden fertig, das andere nach eineinhalb Minuten. In dieser Einheit misst du solche Unterschiede selbst und lernst, wie man aus Messreihen abliest, wie schnell die Laufzeit mit der Eingabe wächst.

Das Beispiel: Duplikate finden

Eine Methode soll prüfen, ob in einem Array eine Zahl doppelt vorkommt. Die erste Idee: Du gehst die Zahlen durch und merkst dir alle bisherigen Zahlen in einer `List`. Bevor du eine Zahl einfügst, schaust du nach, ob sie schon drin ist. Die zweite Variante macht dasselbe mit einem `HashSet`.

Die beiden Methoden sind ein Ausschnitt aus dem Messprogramm weiter unten:

```
static bool HasDuplicateList(int[] data)
{
    List<int> seen = [];
    foreach (int x in data)
    {
        if (seen.Contains(x)) { return true; }      // (1)
        seen.Add(x);
    }
    return false;
}

static bool HasDuplicateHashSet(int[] data)
{
    HashSet<int> seen = [];
    foreach (int x in data)
    {
        if (!seen.Add(x)) { return true; }          // (2)
    }
    return false;
}
```

- **(1)** `List.Contains` vergleicht die Zahl der Reihe nach mit allen Einträgen der Liste. Je mehr Zahlen schon in der Liste sind, desto länger dauert jeder Aufruf.
- **(2)** `HashSet.Add` liefert `false`, wenn die Zahl schon enthalten ist. Ein `HashSet` findet eine Zahl über ihren Hashcode direkt, ohne die anderen Einträge anzusehen.

Beide Methoden liefern immer dasselbe Ergebnis. Ob eine schneller ist und um wie viel, sagt dir nur eine Messung.

Messen mit Stopwatch

Für Zeitmessungen gibt es in .NET die Klasse `Stopwatch` aus `System.Diagnostics`. Sie verwendet den genauesten Zeitgeber des Systems. `DateTime.Now` eignet sich dagegen nicht, denn es ändert sich unter Windows oft nur alle paar Millisekunden.

Das folgende Programm misst beide Methoden für n = 10^3 bis 10^6. Die Testdaten enthalten **kein** Duplikat. Das ist der ungünstigste Fall, denn beide Methoden müssen alle Zahlen ansehen. Füge unten die beiden Methoden von oben an, dann kompiliert das Programm. Den vollständigen Code findest du auch in Beispiele/30_DuplicateSearch.

```
using System.Diagnostics;

int[] warmUp = CreateData(1_000);                                   // (1)
for (int i = 0; i < 100; i++)
{
    HasDuplicateList(warmUp);
    HasDuplicateHashSet(warmUp);
}

int[] sizes = [1_000, 10_000, 100_000, 1_000_000];
Console.WriteLine($"{"n",10} {"List (ms)",12} {"HashSet (ms)",14}");
foreach (int n in sizes)
{
    int[] data = CreateData(n);
    double listMs = Measure(() => HasDuplicateList(data), n < 1_000_000 ? 5 : 1);   //
(2)
    double setMs = Measure(() => HasDuplicateHashSet(data), 5);
    Console.WriteLine($"{n,10} {listMs,12:F3} {setMs,14:F3}");
}

static double Measure(Func<bool> action, int repetitions)
{
    double best = double.MaxValue;
    for (int i = 0; i < repetitions; i++)
    {
        GC.Collect();                                               // (3)
        long start = Stopwatch.GetTimestamp();
        bool result = action();
        TimeSpan elapsed = Stopwatch.GetElapsedTime(start);         // (4)
        if (result) { throw new InvalidOperationException("Test data must not contain
duplicates."); }  // (5)
        best = Math.Min(best, elapsed.TotalMilliseconds);           // (6)
    }
    return best;
}

static int[] CreateData(int n)
{
    int[] data = Enumerable.Range(0, n).ToArray();
    new Random(42).Shuffle(data);                                   // (7)
    return data;
```

```
}
```

So sah das Ergebnis auf einem Intel Core i7-12700KF mit `dotnet run -c Release` aus. Auf deinem Rechner kommen andere Zahlen heraus, das Verhältnis zwischen den Spalten ist aber ähnlich.

```
         n    List (ms)   HashSet (ms)
      1000        0.059          0.013
     10000        4.083          0.073
    100000      244.988          1.506
   1000000    89432.308         28.334
```

Bei 1 000 Zahlen sind beide Varianten praktisch gleich schnell. Bei einer Million braucht die `List`-Variante 89 Sekunden, die `HashSet`-Variante 28 Millisekunden. Das ist ein Faktor von über 3 000, und er wird mit größerem n immer größer.

Richtig messen

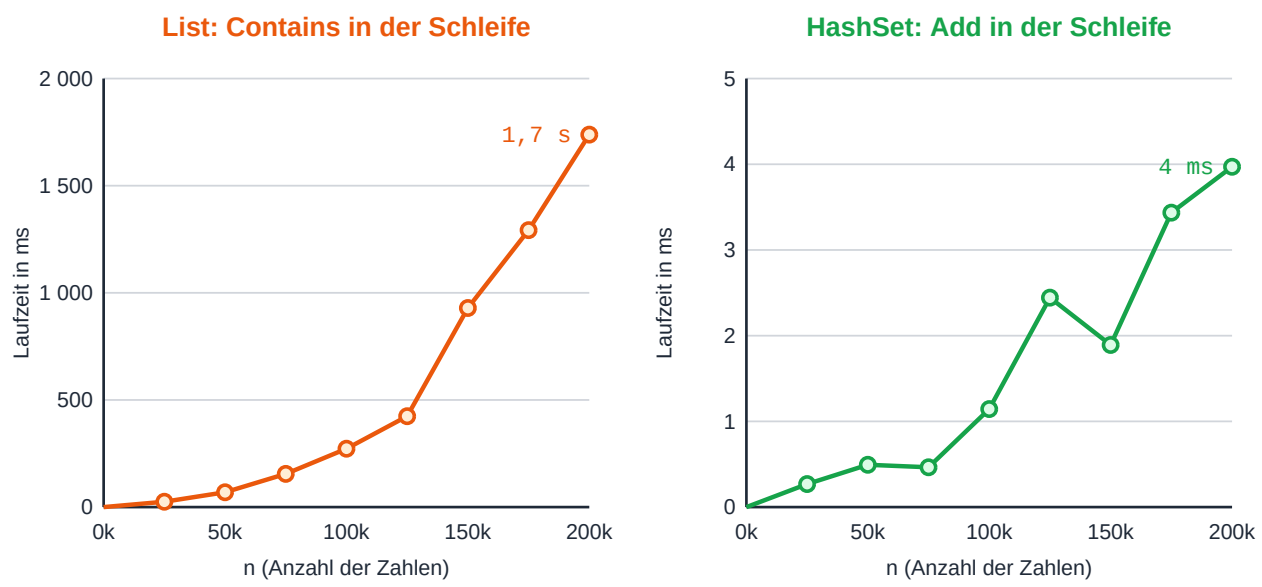
Die markierten Stellen im Programm sind kein Zufall. Ohne sie bekommst du Zahlen, die wenig aussagen.

- **Release-Build:** Starte Messungen immer mit `dotnet run -c Release` bzw. in Visual Studio mit der Konfiguration *Release* und ohne Debugger (Strg+F5). Im Debug-Build optimiert der Compiler nicht, damit du beim Debuggen jede Zeile und jede Variable siehst. Der Code ist dadurch oft um ein Vielfaches langsamer.
- **(1) Aufwärmen:** .NET übersetzt eine Methode erst beim ersten Aufruf mit dem *JIT-Compiler* (Just in Time) in Maschinencode. Zuerst entsteht schnell erzeugter, wenig optimierter Code. Wird eine Methode oft aufgerufen, übersetzt .NET sie im Hintergrund noch einmal mit allen Optimierungen (*Tiered Compilation*). Die ersten Aufrufe messen also auch den Compiler. Deshalb rufst du die Methoden vor der Messung einige Male mit kleinen Daten auf.
- **(2) Wiederholen:** Eine einzelne Messung schwankt, weil andere Programme laufen, der Prozessor den Takt ändert oder Daten noch nicht im Cache liegen. Deshalb misst du mehrmals. Nur die `List`-Variante mit einer Million Zahlen misst das Programm einmal. Sie dauert so lange, dass kleine Störungen keine Rolle spielen.
- **(3) Garbage Collector:** Der GC räumt nicht mehr benötigte Objekte weg und hält dafür das Programm kurz an. `GC.Collect()` vor der Messung sorgt dafür, dass er nicht mitten in der Messung Müll aus früheren Durchläufen wegräumt.
- **(4) Zeit messen:** `Stopwatch.GetTimestamp()` liest den Zeitgeber aus, `Stopwatch.GetElapsedTime` berechnet die vergangene Zeit als `TimeSpan`. Gleichwertig ist `Stopwatch.StartNew()` mit `Stop()` und `Elapsed`.
- **(5) Ergebnis verwenden:** Wird ein Ergebnis nie verwendet, darf der Compiler im Extremfall die Berechnung weglassen. Die Prüfung verhindert das und stellt nebenbei sicher, dass die Testdaten wirklich den gewünschten Fall abbilden.
- **(6) Minimum statt Mittelwert:** Störungen machen eine Messung nur langsamer, nie schneller. Der schnellste Lauf ist deshalb der beste Schätzwert für die Laufzeit des Codes selbst. Auch der Median ist üblich.

- **(7) Feste Testdaten:** Ein fester Seed liefert bei jedem Start dieselbe Reihenfolge. So sind Messungen wiederholbar und vergleichbar.

Messreihen

Vier Messpunkte sagen wenig über die Form der Kurve. Misst du dieselben Methoden für n = 25 000, 50 000, …, 200 000, kannst du die Laufzeit über n zeichnen. Die Werte stammen vom selben Rechner wie oben, jeweils der schnellste von mehreren Läufen:



Achtung: Die y-Achsen sind verschieden. Links reicht sie bis 2 000 ms, rechts bis 5 ms.

Die Kurve der `List`-Variante biegt sich deutlich nach oben. Verdoppelt sich n von 100 000 auf 200 000, wird die Laufzeit mehr als sechsmal so groß. Die `HashSet`-Variante bleibt dagegen bei wenigen Millisekunden. Ihre Messpunkte springen sichtbar: Bei so kurzen Laufzeiten fallen Störungen und Cache-Effekte stark ins Gewicht, die Form der Kurve lässt sich kaum ablesen. Hier hilft der Verdopplungstest mit größeren n.

Die Kurven verraten auch den Grund. Die `List`-Variante ruft n-mal `Contains` auf, und jeder Aufruf durchsucht eine Liste, die mit n wächst. Die Arbeit wächst also mit n · n. Die `HashSet`-Variante macht für jede Zahl ungefähr gleich viel Arbeit, zusammen also n-mal so viel.

Der Verdopplungstest

Aus einer Kurve die Art des Wachstums abzulesen, ist ungenau. Einfacher ist der **Verdopplungstest**: Du verdoppelst n immer wieder und schaust, um welchen Faktor die Laufzeit steigt.

- Wächst die Laufzeit linear, also wie $a \cdot n$, dann ist $T(2n) / T(n) = 2$.
- Wächst sie quadratisch, also wie $a \cdot n^2$, dann ist $T(2n) / T(n) = 4$.
- Allgemein: Wächst sie wie $a \cdot n^k$, dann ist $T(2n) / T(n) = 2^k$.

Die Konstante a kürzt sich weg. Sie hängt vom Rechner ab, der Faktor 2^k aber nicht. Das fol-

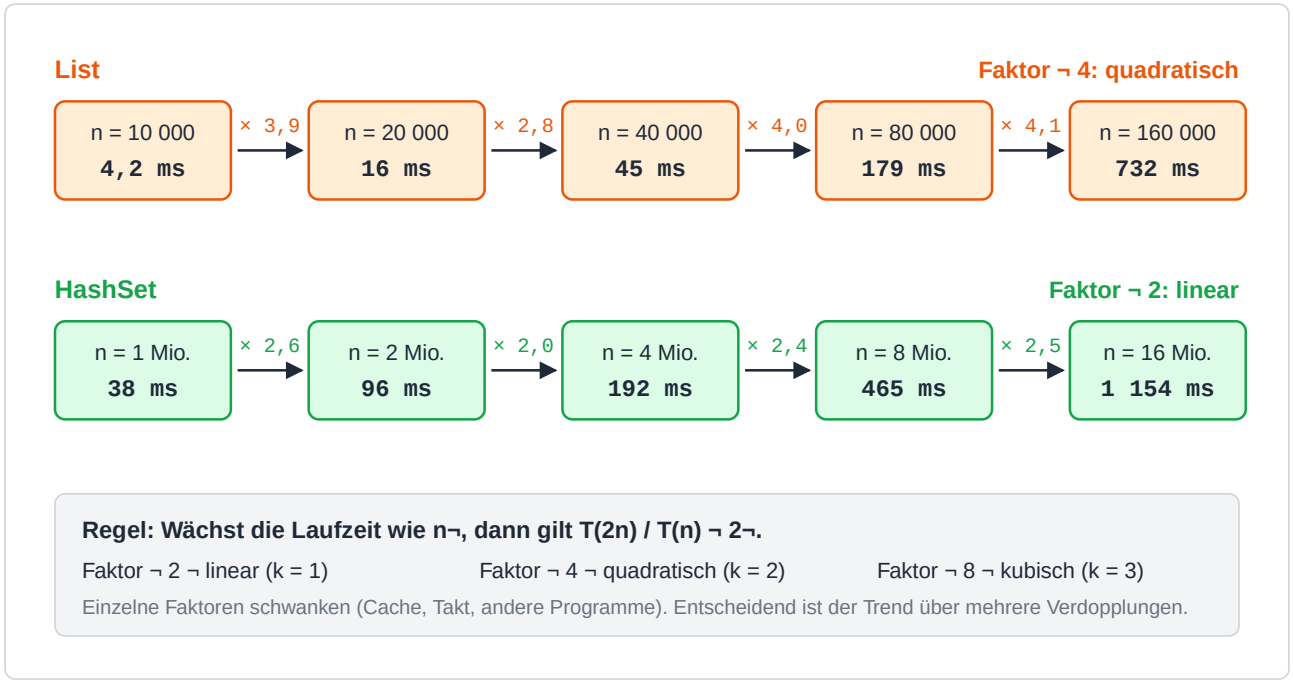
gende Fragment ergänzt das Messprogramm. Es verwendet `Measure` und `CreateData` von oben und ersetzt die Schleife über `sizes`.

```
DoublingTest("List", HasDuplicateList, 10_000, 6);
DoublingTest("HashSet", HasDuplicateHashSet, 1_000_000, 5);

static void DoublingTest(string name, Func<int[], bool> method, int startN, int steps)
{
    Console.WriteLine($"{name}:");
    double previous = 0;
    for (int i = 0, n = startN; i < steps; i++, n *= 2)
    {
        int[] data = CreateData(n);
        double ms = Measure(() => method(data), 5);
        string ratio = previous > 0 ? $"{ms / previous:F2}" : "";     // (1)
        Console.WriteLine($"{n,10} {ms,10:F2} ms {ratio,6}");
        previous = ms;
    }
}
```

- **(1)** Der Quotient aus der aktuellen und der vorherigen Messung ist der Faktor T(2n) / T(n).

```
List:
     10000       4.22 ms
     20000      16.23 ms   3.85
     40000      45.09 ms   2.78
     80000     179.04 ms   3.97
    160000     731.56 ms   4.09
    320000    4169.53 ms   5.70
HashSet:
   1000000      37.55 ms
   2000000      96.40 ms   2.57
   4000000     191.53 ms   1.99
   8000000     465.44 ms   2.43
  16000000    1153.63 ms   2.48
```



Bei der `List` liegt der Faktor meist bei 4, die Laufzeit wächst also quadratisch. Beim `HashSet` liegt er um 2 bis 2,5, das ist ungefähr linear. Einzelne Werte weichen ab, etwa 2,78 und 5,70 bei der `List`. Solche Ausreißer entstehen durch Caches, Taktänderungen und andere Programme. Entscheidend ist der Trend über mehrere Verdopplungen.

Mit dem Faktor kannst du auch **vorhersagen**. Eine Million ist das 6,25-Fache von 160 000. Bei quadratischem Wachstum steigt die Laufzeit um $6{,}25^2 \approx 39$, also auf 732 ms · 39 ≈ 29 s. Gemessen wurden 89 s. Die Vorhersage stimmt in der Größenordnung, liegt aber um den Faktor 3 daneben. Bei großen Listen passen die Daten nicht mehr in den schnellen Cache des Prozessors, und jeder Vergleich wird teurer.

Grenzen der Messung

Messungen sind unverzichtbar, wenn du wissen willst, wie schnell dein Programm wirklich ist. Als Maß für einen **Algorithmus** taugen sie aber nur bedingt:

- **Die Zahlen hängen vom Rechner ab.** Prozessor, Cache, Speicher, Betriebssystem und .NET-Version beeinflussen das Ergebnis. Eine Laufzeit in Millisekunden kannst du nicht mit der eines Kollegen vergleichen.
- **Die Zahlen hängen von den Daten ab.** Steht das Duplikat gleich am Anfang, sind beide Varianten sofort fertig. Eine Messung gilt nur für genau die Daten, mit denen du gemessen hast.
- **Große n lassen sich nicht messen.** Für $n = 10^9$ bräuchte die `List`-Variante bei quadratischem Wachstum etwa $89\text{ s} \cdot 1\,000^2 \approx 9 \cdot 10^7$ s, also rund drei Jahre. Das kannst du nur hochrechnen.
- **Effekte überlagern sich.** Wie die Vorhersage oben zeigt, verändern Caches den Faktor. Bei sehr kurzen Laufzeiten geht das Ergebnis im Rauschen unter.
- **Messen setzt fertigen Code voraus.** Vor der Implementierung kannst du nichts messen. Gerade dann willst du aber wissen, ob ein Ansatz für große Datenmengen taugt.

Was über alle Rechner hinweg gleich bleibt, ist der Faktor beim Verdoppeln, also die **Art des Wachstums**. Sie hängt nur davon ab, wie viele Schritte der Algorithmus abhängig von n braucht. Für dieses maschinenunabhängige Maß gibt es eine eigene Schreibweise, die O-Notation.

Genauer messen mit BenchmarkDotNet

Die Stopwatch-Messung reicht, um große Unterschiede zu sehen. Für kleine Codeteile, bei denen es um Mikrosekunden geht, verwendest du das Paket *BenchmarkDotNet*. Es wärmt automatisch auf, wiederholt die Messung so oft, bis das Ergebnis statistisch stabil ist, und gibt Mittelwert, Streuung und belegten Speicher aus. Wie du ein Benchmark-Projekt anlegst, steht im Kapitel Benchmarks mit Benchmark.NET.

Für Messreihen ist das Attribut `[Params]` praktisch. Es steht auf einem öffentlichen Feld oder einer Property, z. B. `[Params(1_000, 10_000, 100_000)] public int N;`. BenchmarkDotNet führt dann jeden Benchmark für jeden Wert von `N` aus. Testdaten erzeugst du in einer Methode mit dem Attribut `[GlobalSetup]`, damit das Erzeugen nicht mitgemessen wird.

Die O-Notation

Beim Messen der Laufzeit hast du gesehen: Die Zahlen hängen vom Rechner, vom Build, vom Cache und von anderen Programmen ab. Trotzdem war eines immer gleich: Verdoppelt sich n, braucht die Variante mit `List` etwa viermal so lange, die mit `HashSet` etwa doppelt so lange. Genau diese Aussage über das **Wachstum** ist unabhängig vom Rechner. Die O-Notation ist die Sprache, in der man sie aufschreibt.



Zum Selbst-Erarbeiten: der interaktive Trainer zur O-Notation

Im Trainer zur O-Notation vergleichst du die Wachstumsfunktionen in einem Diagramm und rechnest Schrittzahlen in echte Laufzeiten um. Danach ordnest du Funktionen nach ihrem Wachstum, auch knifflige Paare wie n log n und n^1,5. So bekommst du ein Gefühl dafür, was „schnell“ wirklich heißt.

Schritte zählen

Statt Millisekunden zählen wir **Schritte**: Vergleiche, Zuweisungen, Rechenoperationen. Wie lange ein Schritt dauert, hängt vom Rechner ab. Wie viele Schritte nötig sind, hängt nur vom Algorithmus und von der Eingabe ab.

Das folgende Programm sucht wie die `List`-Variante aus der letzten Einheit nach Duplikaten. Es vergleicht jedes Element mit allen Elementen davor und zählt dabei die Vergleiche.

```
int[] sizes = [10, 100, 1_000, 10_000];
foreach (int n in sizes)
{
    int[] data = Enumerable.Range(0, n).ToArray();      // (1)
    long comparisons = CountComparisons(data);
    Console.WriteLine($"n = {n,6}: {comparisons,10} Vergleiche, n²/2 = {(long)n * n /
2,10}");
}

static long CountComparisons(int[] data)
{
    long comparisons = 0;
    for (int i = 0; i < data.Length; i++)
    {
        for (int j = 0; j < i; j++)                     // (2)
        {
            comparisons++;
            if (data[i] == data[j]) { return comparisons; }
        }
    }
    return comparisons;
}
```

```
n =     10:         45 Vergleiche, n²/2 =         50
```

```
n =    100:       4950 Vergleiche, n²/2 =       5000
n =   1000:     499500 Vergleiche, n²/2 =     500000
n =  10000:   49995000 Vergleiche, n²/2 =   50000000
```

- **(1)** Die Zahlen 0 bis n − 1 enthalten kein Duplikat. Die Methode muss also alle Paare vergleichen.
- **(2)** Element i wird mit i Elementen verglichen. Zusammen sind das 0 + 1 + 2 + … + (n − 1) Vergleiche.

Die Summe 0 + 1 + … + (n − 1) ergibt n(n − 1)/2. Die Methode braucht also

T(n) = ½ n² − ½ n Vergleiche.

Der dominante Term

Schon bei n = 1 000 ist −½ n nur noch ein Tausendstel von ½ n². Je größer n wird, desto unwichtiger wird dieser Term. Deshalb behält man nur den **dominanten Term** ½ n². Auch den Faktor ½ lässt man weg. Er hängt davon ab, was man als einen Schritt zählt, und zwei Rechner unterscheiden sich ohnehin um einen konstanten Faktor. Übrig bleibt: Die Methode wächst **quadratisch**, man schreibt O(n²).

Die O-Notation informell

f ∈ O(g) heißt: **f wächst höchstens so schnell wie g**, wenn man konstante Faktoren ignoriert und nur große n betrachtet.

Schritte T(n)	Wachstumsklasse	Warum
3n + 7	O(n)	Der dominante Term ist 3n, ohne Faktor bleibt n.
½ n² − ½ n	O(n²)	Der dominante Term ist ½ n².
5 n log₂ n + 20n	O(n log n)	n log n wächst schneller als n.
1 000 000	O(1)	Eine Konstante, egal wie groß, wächst gar nicht.
$2^n + n^{100}$	$O(2^n)$	Für große n ist 2^n viel größer als jedes Polynom.

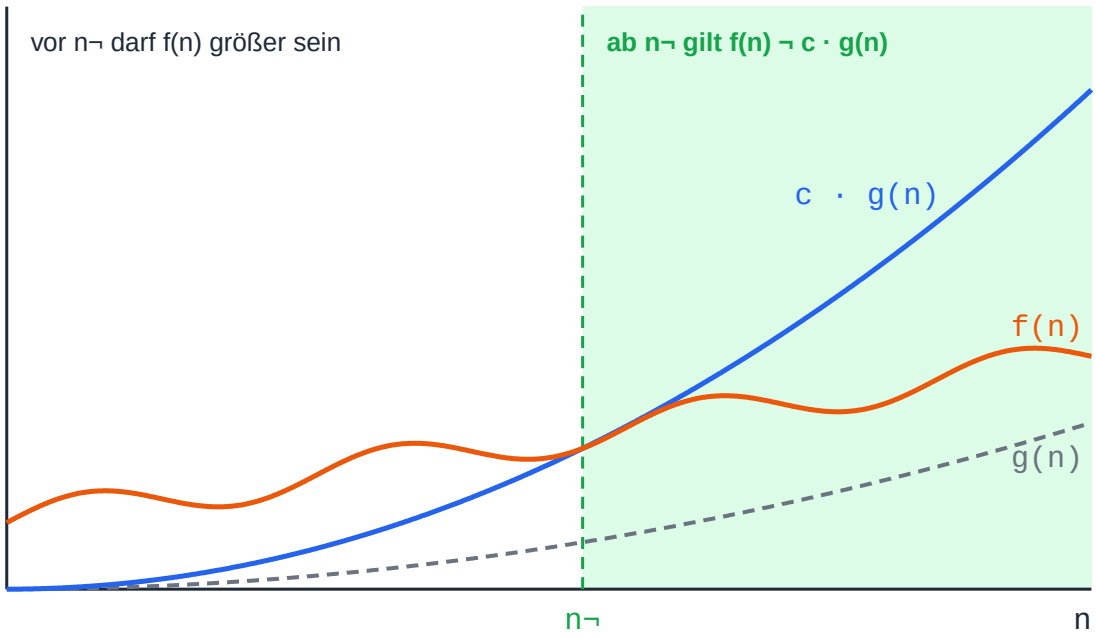
Man liest O(n²) als „O von n Quadrat“ oder „Ordnung n Quadrat“. Oft schreibt man auch T(n) = O(n²). Das Gleichheitszeichen ist dabei nur eine Schreibweise: O(n²) ist eine Menge von Funktionen, und T gehört dazu.

Die formale Definition

„Für große n“ und „bis auf einen Faktor“ lassen sich genau fassen:

> f ∈ O(g) genau dann, wenn es eine Konstante c > 0 und eine Zahl n_0 gibt, sodass **f(n) ≤ c ·**

> **g(n) für alle n ≥ n₀** gilt.



Der Faktor c streckt g(n) nach oben. Ab n¬ bleibt f(n) für immer unter c · g(n).

Der Faktor c erlaubt, g nach oben zu strecken. Die Zahl n_0 erlaubt, kleine n zu ignorieren. Vor n_0 darf f machen, was es will. Ab n_0 muss f für immer unter c · g bleiben.

Beispiel: $4n + 10 \in O(n)$. Wir suchen c und n_0 mit $4n + 10 \leq c \cdot n$.

- Mit c = 5 wird daraus $4n + 10 \leq 5n$, also $10 \leq n$. Das gilt für alle $n \geq 10$, also $n_0 = 10$.
- Mit c = 14 gilt $4n + 10 \leq 4n + 10n = 14n$ schon für alle $n \geq 1$.

Du musst also nur **ein** passendes Paar (c, n_0) finden. Ein größeres c erlaubt meist ein kleineres n_0.

Gegenbeispiel: $n^2 \notin O(n)$. Angenommen, es gäbe c und n_0 mit $n^2 \leq c \cdot n$ für alle $n \geq n_0$. Teilt man durch n, steht da $n \leq c$. Das stimmt aber nicht für $n = c + 1$ oder größer. Egal welches c du wählst: n^2 überholt $c \cdot n$ irgendwann.

Ω und Θ

O ist eine **obere Schranke**. Für untere Schranken und für genaue Angaben gibt es zwei weitere Zeichen:

Schreibweise	Bedeutung	Definition (für alle $n \geq n_0$)	Vergleich
$f \in O(g)$	f wächst höchstens so schnell wie g	$f(n) \leq c \cdot g(n)$	≤

Schreibweise	Bedeutung	Definition (für alle $n \geq n_0$)	Vergleich
$f \in \Omega(g)$	f wächst mindestens so schnell wie g	$f(n) \geq c \cdot g(n)$	$\geq$
$f \in \Theta(g)$	f wächst genau so schnell wie g	$f \in O(g)$ und $f \in \Omega(g)$	=

Für die Duplikatsuche gilt $\frac{1}{2} n^2 - \frac{1}{2} n \in \Theta(n^2)$. Es stimmt aber auch $\frac{1}{2} n^2 - \frac{1}{2} n \in O(n^3)$, denn n^3 ist ebenfalls eine obere Schranke, nur eine schlechte. In der Praxis sagt man „O“ und meint fast immer die **kleinste** obere Schranke, also eigentlich Θ.

Rechenregeln

Mit wenigen Regeln vereinfachst du jeden Ausdruck:

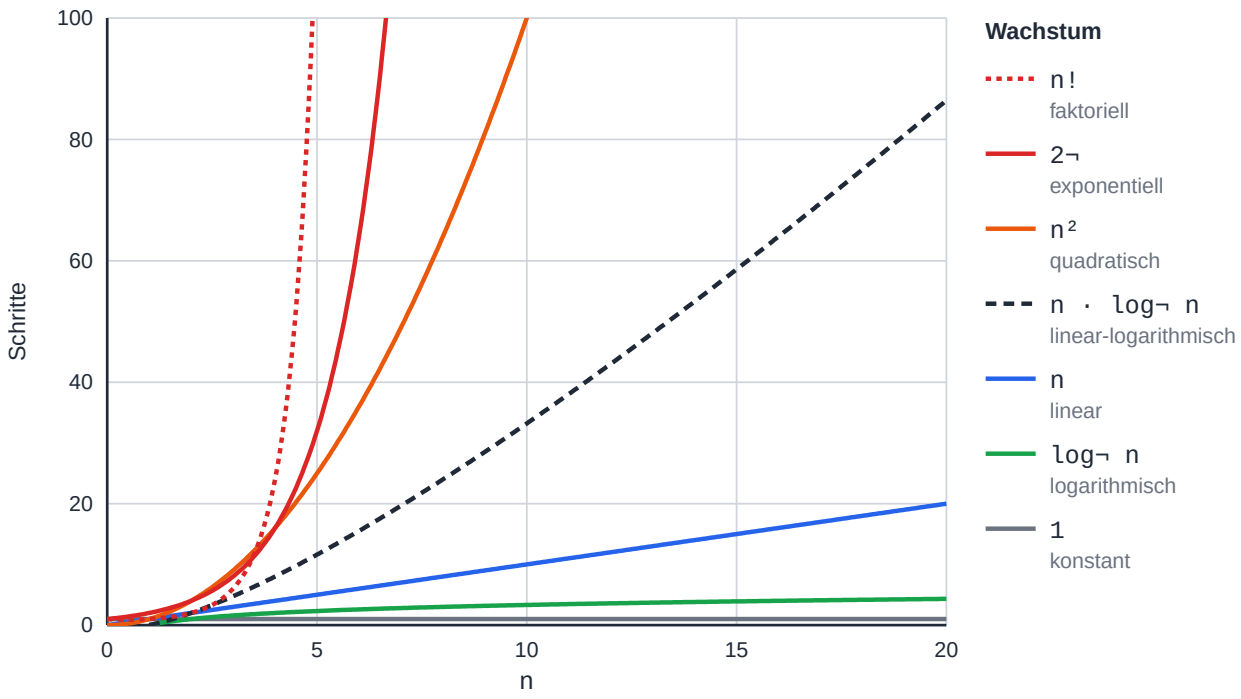
- **Konstante Faktoren weglassen:** $O(5n) = O(n)$, $O(n^2/2) = O(n^2)$.
- **Summenregel:** Laufen zwei Teile nacheinander, zählt der größere. $O(f) + O(g) = O(\max(f, g))$. Erst sortieren ($n \log n$), dann einmal durchlaufen (n), ergibt $O(n \log n + n) = O(n \log n)$.
- **Produktregel:** Wird ein Teil mit $O(g)$ Schritten f-mal wiederholt, ergibt das $O(f \cdot g)$. Eine Schleife über n Elemente, die jedes Mal `List.Contains` (n Schritte) aufruft, braucht $O(n^2)$.
- **Nur der höchste Exponent zählt:** $3n^3 + 100n^2 + n \in O(n^3)$.
- **Die Basis des Logarithmus ist egal:** $\log_2 n = \log_{10} n / \log_{10} 2 \approx 3{,}32 \cdot \log_{10} n$. Die Basen unterscheiden sich nur um einen konstanten Faktor. Man schreibt deshalb einfach $O(\log n)$.
- **Mehrere Eingaben bleiben getrennt:** Bekommt eine Methode zwei Listen der Länge n und m, lautet das Ergebnis z. B. $O(n + m)$ oder $O(n \cdot m)$, nicht $O(n)$.

Die Hierarchie der Wachstumsklassen

Die meisten Algorithmen fallen in eine von wenigen Klassen:

Klasse	Name	Beispiel
$O(1)$	konstant	Zugriff auf `array[i]`, `HashSet.Contains` (im Mittel)
$O(\log n)$	logarithmisch	binäre Suche
$O(n)$	linear	lineare Suche, Summe eines Arrays
$O(n \log n)$	linear-logarithmisch	Mergesort, `Array.Sort`
$O(n^2)$	quadratisch	Duplikatsuche mit `List`, alle Paare vergleichen
$O(2^n)$	exponentiell	alle Teilmengen einer Menge durchprobieren
$O(n!)$	faktoriell	alle Reihenfolgen durchprobieren (Rundreise mit Brute Force)

Es gilt $1 < \log n < n < n \log n < n^2 < 2^n < n!$. Jede Klasse wächst irgendwann schneller als alle links davon. Die Grafik zeigt die Funktionen für kleine n:



n! und 2¬ verlassen das Bild schon vor n = 7, n^2 bei n = 10.

Wie groß der Unterschied wirklich ist, zeigt die Tabelle. Sie nimmt an, dass ein Schritt **1 Nanosekunde** dauert. Das ist ungefähr ein einfacher Rechenschritt eines heutigen Prozessors.

Schritte	**n = 10**	**n = 20**	**n = 50**	**n = 100**	**n = 1 000**	**n = 10^6**
$\log_2 n$	3 ns	4 ns	6 ns	7 ns	10 ns	20 ns
n	10 ns	20 ns	50 ns	100 ns	1 µs	1 ms
$n \log_2 n$	33 ns	86 ns	282 ns	664 ns	10 µs	20 ms
n^2	100 ns	400 ns	2,5 µs	10 µs	1 ms	17 min
2^n	1 µs	1 ms	13 Tage	$4 \cdot 10^{13}$ Jahre	$3 \cdot 10^{284}$ Jahre	$3 \cdot 10^{301013}$ Jahre
n!	3,6 ms	77 Jahre	10^{48} Jahre	$3 \cdot 10^{141}$ Jahre	10^{2551} Jahre	$10^{5565692}$ Jahre

Das Universum ist etwa $1{,}4 \cdot 10^{10}$ Jahre alt. Bis n = 1 000 sind alle Algorithmen bis n^2 in Millisekunden fertig. 2^n und n! sind dagegen schon bei n = 100 hoffnungslos.

Ein schnellerer Rechner hilft dabei kaum. Ist er 1 000-mal schneller, schaffst du in derselben Zeit bei O(n) eine 1 000-mal größere Eingabe, bei $O(n^2)$ eine etwa 32-mal größere. Bei $O(2^n)$ kommst du nur um 10 weiter, denn $2^{10} \approx 1\,000$. Ein besserer Algorithmus bringt fast immer mehr als bessere Hardware.

Best, Average und Worst Case

Die Anzahl der Schritte hängt nicht nur von n ab, sondern auch davon, **welche** Eingabe der Größe n kommt. Man unterscheidet drei Fälle:

- **Best Case:** die günstigste Eingabe der Größe n.
- **Worst Case:** die ungünstigste Eingabe der Größe n.
- **Average Case:** der Durchschnitt über alle Eingaben der Größe n, bei einer angenommenen Verteilung (z. B. alle Reihenfolgen gleich wahrscheinlich).

Lineare Suche

```
int[] numbers = [7, 3, 9, 4, 1];
Console.WriteLine(LinearSearch(numbers, 7));   // best case: first element
Console.WriteLine(LinearSearch(numbers, 8));   // worst case: not found

static int LinearSearch(int[] data, int value)
{
    for (int i = 0; i < data.Length; i++)
    {
        if (data[i] == value) { return i; }
    }
    return -1;
}
```

```
0
-1
```

Fall	Eingabe	Vergleiche	Klasse
Best Case	Der Wert steht ganz vorne.	1	Θ(1)
Average Case	Der Wert kommt vor, jede Position ist gleich wahrscheinlich.	(n + 1)/2	Θ(n)
Worst Case	Der Wert steht ganz hinten oder fehlt.	n	Θ(n)

Der Average Case ist hier nur halb so groß wie der Worst Case. Das ist aber nur ein konstanter Faktor, die Klasse bleibt linear.

Quicksort

Bei Quicksort mit der Lomuto-Partitionierung aus Divide and Conquer ist der Unterschied größer. Das letzte Element ist das Pivot.

- **Best und Average Case:** Das Pivot teilt das Array ungefähr in der Mitte. Es gibt etwa $\log_2 n$

Ebenen, und auf jeder Ebene werden insgesamt etwa n Elemente mit dem Pivot verglichen. Das ergibt $\Theta(n \log n)$.

- **Worst Case:** Das Array ist **schon sortiert**. Das letzte Element ist dann immer das größte, ein Teil ist leer und der andere hat nur ein Element weniger. Es gibt n Ebenen mit $n - 1$, $n - 2$, … Vergleichen, zusammen $n(n - 1)/2$. Das ergibt $\Theta(n^2)$.

Gerade sortierte oder fast sortierte Daten kommen in der Praxis oft vor. `Array.Sort` in .NET verwendet deshalb *Introsort*: Es beginnt mit Quicksort und wechselt zu Heapsort, wenn die Rekursion zu tief wird. So bleibt auch der Worst Case bei $O(n \log n)$.

Beachte: Best, Average und Worst Case sind etwas anderes als O, Ω und Θ. Die Fälle legen fest, **welche Eingabe** du betrachtest. O, Ω und Θ beschreiben, wie die Schrittzahl für diese Eingaben **wächst**. Für jeden Fall kannst du O, Ω und Θ angeben.

Typische Fehler

- **Nicht vereinfachen:** $O(2n)$, $O(n^2 + n)$ oder $O(3 \log n)$ sind nicht falsch, aber unüblich. Richtig vereinfacht heißen sie $O(n)$, $O(n^2)$ und $O(\log n)$.
- **O für kleine n verwenden:** O sagt nichts über kleine Eingaben. $100n$ Schritte sind für $n < 100$ mehr als n^2 Schritte. Deshalb sortiert Introsort kleine Teilarrays mit Insertion Sort, obwohl das ein $O(n^2)$-Verfahren ist.
- **Zwei Größen zu einer machen:** Eine Methode, die für jedes von n Wörtern eine Liste mit m Einträgen durchsucht, braucht $O(n \cdot m)$, nicht $O(n^2)$.
- **Worst Case mit Average Case verwechseln:** `HashSet.Contains` braucht im Mittel $O(1)$. Im Worst Case, wenn alle Elemente denselben Hashcode haben, sind es $O(n)$.
- **Versteckte Kosten übersehen:** Ein Methodenaufruf ist nicht automatisch ein Schritt. `list.Contains(x)`, `list.IndexOf(x)` oder `s1 + s2` bei langen Strings kosten selbst $O(n)$.
- **Gleiche Klasse heißt gleich schnell:** Zwei $O(n \log n)$-Verfahren können sich in der Praxis um einen großen Faktor unterscheiden. O vergleicht das Wachstum, nicht die Stoppuhr.

Zusatzmaterial

Im 31_O_Notation.html vergleichst du die Wachstumsfunktionen in einem Diagramm, rechnest Schrittzahlen in Laufzeiten um und ordnest in mehreren Runden Funktionen nach ihrem Wachstum, auch knifflige Paare wie n log n und n^1,5.

Code analysieren

Mit der O-Notation kannst du ausdrücken, wie stark die Laufzeit mit der Eingabegröße n wächst. Jetzt geht es darum, diese Klasse direkt aus dem Code abzulesen, ohne etwas zu messen. Du zählst dazu, wie oft die Anweisungen ausgeführt werden, und behältst am Ende nur den dominanten Term. Mit ein paar Regeln für Schleifen und Rekursion schaffst du das für fast jeden Code, der dir im Alltag begegnet. Am Ende ordnest du damit die Algorithmen aus dem Block Algorithmen ein.



Zum Selbst-Erarbeiten: das interaktive Laufzeit-Quiz

Im Laufzeit-Quiz siehst du als Animation, wie man die Durchläufe von Schleifen zählt. Danach bestimmst du selbst die Laufzeit von rund 20 C#-Snippets, manche davon mit versteckten Kosten. Damit trainierst du genau das, was du beim Lesen von fremdem Code brauchst.

Die Grundregeln

Konstrukt	Kosten
Einfache Anweisung (Zuweisung, Vergleich, Rechnung, Arrayzugriff)	O(1)
Folge von Anweisungen `A; B;`	Kosten von A + Kosten von B, also das Maximum
`if (Bedingung) A else B`	Bedingung + teurerer Zweig (Worst Case)
Schleife	Anzahl der Durchläufe × Kosten des Rumpfs
Methodenaufruf	Kosten der aufgerufenen Methode, **nicht** automatisch O(1)

Die letzte Zeile ist die häufigste Fehlerquelle. `list.Contains(x)` sieht aus wie eine einfache Anweisung, durchsucht aber die ganze Liste. Bei jedem Aufruf einer Bibliotheksmethode musst du also wissen, was sie intern tut. Die wichtigsten Collections findest du in Datenstrukturen in .NET.

Ein erstes Beispiel:

```
int[] numbers = [4, 8, 15, 16, 23, 42];
Console.WriteLine(Average(numbers));

static double Average(int[] values)
{
    long sum = 0;                          // (1)
    foreach (int v in values)              // (2)
    {
        sum += v;                          // (3)
    }
    return (double)sum / values.Length;    // (4)
```

```
}
```

```
18
```

- **(1)** und **(4)** werden genau einmal ausgeführt, egal wie lang das Array ist.
- **(2)** Die Schleife läuft n-mal, wobei n = `values.Length` ist.
- **(3)** Der Rumpf kostet einen Schritt pro Durchlauf, zusammen also n Schritte.

Insgesamt sind das 1 + n + 1 = n + 2 Schritte. Der dominante Term ist n, die Methode ist also in O(n). Ob du die Prüfung `v < values.Length` und das Weiterzählen als eigene Schritte zählst, ändert nur die Konstante vor dem n. Genau deshalb lässt die O-Notation Konstanten weg: Du musst nicht entscheiden, was „ein Schritt“ ist.

Achte darauf, was n ist. Eine Schleife, die immer genau 10-mal läuft, ist in O(1), auch wenn sie über ein Array mit einer Million Elementen geht. Und eine Schleife bis `n / 2` oder mit `i += 2` ist trotzdem in O(n), denn n/2 unterscheidet sich von n nur um den Faktor ½.

Verschachtelte Schleifen

Steht eine Schleife in einer anderen, multiplizierst du die Durchläufe. Läuft die äußere n-mal und die innere jedes Mal m-mal, wird der innere Rumpf n · m-mal ausgeführt. Bei zwei Schleifen über dasselbe Array sind das n · n = n² Durchläufe.

Zwei Schleifen **hintereinander** addierst du dagegen nur: n + n = 2n ist immer noch O(n). Schau also genau auf die geschweiften Klammern, bevor du multiplizierst.

Abhängige Schleifen

Oft hängt die innere Schleife von der äußeren ab. Das Beispiel sucht alle Paare in einem Array, deren Summe 10 ergibt. Die erste Variante prüft alle Kombinationen, die zweite nur echte Paare:

```
int[] values = [3, 9, 1, 7, 5];
int checksAll = 0;
int checksPairs = 0;

for (int i = 0; i < values.Length; i++)            // (1)
{
    for (int j = 0; j < values.Length; j++)
    {
        checksAll++;
    }
}

for (int i = 0; i < values.Length; i++)            // (2)
{
    for (int j = i + 1; j < values.Length; j++)    // (3)
```

```
    {
        checksPairs++;
        if (values[i] + values[j] == 10)
        {
            Console.WriteLine($"{values[i]} + {values[j]}");
        }
    }
}
Console.WriteLine($"Alle Kombinationen: {checksAll}, nur Paare: {checksPairs}");
```

```
3 + 7
9 + 1
Alle Kombinationen: 25, nur Paare: 10
```

- **(1)** Zwei unabhängige Schleifen: 5 · 5 = 25 Durchläufe. Dabei würde jedes Paar doppelt gefunden (`3 + 7` und `7 + 3`) und jede Zahl auch mit sich selbst kombiniert.
- **(2)** Die äußere Schleife läuft weiterhin n-mal.
- **(3)** Die innere Schleife beginnt erst bei `i + 1`. Für i = 0 läuft sie 4-mal, für i = 1 3-mal und so weiter bis 0-mal für das letzte i.

Die Grafik zeigt das für n = 6. Jede Zeile ist ein Wert von i, jede Spalte ein Wert von j. Blau sind die Kombinationen, die die innere Schleife wirklich prüft:

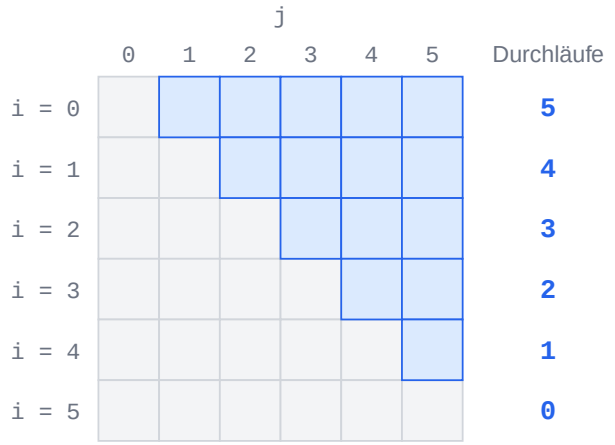
Abhängige Schleife, n = 6

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        count++;
```

Rumpf läuft (j > i)

wird übersprungen (j ¬ i)

Zeile i hat n ¬ 1 ¬ i Durchläufe.



5 + 4 + 3 + 2 + 1 + 0 = 15 = 6 · 5 / 2

Knapp die Hälfte des Quadrats mit n² = 36 Feldern: n(n ¬ 1)/2 Durchläufe, also O(n²)

Die Durchläufe sind eine Summe aufeinanderfolgender Zahlen. Dafür gibt es die **Gaußsche Summenformel**:

1 + 2 + 3 + … + n = n(n + 1) / 2

Für die Paar-Schleife beginnt die Summe bei n − 1, also (n − 1) + (n − 2) + … + 1 + 0 = (n − 1) · n / 2. Für n = 5 sind das 10, für n = 6 sind es 15. Ausmultipliziert ergibt das n²/2 − n/2. Der dominante Term ist n²/2, die Konstante ½ fällt weg. Die abhängige Schleife spart also die Hälfte der Arbeit, bleibt aber in **O(n²)**. Bei n = 100 000 sind das immer noch rund 5 Milliarden Prüfungen.

Mit drei abhängigen Schleifen (`i < j < k`) werden alle Dreiergruppen geprüft. Das sind n(n − 1)(n − 2) / 6 Durchläufe, also O(n³).

Halbierende Schleifen

Eine Schleife muss nicht in Einerschritten zählen. Halbiert sie in jedem Durchlauf die verbleibende Menge, ist sie sehr schnell fertig:

```
foreach (int n in new[] { 8, 1000, 1_000_000, 1_000_000_000 })
{
    int steps = 0;
    for (int size = n; size > 1; size /= 2)       // (1)
    {
        steps++;
    }
    Console.WriteLine($"n = {n,10}: {steps,2} Halbierungen");
}
```

```
n =          8:  3 Halbierungen
n =       1000:  9 Halbierungen
n =    1000000: 19 Halbierungen
n = 1000000000: 29 Halbierungen
```

- **(1)** `size` wird 8 → 4 → 2 → 1. Nach 3 Halbierungen ist die Schleife fertig, und $2^3 = 8$.

Die Frage „Wie oft kann ich n halbieren, bis 1 übrig bleibt?“ beantwortet der **Logarithmus zur Basis 2**: $\log_2 8 = 3$, $\log_2 1000 \approx 9{,}97$. Weil die Ganzzahldivision abrundet, zählt die Schleife $\lfloor \log_2 n \rfloor$ Durchläufe. Eine Schleife, die halbiert oder verdoppelt (`for (int i = 1; i < n; i *= 2)`), ist also in **O(log n)**. Die Basis schreibt man nicht dazu, denn Logarithmen zu verschiedenen Basen unterscheiden sich nur um einen konstanten Faktor.

Das beste Beispiel ist die **binäre Suche** in einem sortierten Array. Sie vergleicht mit dem mittleren Element und sucht danach nur in einer Hälfte weiter:

```
int[] sorted = Enumerable.Range(0, 1_000_000).Select(i => 2 * i).ToArray();
(int index, int steps) = BinarySearch(sorted, 1_234_566);
Console.WriteLine($"Index {index} nach {steps} Schritten");

static (int Index, int Steps) BinarySearch(int[] a, int value)
{
    int lo = 0, hi = a.Length - 1, steps = 0;
    while (lo <= hi)                              // (1)
    {
        steps++;
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == value) { return (mid, steps); }
        if (a[mid] < value) { lo = mid + 1; } else { hi = mid - 1; }   // (2)
    }
```

```
    return (-1, steps);
}
```

```
Index 617283 nach 20 Schritten
```

- **(1)** Die Schleife läuft, solange der Suchbereich nicht leer ist.
- **(2)** In jedem Durchlauf wird der Bereich mindestens halbiert. Nach höchstens $\lfloor \log_2 n \rfloor + 1 = 20$ Durchläufen ist er leer. Bei einer Milliarde Elemente wären es 30.

Kombinationen funktionieren wie gewohnt: Führst du für jedes der n Elemente eine binäre Suche aus, kostet das $n \cdot \log n$ Schritte, also $O(n \log n)$.

Rekursion und der Rekursionsbaum

Bei einer rekursiven Methode kannst du die Durchläufe nicht direkt ablesen. Hier hilft der **Rekursionsbaum**: Jeder Aufruf ist ein Knoten, seine rekursiven Aufrufe sind die Kinder. Die Gesamtkosten sind die Summe der Arbeit in allen Knoten. Zwei Fragen führen zum Ergebnis:

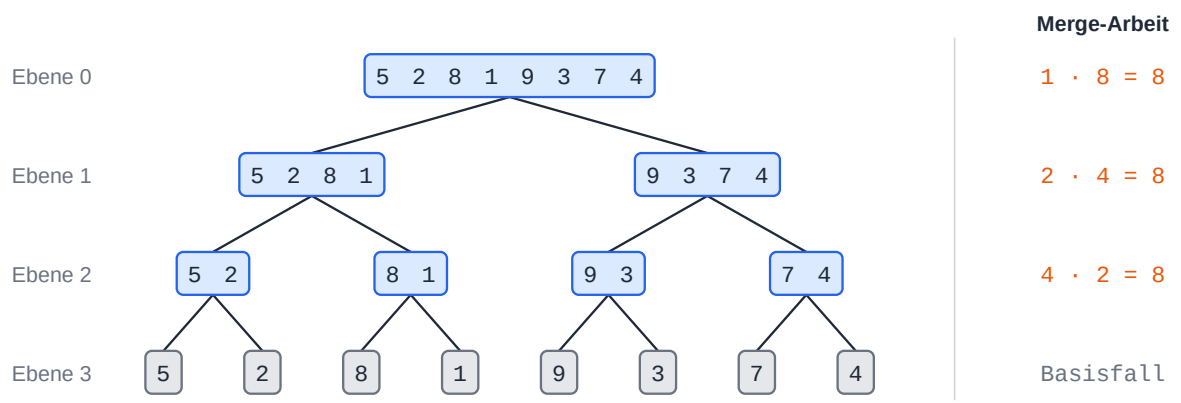
1. Wie viele Aufrufe gibt es (wie viele Ebenen, wie viele Knoten pro Ebene)?
2. Wie viel Arbeit macht ein Aufruf **ohne** seine rekursiven Aufrufe?

Ein Aufruf pro Ebene: binäre Suche rekursiv

Die rekursive binäre Suche ruft sich höchstens einmal selbst auf, mit einem halb so großen Bereich. Der Baum ist eine Kette mit $\log_2 n$ Knoten, und jeder Knoten macht $O(1)$ Arbeit. Das ergibt wie bei der Schleife $O(\log n)$. Dasselbe Muster mit `n - 1` statt `n / 2` (etwa eine rekursive Summe über ein Array) ergibt eine Kette mit n Knoten, also $O(n)$.

Zwei Aufrufe mit halber Größe: Mergesort

Mergesort teilt das Array in zwei Hälften, sortiert beide rekursiv und fügt sie danach mit dem Merge-Schritt zusammen. Das Zusammenfügen von k Elementen kostet k Schritte. Für $n = 8$ sieht der Rekursionsbaum so aus:



log¬ 8 = 3 Ebenen mit je n = 8 Schritten Merge-Arbeit: 3 · 8 = 24 Schritte

Allgemein: log¬ n Ebenen · n Schritte = n · log¬ n, also O(n log n)

Die Ebenen werden breiter, die Teile aber kleiner. Auf jeder Ebene werden zusammen genau n Elemente gemerged: einmal 8, zweimal 4, viermal 2. Es gibt $\log_2 n$ Ebenen mit Merge-Arbeit, weil die Teile bei jeder Ebene halbiert werden. Das ergibt $\mathbf{n \cdot \log_2 n}$ Schritte, also O(n log n). Diese Rechnung „Ebenen × Arbeit pro Ebene“ funktioniert immer, wenn sich die Arbeit gleichmäßig über die Ebenen verteilt. Mergesort braucht außerdem ein Hilfsarray mit n Plätzen.

Zwei Aufrufe mit fast gleicher Größe: naive Fibonacci

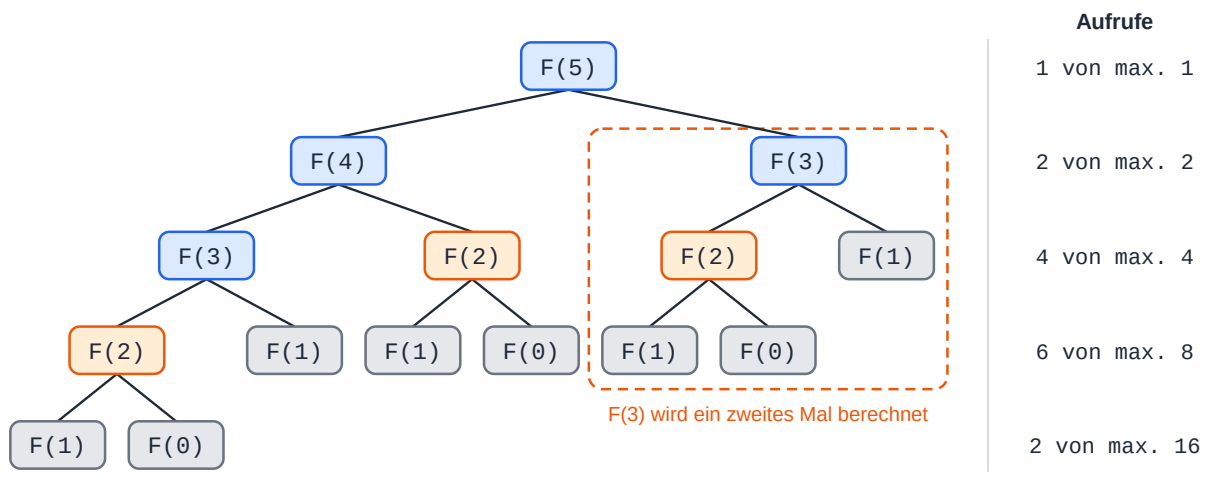
Die naive Fibonacci-Funktion aus Dynamische Programmierung sieht harmlos aus. Der Zähler zeigt, wie oft sie sich aufruft:

```
long calls = 0;
foreach (int n in new[] { 5, 10, 20, 30 })
{
    calls = 0;
    long result = Fib(n);
    Console.WriteLine($"Fib({n}) = {result}, {calls} Aufrufe");
}

long Fib(int n)
{
    calls++;                                        // (1)
    return n < 2 ? n : Fib(n - 1) + Fib(n - 2);     // (2)
}
```

```
Fib(5) = 5, 15 Aufrufe
Fib(10) = 55, 177 Aufrufe
Fib(20) = 6765, 21891 Aufrufe
Fib(30) = 832040, 2692537 Aufrufe
```

- **(1)** `Fib` ist eine lokale Funktion und kann deshalb den Zähler `calls` verwenden.
- **(2)** Jeder Aufruf mit n ≥ 2 erzeugt **zwei** neue Aufrufe. Die Größe sinkt dabei nur um 1 bzw. 2.



15 Aufrufe für F(5). Grau: Basisfall. Orange: F(2) wird dreimal berechnet.

Jede Ebene hat höchstens doppelt so viele Aufrufe wie die vorige: weniger als 2¬ Aufrufe.

Jede Ebene hat höchstens doppelt so viele Knoten wie die vorige, und es gibt bis zu n Ebenen. Das sind höchstens $1 + 2 + 4 + \ldots + 2^{n-1} < 2^n$ Aufrufe, also **$O(2^n)$**. Weil der rechte Ast schneller endet, ist der Baum nicht ganz voll. Genau gerechnet wächst die Zahl der Aufrufe ungefähr mit $1{,}618^n$, das ist trotzdem exponentiell: Jedes zusätzliche n kostet 62 % mehr Aufrufe. Der Grund ist in der Grafik orange markiert: Dieselben Teilprobleme werden immer wieder neu berechnet. Mit Memoization wird jedes F(k) nur einmal berechnet, und die Laufzeit sinkt auf O(n).

Vergleich die beiden Bäume: Mergesort hat zwei Aufrufe mit **halber** Größe und deshalb nur log n Ebenen. Fibonacci hat zwei Aufrufe mit **fast gleicher** Größe und deshalb n Ebenen. Dieser Unterschied macht aus $O(n \log n)$ ein $O(2^n)$.

Quicksort im Worst Case

Quicksort mit Lomuto-Partitionierung nimmt das letzte Element als Pivot. Die Partitionierung eines Bereichs mit k Elementen braucht k − 1 Vergleiche. Danach wird links und rechts vom Pivot rekursiv weitersortiert. Der Zähler zeigt die Vergleiche für ein gemischtes und ein bereits sortiertes Array:

```
int n = 2000;
int[] sorted = Enumerable.Range(0, n).ToArray();
int[] shuffled = [.. sorted];
new Random(42).Shuffle(shuffled);

Console.WriteLine($"gemischt:  {QuickSort(shuffled, 0, n - 1)} Vergleiche");
Console.WriteLine($"sortiert:  {QuickSort(sorted, 0, n - 1)} Vergleiche");
Console.WriteLine($"n(n-1)/2 = {n * (n - 1) / 2}");

static long QuickSort(int[] a, int lo, int hi)
{
    if (lo >= hi) { return 0; }
    int pivot = a[hi];
    int i = lo;
    long comparisons = 0;
    for (int j = lo; j < hi; j++)                    // (1)
```

```
    {
        comparisons++;
        if (a[j] < pivot)
        {
            (a[i], a[j]) = (a[j], a[i]);
            i++;
        }
    }
    (a[i], a[hi]) = (a[hi], a[i]);
    return comparisons + QuickSort(a, lo, i - 1) + QuickSort(a, i + 1, hi);   // (2)
}
```

```
gemischt:  23433 Vergleiche
sortiert:  1999000 Vergleiche
n(n-1)/2 = 1999000
```

- **(1)** Die Partitionierung vergleicht jedes Element des Bereichs mit dem Pivot.
- **(2)** Bei gemischten Daten landet das Pivot meist irgendwo in der Mitte. Der Baum ähnelt dann dem von Mergesort, das ergibt im Mittel O(n log n).

Ist das Array schon sortiert, ist das letzte Element immer das größte. Die Partitionierung liefert dann einen leeren rechten Teil und einen linken Teil mit k − 1 Elementen. Der Rekursionsbaum wird zu einer Kette mit n Ebenen, und die Vergleiche sind (n − 1) + (n − 2) + … + 1 = n(n − 1)/2. Das ist die abhängige Schleife von oben in rekursiver Form: **O(n²)** im Worst Case. Schon bei n = 2000 sind das 85-mal so viele Vergleiche wie beim gemischten Array. Dazu kommt eine Rekursionstiefe von n, die bei großen Arrays zu einer `StackOverflowException` führen kann. Bibliotheken wählen das Pivot deshalb geschickter, und `Array.Sort` wechselt bei zu tiefer Rekursion auf ein anderes Verfahren (*Introsort*). Damit bleibt es auch im Worst Case bei O(n log n).

Die Algorithmen aus dem Block Algorithmen

Mit diesen Werkzeugen kannst du die Algorithmen aus dem Block Algorithmen nachträglich einordnen. Bei Graphen ist V die Anzahl der Knoten (*vertices*) und E die Anzahl der Kanten (*edges*). Die Graphen sind als Adjazenzliste gespeichert.

Algorithmus	Laufzeit	Begründung
BFS und DFS	O(V + E)	Jeder Knoten kommt einmal in die Queue bzw. auf den Stack, jede Kante wird einmal (ungerichtet: zweimal) angesehen.
Dijkstra mit `PriorityQueue`	O((V + E) log V)	Jede Kante kann einen Eintrag in der Queue erzeugen. Einfügen und Herausnehmen kosten je O(log V).

Algorithmus	Laufzeit	Begründung
Topologische Sortierung (Kahn)	O(V + E)	Eingangsgrade zählen geht einmal über alle Kanten, danach wird jeder Knoten und jede Kante einmal verarbeitet.
Binäre Suche	O(log n)	Der Bereich halbiert sich in jedem Schritt.
Mergesort	O(n log n)	log n Ebenen mit je n Schritten Merge-Arbeit, in jedem Fall.
Quicksort	O(n log n) im Mittel, $O(n^2)$ im Worst Case	Bei ungünstigem Pivot wird der Baum zu einer Kette mit n Ebenen.
Backtracking	exponentiell	Der Suchbaum hat bei n Entscheidungen mit je k Möglichkeiten bis zu k^n Blätter. Permutationen: n!, Teilsummen: 2^n, N-Damen: bis zu N! vollständige Stellungen.
Greedy: Aktivitätsauswahl	O(n log n)	Das Sortieren nach Endzeit dominiert, die Auswahl danach ist ein Durchlauf in O(n).
Dynamische Programmierung	Zeilen × Spalten × Arbeit pro Zelle	Fibonacci mit Tabelle: n Zellen → O(n). Wechselgeld: Betrag B × k Münzen → O(B · k). Rucksack: n Gegenstände × Kapazität W → O(n · W). Editierdistanz: O(m · n) für Wörter der Länge m und n.

Beim Backtracking ändert Pruning viel an der tatsächlichen Laufzeit, weil ganze Teilbäume wegfallen. Die Klasse im Worst Case bleibt aber exponentiell. Bei der dynamischen Programmierung ist das Zählen besonders einfach: Jede Zelle der Tabelle wird genau einmal berechnet, und eine Zelle schaut nur auf wenige Nachbarzellen.

Adjazenzmatrix und Adjazenzliste im Vergleich

In Graphen hast du zwei Darstellungen kennengelernt. Jetzt kannst du ihre Kosten vergleichen:

Operation	Adjazenzmatrix	Adjazenzliste
Speicher	$O(V^2)$	O(V + E)
Gibt es die Kante u → v?	O(1)	O(Grad von u)
Alle Nachbarn von u durchlaufen	O(V)	O(Grad von u)

Operation	Adjazenzmatrix	Adjazenzliste
BFS oder DFS über den ganzen Graphen	$O(V^2)$	$O(V + E)$

Bei der Matrix muss die Suche für jeden Knoten eine ganze Zeile mit V Einträgen durchgehen, auch wenn nur zwei davon Kanten sind. Wie viel das ausmacht, hängt davon ab, wie dicht der Graph ist. In einem **dichten** Graphen ist E ungefähr V^2, dann sind beide Darstellungen gleich gut. Die meisten echten Graphen sind aber **dünn**: Eine Kreuzung im Straßennetz hat 3 oder 4 Nachbarn, egal wie groß das Netz ist. Für ein Straßennetz mit einer Million Kreuzungen hätte die Matrix 10^{12} Einträge, die Liste nur einige Millionen. Deshalb ist die Adjazenzliste der Normalfall.

Zusatzmaterial

Im 32_Code_analysieren.html siehst du animiert, wie man die Durchläufe von Schleifen zählt, und bestimmst danach selbst die Laufzeit von rund 20 C#-Snippets, darunter einige mit versteckten Kosten.

Datenstrukturen in .NET

Die meisten Laufzeitprobleme in echten Programmen entstehen nicht durch einen schlechten Sortieralgorithmus, sondern durch die falsche Collection oder durch eine harmlos aussehende Methode, die in einer Schleife steht. Ein `List.Contains` kostet O(n), ein `HashSet.Contains` O(1). In einer Schleife über n Elemente ist das der Unterschied zwischen $O(n^2)$ und O(n). In dieser Einheit lernst du, was die Collections von .NET intern machen, was ihre Operationen kosten und wo in LINQ- und String-Code versteckte Kosten lauern.

Wie man die Collections verwendet, steht in Collections in .NET. Hier geht es nur um ihre Kosten.

Operationen und ihre Kosten

Die Tabelle zeigt die wichtigsten Operationen. n ist die Anzahl der Elemente in der Collection.

Collection	Intern	Lesen	Suchen	Einfügen	Entfernen
`List<T>`	Array	`[i]` O(1)	`Contains` O(n)	`Add` O(1)*, `Insert(0, x)` O(n)	`RemoveAt(Count - 1)` O(1), `RemoveAt(0)` und `Remove(x)` O(n)
`LinkedList<T>`	doppelt verkettete Knoten	`First`, `Last` O(1), kein Index	`Find` O(n)	`AddFirst`, `AddLast`, `AddAfter(node, x)` O(1)	`RemoveFirst`, `Remove(node)` O(1), `Remove(x)` O(n)
`HashSet<T>`	Hashtabelle	kein Index	`Contains` O(1)**	`Add` O(1)**	`Remove` O(1)**
`Dictionary<TKey, TValue>`	Hashtabelle	`[key]` O(1)**	`ContainsKey` O(1)**, `ContainsValue` O(n)	`Add` O(1)**	`Remove` O(1)**
`SortedSet<T>`	Rot-Schwarz-Baum	`Min`, `Max` O(log n)	`Contains` O(log n)	`Add` O(log n)	`Remove` O(log n)
`SortedDictionary<TKey, TValue>`	Rot-Schwarz-Baum	`[key]` O(log n)	`ContainsKey` O(log n)	`Add` O(log n)	`Remove` O(log n)
`Queue<T>`	Ringpuffer (Array)	`Peek` O(1)	`Contains` O(n)	`Enqueue` O(1)*	`Dequeue` O(1)
`Stack<T>`	Array	`Peek` O(1)	`Contains` O(n)	`Push` O(1)*	`Pop` O(1)
`PriorityQueue<TElement, TPriority>`	Heap (Array)	`Peek` O(1)	–	`Enqueue` O(log n)	`Dequeue` O(log n)

* amortisiert: meistens O(1), nur wenn das interne Array voll ist O(n), siehe unten. ** im Mittel und amortisiert. Bei schlechten Hashcodes bis O(n), siehe Abschnitt Hashing.

Ein paar Punkte, die in der Tabelle leicht untergehen:

- Alle Operationen am **Ende** einer `List<T>` sind billig. Am **Anfang** oder in der Mitte muss die Liste alle folgenden Elemente um einen Platz verschieben. Wer Elemente vorne entnimmt, nimmt eine `Queue<T>`. Sie merkt sich Anfang und Ende in einem Ringpuffer und muss nichts verschieben.
- `LinkedList<T>` fügt nur dann in O(1) ein, wenn du den Knoten schon hast. Den Knoten zu einem Wert zu finden kostet O(n), einen Index gibt es nicht. In der Praxis ist `List<T>` trotzdem meist schneller, weil ihre Elemente im Speicher hintereinander liegen und der Prozessor sie gut vorausladen kann.
- `SortedSet<T>` und `SortedDictionary<TKey, TValue>` halten die Elemente immer sortiert. Sie sind langsamer als ihre Hash-Varianten, liefern aber `foreach` in sortierter Reihenfolge, `Min` und `Max` sowie Bereichsabfragen wie `GetViewBetween`.
- `PriorityQueue<TElement, TPriority>` liefert immer das Element mit der kleinsten Priorität. Sie ist ein Heap: Das kleinste Element steht vorne, der Rest ist nur teilweise geordnet. Deshalb gibt es kein `Contains` und auch keine sortierte Aufzählung.
- `Dictionary.ContainsValue` durchsucht alle Einträge. Nur die Suche nach dem **Key** ist O(1).

Wachstum von Arrays und amortisierte Kosten

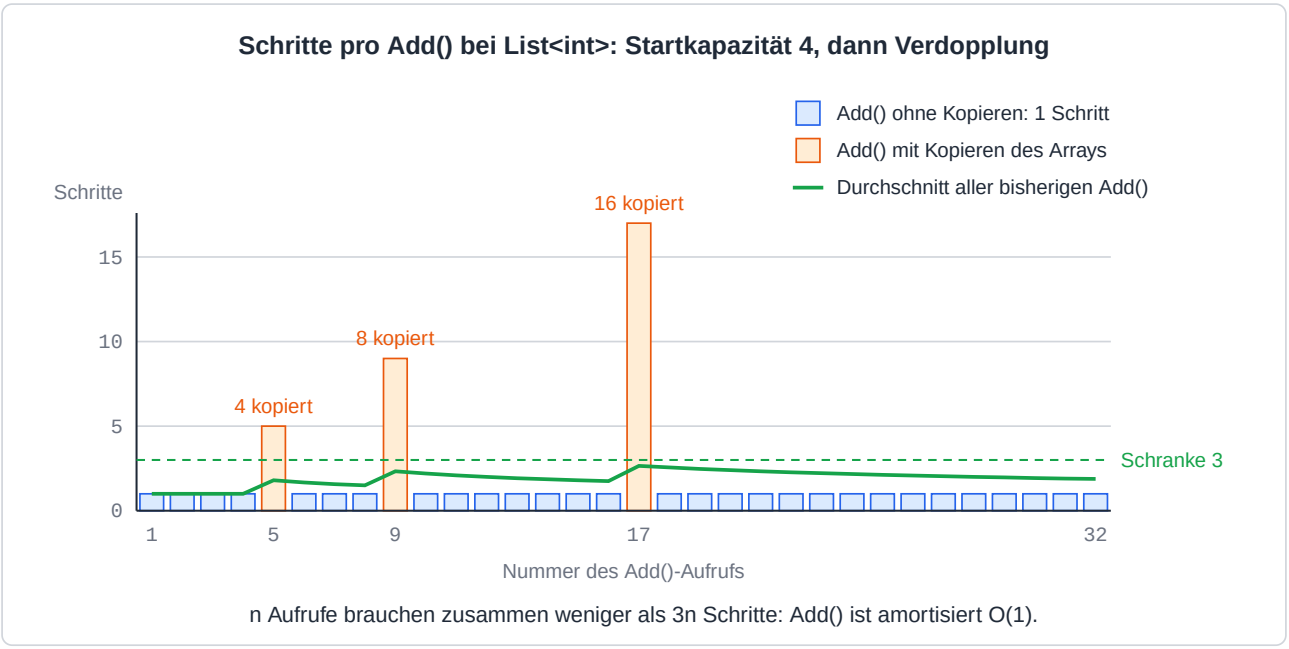
`List<T>`, `Queue<T>`, `Stack<T>`, `HashSet<T>` und `Dictionary<TKey, TValue>` speichern ihre Daten in einem Array. Ein Array hat eine feste Länge. Ist es voll, legt die Collection ein neues, doppelt so großes Array an und kopiert alle Elemente hinüber. Das zeigt das Property `Capacity`:

```
var list = new List<int>();
int lastCapacity = -1;
for (int i = 1; i <= 33; i++)
{
    list.Add(i);
    if (list.Capacity != lastCapacity)                // (1)
    {
        Console.WriteLine($"Add Nr. {i,2}: Capacity {list.Capacity}");
        lastCapacity = list.Capacity;
    }
}
```

```
Add Nr.  1: Capacity 4
Add Nr.  5: Capacity 8
Add Nr.  9: Capacity 16
Add Nr. 17: Capacity 32
Add Nr. 33: Capacity 64
```

- **(1)** Die Kapazität ändert sich nur, wenn das Array voll war. Dann wird kopiert.

Ein einzelnes `Add` kostet also manchmal einen Schritt und manchmal n Schritte. Interessant ist, was **alle** Aufrufe zusammen kosten:



n Aufrufe brauchen zusammen weniger als 3n Schritte: Add() ist amortisiert O(1).

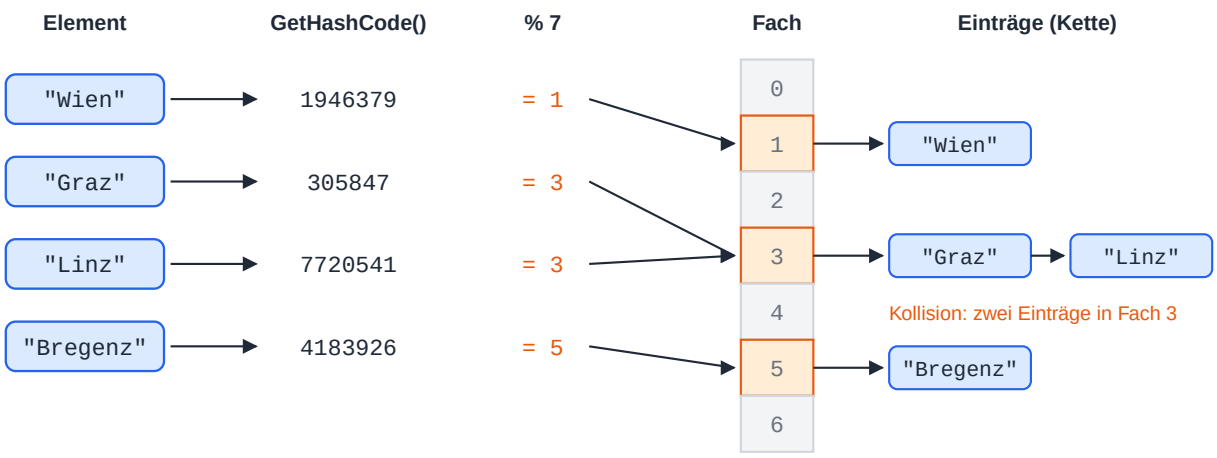
Für n Aufrufe von `Add` werden n Elemente geschrieben. Dazu kommen die Kopien: 4 + 8 + 16 + … bis höchstens n. Jede Kopie ist so groß wie alle vorigen zusammen plus 4, deshalb ist die Summe kleiner als 2n. Insgesamt sind das weniger als 3n Schritte, im Schnitt also weniger als 3 Schritte pro `Add`. Diese Betrachtung heißt **amortisierte Analyse**: Die seltenen teuren Aufrufe werden auf die vielen billigen verteilt. `Add` ist **amortisiert O(1)**.

Das funktioniert nur, weil das Array um einen **Faktor** wächst. Würde es jedes Mal um 10 Plätze wachsen, wären die Kopien 10 + 20 + 30 + … + n ≈ n²/20 Schritte. Dann wäre `Add` im Schnitt O(n), und das Befüllen einer Liste O(n²).

Weißt du vorher, wie viele Elemente kommen, kannst du dir das Kopieren ganz sparen: `new List<int>(1_000_000)` legt das Array sofort in der richtigen Größe an. Dasselbe geht bei `Dictionary`, `HashSet` und `StringBuilder`.

Hashing

`HashSet<T>` und `Dictionary<TKey, TValue>` finden ein Element, ohne die Collection zu durchsuchen. Dafür berechnen sie aus dem Element mit `GetHashCode()` eine Zahl, den **Hashcode**. Der Rest dieser Zahl bei Division durch die Anzahl der Fächer (*Buckets*) gibt an, in welchem Fach das Element liegt:



Contains("Linz"): Hashcode berechnen ¬ Fach 3 ¬ Equals mit "Graz": false ¬ Equals mit "Linz": true

Die Hashcodes sind erfundene Beispielwerte. Echte Hashtabellen haben mehr Fächer.

`Contains("Linz")` berechnet den Hashcode, geht direkt in Fach 3 und vergleicht dort mit `Equals()`. Landen zwei Elemente im selben Fach, ist das eine **Kollision**. Die Elemente eines Fachs bilden eine Kette, die beim Suchen durchlaufen wird. Solange die Hashcodes gut verteilt sind, sind die Ketten kurz, und die Suche kostet im Mittel O(1). Wird die Tabelle zu voll, vergrößert sie sich wie eine Liste und verteilt alle Elemente neu.

Damit das funktioniert, gelten zwei Regeln:

1. **Gleiche Objekte haben gleiche Hashcodes.** Wer `Equals` überschreibt, muss auch `GetHashCode` überschreiben. Der Compiler warnt sonst (CS0659). `record`-Typen erledigen beides automatisch über alle Properties.
2. **Der Hashcode ändert sich nicht, solange das Objekt in der Collection liegt.** Änderst du ein Property, das in den Hashcode eingeht, sucht `Contains` im falschen Fach und findet das Objekt nicht mehr. Verwende deshalb unveränderliche Typen als Keys.

Ein Hashcode darf für verschiedene Objekte gleich sein. Er ist nur dann nutzlos, wenn er für viele Objekte gleich ist. Das zeigt das folgende Beispiel mit einem konstanten Hashcode:

```
using System.Diagnostics;

const int n = 20_000;
Measure("record", Enumerable.Range(0, n).Select(i => new GoodKey(i)));
Measure("konstanter Hashcode", Enumerable.Range(0, n).Select(i => new BadKey(i)));

static void Measure<T>(string label, IEnumerable<T> keys)
{
    var stopwatch = Stopwatch.StartNew();
    var set = new HashSet<T>(keys);
    Console.WriteLine($"{label,-20} {set.Count} Elemente in {stopwatch
.ElapsedMilliseconds,5} ms");
}

record GoodKey(int Value);
```

```
/// <summary>Key with a legal but useless hash code: all instances land in the same
bucket.</summary>
class BadKey(int value)
{
    public int Value { get; } = value;
    public override bool Equals(object? obj) => obj is BadKey other && other.Value ==
Value;
    public override int GetHashCode() => 1;                          // (1)
}
```

```
record               20000 Elemente in     2 ms
konstanter Hashcode  20000 Elemente in   524 ms
```

- **(1)** Alle Objekte landen im selben Fach. Jedes `Add` muss die ganze Kette mit `Equals` vergleichen, um Duplikate auszuschließen. Aus O(1) wird O(n), und das Befüllen kostet O(n²).

Die Zeiten hängen von deinem Rechner ab, das Verhältnis bleibt ähnlich. Für eigene Klassen erzeugst du einen guten Hashcode mit `HashCode.Combine(FirstName, LastName)`. Beachte noch: Der Hashcode eines `string` ist in .NET bei jedem Programmstart anders. Du darfst Hashcodes also nie speichern oder an andere Programme weitergeben.

LINQ-Fallen

LINQ-Methoden sehen aus wie einzelne Anweisungen. Jede davon kann aber die ganze Quelle durchlaufen. Die folgenden Muster findest du in vielen Projekten.

`Contains` in `Where` und `Count()` statt `Any()`

```
List<Order> orders = [.. Enumerable.Range(1, 10_000).Select(i => new Order(i, i % 500,
i % 1000))];
List<int> blockedCustomers = [.. Enumerable.Range(0, 500).Where(c => c % 7 == 0)];

var slow = orders.Where(o => blockedCustomers.Contains(o.CustomerId)).ToList();   //
(1)

HashSet<int> blockedSet = [.. blockedCustomers];
var fast = orders.Where(o => blockedSet.Contains(o.CustomerId)).ToList();         //
(2)
Console.WriteLine($"{slow.Count} = {fast.Count} gesperrte Bestellungen");

bool anyBigSlow = orders.Where(o => o.Total > 900).Count() > 0;                   //
(3)
bool anyBigFast = orders.Any(o => o.Total > 900);                                 //
(4)
Console.WriteLine($"{anyBigSlow} = {anyBigFast}");

record Order(int Id, int CustomerId, decimal Total);
```

```
1440 = 1440 gesperrte Bestellungen
True = True
```

- **(1)** Für jede der n Bestellungen durchsucht `List.Contains` die m gesperrten Kunden. Das sind O(n · m) Schritte.
- **(2)** Einmal ein `HashSet` bauen (O(m)) und danach n-mal in O(1) nachschlagen ergibt O(n + m).
- **(3)** `Count()` zählt **alle** passenden Bestellungen, obwohl nur interessiert, ob es eine gibt.
- **(4)** `Any()` hört beim ersten Treffer auf. Im Worst Case ist es auch O(n), meist aber viel früher fertig.

Ohne Bedingung ist `list.Count()` übrigens O(1), denn LINQ erkennt, dass eine `List<T>` ihre Länge kennt. Nach einem `Where` geht das nicht mehr.

Mehrfache Enumeration

Eine LINQ-Abfrage wird erst ausgeführt, wenn jemand die Ergebnisse durchläuft (*deferred execution*). Durchläufst du sie zweimal, wird sie auch zweimal ausgeführt:

```
int evaluations = 0;
int[] numbers = [.. Enumerable.Range(1, 1000)];
IEnumerable<int> multiplesOfThree = numbers.Where(x =>
{
    evaluations++;
    return x % 3 == 0;
});                                                          // (1)

Console.WriteLine($"Anzahl: {multiplesOfThree.Count()}");    // (2)
Console.WriteLine($"Größte: {multiplesOfThree.Max()}");
Console.WriteLine($"Summe: {multiplesOfThree.Sum()}");
Console.WriteLine($"Bedingung {evaluations}-mal geprüft");

List<int> materialized = [.. numbers.Where(x => x % 3 == 0)];  // (3)
Console.WriteLine($"Anzahl: {materialized.Count}, Größte: {materialized.Max()}");
```

```
Anzahl: 333
Größte: 999
Summe: 166833
Bedingung 3000-mal geprüft
Anzahl: 333, Größte: 999
```

- **(1)** Hier wird noch nichts geprüft. `multiplesOfThree` ist nur eine Beschreibung der Abfrage.
- **(2)** `Count()`, `Max()` und `Sum()` führen die Abfrage jeweils komplett aus.
- **(3)** Brauchst du das Ergebnis mehrmals, materialisiere es einmal mit `ToList()` oder einer Collection Expression.

Bei einem Array kostet das nur Zeit. Liest die Quelle aber eine Datei (`File.ReadLines`) oder eine

Datenbank, wird die Datei mehrmals gelesen oder die Abfrage mehrmals an die Datenbank geschickt.

`OrderBy` oder `ToList` in einer Schleife

Das folgende Fragment (es verwendet `orders` aus dem Beispiel oben) bestimmt für jede Bestellung den Rang nach ihrem Betrag:

```
foreach (Order order in orders)
{
    List<Order> sorted = orders.OrderByDescending(o => o.Total).ToList();   // (1)
    int rank = sorted.IndexOf(order) + 1;                                    // (2)
    Console.WriteLine($"{order.Id}: Rang {rank}");
}
```

- **(1)** Die Liste wird bei **jedem** Durchlauf neu sortiert: n-mal O(n log n).
- **(2)** `IndexOf` durchsucht die Liste noch einmal linear.

Insgesamt ist das $O(n^2 \log n)$, bei 10 000 Bestellungen über eine Milliarde Schritte. Sortiere stattdessen **einmal** vor der Schleife und merke dir den Rang jeder Bestellung in einem `Dictionary<int, int>`. Dann kostet alles zusammen O(n log n). Dasselbe gilt für `ToList()`, `ToArray()` und `ToHashSet()` in einer Schleife: Jeder Aufruf kopiert die ganze Quelle.

Suchst du nur das kleinste oder größte Element, brauchst du gar nicht zu sortieren. `Min()`, `Max()`, `MinBy()` und `MaxBy()` gehen einmal durch die Daten und kosten O(n).

`StringBuilder` statt `+=`

Ein `string` ist in .NET unveränderlich. `s += x` hängt deshalb nichts an, sondern legt einen neuen String an und kopiert den alten Inhalt und `x` hinein. Baust du einen Text mit n Zeichen Stück für Stück auf, werden $1 + 2 + 3 + \ldots + n = n(n + 1)/2$ Zeichen kopiert. Das ist wieder die Gaußsche Summe, also $O(n^2)$.

```
using System.Diagnostics;
using System.Text;

const int n = 50_000;
var stopwatch = Stopwatch.StartNew();
string text = "";
for (int i = 0; i < n; i++)
{
    text += i % 10;                                                  // (1)
}
Console.WriteLine($"+=            {text.Length} Zeichen in {stopwatch
.ElapsedMilliseconds,4} ms");

stopwatch.Restart();
var builder = new StringBuilder();
for (int i = 0; i < n; i++)
```

```
{
    builder.Append(i % 10);                                          // (2)
}
string built = builder.ToString();
Console.WriteLine($"StringBuilder {built.Length} Zeichen in {stopwatch
.ElapsedMilliseconds,4} ms");
```

```
+=            50000 Zeichen in  124 ms
StringBuilder 50000 Zeichen in    0 ms
```

- **(1)** Jeder Durchlauf kopiert den bisherigen Text. Beim letzten sind das fast 50 000 Zeichen.
- **(2)** Der `StringBuilder` sammelt die Zeichen in Puffern. Ist ein Puffer voll, hängt er einen neuen an, ohne den bisherigen Text zu kopieren. `Append` ist damit amortisiert O(1), `ToString()` kopiert am Ende einmal alles: zusammen O(n).

Auch hier hängen die Zeiten von deinem Rechner ab, der Abstand bleibt aber deutlich. Für ein paar Verkettungen außerhalb von Schleifen ist `+` oder ein interpolierter String völlig in Ordnung. Hast du die Teile schon in einer Collection, ist `string.Join(", ", names)` die einfachste Lösung.

Welche Collection passt?

Du brauchst …	Collection
Zugriff über einen Index, Anhängen am Ende	`List<T>` oder Array
Nachschlagen über einen Schlüssel	`Dictionary<TKey, TValue>`
Prüfen, ob etwas vorkommt, oder Duplikate entfernen	`HashSet<T>`
Daten immer sortiert, auch nach Einfügen und Löschen	`SortedSet<T>`, `SortedDictionary<TKey, TValue>`
Abarbeiten in Ankunftsreihenfolge (FIFO)	`Queue<T>`
Das zuletzt Hinzugefügte zuerst (LIFO), z. B. Undo	`Stack<T>`
Immer das dringendste Element zuerst	`PriorityQueue<TElement, TPriority>`
Häufig Einfügen und Löschen mitten in einer Folge, Knoten sind bekannt	`LinkedList<T>`

Ob die Wahl in deinem Programm wirklich etwas bringt, zeigt eine Messung. Wie du mit BenchmarkDotNet zuverlässig misst, steht in Benchmarks mit Benchmark.NET.

Optimierung

Mit der O-Notation kannst du sagen, wie ein Algorithmus wächst. In diesem Kapitel geht es darum, was du mit diesem Wissen anfängst: Wann lohnt es sich, Code schneller zu machen? Wo setzt du an? Und was kostet es? Denn fast jede Optimierung hat einen Preis, meistens bezahlst du mit Speicher.

Erst messen, dann optimieren

Von Donald Knuth stammt der bekannte Satz: *„Premature optimization is the root of all evil.“* Gemeint ist: Wer optimiert, bevor er weiß, wo die Zeit verloren geht, macht den Code komplizierter und meistens kaum schneller. In fast jedem Programm verbringt ein kleiner Teil des Codes den Großteil der Laufzeit. Diese Stelle heißt *Hot Spot* oder *Engstelle*. Eine Optimierung an einer anderen Stelle bringt nichts, egal wie geschickt sie ist.

Gehe deshalb immer in dieser Reihenfolge vor:

1. **Ziel festlegen.** Wie schnell muss es sein? „Die Suche antwortet in unter 100 ms“ ist ein Ziel, „schneller“ ist keines.
2. **Messen.** Für einzelne Methoden eignet sich BenchmarkDotNet (siehe Benchmarks mit Benchmark.NET), für einen ersten Eindruck eine `Stopwatch` im Release-Build.
3. **Engstelle finden.** Bei einem ganzen Programm hilft ein *Profiler*. Visual Studio bringt einen mit (*Debuggen → Leistungsprofiler*, Werkzeug *CPU-Auslastung*). Er zeigt, welche Methoden wie viel Rechenzeit verbrauchen.
4. **Optimieren**, und zwar zuerst den Algorithmus, erst danach Details.
5. **Wieder messen** und die Tests laufen lassen. Eine Optimierung, die falsche Ergebnisse liefert, ist keine.

Ein besserer Algorithmus schlägt jede Mikrooptimierung

Ein klassisches Beispiel ist das Problem *Two-Sum*: Gegeben sind ein Array mit Zahlen und eine Zielsumme. Gibt es zwei Zahlen im Array (an verschiedenen Positionen), deren Summe genau die Zielsumme ist?

Die naheliegende Lösung probiert alle Paare durch. Die bessere Lösung geht das Array nur einmal durch und merkt sich in einem `HashSet` alle Zahlen, die sie schon gesehen hat.

```
int[] numbers = [8, 3, 11, 5, 14, 2, 7];
Console.WriteLine($"Naiv,  Summe 16: {HasPairNaive(numbers, 16)}");
Console.WriteLine($"Naiv,  Summe 30: {HasPairNaive(numbers, 30)}");
Console.WriteLine($"Hash,  Summe 16: {HasPairFast(numbers, 16)}");
Console.WriteLine($"Hash,  Summe 30: {HasPairFast(numbers, 30)}");

static bool HasPairNaive(int[] numbers, int target)
{
    for (int i = 0; i < numbers.Length; i++)
    {
```

```
        for (int j = i + 1; j < numbers.Length; j++)                // (1)
        {
            if (numbers[i] + numbers[j] == target) { return true; }
        }
    }
    return false;
}

static bool HasPairFast(int[] numbers, int target)
{
    HashSet<int> seen = [];                                         // (2)
    foreach (int x in numbers)
    {
        if (seen.Contains(target - x)) { return true; }             // (3)
        seen.Add(x);                                                // (4)
    }
    return false;
}
```

```
Naiv,  Summe 16: True
Naiv,  Summe 30: False
Hash,  Summe 16: True
Hash,  Summe 30: False
```

- **(1)** Die innere Schleife beginnt bei `i + 1`. So wird jedes Paar nur einmal geprüft und keine Zahl mit sich selbst addiert. Das sind n(n − 1)/2 Paare, also O(n²).
- **(2)** Das `HashSet` enthält alle Zahlen *links* von der aktuellen Position.
- **(3)** Die Frage „Gibt es ein y mit x + y = target?“ wird zu „Habe ich `target - x` schon gesehen?“. `Contains` braucht bei einem `HashSet` im Mittel konstante Zeit.
- **(4)** Erst nach der Prüfung wird `x` eingefügt. Sonst würde bei `target = 16` die Zahl 8 mit sich selbst gepaart.

Die zweite Variante braucht n Durchläufe mit je einem `Contains` und einem `Add`, also O(n). Im schlechtesten Fall (es gibt kein Paar) sieht der Unterschied so aus:

n	Paare (naiv)	Durchläufe (HashSet)
1 000	ca. 500 000	1 000
100 000	ca. 5 000 000 000	100 000
1 000 000	ca. 500 000 000 000	1 000 000

Auf einem Testrechner dauerte die naive Variante bei n = 100 000 rund 2 Sekunden, die Variante mit `HashSet` rund 4 Millisekunden. Bei einer Million Zahlen wären es für die naive Variante schon über drei Minuten.

Was wäre mit *Mikrooptimierungen* gegangen? Damit sind kleine Kniffe gemeint, die am Algorithmus nichts ändern: `numbers.Length` in eine lokale Variable legen, `for` statt `foreach` verwenden, Bereichsprüfungen umgehen. Viele dieser Kniffe macht der JIT-Compiler ohnehin schon selbst. Im besten Fall gewinnst du einen konstanten Faktor, etwa die doppelte Geschwindigkeit.

Gegen den Sprung von n^2 auf n kommt das nicht an: Bei n = 100 000 ist n^2 um den Faktor 100 000 größer als n. Mikrooptimierungen lohnen sich erst, wenn der Algorithmus schon passt und die Messung zeigt, dass genau diese Stelle der Hot Spot ist.

Wenn du die Positionen brauchst

Oft willst du nicht nur wissen, *ob* es ein Paar gibt, sondern *welches*. Dann reicht ein `HashSet` nicht, denn es merkt sich nur die Werte. Ein `Dictionary` speichert zu jedem Wert zusätzlich seinen Index:

```
int[] numbers = [8, 3, 11, 5, 14, 2, 7];
Console.WriteLine(FindPair(numbers, 16));
Console.WriteLine(FindPair(numbers, 30));

static (int First, int Second)? FindPair(int[] numbers, int target)
{
    Dictionary<int, int> indexOf = [];
    for (int i = 0; i < numbers.Length; i++)
    {
        if (indexOf.TryGetValue(target - numbers[i], out int j)) { return (j, i); }
        indexOf[numbers[i]] = i;
    }
    return null;
}
```

```
(2, 3)
```

Die Zahlen an den Positionen 2 und 3 sind 11 und 5. Für die Summe 30 liefert die Methode `null`, `Console.WriteLine` gibt dafür eine leere Zeile aus.

Primzahlen: Probedivision und Sieb

Wie viele Primzahlen gibt es bis zu einer Million? Die direkte Lösung testet jede Zahl einzeln mit *Probedivision*: n ist eine Primzahl, wenn keine Zahl von 2 bis n − 1 sie teilt.

```
int limit = 1_000_000;
int count = 0;
for (int n = 2; n <= limit; n++)
{
    if (IsPrime(n)) { count++; }
}
Console.WriteLine($"Primzahlen bis {limit}: {count}");

static bool IsPrime(int n)
{
    if (n < 2) { return false; }
    for (int d = 2; d * d <= n; d++)        // (1)
    {
```

```
        if (n % d == 0) { return false; }
    }
    return true;
}
```

```
Primzahlen bis 1000000: 78498
```

- **(1)** Hier steckt schon eine erste Optimierung. Hat n einen Teiler d mit d · e = n, dann ist einer der beiden Teiler höchstens √n. Es reicht also, bis √n zu testen. Die Bedingung `d * d <= n` vermeidet die Gleitkomma-Funktion `Math.Sqrt`.

Für einen einzelnen Test ist das gut genug. Für alle Zahlen bis N sind es aber bis zu √N Divisionen pro Zahl, also O(N · √N). Bis zu einer Million rechnet das Programm rund 67,7 Millionen Divisionen.

Das Sieb des Eratosthenes

Das Sieb dreht die Frage um. Statt jede Zahl auf Teiler zu prüfen, streicht es von jeder Primzahl aus alle ihre Vielfachen:

1. Schreibe alle Zahlen von 2 bis N auf. Zu Beginn ist keine gestrichen.
2. Nimm die kleinste Zahl p, die nicht gestrichen ist. Sie ist eine Primzahl.
3. Streiche alle Vielfachen von p ab p^2. Kleinere Vielfache wie 2p oder 3p sind schon von 2 bzw. 3 gestrichen worden.
4. Wiederhole Schritt 2 und 3, solange $p^2 \le N$ ist. Alle Zahlen, die dann noch übrig sind, sind Primzahlen.

Für N = 30 sieht das so aus:

```
p = 2: streiche 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30
p = 3: streiche 9, 15, 21, 27          (6, 12, 18, ... sind schon gestrichen)
p = 5: streiche 25                     (10, 15, 20, 30 sind schon gestrichen)
p = 7: 7² = 49 > 30, fertig
übrig: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29
```

Das Sieb kommt ohne eine einzige Division aus. Bis zu einer Million streicht es rund 2,1 Millionen Mal, also etwa dreißigmal weniger Schritte als die Probedivision. Die Laufzeit ist O(N · log log N). log log N wächst so langsam, dass du das in der Praxis fast wie O(N) behandeln kannst.

Der Preis ist Speicher: Das Sieb braucht ein `bool`-Array mit N + 1 Einträgen, bei $N = 10^6$ also rund 1 MB. Die Probedivision kommt mit ein paar lokalen Variablen aus. Bei $N = 10^{10}$ bräuchte das Sieb 10 GB und passt nicht mehr in den Arbeitsspeicher. Wie du das Sieb implementierst und gegen die Probedivision misst, probierst du in den Übungen aus.

Speicher gegen Zeit tauschen

Beide Beispiele folgen demselben Muster. Die schnelle Variante merkt sich etwas, das die langsame Variante immer wieder neu ausrechnet. Das nennt man **Time-Space-Tradeoff**: Du tauschst Speicher gegen Rechenzeit.



Der Tausch lohnt sich, wenn der Speicher vorhanden ist und das Gemerkte oft genug gebraucht wird. Er lohnt sich nicht, wenn die Daten nicht in den Arbeitsspeicher passen oder jedes Ergebnis nur einmal gebraucht wird. Zwei Techniken begegnen dir dabei immer wieder.

Vorberechnung und Lookup-Tabellen

Wird dieselbe Funktion sehr oft für Werte aus einem kleinen Bereich aufgerufen, kannst du alle Ergebnisse einmal im Voraus berechnen und danach nur noch nachschlagen. Ein Beispiel: Wie viele Bits sind in einer Zahl gesetzt? Für ein Byte gibt es nur 256 mögliche Werte, also passt das Ergebnis für jedes Byte in eine kleine Tabelle. Eine 32-Bit-Zahl besteht aus vier Bytes, sie braucht dann nur noch vier Zugriffe auf die Tabelle.

```
int[] bitsInByte = new int[256];
for (int i = 1; i < 256; i++)
{
    bitsInByte[i] = bitsInByte[i >> 1] + (i & 1);                // (1)
}

uint[] values = [0, 1, 255, 0xF0F0, uint.MaxValue];
foreach (uint value in values)
{
    Console.WriteLine($"{value,10}: {CountBits(value)} Bits gesetzt");
}

int CountBits(uint value) =>
    bitsInByte[value & 0xFF] + bitsInByte[(value >> 8) & 0xFF]     // (2)
```

```
    + bitsInByte[(value >> 16) & 0xFF] + bitsInByte[value >> 24];
```

```
         0: 0 Bits gesetzt
         1: 1 Bits gesetzt
       255: 8 Bits gesetzt
     61680: 8 Bits gesetzt
4294967295: 32 Bits gesetzt
```

- **(1)** `i >> 1` ist i ohne das letzte Bit und ist kleiner als i. Sein Eintrag steht also schon in der Tabelle. Dazu kommt das letzte Bit `i & 1`. So wird auch die Tabelle selbst schnell aufgebaut.
- **(2)** Die Zahl wird in ihre vier Bytes zerlegt, jedes Byte wird nachgeschlagen.

Lookup-Tabellen findest du überall: Sinustabellen in alten Spielen, CRC-Prüfsummen, Zeichenklassen in Lexern. Prüfe aber vorher, ob .NET das schon kann. Für das Zählen von Bits gibt es `System.Numerics.BitOperations.PopCount`. Die Methode nutzt einen eigenen Prozessorbefehl und ist noch schneller als jede Tabelle.

Memoization als allgemeine Technik

In Dynamische Programmierung hast du gesehen, wie die Fibonacci-Zahlen mit einem `Dictionary<int, long>` als Gedächtnis schnell werden. Das Prinzip heißt **Memoization** und funktioniert für jede Funktion: Bevor du rechnest, schaust du im Cache nach. Wenn du rechnen musst, legst du das Ergebnis im Cache ab.

Die folgende Methode macht aus einer beliebigen Funktion eine Funktion mit Cache. Der Zähler `calls` zeigt, wie oft die eigentliche Berechnung wirklich läuft.

```
int calls = 0;
Func<int, int> slowSquare = x =>
{
    calls++;
    return x * x;
};
Func<int, int> square = Memoize(slowSquare);

int[] inputs = [3, 5, 3, 3, 5, 7];
foreach (int x in inputs)
{
    Console.Write($"{square(x)} ");
}
Console.WriteLine();
Console.WriteLine($"{inputs.Length} Aufrufe, {calls} Berechnungen");

static Func<TIn, TOut> Memoize<TIn, TOut>(Func<TIn, TOut> function) where TIn : notnull
{
    Dictionary<TIn, TOut> cache = [];                              // (1)
    return x =>
    {
        if (cache.TryGetValue(x, out var cached)) { return cached; }
```

```
        TOut result = function(x);
        cache[x] = result;
        return result;
    };
}
```

```
9 25 9 9 25 49
6 Aufrufe, 3 Berechnungen
```

- **(1)** Das Lambda, das `Memoize` zurückgibt, hält das `Dictionary` fest (*Closure*). Jede memoisierte Funktion hat so ihren eigenen Cache.

Memoization ist nur dann korrekt und sinnvoll, wenn drei Bedingungen erfüllt sind:

- **Die Funktion ist rein.** Dieselbe Eingabe liefert immer dasselbe Ergebnis, und die Funktion hat keine Nebenwirkungen. Eine Methode, die die aktuelle Uhrzeit oder Daten aus einer Datenbank liest, darfst du nicht einfach so cachen.
- **Eingaben wiederholen sich.** Kommt jede Eingabe nur einmal vor, kostet der Cache nur Speicher.
- **Der Cache bleibt beherrschbar.** Er wächst mit jeder neuen Eingabe. In einem Server, der wochenlang läuft, brauchst du deshalb eine Obergrenze oder ein Ablaufdatum für die Einträge.

Übrigens: `Memoize(Fib)` würde die naive Fibonacci-Funktion nicht schnell machen. Die rekursiven Aufrufe innerhalb von `Fib` gehen direkt an die ursprüngliche Methode und am Cache vorbei. Bei rekursiven Funktionen muss der Cache deshalb wie in 27 *in* die Funktion eingebaut werden.

Speicherkomplexität

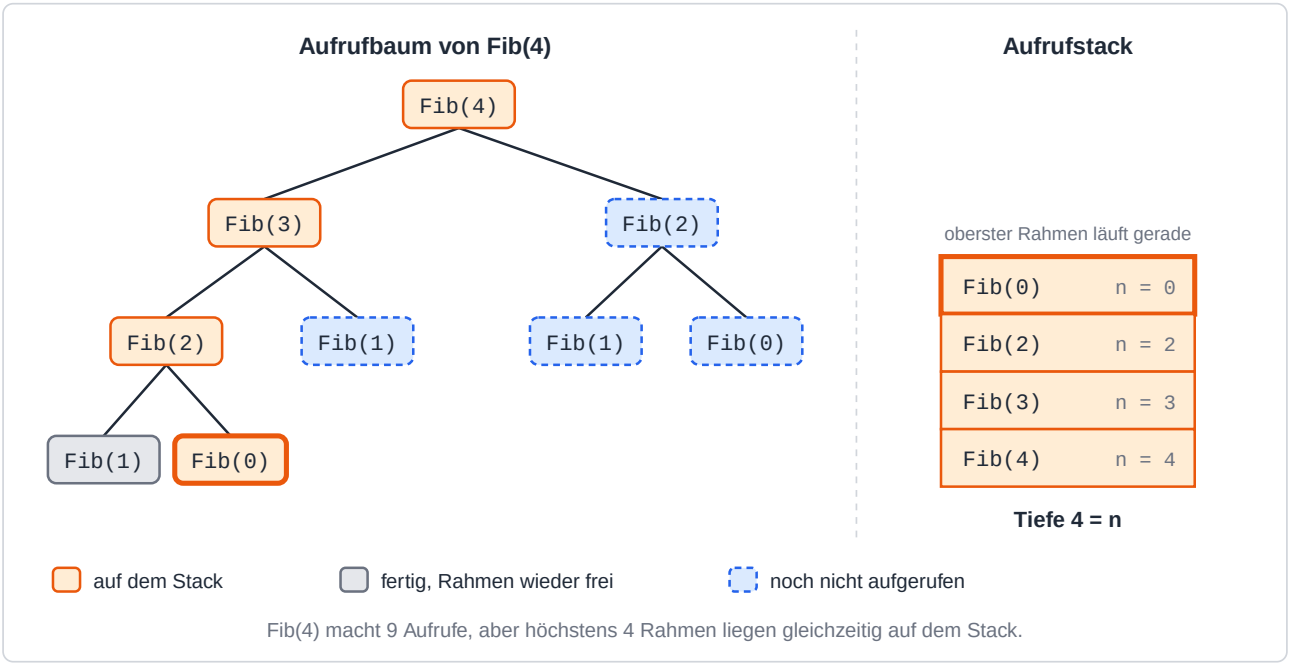
Für den Speicher verwendest du dieselbe Notation wie für die Zeit. Gezählt wird der **zusätzliche Speicher**, den ein Algorithmus neben der Eingabe braucht. Das Array, das du an `HasPairNaive` übergibst, zählt also nicht mit. Die Variablen `i` und `j` brauchen immer gleich viel Platz, egal wie groß n ist. Der zusätzliche Speicher ist O(1).

Algorithmus	Zeit	Zusätzlicher Speicher
Two-Sum naiv	$O(n^2)$	O(1)
Two-Sum mit `HashSet`	O(n)	O(n)
Primzahlen bis N mit Probedivision	$O(N \cdot \sqrt{N})$	O(1)
Sieb des Eratosthenes	$O(N \cdot \log \log N)$	O(N)
Lookup-Tabelle für Bytes	O(1) pro Zahl	O(1), die Tabelle hat immer 256 Einträge

Rekursionstiefe und Stack

Bei rekursiven Methoden versteckt sich Speicher dort, wo du ihn nicht siehst. Jeder Aufruf legt einen *Stackrahmen* (*Stack Frame*) an. Darin stehen die Parameter, die lokalen Variablen und die Rücksprungadresse. Erst wenn der Aufruf zurückkehrt, wird sein Rahmen wieder frei.

Entscheidend ist deshalb nicht, wie *viele* Aufrufe es insgesamt gibt, sondern wie viele *gleichzeitig* offen sind. Das ist die **Rekursionstiefe**. Die Grafik zeigt die naive Fibonacci-Funktion `Fib(n) = Fib(n - 1) + Fib(n - 2)` in dem Moment, in dem `Fib(0)` ganz unten links läuft:



Fib(4) macht 9 Aufrufe, aber höchstens 4 Rahmen liegen gleichzeitig auf dem Stack.

`Fib(4)` macht 9 Aufrufe, aber nie mehr als 4 sind gleichzeitig auf dem Stack. Allgemein ist die Tiefe n, denn jeder Weg von oben nach unten im Aufrufbaum ist höchstens n Ebenen lang. Die naive Variante ist also bei der Zeit katastrophal (exponentiell), beim Speicher aber nur O(n).

Der Stack ist klein. Unter Windows hat der Haupt-Thread eines .NET-Programms 1,5 MB. Das reicht für einige zehntausend Rahmen, je nachdem, wie groß ein Rahmen ist. Eine Rekursion mit linearer Tiefe kommt deshalb schnell an die Grenze:

```
Console.WriteLine(SumRecursive(10_000));
Console.WriteLine(SumIterative(10_000));
Console.WriteLine(SumIterative(1_000_000));

static long SumRecursive(int n) => n == 0 ? 0 : n + SumRecursive(n - 1);

static long SumIterative(int n)
{
    long sum = 0;
    for (int i = 1; i <= n; i++) { sum += i; }
    return sum;
}
```

```
50005000
50005000
500000500000
```

`SumRecursive(100_000)` beendet das Programm mit der Meldung `Stack overflow.`. Auf einem Testrechner lag die Grenze im Debug-Build zwischen 15 000 und 20 000 und im Release-Build zwischen 30 000 und 40 000 Aufrufen. Eine `StackOverflowException` kannst du nicht mit `catch` abfangen, der Prozess wird sofort beendet. Die iterative Variante braucht O(1) Speicher und hat dieses Problem nicht.

Daraus folgt eine Faustregel: Rekursion mit logarithmischer Tiefe (binäre Suche, Mergesort, Bäume, die ausgeglichen sind) ist unproblematisch. Bei linearer Tiefe und großen Eingaben schreibst du die Methode besser iterativ, notfalls mit einem eigenen `Stack<T>` auf dem Heap.

In-place

Ein Algorithmus arbeitet **in-place**, wenn er die Eingabe direkt verändert und nur O(1) zusätzlichen Speicher braucht (bei manchen Definitionen auch O(log n) für die Rekursion). In .NET siehst du den Unterschied beim Sortieren:

```
int[] data = [5, 2, 9, 1, 7];

int[] sortedCopy = data.Order().ToArray();      // (1)
Console.WriteLine($"Kopie:    {string.Join(", ", sortedCopy)}");
Console.WriteLine($"Original: {string.Join(", ", data)}");

Array.Sort(data);                               // (2)
Console.WriteLine($"Original: {string.Join(", ", data)}");
```

```
Kopie:    1, 2, 5, 7, 9
Original: 5, 2, 9, 1, 7
Original: 1, 2, 5, 7, 9
```

- **(1)** LINQ lässt das Original unverändert und baut ein neues Array. Das braucht O(n) zusätzlichen Speicher.
- **(2)** `Array.Sort` sortiert in-place. Es braucht keinen zusätzlichen Platz für die Daten, dafür ist die ursprüngliche Reihenfolge weg.

Auch das ist ein Tradeoff: In-place spart Speicher und Kopierarbeit, verändert aber Daten, die vielleicht noch jemand anderer braucht. Bei großen Arrays oder vielen Aufrufen ist in-place die richtige Wahl. In den meisten anderen Fällen ist Code ohne Seiteneffekte leichter zu verstehen und zu testen.

P und NP

Ein Paketdienst hat 20 Adressen und sucht die kürzeste Route, die alle Adressen besucht und zum Lager zurückführt. Das klingt nach einer Aufgabe für eine Schleife über alle Möglichkeiten. Leider gibt es so viele Möglichkeiten, dass kein Computer der Welt sie rechtzeitig durchprobiert. Und bis heute kennt niemand einen Algorithmus, der dieses Problem immer optimal *und* schnell löst. Dieses Kapitel erklärt informell, warum das so ist, was die Klassen P und NP bedeuten und wie man in der Praxis trotzdem gute Routen findet.

Entscheidungs- und Optimierungsprobleme

Das Problem des Paketdienstes heißt **Problem des Handlungsreisenden** (englisch *Traveling Salesperson Problem*, TSP). Es gibt davon zwei Varianten:

- **Optimierungsproblem:** Finde die kürzeste Rundreise durch alle Städte.
- **Entscheidungsproblem:** Gibt es eine Rundreise, die höchstens K Kilometer lang ist? Die Antwort ist nur *ja* oder *nein*.

Dasselbe gilt für das **Rucksackproblem** aus der dynamischen Programmierung. Die Optimierungsvariante fragt nach dem größten Wert, der in den Rucksack passt. Die Entscheidungsvariante fragt: Gibt es eine Auswahl mit Gewicht höchstens W und Wert mindestens V?

Die Theorie arbeitet mit Entscheidungsproblemen, weil sich Ja/Nein-Fragen leichter vergleichen lassen. Für die Praxis ist das keine Einschränkung. Kannst du die Ja/Nein-Frage schnell beantworten, findest du auch die optimale Länge schnell: Du fragst einfach mit verschiedenen Werten für K, ähnlich wie bei der binären Suche.

Die Klasse P

Ein Problem gehört zur Klasse **P**, wenn es einen Algorithmus gibt, der es in *polynomieller Zeit* löst, also in $O(n^k)$ für eine feste Zahl k. Viele Algorithmen, die du schon kennst, liegen in P:

- Sortieren mit Mergesort: O(n log n)
- Breitensuche in einem Graphen: O(V + E)
- Kürzeste Wege mit Dijkstra: O((V + E) log V)
- Ein Wort mit einem endlichen Automaten prüfen: O(n)

Als Faustregel gilt: Polynomiell heißt *effizient lösbar*. Ein $O(n^{100})$-Algorithmus wäre zwar in der Praxis nutzlos, solche Fälle kommen aber kaum vor. Der eigentliche Unterschied zeigt sich gegenüber exponentiellem Wachstum. Bei 1 ns pro Schritt:

n	n^3 Schritte	2^n Schritte
10	1 µs	1 µs
20	8 µs	1 ms
40	64 µs	18 Minuten

n	n^3 Schritte	2^n Schritte
60	0,2 ms	36 Jahre
100	1 ms	rund 40 Billionen Jahre

Bei kleinen Eingaben ist der Unterschied unsichtbar. Ab einer bestimmten Größe hilft aber auch ein tausendmal schnellerer Computer nicht mehr: Er schafft bei 2^n gerade einmal zehn Elemente mehr, denn $2^{10} \approx 1000$.

Die Klasse NP: Lösungen schnell prüfen

Für das TSP, das Rucksackproblem und viele andere kennt niemand einen polynomiellen Algorithmus. Sie haben aber eine gemeinsame Eigenschaft: **Wenn dir jemand eine Lösung vorlegt, kannst du sie schnell prüfen.**

- **Sudoku:** Ein Rätsel zu lösen kann lange dauern, beim Backtracking probierst du viele Ziffern durch. Ein ausgefülltes Gitter zu prüfen ist dagegen einfach: Jede Zeile, jede Spalte und jeder Block muss jede Ziffer genau einmal enthalten.
- **TSP:** Ist eine vorgelegte Reihenfolge der Städte eine Rundreise mit Länge höchstens K? Du prüfst, ob jede Stadt genau einmal vorkommt, und addierst n Strecken.
- **Rucksack:** Ist eine vorgelegte Auswahl höchstens W schwer und mindestens V wert? Du addierst Gewichte und Werte.

Ein Problem gehört zur Klasse **NP**, wenn sich jede Ja-Antwort mit einem passenden Nachweis (einer vorgelegten Lösung) in polynomieller Zeit prüfen lässt. Achtung: NP steht nicht für „nicht polynomiell“, sondern für *nichtdeterministisch polynomiell*. Ein gedachter Rechner, der wie ein NEA die richtige Lösung *raten* darf, müsste sie nur noch prüfen.

Beim Sudoku ist eine Feinheit wichtig: Ein 9×9-Sudoku hat eine feste Größe, das löst jeder Rechner in Sekundenbruchteilen. Die Theorie betrachtet Sudokus der Größe $n^2 \times n^2$, also 4×4, 9×9, 16×16, 25×25 und so weiter, und fragt, wie der Aufwand mit n wächst.

Prüfen gegen Lösen in C#

Das folgende Programm zeigt den Unterschied am Rucksackproblem. `IsSolution` prüft eine vorgelegte Auswahl in einem Durchlauf. `FindSolution` kennt keinen besseren Weg, als alle Auswahlen durchzuprobieren.

```
Item[] items =
[
    new("Laptop", 3, 9),
    new("Kamera", 1, 4),
    new("Zelt", 4, 8),
    new("Buch", 2, 3),
    new("Jacke", 2, 5),
];
int capacity = 7;
```

```
bool[] proposal = [true, true, false, false, true];
Console.WriteLine($"Vorschlag {Describe(items, proposal)}, Wert 18: "
    + $"{IsSolution(items, proposal, capacity, 18)}");

foreach (int minValue in new[] { 18, 19 })
{
    (bool[]? selection, long tested) = FindSolution(items, capacity, minValue);
    string result = selection is null ? "keine Lösung" : Describe(items, selection);
    Console.WriteLine($"Wert {minValue}: {result} ({tested} von {1L << items.Length}
geprüft)");
}

static bool IsSolution(Item[] items, bool[] selection, int capacity, int minValue)
{
    int weight = 0;
    int value = 0;
    for (int i = 0; i < items.Length; i++)                              // (1)
    {
        if (selection[i])
        {
            weight += items[i].Weight;
            value += items[i].Value;
        }
    }
    return weight <= capacity && value >= minValue;
}

static (bool[]? Selection, long Tested) FindSolution(Item[] items, int capacity, int
minValue)
{
    long count = 1L << items.Length;                                    // (2)
    for (long mask = 0; mask < count; mask++)
    {
        bool[] selection = new bool[items.Length];
        for (int i = 0; i < items.Length; i++)
        {
            selection[i] = (mask & (1L << i)) != 0;                     // (3)
        }
        if (IsSolution(items, selection, capacity, minValue)) { return (selection, mask
+ 1); }
    }
    return (null, count);
}

static string Describe(Item[] items, bool[] selection) =>
    string.Join(", ", items.Where((item, i) => selection[i]).Select(item => item.
Name));

record Item(string Name, int Weight, int Value);
```

```
Vorschlag Laptop, Kamera, Jacke, Wert 18: True
Wert 18: Laptop, Kamera, Jacke (20 von 32 geprüft)
Wert 19: keine Lösung (32 von 32 geprüft)
```

- **(1)** Das Prüfen braucht einen Durchlauf über die Gegenstände, also O(n).
- **(2)** Bei n Gegenständen gibt es 2^n Auswahlen, denn jeder Gegenstand ist dabei oder nicht.
- **(3)** Die Bits von `mask` legen fest, welche Gegenstände ausgewählt sind. Bit i steht für Gegenstand i.

Für eine Ja-Antwort hat die Suche hier Glück und findet nach 20 Versuchen eine Auswahl. Für die Nein-Antwort muss sie alle 32 Auswahlen prüfen, denn erst dann steht fest, dass es keine gibt. Mit 60 Gegenständen wären das 2^{60} Auswahlen, bei 1 ns pro Auswahl laut der Tabelle oben rund 36 Jahre.

Vielleicht wendest du ein, dass die Tabelle aus der dynamischen Programmierung den Rucksack doch schnell löst. Sie hat $n \cdot W$ Zellen. W ist aber eine *Zahl* in der Eingabe, keine Anzahl von Elementen. Bei einer Kapazität von 10^{15} Gramm wird die Tabelle viel zu groß, obwohl die Eingabe nur ein paar Zahlen enthält. Man nennt so einen Algorithmus *pseudopolynomiell*.

P liegt in NP

Jedes Problem aus P liegt auch in NP. Wenn du ein Problem schnell lösen kannst, kannst du eine vorgelegte Lösung auch schnell prüfen: Du löst es einfach selbst und vergleichst. Es gilt also $P \subseteq NP$.

NP-vollständige Probleme

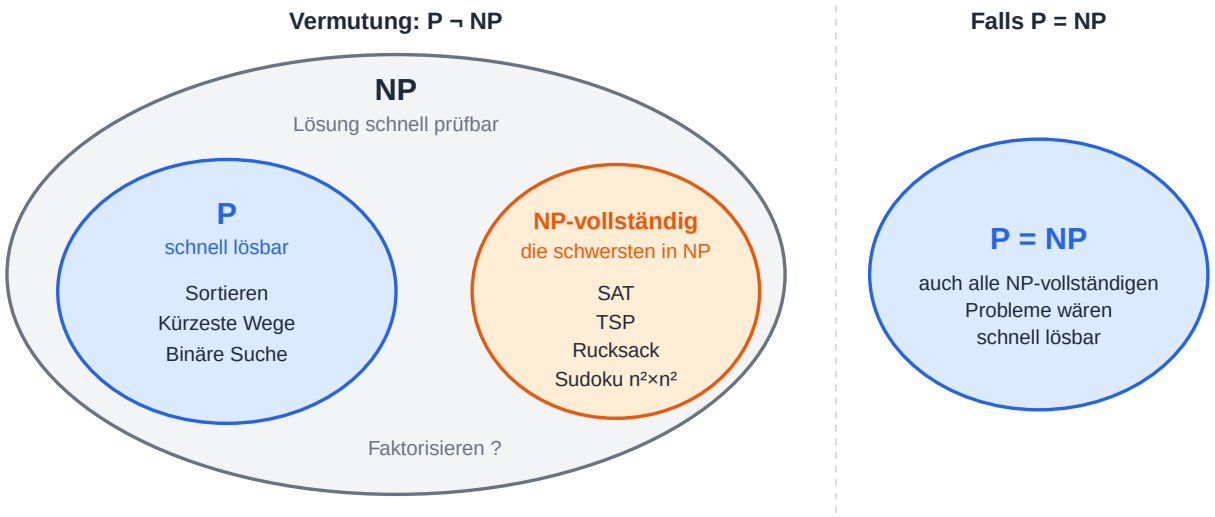
Manche Probleme lassen sich in andere *übersetzen*. Wer zum Beispiel einen Stundenplan ohne Überschneidungen sucht, kann das als Färbung eines Graphen formulieren: Jede Lehrveranstaltung ist ein Knoten. Eine Kante verbindet zwei Lehrveranstaltungen, die nicht gleichzeitig stattfinden dürfen, etwa weil dieselbe Klasse sie besucht. Jede Farbe ist eine Zeitstunde. Ist die Übersetzung selbst schnell (polynomiell), nennt man sie eine **Reduktion**. Dann gilt: Wer das zweite Problem schnell löst, löst auch das erste schnell.

1971 zeigte Stephen Cook (unabhängig davon auch Leonid Levin) etwas Erstaunliches: Es gibt ein Problem in NP, auf das sich *jedes* andere Problem in NP reduzieren lässt. Es heißt SAT und fragt, ob man die Variablen einer logischen Formel so mit `true` und `false` belegen kann, dass die Formel wahr wird. Ein Jahr später zeigte Richard Karp dasselbe für 21 weitere Probleme. Heute kennt man tausende davon.

Ein Problem heißt **NP-vollständig**, wenn es in NP liegt und sich jedes Problem aus NP darauf reduzieren lässt. Die NP-vollständigen Probleme sind damit die schwersten Probleme in NP. Beispiele (jeweils als Ja/Nein-Frage):

- SAT
- TSP: Gibt es eine Rundreise mit Länge höchstens K?
- Rucksack: Gibt es eine Auswahl mit Gewicht höchstens W und Wert mindestens V?
- Sudoku der Größe $n^2 \times n^2$: Hat das Rätsel eine Lösung?
- Graphfärbung: Lässt sich ein Graph mit k Farben färben (Stundenpläne, Registerbelegung in Compilern)?

Die Optimierungsvarianten wie „Finde die kürzeste Rundreise“ sind keine Ja/Nein-Fragen. Sie sind mindestens so schwer und heißen **NP-schwer**.



Die linke Skizze nimmt P ¬ NP an. Ob das stimmt, ist bis heute nicht bewiesen.
Alle Probleme als Ja/Nein-Frage. Für Faktorisieren ist weder ein schneller Algorithmus bekannt noch ein Nachweis, dass es NP-vollständig ist.

Die Frage P = NP

Die NP-vollständigen Probleme hängen alle zusammen. Findet jemand für ein einziges davon einen polynomiellen Algorithmus, sind sie alle in P, und es gilt P = NP. Beweist umgekehrt jemand, dass eines davon nicht in P liegt, gilt das für alle.

Bis heute ist keines von beiden gelungen. **Ob P = NP gilt, ist eine der bekanntesten offenen Fragen der Informatik.** Das Clay Mathematics Institute hat im Jahr 2000 ein Preisgeld von einer Million US-Dollar für die Antwort ausgeschrieben. Die meisten Fachleute vermuten P ≠ NP, weil trotz jahrzehntelanger Suche niemand einen schnellen Algorithmus für ein NP-vollständiges Problem gefunden hat. Eine Vermutung ist aber kein Beweis. Die Grafik oben ist deshalb unter der Annahme P ≠ NP gezeichnet.

Die Frage ist nicht nur theoretisch. Viele Verschlüsselungsverfahren beruhen darauf, dass sich ein Schlüssel leicht prüfen, aber nur sehr schwer finden lässt. Ein praktikabler Algorithmus für NP-vollständige Probleme würde viele davon unsicher machen.

Was das für die Praxis bedeutet

Erkennst du ein Problem als NP-vollständig oder NP-schwer, kannst du dir die Suche nach einem Algorithmus sparen, der *immer* optimal *und* schnell ist. Das heißt aber nicht, dass das Problem unlösbar ist. Es gibt mehrere Auswege.

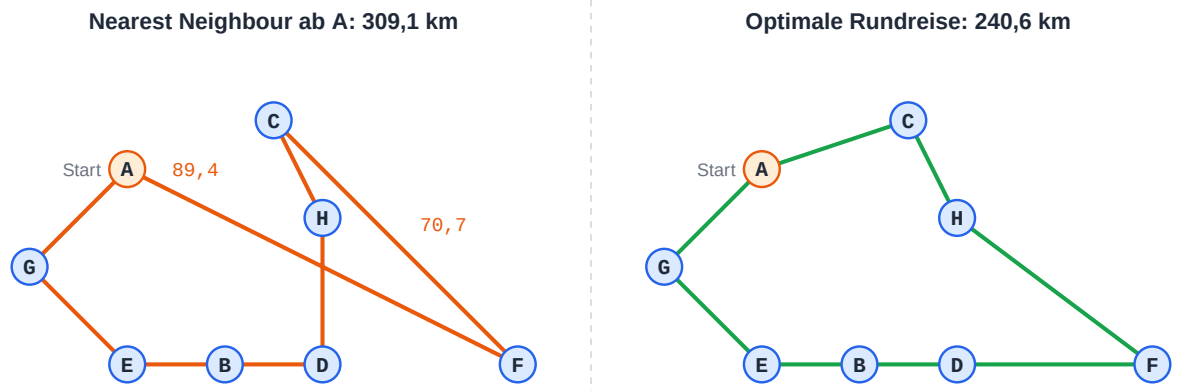
Die Eingabe klein halten

Bei kleinen Eingaben ist exponentielle Laufzeit kein Problem. Ein 9×9-Sudoku löst Backtracking meist in Sekundenbruchteilen. Oft lässt sich ein großes Problem in kleine zerlegen: Ein Paket-

dienst plant nicht eine Route für 2000 Pakete, sondern teilt die Pakete auf Fahrzeuge und Bezirke auf. Pro Fahrzeug bleiben dann vielleicht zehn Stopps, und die kann man exakt optimieren.

Heuristiken

Eine **Heuristik** ist ein Verfahren, das schnell eine *gute*, aber nicht unbedingt die *beste* Lösung findet. Beim TSP ist die einfachste Heuristik **Nearest Neighbour** (nächster Nachbar): Starte in einer Stadt und fahre immer zur nächstgelegenen Stadt, die du noch nicht besucht hast. Am Ende kehrst du zum Start zurück. Das ist ein Greedy-Algorithmus und braucht nur $O(n^2)$ Schritte.



Nearest Neighbour fährt immer zur nächsten noch nicht besuchten Stadt: A, G, E, B, D, H, C, F.
Am Ende bleibt das abgelegene F übrig, der Rückweg F ¬ A ist teuer: 28,5 % länger als das Optimum.

Die Grafik zeigt acht Städte. Nearest Neighbour beginnt in A und besucht die Städte in der Reihenfolge A, G, E, B, D, H, C, F. Die ersten Schritte sind kurz. Am Ende ist nur noch F übrig, das weit weg liegt, und von dort geht es den ganzen Weg zurück nach A. Die Rundreise ist 309,1 km lang. Die optimale Rundreise A, C, H, F, D, B, E, G hat nur 240,6 km. Die Heuristik liegt damit 28,5 % über dem Optimum. Um das Optimum zu finden, hat ein kleines Programm alle Rundreisen durchprobiert.

Zwei Dinge fallen auf:

- Die Nearest-Neighbour-Tour kreuzt sich selbst (F → A schneidet D → H). Eine optimale Tour hat nie Kreuzungen. Das Verfahren *2-Opt* nutzt das: Es löst Kreuzungen auf, indem es zwei Kanten austauscht, solange die Tour dadurch kürzer wird.
- Das Ergebnis hängt von der Startstadt ab. Startet Nearest Neighbour in D, F oder H, findet es bei diesen Städten zufällig das Optimum. Alle Startstädte durchzuprobieren kostet nur den Faktor n.

Näherungsverfahren mit Garantie

Eine Heuristik verspricht nichts. Ein **Näherungsverfahren** (Approximationsalgorithmus) gibt dagegen eine Garantie. Der Algorithmus von Christofides findet zum Beispiel für das TSP, bei dem die Entfernungen wie auf einer Landkarte die Dreiecksungleichung erfüllen, in polynomi-

eller Zeit immer eine Rundreise, die höchstens 50 % länger als die optimale ist. Für das Rucksackproblem gibt es Verfahren, die beliebig nahe an das Optimum herankommen. Je näher, desto länger rechnen sie.

Fertige Solver

Für viele NP-schwere Probleme gibt es hoch optimierte Programmbibliotheken, sogenannte *Solver*. Sie kombinieren geschicktes Backtracking, Pruning und Heuristiken und lösen in der Praxis oft Eingaben mit tausenden Variablen. Beispiele sind SAT-Solver, Solver für lineare Optimierung und Google OR-Tools, das auch ein NuGet-Paket für .NET anbietet. Bevor du selbst ein NP-schweres Problem implementierst, lohnt sich ein Blick auf solche Werkzeuge.

Ausblick

Es gibt sogar Probleme, die *kein* Algorithmus lösen kann, egal wie viel Zeit er hat: Das bekannteste ist das *Halteproblem*, also die Frage, ob ein beliebiges Programm bei einer bestimmten Eingabe irgendwann anhält.